

PrimeSim™ User Guide: Pro and SPICE

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

New in This Release	28
Related Products, Publications, and Trademarks	28
Conventions	29
Customer Support	29
Statement on Inclusivity and Diversity	30

Part 1: Introduction

1. Introduction to the PrimeSim Tool and the PrimeSim Continuum	32
PrimeSim Simulator Overview	32
PrimeSim Pro Features and Benefits	33
PrimeSim SPICE Features and Benefits	34
PrimeSim Continuum Family Overview	34
PrimeSim Pro and PrimeSim SPICE Tool Integration	35
2. Setting Up the PrimeSim Tool	37
Configuring the PrimeSim License	37
Configuring PrimeSim Continuum Token-Based License Mode	38
Configuring Standalone License Mode	39
Setting Up the Path to the Executable	40

Part 2: Input

3. Input Details	42
PrimeSim Input Requirements	42
Supported Netlist Formats	43
Support for Compressed Files	43
Support for Encryption	43

4. Running the PrimeSim Tool	44
Command-Line Syntax	44
Command-Line Options	45
Including PrimeSim Options	51
Restarting Transient Simulations From Snapshots	52
Generating Image Dump Snapshots	52
Restoring From Snapshots	53
Limitations	53
Restarting Simulations With Vector Changes	53
Generating the .restart_db File	54
Restarting From a .restart_db File With New Stimulus	54
Examples	55
Supported Statements in the Stimulus File	56
Limitations	56

5. Digital I/O Vectors	58
Direct VCD Input	58
Vector File Format	61
Vector Pattern Definition	62
Radix	62
NodeName	62
Hierarchical Node Names in Vector Files	63
I/O Statements	63
Parameters as Time Values in the Vector File	64
Waveform Parameter Settings	64
Tunit	65
Slope	65
Tfall	65
Trise	66
idelay_f	66
idelay_r	66
VH or VOH	66
VL or VOL	66
VIH	67
VIL	67
VREF	67
VTH	68
Mask	68
Enable	69

Contents

Check_Window	69
STOP_AT_ERROR	71
Support for logichv and logiclv in Vector Files	71
Delay Statements in Vector Files	72
Support for VCHK_IGNORE in Vector Files	72
Using the Context-Based Control Option (CBC)	72
Tabular Data	73
Arbitrary Time Step	73
Uniform Time Step	73
Cascade Time Step	74
I/O Vector Support for '-' Values	75
PrimeSim Post Vector Check	76
PrimeSim Circuit Examples	78
Case 1 (vec.in)	78
Case 2 (vec.in2)	79
Case 3 (vec.in3)	79

Part 3: Elements and Models

6. Circuit Elements and Models	82
Supported Models	83
Supported Model Features	85
TSMC Model Interface (TMI) Support	85
Si2 Compact Model Coalition - Open Model Interface (OMI) Support	85
STI Effects for BSIM3/4 Models	85
Aging Model NBTI Support	85
MOS Varactor Support	85
Binning Model Support for .mparam	85
IJTH Support	86
Flash Cell Model	86
Rules for Instance, Model, and Global Parameter Interaction	87
Rule 1	87
Rule 2	87
Rule 3	88
Resistors, Capacitors, Inductors, and Diodes	88
Resistor Elements (R-element)	88
Capacitor Elements (C-element)	91
Linear Inductors (L-element)	94

Contents

Mutual Inductors (K-element)	96
Reluctors (L-element)	97
Diode Elements (D-element)	98
Bipolar Junction Transistors (BJTs), JFETs, MESFETs, and MOSFETs	100
BJT Elements (Q-element)	100
JFET and MESFET Elements (J-element)	101
MOSFET Elements (M-element)	103
Independent Sources and Functions	104
Voltage Source (V-element) and Current Source (I-element) Elements	104
Functions for Independent Sources	107
PULSE	107
Piece-Wise-Linear (PWL)	110
.data Driven PWL Source	111
File-driven PWL Source	111
PWLZ Allows High 'z' State	112
Sinusoidal (SIN)	112
Cosinusoidal (COS)	113
Exponential (EXP)	114
Single-Frequency FM (SFFM)	115
Amplitude Modulation (AM)	116
Pseudo Random-Bit Generator Source (LFSR)	117
Pattern Source (PAT)	119
Edgetype: Linear vs. Halfsine	121
Dependent Sources and Functions	122
Voltage Controlled Voltage Source-VCVS (E-element)	123
Current Controlled Current Source-CCCS (F-element)	126
Voltage Controlled Current Source-VCCS (G-element), Voltage Controlled Resistor-VCR, Voltage Controlled Capacitor-VCCAP	128
Current Controlled Voltage Source-CCVS (H-element)	133
S-Element	135
Functions for Dependent Sources	142
Linear Function	142
Piece-Wise-Linear (PWL) Function	142
Polynomial Function	143
Gate Function	143
Transmission Lines	144
Lossless Transmission Lines (T-element)	144
Lossy Transmission Lines (U-element)	147
Transmission Lines (W-element)	148
Subcircuit Instances (X-element)	150

Contents

Port Elements and IBIS Buffers	151
Port Elements (P-element)	151
IBIS I/O Buffers (B-element)	153
Ferroelectric Capacitor (FeCap) Model	155
Overview of the Ferroelectric Capacitor (FeCap) Model	155
FeCap Elements	156
FeCap Model	157
FeCap for Model Parameter Extraction	158
Measuring Ferroelectric Hysteresis Loops	160
FeCap Model Parameter Extraction	161
FeCap Model Temperature Coefficients	163
7. C-Modeling	164
Required Steps to Use a C-Model	165
Main C-Model Structure	166
C-Model Related PrimeSim Options	168
PrimeSim Pro Common C-Functions for C-Models	168
Header Files	168
State Structure Definitions	168
Evaluation Functions	168
Model Definition Functions	169
Port Definition Functions	169
State Structure Memory Allocation	169
Simulation Phases	169
Hash Table Functions for Compact Memory or Array Storage	169
Port Access Functions	169
Self-Generated Events	170
Simulation Time	170
Port Types	170
Port Directions	170
Digital Values	170
Port Properties	171
Usage	171
Error Codes	172
Standard Practices	172
Temporary Local Variable Storage	172
Accessing Ports by Port ID	173

Name Conflicts	173
Debugging C-Models	174
PrimeSim Pro FSC C-Model Functions	174

8. Verilog-A Support	179
Support for Encrypted Verilog-A Files as Input	179
Support for LRM 2.4 Syntax and Features	179

Part 4: Options and Commands

9. PrimeSim Options	182
Using PrimeSim Options	182
General Control Options	183
Using primesim.cfg to Define Common PrimeSim Options	184
Environment Variables	184
PrimeSim Options (by Category)	186
General Options	187
DC Initialization Options	196
Output Reporting Options	197
Partitioning Options	202
Accuracy and Speed Options	203
Back-Annotation Options	206
PrimeSim SPICE Accuracy and Speed Options (-runlvl and .option runlvl) ...	211
PrimeSim Option Reference (Alphabetical List)	213
primesim_ac_at_diff_t	213
primesim_ac_flow	214
primesim_acc_sparse_fcap	215
primesim_adaptive_mt	216
primesim_add_divider	216
primesim_add_instance	217
primesim_aging_spfdivider	217
primesim_aginglib	218
primesim_allow_3t_mosfet	218
primesim_allow_dup_element	218
primesim_allow_dup_port	219

Contents

primesim_analog	219
primesim_auto_gmin_scale	219
primesim_auto_restart	220
primesim_bisection_concurrent_mode	221
primesim_bisection_fail_continue	221
primesim_bisection_output	221
primesim_bisection_summary	222
primesim_bsim3gate_leakage	222
primesim_bytol	223
primesim_c_model	223
primesim_c_model_exclude	224
primesim_capmin	224
primesim_case_sync_in_subckt	224
primesim_cbmodel	225
primesim_cck_post	226
primesim_cck_remove_result	226
primesim_cck_report_limit	227
primesim_ccmodel	227
primesim_cell_char	228
primesim_change_r3cmc_pin_order	229
primesim_check_dummy_mos_size	229
primesim_check_model	230
primesim_check_undefined_param	230
primesim_check_vth	230
primesim_checksum_error_exit	231
primesim_checksum_on	231
primesim_chk_devport	232
primesim_chk_disk_space	232
primesim_chk_format	233
primesim_chk_fsdb	234
primesim_chkblkpwr_pwrnode	234
primesim_chkblkpwr_pwrport	234
primesim_chkblkpwr_specified_only	235
primesim_chkdcpath_gndgroup	235
primesim_chkdcpath_pwrgroup	235
primesim_chkdcpath_rth	236
primesim_chkdevcur_report	237
primesim_chkdevop_disable	237
primesim_chkdevop_enable	237

Contents

primesim_chkdevop_filter_in	238
primesim_chkdevop_filter_out	238
primesim_chkdevop_mode	239
primesim_chkdevop_post	239
primesim_chkdevop_print_info	240
primesim_chkdg	241
primesim_chkexpr_disable	241
primesim_chkexpr_enable	242
primesim_chkrftime_post	242
primesim_chktiming_post	243
primesim_chkznnode_debug_out	244
primesim_chkznnode_rule	244
primesim_chkznnode_vth	245
primesim_cktopt	245
primesim_clampVerilog	246
primesim_connect_mode	247
primesim_consistent_vloop_check	247
primesim_cont_measout	248
primesim_convlevel	248
primesim_cprbtol	250
primesim_cshpt	250
primesim_cutnode	250
primesim_dcalg	251
primesim_dceffort	253
primesim_dcopt	253
primesim_delmax	254
primesim_delmax_level	255
primesim_delmin	255
primesim_delvto	256
primesim_detect_lvdd	257
primesim_detect_lvdd_static	257
primesim_detect_vcvs_loop	257
primesim_double_precision_output	258
primesim_dpf	258
primesim_dpf_mos_only	258
primesim_dpfadddev	259
primesim_dpfddiv	259
primesim_dpfdmodule	260
primesim_dpfdprefix	260

Contents

primesim_dpfscale	260
primesim_dpfsuffix	261
primesim_dvmax	261
primesim_elapsed_tflush	262
primesim_enhanced_tcl_mode	262
primesim_enprefix	262
primesim_eqr2spres, primesim_spresout	263
primesim_evlevel	265
primesim_exit	266
primesim_exit_dc_fail	266
primesim_exit_invopt	266
primesim_exitwarn	267
primesim_expand_instance_mfactor	267
primesim_extended_pwrblock	268
primesim_fast_corner	268
primesim_fast_corner_sweep	269
primesim_fcapmin	270
primesim_fcapmodel	271
primesim_fflush	271
primesim_finger_current	272
primesim_fingerchar	272
primesim_flash	273
primesim_flashcell_erase_update	273
primesim_flatsize	274
primesim_floating_gate_gshunt	274
primesim_fmiflag	275
primesim_fsc_auto_detect	275
primesim_fsc_vdd	275
primesim_fsdb_limit	276
primesim_fsdb_max_size	276
primesim_fsdb_split_pg	277
primesim_fsdb_split_time	277
primesim_fsdb_term_var_hier	277
primesim_fsdb_version	278
primesim_gdcpath	278
primesim_gen_ac_op	279
primesim_gen_ic_op	279
primesim_gen_mc0	280
primesim_gen_op_captab	280

Contents

primesim_gen_pn_c	281
primesim_gic	281
primesim_gmax	282
primesim_gmi_telem	282
primesim_goff	282
primesim_gpd	282
primesim_gpuid	283
primesim_gpu_mode	283
primesim_gshpt	284
primesim_hb_autoharm_tol	284
primesim_hier_delimiter	285
primesim_hiersim	285
primesim_hstolscale	286
primesim_ic_priority	286
primesim_icsweep	287
primesim_ichier	289
primesim_identical_mc_instance_file	289
primesim_ignore	290
primesim_ignore_chkfunc_error	291
primesim_ignore_device	291
primesim_ignore_flat_device	292
primesim_ignore_floating_node	292
primesim_ignore_layout_only_device	292
primesim_ignore_no_vec_tdata_error	293
primesim_ignore_op_analysis	293
primesim_ignore_option_error	294
primesim_ignore_subblk_option_error	294
primesim_ignore_va	295
primesim_inactive_chkanode_dv	295
primesim_include_instance	295
primesim_indn	296
primesim_ind_matrix_passivity	296
primesim_ind_matrix_passivity_dim_limit	297
primesim_info_limit	297
primesim_iovec_abs_value	298
primesim_iovec_option	298
primesim_iovec_vih	298
primesim_iovec_vil	299
primesim_iovec_voh	299

Contents

primesim_iovec_vol	300
primesim_iprbtol	300
primesim_iprobe_leaf_level	300
primesim_isolated_vsrc_res	301
primesim_keep_0v_dcvs	301
primesim_keepzeroparms	302
primesim_leakage_mode	302
primesim_list_noisesources	303
primesim_loadmodel	303
primesim_lprobe_hiz	304
primesim_lprobe_vh	304
primesim_lprobe_vl	305
primesim_lsf_format_chars	305
primesim_map_prbport	306
primesim_max_parallel	306
primesim_max_swap_allowed	307
primesim_max_width_tol	307
primesim_maxicout	308
primesim_mc_sigma_scale	308
primesim_mc_stats_report	308
primesim_mcbrief	309
primesim_mcseed	309
primesim_meas_from_to	310
primesim_measout	310
primesim_meastolscale	311
primesim_merged_meas_file	311
primesim_merge_fsdb	312
primesim_merge_parmosdio	312
primesim_merge_pinstance	313
primesim_merge_vector_errmsg	313
primesim_method	314
primesim_mode	314
primesim_model	317
primesim_model_cache_dc	318
primesim_model_dup_rule	318
primesim_model_verification_mode	319
primesim_module_instance	319
primesim_monitor	320
primesim_monte_cmd_overwrite	322

Contents

primesim_moscapcut	322
primesim_mosfet_fast_probe_lx34	322
primesim_mparcheck	323
primesim_negcap	323
primesim_negres	323
primesim_no_swap	323
primesim_noise_format	324
primesim_noise_off	325
primesim_noise_on	326
primesim_npc_max	327
primesim_num_meas_log	328
primesim_num_meas_per_line	328
primesim_op_compatible	329
primesim_op_parasitic_capacitance	329
primesim_op_region_display	330
primesim_output	330
primesim_output_ac	331
primesim_output_fname_type	331
primesim_output_gzip	333
primesim_output_op	334
primesim_output_range	334
primesim_pahier_mode	335
primesim_partition	336
primesim_pdn	337
primesim_phasenoise_lorentz	337
primesim_phasenoise_min_harmfreq_offset	338
primesim_pinst_in_p	339
primesim_power_mode	339
primesim_prb_skipmeas	340
primesim_prb_tmin	340
primesim_prbexprvar	340
primesim_prbport	341
primesim_prbtol_disable_visrc	341
primesim_precision	342
primesim_prelayout_models	343
primesim_print_global_param	343
primesim_print_max_con_node	344
primesim_print_period	344
primesim_print_rchk_resource	345

Contents

primesim_print_to_probe	345
primesim_prll_opt	346
primesim_pro_dcalg	346
primesim_probe_passive_device	347
primesim_probe_protected	347
primesim_probe_resource_usage	348
primesim_profile	349
primesim_pseudotrandbg	350
primesim_psfdir	350
primesim_pt0_format	351
primesim_pva_cross_ttol	351
primesim_pva_flow	352
primesim_pva_opt_port	352
primesim_pwl_r_end	353
primesim_pwrblock	353
primesim_pwrloadmodel	354
primesim_pwrtol	355
primesim_ra_cmd	355
primesim_ra_net	356
primesim_ra_tcl	356
primesim_rclamp	356
primesim_rcred	357
primesim_rcredscale	357
primesim_remove_floating_vsrc	358
primesim_remove_probe_prefix	358
primesim_remove_va_so_files	359
primesim_repdot	359
primesim_report_large_no_instances	359
primesim_resmax	360
primesim_resmin	360
primesim_restore (.SNAPSHOT)	361
primesim_reuse_mos_model	362
primesim_reuse_op	362
primesim_rng	363
primesim_rpitft_mode	363
primesim_runtime_meas	364
primesim_scale	364
primesim_set_cpu_time	365
primesim_set_floating_node	365

Contents

primesim_set_floating_node_format	366
primesim_set_floating_node_type	367
primesim_set_special_char	367
primesim_setenv	368
primesim_short_rnet	368
primesim_simple_em_naming	369
primesim_single_bin_model_check	369
primesim_skip_flat_pl_node	370
primesim_skip_meas_error	370
primesim_skip_unused_param	371
primesim_skipwarn	371
primesim_soa_check_on_dc_init	372
primesim_soa_maxwarns	372
primesim_soa_warn	373
primesim_soa_warn_format	373
primesim_soa_warn_range	374
primesim_soa_warn_skipwindow	375
primesim_soa_warn_to_file	375
primesim_sparse_fcap	376
primesim_speed	376
primesim_spf	378
primesim_spf_acc_irem	379
primesim_spf_add_irem_window	379
primesim_spf_add_netpin	380
primesim_spf_add_netpin_file	381
primesim_spf_allow_rc_parasitic_instance	382
primesim_spf_bus_char	382
primesim_spf_cc_parse	383
primesim_spf_delete_netpin	384
primesim_spf_delete_netpin_file	385
primesim_spf_em_version	386
primesim_spf_exclude	386
primesim_spf_include	386
primesim_spf_fingerchars	387
primesim_spf_finger_res_policy	387
primesim_spf_histogram	388
primesim_spf_layout_only_device_models	388
primesim_spf_layout_only_net	389
primesim_spf_map_subckt_port	389

Contents

primesim_spf_matcheffort	390
primesim_spf_port_name_check	390
primesim_spf_removetoprc	391
primesim_spf_sch_check_mode	392
primesim_spf_sensitive	392
primesim_spf_skip_res_for_ivsrc	393
primesim_spf_spice_names	393
primesim_spf_user_defined_gnd_list	394
primesim_spfallowerror	394
primesim_spfcnet	394
primesim_spfdb_version	394
primesim_spfeqr, primesim_spf2eqr, primesim_spfeqrfile, primesim_spfeqr_only, primesim_spf2eqr_select	395
primesim_spffcmin	397
primesim_spffcnet	397
primesim_spfhdiv	398
primesim_spfinst	398
primesim_spfkeepnodecap	399
primesim_spfmergeport	399
primesim_spfmodule	400
primesim_spfnet_wildcard_mode	400
primesim_spfnonet	401
primesim_spfpost	401
primesim_spfpost_out_only	401
primesim_spfprb	402
primesim_spfprb_finger	402
primesim_spfprb_mode	403
primesim_spfprb_sync	403
primesim_spfprefix	404
primesim_spfrc_ctype	404
primesim_spfrcnet	405
primesim_spfrepgate	407
primesim_spfreplast	407
primesim_spfrgcnet	409
primesim_spfrmax	409
primesim_spfrmin	410
primesim_spfrnet	410
primesim_spfrptrmax	411
primesim_spfscale	411

Contents

primesim_spfspef	411
primesim_spfsplitnet	412
primesim_spfsuffix	412
primesim_spfwarn	413
primesim_spred	413
primesim_spredtc	414
primesim_subckt_dup_rule	415
primesim_sweep_brief	415
primesim_syntax_check	416
primesim_tcl_init_file	416
primesim_tflush	417
primesim_title_in_alter_filenames	418
primesim_tmiage_reltol	419
primesim_tmishe_abstol	419
primesim_tolscale	419
primesim_tran_conv_retry	421
primesim_tsc	422
primesim_tstop	422
primesim_tunit	423
primesim_use_old_trout	423
primesim_vdd	423
primesim_vector	424
primesim_vector_errmsg	425
primesim_vector_mode	425
primesim_veriloga (.hdl)	426
primesim_veriloga_bypass	426
primesim_veriloga_cross_method	427
primesim_veriloga_dkind_def	427
primesim_veriloga_dkind_mode	428
primesim_veriloga_table_fill_in_policy	428
primesim_veriloga_var_opt	429
primesim_vprbtol	429
primesim_vpwltol	429
primesim_warn_limit	430
primesim_wdf_limit	430
primesim_wdf_mode	430
primesim_write_instance_param	431
primesim_write_instance_table	432
primesim_write_mcparam	432

	pva_genso_filebase	433
	pva_output_dir	434
	senssort	434
	s_cache_reuse	435
	PrimeSim Commands	435
	.AUTOSETTLE	435
	.FORCE	437
	.RELEASE	438
10.	PrimeSim HSPICE-Compatible Commands and Control Options	440
	PrimeSim HSPICE-Compatible Commands	440
	.NET	443
	.XF	444
	PrimeSim HSPICE-Compatible Control Options	444
	ge_rational_func	448
	hisimhv_version_update	448
	s_rational_func_preset	449
	stress_input	449
	Optical Port Mapping	450
	.Convert_Port	450
	.OPTION OPTICAL_PORT	451
	.PortMapping	451
	Port Mapping Example	452
	Bisection Optimization	452
	Example for Setup Time Analysis With Bisection	455
	Example for Minimal Pulse Width With Passfail	456
	Pushout Bisection	457
	Bisection Output Convention	458
	Bisection Analysis With Two Measurements	458
	Concurrent Bisection for Independent Circuit Blocks	458
11.	PrimeSim SPICE Commands and Control Options	460
	PrimeSim SPICE Commands	460
	.AC	461
	.ACMATCH	463
	.DC	464

Contents

.DCMATCH	467
.ENV	467
.HB	469
.HBAC	475
.HBLIN	478
.HBLSP	481
.HBNOISE	482
.HBOSC	488
.HBSTB	492
.HBXF	493
.LIN	496
.LSTB	498
.NOISE	499
.PHASENOISE	502
.SENS	508
.SN	509
.SNAC	512
.SNLIN	514
.SNNOISE	516
.SNOSC	522
.SNSTB	524
.SNXF	525
.STATEYE	529
.TRAN	538
.XF	544
PrimeSim SPICE Control Options	545
S_IGNORE_PARAM	546

Part 5: Multi-CPU and GPU Support

12. PrimeSim Multi-CPU	548
Introduction to Multi-CPU	548
Why Multi-CPU SPICE?	549
Multi-CPU Computers vs. Clusters of Computers	549
System Requirements	550
Hardware	550

Contents

Software	550
Command-Line Options for Running Multi-CPU Simulations	550
Running PrimeSim Multi-CPU on a Single Machine	556
Running PrimeSim Multi-CPU on Multiple Machines	556
How to Set Up Password-less SSH Login	556
Machine File	557
LSF	557
LSF HPC	558
Using *.bkill to Terminate LSF Processes	558
SGE	558
Using *.qdel to Terminate SGE Processes	559
License Suspend/Resume Feature	559
Time-Out Messages	560
PrimeSim Client-Server Mode	560
Start the Server	560
Start the Clients	561
Stop the Server	562
Distributed Processing, CDPL, and Dynamic Scaling	563
Running Distributed Processing on the Current Machine	564
Running Distributed Processing on a Network Grid	565
Dynamic Scaling	567
Terminating Distributed Processing Simulation in an LSF Farm Environment ..	568
Using .primesim_parallel.ini to Simplify Multi-CPU Calls	568
Automatic Determination of Optimal Number of Parallel Processes	569
13. PrimeSim GPU	570
Introduction	570
GPU Hardware and Software Requirements	572
Running GPU Simulation	573
GPU Card Selection	574
GPU Extended Log File (.glog)	575
CUDA Error Codes	576
NVIDIA System Management Interface	576

14. Back-Annotation	580
Setting Defaults for DSPF and DPF Options	580
DSPF Annotation	582
Option Precedence	586
Probing Internal Nodes	586
Back-Annotation Support Using Expressions	587
SPEF Annotation	587
DPF Annotation	587
SPF RC Reduction	588
Active Net Flow	588
.spf_active_ba	588

15. Circuit Checks	591
Circuit Check Commands	591
Check Active/Inactive Nodes (.CHKANODE)	592
Check Block Power (.CHKBLKPWR)	593
Check DC Path (.CHKDCPATH)	594
Check Leakage Current Path (.CHKDCPATH, zgate=on)	597
Check Device Current (.CHKDEVCUR)	597
Check Device Operation Point (.CHKDEVOP)	599
Check Expression (.CHKEXPR)	601
Check OFF Device Leakage Current (.CHKOFFDEVLEAK)	605
Check Rise/Fall Transition Time (.CHKRFTIME)	611
Check Signal Voltage Difference (.CHKSIGDIFF)	613
Static Circuit Checks (.CHKSTATICERC)	614
Check Timing Setup/Hold/Delay/Width (.CHKTIMING)	619
Check and Report Toggle Count (.CHKTOGGLE)	623
Check High Impedance State Node (.CHKZNODE)	624
Circuit Check Post-Processing Options	627

16. Tcl Interactive Mode	628
Interactive Mode	628
Interactive Commands	628
circheck	630
clearlog	631

Contents

cont	631
dataflush	631
dn	632
exi	632
exit	633
fn	633
fset	634
help	634
ni	634
now	635
op	635
openwv	635
pd	635
pdp	636
pi	636
pn	636
pvar	637
quit	637
rn	637
snapshot save	637
stop	638
svar	638
tn	638
tnet	639
17. Monte Carlo Analysis	642
Monte Carlo Analysis in the PrimeSim Tool	643
Input Syntax	645
Monte Carlo-Specific Variation Block Options	646
Output for Variation Block Monte Carlo	647
Simulation Listing	647
Measurement Output File	648
Bi-Section Runs	648
Incremental Runs	648
Random Number Generation Sequence	649
Overview of the Variation Block	649
Variation Block Structure	650

Contents

General Section	651
Subblocks for Global and Local Variations	651
Independent Random Variables	652
Dependent Random Variables	654
Absolute Versus Relative Variation	655
Variations on Model Parameters	655
Variations on Subcircuit Parameters	656
Variations on Top-Level Parameters	659
Variations on Temperature	659
Access Functions, Get_E(), Get_P(), Get_M(), Get_O()	659
Special Rules Regarding Variation Block Usage	665
External Sampling	665
Syntax	666
Controlling the Read-in of an External File	666
Reusing a Modified .mc0 File	667
Data Mining Using PrimeSim With PrimeSim AVA	667
One-Step Flow	668
Two-Step Flow	668

18. Multi-Technology Simulation of 3-D Integrated Circuit (3DIC)	670
3DIC Netlist Construct and Usage	670
Full Circuit Example	671
Module-Based PrimeSim Options	671
Module-Based Back-Annotation	672
SPF Back-Annotation Example	672
DPF Back-Annotation	673
Applying a 3DIC Module for a Regular Netlist	673

Part 7: Output

19. Output Details	676
Output Naming Conventions	676
PrimeSim SPICE Default Output Format	678
Output Files	678
Transient Analysis	679
DC Analysis	679

Output Control Statements	680
Log Files	680
Exit Codes	680
20. Probing and Measuring	683
Algebraic Expressions	683
Built-In Functions	684
External Table Parameter File Interface	686
String Compare Function (strcmp)	687
.PROBE	688
Probing Block Current	693
Probing and Exceptions to Probing	693
Logic Probes (Digital Waveforms)	694
Probing With Regular Expressions	694
Exceptions to Probing	695
Support for Power Reporting	696
Support for Measuring Power Dissipation: pd()	696
Probing Element Parameters	697
.MEASURE	705
Support for EM in .MEASURE	706
Hierarchical Parameter Measurements	706
Measurement Analysis (.MEASURE) and its Modes	706
Rise, Fall, and Delay	707
FIND and WHEN Functions	709
Equation Evaluation	711
AVG, RMS, MIN, MAX, and Peak-to-Peak (PP)	711
INTEGRAL Function	713
DERIVATIVE Function	714
Peak-to-Peak Period Jitter (PPJITTER)	715
Continuous Measure	716
Pulse Width Measurements	716
.STROBEJITTER	717
.LPROBE	719
21. PrimeSim Utilities	720
Using fscript	720

Contents

fscript Timeout Feature	720
fscript Commands	721
calc	722
close	723
cmdlist	723
dinit	723
dir	723
dlist	723
dsignal	724
dump	724
exit/quit	725
fsdb2vcd	725
fsdbinfo	726
fset	726
list	729
load	729
meas/measure	730
open	730
phnmeas	731
pn2tbl	731
pwl	731
run	733
signal	733
sp2fsdb	734
valias	734
vbus	734
vinit	735
vlist	735
vrun	735
vsignal	736
Wildcard Support for Signal Names	737
fscript Examples	737
Example 1: Convert FSDB to PWL	737
Example 2: Convert VCD to Vector	738
Example 3: Dump Signal	739
Example 4: Convert tr0 to FSDB	739
Example 5: Convert FSDB to VCD	740
Example 6: Create IC File From Simulation Output Files	740

Contents

Example 7: Post-measure With Output Data File	740
Example 8: Phase Noise Analysis	740
Using IM	741
Filter Options	741
Batch Mode Options	741
Input Files	742
Examples	742

About the PrimeSim User Guide: Pro and SPICE

The *PrimeSim User Guide: Pro and SPICE* describes how to use the Synopsys PrimeSim Pro and PrimeSim SPICE tools.

This preface includes the following sections:

- [New in This Release](#)
- [Related Products, Publications, and Trademarks](#)
- [Conventions](#)
- [Customer Support](#)
- [Statement on Inclusivity and Diversity](#)

New in This Release

Information about new features, enhancements, and changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the *PrimeSim Release Notes* on the SolvNetPlus site.

Related Products, Publications, and Trademarks

For additional information about the PrimeSim tool, see the documentation on the Synopsys SolvNetPlus support site at the following address:

<https://solvnetplus.synopsys.com>

You might also want to refer to the documentation for the following related Synopsys products:

- PrimeSim™ Continuum
- PrimeSim™ HSPICE® (formerly HSPICE®)
- PrimeSim™ XA (formerly CustomSim™)
- VCS® PrimeSim™ AMS
- PrimeWave™ Design Environment

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
Courier	Indicates syntax, such as <code>write_file</code> .
<i>Courier italic</i>	Indicates a user-defined value in syntax, such as <code>write_file design_list</code>
Courier bold	Indicates user input—text you type verbatim—in examples, such as <code>prompt> write_file top</code>
Purple	• Within an example, indicates information of special interest. • Within a command-syntax section, indicates a default, such as <code>include_enclosing = true false</code>
[]	Denotes optional arguments in syntax, such as <code>write_file [-format fmt]</code>
...	Indicates that arguments can be repeated as many times as needed, such as <code>pin1 pin2 ... pinN</code> .
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Bold	Indicates a graphical user interface (GUI) element that has an action associated with it.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing C.

Customer Support

Customer support is available through SolvNetPlus.

Accessing SolvNetPlus

The SolvNetPlus site includes a knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The SolvNetPlus site also gives you access to a wide range of Synopsys online services including software downloads, documentation, and technical support.

To access the SolvNetPlus site, go to the following address:

<https://solvnetplus.synopsys.com>

If prompted, enter your user name and password. If you do not have a Synopsys user name and password, follow the instructions to sign up for an account.

If you need help using the SolvNetPlus site, click REGISTRATION HELP in the top-right menu bar.

Contacting Customer Support

To contact Customer Support, go to <https://solvnetplus.synopsys.com>.

Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

Part 1: Introduction

Describes the PrimeSim tool, the PrimeSim Continuum, and how to configure the different licensing modes.

This manual is organized into the following Parts:

- [Part 1, Introduction](#)
- [Part 2, Input](#)
- [Part 3, Elements and Models](#)
- [Part 4, Options and Commands](#)
- [Part 5, Multi-CPU and GPU Support](#)
- [Part 6, PrimeSim Features](#)
- [Part 7, Output](#)

This Part presents the following chapters:

- [Chapter 1, Introduction to the PrimeSim Tool and the PrimeSim Continuum](#)
- [Chapter 2, Setting Up the PrimeSim Tool](#)

1

Introduction to the PrimeSim Tool and the PrimeSim Continuum

This chapter provides an overview of the basic features of the PrimeSim Pro and PrimeSim SPICE tools.

This chapter covers the following topics:

- [PrimeSim Simulator Overview](#)
- [PrimeSim Continuum Family Overview](#)

PrimeSim Simulator Overview

PrimeSim technology consists of two simulators: the PrimeSim Pro and PrimeSim SPICE tools.

- The PrimeSim Pro tool represents a next-generation FastSPICE architecture for fast and high-capacity analysis of modern DRAM and Flash memory designs. The new architecture is highly optimized to handle full-chip memory and image designs with high bandwidth, large power distributed networks and strict density requirements. Additionally, the PrimeSim Pro tool's unique heterogeneous architecture enables GPU simulation for even faster performance.

The FastSPICE engine in the PrimeSim Pro tool has several distinctive features that break down barriers commonly found in mixed-signal SoC simulations. The PrimeSim Pro tool takes advantage of improved circuit partitioning algorithms that solve complex topologies in all aspects of mixed-signal design, including custom, analog, and memory designs. The PrimeSim Pro tool can provide improvements in simulation speed and design capacity by running over multiple CPUs and GPU cards using multi-CPU and GPU technology for large custom blocks in FastSPICE.

See [PrimeSim Pro Features and Benefits](#) for details.

- The PrimeSim SPICE tool is a high-performance SPICE circuit simulator for analog, RF, and mixed-signal applications. The PrimeSim SPICE tool offers a unique multi-core/multi-machine capability that delivers highly scalable performance with sign-off accuracy, as well as a state-of-the-art engine leveraging GPU technology to further

increase performance. Additionally, the PrimeSim SPICE tool supports efficient high frequency noise analysis and S-Parameter handling, while offering advanced analysis capabilities for periodic and non-periodic time-domain and frequency-domain applications.

The PrimeSim SPICE tool is a true SPICE engine that has a single matrix solver and unique capability of solving the SPICE matrix over multiple CPUs/machines and GPU cards (multi-CPU and GPU technology) with linear scaling and true SPICE accuracy. This SPICE technology is well suited for sensitive analog blocks such as phase-locked loops (PLLs), analog-to-digital converters (ADCs), charge pumps, and other traditionally difficult circuits for SPICE.

See [PrimeSim SPICE Features and Benefits](#) for details.

PrimeSim Pro Features and Benefits

The PrimeSim Pro tool offers the following features and benefits:

- Highest performance while maintaining accuracy using proprietary synchronization techniques that improve handling of event propagation between partition and more accurate modeling
- Innovative RC optimization techniques with compensation schemes to providing fast, accurate simulation results, including IR drop and electromigration simulation, for fully extracted power networks
- Enhanced parallel scalability with cutting edge algorithms for matrix-level parallel partitioning and solving
- Efficient full-chip verification flows that significantly reduce TAT for multiple vector simulation
- Advanced self-heating effect algorithms that speed up self-heat and aging simulation
- Comprehensive dynamic circuit check feature set integrated with the PrimeWave Design Environment for design debug
- Foundation built on the golden PrimeSim HSPICE device model library, guaranteeing device model correlation among all PrimeSim Continuum engines
- Full set of Monte Carlo capabilities including Sigma Amplification
- Cloud ready, built-in support containers for running the tool in a container, as well as the Synopsys cloud environment

PrimeSim SPICE Features and Benefits

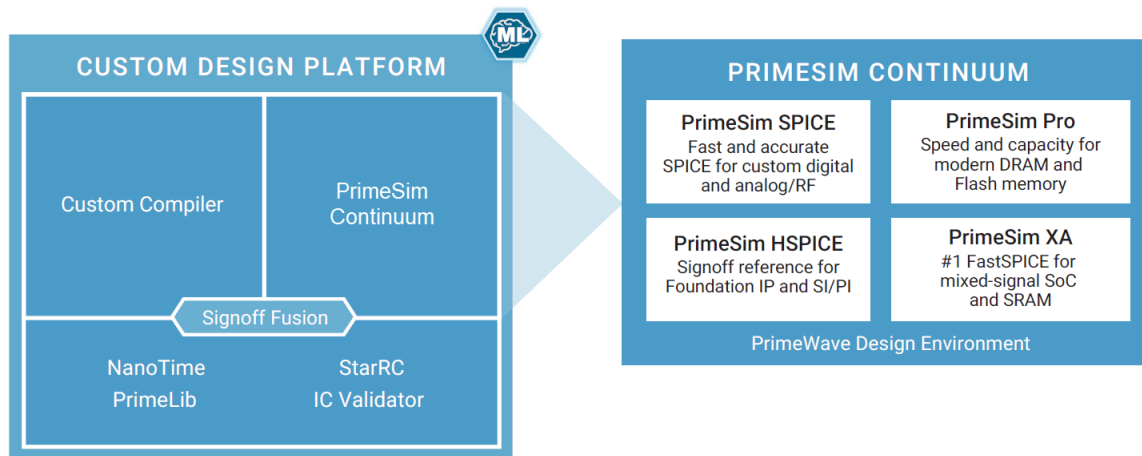
The PrimeSim SPICE tool offers the following features and benefits:

- Industry-leading multi-core and multi-machine capability complemented by support for GPU architectures for solving dense parasitic networks
- Period time and frequency domain analysis capability using Shooting Newton (SN) and Harmonic Balance (HB) based algorithms
- Efficient support for flicker, shot, and thermal noise types of transient noise analysis
- Optimized convergence algorithms providing superior results with post-layout circuits
- Advanced S-parameter algorithm that delivers faster performance for Signal Integrity Analysis
- Advanced self-heating effect algorithms that speed up self-heat simulation
- Comprehensive dynamic circuit check feature set integrated with the PrimeWave Design Environment for design debug
- Foundation built on the golden PrimeSim HSPICE device model library, guaranteeing device model correlation among all PrimeSim Continuum engines
- Full set of Monte Carlo capabilities including Sigma Amplification and High Sigma Monte Carlo
- Cloud ready, built-in support containers for running the tool in a container, as well as the Synopsys cloud environment

PrimeSim Continuum Family Overview

As shown in [Figure 1](#), the PrimeSim Pro and PrimeSim SPICE tools are key components of the PrimeSim Continuum family of simulators.

Figure 1 Key Components of PrimeSim Continuum



PrimeSim Pro and PrimeSim SPICE Tool Integration

The PrimeSim Pro and PrimeSim SPICE tools are designed to integrate seamlessly with the following Synopsys tools.

- *PrimeWave Design Environment*

The PrimeSim Pro and PrimeSim SPICE tools are fully integrated with the PrimeWave Design Environment, a new design environment and waveform viewer that encapsulates analysis and display requirements and significantly improves productivity.

The PrimeWave Design Environment combines simulation setup, analysis, waveform viewing, and post-processing onto one unified platform. It provides native integration to all PrimeSim Continuum simulation engines and all the advanced capabilities available within them, from analog and RF analysis, advanced variation analysis, custom fault analysis, device aging, to electromigration and IR drop analysis. The integrated waveform viewer offers fast loading, display scrolling, and zooming of very large waveform files as well support for multiple waveform file formats.

- *PrimeSim Reliability Analysis*

The PrimeSim Pro and PrimeSim SPICE tools are key engines inside the PrimeSim Reliability Analysis suite, a set of comprehensive solutions (fault simulation, static circuit check, MOS aging, IR/EM, and variation) that analyzes the full spectrum of a product life cycle, from early-life to end-of-life.

The PrimeSim Reliability Analysis suite is a set of comprehensive solutions that not only cover the verification of the entire product lifetime, early-life to end-of-life, but also span the full range of design validation from pre-layout phase to the post-layout stage. The solution suite includes PrimeSim CCK (static circuit check), PrimeSim Custom Fault (custom fault simulation and analysis), PrimeSim AVA (advanced

variation analysis), PrimeSim MOSRA (device aging and analysis), PrimeSim EMIR (electromigration and IR drop analysis), and SPRES (static power-net resistance).

- *VCS PrimeSim AMS*

The PrimeSim Pro and PrimeSim SPICE tools are tightly integrated into Synopsys' VCS digital simulator through a direct-kernel integration (DKI) which enables fast development of a rich feature set and continuous innovation. This mixed-signal solution delivers the highest throughput by combining the fastest transistor-level engine and the fastest digital engine.

The VCS PrimeSim AMS tool offers a flexible use model that allows for a broad mixture of mixed-signal modeling languages and abstractions levels and is integrated with both the PrimeWave Design Environment for design and verification, as well as the Verdi AMS Debug Environment for advanced digital and mixed-signal debug.

2

Setting Up the PrimeSim Tool

This chapter describes how to set up the PrimeSim tool.

This chapter covers the following topics:

- [Configuring the PrimeSim License](#)
- [Setting Up the Path to the Executable](#)

Configuring the PrimeSim License

You must set up one of the following environment variables to point to the license paths:

- `SNPSLMD_LICENSE_FILE` – A pointer to the Synopsys license paths
- `LM_LICENSE_FILE` – A pointer to all license paths

Note:

If you specify both environment variables, `SNPSLMD_LICENSE_FILE` takes precedence over `LM_LICENSE_FILE`.

When specifying the pointer to the license paths, use the full path of the license file, the `port@host` syntax, or a combination of the two syntax types in a colon-separated list.

In C-Shell:

```
setenv SNPSLMD_LICENSE_FILE pointer_to_license_paths
```

In Bourne, Korn, or Bash shells:

```
export SNPSLMD_LICENSE_FILE=pointer_to_license_paths
```

The PrimeSim tool supports the license modes described in the following sections:

- [Configuring PrimeSim Continuum Token-Based License Mode](#)
- [Configuring Standalone License Mode](#)

Configuring PrimeSim Continuum Token-Based License Mode

Use one of the following methods to enable PrimeSim Continuum token-based license mode:

- Define the shell environment variable `PRIMESIM` to 1.

In C-Shell:

```
setenv PRIMESIM 1
```

In Bash shell:

```
export PRIMESIM=1
```

For example:

```
% setenv PRIMESIM 1  
% primesim netlist.sp
```

- Use the `-primesim 1` command-line option.

For example:

```
% primesim netlist.sp -primesim 1
```

Note:

If the shell environment variable is set and the `-primesim` command line option is used, the command-line option takes precedence and overrides the shell environment variable setting.

[Table 1](#) lists the shell environment variables that control the token-based license mode and license queuing.

Table 1 Variables for Controlling Token-Based License Mode and License Queuing

Environment Variable	Description
<code>PRIMESIM 0 1 2</code>	This variable enables PrimeSim Continuum token-based license mode. Valid values: • 0: Disables token-based license mode. • 1: Enables token-based license mode. • 2: (Default) Enables standalone and token-based license mode. By default, the standalone keys are attempted first (see <code>PRIMESIM_ORDER</code>). This variable can also be enabled via the <code>-primesim</code> command line option.

Table 1 *Variables for Controlling Token-Based License Mode and License Queuing*
(Continued)

Environment Variable	Description
<code>PRIMESIM_ORDER 0 1</code>	This variable controls whether standalone keys or token-based keys are attempted first when the <code>PRIMESIM</code> variable is set to 2. • 0: (Default) Standalone keys are attempted first. • 1: Token-based keys are attempted first.
<code>PRIMESIM_WAIT_LICENSE 0 1</code>	This variable enables and disables license queuing. • 0: Disables license queuing. • 1: (Default) Enables license queuing.
<code>PRIMESIM_WAIT_LICENSE_TIMEOUT value</code>	This variable controls the timeout period for license queuing. The value is in minutes and cannot exceed 43,200 minutes (30 days). The range is [1, 43200] in minutes. The default period is 2 minutes. Non-positive integer values will result in an error. The following example sets the queue timeout period to 30 minutes: <pre>% setenv PRIMESIM_WAIT_LICENSE_TIMEOUT 30</pre>
<code>PRIMESIM_WAIT_LICENSE_INTERVAL value</code>	This variable controls the timeout period of sleeping during license queuing. The value is in seconds and cannot exceed 60 seconds and cannot be less than 1 second. The default period is 30 seconds. Integer values beyond [0, 60] will result in a warning message with resetting into the default 30 seconds. The following example sets the queue timeout period to 2 seconds: <pre>% setenv PRIMESIM_WAIT_LICENSE_LICENSE 2</pre>
<code>PRIMESIM_ONCE_CHECKOUT_LIC 1</code>	Set this environment variable to 1 to control whether to turn off the license incremental checkout function. By default, the incremental license checkout is enabled in PrimeSim SPICE and Pro, and PrimeSim HSPICE, except for PrimeSim XA.

Configuring Standalone License Mode

This is the default license mode if PrimeSim Continuum token-based license mode is not enabled.

If the required license keys for the PrimeSim tool are not available when it is invoked, the tool ends with an error. You can instruct the PrimeSim tool to queue for a license by setting the `PRIMESIM_WAIT_LICENSE` environment variable. When `PRIMESIM_WAIT_LICENSE` is

specified and the license keys are not available, the PrimeSim tool remains alive and idle until a license key becomes available. For example:

In C-Shell:

```
setenv PRIMESIM_WAIT_LICENSE 1
```

In Bourne, Korn, or Bash shells:

```
export PRIMESIM_WAIT_LICENSE=1
```

When the wait license feature is enabled, the PrimeSim tool queues indefinitely for an available license key. To time out the license wait feature, set the `PRIMESIM_WAIT_LICENSE_TIMEOUT` environment variable. The specified value is in minutes.

For example:

In C-Shell:

```
setenv PRIMESIM_WAIT_LICENSE_TIMEOUT timeout_in_minutes
```

In Bourne, Korn, or Bash shells:

```
export PRIMESIM_WAIT_LICENSE_TIMEOUT timeout_in_minutes
```

Setting Up the Path to the Executable

The PrimeSim executable is called by the launch script path. The launch script is in the `PRIMESIM_INSTALL_DIR/bin` directory. You must set up the shell path to include the path to the PrimeSim launch script. For example:

C-Shell:

```
set path = (PRIMESIM_INSTALL_DIR/bin $path)
```

Bourne, Korn, or Bash shells:

```
export PATH=PRIMESIM_INSTALL_DIR/bin:$PATH
```


Part 2: Input

Describes how to invoke the PrimeSim tool.

This Part presents the following chapters:

- [Chapter 3, Input Details](#)
- [Chapter 4, Running the PrimeSim Tool](#)
- [Chapter 5, Digital I/O Vectors](#)

3

Input Details

This chapter describes PrimeSim tool's supported netlist formats, as well as support for compression and encryption.

This chapter covers the following topics:

- [PrimeSim Input Requirements](#)
- [Supported Netlist Formats](#)
- [Support for Compressed Files](#)
- [Support for Encryption](#)

PrimeSim Input Requirements

The PrimeSim tool requires the following inputs:

- A transistor-level netlist.
- Process SPICE models (such as `BSIM4.8`).
- A flat or hierarchical Detailed Standard Parasitic Format (DSPF) file (required for back-annotation simulation only).

If you have a SPICE simulation environment for another simulator, the PrimeSim tool can use that as-is without any modifications.

For a list of supported output formats, see [Output](#).

Supported Netlist Formats

The PrimeSim tool supports PrimeSim HSPICE-formatted and 3rd-party netlists.

Support for Compressed Files

To save disk space and to improve performance on large netlists, the PrimeSim tool supports the `gzip` (.gz) file and `zstd` (.zst) file formats for both input files (`design.sp`) and the files associated with the following statements:

- `.INCLUDE / INC / INCL`
- `.LIB`
- `.LOAD`
- `.option primesim_restore (.SNAPSHOT)`

When both `design.sp` and `design.sp.gz` exist, the PrimeSim tool always selects the exact match file first, if it exists. For example:

1. If `.include design.sp` is in the netlist, then the PrimeSim tool picks `design.sp`.
2. If `.include design.sp.gz` is in the netlist, then the PrimeSim tool picks `design.sp.gz`.
3. If `.include design.sp` is in the netlist and only `design.sp.gz` exists, then the PrimeSim tool chooses `design.sp.gz`.

Note:

The PrimeSim tool does not support the S-parameter and RLGC file input types in the `gzip` and `zstd` file formats.

Support for Encryption

The PrimeSim tool supports encrypted models and file structures (see `.PROTECT / PROT` and `.UNPROTECT / UNPROT`) through the `metaencrypt` utility.

For more information, see the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

4

Running the PrimeSim Tool

This chapter describes how to run the PrimeSim tool and use its command-line options.

This chapter covers the following topics:

- [Command-Line Syntax](#)
- [Command-Line Options](#)
- [Including PrimeSim Options](#)
- [Restarting Transient Simulations From Snapshots](#)
- [Restarting Simulations With Vector Changes](#)

Command-Line Syntax

The PrimeSim Pro and PrimeSim SPICE tools share a unified command-line syntax. You can run a single CPU, multi-CPU, GPU, or multi-machine environment with either the PrimeSim Pro or PrimeSim SPICE tool by using the same command:

```
$ primesim [options] SPICE_deck_file_name
```

Examples

- `primesim in.sp` – This invokes the PrimeSim Pro tool in single CPU mode.
- `primesim -spice in.sp` – This invokes the PrimeSim SPICE tool in single CPU mode.
- `primesim -np no_of_process in.sp` – This option calls the PrimeSim Pro tool and runs in parallel mode.
- `primesim -np no_of_process -spice in.sp` – This option calls the PrimeSim SPICE tool and runs in parallel mode.

- `primesim -np no_of_process -ngpu no_of_gpus in.sp` – This option calls the PrimeSim Pro tool and runs in parallel mode on CPU cores and GPU cards.
- `primesim -np no_of_process -ngpu no_of_gpus -spice in.sp` – This option calls the PrimeSim SPICE tool and runs in parallel mode on CPU cores and GPU cards.

Command-Line Options

Table 2 lists the PrimeSim command-line options (for additional Multi-CPU command-line options, see Table 30 in [Command-Line Options for Running Multi-CPU Simulations](#)).

Table 2 PrimeSim Command-Line Options

Command-Line Option	Description
<code>-arch COMPATIBLE</code>	Switches to one more compatible math lib used instead of default Intel math lib.
<code>-webhelp</code> <code>-webhelp=srch_term</code> <code>-webhelp=[srch_term1 srch_term2 ...]</code>	Opens a full online Help collection in a web browser. Click “Using Help” in the left-side navigation bar of the Help window for details about how to use Help features such as the Help icons, Search, and Bookmarks. If you include a search term, the online help opens directly to a Search Results page. To search for a multiple word string, enclose the set of search terms in square brackets ([]).
<code>-h</code>	Prints a message that lists the command-line options.
<code>-primesim [0 1 2]</code>	Set to 1 to enable PrimeSim Continuum token licensing mode. Default is 2. See Configuring the PrimeSim License for details.
<code>-p</code>	Prints usage information for parallel simulation (see Table 30).
<code>-np N</code>	Runs parallel simulation with the specified number (<i>N</i>) of processors. Use <code>primesim -p</code> to get more detailed information or see PrimeSim Multi-CPU . Additional multi-CPU command-line options are described in Table 30.
<code>-spectre</code>	Specifies the name of the input netlist in the Spectre format.
<code>-spice</code>	Runs the PrimeSim SPICE tool; without this option, it runs the PrimeSim Pro tool.

Table 2 PrimeSim Command-Line Options (Continued)

Command-Line Option	Description
<code>-runlvl 1 2 3 4 5 6</code>	Specifies the accuracy level setting for transient analog simulation in the PrimeSim SPICE tool only, where 1 is best performance and 6 is best accuracy. Default if not specified on the command line is 4. See PrimeSim SPICE Accuracy and Speed Options (-runlvl and .option runlvl) for details.
<code>-spice_lic</code>	Runs the PrimeSim Pro tool's SPICE modes (<code>spicehd</code> , <code>spicemd</code> , and <code>spicexd</code>) with SPICE license check-out only. As of the S-2021.09 release, any localized <code>primesim_mode</code> SPICE settings in the netlist are also honored when <code>-spice_lic</code> is set.
<code>-mode value</code>	Overrides the <code>primesim_mode</code> value. (PrimeSim Pro only)
<code>-speed value</code>	Overrides the <code>primesim_speed</code> value. (PrimeSim Pro only)
<code>-model value</code>	Overrides the <code>primesim_model</code> value.
<code>-rcred value</code>	Overrides the <code>primesim_rcred</code> value.
<code>-tolscale value</code>	Overrides the <code>primesim_tolscale</code> value.
<code>-hspice file_name</code> <code>-i file_name</code>	Specifies that the input file, <code>file_name</code> , is in PrimeSim HSPICE netlist format.
<code>-o prefix</code> <code>-out prefix</code>	Specifies the prefix for the output file.
<code>-log file_name</code>	Specifies the log file name.
<code>-w</code>	Suppresses warning messages.
<code>-method integ_method</code>	Overrides the <code>primesim_method</code> value.
<code>-format type</code> <code>-wavefmt type</code>	Overrides the <code>primesim_output</code> value.
<code>-stop time</code> <code>-t time</code> <code>-time time</code>	Specifies the simulation stop time.
<code>-istop [time]</code>	If invoked <i>with</i> a time, specifies the time to interrupt the simulation and enter Interactive Mode . If invoked <i>without</i> a time, enters interactive mode immediately and the simulation runs to the <code>.tran</code> line end value.

Table 2 PrimeSim Command-Line Options (Continued)

Command-Line Option	Description
<code>-istop_file file_name</code>	Specifies a batch file containing Interactive Commands to be used when the <code>-istop</code> command-line option is used to enter interactive mode. If <code>-istop_file</code> is used without <code>-istop</code>, the tool exits with an error.
<code>-auto_restart [-stop new_tstop]</code>	Looks for any snapshots created with the same output prefix and restores the transient simulation from the last saved state. If the snapshot was created with primesim_auto_restart modes 2 or 3, the PrimeSim tool skips input parsing and starts at the last saved state. If <code>-stop new_tstop</code> is specified with a new <code>tstop</code> value, the original transient simulation <code>tstop</code> is extended to the <code>new_tstop</code> time. If the <code>new_tstop</code> is less than or equal to the original <code>tstop</code>, this is ignored. When there are no snapshots with the same output name, this option is ignored. See Restarting Transient Simulations From Snapshots for more information.
<code>-syntaxcheck</code>	Checks the netlist syntax.
<code>-inc path</code>	Specifies the include path.
<code>-ifile file_name</code>	Specifies the file to insert.
<code>-afile file_name</code>	Specifies the file to append.
<code>-aopt option[=value]</code>	Specifies an option to append, taking precedence over the same option in the netlist or global configuration file. Example: <pre>primesim input.sp -aopt primesim_fcamin=10f</pre> This command appends the <code>primesim_fcamin=10f</code> option to the simulation. If the <code>primesim_fcamin</code> option appears in the netlist or global configuration file, this command-line option takes precedence.
<code>-top subcircuit_name</code>	Specifies the top-level subcircuit name.
<code>-case [0 1]</code>	Enables case sensitivity in the input netlist. Specify <code>-case 1</code> to enable case sensitivity. The default behavior depends on netlist format. For HSPICE format, it is “case-insensitive”, while for spectre format, it is “case-sensitive”.

Table 2 PrimeSim Command-Line Options (Continued)

Command-Line Option	Description
<code>-rchk netlist.sp</code>	The file after <code>-rchk</code> is used to recheck the database. It should include the same netlist when the wave file is created, but be allowed to modify the <code>.chkdevop</code> <code>.chkrftime</code> <code>.chktiming</code> condition. The PrimeSim tool checks the netlist and is required to parse to match the device to the waveform information. If <code>netlist.sp</code> does not have any <code>.chkdevop</code> <code>.chkrftime</code> <code>.chktiming</code>, then the PrimeSim tool errors out with an error message.
<code>-wvfile netlist.fsdb</code>	The file after <code>-wvfile</code> is used to recheck the post-process with <code>-rchk netlist.sp</code>. Currently, the PrimeSim tool supports FSDB format only.
<code>-pvchk file_name</code>	Specifies the FSDB file for PrimeSim Post Vector Check .
<code>-sdp</code>	Enables the Synopsys Diagnostic Platform.
<code>-container</code>	Runs the PrimeSim tool in a container. A container bundles applications and their system dependencies so the applications can run on any host machine regardless of which packages are installed on the host. This is especially useful for running applications in cloud environments. Note: Alternatively, you can set the <code>SNPS_CONTAINER</code> environment variable to 1, although this command-line option is the preferred method.
<code>-datamining</code>	Executes the PrimeSim AVA tool's standalone data mining flow based on Monte Carlo results. See Data Mining Using PrimeSim With PrimeSim AVA for more information.
<code>-v</code>	Displays version information to the terminal.
<code>-version</code>	

Table 2 PrimeSim Command-Line Options (Continued)

Command-Line Option	Description
<code>-gz</code>	Enables <code>gzip</code> file compression of the text format output files for the following: • <code>.IC</code> results • <code>.OP</code> results • <code>.AC / .DC / .NOISE</code> results • Circuit Checks commands results • <code>.MEASURE / MEAS</code> results • <code>.PRINT</code> results • <code>.ALTER</code> results (<code>.tr#</code> files) Note: <code>gzip</code> file compression of the log file is not supported.
<code>-r input.tcl</code>	Runs the SPRES solver using commands from <code>input.tcl</code> .
<code>-rout rout_file</code>	Post-processes files with sorted resistances to print only resistances for selected pins.
<code>-soacheck</code>	Runs SOA checks on the first two time steps in Transient Analysis and then exits the simulation. See primesim_soa_check_on_dc_init for more information.
<code>-lock [0 1 2]</code>	Specifies how the PrimeSim tool should handle multiple processes running simulations with the same output prefix name: • 0: No lock; files are overwritten. • 1: (default) Rename log file with unique name and proceed with the simulation. • 2: Exit with an error message.
<code>-map_op</code>	Use with 3rd-party netlists that contain both AC and DCOP analyses requests. This option instructs the PrimeSim tool to ignore redundant DCOP requests and generate the IC file from the AC analysis' DC operating point analysis. Note: This option should not be used in an "AC at Diff T" simulation (see primesim_ac_at_diff_t).
<code>-bslash</code>	Removes any backslashes (<code>\</code>) being used in all kind of names, such as instance names, node names, subckt names, model names, parameter names, and names in control statements.
<code>-3dic="subckt_list"</code>	Use with 3rd-party netlists to specify subcircuits for Multi-Technology Simulation (3DIC) modules where <code>subckt_list</code> supports wildcard characters: • <code>*</code> matches a sequence of characters. • <code>?</code> matches a single character.

Table 2 PrimeSim Command-Line Options (Continued)

Command-Line Option	Description
<code>-im [-opt] file [file2 ...]</code>	Invokes IM, a GUI-based post-processing tool for inspecting measurements. See Using IM for more information.
<code>-vec2pwl vector_file</code>	Converts the specified vector file into a PWL file.
<code>-incremental directory/output_prefix</code>	Use to add more samples to a completed Monte Carlo analysis run and merge the results. <i>directory/output_prefix</i> specifies the output directory and the output prefix of the completed samples. See Incremental Runs for more information.
<code>-restart_dbfile=Base_name[@time]</code>	Specifies the base file name for the <code>.restart_db</code> file for both initial and restart runs. - If the <code>-restart_vector</code> option is not used, the PrimeSim tool generates the specified <code>.restart_db</code> file. Note: If the <code>-restart_dbfile</code> option is used in the initial run without the <code>-restart_time</code> option, the PrimeSim tool generates one <code>.restart_db</code> file at time 0. - If the <code>-restart_vector</code> option is used, the PrimeSim tool restarts simulation after reading the specified <code>.restart_db</code> file. Note: The <i>time</i> argument should only be specified for the restart run. See Restarting Simulations With Vector Changes for more information and examples.
<code>-restart_time="t1 t2 ..."</code>	Can be used by itself or with the <code>-restart_dbfile</code> option in the initial run to generate <code>.restart_db</code> files at the specified times (<i>tn</i>). Note: If the <code>-restart_dbfile</code> option is used in the initial run without the <code>-restart_time</code> option, the PrimeSim tool generates one <code>.restart_db</code> file at time 0. See Restarting Simulations With Vector Changes for more information and examples.

Table 2 PrimeSim Command-Line Options (Continued)

Command-Line Option	Description
<code>-restart_vector="stimulus_file1 stimulus_file2 ..."</code>	Specifies one or more stimulus files to use in a restart run. Note: This option must be used with the <code>-restart_dbfile</code> option, so that the PrimeSim tool restarts the simulation with the specified <code>.restart_db</code> file using the specified stimulus files. See Restarting Simulations With Vector Changes for more information and examples.

Including PrimeSim Options

The PrimeSim tool options start with `.option` and can be inserted either in the SPICE deck input file or grouped into one file, such as `option.inc`, that is included in the original SPICE deck.

Example

SPICE deck `in.sp` file in tutorial directory:

```
*** 128 inv chain ***
* primesim option
* primesim option is added through a separate option file
.inc './option.inc'
* primesim option is added directly in SPICE deck
*.option primesim_mode=promd
.probe v(*)
.param supplyd=1.5v
VDDD2 VDD! 0 supplyd
VSSD2 GND! 0 0
.global VDD! GND!
VIN IN 0 pw1( 0 0 5n 0 5.3n supplyd )
.measure tran delay trig v(in) val='supplyd/2' rise=1
+          targ v(out) val='supplyd/2' rise=1

.tran 1ps 30n
:
:
```

Where the `option.inc` file contains:

```
.option primesim_mode=promd
```

To run the PrimeSim Pro tool on this circuit, enter the following command:

```
% primesim in.sp
```

These commands generate the following output files:

```
in.log : PrimeSim Pro log file;  
in.fsdb : Waveform in .fsdb format can be viewed by PrimeWave Design  
Environment WaveView;  
in.ic : .ic file from DC initialization process;  
in.mt0 : measurement values from .measure card.
```

Restarting Transient Simulations From Snapshots

By using a combination of the `primesim_auto_restart` option and the `-auto_restart` command-line option, you can take periodic snapshots of the entire memory image of the current simulation state and then use a snapshot image to restore the transient simulation to the exact same state. This can be useful if the simulation ends unexpectedly or if you want to extend the originally defined stop time because when you restore the simulation this way, the PrimeSim tool skips all parsing and elaboration and restarts directly from the last saved state.

For details, see the following topics:

- [Generating Image Dump Snapshots](#)
- [Restoring From Snapshots](#)
- [Limitations](#)

Generating Image Dump Snapshots

The PrimeSim tool supports the following methods for generating snapshots of the simulation:

- Use the `primesim_auto_restart` option. For example, the following statement generates snapshots at every 20% of `tstop` and keeps the last one at 100% (because mode 3 is specified):

```
.option primesim_auto_restart="3 20"
```

- Send `kill` signal 12 from the same machine that is running the simulation to generate a snapshot at the transient time when the PrimeSim process is terminated. In the following example, `pid` is the process ID of the PrimeSim job:

```
kill -12 pid
```

- When using LSF, send `bkill` signal 12 to generate a snapshot at the transient time when the PrimeSim job is terminated. For example:

```
bkill -s 12 job_id
```

Restoring From Snapshots

Run the PrimeSim tool using the `-auto_restart` command-line option, as follows:

- To restart simulation directly from the last saved state:

```
primesim input.sp -o output_name -auto_restart
```

Note:

The output name must be the same as when the snapshot was generated. If it is different, the PrimeSim tool reruns the simulation from the start.

- To restore from the last snapshot and extend the transient simulation to new `tstop`:

```
primesim input.sp -o output_name -auto_restart -stop new_tstop
```

Note:

The value for `new_tstop` must be greater than the original `tstop`. This feature only works if `primesim_auto_restart` mode 3 was used to generate the snapshot.

Limitations

This feature has the following limitations:

- The PrimeSim binary must be the same for snapshot generation and restart.
- The Linux Kernel and System Architecture must be the same for snapshot generation and restart.
- Restart may not work if the CPU model is different in the restart run even with the same kernel.
- The simulation condition, such as changing the number of CPUs, cannot be changed.
- Does not work with DATA sweeps including Monte Carlo.
- Any changes in the netlist or options are ignored with `primesim_auto_restart` modes 2 or 3.

Restarting Simulations With Vector Changes

You can use the Image Dump Reuse (IDR) Vector Change Flow feature to run an initial simulation while generating one or more snapshots of it (`.restart_db` files), then restart the simulation using different vector stimuli.

For details, see the following topics:

- [Generating the .restart_db File](#)
- [Restarting From a .restart_db File With New Stimulus](#)
- [Examples](#)
- [Supported Statements in the Stimulus File](#)
- [Limitations](#)

Generating the .restart_db File

Generate one or more .restart_db files using either or both of the following [Command-Line Options](#):

- `-restart_dbfile=Base_name`
Specifies the base file name for the .restart_db file(s) to be generated.
- `-restart_time="t1 t2 ..."`
Specifies one or more times at which to generate the .restart_db file.

Examples

The following example generates a file at time 0 called `ABC@0.restart_db`:

```
primesim input.sp -o output -restart_dbfile=ABC
```

The following example generates a file at time 1u called `output@1u.restart_db`:

```
primesim input.sp -o output -restart_time=1u
```

The following example generates two files, one at time 0 and another at time 1u, called `ABC@0.restart_db` and `ABC@1u.restart_db`, respectively:

```
primesim input.sp -o output -restart_dbfile=ABC -restart_time="0 1u"
```

Restarting From a .restart_db File With New Stimulus

After generating one or more .restart_db files, restart simulation using the following two [Command-Line Options](#):

- `-restart_dbfile=Base_name@time`
Specifies the .restart_db file to be read before restarting simulation.

- `-restart_vector="stimulus_file1 stimulus_file2 ..."`

Specifies one or more stimulus files to use with the restarted simulation. A stimulus file can be a complete vector file or a file that contains other allowed statements as described in [Supported Statements in the Stimulus File](#).

Examples

The following example reads `ABC@0.restart_db` and restarts the simulation from time 0 with the new stimulus specified in `stim1`:

```
primesim input.sp -o output -restart_dbfile=ABC@0 -restart_vector="stim1"
```

The following example reads `ABC@0.restart_db` and restarts the simulation from time 0 with the new vector file `read.vec`:

```
primesim input.sp -o output -restart_dbfile=ABC@0  
-restart_vector="read.vec"
```

The following example reads `ABC@1u.restart_db` and restarts the simulation from time 1u with the new stimulus specified in `stim1` and `read.vec`:

```
primesim input.sp -o output -restart_dbfile=ABC@1u -restart_vector="stim1  
read.vec"
```

Examples

The following examples show both steps of the IDR-based Vector Change Flow feature.

Example 1

The following example generate a `.restart_db` file at time 0 with the base name of `ABC`. Then it restarts simulation with a new vector file called `stim1` in one run and `stim2` in the next run:

```
primesim input.sp -o output -restart_dbfile=ABC  
  
primesim input.sp -o output -restart_dbfile=ABC@0 -restart_vector="stim1"  
primesim input.sp -o output -restart_dbfile=ABC@0 -restart_vector="stim2"
```

Example 2

The following example generates `.restart_db` files at time 0 and 1u with the base name of `ABC`. Then it restarts simulation at time 0 with a new vector file called `stim1` and restarts simulation again at 1u with a new vector file called `read.vec`:

```
primesim input.sp -o output -restart_dbfile=ABC -restart_time="0 1u"  
  
primesim input.sp -o output -restart_dbfile=ABC@0 -restart_vector="stim1"  
primesim input.sp -o output -restart_dbfile=ABC@1u  
-restart_vector="read.vec"
```

Supported Statements in the Stimulus File

The following statements in the stimulus file are supported:

- Voltage Source (PWL/PULSE...) statement – The use of PWL sources is recommended. See [Functions for Independent Sources](#).
- Behavioral Voltage Source statement (Exxx a gnd VOL='expression'). See [Voltage Source \(V-element\) and Current Source \(I-element\) Elements](#).
- Parameter statement (.PARAM) – The new parameters defined affect only the new stimulus definitions even if the same parameter names existed in the original netlist. See .PARAM / PARAMETER / PARAMETERS.
- Vector statement (.VEC) – A complete vector file can be used. If a new *tstop* is not specified with .TRAN in one of the stimulus files, the last time point in the vector file is taken as the new *tstop* if it is larger than the original *tstop*. See .VEC.
- .FORCE and .RELEASE statements.
- Include statement (.INC) – The included file can have other supported statements. See .INCLUDE / INC / INCL.
- Transient statement (.TRAN) – Only *tstop* is used in restart simulation to change the simulation end time according to the stimulus change. If .TRAN with a new *tstop* is not specified, the PrimeSim tool uses the *tstop* in the original netlist. See .TRAN.

Limitations

This feature has the following limitations:

- The voltage level for all nodes in the circuit at the time of .restart_db file generation must remain the same in the newly-defined voltage sources. Otherwise, the PrimeSim tool ends simulation with an error.
- The PrimeSim binary must be the same for .restart_db file generation and restart runs.
- The initial and restart runs must be on the same Linux kernel and system architecture.
- The restart run may not work if the CPU model is different in the initial run even with the same kernel.
- The same number of cores must be used for .restart_db file generation and restart runs. During the restart runs, the PrimeSim tool uses the same number of cores as when the snapshot was generated even if you specify a different number.

- This feature does not work with Monte Carlo or `.ALTER`.
- Any changes in the input netlist are ignored by default during the restart runs (`primesim_checksum_on=0` by default). In restart runs, the PrimeSim tool reads only the stimulus statements specified by the `-restart_vector` option.

5

Digital I/O Vectors

This chapter describes the PrimeSim tool support of digital I/O vector files.

The PrimeSim tool usually reads a stimulus directly from a SPICE deck, such as PWL for input CLK. But for digital circuits, which generate and read vectors as stimuli, the PrimeSim tool supports a tabular I/O-vector format to accommodate the digital simulation design environment. This vector format has time-varying input signal patterns and expected output patterns that are used for simulation result comparison.

To read the I/O vector file, you must set the vector file name as in the following example:

```
.option primesim_vector='control.vec'
```

This chapter covers the following topics:

- [Direct VCD Input](#)
- [Vector File Format](#)
- [PrimeSim Post Vector Check](#)
- [PrimeSim Circuit Examples](#)

Direct VCD Input

The PrimeSim tool can read in the VCD (Value Change Dump) file by converting the VCD file into a vector file automatically. You need either an fscript command file or signal file in order to use this feature. Alternatively, you can use the `fscript` utility to first convert your VCD files to `io_vector` files and then input them into the PrimeSim tool directly. For more details about this utility, see [PrimeSim Utilities](#). The PrimeSim tool can read in VCD files directly by using the `.vcd2vec` command, as follows:

```
.vcd2vec fscript_command_file
```

or:

```
.vcd2vec vcd_file signal_file
```

The PrimeSim tool has limited support for a `signal_file` that is commonly used by other simulators for converting VCD to VEC. The list of `signal_file` features includes:

```
#tunit - fset tunit
#trise - fset rise
#tfall - fset fall
#slope - fset slope
#vih - fset vih
#vil - fset vil
#voh - fset voh
#vol - fset vol
#scale - fset scale
#idelay - fset tdelay
#scope - fset scope
#in - vsignal ... in bin
#out - vsignal ... out bin
#bidirectional|#bi sname1 [... snameN][in|out enable_condition]
#alias - valias
*fset fscript_fset_command
```

The `bidirectional` vector becomes input (if `in` is specified) or output (if `out` is specified), if the `enable_condition` argument is true. You can define a single signal in `enable_condition` or a logical expression with multiple signals.

Example

```
#bi data (out en)
#bi iobus* (in write_en & cs)
```

The `bidirectional` vector supports the following logical expression operators:

- NOT: `~`, `!`
- AND: `&`, `&&`
- OR: `|`, `||`
- XOR: `^`

Note:

Signals used in `enable_condition` should be vector signals.

An asterisk (*) usually dictates comments in a signal file. In order to avoid conflicting syntax, because `fset` is a script-specific command, `fscript fset` commands can be set using `*fset`.

Example

```
*fset vcd2vec bus_separator < >
```

The above is converted into `"fset vcd2vec bus_separator < >"`.

Consider the following example, where there is a `vcd2vec` statement in the netlist:

```
.vcd2vec "sample.sig"
```

The fscript command file `sample.sig` is as follows:

```
load vcd sample.vcd
open sample.vec

fset vih 2.5
fset vil 0.0
fset slope 1ns

vinit
vsignal x1.a in bin
vsignal x1.z out bin
vrun
```

The output vector file `sample.vec` is generated as follows and the PrimeSim tool reads this vector file automatically.

```
; input file : sample.vcd
; vector pattern definitions

radix
+ 1
+ 1

vname
+ x1.a
+ x1.z

io
+ i
+ o

; waveform parameter setting
tunit ns
      slope      1
      vil        0
      vih        2.5

; tabular data (these values may differ in case)
      0 0 0
     10 1 0
     12 1 1
```

Vector File Format

An I/O vector file has three sections in the following order:

1. [Vector Pattern Definition](#)
2. [Waveform Parameter Settings](#)
3. [Tabular Data](#)

The vector pattern definition declares signal (vector) names, vector sizes, and vector types. A vector type specifies whether the signal is an input or an expected output that should be used for comparison.

The waveform parameter settings define a variety of signal attributes. For example, they define the time unit, the rise and fall time, the driving strength, the thresholds for logic *high* or *low*, and so on.

The tabular data lists the values of input signals at specified times. The time may be listed in the first column, followed by signal values, in the order specified by the `nodename` statement of the vector pattern definition.

By default, the PrimeSim tool errors out and ends the simulation if there is no tabular data, but you can set the `primesim_ignore_no_vec_tdata_error` option to 1 to keep the simulation running.

To take the rise and fall time into consideration when the vector switches between 1 and 0 or vice versa, the PrimeSim tool provides the `pwl_type` definition you specify before the description of the tabular data but preceded by comment character `;`.

The following is an example I/O vector file:

```
; Vector Pattern Definition
radix 1 1 4 4 4 4 1 4 4
nodename clk out addr[15-0] R data[7:0]
io i o i i i i x b b
; Waveform Parameter Setting
Slope 2.1
VIH 5
; Tabular Data
; pwl_type 0 (or 1) ; Default is 0
1.0 L H 1 1 a e 1 0 0
2.5 1 0 z x 0 0 1 x 0
5.6 0 1 a b c d 0 1 2
```

In this example, the colon character (`:`) always means that this line is a comment line.

Vector Pattern Definition

Vector patterns are defined by the following parameters:

- [Radix](#)
- [NodeName](#)
- [Hierarchical Node Names in Vector Files](#)
- [I/O Statements](#)
- [Parameters as Time Values in the Vector File](#)

Radix

The Radix statement must be included as a non-comment line in the I/O vector file. Its valid values are 1 to 4, as defined in the following table.

Table 3 Radix Values

Radix	Number System	Ranges
1	binary	0-1
2	quaternary	0-3
3	octal	0-7
4	hexadecimal	0-F

Example

```
radix 1 1 4 4 4 4 1 4 4;
```

NodeName

The `nodeName`, `signal`, `sname`, or `vname` parameter defines the node name of each vector. Single-bit signals have one NodeName, and multiple-bit signals (bus signals) have bus notation with `[i:j]`, `[i~j]`, `[i~j]`, `<i:j>`, `<i~j>`, or `<i~j>`.

When the PrimeSim tool converts a bus signal to a bit signal, both a PrimeSim HSPICE rule and legacy HSIM rule are supported according to the set of `.vec`, `primesim_vector`, and `primesim_vector_mode` options. Therefore, it could be `a[1:2] -> a1, a2` (PrimeSim HSPICE rule) or `a[1:2] -> a[1], a[2]` (legacy HSIM rule). For more information, see [primesim_vector_mode](#).

Example

```
nodename clk out addr[15-0] R data[7:0];
```

In this example, `clk`, `out`, and `R` are single-bit signals and `addr` and `data` are bus signals.

When using `.vec` or `primesim_vector_mode=0`, the PrimeSim tool always ignores the bus bracket for PrimeSim HSPICE compatibility.

Example (`.vec` or `primesim_vector_mode=0`)

```
a<1~2> -> a1, a2
a<1:2> -> a1, a2
a[1:2] -> a1, a2
a<[1:2]> -> a<1>, a<2>
a[[1:2]] -> a[1], a[2]
a[<1:2>] -> a[1], a[2]
a_[1:2]_ -> a_1_, a_2_
a_[1:2] -> a_1, a_2
```

When using `primesim_vector` or `primesim_vector_mode=1`, the PrimeSim tool only ignores the bus bracket in the following two cases:

- The bus delimiter is a tilde (~).
- There is a string after the bus bracket.

Example (`primesim_vector` or `primesim_vector_mode=1`)

```
a<1~2> -> a1, a2 (ignore angle bracket because of '~')
a<1:2> -> a<1>, a<2>
a[1:2] -> a[1], a[2]
a<[1:2]> -> a<1>, a<2> (ignore square bracket because there is '>')
a[[1:2]] -> a[1], a[2]
a[<1:2>] -> a[1], a[2]
a_[1:2]_ -> a_1_, a_2_
a_[1:2] -> a_[1], a_[2] (PrimeSim keeps the square bracket because there
is no character after bus description)
```

Hierarchical Node Names in Vector Files

The PrimeSim tool supports hierarchical names for input signals in a vector file. It checks voltage sources for internal nodes, copying subcircuits when needed. For example, `signal x1.x2.a` can be an input or a bidirectional signal. The PrimeSim tool supports all levels of signals for value checking in vector files.

I/O Statements

An I/O statement starts with a keyword `io` and is followed by a string of `i`, `o`, `b`, `m`, or `x(u)`. The `io` types are defined in the following table.

Table 4 I/O Types

State	Meaning
i	An input stimulus signal
o	An expected output pattern
b	A bidirectional vector
m	The voltage range between <i>vih</i> and <i>vil</i>
x, u	Ignored

Example

```
io i o i i i i x b b ;
```

In this example, the first signal is an input, the second an output, the third through sixth input, the seventh ignored, and the eighth and ninth bidirectional.

Parameters as Time Values in the Vector File

The PrimeSim tool supports using parameters for the time values specified in a vector file. For example, a `.vec` file:

```
radix 1 1 1 1
nodename clk out in in1
io i o i i
Slope 1
VIH 2
time1 0110
time2 0111
```

where `time1` and `time2` are parameters in the input deck:

```
.param time1 = 10n
.param time2 = 20n
```

Waveform Parameter Settings

Waveform parameters for the PrimeSim tool are defined as follows:

- [Tunit](#)
- [Slope](#)
- [Tfall](#)
- [Trise](#)

- [idelay_f](#)
- [idelay_r](#)
- [VH or VOH](#)
- [VL or VOL](#)
- [VIH](#)
- [VIL](#)
- [VREF](#)
- [VTH](#)
- [Mask](#)
- [Enable](#)
- [Check_Window](#)
- [STOP_AT_ERROR](#)
- [Support for logicv and logiclv in Vector Files](#)
- [Delay Statements in Vector Files](#)
- [Support for VCHK_IGNORE in Vector Files](#)
- [Using the Context-Based Control Option \(CBC\)](#)

Tunit

Sets the time unit in the vector file. The default value is 1 ns.

Example

```
Tunit 0.1n;
```

Slope

Sets the rise and fall time slopes for an input signal. Slope is defined as 0-100% of a signal swing in the PrimeSim tool. This is overridden by the [Trise](#) or [Tfall](#) statement when used. The default value is 1 ps.

Example

```
Slope 1;
```

Tfall

Sets the fall time slope of an input signal. If [Tfall](#) is not specified, [Tfall](#)=[Slope](#).

Example

```
Tfall 1;
```

Trise

Sets the rise time slope of an input signal. If `Trise` is not specified, `Trise=Slope`.

Example

```
Trise 0.5;
```

idelay_f

Specifies the delay time for falling input vectors, relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is set to the value specified in the `delay` or `tdelay` statement (see [Delay Statements in Vector Files](#)).

Example

```
idelay_f 10;
```

idelay_r

Specifies the delay time for rising input vectors, relative to the absolute time of each row in the tabular data section. The delay time can be positive or negative. The default is set to the value specified in the `delay` or `tdelay` statement (see [Delay Statements in Vector Files](#)).

Example

```
idelay_r 10;
```

VH or VOH

Sets the logic threshold of `output 1` value. It is used for output pattern checking. The default value is `VIH/2`.

Examples

```
VH 3.0;  
VOH 3.0;
```

VL or VOL

Sets the logic threshold of `output 0` value. It is used for output pattern checking. The default value is `VIH/2`.

Examples

```
VL 2.0;  
VOL 2.0;
```

VIH

Sets the voltage level for `input 1-state`. VIH has no default value. If there is no description, the PrimeSim tool terminates with an error message.

Example

```
VIH 3.3;
```

VIL

Sets the voltage value for `input 0-state`. The default value is 0 V.

Example

```
VIL 0.8;
```

VREF

Specifies the name of the reference voltage for each input vector to which the mask applies.

The `VREF` command applies only to input signals and has no effect on the output signals. If you do not specify the reference voltage name of the signals in a `VREF` command, the PrimeSim tool assumes 0. If you apply more than one `VREF` command, the last command overrides the previous commands.

Syntax

```
VREF reference_voltage
```

or:

```
VREF reference_voltage_node [mask]
```

Examples

```
VNAME v1 v2 v3 v4 v5[[1:0]] v6[[2:0]] v7[[0:3]] v8 v9 v10  
VREF 0  
VREF 0 111 137F 000  
VREF vss 0 0 0 0000 111
```

When the PrimeSim tool implements these commands into the netlist, the voltage source realizes `v1`:

```
v1 V1 0 pw1(.....)
```

as well as `v2`, `v3`, `v4`, `v5`, `v6`, and `v7`.

However, `v8` is realized by:

```
V8 V8 vss pw1(.....)
```

`v9` and `v10` use a syntax similar to `v8`.

VTH

Specifies the logic threshold voltage for each output signal to which the mask applies.

The `VTH` command applies only to output signals and has no effect on the input signals. If `VTH` is not specified, then use `VIH/ 2.0`. If `VIH` is also not defined, then the tool errors out. If you apply more than one `VTH` command, the last command overrides the previous commands.

Syntax

```
VTH logic-threshold_voltage
```

Examples

```
VTH 1.75
VTH 2.5 1 1 1 137F 00000000
VTH 1.75 0 0 0 0000 11111111
```

Mask

Sets a mask function for signals defined in the vector file. It works on all keywords such as `vil`, `vih`, `trise`, `tfall`, `slope`, `tunit`, and `delay`. It is used when signals require special values other than the values defined globally. The signals used in `check_window` statement also can be defined with mask.

Syntax

```
mask mask_name signal1 [signal2 signal3 .....]
```

or:

```
mask mask_name mask_pattern
```

`mask_name` is the mask name referenced in the `check_window` statement or other parameter definitions.

Example

```
signal in1 in2 in3 in4
radix 1 1 1 1
io i i i i
mask mask1 in1 in4
mask mask2 0 1 0 1
vil 1v mask1
vil 0.75v mask2
```

```
vil 0.5v
.....
```

In the previous example, the `vil` for signal `in1` is `1v`; It is enabled by the first `vil` definition. The `vil` for `in2` is `0.75v`; It is enabled by the second definition of `vil`. The `vil` for `in3` is `0.5v`, it is disabled by the first two `vil` statements, and uses the default value. The `vil` for signal `in4` is `0.75v`. Actually the first and second definitions have a conflict for `in4`; for such situations, the last one is used.

Enable

Defines the bidirectional signals as input or output based on *enable_condition*, which can be a single signal or a logical expression with multiple signals.

When *enable_condition* is true, then bidirectional signals are treated as output. The bidirectional signals are selected based on *pattern* or *mask_name*.

Syntax

```
enable enable_condition pattern|mask_name
```

Example 1

```
nodename in1 in2 cs data[7:0];
radix 1 1 1 4 4
io i I i b b
enable cs 0 0 FF
```

Example 2

```
nodename addr[3:0] cs wen data[15:0];
radix 4 1 1 4444
mask exp_lowout 0 0 0 00FF
enable (cs && !wen) exp_lowout
```

Check_Window

Specifies the time window of the vector strobe that is defined as an output of the `io` statement. The output comparison is done within the time window.

Syntax

```
check_window begin_offset end_offset steady [mask_name]
```

or:

```
check_window begin_offset end_offset steady period_time first_time
[mask_name]
```

For the first syntax statement, the *begin_offset* and *end_offset* are the start and stop offset values, respectively. The default value and unit are `0` and `ns`. The time window is $[t - \text{begin_offset}, t + \text{end_offset}]$, where t is the vector stop time. The *steady* can be

defined as 1 or 0. If *steady*=1 and the unexpected output range or time exists in the check window, the result is an error. If *steady*=0 and the expected output range or time exists in the check window, the result is correct. If *mask_name* is specified, the check window is applied to the signals defined in the *mask* statement.

For the second syntax statement, the *begin_offset* and *end_offset* are the same with the first example. The time window is [*t* - *begin_offset*, *t* + *end_offset*], where *t* is *first_time*. The check is repeated every *period_time*. The *steady* can be defined as 3 or 2. If *steady*=3 and the unexpected output range or time exists in the check window, the result is an error. If *steady*=2 and the expected output range or time exists in the check window, the result is correct. If *mask_name* is specified, the check window is applied to the signals defined in the *mask* statement.

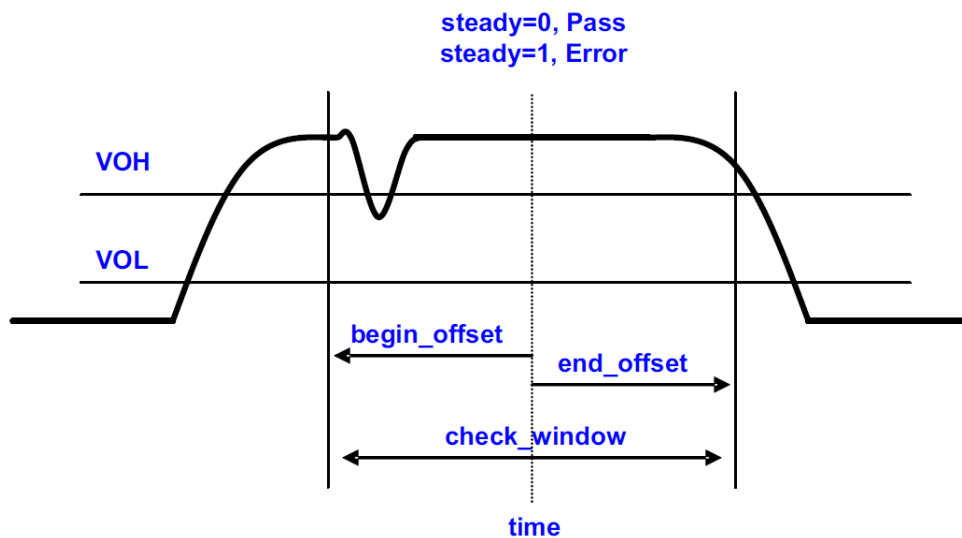
Example 1

```
signal DQ
radix 1
io o
check_window 0.5 0.5 1 mn1
mask mn1 DQ
time0 x
time1 1
.....
```

Example 2

```
signal Dout1 Dout2 Din1
radix 1 1 1
io o o i
check_window 1 1 3 20 5
timen 0 0 1
timen+1 1 0 0
```

Figure 2 *check_window Example Output*



STOP_AT_ERROR

Stops circuit simulation if output comparisons are performed resulting in mismatched outputs.

Example

```
vname clk D2 D3
radix 1 1 1
io i o o
mask m1 D2
stop_at_error
check_window 0.2 0.2 3 10 4 m1
```

Support for logichv and logiclv in Vector Files

The PrimeSim tool supports **logichv** and **logiclv**, which are equivalent to the existing **vih** and **vil** commands, respectively. Consider the following usage example:

```
logichv|logiclv value
```

In vector files, you can use **logichv** for describing voltage values of logic 1 and you can use **logiclv** for describing voltage values of logic 0.

Example

```
"logichv 1.2" -> logic '1' is replaced by 1.2v.
"logiclv 0.0" -> logic '0' is replaced by 0.0v.
```

Delay Statements in Vector Files

The PrimeSim tool supports the `delay` parameter in vector files. The reserved keywords are `delay` or `tdelay`. The usage is as follows:

```
delay|tdelay value
```

The value is scaled by the `tunit` value. The default unit is `ns` scale.

Example

```
tunit 0.1n  
delay 10
```

In the previous example, the actual delay is 1 ns because the time unit is `0.1n`. If there is no `tunit`, the delay is 10 ns.

Support for VCHK_IGNORE in Vector Files

The PrimeSim tool supports the `VCHK_IGNORE` command in the vector file. The usage is as follows:

```
VCHK_IGNORE t1 t2 [mask]
```

This causes the PrimeSim tool to ignore checking for all nodes specified when you use the optional `mask` between times specified by `t1` and `t2`.

If `mask` is not specified, the PrimeSim tool ignores all signals between `t1` and `t2`. To ignore selected signals over the entire time period, specify the `t1` start and `t2` ending times. This command can be repeated for cumulative effect.

Example

```
vchk_ignore 0 2 0101
```

This example ignores the output check for the signals impacted by the specified mask from 0 to 2 ns.

Using the Context-Based Control Option (CBC)

The `OPTION CBC` (Context-Based Control) specifies the direction of bidirectional signals. A bidirectional signal is an input if its value is 0, 1, or Z; conversely, a bidirectional signal is an output if its value is H, L, U, or X.

For example:

```
RADIX 1 1 1  
IO I O B  
VNAME A Z B  
OPTION CBC  
10.0 0 X L
```



```
20.0 1 1 H
30.0 1 0 Z
```

This example sets up three vectors, named **A**, **Z**, and **B**. Vector **A** is an input, vector **Z** is an output, and vector **B** is a bidirectional signal (defined in the **IO** statement).

The **OPTION CBC** line turns on context-based control. The next line sets vector **A** to a logic-low at 10.0 ns, and vector **Z** is “do not care.” Because the **L** value is under vector **B**, the PrimeSim tool expects a logic-low output.

At 20 ns, vector **A** transitions high, and the expected outputs at vectors **Z** and **B** are high. Finally, at 30 ns, the PrimeSim tool expects vector **Z** to be low, vector **B** changes from an output to a high-impedance input, and vector the **A** signal does not change.

Tabular Data

Tabular data has the following types:

- [Arbitrary Time Step](#)
- [Uniform Time Step](#)
- [Cascade Time Step](#)

Arbitrary Time Step

```
T1 s1 s2 s3 ...
T2 s1 s2 s3 ...
T3 s1 s2 s3 ...
```

Uniform Time Step

```
period Tstep ;
s1 s1 s3 ...
s1 s2 s3 ...
:
:
```

period

Defines the time interval of the tabular data. If a **period** statement is specified, the tabular data only contains signal values; there is no time.

Example

```
period 25 ;
1000 0001 ;
1110 0111 ;
1100 0011 ;
```

This example is the same as:

```
0 1000 0001 ;
25 1110 0111 ;
50 1100 0011 ;
```

tskip

Ignores the absolute times specified at the beginning of each row in the tabular data section and instead uses the time interval defined by the `period` statement.

Example

```
period 25 ;
tskip
0.0 1000 0001 ;
11.0 1110 0111 ;
15.0 1100 0011 ;
```

This example is the same as:

```
0 1000 0001 ;
25 1110 0111 ;
50 1100 0011 ;
```

Cascade Time Step

```
T1 s1 s2 s3
T2 s1 s2 s3
T3 s1 s2 s3
cascade;
T4 s1 s2 s3
T5 s1 s2 s3
T6 s1 s2 s3
```

The preceding is the same as (no `cascade`):

```
T1      s1 s2 s3
T2      s1 s2 s3
T3      s1 s2 s3
T3+T4   s1 s2 s3
T3+T5   s1 s2 s3
T3+T6   s1 s2 s3
```

cascade

Forces the tabular data to be cascaded with the previous data. The time steps after the `cascade` keyword are added to the last time step before the `cascade` keyword.

The states are defined in the following table.

Table 5 *State Definitions*

State	Meaning
0	Drive to ground
1	Drive to one (VIH)
Z	Floating high impedance
X, U	Don't Care, set to ground
L	Currently, same as state 0
H	Currently, same as state 1
M	Meta-stability state defined as the voltage range in between <code>vih</code> and <code>vil</code>

pwl_type

Defines the slope as a PWL rise or fall around `time=T`. Its values are defined as:

- `pwl_type 0` : The slope is the time interval between `T` and `T+slope`
- `pwl_type 1` : The slope is the time interval between `T-slope/2` and `T+slope/2`

The default value is `0`. This statement comes before the first tabular data line. The `pwl_type` is one of options coming after comment, currently, `'`, `*`, `$` is managed as comment.

```
* pwl_type 1
```

I/O Vector Support for '-' Values

In the period during which an output variable has '-' values, output comparison is disabled during simulation time. The hyphen (-) character in an I/O vector file implies a `don't care` region of out comparison checking. In the following example, the `out` signal is checked during simulation because it is output.

If the value of signal is different from the expected value in an I/O vector file, the PrimeSim tool reports that information. But the output comparison is disabled for the period of '-', that is, 20ns ~ 30ns, as shown in the following example:

```
radix 11
io io
nodename clk out
0.00 10
10.00 00
20.00 1-
```

```
30.00 0-  
40.00 11  
50.00 01
```

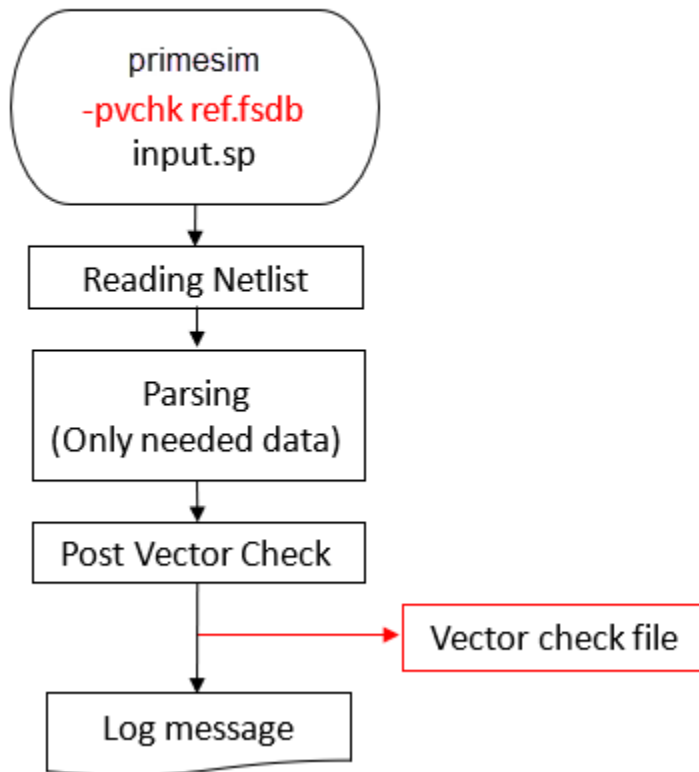
PrimeSim Post Vector Check

Description

Use the PrimeSim tool Post Vector Check feature (`-pvchk`) to perform an output vector comparison with an FSDB file using a modified vector file without the need for rerunning the simulation.

The feature generates a vector check file `[output_prefix].err0` after the check.

PrimeSim Post Vector Check Procedure



Syntax

```
$ primesim -pvchk [FSDB_file] input.sp
```

Examples

```
$ primesim -pvchk ref.fsdb input.sp
```

PrimeSim Circuit Examples

```
**** Test I/O vector ****
.option post probe
.option primesim_vector=vec.in
*** vec.in2 is a case for cascade io vector
*.option primesim_vector=vec.in2
*** vec.in3 is the same as vec.in2 without cascade io vector
*.option primesim_vector=vec.in3
.model nch nmos level=49
.model pch pmos level=49
.subckt inv out in d
mp1 out in d d pch w=20u l=0.6u
mn1 out in 0 0 nch w=10u l=0.6u
.ends inv
vcc vcc 0 5.0
vin vin 0 pulse(0 4.3 2n 3n 3n 10n 20n)
x0 d0 in0 vcc inv
x1 d1 in1 vcc inv
x2 d2 vin vcc inv
x3 d3 in3 vcc inv
x4 d4 in4 vcc inv
.tran 0.1n 100n
.probe v(*)
.end
```

This circuit has five inverters and has its input signal given by three I/O vector files. The first case, `vec.in`, is the input signal and output check declaration via I/O vector in print-on-change format. The second case, `vec.in2`, is the input signal using cascaded tabular data format, which is the same as the third case, `vec.in3`.

The following examples show each case in the vector format.

- [Case 1 \(vec.in\)](#)
- [Case 2 \(vec.in2\)](#)
- [Case 3 \(vec.in3\)](#)

Case 1 (vec.in)

```
; input node:in1-4 output node:d0-d4 check node:d2, d3
radix 1111 1 1 1 1 1 1 1
io bbbb b i i i i o o
nodename d[0~3] d4 in0 in1 in3 in4 d2 d3
tunit 1n;
slope 0.5;
vih 2.5;
vil 0.8;
voh 3.8;
```

```

vol 2.0;
;tabular data
;time d0 d1 d2 d3 d4 in0 in1 in3 in4 d2 d3
; print on change
5.0 z z z z z 1 0 0 1 x x
10.0 z z z z z 1 0 1 0 x x
13.0 z z z z z 0 1 0 1 x 1
15.0 1 1 0 0 z 0 1 1 1 0 x
28.0 z z z z z 1 0 0 1 1 1
38.0 z z z z 1 1 1 1 1 0 0
45.0 z z 1 z 1 0 0 0 0 1 1
78.0 z z 1 z 1 0 1 0 0 0 x
98.0 z z z 1 0 0 0 1 1 x x

```

Case 2 (vec.in2)

```

; input node:in1-4
radix 1111
io iiii
nodename in0 in1 in3 in4
tunit 1n;
slope 0.5;
vih 2.5;
vil 0.8;
vh 3.8;
vl 2.0;
;tabular data
;time in0 in1 in3 in4
0.0 0 0 0 0
5.0 1 0 0 1
10.0 1 0 1 0
cascade
1.0 0 1 0 1
3.0 0 1 1 1
8.0 1 0 0 1
cascade
0.0 1 1 1 1
5.0 0 0 0 0
11.0 0 1 0 0
cascade
3.0 0 0 1 1
5.0 0 0 0 0

```

Case 3 (vec.in3)

```

; input node:in1-4
radix 1111
io iiii
nodename in0 in1 in3 in4
tunit 1n;

```

```
slope 0.5;
vih 2.5;
vil 0.8;
vh 3.8;
vl 2.0;
;tabular data
;time in0 in1 in3 in4
; same time point as vec.in2
0.0 0 0 0 0
5.0 1 0 0 1
10.0 1 0 1 0
11.0 0 1 0 1
13.0 0 1 1 1
18.0 1 1 1 1
23.0 0 0 0 0
29.0 0 1 0 0
32.0 0 0 1 1
34.0 0 0 0 0
```


Part 3: Elements and Models

Describes the elements and models used by the PrimeSim tool.

This Part presents the following chapters:

- [Chapter 6, Circuit Elements and Models](#)
- [Chapter 7, C-Modeling](#)
- [Chapter 8, Verilog-A Support](#)

For more information, see the following PrimeSim Continuum manuals:

- *PrimeSim™ Continuum User Guide: Elements*
- *PrimeSim™ Continuum Reference Manual: Device Models*
- *PrimeSim™ Continuum Reference Manual: MOSFET Models*

6

Circuit Elements and Models

This chapter describes the circuit elements and models that the PrimeSim tool supports.

Elements are categorized into four general classes:

- Passive elements
- Independent sources
- Dependent sources
- Active elements

An element statement defines the type of the device, its terminal nodes, the parameter values that describe the operating electrical characteristics of the device, and references to model statements that define the electrical parameters of the element itself. A netlist is simply the element instances for all the devices in a circuit and their connections through nodes.

This chapter covers the following topics:

- [Supported Models](#)
- [Rules for Instance, Model, and Global Parameter Interaction](#)
- [Resistors, Capacitors, Inductors, and Diodes](#)
- [Bipolar Junction Transistors \(BJTs\), JFETs, MESFETs, and MOSFETs](#)
- [Independent Sources and Functions](#)
- [Dependent Sources and Functions](#)
- [Transmission Lines](#)
- [Port Elements and IBIS Buffers](#)
- [Ferroelectric Capacitor \(FeCap\) Model](#)

Supported Models

The PrimeSim Pro and PrimeSim SPICE tools support many industry-standard models, including passive element models, diode models, BJT models, and MOSFET models.

Passive element models:

- Reluctor models
- RLC models
- CMC R3 model
- Diffusion resistor
- Physical resistor
- MOSVAR 1.3

Diode models:

- HSPICE level 1 (Berkeley diode model)
- HSPICE level 2 (modified Berkeley diode model)
- HSPICE level 3 (Fowler-Nordheim)
- HSPICE level 4 (Juncap)
- HSPICE level 5 (Philips diode level 500)
- HSPICE level 6 (Juncap2, up to 200.6)
- HSPICE level 7 (DIODE_CMC, up to 2.0)

BJT models:

- HSPICE level 1 - Gummel-Poon Models
- HSPICE level 4 - VBIC95 1.2 model
- HSPICE level 6 - Mextram 503
- HSPICE level 6 - Mextram 504, up to 504.13
- HSPICE level 6 - Mextram 505, up to 505.1
- HSPICE level 8 - HiCUM L2 up to 3.0.0
- HSPICE level 9 - VBIC99 model

- HSPICE level 10 - Philips MODELLA model
- HSPICE level 13 - HiCUM L0 up to 2.1.0

MOSFET models:

- HSPICE level 1 - MOS1 (level=1) model of UC Berkeley SPICE
- HSPICE level 2 - MOS1 (level=2) model of UC Berkeley SPICE
- HSPICE level 3 - MOS3 (level=3) models of UC Berkeley SPICE
- HSPICE level 49 - BSIM 3.2 MOS model of UC Berkeley SPICE
- HSPICE level 50 - Philips MM9 (Level 903) model
- HSPICE level 53 - BSIM 3.3.0 model of UC Berkeley SPICE
- HSPICE level 54 - BSIM4 MOS model of UC Berkeley SPICE, up to 4.8
- HSPICE level 55 - EKV v2.6 model
- HSPICE level 57 - BSIM3SOI models of UC Berkeley SPICE, up to 3.2
- HSPICE level 61- RPI a-Si TFT model, up to 1.0
- HSPICE level 62- RPI poly-Si TFT model, up to 2.0
- HSPICE level 63 - Philips MM11 model
- HSPICE level 66 - Synopsys Proprietary High Voltage CMOS Model, up to 2.1
- HSPICE level 68 - STARC HiSIM2 model, up to 3.1.1
- HSPICE level 69 - PSP 102 model, up to 102.5 (w/o NQS)
- HSPICE level 69 - PSP 103 model, up to 103.8.2
- HSPICE level 70 - BSIMSOI4 model of UC Berkeley SPICE, up to 4.61, and 100.1
- HSPICE level 72 - BSIM-CMG model of UC Berkeley SPICE, up to 111.10
- HSPICE level 73 - STARC HiSIM-HV/LD MOS model, up to 2.5
- HSPICE level 76 - UTSOI 1.13, 1.14, 2.0, 2.1, 2.1.1, 2.4, and up to 102.7.
- HSPICE level 77 - BSIM-BULK (formerly BSIM6) model, up to 107.0.0
- HSPICE level 78 - BSIM-IMG model of UC Berkeley SPICE, up to 102.9.4
- HSPICE level 79 - EKV3 model
- HSPICE level 80 - HiSIM-SOI, up to 1.4

- HSPICE level 81 - ASM-HEMT, up to 101.4
- HSPICE level 82 - MVSG-HEMT, up to 4.0
- HSPICE level 83 - HiSIM-HMG, up to 2.0
- HSPICE level 85 - ASM-ESD, up to 101.0

Supported Model Features

TSMC Model Interface (TMI) Support

The PrimeSim Pro tool supports TSMC TMI models up to 2.0.1. To use TMI models, add the library path to your input netlist.

The default `.OPTION MACMOD` is now set to 3 for TMI models. If the `.OPTION TMIFLAG` option is set, `MACMOD` is automatically set to 3. Otherwise, the default for `MACMOD` is 0.

Si2 Compact Model Coalition - Open Model Interface (OMI) Support

The PrimeSim tool supports Si2 Compact Model Coalition OMI models. OMI model usage is defined in the *Si2 Compact Model Coalition OMI Functional Specification* and the foundry's or IDM's OMI model usage guide.

STI Effects for BSIM3/4 Models

The PrimeSim tool supports STI stress effect for BSIM3/4 models.

Aging Model NBTI Support

The PrimeSim tool supports NBTI aging model. Contact your local Synopsys Applications Consultant for additional information.

MOS Varactor Support

The PrimeSim tool supports MOS Varactor, which is a capacitor model.

Syntax

```
C1 g b mname l=1u w=1u ...  
.model mname c level=7 ...
```

Binning Model Support for .mparam

Syntax

```
.mparam mname=model_name [subckt=ckt_name] param_name=value
```

If no subcircuit is specified, the PrimeSim tool searches models with `mname` in the top level. Otherwise, it searches for the local model in the subcircuit.

IJTH Support

The PrimeSim tool supports the model parameter `IJTH` for adjustable current limiting in the junction diode current model. This is implemented for BSIM3.

Flash Cell Model

The flash cell model is an extension of a base transistor model, such as BSIM3, BSIM4, and MOS1. The base transistor model card is defined using the `.model` statement. The flash extension consists of an additional set of model parameters, which can be defined with the `.model` statement and appended to the base model card using the `.appendmodel` statement.

Example

```
.model bsim4 nmos level=54 version=4.8
.model flash flashcell flashlevel=2
.appendmodel flash bsim4
m1 d g s b bsim4 L=0.1u W=0.2u
```

The flash cell model is supported at the following levels:

- `flashlevel=1` is a simple model that targets NOR flash technology. The programming and erasing conditions are fixed in terms of maximum or minimum voltage on the device terminals. All Vth programming and erasing behavior is controlled by built-in equations.
- `flashlevel=2` is based on the same concept as `flashlevel=1` but offers more flexibility, which makes it suitable to model NAND technology, for example.
- `flashlevel=3` is used for simple models that target split-gate flash technology. All cell programming and erasing behavior is controlled by built-in equations.

The flash cell model is supported only for the following MOSFET models: MOS1, MOS3, BSIM3, and BSIM4.

Rules for Instance, Model, and Global Parameter Interaction

Before you learn about each element's definition and use, review the three rules that the PrimeSim tool follows to determine the right value in case instance parameters are in conflict with model or global parameters.

- [Rule 1](#)
- [Rule 2](#)
- [Rule 3](#)

Rule 1

The instance length L and width W are scaled by the global scale statement,

```
.OPTION SCALE=val
```

However, the instance values (for example, resistance and capacitance) are scaled by the value in the instance statement. In the following discussion, $SCALE(instance)$ represents instance scaling, and $SCALE(option)$ represents the global dimensional scale declaration.

Example

```
.OPTION SCALE=1u
R1 rn1 rn2 R=5 SCALE=1k L=0.5 W=20
```

In this example, L and W are scaled in μm by $SCALE(option)$. That is, $L=0.5\mu m$ and $W=20\mu m$, respectively. The resistance value of $R1$ is scaled to $1.0e3$ by $SCALE(instance)$. That is, $R=5k\ ohm$.

Rule 2

If the model name of an instance is the same as that of a parameter, the model name takes precedence.

Example

```
.PARAM name=1.5
R1 in out1 name TC1=0.1
.MODEL name R RSH=1k
```

In this example, $name$ is the model name of $R1$, not the parameter 1.5 .

Rule 3

Some parameters can come from instance and `.MODEL` statements. If a parameter is in both the instance and the `.MODEL` statements, the value in the instance statement is used.

Example

```
R1 in out1 Rmodel 4.7k TC1=0.08 TC2=0.04  
.MODEL Rmodel R DW=0.3u TNOM=27 TC1=0.4 TC2=0.18
```

In this example, the temperature coefficient of R1 is TC1=0.08, TC2=0.04, and not TC1=0.4, TC2=0.18 as defined in the `.MODEL` statement.

Note:

Reserved node names for ground: 0, GND, GND!, GROUND. The PrimeSim tool treats those nodes as the absolute ground node. But if they are used in a subcircuit's port name, it is treated as a normal node inside of that subcircuit.

Resistors, Capacitors, Inductors, and Diodes

This section describes the following:

- [Resistor Elements \(R-element\)](#)
- [Capacitor Elements \(C-element\)](#)
- [Linear Inductors \(L-element\)](#)
- [Mutual Inductors \(K-element\)](#)
- [Reluctors \(L-element\)](#)
- [Diode Elements \(D-element\)](#)

Resistor Elements (R-element)

This section details resistors in the PrimeSim tool.

Syntax

```
Rxxx n1 n2 <mname> Rval <TC1 <TC2 <SCALE>>> <M=val>  
+ <AC=val> <DTEMP=val> <L=val> <W=val> <C=val>
```

or:

```
Rxxx n1 n2 <mname> R=val <TC1=>val> <TC2=>val> <SCALE=val>  
+ <M=val> <AC=val> <DTEMP=val> <L=val> <W=val> <C=val>
```


or:

```
Rxxx n1 n2 <mname> R='algebraic expression' <TC1 <TC2>>  
+ <SCALE=val> <M=val> <AC=val> <DTEMP=val> <L=val>  
+ <W=val> <C=val>
```

Arguments

Argument	Description
AC	Resistance of AC analysis. If not specified, the same value is used for AC analysis.
C	Capacitance connected from node n1 to bulk and node n2 to bulk. Default is 0.0, if C is not specified in a resistor model.
DTEMP	Temperature difference between the element and the circuit in Celsius. The default value is 0.0. DTEMP is available with TC1 and TC2. Equation: $R = R0 * (1 + TC1*\Delta t + TC2*\Delta t^2)$ $\Delta t = \text{temperature} - \text{TNOM} + \text{DTEMP}$ where R0 is the user-given resistance value.
L	Resistor length. The value is 0.0 if L is not set in the model mname. Scaled length is $L_{scaled} = L * \text{SCALE}(\text{option})$.
M	Multiplier used to simulate parallel connection of the resistors. The default value is 1.0.
mname	Model name for the resistor.
N1, N2	Two instance nodes.
Rxxx	Resistor instance name.
SCALE	Instance scale factor for resistor. The default value is 1.0. Rval should be set before SCALE(instance) is used.
TC1	First-order temperature coefficient for the resistor.
TC2	Second-order temperature coefficient for the resistor.
VALUE	The resistance, either a value (in ohms) or an equation. ($R=\text{val}$ or $R=\text{'expression'}$) $\text{Reff} = R * \text{SCALE}(\text{instance}) / M$
W	Resistor width. The default value is 1e-6 meters if W is not set in the model mname. Scaled width is: $W_{scaled} = W * \text{SCALE}(\text{option})$.

Description

This section covers the semiconductor resistor model and frequency-dependent resistors.

Semiconductor Resistor Model

In the following example:

```
.MODEL mname R(RES) par1=val1 par2=val2 ...
```

where `mname` is the model name, `R(or RES)` is the keyword for the resistor model, and `par1, par2, ...` list the values of parameters.

Frequency-Dependent Resistors

The syntax to define frequency-dependent resistors is:

```
Rxxx n1 n2 [R=] 'equation' [CONVOLUTION=[0|1|2]] [FBASE=value]  
+ [FMAX=value]
```

Argument	Description
CONVOLUTION=[0 1 2]	Indicates which method to use (at the instance level only). 0 acts the same as the conventional method. This is the default. 1 applies recursive convolution, and if the rational function is not accurate enough, it switches to linear convolution. 2 applies linear convolution.
FBASE=value	Specifies the lower bound of the transient analysis frequency: - CONVOLUTION=1 (default=0 Hz); FBASE is the starting sampling frequency. - CONVOLUTION=2 (default=reciprocal of transient period); FBASE is the base frequency point for Inverse Fourier Transformation.
FMAX=value	Specifies the possible maximum frequency of interest. The default value is the frequency point where the function reaches close enough to infinity value. This assumes that frequency point reaches 10 THz and the monotonous function is approaching the infinity value. The equation should be a function of HERTZ. If CONVOLUTION turns on when a HERTZ keyword is not in the equation, it is automatically turns off to let the resistor behave conventionally. The equation can be a function of temperature, but it cannot be node voltage or branch current and time.

The equation can only be a function of time-independent variables such as HERTZ, and temperature. You can model a frequency-dependent resistor and use it in the time domain only when `CONVOLUTION=1`. For example:

```
R1 1 2 r='1.0 + 1e-5*sqrt(HERTZ)' CONVOLUTION=1
```

Examples

```
* Example of Resistor model *
.option post
R123 in out1 1k
c1 out1 0 1p
Rw in out2 R=10 SCALE=1e3 M=3
c2 out2 0 1p
Rtest in out3 R='2.345*1.4' SCALE=1e6
c3 out3 0 1p
Rk1 in out4 R_model L=10u W=200u
c4 out4 0 1p
RCC in out5 R=100k
c5 out5 0 1p
Rk2 in out6 R_model L=10u W=200u DTEMP=90
c6 out6 0 1p
.model R_model R RSH=1k DW=0.3u TNOM=27 TC1=0.4 TC2=0.18
vin in 0 pw1( 0 0 5n 0 5.5n 2.5 )
.tran 0.1n 200n
.end
```

Capacitor Elements (C-element)

This section details capacitors in the PrimeSim tool.

Syntax

```
Cxxx n+ n- <mname> VALUE <TC1 <TC2 <SCALE>>> <L=val>
+ <W=val> <M=val> <IC=val> <DTEMP=val>
```

or:

```
Cxxx n+ n- <mname> C=val <TC1=>val> <TC2=>val> <SCALE=val>
+ <L=val> <W=val> <M=val> <IC=val> <DTEMP=val>
```

or:

```
Cxxx n+ n- <mname> C='algebraic expression' <<TC1=>val>
+ <<TC2=>val> <SCALE=val> <L=val> <W=val> <M=val>
+ <IC=val> <DTEMP=val>
```

Arguments

Argument	Description
Cxxx	Capacitor instance name.

Argument	Description
DTEMP	Temperature difference between the element and the circuit in Celsius. The default value is 0.0. DTEMP is available with TC1 and TC2. Equation: $C = C0 * (1 + TC1*\Delta t + TC2*\Delta t^2)$ $\Delta t = \text{temperature} - \text{TNOM} + \text{DTEMP}$ where C0 is the user-given capacitance value.
IC=val	Sets the initial voltage across the capacitor in volts. If the input netlist contains an .IC statement, it overrides the initial assignment in the instance statement.
L	Capacitor length. The value is 0.0 if L is not set in the model mname. Scaled length is $L_{\text{scaled}} = L * \text{SCALE}(\text{option})$.
M	Multiplier used to simulate parallel connection of the capacitors. The default value is 1.0.
mname	Model name for the capacitor.
N-	Negative instance node.
N+	Positive instance node.
SCALE	Instance scale factor for the capacitor. The default value is 1.0. VALUE should be set before SCALE(instance) is used. $C_{\text{eff}} = C * \text{SCALE}(\text{instance}) * M$
TC1	First-order temperature coefficient for the resistor.
TC2	Second-order temperature coefficient for the resistor.
VALUE	Capacitance in Farads, or as (C=val or C='expression'). Specify infinite capacitance as: C=infinity or C=inf

Description

This section covers:

- Capacitor model
- Mosvar model
- Charged-based capacitors

Capacitance Model

In the following example:

```
.MODEL mname C par1=val1 par2=val2 ...
```

where `mname` is the model name, `C` is the keyword for capacitor model, and `par1`, `par2`, ... list the values of the parameters.

Three Terminal Mosvar Model in 3rd-party and SPICE Format

The PrimeSim tool supports a three terminal mosvar model, and this element is recognized as a MOSVAR element in the log file.

Example

```
# MOSVAR : 1
```

Charge-Based Capacitors

The PrimeSim tool supports charge-based capacitors, where the charge can be given by an expression which may depend on node voltages.

Syntax

```
Cxxx n1 n2 q=expr
```

`expr` supports nodal voltages parameters.

Example

```
C1 1 2 Q='cosh(0.5*V(1,2))'
```

Examples

```
* Example of Capacitor model *
.option post accurate
C11 out1 0 1pF
r1 in out1 1k
Ceq out2 0 c='22p + 11*4e-12'
r2 in out2 1k
Cval out3 0 cmodel L=10u W=20u M=4
r3 in out3 1k
Ctt out4 0 cmodel IC=1.5V
r4 in out4 1k
.model cmodel C CAP=1p TC1=0.01 TC2=0.005
vin in 0 pw1( 0 0 5n 0 5.5n 2.5 )
.tran 0.1n 200n
.end
```

Capacitor device in SPICE syntax

The capacitor component can also have the frequency-dependent Q-factor. You should set two optional parameters, `q` and `fq` to activate this feature. One resistor (`res`) that is parallel with one capacitor `Ceff` is used to replace the original capacitor (`C`).

$$C_{\text{eff}} = C$$

$$\text{res} = \frac{q}{2\pi f q c}$$

where

- q denotes the Q factor measured at the frequency f_q (default $q=50$).
- f_q denotes the frequency (in Hz) which specifies the frequency-dependent capacitance and Q factor (default $f_q=1.0e8$).

Example:

```
C1 n1 n2 c=2.5e-6 fq=1.3e9 q=120
```

Linear Inductors (L-element)

Description

This section details inductors in the PrimeSim tool.

Note:

For information on the ideal transformer format and the `TRANSFORMER_NT` argument, see “Ideal Transformer Format” topic in the PrimeSim Continuum: Elements User guide.

Syntax

```
Lxxx n+ n- VALUE <TC1 <TC2>> <SCALE=val> <M=val> <IC=val>  
+ <DTEMP=val>
```

or:

```
Lxxx n+ n- L=val <<TC1=>val> <<TC2=>val> <SCALE=val> <M=val>  
+ <IC=val> <DTEMP=val>
```

or:

```
Lxxx n+ n- L='algebraic expression' <<TC1=>val> <<TC2=>val>  
+ <SCALE=val> <M=val> <IC=val> <DTEMP=val>
```

Arguments

Argument	Description
DTEMP	Temperature difference between the element and the circuit in Celsius. The default value is 0.0. DTEMP is available with TC1 and TC2. Equation: $L = L0 * (1 + TC1*\Delta t + TC2*\Delta t^2)$ $\Delta t = \text{temperature} - T_{NOM} + DTEMP$ where $L0$ is the user-given inductance value.

Argument	Description
IC	Sets the initial current through the inductor in amperes.
Lxxx	Linear inductor instance name.
M	Multiplier used to simulate parallel connection of the inductors. The default value is 1.0.
mname	Model name for the inductor.
N-	Negative instance node.
N+	Positive instance node.
SCALE	Instance scale factor for the inductor. The default is 1.0. VALUE should be set before <code>SCALE(instance)</code> is used. $L_{eff} = 1 \times SCALE(instance) \times M$
TC1	First-order temperature coefficient for the resistor.
TC2	Second-order temperature coefficient for the resistor.
VALUE	Inductance in Henries (H), or as (<code>L=val</code> or <code>L='expression'</code>). Specify infinite inductance as: <code>L=infinity</code> or <code>L=inf</code>

Examples

```
* example of Inductor *
.option post accurate
Lcouple l1 lout1 L=10uH M=3
r1 in l1 100
c1 lout1 0 0.5p
Ls l2 lout2 45 SCALE=1e-5
r2 in l2 100
c2 lout2 0 0.5p
LIC l3 lout3 70uH IC=0.5mA
r3 in l3 100
c3 lout3 0 0.5p
vin in 0 pw1( 0 0 5n 0 5.5n 2.5 )
.tran 0.1n 200n
.end
```

Inductor device in SPICE syntax

The inductor component can also have the frequency-dependent Q-factor. You should set three optional parameters, `q`, `fq`, and `rdc`, to activate this feature. One resistor (`res`) that is in series with one inductor L_{eff} is used to replace the original inductor (L).

$$L_{eff}=L$$

$$res = 2 \pi f_q \frac{L}{Q} + rdc$$

where

- q : denotes the Q factor measured at the frequency f_q (default $q=0$).
- f_q : denotes the frequency (in Hz) which specifies the frequency-dependent inductance and Q factor (default $f_q=0$).
- rdc : denotes the DC resistance (default $rdc=0$).

Example:

```
L1 n1 n2 l=3.8e-10 q=16 fq=1.086e9 rdc=0.05
```

Mutual Inductors (K-element)

Description

This section details mutual inductors in the PrimeSim tool.

Syntax

```
Kxxx Lyyy Lzzz VALUE
```

Arguments

Argument	Description
Kxxx	Mutual inductor instance name.
Lyyy, Lzzz	Names of the two coupled inductors.
VALUE	The coupling coefficient, which must be greater than 0 and less than 1, or it can be expressed as $K=val$ or $K='expression'$.

Examples

```
** K element example **
.option accurate post rmax=.05
Vin in 0 sin (0 5 60)
Rin in rin 10
Lin rin 0 L=20u
Km Lin Lm K=0.8
Rs 0 rs 10
Lm rm rs L=50u
Rm rm rmc 5
Cm rmc 0 50p
.tran 1m 25m
.end
```


The PrimeSim tool allows the two inductors that are used to define a mutual inductor (K element) to be hierarchical. Consider the following example:

```
Kmutual X1.LA X1.LB 0.5
```

Where LA and LB are defined under the hierarchical structure of X1 as follows:

```
.SUBCKT subinductor
LA 1 0 1m
LB 1 0 0.5m
.ENDS
X1 subinductor
```

Reluctors (L-element)

Description

This section details reluctor inductors in the PrimeSim tool.

Syntax

```
Lxxx n1+ n1- n2+ n2- ... ni+ ni- ... nj+ nj- ... nN+ nN-
+ RELUCTANCE=(1, 1, rluc11, 1, 2, rluc12, ..., i, i, rlucii,
+ i, j, rlucij, ..., N, N, rlucNN)
```

Arguments

Argument	Description
Lxxx	N-port mutual reluctor instance name.
ni+	Positive instance node name of port i.
ni-	Negative instance node name of port i.
RELUCTANCE	Described in triplets (index of port<i>i</i>, index of port<i>j</i>, reluctance value in inverse Henries (1/H)). When <i>i</i> equals <i>j</i>, the third item of the triplet is self reluctance, otherwise it is mutual reluctance. Mutual reluctance is symmetric: the reluctance between port <i>i</i> and port <i>j</i> equals the reluctance between port <i>j</i> and port <i>i</i>, so you only need to specify one of the triplets, (<i>i</i>, <i>j</i>, <i>rluc</i>) or (<i>j</i>, <i>i</i>, <i>rluc</i>) . Also, you only need to include non-zero reluctance in each triplet.

Examples

```
.option post accurate
L1 1 2 0 3 6 7 RELUCTANCE = (1, 1, 1.6e9, 1, 2, -8.5e8, 1, 3, -2.1e8, 2,
2, 1.8e9, 2, 3, -5.5e8, 3, 3, 1.4e9)
R1 2 4 1.0
R2 3 5 1.0
R3 7 8 1.0
```

```
C1 6 0 0.1p
C2 4 5 0.1p
C3 5 8 0.1p
vin 1 0 pw1(0 0 0.1n 0 0.2n 2.5 1n 2.5)
.tran 1p 1n
.end
```

Diode Elements (D-element)

This section details diode elements in the PrimeSim tool.

Syntax

```
Dxxx N+ N- MNAME <OFF> <DTEMP=val> <M=val> <IC=vd>
+ <AREA=val> <PJ=val> <L=val> <W=val> +<WP=val>
+ <LP=val> <WM=val> <LM=val>
```

Arguments

Argument	Description
AREA	Area multiplier factor. The default is 1.0.
DTEMP	Temperature difference between the element and the circuit in Celsius. The default value is 0.0.
IC=vd	Initial voltage across the diode. Intended for use with the UIC option on the .TRAN statement when starting transient analysis from a point other than the quiescent operating point.
L	Length of diode in meters (for LEVEL=3 only).
LM	Length of metal capacitor in meters (for LEVEL=3 only). Overrides WM in diode model. The default is 0.0.
LP	Length of polysilicon capacitor in meters (for LEVEL=3 only). Overrides LP in diode model. The default is 0.0.
M	Multiplier for Diode.
mname	Model name for the diode.
N+, N-	Positive and negative nodes, respectively.
OFF	Initial condition OFF for this instance in DC analysis. Default is ON.
PJ	Periphery of junction.
W	Width of diode in meters (for LEVEL=3 only).
WM	Width of metal capacitor in meters (for LEVEL=3 only). Overrides WM in diode model. The default is 0.0.

Argument	Description
WP	Width of polysilicon capacitor in meters (for LEVEL=3 only). Overrides WP in diode model. The default is 0.0.

Description

This section covers geometry calculation and the diode model.

Geometry Calculation Note

Diode model LEVEL=1 is a *non-geometric* and *scale-less* junction diode model. This means all settings of L, W, SCALE(option), and SCALE(instance), in both the diode instance and the model, are ignored.

Parameters (AREA, PJ, L, W) are common in instance and model parameters. If they appear in both instance and model lines, instance parameters take precedence.

For LEVEL=1:

$$\begin{aligned} \text{AREA}_{\text{eff}} &= \text{AREA} * M \\ \text{PJ}_{\text{eff}} &= \text{PJ} * M \end{aligned}$$

For LEVEL=3:

Case 1: If AREA or PJ equations are given, and L and W are not given:

$$\begin{aligned} \text{AREA}_{\text{eff}} &= \text{AREA} * M * \text{SCALE}(\text{option}) * \text{SCALE}(\text{option}) \\ \text{PJ}_{\text{eff}} &= \text{PJ} * M * \text{SCALE}(\text{option}) \end{aligned}$$

Case 2: If L and W are given, AREA or PJ are ignored whether given or not, and:

$$\begin{aligned} L_{\text{eff}} &= L * \text{SCALE}(\text{option}) + XW \\ W_{\text{eff}} &= W * \text{SCALE}(\text{option}) + XW \end{aligned}$$

and

$$\begin{aligned} \text{PJ}_{\text{eff}} &= (2 * W_{\text{eff}} + 2 * L_{\text{eff}}) * M \\ \text{AREA}_{\text{eff}} &= W_{\text{eff}} * L_{\text{eff}} * M \end{aligned}$$

Diode Model

In the following example:

```
.MODEL MNAME D <par1=val1 par2=val2 ...>
```

where MNAME is the model name, and D is the keyword to indicate the diode model. par1, par2, ... list the value of the parameters. For specific details about which models the PrimeSim tool supports, see [Supported Models](#).

Examples

```
Diode pnode nnode diodemodel M=2
Dclamp in out dmodel area=3 IC=0.4
```

Bipolar Junction Transistors (BJTs), JFETs, MESFETs, and MOSFETs

This section describes the following:

- [BJT Elements \(Q-element\)](#)
- [JFET and MESFET Elements \(J-element\)](#)
- [MOSFET Elements \(M-element\)](#)

BJT Elements (Q-element)

This section details BJT elements in the PrimeSim tool.

Syntax

```
Qxxx NC NB NE <NS> mname <area> <OFF> <DTEMP=val> <M=val>
+ <IC=VBE, VCE>
```

or:

```
Qxxx NC NB NE <NS> mname <OFF> <DTEMP=val>
+ <M=val> <AREA=val> <AREAB=val> <AREAC=val> <VBE=val>
+ <VCE=val>
```

Arguments

Argument	Description
AREA	Area multiplier factor. The default is 1.0.
AREAB	Base Area multiplier factor. The default is AREA.
AREAC	Collector Area multiplier factor. The default is AREA.
DTEMP	Temperature difference between the element and the circuit in Celsius. The default value is 0.0.
IC=VBE, VCE	Initial internal voltages, base-emitter and collector-emitter voltage, respectively. Intended for use with the UIC option in the .TRAN statement when starting a transient analysis from a point other than the quiescent operating point.
M	Multiplier for BJT.

Argument	Description
<code>mname</code>	Model name for the BJT.
<code>NC, NB, NE</code>	The collector, base and emitter nodes, respectively.
<code>NS</code>	Optional substrate node. The default is ground node.
<code>OFF</code>	Initial condition <code>OFF</code> for this instance in DC analysis. Default is <code>ON</code> .
<code>VBE=val</code>	Initial voltage drop between base and emitter.
<code>VCE=val</code>	Initial voltage drop between collector and emitter

Description

This section covers geometry calculation and the BJT model.

Geometry Calculation Note

Some parameters used in a BJT instance or model depend on the geometry of the collector, base and emitter. Also, these calculations depend on the type of BJT—vertical or lateral.

BJT Model

In the following example:

```
.MODEL MNAME NPN <par1=val1 par2=val2 ...>
```

or:

```
.MODEL MNAME PNP <par1=val1 par2=val2 ...>
```

where `MNAME` is the model name, and `NPN` and `PNP` are the keywords to indicate `NPN` and `PNP` type of BJT models, respectively. `par1`, `par2`, ... list the value of parameters.

The PrimeSim tool supports the Gummel-Poon (GP) model (HSPICE level 1), the VBIC95 1.2 model (HSPICE level 4), and the Philips Mextram 503 model, which is equivalent to an HSPICE level 6 model.

Examples

```
Q1 C B E Qmodel M=3 IC=0.6, 3.3
Qw 1 23 4 BJTm AREA=3
```

JFET and MESFET Elements (J-element)

This section details JFET and MESFET elements in the PrimeSim tool.

Syntax

```
Jxxx ND NG NS <NB> mname <AREA=area | W=val L=val>  
+ <OFF> <M=val> <DTEMP=val>
```

Arguments

Argument	Description
AREA	The area of JFET/MESFET.
DTEMP	The difference between the element temperature and the circuit temperature, in degrees Celsius. Default is 0.0.
L	Gate length.
M	Multiplier for the JFETs/MESFET
mname	model name of JFET or MESFET instance
ND, NG, NS, NB	The drain, gate, source, and bulk (substrate) nodes, respectively.
OFF	Initial condition for this instance in DC analysis
W	Gate width.

Description

This section covers the JFET/MESFET Model

JFET/MESFET Model

In the following example:

```
.MODEL MNAME PJF LEVEL=val <par1=val par2=val ...>
```

or:

```
.MODEL MNAME NJF LEVEL=val <par1=val par2=val ...>
```

where `MNAME` is the model name, and the `PJF` and `NJF` are keywords that indicate the P-channel and N-channel type of JFET/MESFET models, respectively. `par1`, `par2`, ... list the value of the parameters.

The PrimeSim tool supports HSPICE level 1 and 2.

Examples

```
J1 d g s b njfet L=10u W=5u  
J2 2 4 0 0 njfet area=5p M=3 DTEMP=50
```

MOSFET Elements (M-element)

This section details MOSFET elements in the PrimeSim tool.

Syntax

```
Mxxxx ND NG NS NB mname <L=>val <W=>val
+ <AS=val> <AD=val> <PS=val> <PD=val> <NRD=val> <NRS=val>
+ <OFF> <IC=VDS,VGS,VBS> <DELVTO=val> <DTEMP=val>
+ <GEO=val> <M=val>
```

Arguments

Argument	Description
AS, AD	The area of the drain and source diffusions. The units are m^2 . The defaults are DEFAS and DEFAD as defined in the .OPTION statement for AS and AD, respectively.
DELVTO	Zero-bias threshold voltage shift. The default is 0.0.
DTEMP	Temperature difference between the element and the circuit in Celsius. The default value is 0.0.
GEO	Source/drain sharing selector for ACM=3. The default is 0.0.
IC=VDS, VGS, VBS	Initial voltages across the external drain-source, gate-source, and bulk-source, respectively. Intended for use with the UIC option in the .TRAN statement when starting a transient analysis from a point other than the quiescent operating point.
L=val, W=val	Channel length and width of MOSFET, respectively. If L= and W= are omitted, these two values assign length first and then width in defaults. Also, L and W are scaled by the SCALE(option). The defaults are DEFL and DEFW in the .OPTION statement for length and width, respectively.
M	Multiplier for the MOSFET.
mname	Model name of MOSFET instance.
ND, NG, NS, NB	The drain, gate, source, and bulk (substrate) nodes, respectively.
NRD, NRS	The equivalent number of squares of the drain and source diffusion for resistance calculation. The default is DEFNRD, DEFDRS as defined in the .OPTION statement for NRD and NRS, respectively.
OFF	Initial condition OFF for this instance in DC analysis. Default is ON.
PD, PS	The perimeters of the drain and source diffusions. Units are m.

Description

This section covers the MOSFET model.

MOSFET Model

In the following example:

```
.MODEL MNAME PMOS LEVEL=val <par1=val par2=val ...>
```

or:

```
.MODEL MNAME NMOS LEVEL=val <par1=val par2=val ...>
```

where `MNAME` is the model name, and the `PMOS` and `NMOS` are keywords that indicate the `PMOS` and `NMOS` type of MOSFET models, respectively. `par1`, `par2`, ... list the value of the parameters.

A MOSFET is defined by the MOSFET model and element parameters. MOSFET models are either p-channel or n-channel models. They are classified according to level such as LEVEL 1 or LEVEL 50.

The detailed description of MOSFET model parameters can be found in the original UC Berkeley SPICE and Philips documents. For specific details about which models the PrimeSim tool supports, see [Supported Models](#).

Examples

```
M1 d g s b MOD1 L=10u W=5u AD=100p AS=100p  
+ PD=40u PS=40u  
M2 2 4 0 0 nch 0.5u 10u M=3 DTEMP=50  
M7 8 9 vdd vdd pch W=4u L=0.8u IC=-3, -1, 0
```

Independent Sources and Functions

This section describes the following:

- [Voltage Source \(V-element\) and Current Source \(I-element\) Elements](#)
- [Functions for Independent Sources](#)

Voltage Source (V-element) and Current Source (I-element) Elements

Description

This section details voltage source and current source elements in the PrimeSim tool.

The V (voltage source) and I (current source) elements include extensions that allow you to use them as sources of steady-state sinusoidal signals for HB, HBAC, and SNAC analyses.

Syntax

Voltage source

```
Vxxx N+ N- <<DC=>dcval> function <AC=acmag <,acphase>>
+ <RMSJITTER=val> <PERJITTER=val> <SEED=val>
+ <HBAC=>hbac_mag <,hbac_phase>>
+ <SNAC=>snac_mag <,snac_phase>>
+ <HB=>hb_mag <,hb_phase>>
```

Current source

```
Ixxx N+ N- <<DC=>dcval> function <AC=acmag <,acphase>>
+ <RMSJITTER=val> <PERJITTER=val> <SEED=val>
+ <M=val>
+ <HBAC=>hbac_mag <,hbac_phase>>
+ <SNAC=>snac_mag <,snac_phase>>
+ <HB=>hb_mag <,hb_phase>>
```

Arguments

Argument	Description
AC	AC source keyword, for use in AC small-signal analysis.
acmag	Magnitude (RMS) of the AC source.
acphase	Phase of AC source in degrees. Default is 0.0.
DC=dcval	DC source value. Default is 0.0.
function	Time-dependent function for transient analysis. The following functions are supported by the PrimeSim tool: - PULSE - Piece-Wise-Linear (PWL) - Sinusoidal (SIN) - Cosinusoidal (COS) - Exponential (EXP) - Single-Frequency FM (SFFM) - Amplitude Modulation (AM) - Pseudo Random-Bit Generator Source (LFSR) - Pattern Source (PAT)
HB	Harmonic balance (HB) source keyword, for use in single or multi-tone HB algorithm for periodic steady-state analysis for a driven circuit.
hb_mag	Magnitude (RMS) of the HB source.

Argument	Description
<i>hb_phase</i>	Phase of the HB source in degrees. Default is 0.0.
HBAC	Harmonic balance periodic AC (HBAC) source keyword, for use in HBAC analysis on circuits operating in a large-signal periodic steady state.
<i>hbac_mag</i>	Magnitude (RMS) of the HBAC source.
<i>hbac_phase</i>	Phase of the HBAC source in degrees. Default is 0.0.
M	Multiplier used to simulate parallel connection of current source. The default is 1.0.
N+, N-	Positive and negative nodes for the source, respectively. Note that the voltage source, in addition to being used as power supply of the circuit, can also be used as <i>current meter</i> for the circuit. That is, a zero volt voltage source may be inserted into the circuit to measure the current. Positive current is defined to flow from the positive node, through the source, to the negative node.
PERJITTER	RMS value for period jitter, used to adjust the magnitude of the random time.
RMSJITTER	RMS value for random phase jitter, in seconds
SEED	Used to generate random number sequences with a different seed value. The value is a negative integer, defaults to -1.
SNAC	Shooting Newton AC (SNAC) source keyword, for use in running a frequency sweep across a range for the input signal based on a Shooting Newton algorithm.
<i>snac_mag</i>	Magnitude (RMS) of the SNAC source.
<i>snac_phase</i>	Phase of the SNAC source in degrees. Default is 0.0.

Examples

```
Vdd vdd 0 2.5
Vss Vss 0 gnd_value
Vin in vss PULSE(0 2.5 1n 2n 2n 8n 20n)
Vm Iin Iout 0
IG vdd iin SIN(0 1e-3 10MEG)
Iac vdd 3 SFFM( 0 1 10MEG 5 1MEG)
```

Functions for Independent Sources

Independent sources make use of the following functions:

- [PULSE](#)
- [Piece-Wise-Linear \(PWL\)](#)
- [.data Driven PWL Source](#)
- [File-driven PWL Source](#)
- [PWLZ Allows High 'z' State](#)
- [Sinusoidal \(SIN\)](#)
- [Cosinusoidal \(COS\)](#)
- [Exponential \(EXP\)](#)
- [Single-Frequency FM \(SFFM\)](#)
- [Amplitude Modulation \(AM\)](#)
- [Pseudo Random-Bit Generator Source \(LFSR\)](#)
- [Pattern Source \(PAT\)](#)
- [Edgetype: Linear vs. Halfsine](#)

PULSE

Specifies a PULSE function.

Syntax

```
PULSE (V1 V2 TD TR TF PW PER <<RMSJITTER=val> <PERJITTER=val>  
+ <SEED=val> <EDGETYPE=LINEAR|HALFSINE> <THETA_DAMP=val> <THETA_REL=val>  
<THETA_ABS=val>>)
```

Arguments

Argument	Description
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: <code>LINEAR</code> (default) or <code>HALFSINE</code> . See Edgetype: Linear vs. Halfsine for more information.
PER	Period. The default value is the <code>TSTOP</code> value of the <code>.TRAN</code> statement.
PERJITTER	RMS value for period jitter, used to adjust the magnitude of the random time.

Argument	Description
PW	Pulse width. The default value is <code>TSTOP</code> of the <code>.TRAN</code> statement. Note that if <code>PW=0</code> , this pulse function generates a triangular wave.
RMSJITTER	RMS value for random phase jitter, in seconds
SEED	Used to generate random number sequences with a different seed value. The value is a negative integer, defaults to <code>-1</code> .
THETA_DAMP	Specifies the theta damp factor. When <code>THETA_DAMP=val</code> (factor) is given, each pulse (rising and falling) is divided by the value.
THETA_REL	Specifies the percentage for relative theta reduction. When <code>THETA_REL=val</code> (percent) is given, each pulse (rising and falling) is reduced by the given percentage value.
THETA_ABS	Specifies absolute time for theta reduction. When <code>THETA_ABS=val</code> (time) is given, each pulse (rising & falling) is reduced by the given value.
TD	Delay time. The default value is <code>0</code> .
TF	Fall time. The default value is <code>TSTEP</code> of the <code>.TRAN</code> statement.
TR	Rise time. The default value is <code>TSTEP</code> of the <code>.TRAN</code> statement.
V1	Initial value before the pulse onset.
V2	Pulsed value.

Description

A single pulse so specified is described by the following table:

Table 6 Pulse Pattern

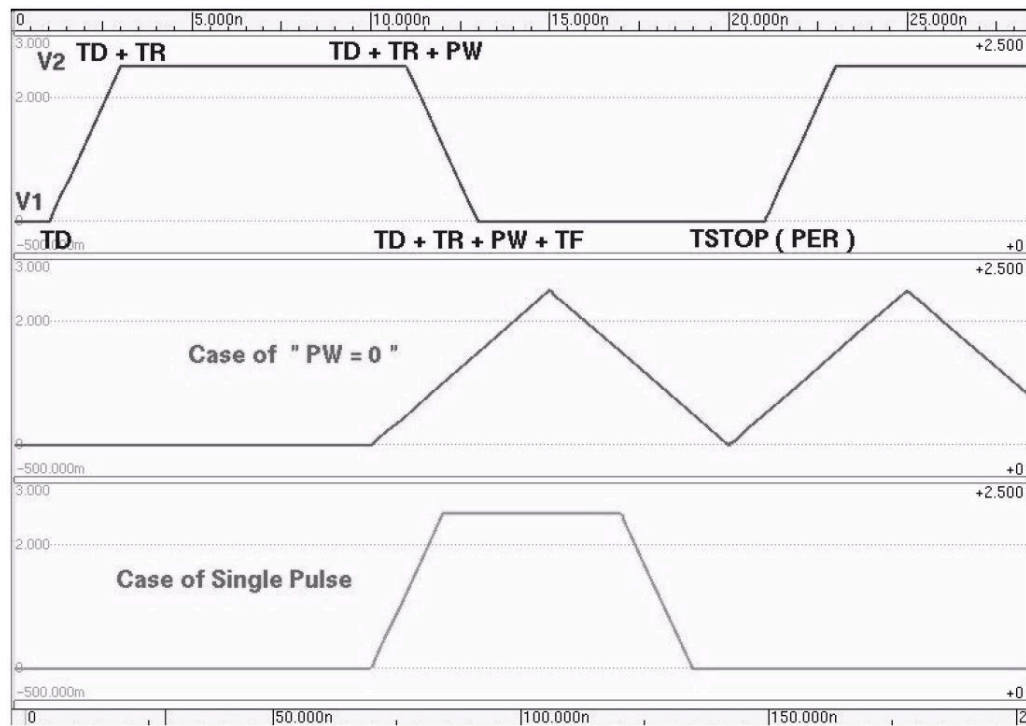
Time (secs)	Value (volt or amp)
0	V1
TD	V1
TD+TR	V2
TD+TR+PW	V2
TD+TR+PW+TF	V1
TSTOP	V1

Examples

```
PULSE ( 0 2.5 1n 2n 2n 8n 20n )
PULSE (0 2.5 10n 5n 5n 0 10n) $ triangular wave
PULSE (0 2.5 10n 2n 2n 5n ) $ single pulse
```

These signals are shown below.

Figure 3 Example Pulse Signals



The following example specifies a pulse with a theta damp factor of 2. This means that the amplitude of each pulse (rising and falling) is reduced by a factor of 2.

```
Va a 0 pulse 0 1 0 10p 10p 10n 20n theta_damp=2
```

The following example specifies a pulse with a theta relative percentage of 10%. This means that the amplitude of each pulse (rising and falling) is reduced by a percentage of 10%.

```
Va a 0 pulse 0 1 0 10p 10p 10n 20n theta_rel=10
```

The following example specifies a pulse with a absolute time of 1 ns. This means that the amplitude of each pulse (rising and falling) is reduced by the specified time.

```
Va a 0 pulse 0 1 0 10p 10p 10n 20n theta_abs=1n
```

Piece-Wise-Linear (PWL)

Description

Specifies a piece-wise-linear function.

Syntax

```
PWL (T1 V1 <T2 V2 T3 V3 ...> <R<=repeat>> <TD=delay>  
+ <EDGETYPE=LINEAR|HALFSINE> <RSTOP=val>  
+ <STOPVALUE=val> <STOPSLOPE=val>  
  
+ <SCALE =val> <STRETCH=val> <OFFSET=val> )
```

Arguments

Argument	Description
SCALE	Amplitude scale factor for the PWL waveform. Default value=1.0
STRETCH	Time scale factor for the PWL waveform. Default value=1.0
OFFSET	DC offset for the PWL waveform. Default value=0 V.
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: LINEAR (default) or HALFSINE. See Edgetype: Linear vs. Halfsine for more information.
R	Keyword and time value to specify a repeating function. With no argument, the source repeats from the beginning of the function. <i>repeat</i> is the time, in units of seconds, which specifies the start point of the waveform to repeat. This time must be less than the greatest time point, T_n .
RSTOP	Specifies the stop time for the repeat, R.
STOPSLOPE	Switching time from the last PWL value to the STOPVALUE. Default is 30 ps.
STOPVALUE	Specifies the value of the current/voltage source at the time of RSTOP. STOPVALUE can be either a real number or Z for high-impedance state.
TD	Delay time. The default value is 0.
$T_n V_n$	Each pair $(T_n V_n)$ specifies that the value of the source is V_n at $\text{time}=T_n$.

Note:

When scale/stretch/offset is specified, many PWL parameters needs to be re-calculated.

- T-V pairs: $t = t \cdot \text{stretch}$, $v = v \cdot \text{scale} + \text{offset}$.
- New parameters: $r = r \cdot \text{stretch}$,
 $r_{\text{stop}} = r_{\text{stop}} \cdot \text{stretch}$, $\text{stopslope} = \text{stopslope} \cdot \text{stretch}$,
 $\text{stopvalue} = \text{stopvalue} \cdot \text{scale} + \text{offset}$.

Examples

```
PWL (0 0 3n 2.5 15n 2.5 18n 0 38n 2.5 59n 2.5 60n 0 R TD=2n)
PWL (0 0 4n 0 4.5n vddval 9.5n vddval 10n 0 )
PWL (0 0n 0 4n vddval 4.5n vddval 9.5n 0 10n )

PWL (60n 0v 120n 0v 130n 5v 170n 5v 180n 0v R=60n RSTOP=100u)
PWL (60n 0v 120n 0v 130n 5v 170n 5v 180n 0v R=60n RSTOP=100u
STOPVALUE=2.5)
PWL (60n 0v 120n 0v 130n 5v 170n 5v 180n 0v R=60n RSTOP=100u
STOPVALUE=2.5 STOPSLOPE=1n)
PWL (60n 0v 120n 0v 130n 5v 170n 5v 180n 0v R=60n RSTOP=100u
STOPVALUE=2.5 STOPSLOPE=1n SCALE=0.5 STRETCH=2 OFFSET=0.3)
```

.data Driven PWL Source

The PrimeSim tool supports a `.data` driven PWL source.

Syntax

```
vxxx sigA gnd PWL(time, sigB)
.tran data=data_name
.data data_name
time sigB
0      0
1e-9 0.5
1.5e-0 1
...
2e-6 1.2
.enddata
```

Description

In `.tran`, the PrimeSim tool is taking the second time in `.data` as *tstep* and the last time as *tstop*.

In the above `.data` example, the transient time step and stop time are 1e-9 and 2e-6, respectively, as with `.tran 1e-9 2e-6`.

File-driven PWL Source

Description

The PrimeSim tool supports specifying a text file containing time, voltage (or current) pairs into a PWL source.

Syntax

```
Vxxx n1 n2 PWL PWLFILE='filename' [TD=delay]
Ixxx n1 n2 PWL PWLFILE='filename' [TD=delay]
```

PWLZ Allows High 'z' State

PWLZ is a PWL source that allows a high 'z' state.

Syntax

```
Vin in 0 PWLZ (t1 v1 <t2 v2 t3 v3 ...>) <logic=0|1>
```

Description

If the `logic` argument is set to 1, then all values of the source (`v1`, `v2`, ...) would be described as logic values 0, 1, and z. The PrimeSim tool automatically sets the proper voltage level by detecting the proper voltage level that corresponds to the specified logic state. For example:

```
PWLZ (0 0 3n 2.5 15n Z 18n Z 38n 2.5 59n 2.5 60n 0)
```

PWLZ can apply on subcircuit nodes using the `subckt=` argument. For example:

```
v1 tru 0 pwlz(0 z 9.8u z 9.9u 0 10u z) subckt="cell"
```

Sinusoidal (SIN)

Specifies a sinusoidal pattern.

Syntax

```
SIN (VO VA <FREQ <TD <THETA <PHASE>>>> <<PERJITTER=val>
+ <SEED=val>>>)
```

Arguments

Argument	Description
FREQ	Specifies the frequency. The default is $1/TSTOP$.
PERJITTER	RMS value for period jitter, used to adjust the magnitude of the random time.
PHASE	Specifies the phase delay in degrees. The default is 0.0.
SEED	Used to generate random number sequences with different seed value. The value is a negative integer and defaults to -1.
TD	Specifies the delay time. The default is 0.
THETA (q)	Specifies the dampening factor in $1/seconds$. The default is 0.0.

Argument	Description
VA	Specifies the amplitude.
VO	Specifies the offset of voltage or current.

Description

The following table specifies a sinusoidal pattern:

Table 7 Sinusoidal Pattern

Time (secs)	Value (Volt or Amp.)
0 to TD	$VO + VA * \sin(2 * \pi * \phi / 360)$
TD to TSTOP	$VO + VA * \exp[-\text{theta} * (\text{time} - \text{TD})] * \sin\{ 2 * \pi * [\text{FREQ} * (\text{time} - \text{TD}) + \phi / 360] \}$

Examples

```
SIN(1 3.3 10MEG 50n 4e6 180)
SIN(1.5 0.5 10MEG 50n 0 90)
```

Cosinusoidal (COS)

Specifies a cosinusoidal pattern.

Syntax

```
COS(VO VA <FREQ <TD <THETA <PHASE>>>> <<PERJITTER=val>
+ <SEED=val>>)
```

Arguments

Argument	Description
FREQ	Specifies the frequency. The default is 1/TSTOP.
PERJITTER	RMS value for period jitter, used to adjust the magnitude of the random time.
PHASE	Specifies the phase delay in degrees. The default is 0.0.
SEED	Used to generate random number sequences with different seed value. The value is a negative integer and defaults to -1.
TD	Specifies the delay time. The default is 0.

Argument	Description
THETA (q)	Specifies the dampening factor in 1/seconds. The default is 0.0.
VA	Specifies the amplitude.
VO	Specifies the offset of voltage or current.

Description

The following table specifies a cosinusoidal pattern:

Table 8 *Cosinusoidal Pattern*

Time (secs)	Value (Volt or Amp.)
0 to TD	$VO + VA * \cos(2 * \pi * \phi / 360)$
TD to TSTOP	$VO + VA * \exp[-\text{theta} * (\text{time} - \text{TD})] * \cos\{ 2 * \pi * [\text{FREQ} * (\text{time} - \text{TD}) + \phi / 360] \}$

Examples

```
COS(1 3.3 10MEG 50n 4e6 180)
COS(1.5 0.5 10MEG 50n 0 90)
```

Exponential (EXP)

Specifies an exponential source.

Syntax

```
V1 out gnd EXP (V1 V2 TD1 TAU1 TD2 TAU2 TD3 TAU3 TD4 TAU4 ...)
```

Arguments

Argument	Description
TAU1 (τ 1)	Rise time constant. The default is TSTEP.
TAU2 (τ 2)	Fall time constant.
TD1	Rise delay time. The default is 0.0.
TD2	Fall delay time. The default is TD1+TSTEP.
V1	Initial value.
V2	Pulsed value.

Description

The standard exponential source allows multiple time delay and time constant pairs without a limit. It is required that each successive delay time for multiple pairs be greater than the last, because the delay is measured from time zero.

The following table specifies an exponential function:

Table 9 *Exponential Pattern*

Time (secs)	Value (Volt or Amp.)
$0 < t \leq TD1$	$V_o = V1$
$TD1 < t \leq TD2$	$V_o = V1 + (V2 - V1) * (1 - \text{Exp}(-(t - TD1) / \text{TAU1}))$
$TD2 < t \leq TD3$	$V_o = (V_o t = TD2) + (V1 - V2) (1 - \text{Exp}(-(t - TD2) / \text{TAU2}))$
$TD3 < t \leq TD4$	$V_o = (V_o t = TD3) + (V2 - V1) * (1 - \text{Exp}(-(t - TD3) / \text{TAU3}))$
$TD4 < t \leq TD5$	$V_o = (V_o t = TD4) + (V1 - V2) (1 - \text{Exp}(-(t - TD4) / \text{TAU4}))$
$TDn < t \leq TSTOP$	$V_o = (V_o t = TDn-1) + (V1 - V2) (1 - \text{Exp}(-(t - TDn-1) / \text{TAUn-1}))$ when $n = \text{even}$ $V_o = (V_o t = TDn-1) + (V2 - V1) (1 - \text{Exp}(-(t - TDn-1) / \text{TAUn-1}))$ when $n = \text{odd}$

Single-Frequency FM (SFFM)

Specifies a single-frequency FM function.

Syntax

SFFM (VO VA FC MDI FS)

Arguments

Argument	Description
FC	Carrier frequency in Hz. The default is $1/TSTOP$.
FS	Frequency in Hz. The default value is $1/TSTOP$.
MDI	Modulation index. The default is 0.0.
VA	Specifies the amplitude.
VO	Specifies the offset.

Description

The following table specifies a single-frequency FM function.

Table 10 Single-Frequency FM Pattern

Time (sec.)	Value (Volt or Amp.)
time	$VO+VA * \sin[2*\pi*FC*time + MDI*\sin(2*\pi*FS*time)]$

Examples

Example

```
SFFM(1 3.3 3MEG 18 3MEG)
```

Amplitude Modulation (AM)

Specifies an amplitude modulation function.

Syntax

```
AM (SA OC FM FC <TD>)
```

Arguments

Argument	Description
FC	Specifies the carrier frequency in Hz. The default value is 1/TSTOP.
FM	Specifies the modulation frequency in Hz. The default value is 1/TSTOP.
OC	Specifies the offset constant, a unitless constant that determines the absolute magnitude of the modulation. The default is 0.0.
SA	Specifies the signal amplitude in volts or amps. The default is 0.0.
TD	Specifies the delay time before start of signal in seconds. The default is 0.0.

Description

The following table specifies an amplitude modulation function:

Table 11 Amplitude Modulation Pattern

Time (sec.)	Value (Volt or Amp.)
time	$SA * \{OC + \sin[2*\pi*FM*(time - TD)]\} * \sin[2*\pi*FC*(time - TD)]$

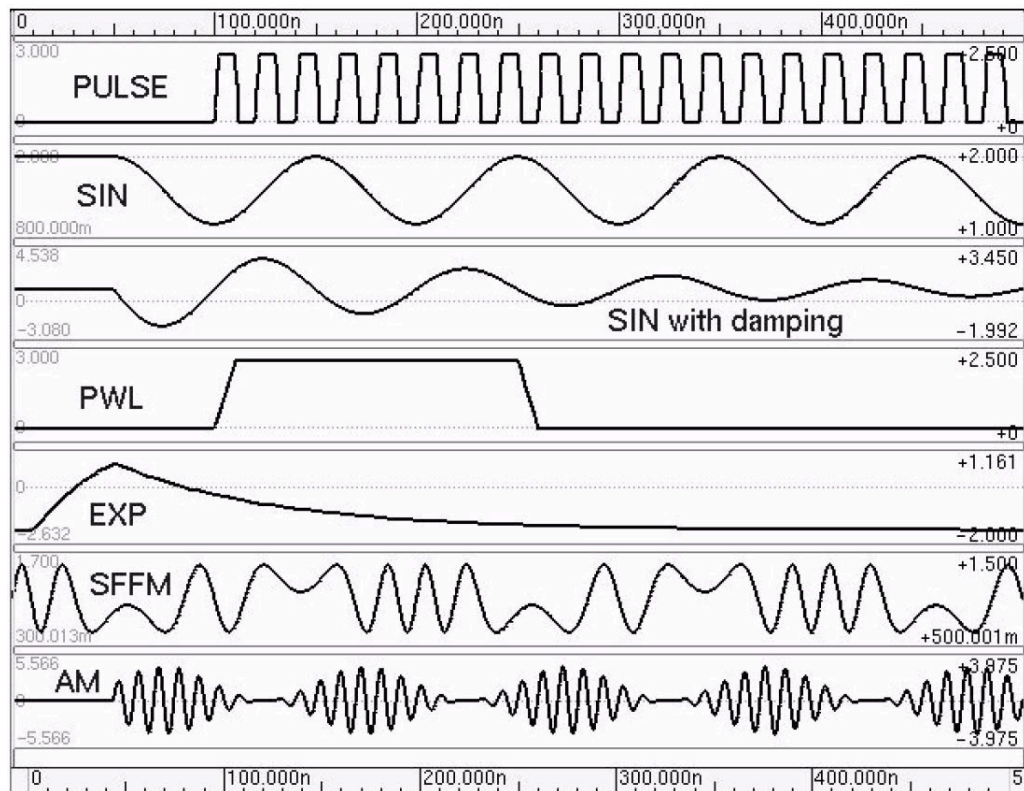
Examples

Example

AM (3.3, 1, 10MEG, 100MEG, 40n)

These functions' waveforms are shown below.

Figure 4 Source Waveforms



Pseudo Random-Bit Generator Source (LFSR)

Description

Specifies a pseudo random-bit generator source.

Syntax

```
LFSR (vlow vhigh tdelay trise tfall rate seed <taps>
+ <ENCODE=DW8b10b> <RD_INIT=0|1> <EDGETYPE=LINEAR|HALFSINE> <rout=val>)
```

Arguments

Argument	Description
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: <code>LINEAR</code> (default) or <code>HALFSINE</code> . See Edgetype: Linear vs. Halfsine for more information.
ENCODE=DW8b10b	Specifies 8b/10b encoding.
rate	Specifies the bit rate.
RD_INIT=0 1	Specifies the initial value of Running Disparity. The one-bit memory that recalls the bias of the last unbalanced code word is the Running Disparity: 0 (default): Specifies that a Running Disparity value of one is synonymous with negative Running Disparity (-). 1: Specifies that a Running Disparity value of one is synonymous with positive Running Disparity (+).
seed	Specifies the initial value loaded into the shift register.
taps	Specifies the bits used to generate feedback.
tdelay	Specifies the initial time delay to the first transition.
tfall	Specifies the duration of the recovery ramp, from the pulse plateau back to the initial value.
trise	Specifies the duration of the onset ramp, from the initial value to the pulse plateau value.
vhigh	Specifies the maximum level of voltage or current.
vlow	Specifies the minimum level of voltage or current.
rout=val	Specifies the output resistance.

Examples

```
vin in gnd LFSR (0 1 1m 1n 1n 10meg 1 [5, 2] rout=10)
```

Comparison of PrimeSim HSPICE-compatible and 3rd-party Syntax

For PrimeSim HSPICE-compatible LFSR source, positive transitions occur at $td + (n / rate) - tr$, and the negative transitions occur at $td + (n / rate) - tf$, where:

- td is the time delay
- $1 / rate$ is the UI

- t_r and t_f are the rise and fall time, respectively
- n is the n th bit in the LFSR sequence

For example:

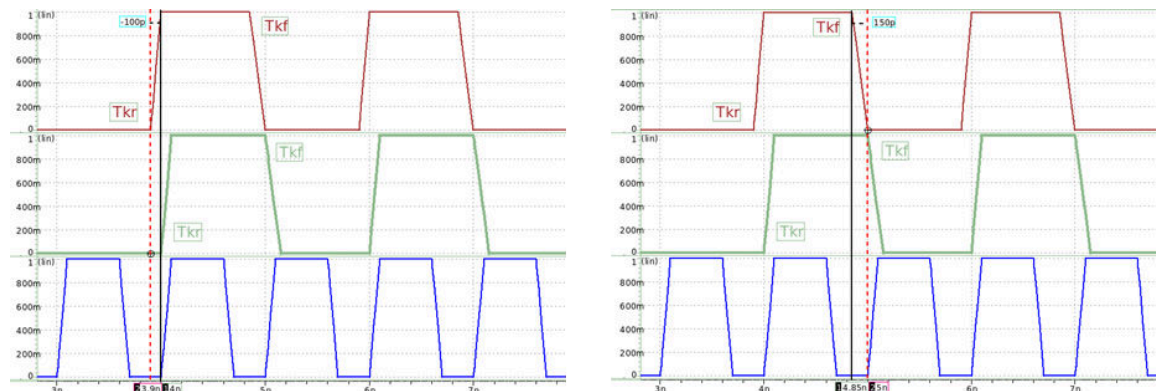
```
.OPTION POST
.PARAM rate=1e9 Tr=0.1n Tf=0.15n Td=0 Vh=1 Vl=0 seed=1
.PARAM pw=0.5ns per=1ns

Vlfsr 1 0 LFSR ( Vl Vh Td Tr Tf rate seed [4, 2] )
Vclk 2 0 pulse( Vl Vh Td Tr Tf pw per )

r1 1 0 1e4
r2 2 0 1e4
.TRAN 0.1n 10n
.END
```

The PrimeSim HSPICE-compatible behavior is in contrast to some 3rd-party simulators, where the transitions occur at $t_d + (n/\text{rate})$, as shown in Figure 5 where the red line shows PrimeSim HSPICE-compatible output, the green line shows 3rd-party output, and the blue line shows the reference clock:

Figure 5 LFSR Sample Output: Rising (l) and Falling (r)



Pattern Source (PAT)

Specifies a pattern source.

Syntax

```
PAT (vhi vlo td tr tf tsample data <RB=val> <R=val>
+ <EDGETYPE=LINEAR|HALFSINE>
+ <RJRMS=val> <RJSEED=val>
+ <PJAMP=val1,...,valN> <PJFREQ=val1,...,valN>
+ <PJTYPE=SINE|SAWTOOTH|SQUARE>)
```

or

```
PAT (vhi vlo td tr tf tsample [component 1, ..., component
+ n] <RB=val> <R=val> <EDGETYPE=LINEAR|HALFSINE>
+ <RJRMS=val> <RJSEED=val>
+ <PJAMP=val1,val2,val3> <PJFREQ=val1,val2,val3>
+ <PJTYPE=SINE|SAWTOOTH|SQUARE>)
```

Arguments

Argument	Description
component 1 ... n	The components that make up nested structures. These structures can be a bit-pattern or a pattern-name defined in another .pat command. RB or R can be used in the component.
data	Bit string of 1, 0, m, or z starting with B, or a pattern-name defined in other .pat command.
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: LINEAR (default) or HALFSINE. See Edge type: Linear vs. Halfsine for more information.
PAT	Keyword for a time-dependent pattern source.
PJAMP=val1,...,valN	List of periodic jitter amplitude. The unit is time (seconds). The default is an empty list.
PJFREQ=val1,...,valN	List of periodic jitter frequencies. The unit is frequency (Hz). The default is an empty list.
PJTYPE	List of periodic jitter types. The element choices are: SINE, SAWTOOTH, and SQUARE. The default is SINE.
R=val	Keyword to specify the number of repeating operations. R must be an integer. It can be set to 0 or -1. When set to -1, the repeating operation continues forever. When this value is set to less than -1, the PrimeSim Pro tool automatically resets it to 0.
RB=val	Keyword to specify the starting bit when repeating. Default value is 1.
RJRMS=val	Root-mean-square value of random jitter. The unit is time (seconds) and the default is 0.
RJSEED=val	Seed to generate random jitter. The default is 1.
td	Delay time from the beginning of the transient interval to the first onset ramp.
tf	Duration of the recovery ramp from the high to the low value.
tr	Duration of the onset ramp from the low to the high value.

Argument	Description
<code>tsample</code>	Time spent at 1, 0, m, or z pattern value.
<code>vhi</code>	High voltage or current value.
<code>vlo</code>	Low voltage or current value.

Description

The PrimeSim tool supports the time dependent bit pattern (PAT) function. This special function is used to describe some bit patterns in place of PWL in the independent voltage or current sources. The PrimeSim Pro tool also supports nested structure and pattern-command (`.PAT`) driven structure for the pattern source function to construct complex bit patterns. The nested structure starts with an open square bracket ('[') and ends with closing square bracket (']').

To create complex bit patterns, the pattern source function can be written using either a nested structure notation or a pattern-command driven notation. (See `.PAT` in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options* for more information).

Examples

Example 1 (simple form):

```
v1 n1 n2 PAT (5 0 1n 2n 5n 10n b1010111 r=2 rb=2 b10m1z)
```

Example 2 (nested structure form):

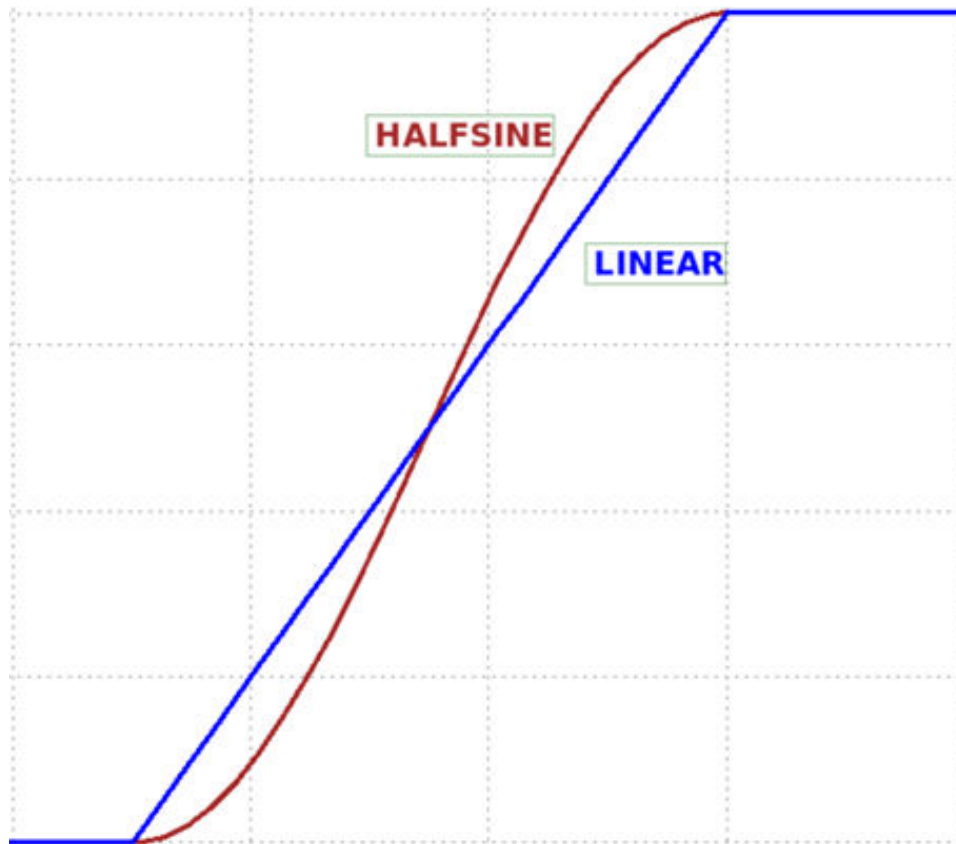
```
v1 n1 n2 PAT (5 0 1n 2n 5n 10n [b1101 r=1 rb=2 [b10m1z r=2 rb=2]] r=2
rb=2)
v1 n1 n2 PAT (5 0 1n 2n 5n 10n [b1101 [b10m1z r=2 rb=2]] r=2 rb=2)
```

Edgetype: Linear vs. Halfsine

Figure 6 illustrates the difference between `EDGETYPE=LINEAR` and `EDGETYPE=HALFSINE`.

The linear (default) edgetype waveform has a corner transition at the rising and falling edges, while the halfsine edgetype waveform has a sinusoidal roll off at the rising and falling edges.

Figure 6 Comparison of Linear and Halfsine Edgetypes



Dependent Sources and Functions

The PrimeSim Pro tool supports the following types of dependent sources/instances:

- **E-element:** Voltage controlled voltage source (VCVS)
- **F-element:** Current controlled current source (CCCS)
- **G-element:** Voltage controlled current source or resistor or capacitor (VCCS, VCR, and VCCAP).
- **H-element:** Current controlled voltage source (CCVS)

The dependency between these elements and their control source can be expressed by linear, or [Piece-Wise-Linear \(PWL\)](#) functions.

This section describes the following:

- [Voltage Controlled Voltage Source-VCVS \(E-element\)](#)
- [Current Controlled Current Source-CCCS \(F-element\)](#)
- [Voltage Controlled Current Source-VCCS \(G-element\), Voltage Controlled Resistor-VCR, Voltage Controlled Capacitor-VCCAP](#)
- [Current Controlled Voltage Source-CCVS \(H-element\)](#)
- [S-Element](#)
- [Functions for Dependent Sources](#)

Voltage Controlled Voltage Source-VCVS (E-element)

The PrimeSim tool supports voltage-controlled voltage sources VCVS (E-elements). These sources are functions of the input voltages (linear, piece-wise-linear, polynomial, gates, delays, and behavioral).

Syntax

Linear:

```
Exxx N+ N- <VCVS> in+ in- gainval <MAX=val> <MIN=val>  
+ <SCALE=val> <TC1=val> <TC2=val> <ABS=1> <IC=val>
```

PWL:

```
Exxx N+ N- <VCVS> PWL(1) in+ in- <SCALE=val> <TC1=val>  
+ <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

Polynomial:

```
Exxx N+ N- <VCVS> POLY(k) in1+ in1- ... ink+ ink- <SCALE=val>  
+ <TC1=val> <TC2=val> <MAX=val> <MIN=val> <ABS=1> P0 P1  
+ ... <IC=val>
```

Gates:

```
Exxx N+ N- <VCVS> type(k) in1+ in1- ... ink+ ink- <SCALE=val>  
+ <TC1=val> <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

Delay:

```
Exxx N+ N- <VCVS> DELAY in+ in- TD=val <SCALE=val> <TC1=val>  
+ <TC2=val>
```

Ideal Transformer (Element):

```
Exxx N+ N- TRANSFORMER in+ in- k
```

Behavioral Voltage Source:

```
Exxx N+ N- VOL="equation" <MAX=val> <MIN=val> <SCALE=val>
```

Laplace Transform:

```
Exxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0, d1, ..., dm  
+ <SCALE=val> <TC1=val> <TC2=val>
```

Pole-Zero Function:

```
Exxx n+ n- POLE in+ in- a az1, fz1, ..., azn, fzn / b,  
+ ap1, fp1, ..., apm, fpm <SCALE=val> <TC1=val>  
+ <TC2=val>
```

Frequency Response Table:

```
Exxx n+ n- FREQ in+ in- f1, a1, f1, ..., fi, ai, f1  
+ <DELF=val> <MAXF=val> <SCALE=val> <TC1=val>  
+ <TC2=val> <LEVEL=val> <ACCURACY=val>
```

Foster Pole-Residue Form:

```
Exxx n+ n- FOSTER in+ in- k0 k1  
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})  
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})  
+ (Re{A3}, Im{A3}) / (Re{p3}, Im{p3})  
+ ...
```

See also Behavioral Voltage Noise Source in the *PrimeSim™ Continuum User Guide: Elements*

Arguments

Argument	Description
ABS	If ABS=1, output is absolute value.
DELAY	Keyword to indicate a delayed voltage source.
FOSTER	Keyword to indicate a Foster pole-residue form.
FREQ	Keyword to indicate a frequency response table.
gainval	Value of voltage gain. The default is 1.
IC	The initial estimate of the values of the controlling voltage. The default is 0.0.
in+, in-	Positive or negative controlling nodes. Specify one pair for each dimension.

Argument	Description
k	Dimension of the polynomial function or number of inputs to the gate function.
LAPLACE	Keyword to indicate a Laplace transform.
MAX, MIN	Maximum and minimum output voltage values. Note: To match 3rd-party behavior, the PrimeSim tool supports the MAX and MIN arguments in E-elements when the DELAY keyword is specified in 3rd-party netlists. To match PrimeSim HSPICE behavior, the PrimeSim tool ignores the MAX and MIN arguments when the E-element is used as a delay element in SPICE netlists.
N+, N-	Positive and negative nodes for the controlled element, respectively.
P0, P1 ...	Polynomial coefficients.
POLE	Keyword to indicate a pole-zero function.
POLY	Keyword to indicate a polynomial function.
PWL(1)	Keyword to indicate a piece-wise-linear function.
SCALE	Scale factor for element value.
TC1, TC2	First and second order temperature coefficients. $SCALE_{eff} = SCALE * (1 + TC1 * dT + TC2 * dT^2)$
TD	Delay time.
TRANSFORMER	Keyword for an ideal transformer.
type	Type of gates, which must be and, nand, or, or nor.
VCVS	Optional keyword to indicate voltage controlled voltage source.
VOL	Voltage output that flows from N+ to N-. The expression that you define can be a function of: • Node voltages • Branch currents • Time (time variable) • Temperature (temper variable) • Frequency (hertz variable)
x1, x2, ...	Value of controlling voltage between node in+ and in-.
y1, y2, ...	Corresponding element value of the controlled voltage.

Description

Specifies a voltage-controlled voltage source.

Behavioral Time Derivative Voltage Source

The E-element can be used as a behavioral time derivative voltage source. The function `ddt()` computes the time derivative, $\frac{d}{dt}x(t)$, of the argument where the argument is an expression that can include state and input variables, parameters, node voltages or currents.

For voltage sources using the `ddt()` function, the E-element takes the form:

```
Exxx node1 node2 ddt('expression')
```

Current Controlled Current Source-CCCS (F-element)

Description

The PrimeSim tool supports current-controlled current sources. These sources are functions of the input voltages (linear, piece-wise-linear, polynomial, gates, delays, and behavioral).

Syntax

Linear:

```
Fxxx N+ N- <CCCS> vname <lin> gainval <MAX=val> <MIN=val>  
+ <SCALE=val> <TC1=val> <TC2=val> <M=val> <ABS=1>  
+ <IC=val>
```

PWL:

```
Fxxx N+ N- <CCCS> PWL(1) vname <M=val> <SCALE=val> <TC1=val>  
+ <TC2=val> <ABS=1> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

Polynomial:

```
Fxxx N+ N- <CCCS> POLY(k) vin1... vink <M=val> <SCALE=val>  
+ <TC1=val> <TC2=val> <MAX=val> <MIN=val> <ABS=1> P0 P1  
+ ... <IC=val>
```

Gates:

```
Fxxx N+ N- <CCCS> type(k) vin1... vink <M=val> <SCALE=val>  
+ <TC1=val> <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

Delay:

```
Fxxx N+ N- <CCCS> DELAY vname TD=val  
+ <SCALE=val> <TC1=val> <TC2=val>
```

Arguments

Argument	Description
ABS	If ABS=1, output is absolute value.
CCCS	Optional keyword to indicate the current controlled current source.
DELAY	Keyword to indicate a delayed instance.
gainval	Value of voltage gain. The default is 1.
IC	The initial estimate of the values of the controlling current. The default is 0.0.
k	Dimension of the polynomial function or number of inputs to the gate function.
lin	Optional keyword to indicate a linear function.
M	Multiplier for parallel connection of current source.
MAX, MIN	Maximum and minimum output current values. Note: To match 3rd-party behavior, the PrimeSim tool supports the MAX and MIN arguments in F-elements when the DELAY keyword is specified in 3rd-party netlists. To match PrimeSim HSPICE behavior, the PrimeSim tool ignores the MAX and MIN arguments when the F-element is used as a delay element in SPICE netlists.
N+, N-	Positive and negative nodes for the controlled instance, respectively.
P0, P1 ...	Polynomial coefficients.
POLY	Keyword to indicate a polynomial function.
PWL(1)	Keyword to indicate a piece-wise-linear function.
SCALE	Scale factor for element value.
TC1, TC2	First and second order temperature coefficients. $SCALE_{eff} = SCALE * (1 + TC1 * dT + TC2 * dT^2)$
TD	Delay time for delayed instance. The default value is 0.0.
type	Type of gates, which must be and, nand, or or nor.
vname, ...	Names of voltage source through which the controlling current flows. At least one voltage source name should be defined.

Argument	Description
$x1, x2, \dots$	Current through controlling voltage source $vname$. You can specify up to 100 points.
$y1, y2, \dots$	Corresponding current value of the controlled instance.

Examples

```
Fp in out CCCS Vzero 0.4 max=1m min=1u M=4
Fil 0 out PWL(1) VSRC 1m 10m -2m 0.1m
```

The first example is a CCCS connected between `in` and `out`. The current that controls the value of `Fp` flows through the voltage source `Vzero`. The value is $0.4 \times I(Vzero) \times 4$ and is clamped by `(1m, 1u)`.

The second example is a CCCS connected between `0` and `out`. When the current through `VSRC` is 1mA, the current value is 10mA. When the current through `VSRC` is -2mA, the current value is 0.1mA.

Voltage Controlled Current Source-VCCS (G-element), Voltage Controlled Resistor-VCR, Voltage Controlled Capacitor-VCCAP

Specifies a voltage-controlled current, resistor, or capacitor source.

Syntax

Linear:

```
Gxxx N+ N- <VCCS> in+ in- transconductance <M=val> <MAX=val>
+ <MIN=val> <SCALE=val> <TC1=val> <TC2=val> <ABS=1>
+ <IC=val>
```

or:

```
Gxxx N+ N- VCR in+ in- transfactor <M=val> <MAX=val>
+ <MIN=val>
```

PWL:

```
Gxxx N+ N- <VCCS> PWL(1) in+ in- <M=val> <SCALE=val>
+ <TC1=val> <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

or:

```
Gxxx N+ N- VCR PWL(1) in+ in- <M=val> <SCALE=val> <TC1=val>
+ <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```


or:

```
Gxxx N+ N- VCCAP PWL(1) in+ in- <M=val> <SCALE=val> <TC1=val>  
+ <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

NPWL and PPWL:

```
Gxxx N+ N- <VCCS> NPWL(1) in+ in- <DELTA=val> <SCALE=val>  
+ <M=val> <TC1=val> <TC2=val> x1,y1 x2,y2 ... x100,y100  
+ <IC=val>
```

Polynomial:

```
Gxxx N+ N- <VCCS> POLY(k) vin1... vink <M=val> <SCALE=val>  
+ <TC1=val> <TC2=val> <MAX=val> <MIN=val> <ABS=1> P0 P1  
+ ... <IC=val>
```

Gates:

```
Gxxx N+ N- <VCCS> type(k) vin1 ... vink <M=val> <SCALE=val>  
+ <TC1=val> <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

Delay:

```
Gxxx N+ N- <VCCS> DELAY in+ in- TD=val <SCALE=val> <TC1=val>  
+ <TC2=val>
```

Laplace Transform:

```
Gxxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0, d1, ..., dm  
+ <SCALE=val> <TC1=val> <TC2=val> <M=val>
```

Pole-Zero Function:

```
Gxxx n+ n- POLE in+ in- a az1, fz1, ..., azn, fzn / b,  
+ ap1, fp1, ..., apm, fpm <SCALE=val> <TC1=val>  
+ <TC2=val> <M=val>
```

Frequency Response Table:

```
Gxxx n+ n- FREQ in+ in- f1, a1, f1, ..., fi, ai, f1  
+ <DELF=val> <MAXF=val> <SCALE=val> <TC1=val>  
+ <TC2=val> <M=val> <LEVEL=val> <ACCURACY=val>
```

Foster Pole-Residue Form:

```
Gxxx n+ n- FOSTER in+ in- k0 k1  
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})  
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})  
+ (Re{A3}, Im{A3}) / (Re{p3}, Im{p3})  
+ ...
```

See also Behavioral Current Noise Source in the *PrimeSim™ Continuum User Guide: Elements*.

Arguments

Argument	Description
ABS	If ABS=1, output is absolute value.
DELAY	Keyword to indicate a delayed current source.
FOSTER	Keyword to indicate a Foster pole-residue form.
FREQ	Keyword to indicate a frequency response table.
IC	The initial estimate of the values of the controlling voltage. The default is 0.0.
in+, in-	Positive or negative controlling nodes. Specify one pair for each dimension.
k	Dimension of the polynomial function or number of inputs to the gate function.
LAPLACE	Keyword to indicate a Laplace transform.
M	Multiplier for parallel connection of controlled instances
MAX, MIN	Maximum and minimum current or resistance values. Note: To match 3rd-party behavior, the PrimeSim tool supports the MAX and MIN arguments in G-elements when the DELAY keyword is specified in 3rd-party netlists. To match PrimeSim HSPICE behavior, the PrimeSim tool ignores the MAX and MIN arguments when the G-element is used as a delay element in SPICE netlists.
N+, N-	Positive and negative nodes for the controlled instance, respectively.
P0, P1 ...	Polynomial coefficients.
POLE	Keyword to indicate a pole-zero function.
POLY	Keyword to indicate a polynomial function.
PWL(1)	Keyword to indicate a piece-wise-linear function.
SCALE	Scale factor for element value.
TC1, TC2	First and second order temperature coefficients. $SCALE_{eff} = SCALE * (1 + TC1 * dT + TC2 * dT^2)$
TD	Delay time.
transconductance	Voltage to current conversion factor. The default is 1.

Argument	Description
transfactor	Voltage to resistance conversion factor. The default is 1.
type	Type of gates, which must be <code>and</code> , <code>nand</code> , <code>or</code> , or <code>nor</code> .
VCCAP	Keyword for voltage controlled capacitor
VCCS	Optional keyword to indicate voltage controlled current source. This is the default mode for a G-element.
VCR	Keyword for voltage controlled resistor.
x1,x2, ...	Value of controlling voltage between node <code>in+</code> and <code>in-</code> . You can specify up to 100 points.
y1,y2, ...	Corresponding current/resistance/capacitance value of the controlled instance.

Description

The PrimeSim tool supports G-element for foster pole-residue form. The form is:

```
Gxxx N+ N- FOSTER IN+ IN- k0 k1
+ (Re{A1}, Im{A1}) / (Re{p1}, Im{p1})
+ (Re{A2}, Im{A2}) / (Re{p2}, Im{p2})
+ ...
```

A pole-residue pair is represented by four numbers (real and imaginary part of the residue, then real and imaginary part of the pole).

Behavioral Current Source

This section shows a behavioral current source example:

```
gxxx node1 node2 noise='noise_expression'
```

The above syntax creates a simple two-terminal current noise source. The output noise is `noise_expression*H`, where `H` is the transfer function from the terminal pair (`node1,node2`) to the circuit output, where the output noise is measured. For example:

```
gnoise 1 2 noise='4*1.3806266e-23*(TEMPER+273.15)*0.001'
```

The noise data can also be expressed in data format:

```
Gxxx node1 node2 noise data=dataname
.DATA dataname
+ pname1 pname2
+ freq1 noise1
+ freq2 noise2
+ ...
.enddata
```

The data form defines a basic frequency-noise table. The `.DATA` statement contains two parameters, frequency and noise, to specify the noise value at each frequency point. The unit for frequency is hertz, and the unit for noise is A^2/Hz .

The G-element with noise sources supports the following syntax:

```
Gxxx n1 n2 noise data=dataname
```

Behavioral Time Derivative Current Source

The G-element can be used as a behavioral time derivative current source. The function `ddt()` computes the time derivative, $\frac{d}{dt}x(t)$, of the argument where the argument is an expression that can include state and input variables, parameters, node voltages or currents.

For current sources using the `ddt()` function, the G-element takes the form:

```
Gxxx node1 node2 ddt('expression')
```

NPWL Function

If the node is connected, `in-` is connected to `n-`, as follows:

If $v(N+, N-) > 0$, the controlling voltage is $v(in+, in-)$.

Otherwise, the controlling voltage is $v(in+, N+)$.

If node `in-` is connected to `n+`, then if $v(N+, N-) < 0$, the controlling voltage is $v(in+, in-)$. Otherwise, the controlling voltage is $v(in+, N+)$.

PPWL Function

If the node is connected, `in-` is connected to `n-`, as follows:

If $v(N+, N-) < 0$, the controlling voltage is $v(in+, in-)$.

Otherwise, the controlling voltage is $v(in+, N+)$.

For node `in-` connected to `N+`, if $v(N+, N-) > 0$, the controlling voltage is $v(in+, in-)$. Otherwise, the controlling voltage is $v(in+, N+)$.

Laplace Function

The PrimeSim Pro tool supports the Laplace function.

Syntax

```
Gxxx n+ n- LAPLACE in+ in- k0, k1, ..., kn / d0, d1, ..., dm
```

$H(s)$ is a rational function, in the following form:

$$H(s) = (k_0 + k_1s + \dots + k_n s^n) / (d_0 + d_1s + \dots + d_m s^m)$$

The following parameters can be used to define the values of all coefficients (k_0 , k_1 , ..., d_0 , d_1 , ...).

Example

```
G22 0 out LAPLACE in 0 1.0 / 1.0 2.0 2.0 1.0
```

The G22 element statement describes a third-order low-pass filter, with the transfer function:

$$H(s)=1/(1 + 2s + 2s^2 + s^3)$$

Examples

```
G2 out 0 VCCS pnode nnode 1e-6 M=2
Gswitch 1 2 VCR PWL(1) Vswitch 0 0, 10MEG 1, 1e-6
Gcap 22 19 VCCAP PWL(1) V+ V-
+ 1 1e-12
+ 2 2e-12
+ 3 3e-12
```

The first example is a current source connected between `out` and 0. It depends on the voltage between `pnode` and `nnode` and the transconductance is `1e-6` with a multiplier of 2.

The second example is a way to describe the operation of the switch. That is, when the controlled voltage is 0, it is off (with high resistance `1e7`) and when the voltage is 1 V, it is on (with the low resistance `1e-6`).

The third example is a voltage dependent capacitor with the capacitance value 1 pF when the controlled voltage is 1 V and 2 pF when the controlled voltage is 2 V and so on.

Current Controlled Voltage Source-CCVS (H-element)

Description

Specifies a current-controlled voltage source.

Syntax

Linear:

```
Hxxx N+ N- <CCVS> vname <lin> transresistance <MAX=val>
+ <MIN=val> <SCALE=val> <TC1=val> <TC2=val> <ABS=1>
+ <IC=val>
```

PWL:

```
Hxxx N+ N- <CCVS> POLY(k) vin1... vink <SCALE=val> <TC1=val>
+ <TC2=val> <MAX=val> <MIN=val> <ABS=1> P0 P1 ... <IC=val>
```

Gates:

```
Hxxx N+ N- <CCVS> type(k) vin1 ... vink <SCALE=val> <TC1=val>
+ <TC2=val> x1,y1 x2,y2 x3,y3 ... <IC=val>
```

Delay:

```
Hxxx N+ N- <CCVS> DELAY vname TD=val
+ <SCALE=val> <TC1=val> <TC2=val>
```

Arguments

Argument	Description
CCVS	Optional keyword to indicate current controlled voltage source.
DELAY	Keyword to indicate delayed voltage source.
IC	The initial estimate of the values of the controlling current. The default is 0.0.
k	Dimension of polynomial function or the number of inputs to the gate function.
lin	Optional keyword to indicate linear function.
MAX, MIN	Maximum and minimum output voltage values. Note: To match 3rd-party behavior, the PrimeSim tool supports the <code>MAX</code> and <code>MIN</code> arguments in H-elements when the <code>DELAY</code> keyword is specified in 3rd-party netlists. To match PrimeSim HSPICE behavior, the PrimeSim tool ignores the <code>MAX</code> and <code>MIN</code> arguments when the H-element is used as a delay element in SPICE netlists.
N+, N-	Positive and negative nodes for the controlled instance, respectively.
P0, P1 ...	Polynomial coefficients.
POLY	Keyword to indicate a polynomial function.
PWL(1)	Keyword to indicate piece-wise-linear function.
SCALE	Scale factor for element value.
TC1, TC2	First and second order temperature coefficients. $SCALE_{eff} = SCALE * (1 + TC1 * dT + TC2 * dT^2)$
TD	Delay time.
transresistance	Current to voltage conversion factor.

Argument	Description
<code>type</code>	Type of gates, which must be <code>and</code> , <code>nand</code> , <code>or</code> , or <code>nor</code> .
<code>vname, ...</code>	Names of voltage sources through which the controlling current flows. At least one voltage name must be defined.
<code>x1, x2, ...</code>	Controlling current through <code>vname</code> . You can specify up to 100 points.
<code>y1, y2, ...</code>	Corresponding voltage values of the controlled instances.

Examples

```
Hqq 2 1 Vsuppply 1000
```

This example is a voltage source dependent on the current through `Vsupply` and the trans-resistance is `1000`.

S-Element

The PrimeSim tool supports running simulations with the S-element.

Syntax

```
Sxxx nd1 nd2 ... ndN ndRef [MNAME=Smodel_name] [TYPE=s|y]
+ [Z0=value|vector_value] [FBASE=base_frequency]
+ [FMAX=maximum_frequency]
+ [INTERPOLATION=STEP|LINEAR|SPLINE|HYBRID]
+ [INTDAT TYP=RI|MA|DBA] [HIGHPASS=1|2|3|4]
+ [LOWPASS=0|1|2|3] [DELAYHANDLE=1|0|ON|OFF]
+ [DELAYFREQ=val] [MIXEDMODE=0|1] [DATATYPE=data_string]
+ [DTEMP=val] [RATIONAL_FUNC=[0|1]]
+ [RATIONAL_FUNC_REUSE=[0|1|2]] [PASSIVE=0|1]
+ [PASSIVE_TOL=val] [COLSUM_LIMIT=val]
+ [ENFORCE_PASSIVE=0|1] [STAMP=S|Y|YSTS|SSTS|DEEMBED]
+ [M=int] [PREC FAC=val]
+ [RATIONAL_FUNC_CACHE_DIR=dir_str]
+ [RATIONAL_FUNC_UNDERSAMPLE=val]
```

Description

The S-element contains two parts: an instantiation line for each S-element (see *S-Element Syntax* above) and a `.model` section to specify the modeling of the S-element (see *S-Model Syntax* below). For additional information on S-element simulation, see [Support for PrimeSim HSPICE S-Parameter and Modeling](#).

The following example illustrates the *nd1 nd2...ndN*—*no reference, single reference, and multi-reference parameters*.

```
*S-parameter example
.OPTION post primesim_mode=spicemd
.probe v(*)
V1 n1 0 ac=1v PULSE 0v 5v 5n 0.5n 0.5n 25n
.ac lin 500 1Hz 30MegHz
.tran 0.1ns 10ns
```

```
* reference node is set
S1 n1 n2 0 mname=s_model
```

```
* no reference node
S2 n1 n3 mname=s_model
```

```
* multi-reference node
S3 n1 0 n4 0 mname=s_model
```

```
* S parameter
.model s_model S TSTONEFILE = ss_ts.s2p
Rt1 n2 0 50
Rt2 n3 0 50
Rt3 n4 0 50
.end
```

```
TOUCHSTONE file ss_ts.s2p as below:
!
! touchstone file example "ss_ts.s2p"
!
# GHz S DB R 50
!
! - RI -
!1.0 0.3333333333 0.0 0.6666666667 0.0
! 0.6666666667 0.0 0.3333333333 0.0
! - DB -
1.0 -9.54242510308 0.0 -3.52182518107 0.0
-3.52182518107 0.0 -9.54242510308 0.0
! end of file
```

S-Model Syntax

```
.MODEL Smodel_name S [N=dimension]
+ [TSTONEFILE=filename | CITIFILE=filename |
+ RFMFILE=file_name.rfm]
+ [TYPE=s|y] [Z0=value | vector_value]
+ [FBASE=base_frequency] [FMAX=maximum_frequency]
+ [INTERPOLATION=STEP|LINEAR|SPLINE|HYBRID]
+ [INTDAT TYP=[RI|MA|DBA]]
+ [HIGHPASS=0|1|2|3|4] [LOWPASS=0|1|2|3]
+ [DELAYHANDLE=1|0|ON|OFF] [DELAYFREQ=val]
+ [MIXEDMODE=0|1]
+ [DATATYPE=data_string] [XLINELENGTH=val] [PASSIVE=[0|1]]
```



```
+ [SMOOTH=val] [SMOOTHPTS=val]
+ [RATIONAL_FUNC=[0|1] [RATIONAL_FUNC_REUSE=0|1|2]
+ [PASSIVE=[0|1] [PASSIVE_TOL=val] [COLSUM_LIMIT=val]
+ [ENFORCE_PASSIVE=0|1] [STAMP=S|Y|YSTS|SSTS|DEMBED]
+ [PRECFACT=val]
+ [RATIONAL_FUNC_CACHE_DIR=dir_str]
+ [RATIONAL_FUNC_UNDERSAMPLE=val]
```

Arguments

Argument	Description
CITIFILE	Specifies the name of the CITIfile, which is a data file that contains frequency-dependent data. Note that string parameters are supported for calling a CITIFILE. For details, see <i>Using Instruments with ADS</i> by Agilent Technologies (http://www.agilent.com).
COLSUM_LIMIT	Maximum value allowed for S-parameter matrix column summation. The limiting value is used for the passivity checker and passivity enforcement. Default value is 1.0.
DATATYPE	A string used to determine the order of the indices of the mixed-signal incident or reflected vector. The string must be an array of a letter and a number (X_n) where: $X = D$ to indicate a differential term $X = C$ to indicate a common term $X = S$ to indicate a single (grounded) term n = the port number
DELAYFREQ	Delay frequency for transmission-line type parameters. The default is FMAX. If the DELAYHANDLE is set to OFF, but DELAYFREQ is nonzero, the PrimeSim tool still simulates the S-element in the delay mode.
DELAYHANDLE	DELAYHANDLE extracts the system delay before constructing the system impulse response. This might improve transient accuracy when the system does have delay, such as a transmission line system, and RATIONAL_FUNC=0 is used. When RATIONAL_FUNC=1 (default) is used, it is recommended to not specify the DELAYHANDLE keyword. The S-element automatically detects and handles the propagation delay. If DELAYHANDLE is ON (or 1), the S-element extracts the propagation delay to simplify transfer functions, and then proceeds for approximation. The extracted delay is handled separately in the time domain. You must set the delay handler, if the delay of the model is longer than the base period specified in the FBASE parameter. If you set DELAYHANDLE=OFF (or 0), but DELAYFREQ is not zero, the PrimeSim tool simulates the S-element in the delay mode. For Y-parameter models, DELAYHANDLE is not supported and is ignored.

Argument	Description
DTEMP	Temperature difference between the element and the circuit, expressed in °C. The default is 0.0. Element temperature is calculated as: $T = \text{Element temperature (°K)} \\ = 273.15 \text{ (°K)} + \text{circuit temperature (°C)} \\ + \text{DTEMP (°C)}$ Where circuit temperature is specified using either the .TEMP statement, or by sweeping the global TEMP variable in .DC, .AC, or .TRAN statements. When a .TEMP statement or TEMP variable is not used, the circuit temperature is set by .OPTION TNOM, which defaults to 25°C unless you use .OPTION SPICE, which raises the default to 27°C.
ENFORCE_ PASSIVE	With the ENFORCE_PASSIVE=1 keyword, the S-element checks passivity of all the given frequency sampling points. Once passivity violations are found, the S-element seeks a minimum amount of loss property which restores passivity of all the violated points. Then the S-element adds the loss to all the given frequency points. Note: Passivity enforcement function generates cache data and stores it in a <model_name>.penf file. When the cache file exists, the PrimeSim tool S-element reuses passivity enforcement result without re-enforcement.
FBASE	Base frequency used for transient analysis. The PrimeSim tool uses this value as the base frequency point for IFFT: • If FBASE is not set, the PrimeSim tool uses a reciprocal of the transient period as the base frequency. • If FBASE is set smaller than the reciprocal value of transient period, transient analysis performs circular convolution by using the reciprocal value of FBASE as a base period.
FMAX	Maximum frequency for transient analysis. Used as the maximum frequency point for IFFT.
HIGHPASS	Specifies high-frequency extrapolation: • 0: Use zero in Y dimension (open circuit). • 1: Use highest frequency. • 2: Use linear extrapolation with the highest two points. • 3: (default) Apply window function. • 4: Estimates average derivatives of the phase and magnitude from highest 10% of sampling points. Extrapolation is performed using the highest sampling point and these derivatives. This option overrides EXTRAPOLATION in .MODEL SP.
INTDATTYPE	Data type for the interpolation of the complex data. Is used only with linear and spline interpolation. • RI: Real-imaginary based interpolation • DBA: dB-angle based interpolation • MA: (default) Magnitude-angle based interpolation

Argument	Description
INTERPOLATION	The interpolation method: • STEP: Piecewise step • SPLINE: B-spline curve fit • LINEAR: (default) Piecewise linear • HYBRID: The PrimeSim tool combines different interpolation/extrapolation methods, and switches automatically between them to get the best accuracy. If needed, it also does causality correction down to DC. It is most useful for the S-parameters showing local resonances, and provides the proper interpolation and low-frequency extrapolation method for each entry of the S matrix, which shows different behaviors. For best accuracy, low frequency examples should be provided.
LOWPASS	Method to extrapolate lower frequency points: • 0: Cut off • 1: (default) Make use of the S-matrix at the magnitude of the lowest given frequency point. Set the magnitude value of each entry as the element of DC matrix. The sign of each value is determined by the real part of the extrapolated value at DC point. • 2: Perform linear extrapolation using the magnitude of the lowest two points. • 3: Perform rational function approximation based on low end frequency extrapolation. The LOWPASS option overrides EXTRAPOLATION in .MODEL SP.
MIXEDMODE	Set to 1 if the parameters are represented in the mixed mode.
N	S-model dimension, which is equal to the terminal number of an S-element and excludes the reference node.
PASSIVE	Activates the passive checker to help debug passive models. The default is 0 for the S-element where 0=deactivate and 1=activate. Using the tolerance value specified by the PASSIVE_TOL keyword, the eigenvalues of matrix $(I-S*S')$, $ev[i]$, are checked. • If any frequency point violates $RE(ev[i]) > -(TOL*0.1)$, the PrimeSim tool issues a warning containing a list of violating frequencies with an E flag. Also, the checker verifies potential passivity violations by checking the summation of each S-parameter matrix column. • If $Sum > COLSUM_LIMIT$, the PrimeSim tool issues a warning containing a list of those violating frequencies with a C flag.
PASSIVE_TOL	Tolerance for eigenvalue checking activated by PASSIVE keyword. Default value is 0.01.
PRECFAC	In almost all cases, you do not need to specify a value for this parameter. This parameter specifies the precondition factor keyword used for the precondition process of the S-parameter. A precondition is used to avoid an infinite admittance matrix. The default is 0.75, which is good for most cases.

Argument	Description
RATIONAL_FUNC	0: Perform the same as conventional S-element. FBASE/FMAX-based linear convolution is performed. 1: (default) Perform rational function approximation then recursive convolution; also handles noncausal S-parameters. When RATIONAL_FUNC=1 is used, it is recommended to not specify the DELAYHANDLE keyword. S-element automatically detects and handles the propagation delay. See also .OPTION S_RATIONAL_FUNC in the <i>PrimeSim™ Continuum Reference Manual: Commands and Control Options</i>.
RATIONAL_FUNC_CACHE_DIR	Specifies the directory to write (for reuse) the generated cache files, such as rational function model generation.
RATIONAL_FUNC_REUSE	0: Discard previously extracted rational function data and re-run the rational function approximation. 1: Reuse rational function data if available. The srf/yrf file must have been generated previously using the same version of the PrimeSim tool; otherwise, the srf/yrf file is updated and regenerated. 2: (default) Reuse rational function data if available and make no change in parameter source file (time stamp), FBASE, FMAX, HIGHPASS, LOWPASS, and passivity enforcement configurations; otherwise, rerun the rational function approximation.
RATIONAL_FUNC_UNDER_SAMPLE	Specifies that the rational function takes one sample in every <i>value</i> S-parameter samples. That is, it takes samples #1, #(value+1), #(2*value+1), ... #(k*value+1) of the original S-parameter sample for the rational function. For example, if <i>value</i> is 10, then it takes samples #1, #11, #21, #31, ... of the original S-parameter samples for the rational function.
RFMFILE	Specifies S-element rational function (RFM) file.
S	Specifies that the model type is an S model.
Smodel_name	Name of the S model.
SMOOTH	An integer value to choose one of following methods: 0: (default) No smoothing 1: Mean 2: Median 3: 2nd order polynomial fit 4: 4th order polynomial fit
SMOOTHPTS	An integer value to specify width of the smoothing window on each side of the target point. In total, 2*x + 1 point is taken at each point calculation.

Argument	Description
STAMP	- Y: Conventional admittance based stamp. - S: Scattering parameter based stamp (see Note 1). - YSTS: Admittance parameter based state space stamp (see Note 2). - SSTS: Scattering parameter based state space stamp (see Note 2). - DEEMBED: Produces negated stamp to de-embed given a S-parameter block from the adjacent DUT connected in series. Note 1: Although Y and S stamp types behave mathematically equivalent, when the S-type is selected, the S-element activates a procedure to reduce memory consumption by considering matrices sparseness. Note 2: YSTS and SSTTS stamp methods may be activated when <code>RATIONAL_FUNC=1</code> is used. The state space stamping embeds all the state variables for extracted rational function matrix into the modified nodal analysis (NMA) matrix instead of performing recursive convolution integration. Although this stamping method may incur additional computational cost, since it produces frequency an invariant NMA matrix, it enables time domain steady state analysis (also called .SN analysis) to handle frequency-dependent S-parameter blocks.
TSTONEFILE	Specifies the name of a Touchstone file v 1.0/2.0. Data contains frequency-dependent array of matrixes. Touchstone v1.x files must follow the .s#p file extension rule, where # represents the dimension of the network. You can specify the Touchstone file by specifying the file name within single quotation marks or by using string parameters. Example of using <code>str()</code> to specify a Touchstone file: <pre>.subckt sparam n1 n2 tsfile=str('ss_ts.s2p') S1 n1 n2 0 mname=s_model .model s_model S TSTONEFILE=str(tsfile) .ends x1 A B sparam tsfile=str('ss_ts.s2p') ...</pre> For details, see <i>Touchstone® File Format Specification</i> by the IBIS Open Forum (https://ibis.org/).
TYPE	Parameter type: - S: (default) Scattering - Y: Admittance
XLINELLENGTH	The line length of the transmission line system where the S-parameters are extracted. This keyword is required only when the S Model is used in a W-element.
Z0	Characteristic impedance value of the reference line (frequency-independent). For multi-terminal lines (N>1), the PrimeSim tool assumes that the characteristic impedance matrix of the reference lines are diagonal, and their diagonal values are set to Z0. You also can set a vector value for nonuniform diagonal values. Use Z0 to specify more general types of a reference-line system. The default is 50.

For the PrimeSim SPICE tool, see the `S_IGNORE_PARAM` control option to specify whether to ignore some or all parameters in a PrimeSim SPICE or 3rd-party netlist.

Support for PrimeSim HSPICE S-Parameter and Modeling

The PrimeSim tool supports the PrimeSim HSPICE S-parameter and modeling features as described in the S-parameter Modeling Using the S-element section of the *PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis*, with the following unsupported exceptions:

- De-embedding S-Parameters

The PrimeSim tool supports `sputil`, a standalone utility program that allows you to access S-element features without invoking a simulation and offers additional utility functions such as predicting and verifying the behavior of given S-parameters in advance of simulation runs.

Complete details on `sputil` are available in the S-parameter Standalone Manipulation Utility section of the *PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

Functions for Dependent Sources

Dependent sources make use of the following functions:

- [Linear Function](#)
- [Piece-Wise-Linear \(PWL\) Function](#)
- [Polynomial Function](#)
- [Gate Function](#)

Linear Function

Use a gain factor to describe the relationship between the controlling source and the controlled element.

Syntax

```
(controlled)=gain×val (controlling)
```

Description

The final value is clamped by `MAX` and `MIN` assigned in the instance line. `MAX` and `MIN` should be set simultaneously and `MAX` should be greater than `MIN`. Otherwise, the clamp assignment is ignored.

Piece-Wise-Linear (PWL) Function

Use the pairs of data points (at least two) to describe the relationship between the controlling source and the controlled element.

Syntax

PWL (1)

Description

This value is the keyword to declare the PWL function. The pairs of data points (x_1 y_1 x_2 y_2 x_3 y_3 ... x_i y_i) list the values of the controlling nodes (x_i) and the corresponding value of the controlled node (y_i). The x values must be in increasing order.

Polynomial Function

Use a polynomial function to describe the relationship between the controlling source and the controlled element.

Syntax

POLY (k)

Description

The POLY (k) keyword declares the polynomial function and the coefficients P_0 , P_1 , ..., that make up the polynomial function. K is the dimension of the polynomial function, and can be 1, 2, or 3.

When the input value (current or voltage) I_1, I_2, I_3 is given, the function value F is determined by the following expression:

When $k=1$:

$$F = P_0 + P_1 \cdot I_1 + P_2 \cdot I_2 + P_3 \cdot I_3$$

When $k=2$:

$$F = P_0 + P_1 \cdot I_1 + P_2 \cdot I_2 + P_3 \cdot I_3 + P_4 \cdot I_1^2 + P_5 \cdot I_1 \cdot I_2 + P_6 \cdot I_1 \cdot I_3 + P_7 \cdot I_2^2 + P_8 \cdot I_2 \cdot I_3 + P_9 \cdot I_3^2$$

When $k=3$:

$$F = P_0 + P_1 \cdot I_1 + P_2 \cdot I_2 + P_3 \cdot I_3 + P_4 \cdot I_1^2 + P_5 \cdot I_1 \cdot I_2 + P_6 \cdot I_1 \cdot I_3 + P_7 \cdot I_2^2 + P_8 \cdot I_2 \cdot I_3 + P_9 \cdot I_3^2 + P_{10} \cdot I_1^3 + P_{11} \cdot I_1^2 \cdot I_2 + P_{12} \cdot I_1^2 \cdot I_3 + P_{13} \cdot I_1 \cdot I_2^2 + P_{14} \cdot I_1 \cdot I_2 \cdot I_3 + P_{15} \cdot I_1 \cdot I_3^2$$

Gate Function

Use the gate function to describe the relationship between the controlling source and the controlled element.

Syntax

type (k)

Description

The `type(k)` keyword declares the gate function. `type` can be `AND`, `NAND`, `OR`, or `NOR`, and `k` is the number of inputs. The function is determined by the given PWL function and only one of inputs determines its output. For `AND/NAND`, the smallest value among the inputs is used as the input of the PWL function. For `OR/NOR`, the largest value among the inputs is used as the input of the PWL function.

Transmission Lines

For transmission line, the PrimeSim tool supports both the ideal transmission line T-element and the lossy U-element statement. The ideal transmission line only delays the difference between the signal and the reference. Like SPICE, the PrimeSim tool also uses a U-element to model single and coupled lossy transmission lines for various line structures.

This section describes the following:

- [Lossless Transmission Lines \(T-element\)](#)
- [Lossy Transmission Lines \(U-element\)](#)
- [Transmission Lines \(W-element\)](#)
- [Subcircuit Instances \(X-element\)](#)

Lossless Transmission Lines (T-element)

Specifies lossless transmission lines.

Syntax

```
Txxx in refin out refout Z0=val F=freq <NL=NRMLLEN>  
+ <IC=v1, i1, v2, i2>
```

or

```
Txxx in refin out refout Z0=val TD=val <L=val>  
+ <IC=v1, i1, v2, i2>
```

Arguments

Argument	Description
F	Frequency at which the transmission line has the electrical length given by NL.

Argument	Description
<code>IC=v1,i1,v2,i2</code>	Initial conditions of the transmission line. <code>v1</code> specifies the voltage on the input port. <code>i1</code> is the current into the input port. <code>v2</code> and <code>i2</code> are the voltages on the output port and the current into the output port, respectively.
<code>in</code>	Signal input node.
<code>L</code>	Physical length of the transmission line (in unit <code>m</code>). The default is <code>1</code> .
<code>NL</code>	Normalized electrical length of the transmission line at the frequency <code>F</code> , in units of wavelengths per line length. The default is <code>0.25</code> , which corresponds to a quarter-wavelength.
<code>out</code>	Signal output node.
<code>refin</code>	Ground reference for input signal.
<code>refout</code>	Ground reference for output signal.
<code>TD</code>	Signal delay from the transmission line (in unit <code>sec/m</code>)
<code>Z0</code>	Characteristic impedance of the transmission line.

Description

Nodes `in` and `refin` are the nodes at port 1; `out` and `refout` are the nodes at port 2; and `Z0` is the characteristic impedance. The length of the line may be expressed in either of two forms. The transmission delay, `TD`, may be specified directly (as `TD=10ns`, for example). Or, a frequency `F` may be given, together with `NL`, the normalized electrical length of the transmission line with respect to the wavelength in the line at the frequency `F`. If a frequency is specified but `NL` is not, `0.25` is assumed. That is, the frequency is assumed to be the quarter-wave frequency. Although both forms for expressing the line length are indicated as optional, one of the two must be specified.

This instance models only one propagating mode. If all four nodes are distinct in the actual circuit, then two modes may be excited. To simulate such a situation, two transmission-line instances are required. The initial condition (`IC`) specification consists of the voltage and current at each of the transmission line ports. The initial conditions (if any) apply *only* if the `UIC` option is specified on the `.TRAN` statement.

A lossy transmission line with zero loss may be more accurate than the loss-less transmission line due to implementation details. For example:

```
Td 1 0 2 0 Z0=55 TD=12ns
```

Examples

```
*** Lossless Transmission Line (match) ***
.global dd
```

```
.subckt inv y a
mpa y a dd dd p l=0.5u w=100u ad=2.1p as=2.1p ps=1u pd=1u
mna y a 0 0 n l=0.5u w=50u as=2.1p ad=2.1p ps=1u pd=1u
.ends inv
.model n nmos level=49
.model p pmos level=49
.param dd=3.3
vdd dd 0 dd
vinp inp 0 pulse(0 dd 5n 0.1n 0.1n 5n 10n)
x1 out1 inp inv

*** characteristic impedance of t1 matches rout2 ***
t1 out1 0 out2 0 z0=50
rout2 out2 vx 50
vx vx 0 'dd/2'
.tran 0.1n 30n
.probe v(out1) v(out2) v(inp)
.end
```

This case is a loss-less transmission line with impedance match. The next case shows the results when impedance is mismatched.

```
*** Lossless Transmission Line (not matched) ***
.global dd
.subckt inv y a
mpa y a dd dd p l=0.5u w=100u ad=2.1p as=2.1p ps=1u pd=1u
mna y a 0 0 n l=0.5u w=50u as=2.1p ad=2.1p ps=1u pd=1u
.ends inv
.model n nmos level=49
.model p pmos level=49
.param dd=3.3
vdd dd 0 dd
vinp inp 0 pulse(0 dd 5n 0.1n 0.1n 5n 10n)
x1 out1 inp inv

*** characteristic impedance of t1 does not match rout2 ***
t1 out1 0 out2 0 z0=50 td=10n
rout2 out2 vx 100
vx vx 0 'dd/2'
.tran 0.1n 30n
.probe v(out1) v(out2) v(inp)
.end
```

Lossy Transmission Lines (U-element)

Description

Specifies lossy transmission lines. A lossy transmission line with zero loss may be more accurate than the lossless transmission line due to implementation details. For example:

Example

```
U12 1 0 2 0 model_loss
```

Syntax

```
Uxxx in1 <in2 <...in5>> refin out1 <out2 <...out5>> refout  
+ MNAME L=val <LUMPS=val>
```

Arguments

Argument	Description
F	Frequency at which the transmission line has the electrical length given by NL.
inn	Signal input node for the n th transmission line (in1 is required.).
L	Physical length of the transmission line (in meters).
LUMPS	Number of lumped-parameter sections.
mname	U-model lossy transmission-line model reference name.
NL	Normalized electrical length of the transmission line at the frequency F, in units of wavelengths per line length. The default value is 0.25, which corresponds to a quarter-wavelength.
outn	Signal output node for the n th transmission line (each input port must have a corresponding output port).
refin	Ground reference for input signal.
refout	Ground reference for output signal.

Examples

```
*** Lossy Transmission Line (U-element) ***  
.option post  
  
vin 12 0 pw1 (0 0v 250ps 0v 350ps 2v 10n 2v)  
RG 12 10 50  
RLD1 1 0 50  
C1 10 0 2p
```

```
C2 1 0 2p
Uelem1 3 10 2 0 5 1 4 0 example L=0.178 lumps=1
Uelem2 3 10 2 0 5 1 4 0 example L=1 lumps=100
R2 2 0 50
R3 3 0 50
R4 4 0 50
R5 5 0 50

.model example U Level=3 NL=3 Elev=2 Llev=0 Plev=1 Nlay=2
+ L11=2.311uH L12=0.414uH L22=2.988uH L33=2.813uH
+ Cr1=17.43pF C12=5.41pF Cr2=10.1pF
+ C13=1.08pF C23=5.72pF Cr3=17.67pF
+ R1c=42.5 R2c=41.0 R3c=33.5
+ Gr1=0.44387mS G12=0.1419mS Gr2=0.3671mS
+ G13=23.23uS G23=90uS Gr3=0.38877mS
+ R1s=0.00135 R2s=0.001303 R3s=0.001064

.tran 10ps 20ns
.probe v(*)
.end
```

Transmission Lines (W-element)

The PrimeSim tool supports Transmission Line (“W”) elements, according to the following syntax:

Syntax

```
Wxxx in1 ... inN refin out1 ... outN refout N=val L=val
+ <RLGCMODEL=name or RLGCFILE=name>
```

Arguments

Argument	Description
in1 ... inN	Signal input nodes.
L	Length of a transmission line.
N	Number of signal conductors (excluding the reference conductor).
out1 ... outN	Signal output nodes.
refin	Reference for input signal.
refout	Reference for output signal.
RLGCFILE	Name of the external file with RLGC parameter.
RLGCMODEL	Name of the RLGC model.

Description

This section shows a `RLGCMODEL` example:

Example

```
* RLGCMODEL example matrices (N=2)
.MODEL rlgc_model W MODELTYPE=RLGC N=2
+ Lo=
+ 1.0e-6
+ 5.0e-7 1.2e-6
+ Co=
+ 2.0e-11
+ -4.0e-12 2.1e-11
+ Ro=
+ 30
+ 0 28
+ Go=
+ 5.0e-4
+ -1.0e-4 4.5e-4
.end
RLGCFILE:
* example.rlgc
* N
2
* Lo
1.0e-6
5.0e-7 1.2e-6
* Co
2.0e-11
-4.0e-12 2.1e-11
* Ro
30
0 28
* Go
5.0e-4
-1.0e-4 4.5e-4
```

Where L_o , C_o , R_o , and G_o are DC inductance matrix per unit length, DC capacitance matrix per unit length, DC resistance matrix per unit length, and DC shunt conductance matrix per unit length, respectively.

Skin-Effect in W-elements

The PrimeSim tool supports skin effect resistance matrix R_s for a transmission line (W-element). This matrix can actually be a single line or large matrix. For example, in a RLGC model, you might have R_s defined as follows:

```
:
+ R_s = 1.8066439e-03
```

Or you might have it defined as a matrix:

```
:
+ Rs = 1.8247631e-03
+ _      2.4873219e-06 1.8247631e-03
```

Examples

```
W2 1 2 3 0 4 5 6 0 N=3 L=0.6 RLGCFILE=example.rlgc
W3 1 2 0 4 5 0 N=2 L=0.6 RLGCMODEL=rlgc_model
```

Subcircuit Instances (X-element)

Description

A subcircuit instance is a call statement which instantiates a `.SUBCKT` or `.MACRO` definition. The subcircuit definition, `.SUBCKT`, must be predefined for the creation of a reusable circuit (instance).

A simple method for naming the subcircuit nodes and elements is to prefix them with the subcircuit call name `Xyyyyy`.

Syntax

```
Xyyyyy N1 <N2 N3 ...> subname <param=val ...> <M=val>
```

Arguments

Argument	Description
M	Multiplier. Makes the subcircuit appear as M subcircuits in parallel.
N1,N2,N3,...	The list of input/output signal names of the subcircuit.
param	A parameter name set to a value for use only in the subcircuit.
subname	Subcircuit or macro model reference name.
Xyyyyy	Subcircuit or macro element name, which must begin with an x.

Examples

```
X1 2 4 17 31 MULTI WN = 100 LN = 5
```

This example calls a subcircuit model named `MULTI`. It assigns the parameters `WN=100` and `LN=5` to the parameters `WN` and `LN` given in the `.SUBCKT` statement (not shown). The instance name is `X1`.

Port Elements and IBIS Buffers

This section describes the following:

- [Port Elements \(P-element\)](#)
- [IBIS I/O Buffers \(B-element\)](#)

Port Elements (P-element)

Description

The port element (P-element) identifies the ports used in most analyses and behaves as either a noiseless impedance or a voltage source in series with the port impedance. Each port element requires a unique port number. Each port has an associated system impedance, Z_0 . If you do not explicitly specify the system impedance, the default is 50 ohms. You can use this element as a pure terminating resistance or as a voltage source.

The port element accepts transient waveforms AM, EXP, PULSE, PWL, SFFM, SIN, LFSR and, for signal integrity usage, the PAT source.

Syntax

```
Pxxx N+ N- <PORT=num> <Z0=num> <X=num> <EDGETYPE=LINEAR|HALFSINE>
+ <HBAC hbacmag<,hbacphase>> <SNAC snacmag,<snacphase>>
+ <<DC=>dcval> function <AC=acmag<,acphase>> POWER=0|1|2 <M=val>
+ SIN (VO VA <FREQ <TD <THETA <PHASE>>>> <<PERJITTER=val> <SEED=val>>)
+ SIN2 (VO2 VA2 <FREQ2 <0 0 <PHASE2>>>>)
```

Arguments

Argument	Description
AC	AC voltage or power source keyword.
acmag	Magnitude (RMS) of the AC source, in volts.
acphase	Phase of AC source in degrees. Default is 0.0.
DC=dcval	DC voltage or power source value. Default is 0.0.
EDGETYPE	Type of the rising and falling edges of the waveform. Supported types are: LINEAR (default) or HALFSINE. See Edgetype: Linear vs. Halfsine for more information.

Argument	Description
function	Time-dependent function for transient analysis. The following functions are supported: • Amplitude Modulation (AM) • Exponential (EXP) • PULSE • Piece-Wise-Linear (PWL) • Single-Frequency FM (SFFM) • Sinusoidal (SIN) • Pseudo Random-Bit Generator Source (LFSR) • Pattern Source (PAT)
HBAC	Harmonic Balance AC voltage or power source keyword.
hbmag	Magnitude (RMS) of the HBAC source: If POWER=0, the port is a voltage source and hbmag is in V. If POWER=1, the port is a power source and hbmag is in W. If POWER=2, the port is a power source and hbmag is in dBm.
hbmphase	Phase of HBAC source in degrees. Default is 0.0.
M=val	Multiplier for ports.
N+, N-	Positive and negative nodes, respectively.
PORT=num	The port index. Numbered sequentially beginning with 1 with no shared port indexes.
POWER=0 1 2	Specifies whether the port is considered a voltage or power source. • 0: (default) Port is considered a voltage source measured in V. • 1: Port is considered a power source measured in W. • 2: Port is considered a power source measured in dBm.
Pxxx	Port instance name.
SIN()	If function=sine, SIN uses the following arguments: • VO: Specifies the offset of voltage. • VA: Specifies the amplitude: If POWER=0, the port is a voltage source and VA is in V. If POWER=1, the port is a power source and VA is in W. If POWER=2, the port is a power source and VA is in dBm. • FREQ: Specifies the frequency. Default is 1/TSTOP. • TD: Specifies the delay time. The default is 0. • THETA: Specifies the dampening factor in 1/seconds. Default is 0.0. • PHASE: Specifies the phase delay in degrees. Default is 0.0. • PERJITTER=val: RMS value for period jitter, used to adjust the magnitude of the random time. • SEED=val: Used to generate random number sequences with different seed value. The value is a negative integer, defaults to -1.

Argument	Description
<code>SIN2 ()</code>	If <code>function=sine</code>, <code>SIN2</code> uses the following arguments: <code>VO2</code>: Specifies the offset of voltage. <code>VA2</code>: Specifies the amplitude. - If <code>POWER=0</code>, the port is a voltage source and <code>VA2</code> is in V. - If <code>POWER=1</code>, the port is a power source and <code>VA2</code> is in W. - If <code>POWER=2</code>, the port is a power source and <code>VA2</code> is in dBm. <code>FREQ2</code>: Specifies the frequency. Default is <code>1/TSTOP</code>. <code>PHASE2</code>: Specifies the phase delay in degrees. Default is <code>0.0</code>.
<code>SNAC</code>	Shooting Newton AC voltage or power source keyword.
<code>snacmag</code>	Magnitude (RMS) of the <code>SNAC</code> source: If <code>POWER=0</code>, the port is a voltage source and <code>snacmag</code> is in V. If <code>POWER=1</code>, the port is a power source and <code>snacmag</code> is in W. If <code>POWER=2</code>, the port is a power source and <code>snacmag</code> is in dBm.
<code>snacphase</code>	Phase of <code>SNAC</code> source in degrees. Default is <code>0.0</code> .
<code>X=num</code>	Reference reactance (Ohms). This argument is effective for frequency domain analyses and ignored for time domain analyses. Default is <code>0</code> .
<code>Z0=num (or Zo=num)</code>	Port impedance (Ohms). Sets port characteristic impedance used for <code>.LIN</code> analysis, sets port termination impedance for other analyses, and also sets source impedance when you use the port element as a signal source. Default is <code>50</code> .

IBIS I/O Buffers (B-element)

Description

The general syntax of a B-element card for IBIS I/O buffers is:

Syntax

```
Bxxx node_1 node_2 ... node_N
+ file='filename' model='model_name'
+ keyword_1=value_1 ... [keyword_M=value_M]
```

Arguments

Argument	Description
<code>Bxxx</code>	Buffer name; starts with the letter B followed by up to 1023 alphanumeric and special characters.
<code>file='filename'</code>	Name of the IBIS file.
<code>node_1 node_2 ... node_N</code>	List of I/O buffer external nodes. The number of nodes and their meaning are specific to different buffer types.

Argument	Description
<code>model='model_name'</code>	Name of the model.
<code>keyword_i=value_i</code>	Assigns a value of <code>value_i</code> to the <code>keyword_i</code> keyword. Specify optional keywords in brackets ([]). For more information about IBIS keywords, see Specifying Required and Optional Common Keywords in the <i>PrimeSim™ Continuum User Guide: Elements</i> .

Examples

Example 1

The input buffer name is B1. The four terminal names are `nd_pc`, `nd_gc`, `nd_in` and `nd_out_of_in`. The IBIS model named `IBIS_IN` is in the file named `test.ibs`. The values specified in `IBIS_IN` guide the PrimeSim tool to connect `nd_pc` and `nd_gc` to the voltage sources. Do *not* manually connect voltage sources to these nodes.

```
B1 nd_pc nd_gc nd_in nd_out_of_in
+ file='test.ibs'
+ model='IBIS_IN'
```

Example 2

The output buffer name is B2. The six terminal names are `nd_pu`, `nd_pd`, `nd_out`, `nd_in`, `nd_pc`, and `nd_gc`. Here, `nd_pc` and `nd_gc` are optional terminals. If you do not set these names, the netlist uses the optional terminal names instead. The IBIS model named `IBIS_OUT` in the file named `test.ibs`. The values specified in `IBIS_OUT` guide the PrimeSim tool to connect `nd_pc` and `nd_gc` to the voltage sources. Do *not* manually connect voltage sources to these nodes.

```
B2 nd_pu nd_pd nd_out nd_in [nd_pc nd_gc]
+ file = 'test.ibs'
+ model = 'IBIS_OUT'
```

Example 3

The I/O buffer name is B3. The eight terminal names are `nd_pu`, `nd_pd`, `nd_out`, `nd_in`, `nd_en`, `nd_out_of_in`, `nd_pc`, and `nd_gc`. Here, `nd_pc` and `nd_gc` are optional terminals. If you do not set these names, the netlist uses the optional terminal names instead. The IBIS model named `IBIS_IO` is in the IBIS file named `test.ibs`. The values specified in `IBIS_IO` guide the PrimeSim tool to connect `nd_pc` and `nd_gc` to the voltage sources. Do *not* manually connect voltage sources to these nodes.

```
B3 nd_pu nd_pd nd_out nd_in nd_en nd_out_of_in [nd_pc nd_gc]
+ file = 'test.ibs'
+ model = 'IBIS_IO'
```

For more examples, see the Modeling Input/Output Buffers Using IBIS Files chapter in the *PrimeSim™ Continuum User Guide: Elements*.

Ferroelectric Capacitor (FeCap) Model

Describes the ferroelectric capacitor (FeCap) model, FeCap elements, and FeCap model parameter extraction.

- [Overview of the Ferroelectric Capacitor \(FeCap\) Model](#)
- [FeCap Elements](#)
- [FeCap Model](#)
- [FeCap for Model Parameter Extraction](#)

Overview of the Ferroelectric Capacitor (FeCap) Model

The PrimeSim tool supports the ferroelectric capacitor (FeCap) model. Ferroelectric random access memory (FRAM) is nonvolatile memory featuring fast read/write operation as low power battery-backed SRAM. Using FRAM eliminates the need for a battery.

Ferroelectric material is distinguished from other materials by the P-V hysteresis loop. When voltage is externally applied to a ferroelectric capacitor, sweeping from a large negative to a large positive voltage and back to the original negative voltage, the corresponding polarization (which is directly related to the charge quantify for circuit design purpose) changes from a negative value to a positive value along the lower characteristic curve and then back to the original negative value along an upper characteristic curve.

The hysteresis loop of a ferroelectric film is not fixed. It depends on several factors including:

- Film history
- Peak and frequency of the applied voltage
- Operating temperature

The hysteresis loop corresponding to the maximum peak voltage in one application is called the major hysteresis loop, while the other loops are called the sub-loops (or minor loops).

In the FRAM operation, the changes in the operating point switch among the loops during the transient period according to the switching conditions determined by the following factors:

- Ferroelectric capacitor history
- Current state
- Voltage applied

FeCap Elements

The FeCap element can be handled in the PrimeSim tool as a capacitor with model parameter level=6.

Syntax

```
Caa n1 n2 model_name area=val1 p0=val2 v0=val3
```

Parameters

FeCap parameters are described in [Table 12](#).

Table 12 *FeCap Element Parameters*

Parameter	Description
Caa	Capacitor element name, which must begin with the character c.
n1	Positive node name.
n2	Negative node name.
model_name	Capacitor model name.
area	Area of the ferroelectric capacitor (units in square meter). The default value is 1.0e-12.
p0	Polarization (units in uC/cm ²). The default value is 0.
v0	Initial voltage (units in volt). The default value is 0.

Example

```
c1 1 2 frmC area=3p p0=0 v0=0
```

FeCap Model

The hysteresis loops of the FeCap model from Ramtron are symmetric with respect to the origin point in a P-V plot.

Syntax

```
.model model_name c level=6 <parameter=val1>
```

Parameters

FeCap model parameters are shown in [Table 13](#).

Table 13 *FeCap Model Parameters*

Parameter	Default Value	Description
vmax	5.0	Maximum voltage used in measured data
pmax	30.27	Maximum polarization at vmax in uC/cm ²
kpmax	-3.47e-2	Temperature coefficient of pmax
vsat	2.0	Saturation voltage
vcr	0.5	Minimum voltage for domain switching on unpolarized cap only
as1	2.5000e-1	Curve fitting parameter for a saturated loop
bs1	0.12390e1	Curve fitting parameter for a saturated loop
cs1	0.13483e1	Curve fitting parameter for a saturated loop
as2	5.6726e-2	Curve fitting parameter for a saturated loop
bs2	2.4269e-1	Curve fitting parameter for a saturated loop
cs2	-3.8183e-2	Curve fitting parameter for a saturated loop
ds0	5.0536e-1	Curve fitting parameter for a saturated loop
au1	5.0152e-1	Curve fitting parameter for a unsaturated loop
bu1	1.4908	Curve fitting parameter for a unsaturated loop
cu1	1.0950	Curve fitting parameter for a unsaturated loop
au2	7.4715e-2	Curve fitting parameter for a unsaturated loop
bu2	1.3517e-7	Curve fitting parameter for a unsaturated loop
cu2	-8.2528	Curve fitting parameter for a unsaturated loop

Table 13 *FeCap Model Parameters (Continued)*

Parameter	Default Value	Description
du0	2.1247e-3	Curve fitting parameter for a unsaturated loop
kas1	-1.0107e-3	Temperature coefficient for a saturated loop
kbs1	1.4579e-3	Temperature coefficient for a saturated loop
kcs1	-3.0895e-3	Temperature coefficient for a saturated loop
kas2	4.8937e-4	Temperature coefficient for a saturated loop
kbs2	6.2892e-4	Temperature coefficient for a saturated loop
kcs2	7.2631e-3	Temperature coefficient for a saturated loop
kds0	9.7153e-5	Temperature coefficient for a saturated loop
kau1	-9.5819e-4	Temperature coefficient for an unsaturated loop
kbu1	2.6618e-3	Temperature coefficient for an unsaturated loop
kcu1	-6.1257e-3	Temperature coefficient for an unsaturated loop
kau2	3.7970e-4	Temperature coefficient for an unsaturated loop
kbu2	2.7597e-4	Temperature coefficient for an unsaturated loop
kcu2	1.0525	Temperature coefficient for an unsaturated loop
kdu0	-3.4559e-2	Temperature coefficient for an unsaturated loop

Example

```
.model frm c level=6
```

FeCap for Model Parameter Extraction

There are 14 parameters in the ferroelectric capacitor model, FeCap obtained from curve fitting of a saturated loop or an unsaturated loop as shown in [Table 14](#).

Table 14 *FeCap Model Parameters From Curve-fitting Saturated and Unsaturated Loops*

Saturated Loop	Unsaturated Loop
as1	au1

Table 14 *FeCap Model Parameters From Curve-fitting Saturated and Unsaturated Loops (Continued)*

Saturated Loop	Unsaturated Loop
bs1	bu1
cs1	cu1
as2	au2
bs2	bu2
cs2	cu2
ds0	du0

Each of these parameters has temperature coefficients to estimate its variation with temperature. V_{max} is the maximum voltage in the parameter extraction measurement. Usually, V_{max} is the maximum V_{DD} in the application of the film.

P_{max} is the polarization when V_{max} is applied on the ferroelectric capacitor.

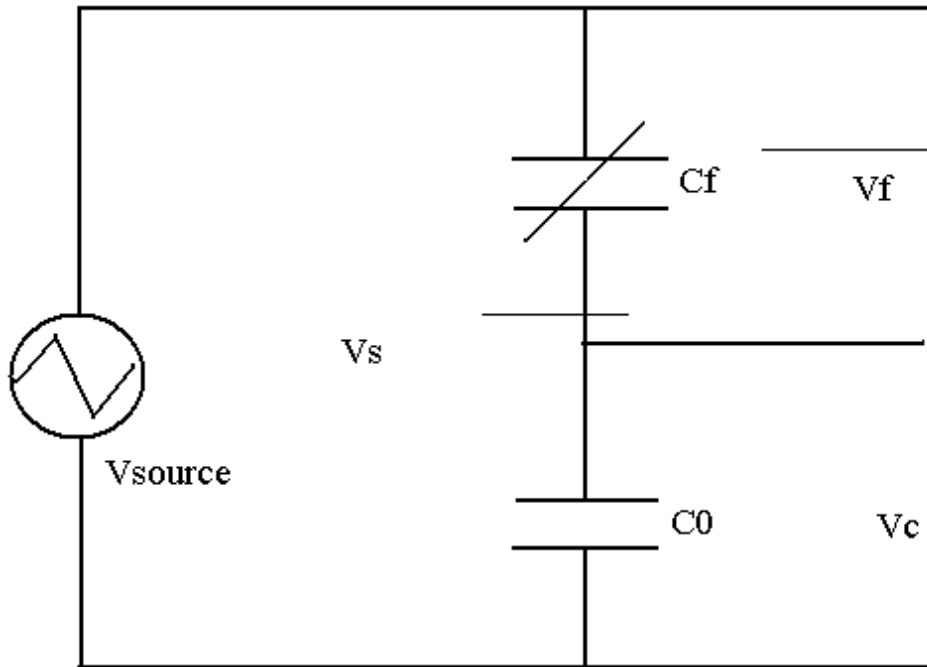
V_{sat} is the criteria voltage to distinguish saturated and unsaturated loops. When a peak voltage applied on the capacitor is higher than V_{sat} , the parameters for saturated loops are used; otherwise, the parameters for unsaturated loops are used. V_{sat} should be larger than the coercive voltage of the film. The coercive voltage is defined as the voltage corresponding to the zero polarization point when the applied voltage sweeps from $-V_{max}$ to V_{max} .

V_{cr} is the criteria voltage for domain switching. When a voltage applied on an unpolarized ferroelectric capacitor is lower than V_{cr} , it is assumed there is no domain switching and the capacitor is a linear capacitor. When the voltage is higher than V_{cr} , domain switching starts and the P-V curve exhibits hysteresis loops. Usually, V_{cr} is about 0.5V for 5V films. V_{cr} should be less than the coercive voltage of the film.

Measuring Ferroelectric Hysteresis Loops

All model parameters are extracted from hysteresis loops that are measured with a Sawyer-tower circuit, as shown in [Figure 7](#).

Figure 7 Sawyer-Tower Circuit for Hysteresis Loop Measurement



In [Figure 7](#):

- V_{source} is a function generator that generates either a sinusoidal wave or a triangular wave. Typically, a lower frequency such as 2 KHz is used.
- C_0 is a high precision linear capacitor.
- C_f is a ferroelectric capacitor.

C_0 is usually chosen to be much larger than C_f so that most of V_s are dropped on C_f and the hysteresis loops can be conveniently monitored with an oscilloscope.

In a ferroelectric film with an area of $2500\mu\text{m}^2$, the maximum value of C_f is about 500pF. We choose C_0 to be about 5000pF. In the measurement, V_s and V_c are sampled and stored. V_f is obtained from $V_s - V_c$.

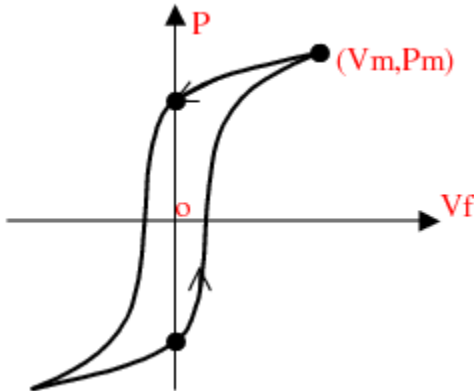
The polarization in the ferroelectric capacitor is calculated using the following equation:

$$P = C_0 \times V_c / A_f$$

where A_f is the area of the ferroelectric capacitor.

If a plot of P vs. V_f , or V_c vs. V_f is conducted, a loop is observed as V_f goes from $-V_{max}$ to V_{max} and then returns to $-V_{max}$. A typical ferroelectric hysteresis loop is shown in [Figure 8](#).

Figure 8 Ferroelectric Hysteresis Loop Example



Two hysteresis loops are used for parameter extraction. One is a saturated loop which corresponds to $V_{max}=V_{DDmax}$, where V_{DDmax} is the maximum source voltage in the application. The other one is an unsaturated loop, which corresponds to $V_{max} @ V_{DD}/2$. Actually, V_{max} should be slightly larger than the coercive voltage of the ferroelectric film in this case. An unsaturated loop is used to obtain parameters, $au1$, $bu1$, $cu1$, $au2$, $bu2$, $cu2$, and $du0$.

For a 5V film, $V_{max}=5.5V$. A saturated loop is used to obtain parameters, $as1$, $bs1$, $cs1$, $as2$, $bs2$, $cs2$ and $ds0$.

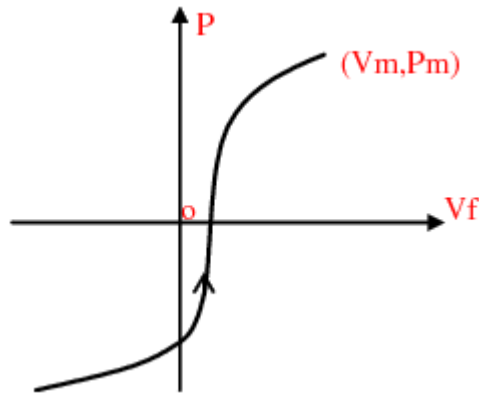
FeCap Model Parameter Extraction

The saturated parameters, $as1$, $bs1$, $cs1$, $as2$, $bs2$, $cs2$ and $ds0$, are extracted from a saturated hysteresis loop.

The steps are as follows:

1. Obtain a saturated hysteresis loop from measurement data.
2. Remove the upper-branch of the hysteresis loop. Since the hysteresis loop is assumed to be symmetric about the origin, only the lower-branch is needed, as shown in [Figure 9](#).

Figure 9 Lower Branch of a Hysteresis Loop

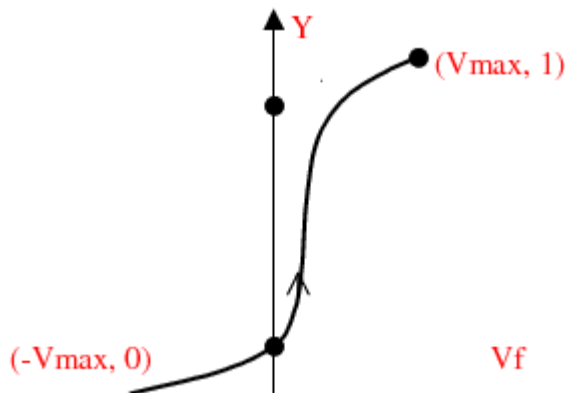


3. Normalize the lower branch of the hysteresis loop, as shown in Figure 10. For each data point (V, P), it is normalized by:

$$Y = (P - P_{\min}) / (P_{\max} - P_{\min})$$

where $P_{\max}$ and $P_{\min}$ are the maximum and the minimum values of P between $-V_{\max}$ and $V_{\max}$.

Figure 10 Normalized Lower Branch of a Hysteresis Loop



4. Curve-fit the normalized lower-branch to the following equation to obtain the parameters, a_1 , b_1 , c_1 , a_2 , b_2 , c_2 and d_0 , which are the values of a_{s1} , b_{s1} , c_{s1} , a_{s2} , b_{s2} , c_{s2} and d_{s0} , respectively.

$$Y = d_0 + (a_1/b_1) \times \arctan(b_1(x-c_1)) + (a_2/b_2) \times \arctan(b_2(x-c_2))$$

The initial values for curve fitting are: $c_1 = V_{\text{sat}}/2$, and $c_2 = 2 \cdot V_{\text{sat}}$.

The steps for extracting the unsaturated parameters `au1`, `bu1`, `cu1`, `au2`, `bu2`, `cu2`, and `du0` are the same, except when an unsaturated hysteresis loop is used.

FeCap Model Temperature Coefficients

Saturated and unsaturated hysteresis loops are measured at different temperatures. The parameters, `Pmax`, `as1`, `bs1`, `cs1`, `as2`, `bs2`, `cs2`, `ds0`, `au1`, `bu1`, `cu1`, `au2`, `bu2`, `cu2`, and `du0`, are extracted for each temperature point. Then, temperature coefficients for each of these parameters are obtained by linear fitting.

The temperature coefficient `kas1` of `as1` is obtained by curve-fitting the following equation:

$$as1@T = as1@25C + kas1 \times (T - 25)$$

The temperature coefficient for `Pmax` is obtained from the saturated `Pmax` vs. Temperature measurements.

7

C-Modeling

This chapter describes the C-modeling capabilities of the PrimeSim Pro tool.

C-based models have the advantage of the flexibility of the C programming language, whereas dedicated modeling languages, such as Verilog or Verilog-A, are mostly restricted to dedicated functions defined within the boundaries of their associated IEEE standards. Each has their place in the design space.

However, C-models can be written to model both analog and digital behavior simultaneously, whereas digital HDL Verilog models cannot model analog behavior and Verilog-A models cannot model digital behavior in the pure digital domain. In some cases, performance can be greatly increased. In others, the C-model approach can allow simulation of regular array structures that are not possible with any other modeling approach.

This chapter covers the basic C-model structure as well as a list of the C-based functions available within the PrimeSim Pro tool.

- [Required Steps to Use a C-Model](#)
- [Main C-Model Structure](#)
- [C-Model Related PrimeSim Options](#)
- [PrimeSim Pro Common C-Functions for C-Models](#)
- [Error Codes](#)
- [Standard Practices](#)
- [Debugging C-Models](#)
- [PrimeSim Pro FSC C-Model Functions](#)

Required Steps to Use a C-Model

The following steps must be done before the PrimeSim Pro tool can execute a simulation with a C-model:

1. Compile the model to create the shared object `model.so` file.
2. Reference the compiled model files in the netlist.
3. Set the supply voltage level for digital translation in the netlist.
4. Instantiate the model in the netlist.
5. Run the simulation.

Compiling the model is usually done with the standard “gcc” compiler provided with all Linux OS installations. The `build_c_model` utility provided with the PrimeSim Pro tool is used to do the compilation.

```
% build_c_model model.c
```

This creates a `model.so` file that the PrimeSim Pro tool can then read in. To do so, a command must be provided in the netlist. Then the supply voltage for the model must be specified as in the example below. If a given instance of the same model were to be placed in a different supply voltage domain, then other C-functions are provided to change the voltage levels corresponding to the required digital values. This is covered later in this chapter.

```
.option primesim_c_model = "model.so"  
.option primesim_fsc_vdd = 1.8
```

The model must be instantiated in the netlist somewhere. When a model is provided and there is a transistor-level subcircuit with the same name, the instances with the same name as defined in the model C-code are replaced with the C-model. It is assumed that if the `primesim_c_model` option is present in the netlist the intent is that instances with the model name are to be replaced with the C-model equivalent. An extra command to tell the simulator to do the substitution is not necessary.

Running the simulation then just requires executing the appropriate command line normally used to run the PrimeSim Pro tool such as:

```
% primesim input.sp
```

Multiple C-model files can be compiled and included separately as needed. They do not need to be all placed in a single C-code file. It is only necessary to use additional netlist option lines to include the other compiled object files. This may be different than other C-model approaches used with other FastSPICE simulation tools.

```
.option primesim_c_model = "model1.so"  
.option primesim_c_model = "model2.so"
```

It is also not necessary to have an empty subcircuit definition as a placeholder in the netlist, though it might be a good idea so that the port order is guaranteed to match the real transistor-level subcircuit when the subcircuit is eventually included. The empty subcircuit should be generated from a schematic netlist to ensure consistency of port order matching later in the design process.

Main C-Model Structure

There are four main parts of a typical C-model file:

1. Header file includes
2. Structure definition, if needed
3. Evaluation function
4. Model definition

Below is a simple NAND gate example of the basic C-model structure. First is the header file `include` statements, then the evaluation function followed by the model definition function where the model name and the port names are defined:

```
#include <stdio.h>
#include "fsc.h"
// Evaluation function; It is called whenever there is an event on an
// input.
static void nand_eval()
{
    fsc_digital_value_t a, b, y;

    a = fsc_get_port_value("A");
    b = fsc_get_port_value("B");
    y = !(a&&b);
    fsc_set_port_value("Y", y);
    fsc_mesg("DEBUG: Setting value to %d\n", y);
}

void fsc_define_models()
{
    // Define the model
    fsc_model_t* m = fsc_define_model("nand", nand_eval);

    // Define the ports. The port order is determined by the subckt
    // definition.
    fsc_define_port(m, "A", FSC_DIGITAL, FSC_INPUT);
    fsc_define_port(m, "B", FSC_DIGITAL, FSC_INPUT);
    fsc_define_port(m, "Y", FSC_DIGITAL, FSC_OUTPUT);
}
```

The above example consists of only three sections, since a structure for locally stored variables was not required. The next example shows a register model with all four sections:

```
#include <stdio.h>
#include "fsc.h"
typedef struct d_register_state_t {
    fsc_digital_value_t prev_clk;
} d_register_state_t;

// Evaluation function
static void d_register_eval()
{
    fsc_digital_value_t clk, d[8];
    d_register_state_t* state = fsc_get_state();
    clk = fsc_get_port_value("CLK");

    // On a rising clock set the value of q to d.
    if (state->prev_clk == FSC_ZERO && clk == FSC_ONE) {
        fsc_get_vector_port_values("D", d);
        fsc_set_vector_port_values("Q", d);
    }
    // Save the previous clock value
    state->prev_clk = clk;
}

void fsc_define_models()
{
    // Define the model
    fsc_model_t* m = fsc_define_model("d_register", d_register_eval);
    // Define the ports
    fsc_define_port(m, "CLK", FSC_DIGITAL, FSC_INPUT);
    fsc_define_vector_port(m, "D", 7, 0, FSC_DIGITAL, FSC_INPUT);
    fsc_define_vector_port(m, "Q", 7, 0, FSC_DIGITAL, FSC_OUTPUT);
    fsc_define_port(m, "PREV_CLK", FSC_DIGITAL, FSC_STATE);
    // Allocate memory for state structure
    fsc_alloc_state(m, sizeof(d_register_state_t));
}
```

The extra structure definition is used to store local variables that are unique to each instance of the model. This is necessary if there are multiple instances of the same model because each instance likely has different values for those local variables. One instance cannot be allowed to overwrite the local stored variables of another instance. Memory for the structure is allocated in the model definition section as shown at the end of the above example.

An example of such a local stored variable is the last state of a clock signal that is used to determine if there is a rising edge or a falling edge on the clock to trigger evaluation of a section of the model code. To do this it is necessary to store a short history of the clock state for later recall to test for the desired condition. If the last clock state was low and the

present clock state is high then a rising edge has occurred. Such variables can be integer for digital checks or floating point types for analog voltage or time checks.

C-Model Related PrimeSim Options

The following table describes related C-Model options.

Option	Description
<code>primesim_c_model</code>	Inputs the C-model <code>.so</code> files into the PrimeSim tool.
<code>primesim_fsc_auto_detect</code>	Automatically detects vdd instead of using <code>primesim_vdd</code> .
<code>primesim_c_model_exclude</code>	Excludes some instances of C-models from being replaced with the model that was created.
<code>primesim_fsc_vdd</code>	Specifies the supply voltage for the C-model interface.

PrimeSim Pro Common C-Functions for C-Models

All PrimeSim Pro C-functions begin with `fsc_` to identify them as specific and unique to PrimeSim Pro C-models. These functions are defined in a header file called `fsc.h` that must be put at the top of every C-model file. Other header files may be needed, such as `stdio.h` and `math.h`, which are standard C-code header files. Typically, the `stdio.h` header file is always required. It is up to you to determine if additional header files are needed.

Header Files

```
include "fsc.h"
```

This line must appear at the top of the file to define the core C-functions.

State Structure Definitions

```
typedef struct model_state_t {  
}model_state_t;
```

Evaluation Functions

```
static void model_eval() { }
```

where `model_eval` name can be anything.

Model Definition Functions

```
void fsc_define_models ()
{
    fsc_model_t* m=fsc_define_model("subckt_name", eval_function_name);
}
```

Port Definition Functions

```
fsc_define_port(m, "name", type, direction);
fsc_define_vector_port(m, "name", msb, lsb, type, direction);
fsc_define_variable_vector_port(m, "name", "width_param", type,
    direction);
```

State Structure Memory Allocation

```
fsc_alloc_state(m, sizeof(model_state_t) );
```

where `model_state_t` must match previously defined structure name.

Simulation Phases

```
FSC_START
FSC_DC
FSC_TRAN
FSC_END
```

Usage

```
if(fsc_get_phase()==FSC_TRAN) { //Then do this }
```

Hash Table Functions for Compact Memory or Array Storage

```
fsc_define_table(fsc_model_t* model, const char* name, unsigned key_size,
    unsigned value_size);
fsc_get_table_id(const char* name);
fsc_set_table_value(const char* name, void* key, void* value);
fsc_set_table_value_by_id(fsc_table_id_t* id, void* key, void* value);
fsc_get_table_value(const char* name, void* key);
fsc_get_table_value_by_id(fsc_table_id_t* id, void* key);
```

Port Access Functions

```
fsc_get_port_value("name");
fsc_set_port_value("name", value);
fsc_get_vector_port_values("D", array);
```

```
fsc_set_vector_port_values("Q", array);  
fsc_get_port_voltage("name");  
fsc_set_port_voltage("name", value);  
fsc_get_vector_port_voltages("name", array.);
```

Self-Generated Events

```
fsc_wakeup(time);
```

where time units are in seconds.

Simulation Time

```
t = fsc_current_time();
```

where time units are in seconds.

Port Types

```
FSC_DIGITAL  
FSC_ANALOG
```

Port Directions

FSC_INPUT – Input port

FSC_OUTPUT – Output port

FSC_BIDI – Bidirectional port

FSC_STATE – State port for a local variable storage structure

Digital Values

FSC_ZERO – Digital zero [0]

FSC_ONE – Digital one [1]

FSC_X – Digital unknown [2]

Port Properties

Property	Description	Default
FSC_CAPACITANCE	Port capacitance.	Analog/BIDI: 100ff Input/Dig Output: 0
FSC_DRIVE_RESISTANCE	Drive resistance to port.	1k ohm
FSC_ABSTOL	Absolute tolerance (<code>primesim_abstol</code>). Controls analog event triggering.	
FSC_RELTOL	Relative tolerance (<code>primesim_reltol</code>). Controls analog event triggering.	
FSC_VIL	Digital logic zero threshold.	VDD/2
FSC_VIH	Digital logic one threshold.	VDD/2
FSC_VOL	Digital logic zero voltage.	0 volts
FSC_VOH	Digital logic one voltage.	VDD
FSC_DELAY	Intrinsic delay from input to output.	0
FSC_RISE_DELAY	Intrinsic delay for a rising output.	0
FSC_FALL_DELAY	Intrinsic delay for a falling output.	0
FSC_TRANSITION_TIME	Transition time.	<code>primesim_tunit</code>
FSC_RISE_TIME	Rising transition time.	<code>primesim_tunit</code>
FSC_FALL_TIME	Falling transition time.	<code>primesim_tunit</code>

Usage

```
fsc_set_port_property("name", FSC_DRIVE_RESISTANCE, 1.0e4);
fsc_set_port_property("name", FSC_VOH, 2.2);
fsc_set_port_property("name", FSC_VIH, 2.0);
fsc_set_port_property("name", FSC_VIL, 0.2);
fsc_set_port_property("name", FSC_TRANSITION_TIME, 50e-12);
```

Error Codes

If there is an error found in a C-model during runtime after a clean compile of the model, the PrimeSim Pro tool reports an error code. Below is a list of error codes:

- `FSC_NOT_IMPLEMENTED` – This function is not yet implemented.
- `FSC_INVALID_DIRECTION` – This operation is invalid due to its port direction.
- `FSC_INVALID_TYPE` – This operation is invalid due to its port type.
- `FSC_INVALID_ARG` – An argument is invalid, usually `NULL`.
- `FSC_INTERNAL_ERROR` – Other errors in the PrimeSim Pro tool.

Standard Practices

C-code functions that are pre-defined in the `fsc.h` header file all begin with the `fsc_` identifier. All of the `fsc_` function definitions can be found in [PrimeSim Pro FSC C-Model Functions](#).

Temporary Local Variable Storage

One recommended approach in coding C-models is to use the state pointer method to access and save local state variables such as last clock states or any other variable that needs temporary storage.

Examples:

```
if(state->prev_clk==0 && clk==1) { // rising edge of clk }
```

or:

```
if(state->prev_clk==FSC_ZERO && clk==FSC_ONE ) { // rising edge of clk }
```

The `state->prev_clk` action checks the `prev_clk` stored state to see if it equals 0. This is a handy shorthand to simplify coding to check state variable values. When it is necessary to write a new value, such as the current clock state into the state variable, it would look like this:

```
state->prev_clk = clk;
```

To copy the state variable into another variable, use the following:

```
new_clk = state->prev_clk;
```

When defining port types, try to stick with defining `FSC_INPUT` and `FSC_OUTPUT` ports. Bidirectional `FSC_BIDI` ports are rarely needed. If true bidirectional ports are required,

such as for a bidirectional data bus, the active direction must be set before using. The `fsc_enable_port()` function activates output direction. The `fsc_disable_port()` function changes it to an input. An `FSC_BIDI` port does not function as an input unless disabled as an output. By default, an `FSC_BIDI` port is enabled as an output.

Defining vector ports requires paying attention to the port order and the MSB and LSB definitions.

Examples

```
fsc_define_vector_port(m, "IN", 7, 0, FSC_DIGITAL, FSC_INPUT);  
fsc_define_vector_port(m, "IN", 0, 7, FSC_DIGITAL, FSC_INPUT);
```

Note that the MSB and LSB locations can be swapped, which would reverse the port order. The vector port description does not imply a specific port naming convention (for example, `IN[0]`, `IN[1]`, ...) and only defines whether the LSB to MSB port ordering is left-to-right or right-to-left. The actual subcircuit port names in the netlist can be anything.

Accessing Ports by Port ID

There are two ways to access ports in a model. One is by name, which makes for more readable code, and the other is by port ID. The “by_id” method permits faster access because the ID does not have to be looked up first. However, it requires extra programming steps to get the port ID and use the alternate “by_id” versions of the port access functions.

For models that are small or do not see a lot of activity during simulation, the access by name approach is fine. For high activity models or large footprint models, the “by_id” method may be a good one to consider:

Examples

The following example defines the pointer for `A_id`:

```
fsc_id_t* A_id;
```

The following example gets the ID for port `A`:

```
A_id = fsc_get_port_id("A");
```

The following example copies the port value using `A_id` into variable `A`:

```
A = fsc_get_port_value_by_id(A_id);
```

Name Conflicts

Name conflicts between local functions and identically named global functions can lead to crashes or incorrect behaviors which are hard to pinpoint. It is therefore recommended that all functions local to the C-model file be declared `static`.

Another useful practice is to choose function names which attempt to be unique, such as by prefixing the company or project name.

Examples

```
int open(const char *name, int flags);
```

The preceding example might become:

```
static int open(const char *name, int flags);
```

or:

```
int company_or_project_open(const char *name, int flags);
```

Debugging C-Models

Debugging C-models is normally done by using embedded print commands to print out variable values or other information to determine if the code is performing the intended function. The standard C `printf` function can be used. The PrimeSim tool also provides the `fsc_msg` function that prints to the screen and the log file. The formatting syntax for the `fsc_msg` function is the same as for the standard `printf` command. The following are some examples for printing floating point and integer variables.

Examples:

```
fsc_msg("time=%f \n", fsc_current_time()*1e9);  
fsc_msg("Q=%d \n", fsc_get_port_value("q"));  
printf("time=%f \n", fsc_current_time()*1e9);  
printf("Q=%d \n", fsc_get_port_value("q"));
```

The preceding functions print to the screen as well as the log file, allowing inspection of the results after a simulation has completed.

PrimeSim Pro FSC C-Model Functions

The header file defining all FSC C-modeling functions must be included at the top of all C-model files that are separately compiled.

```
fsc.h  
fsc_get_instance_name();
```

Example:

```
fsc_msg("Instance = %s\n", fsc_get_instance_name());  
fsc_get_port_value("name");
```

Example:

```
fsc_get_port_value("A");  
fsc_get_port_value_by_id(id);  
fsc_set_port_value("name", value);
```

Example:

```
fsc_set_port_value("A", 0);  
fsc_set_port_value("A", 1);  
fsc_set_port_value_by_id(id);
```

Example:

```
id_A = fsc_get_port_id("A");  
fsc_set_port_value_by_id(id_A, 1);  
fsc_get_vector_port_values("D", array);  
fsc_get_vector_port_values_by_id(id);  
fsc_set_vector_port_values("Q", array);  
fsc_set_vector_port_values_by_id(id);  
fsc_get_port_voltage("name");
```

Example:

```
in1 = fsc_get_port_voltage("A");  
fsc_get_port_voltage_by_id(id);  
fsc_set_port_voltage("name", value);
```

Example:

```
fsc_set_port_voltage("A", vdd);  
fsc_set_port_voltage("A", 1.2);  
fsc_set_port_voltage_by_id(id, value);
```

Example:

```
id_A = fsc_get_port_id("A");  
fsc_set_port_voltage_by_id(id_A, 1.2);  
fsc_get_vector_port_voltages("name", array);  
fsc_get_vector_port_voltages_by_id(id);  
fsc_get_port_id("name");
```

Example:

```
IN_id = fsc_get_port_id("IN");  
fsc_define_port(m, "name", type, direction);
```

Example:

```
fsc_define_port(m, "IN", FSC_DIGITAL, FSC_INPUT);  
fsc_define_vector_port(m, "name", lsb, msb, type, direction);  
fsc_define_variable_vector_port(m, "name", "width_param", type,  
direction);
```

Example:

```
fsc_define_vector_port(m, "ADDR", 0, 15, FSC_DIGITAL, FSC_INPUT );  
fsc_set_port_property("name", property, value);
```

Example:

```
fsc_set_port_property("IN", FSC_RISE_TIME, 0.1e-9 );  
fsc_enable_port("name");
```

Example:

```
fsc_enable_port("IN");  
fsc_enable_port_by_id(id);
```

Example:

```
OUT_id = fsc_get_port_id("OUT");  
fsc_enable_port_by_id(OUT_id);  
fsc_disable_port("name");
```

Example:

```
fsc_disable_port("IN");  
fsc_disable_port_by_id(id);
```

Example:

```
OUT_id = fsc_get_port_id("OUT");  
fsc_disable_port_by_id(OUT_id);  
fsc_disable_all_ports(port_type, port_direction);
```

Example:

```
fsc_disable_all_ports(FSC_INPUT, FSC_DIGITAL);  
fsc_activate_port("name");
```

Example:

```
fsc_activate_port("OUT");  
fsc_activate_port_by_id(id);
```

Example:

```
OUT_id = fsc_get_port_id("OUT");  
fsc_activate_port_by_id(OUT_id);  
fsc_deactivate_port("name");
```

Example:

```
fsc_deactivate_port("OUT");  
fsc_deactivate_port_by_id(id);
```


Example:

```
OUT_id = fsc_get_port_id("OUT");  
fsc_deactivate_port_by_id(OUT_id);  
fsc_port_changed("name");
```

Example:

```
fsc_port_changed("IN");  
fsc_port_changed_by_id(id);
```

Example:

```
IN_id = fsc_get_port_id("IN");  
fsc_port_changed_by_id(IN_id);  
fsc_vector_port_changed("name", start, end);
```

Example:

```
fsc_vector_port_changed("IN");  
fsc_vector_port_changed_by_id(id, start, end);
```

Example:

```
IN_id = fsc_get_port_id("IN");  
fsc_vector_port_changed_by_id(IN_id);  
fsc_mesg("Text %format\n", variable_name);
```

Example:

```
fsc_mesg("IN = %d", in);  
fsc_get_instance_name();
```

Example:

```
fsc_mesg("Instance Name=%s \n", fsc_get_instance_name() );  
fsc_get_prefix_name();
```

Example:

```
fsc_mesg("prefix Name=%s \n", fsc_get_prefix_name() );  
fsc_define_models();
```

Example:

```
void fsc_define_models() {--Definitions--}  
fsc_define_model("subckt_name", eval_function_name);
```

Example:

```
fsc_model_t* m = fsc_define_model("nand", nand_eval );  
fsc_alloc_state(m, sizeof(structure_name));
```

Example:

```
fsc_alloc_state(m, sizeof(rom_state_t));  
fsc_get_phase();
```

Example:

```
if(fsc_get_phase == FSC_TRAN){--execute transient code--}  
fsc_wakeup(time);
```

Example:

```
fsc_wakeup(fsc_current_time() + tstep);  
fsc_current_time();
```

Example:

```
time = fsc_current_time();  
fsc_get_state();
```

Example:

This function is typically replaced by this form:
x = state->local_variable, or this: state->local_variable = y;
fsc_get_string_param_value("name", value);

Example:

```
const char* file=fsc_get_string_param_value("init_file", NULL);  
fsc_get_vdd();
```

Example:

```
vdd = fsc_get_vdd();
```

The above example gets the VDD value specified by:

```
.option primesim_fsc_vdd = 1.20 in the netlist
```

8

Verilog-A Support

This chapter describes support for Verilog-A models.

Verilog-A derives from the IEEE 1364 Verilog Hardware Description Language (HDL) specification for describing behavior in analog systems. The Verilog-A language that the PrimeSim tool supports is compliant with *Verilog-AMS Language Reference Manual, Version 2.2*.

The Verilog-A implementation in the PrimeSim tool supports a mixed design of Verilog-A descriptions and transistor-level SPICE netlists with a simple use model. Verilog-A supports most analysis features available in the PrimeSim tool for Verilog-A based devices, including AC, DC, transient analysis, statistical analysis, and optimization.

For more information on selecting which Verilog-A engine to use, see the PrimeSim option, [primesim_pva_flow](#).

For more detailed information on Verilog-A support, see the Using Verilog-A chapter in the *PrimeSim™ Continuum Overview*.

This chapter covers the following topics:

- [Support for Encrypted Verilog-A Files as Input](#)
- [Support for LRM 2.4 Syntax and Features](#)

Support for Encrypted Verilog-A Files as Input

The PrimeSim tool supports reading metaencrypted Verilog-A files as input for the `.hdl` command. This allows the user to encrypt Verilog-A models using the `metaencrypt` utility and use them in PrimeSim simulation. This feature is supported for SPICE netlist format.

Support for LRM 2.4 Syntax and Features

The PrimeSim tool supports Language Reference Manual (LRM) 2.4 features on a case-by-case basis. If you require additional LRM 2.4 feature support, contact the Synopsys Technical Support Center.

You can:

- Go to <https://solvnetplus.synopsys.com> (Synopsys user name and password required).
- Send an e-mail message to your local support center:
 - E-mail support_center@synopsys.com from within North America.
 - Find other local support center e-mail addresses at <https://www.synopsys.com/support/global-support-centers.html>.

Note:

The PrimeSim tool does not support LRM versions prior to 2.2.

Part 4: Options and Commands

Describes options and controls for the PrimeSim tool.

This Part presents the following chapters:

- [Chapter 9, PrimeSim Options](#)
- [Chapter 10, PrimeSim HSPICE-Compatible Commands and Control Options](#)
- [Chapter 11, PrimeSim SPICE Commands and Control Options](#)

9

PrimeSim Options

This chapter describes how to use the PrimeSim simulation controls, and most importantly, its various simulation mode options.

This chapter covers the following topics:

- [Using PrimeSim Options](#)
- [PrimeSim Options \(by Category\)](#)
- [PrimeSim Option Reference \(Alphabetical List\)](#)
- [PrimeSim Commands](#)

Using PrimeSim Options

The PrimeSim tool is a unified simulation engine with Pro and SPICE modes. These modes can be applied to different design types based on accuracy, performance and capacity requirements. Each mode handles a design differently but all start by reading in a transistor level netlist and SPICE models.

The PrimeSim Pro tool analyzes the sea of transistors and categorizes them into different Channel Connected Components (CCC) groups, called partitions, while SPICE mode puts all transistors into one partition. For each partition, the PrimeSim Pro tool builds and solves a matrix as any other SPICE simulator does. The electrical communication from partition to partition is facilitated through an event-driven procedure that controls the simulation calculation process from primary inputs to primary outputs. If the event control signal at a partition boundary is not crossing the predefined threshold, the partition driven by this signal is not activated and no calculation is performed to save simulation time. By employing this event-driven technique for partitions, the PrimeSim Pro tool can run up to 1000X faster than SPICE mode, which usually takes much longer to solve its single matrix.

The PrimeSim Pro and PrimeSim SPICE tools provide a rich set of options to control simulation accuracy and performance. For example, a MOSFET transistor can be modeled at different levels of detail to represent its electrical characteristics. There are also various numerical integration methods and error tolerances that help you meet your simulation needs. For designs with repeated structures such as memories, the PrimeSim Pro tool

automatically applies its adaptive hierarchical database to speed up simulation and maintain high accuracy. This chapter presents basic options that control the accuracy, performance, and capacity in a PrimeSim Pro simulation. If they conflict with options in the native SPICE deck from other SPICE simulators, PrimeSim options take precedence. For example, the *accurate* option in other simulators is replaced by a combination of PrimeSim options that address accuracy considerations differently.

General Control Options

When specifying general control options, you can use wildcards for subcircuit names that appear in any PrimeSim option. For example, in the following command, all nand cells and aoi cells are in `model=2` while the rest of the design is in `model=4`.

```
.option primesim_model="4 nand*:2 aoi*:2"
```

You can specify all global default options in the `primesim.cfg` file. The configuration file is a collection of commonly used PrimeSim options. This file serves as a central location for maintaining and managing default values. You can find the `primesim.cfg` file in the installation directory. Upon installation, it is an empty file. It is your responsibility to create and maintain this file. Because these options are used for any PrimeSim simulation, you must add options to this global configuration file with extreme care.

When the `$PRIMESIM_HOME` environment variable sets the directory (that is, the `primesim.cfg` file is properly located within the installation directory), the PrimeSim tool reads `primesim.cfg` and changes all PrimeSim Pro options as specified in this file.

You can use `primesim -h` to get a list of the PrimeSim controls and options that can be used on the command line.

Note:

The PrimeSim tool supports user environment variables and the UNIX home tilde (~) character. However, the dollar sign (\$) is also a comment character in the SPICE netlist format. So, a single quotation mark is required when you use the \$ character. Consider the following examples:

```
.inc '$process/model.inc'  
.inc ~/device/process/model.inc
```

Note:

The PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE modes also rely on this file. You must use extreme care when adding PrimeSim Pro options to this file.

You can specify PrimeSim options using the `primesim.cfg` file or in the input SPICE deck or in any one of the included files. Options specified locally always override any options in the `primesim.cfg` file.

The PrimeSim `.log` file prints out all options in `primesim.cfg` but separates them from those options defined in the input SPICE deck or included files.

A sample PrimeSim Pro configuration file might appear as follows:

```
.option primesim_fcamin=0.1f
.option primesim_resmin=0.01
.option primesim_cutnode=1000
.option primesim_spred=2
```

Using primesim.cfg to Define Common PrimeSim Options

The PrimeSim tool has extended support for the `primesim.cfg` file. The PrimeSim tool searches within four directories to find this file. The directory names and their priorities in searching are as follows:

1. `$PRIMESIM_HOME` – Installation directory
2. `$PRIMESIM_PROJ_CFG` – Project directory
3. `$HOME` – Home directory
4. The directory of the input file

The search is performed in the order listed, with the input file directory being the highest priority. The highest priority `primesim.cfg` file can incrementally add the options not yet set and also override the options set in the lower priority `primesim.cfg` files. For example, if no `$PRIMESIM_PROJ_CFG` variable is defined, the search for `primesim.cfg` is initiated in the order `$PRIMESIM_HOME`, `$HOME`, and then the input file directory. If a `primesim.cfg` file exists in the current working directory and contains options already specified in a lower-level `primesim.cfg` file, the options in the input file directory's `primesim.cfg` file take precedence over any `primesim.cfg` instances in `$PRIMESIM_HOME` or `$HOME`. Any unique options defined in `$PRIMESIM_HOME` or `$HOME` and not defined in the `primesim.cfg` file in the input file directory are retained.

Note:

Any PrimeSim option defined in the simulation deck always overrides the same option in any `primesim.cfg` file.

Environment Variables

The following is a list of environment variables used by the PrimeSim tool.

Table 15 PrimeSim Environment Variables

Variable	Description
<code>PRIMESIM_VA2C_SO_DIR</code>	The directory to output the <code>.so</code> files for Verilog-A compilation.
<code>PRIMESIM_PROJ_CFG</code>	Secondary locations to look for the <code>primesim.cfg</code> file.

Table 15 PrimeSim Environment Variables (Continued)

Variable	Description
PRIMESIM_USE_CONFIG=[0 1 2 3 4 5]	0: The PrimeSim tool does not read any global configuration file. 1: (default) The PrimeSim tool reads (in order): <code>\${PRIMESIM_HOME}/primesim.cfg,</code> <code>\${PRIMESIM_PROJ_CFG}/primesim.cfg,</code> <code>\${HOME}/primesim.cfg,</code> <directory of top-level netlist if not <code>cwd>/primesim.cfg,</code> <code>./primesim.cfg</code> 2: The PrimeSim tool reads (in order): <code>\${PRIMESIM_PROJ_CFG}/primesim.cfg,</code> <code>\${HOME}/primesim.cfg,</code> <directory of top-level netlist if not <code>cwd>/primesim.cfg,</code> <code>./primesim.cfg</code> 3: The PrimeSim tool reads (in order): <code>\${HOME}/primesim.cfg,</code> <directory of top-level netlist if not <code>cwd>/primesim.cfg,</code> <code>./primesim.cfg</code> 4: The PrimeSim tool reads (in order): <directory of top-level netlist if not <code>cwd>/primesim.cfg,</code> <code>./primesim.cfg</code> 5: The PrimeSim tool reads: <code>./primesim.cfg</code>
SNPS_CONTAINER	Set this variable to 1 to enable running the PrimeSim tool in a container. A container bundles applications and their system dependencies so the applications can run on any host machine regardless of which packages are installed on the host. This is especially useful for running applications in cloud environments. Note: Alternatively, you can invoke the PrimeSim tool using the <code>-container</code> command-line option, which is the preferred method.

PrimeSim Options (by Category)

The PrimeSim tool provides the following categories of configuration options:

- [General Options](#)
- [DC Initialization Options](#)
- [Output Reporting Options](#)
- [Partitioning Options](#)
- [Accuracy and Speed Options](#)
- [Back-Annotation Options](#)
- [PrimeSim SPICE Accuracy and Speed Options \(-runlvl and .option runlvl\)](#)

General Options

The PrimeSim tool provides the following options.

Table 16 *General Options*

Option	Description
<code>primesim_ac_at_diff_t</code>	Controls whether the starting point of an AC analysis is a user-specified "non-zero" transient analysis timepoint.
<code>primesim_ac_flow</code>	Controls whether small signal AC analyses of 3rd-party netlists are run in a PrimeSim HSPICE-compatible flow or in a 3rd-party compatible flow (default). This option is ignored in small signal AC analyses of PrimeSim HSPICE-compatible netlists.
<code>primesim_acc_sparse_fcap</code>	Specifies whether to selectively add back sparse coupling capacitors between the threshold specified by the <code>primesim_sparse_fcap</code> option and 1e-18.
<code>primesim_add_divider</code>	Defines a character to use as a hierarchy divider.
<code>primesim_add_instance</code>	Automatically creates a subcircuit call line.
<code>primesim_aginglib</code>	Loads a model aging shared library.
<code>primesim_allow_3t_mosfet</code>	Allows three-terminal MOSFETs.
<code>primesim_allow_dup_element</code>	Specifies whether duplicate instance definitions are allowed.
<code>primesim_allow_dup_port</code>	Allows duplicated ports.
<code>primesim_analog</code>	Provides a turnkey accuracy setting to run large post-layout analog centric circuits in PrimeSim Pro.
<code>primesim_auto_gmin_scale</code>	Specifies whether to auto-scale values for gmin and gmindc in FinFET designs.
<code>primesim_auto_restart</code>	Allows you to create snapshots during transient simulation that can be used in conjunction with the <code>-auto_restart</code> command-line option.
<code>primesim_c_model</code>	Allows you to code behavioral models using provided types/functions.
<code>primesim_c_model_exclude</code>	Excludes some instances of c-models from being replaced with the model that was created.
<code>primesim_capmin</code>	Specifies a threshold at which linear capacitors are ignored.
<code>primesim_case_sync_in_subckt</code>	Resolves potential error for mismatched node names caused by mixed case-sensitivity within subcircuit.
<code>primesim_ckk_post</code>	Sets all three post-processing options: • <code>primesim_chkdevop_post</code> • <code>primesim_chkrftime_post</code> • <code>primesim_chktiming_post</code>

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_consistent_vloop_check</code>	Enables to configure how the simulation behaves when PrimeSim encounters a consistent voltage loop.
<code>primesim_change_r3cmc_pin_order</code>	Specifies the pin order to use when assigning pins for the CMC R3 model.
<code>primesim_check_dummy_mos_size</code>	Specifies whether to skip shorting dummy MOSFETs during parsing and check their size during elaboration for violations.
<code>primesim_check_model</code>	Determines if RC models exist.
<code>primesim_check_undefined_param</code>	Specifies whether to check for undefined parameters in the netlist and then, if found, exit the simulation before circuit elaboration or SPF back-annotation.
<code>primesim_check_vth</code>	Determines if Vth or <code>primesim_goff</code> is used for determining the ON/OFF state of the transistor.
<code>primesim_checksum_error_exit</code>	If MD5 checksum error checking is enabled for input files, this option specifies whether to exit simulation in the event of a checksum error.
<code>primesim_checksum_on</code>	Specifies whether to enable MD5 checksum error checking for input files.
<code>primesim_chk_disk_space</code>	Checks available disk space at regular intervals.
<code>primesim_chkblkpwr_pwrnode</code>	Marks nodes as supply nodes in the power report.
<code>primesim_chkblkpwr_pwrport</code>	Defines the power port of all subcircuits.
<code>primesim_chkblkpwr_specified_only</code>	Reports only the block power/ground for the specified node when using the <code>primesim_chkblkpwr_pwrnode</code> or <code>primesim_chkblkpwr_pwrport</code> options.
<code>primesim_chkdcpath_gndgroup</code>	Specifies whether any node connected to an ideal ground (zero) voltage source through a zero voltage source is treated separately or as the same ground group by the <code>.CHKDCPATH</code> command.
<code>primesim_chkdcpath_pwrgroup</code>	Specifies whether any node connected to an ideal power (non-zero) voltage source through a zero voltage source is treated separately or as the same power group by the <code>.CHKDCPATH</code> command.
<code>primesim_chkdcpath_rth</code>	Specifies the value for determining whether a resistor is treated as a parasitic resistor and part of a resistor network or as a normal device and not part of a resistor network.
<code>primesim_chkdevop_disable</code>	Disables <code>.chkdevop</code> for those circuits identified with the specified <code>tag=tag_name</code> arguments (see Check Device Operation Point (.CHKDEVOP)) and enables <code>.chkdevop</code> for all other circuits including those without a <code>tag</code> argument.

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_chkdevop_enable</code>	Enables <code>.chkdevop</code> for those circuits identified with the specified <code>tag=tag_name</code> arguments (see Check Device Operation Point (.CHKDEVOP)) and disables <code>.chkdevop</code> for all other circuits including those without a <code>tag</code> argument.
<code>primesim_chkdevop_filter_in</code>	Controls which devices to include under the specified subcircuit's instances and the specified instances when for checking device operation condition.
<code>primesim_chkdevop_filter_out</code>	Controls which devices to exclude under the specified subcircuit's instances and the specified instances when for checking device operation condition.
<code>primesim_chkdevop_mode</code>	Checks lower hierarchies of the subckt for <code>.chkdevop</code> when <code>subckt=subckt_name</code> is specified.
<code>primesim_chkdevop_post</code>	Allows you to post-process <code>.chkdevop</code> (see Check Device Operation Point (.CHKDEVOP)).
<code>primesim_chkexpr_disable</code>	Disables <code>.chkexpr</code> for those circuits identified with the specified <code>tag=tag_name</code> arguments (see Check Expression (.CHKEXPR)) and enables <code>.chkexpr</code> for all other circuits including those without a <code>tag</code> argument.
<code>primesim_chkexpr_enable</code>	Enables <code>.chkexpr</code> for those circuits identified with the specified <code>tag=tag_name</code> arguments (see Check Expression (.CHKEXPR)) and disables <code>.chkexpr</code> for all other circuits including those without a <code>tag</code> argument.
<code>primesim_chkrftime_post</code>	Allows you to post-process <code>.chkrftime</code> (see Check Rise/Fall Transition Time (.CHKRFTIME)).
<code>primesim_chktiming_post</code>	Allows you to post-process <code>.chktiming</code> (see Check Timing Setup/Hold/Delay/Width (.CHKTIMING)).
<code>primesim_chkznode_rule</code>	Specifies a global option in checking conductivity of MOSFET.
<code>primesim_chkznode_vth</code>	Sets the V_{th} value in calculating the ON/OFF state of MOSFET.
<code>primesim_clampVerilog</code>	Clamps stamped conductance value to <code>primesim_gmax</code> .
<code>primesim_connect_mode</code>	Specifies what the <code>.CONNECT</code> PrimeSim HSPICE-compatible command uses to connect two nodes.
<code>primesim_deltvto</code>	Sets options for the flash core cell.
<code>primesim_dcopt</code>	Applies DC performance optimization.
<code>primesim_detect_lvdd</code>	Provides auto-detection of the VDD level for <code>lprobe</code> .
<code>primesim_detect_lvdd_static</code>	Sets logic probes for internal voltage sources generated, non-ideal sources, such as voltage regulator.

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_detect_vcvs_loop</code>	Enables the PrimeSim Pro tool to detect loops between VCVS sources and Verilog-A and then prevent partitioning from breaking the loops causing "time step too small" errors in transient analysis.
<code>primesim_enhanced_tcl_mode</code>	Allows Tcl to perform model name matching.
<code>primesim_exit</code>	Forces the PrimeSim tool to terminate after parsing, SPF back-annotation, or elaboration.
<code>primesim_exit_dc_fail</code>	Specifies whether to issue a warning message and continue the simulation or exit the simulation entirely if DC does not converge.
<code>primesim_exitwarn</code>	Changes a PrimeSim Pro warning to an error by defining the warning string.
<code>primesim_expand_instance_mfactor</code>	Expands subcircuit instances based on the instance's multiplier parameter (M) and assigns a starting index.
<code>primesim_fast_corner</code>	Enables fast multi-corner simulation to enhance throughput when you use the same input netlist and input vector with variations of PVT (Process, Voltage and Temperature) using .ALTER statements.
<code>primesim_fast_corner_sweep</code>	Enables fast multi-corner simulation to enhance throughput when you use the same input netlist and input vector with variations of PVT (Process, Voltage and Temperature) using .DATA SWEEP simulation.
<code>primesim_fcmodel</code>	Controls the modeling of floating capacitances.
<code>primesim_fingerchar</code>	Sets the finger delimiter character for devices and instances.
<code>primesim_flash</code>	Provides auto-detection and partitioning of internally-generated power supplies in flash design simulations, and enables the advanced algorithm of massive array detection and partitioning.
<code>primesim_flashcell_erase_update</code>	Specifies <code>flashlevel=2</code> erase behavior (see Flash Cell Model).
<code>primesim_floating_gate_gshunt</code>	Adds a conductance between floating gate node and ground.
<code>primesim_fsc_auto_detect</code>	Voltage levels corresponding to logic levels or setting the VDD for a C-model is handled automatically using this option.
<code>primesim_fsc_vdd</code>	Specifies the supply voltage for the C-model interface.
<code>primesim_gdcpath</code>	Adds gshunts to nodes having no DC path to ground.
<code>primesim_gmi_telem</code>	Specifies which T-element implementation to use.
<code>primesim_gpuid</code>	Specifies the GPU card to use for the current process. For more information, see GPU Card Selection .
<code>primesim_gpu_mode</code>	Specifies the mode of operation when using GPU simulation. For more information, see Running GPU Simulation .

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_goff</code>	Works with the <code>.CHKZNODE</code> statement to determine if a MOSFET is <code>on</code> or <code>off</code> .
<code>primesim_hb_autoharm_tol</code>	Controls the tolerance for choosing the number of harmonics to be used by Harmonic Balance analysis when used with the <code>AUTOHARMS</code> argument (see <code>.HB</code> in the <i>PrimeSim SPICE Command Reference</i>).
<code>primesim_ic_priority</code>	Forces the PrimeSim tool to take the <code>.IC</code> value even if there is a <code>.NODESET</code> value defined later on the same node.
<code>primesim_ignore</code>	Ignores instances or subcircuits in simulation for higher performance.
<code>primesim_ignore_device</code>	Makes quick netlist changes by ignoring specified devices during simulation.
<code>primesim_ignore_flat_device</code>	Ignores one or more specified flattened devices (that is, devices with no top-level subcircuit) during the simulation.
<code>primesim_ignore_floating_node</code>	Suppresses WARNINGS about "No DC path to ground on MOSFET...".
<code>primesim_ignore_layout_only_device</code>	Skips the specified devices for the include flow.
<code>primesim_ignore_no_vec_tdata_error</code>	Specifies whether to error out if there is no tabular data in the I/O vector file.
<code>primesim_ignore_op_analysis</code>	Specifies whether to ignore <code>.OP</code> analysis in the netlist.
<code>primesim_ignore_option_error</code>	This option ignores misinput for the <code>primesim_hiersim</code> and <code>primesim_pwrblock</code> options.
<code>primesim_ignore_subblk_option_error</code>	Prints a warning message if a subcircuit name is not defined.
<code>primesim_inactive_chkanode_dv</code>	When performing fast multi-corner simulation, inserts a <code>.CHKANODE</code> option internally in the input run file to detect information on inactive nodes from the first run.
<code>primesim_include_instance</code>	Includes the specified instances in the simulation. All other instances are ignored.
<code>primesim_ind_matrix_passivity</code>	Checks and handles passivity of inductance matrices.
<code>primesim_ind_matrix_passivity_dim_limit</code>	Used along with <code>primesim_ind_matrix_passivity=1</code> or <code>2</code> , to set the matrix dimension limit for inductance matrices passivity check.
<code>primesim_info_limit</code>	Sets the number of "INFO:" messages to a limited number of user-specified lines.
<code>primesim_isolated_vsrc_res</code>	Specifies whether to detect floating voltage sources which are referenced by other current controlled sources and add a shunt resistance of 1M to them.
<code>primesim_keepzeroparms</code>	Use zero values for PS, PD, AS, and AD parameters for BSIM models.

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_lprobe_hiz</code>	Specifies whether to check the Hi-Z node and display the "Z" state in the waveform.
<code>primesim_lprobe_vh</code>	Sets the high threshold value for signals with <code>.LPROBE</code> statement.
<code>primesim_lprobe_vl</code>	Sets the low threshold value for signals with <code>.LPROBE</code> statement.
<code>primesim_lsf_format_chars</code>	Allows usage of 3rd-party Verilog-A special characters.
<code>primesim_max_swap_allowed</code>	Controls the maximum amount of swap memory that can be used during a simulation.
<code>primesim_max_width_tol</code>	Sets tolerance range for transistor width when it exceeds maximum defined width.
<code>primesim_mc_sigma_scale</code>	Scales the Monte Carlo distribution.
<code>primesim_meas_from_to</code>	Specifies whether the <code>FROM</code> and <code>TO</code> parameters for the <code>TARG</code> parameter of a measurement are applied to the <code>TRIG</code> parameter of the measurement.
<code>primesim_meastolscale</code>	Specifies the compression factor of internally saved data when performing runtime measurement.
<code>primesim_merge_vector_errmsg</code>	Specifies whether to merge the <code>.err</code> files from multiple runs into a single <code>.err</code> file.
<code>primesim_model_dup_rule</code>	Specifies how to handle duplicate <code>.MODEL</code> definitions.
<code>primesim_model_verification_mode</code>	Use tighter simulation settings for simulations with model qualification.
<code>primesim_module_instance</code>	Applies a 3DIC module for a regular netlist without modifying the input netlist.
<code>primesim_monte_cmd_overwrite</code>	Overwrites the <code>SWEEP MONTE=MCcommand</code> section of the analysis statements without having to modify the netlist.
<code>primesim_moscapcut</code>	Specifies when the MOS capacitor should be cut for partitioning based on its estimated capacitance value.
<code>primesim_negres</code>	Keeps negative resistors in the netlist.
<code>primesim_no_swap</code>	Controls whether disk swap can be used when the simulation runs out of memory.
<code>primesim_noise_off</code>	Disables noise generation only for noise sources of the specified type. Noise of all other types is enabled.
<code>primesim_noise_on</code>	Enables noise generation only for noise sources of the specified type. Noise of all other types is disabled.
<code>primesim_parhier_mode</code>	Specifies how the PrimeSim HSPICE-compatible <code>.OPTION PARHIER</code> functions when set to <code>LOCAL</code> .

Table 16 General Options (Continued)

Option	Description
<code>primesim_phasenoise_lorentz</code>	Used with phase noise analysis, specifies whether to calculate and output the Lorentzian corner frequency and apply the Lorentzian model on phase noise at a low frequency offset.
<code>primesim_phasenoise_min_harmfreq_offset</code>	In phase noise analysis, the output noise approaches infinity when the output frequency gets close to harmonic frequencies. Use this option to skip frequency points that are too close to harmonic frequencies.
<code>primesim_prll_opt</code>	Controls how the <code>-np X</code> command-line option functions (see Command-Line Options for Running Multi-CPU Simulations).
<code>primesim_pinst_in_p</code>	Specifies how the <code>p()</code> power reporting function works when Probing and Measuring .
<code>primesim_pva_flow</code>	Simulates Verilog-A Elements using the Portable Verilog-A Engine compatible with PrimeSim HSPICE.
<code>primesim_remove_floating_vsrc</code>	Removes any floating voltage sources and issues a warning message.
<code>primesim_restore (.SNAPSHOT)</code>	Restores the snapshot of the dynamic state of the circuit in motion that has been captured by the <code>.snapshot</code> option.
<code>primesim_rng</code>	Specifies the random number generator (RNG) used by the simulator.
<code>primesim_rpitft_mode</code>	Enables PrimeSim support for SmartSpice TFT Model correlation.
<code>primesim_runtime_meas</code>	Uses runtime measurement instead of post measurement.
<code>primesim_scale</code>	Applies the scale factor for subcircuits.
<code>primesim_set_cpu_time</code>	Sets the maximum elapse time for a simulation.
<code>primesim_set_floating_node</code>	Sets a user-specified voltage value for either all floating nodes or floating gate nodes only (see primesim_set_floating_node_type) at either the DC operating point only or also throughout the transient simulation (see primesim_set_floating_node_format).
<code>primesim_set_floating_node_format</code>	Determines whether the floating node value specified by primesim_set_floating_node is applied in DC initialization only or also throughout the transient simulation.
<code>primesim_set_floating_node_type</code>	Determines whether the floating node value specified by primesim_set_floating_node is applied to all floating nodes or to floating gate nodes only.
<code>primesim_setenv</code>	Specifies an environment variable <i>name</i> and sets it to <i>value</i> .
<code>senssort</code>	Sorts the sensitivity results from big to small for DC sensitivity or AC sensitivity if its value is 1.
<code>primesim_short_rnet</code>	Shorts the resistors in an RC network that includes the specified node name.

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_single_bin_model_check</code>	Apply a similar model binning check for non-binning models.
<code>primesim_skip_flat_pl_node</code>	Reduces FSDB file size by reducing the number of nodes probed.
<code>primesim_skip_meas_error</code>	Specifies whether simulation should error out or continue with a warning if a <code>.MEASURE / MEAS</code> statement is missing a parameter.
<code>primesim_skip_unused_param</code>	Ignores legacy HSPICE-specific parameters to avoid ineffective warning messages.
<code>primesim_spred</code>	Specifies RC reduction for extracted netlist to reduce the runtime penalty.
<code>primesim_soa_check_on_dc_init</code>	Runs SOA checks on the first two time steps in Transient Analysis and then exits the simulation.
<code>primesim_subckt_dup_rule</code>	Selects which subcircuit definition to use in simulation.
<code>primesim_tcl_init_file</code>	Specifies the Tcl file for PrimeSim to source for the simulation.
<code>primesim_tmiage_reltol</code>	For TMI aging simulation, devices can share the same table if the difference between their model and instance parameters is less than the relative tolerance specified by this option.
<code>primesim_tmishe_abstol</code>	For TMI self-heating simulation, devices can share the same table if the difference between their temperatures is less than the absolute tolerance specified by this option.
<code>primesim_tran_conv_retry</code>	If transient simulation fails to converge the first time, the PrimeSim tool reruns transient simulation a second time with better convergence settings.
<code>primesim_tsc</code>	Sets the algorithm mode to be used for time step control during transient analysis.
<code>primesim_tstop</code>	Modifies the <code>tstop</code> specified in the <code>.tran</code> command line, to shorten/extend simulation length.
<code>primesim_vector</code>	Specifies a vector file for comparison with simulation results.
<code>primesim_vector_errmsg</code>	Specifies how errors are handled during vector comparison.
<code>primesim_vector_mode</code>	Sets the I/O vector file format for bus signals.
<code>primesim_veriloga (.hdl)</code>	Includes a Verilog-A file in the netlist.
<code>primesim_veriloga_bypass</code>	Enables the model bypass of entire Verilog-A models in the design.
<code>primesim_veriloga_cross_method</code>	Specifies what value (<code>true false</code>) the Verilog-A <code>cross(x)</code> function returns when <code>x</code> is exactly zero.

Table 16 *General Options (Continued)*

Option	Description
<code>primesim_veriloga_dkind_def</code>	Defines the device type for the Verilog-A device model, allowing the dynamic circuit checker to operate normally on the Verilog-A device model just as with normal primitive devices.
<code>primesim_veriloga_dkind_mode</code>	Specifies whether to enable (1) or disable (0) the <code>primesim_veriloga_dkind_def</code> option.
<code>primesim_veriloga_table_fill_in_policy</code>	Specifies the direction to start extrapolation if there are missing indexed entries when using the <code>table_param</code> built-in function to specify a file that contains a parameter values table and return the linear interpolated value from that table.
<code>primesim_veriloga_var_opt</code>	Sets some local variables within Verilog-A modules in order to have the variables shared among instances.
<code>primesim_warn_limit</code>	Sets the number of all warning messages to a specific number of lines.
<code>primesim_write_instance_param</code>	Dumps out a list of resolved instance parameters with full scope names.
<code>primesim_write_instance_table</code>	Generates an instance list file.
<code>primesim_write_mcparam</code>	Saves the generated parameters in each run to a file.
<code>pva_genso_filebase</code>	Automatically generates the <code>.so</code> files per VA files based on all <code>.hdl</code> VA files or reuse <code>.so</code> files during simulation. The default is 0.
<code>pva_output_dir</code>	Controls the cache directory or sets the output directory of the Portable Verilog-A Engine (pVA) to the desired location. It is very useful for reusing <code>.so</code> files.
<code>s_cache_reuse</code>	Global option to turn on and off S parameter cache reuse.

DC Initialization Options

The PrimeSim tool provides the following options to control DC initialization.

Table 17 *DC Initialization Options*

Option	Description
<code>primesim_convlevel</code>	Determines how PrimeSim attempts to solve the "time step too small" error.
<code>primesim_cshpt</code>	Adds a grounded capacitor with the specified value to each node during pseudo-transient in DC.
<code>primesim_dcalg</code>	Sets algorithm for DC convergence (SPICE mode only).
<code>primesim_dceffort</code>	Controls effort applied for DC convergence.
<code>primesim_gen_ic_op</code>	Generates the <code>.op</code> file when it is not created by default.
<code>primesim_gen_op_captab</code>	Generates node capacitance information in the <code>.op</code> file.
<code>primesim_gic</code>	Sets conductance value of the resistor in the Norton-type voltage source.
<code>primesim_keep_0v_dcvs</code>	Keeps zero voltage DC sources in the simulation database.
<code>primesim_pro_dcalg</code>	Sets the approach for DC convergence (Pro modes only).

Output Reporting Options

The PrimeSim tool provides the following options to control output reporting.

Table 18 *Output Reporting Options*

Option	Description
<code>primesim_bisection_concurrent_mode</code>	Controls the behavior of bisection optimization when performing Concurrent Bisection for Independent Circuit Blocks .
<code>primesim_bisection_fail_continue</code>	Allows the simulation to continue to other sweeps outside the bisection in the event that the bisection sweep fails.
<code>primesim_bisection_output</code>	Controls the measure file and waveform database output for each bisection sweep.
<code>primesim_bisection_summary</code>	Controls the bisection summary file <code>.bisect_mt0</code> .
<code>primesim_cck_remove_result</code>	Removes the circuit check result file when there are no errors.
<code>primesim_cck_report_limit</code>	Controls the maximum number of error reports generated for each file.chkdevop file during simulation.
<code>primesim_chk_devport</code>	Checks device port name during wildcard matching of <code>.PROBE</code> statement.
<code>primesim_chk_format</code>	Specifies whether <code>.CHKDEVOP</code> commands use the PrimeSim XA Tcl-CCK output format.
<code>primesim_chkdevcur_report</code>	Specifies whether the <code>.CHKDEVCUR</code> command generates two log files: the main file and the summary file, or just the summary file..
<code>primesim_chkdevop_print_info</code>	Specifies whether to append W,L,m device information to the <code>.CHKDEVOP</code> violations report.
<code>primesim_chkdg</code>	Sets the number of digits displayed after the decimal by the <code>.CHKRETIME</code> and <code>.CHKTIMING</code> commands.
<code>primesim_chkznode_debug_out</code>	Specifies that device turn-on/off information be printed out for Hi-Z node debugging.
<code>primesim_cont_measout</code>	Controls the continuous measurement file format.
<code>primesim_cprbtol</code>	Sets the capacitance tolerance for the capacitance print out.
<code>primesim_double_precision_output</code>	Changes output file value from 4 bytes to 8 bytes.
<code>primesim_elapsed_tflush</code>	Flushes output files after the specified number of seconds in elapsed runtime since the previous flush.
<code>primesim_fflush</code>	Sets the PrimeSim tool to output simulation data to a file based on elapsed time or simulation progress.
<code>primesim_finger_current</code>	Specifies whether to report the total device current or only the main device's finger current contribution.

Table 18 *Output Reporting Options (Continued)*

Option	Description
<code>primesim_fsdb_limit</code>	Specifies a size limit for FSDB files.
<code>primesim_fsdb_split_pg</code>	Specifies to split Power/Ground R-Network nodes in non-ideal Power/Ground simulation from all other signals and then output them into a separate FSDB file.
<code>primesim_fsdb_split_time</code>	Allows splitting of FSDB files.
<code>primesim_fsdb_term_var_hier</code>	Changes the terminal delimiter, in hierarchical signal names, from a colon (:) to a period (.).
<code>primesim_fsdb_version</code>	Specifies what version of FSDB writer the PrimeSim tool should use.
<code>primesim_gen_ac_op</code>	Controls whether operating point files or initial condition files are generated during an "AC at Diff T" simulation (see <code>primesim_ac_at_diff_t</code>).
<code>primesim_gen_mc0</code>	Specifies whether to generate the *.mc0 file for Monte Carlo analysis.
<code>primesim_gen_pn_c</code>	Enables output to the *.pn_c0 file in .NOISE analysis.
<code>primesim_identical_mc_instance_file</code>	Generates the same random number sequence for the Monte Carlo simulation.
<code>primesim_ignore_chkfunc_error</code>	Continues the simulation with a warning instead of an error and exiting.
<code>primesim_iovec_abs_value</code>	Treats all the time values in the vector file as absolute time.
<code>primesim_iovec_option</code>	Specifies whether PrimeSim accuracy options can be used inside a vector file using the <code>fscript</code> utility <code>fset</code> command.
<code>primesim_iovec_vih</code>	Sets the <code>logichv</code> outside of a vector file.
<code>primesim_iovec_vil</code>	Sets the <code>logiclv</code> outside of a vector file.
<code>primesim_iovec_voh</code>	Sets the default <code>voh</code> value for the vector file.
<code>primesim_iovec_vol</code>	Sets the default <code>vol</code> value for the vector file.
<code>primesim_iprbtol</code>	Sets the current tolerance for the current print out.
<code>primesim_iprobe_leaf_level</code>	Specifies whether to output device currents at the lowest level of hierarchy in the design.
<code>primesim_list_noisesources</code>	Specifies whether to include noise contributors in the output waveform file from the following analyses: <code>.HBNOISE</code> , <code>.LIN</code> , <code>.NOISE</code> , <code>.PHASENOISE</code> , and <code>.SNNNOISE</code> .
<code>primesim_map_prbport</code>	Maps probe statements from <code>: [dg sb]</code> to <code>v [1234]</code> syntax.
<code>primesim_maxicout</code>	Sets maximum number of node voltages included in the <code>.ic</code> file.
<code>primesim_mc_stats_report</code>	Controls printing of Monte Carlo statistics.

Table 18 *Output Reporting Options (Continued)*

Option	Description
<code>primesim_mcbrief</code>	Limits Monte Carlo information written to the log file.
<code>primesim_measout</code>	Controls the measurement file format.
<code>primesim_merged_meas_file</code>	Specifies whether to output separate <code>.mt0</code> files or a single merged <code>.mt0</code> file for sweep and bisection.
<code>primesim_merge_fsdb</code>	Specifies whether to merge the PrimeSim tool's FSDB output with a digital FSDB file for mixed-signal simulation using the VCS PrimeSim AMS tool.
<code>primesim_monitor</code>	Provides CPU monitoring functionality when the PrimeSim tool is running on a server.
<code>primesim_mparcheck</code>	Determines whether MOSFET model parameters are checked.
<code>primesim_noise_format</code>	Controls the format of the <code>.pn0</code> output file.
<code>primesim_num_meas_log</code>	Limits number of measurement results output to the screen and log file.
<code>primesim_num_meas_per_line</code>	Outputs measurement values on multiple lines.
<code>primesim_op_compatible</code>	Specifies which operating point (<code>.OP</code>) analysis output format to use.
<code>primesim_op_region_display</code>	Specifies which values to use for different regions of MOS operation when using waveform display tools such as PrimeWave Design Environment WaveView.
<code>primesim_output</code>	Sets the output file format for saving transient analysis results.
<code>primesim_output_fname_type</code>	Defines the naming convention for output files when the netlist has an <code>.ALTER</code> and/or a <code>.TEMP / TEMPERATURE</code> sweep parameter.
<code>primesim_op_parasitic_capacitance</code>	Specifies whether to output parasitic passive capacitors into the OP result file.
<code>primesim_output_gzip</code>	Enables <code>gzip</code> file compression of the output files.
<code>primesim_output_op</code>	Specifies the output format for the operating point (<code>.op</code>) report.
<code>primesim_output_range</code>	Limits output of transient results of waveforms to the waveform file.
<code>primesim_prb_skipmeas</code>	Specifies whether to skip saving probed signals from <code>.MEASURE</code> statements in the FSDB file.
<code>primesim_prb_tmin</code>	Specifies that the waveform file records data points only after the minimum transient time set by this option has passed.
<code>primesim_prbexprvar</code>	Determines whether to save any signals in the expression of a <code>.PROBE</code> statement.
<code>primesim_prbport</code>	Determines whether port voltages are probed, depending on tolerance.

Table 18 *Output Reporting Options (Continued)*

Option	Description
<code>primesim_prbtol_disable_visrc</code>	Disables <code>primesim_vprbtol</code> or <code>primesim_iprbtol</code> for any independent voltage source(V)'s node voltage and current source(I)'s current, respectively.
<code>primesim_print_global_param</code>	Prints global parameter names and resolved values.
<code>primesim_print_period</code>	Defines the fixed interval at which the <code>.PRINT</code> command is printed to the output file.
<code>primesim_print_rchk_resource</code>	Reports the detailed time and memory usage for the <code>-rchk</code> command-line option when rechecking the database (see Command-Line Options).
<code>primesim_print_to_probe</code>	Converts all <code>.PRINT</code> statements to <code>.PROBE</code> statements.
<code>primesim_probe_passive_device</code>	Specifies whether to probe passive devices.
<code>primesim_probe_protected</code>	Specifies whether to allow the probing of subcircuits within protected or encrypted blocks.
<code>primesim_profile</code>	Generates profile data in a file after transient analysis.
<code>primesim_pseudotrandbg</code>	Controls whether an FSDB file is generated from pseudo-transient analysis.
<code>primesim_pt0_format</code>	Changes the <code>.PRINT</code> output default format into a list format.
<code>primesim_pwl_r_end</code>	Changes the meaning of parameter R for PWL source.
<code>primesim_remove_probe_prefix</code>	Removes the prefix from signals from <code>.PROBE</code> statements.
<code>primesim_report_large_no_instances</code>	Reports subcircuit instances that contain a large number of devices.
<code>primesim_set_special_char</code>	Appends a backslash (\) before special characters during PSF output.
<code>primesim_skipwarn</code>	Determines amount and type of information output to the log file.
<code>primesim_soa_maxwarns</code>	Sets the maximum number of warnings for <code>primesim_soa_warn</code> .
<code>primesim_soa_warn</code>	Specifies whether to enable Safe Operating Area (SOA) checking.
<code>primesim_soa_warn_format</code>	Specifies time window-based SOA reporting.
<code>primesim_soa_warn_range</code>	Limits SOA checking to one or more user-specified time ranges.
<code>primesim_soa_warn_skipwindow</code>	Controls whether to skip the SOA warning report if the violation time window is less than a specified value.
<code>primesim_soa_warn_to_file</code>	Controls whether to output the SOA warnings in the log file or separate file.
<code>primesim_sweep_brief</code>	Limits parametric sweep information written to the log file.

Table 18 *Output Reporting Options (Continued)*

Option	Description
<code>primesim_tflush</code>	Flushes output files at the specified time interval of analysis.
<code>primesim_title_in_alter_filenames</code>	Specifies whether to append the <i>title_string</i> from <code>.ALTER</code> statements to the output file name prefix.
<code>primesim_use_old_trout</code>	Determines whether to use new or previous output file name extensions for <code>.PRINT</code> and <code>.MEASURE / MEAS</code> .
<code>primesim_vprbtol</code>	Sets the voltage tolerance for voltage print out.
<code>primesim_vpwltol</code>	Removes small changes in the voltage input waveform in PWL format.
<code>primesim_wdf_limit</code>	Splits wdf output files that exceed a user-specified value.
<code>primesim_wdf_mode</code>	Controls the amount of compression for wdf output file.

Partitioning Options

The PrimeSim Pro tool provides the following options to control partitioning.

Table 19 *Partitioning Options*

Option	Description	Mode
<code>primesim_cutnode</code>	Reduces partition size for signal nodes.	Pro Modes Only
<code>primesim_extended_pwrblock</code>	Enables power block detection for topologies that have feedback and/or circuits that must be in the same partition.	Pro Modes Only
<code>primesim_flatsize</code>	Sets a threshold to flatten subcircuits before partitioning starts.	Pro Modes Only
<code>primesim_gmax</code>	Clamps the stamped conductance value to <code>primesim_gmax</code> .	Pro Modes Only
<code>primesim_hiersim</code>	Selects hierarchical simulation mode.	Pro Modes Only
<code>primesim_hstolscale</code>	Sets a multiplier applied to all internal tolerance values for subcircuits that are simulated with the <code>hiersim</code> method.	Pro Modes Only
<code>primesim_partition</code>	Specifies how partitioning is done.	Pro Modes Only
<code>primesim_print_max_con_node</code>	Identifies nodes with the greatest number of connections for connectivity report.	Pro Modes Only
<code>primesim_pwrblock</code>	Detects transistors that control or regulate power.	Pro Modes Only
<code>primesim_pwrloadmodel</code>	Enables a power load model especially well-suited for power supply networks.	Pro Modes Only
<code>primesim_pwrtol</code>	Specifies the voltage tolerance to control an event between a power block partition and the other partitions.	Pro Modes Only

Accuracy and Speed Options

The PrimeSim tool provides the following options for accuracy and speed.

Table 20 *Accuracy and Speed Options*

Option	Description
<code>primesim_adaptive_mt</code>	Enables an optimized method for handling multi-threading on an overloaded machine.
<code>primesim_bytol</code>	Sets the voltage value to use in determining whether a MOS is latent.
<code>primesim_ccmodel</code>	Controls the modeling of coupling capacitors based on the total coupling capacitance ratio.
<code>primesim_cell_char</code>	Enhances performance when using the PrimeSim tool for cell characterization.
<code>primesim_ckptopt</code>	Applies circuit optimization.
<code>primesim_delmax</code>	Sets maximum value of internal time step in simulation.
<code>primesim_delmax_level</code>	Controls whether to use synchronous or asynchronous variable <code>delmax</code> .
<code>primesim_delmin</code>	Specifies the minimum time step in a simulation.
<code>primesim_dvmax</code>	Sets the maximum voltage change of a node in a single time step.
<code>primesim_evlevel</code>	Improves performance especially in post-layout circuits with many coupling capacitors such as SRAM/DRAM.
<code>primesim_fcaptop</code>	Converts a small coupling capacitance into two grounded capacitances.
<code>primesim_gshpt</code>	Adds conductance from each node to ground during pseudo-transient simulation. After pseudo-transient converges, the conductance value is ramped down to achieve better DC accuracy.
<code>primesim_ignore_va</code>	Improves front-end performance by ignoring Verilog-A file parsing.
<code>primesim_indn</code>	Specifies whether the PrimeSim tool uses a Norton equivalent or a more accurate formulation to model a simple inductor.
<code>primesim_leakage_mode</code>	Uses more conservative leakage tolerances for better accuracy.
<code>primesim_loadmodel</code>	Sets the capacitive load model that a partition output drives.
<code>primesim_max_parallel</code>	For parallel simulation in PrimeSim Pro mode with multiple CPUs, sets whether the order of internal events for the partition evaluation should be fixed (more consistent results run-to-run) or not (better runtime performance).
<code>primesim_mcseed</code>	Provides a seed to produce the same sequence of random numbers in Monte Carlo simulation.

Table 20 *Accuracy and Speed Options (Continued)*

Option	Description
<code>primesim_merge_parmosdio</code>	Optimizes MOSFET device instances that are in parallel.
<code>primesim_merge_pinstance</code>	Optimizes subcircuit instances that are in parallel.
<code>primesim_method</code>	Sets the numerical integration method used during a transient analysis.
<code>primesim_mode</code>	Sets the simulation mode.
<code>primesim_model</code>	Sets the modeling method of the MOSFETs.
<code>primesim_mosfet_fast_probe_lx34</code>	Speeds up probing the LX34 MOS parameter for the level=129 CMI model.
<code>primesim_npc_max</code>	Specifies that the PrimeSim SPICE tool should take extra effort to ensure that simulation results are consistent across a different number of threads.
<code>primesim_pdn</code>	Provides autodetection of power and ground nets and optimized RC reduction in PDN (Power Distribution Network) design simulations, and enables advanced event control between circuit and power networks automatically.
<code>primesim_precision</code>	Specifies whether to serve as the reference with the highest precision and ignore the <code>runlvl</code> option.
<code>primesim_prelayout_models</code>	Sets which model PrimeSim should use.
<code>primesim_pva_cross_ttol</code>	Specifies the PrimeSim / Portable Verilog-A Engine timing accuracy for the <code>CROSS</code> and <code>ABOVE</code> functions.
<code>primesim_pva_opt_port</code>	Specifies whether to treat the Verilog-A port as non-voltage, and so not subject to per-iteration limiting.
<code>primesim_rclamp</code>	Specifies the value at which to clamp those resistors not shorted by <code>primesim_resmin</code> .
<code>primesim_rcred</code>	Sets the level of acceleration to use when performing post-layout analog simulation with RC reduction.
<code>primesim_rcreditscale</code>	Sets a scaling factor applied for RC reduction performed using the <code>primesim_rcred</code> command.
<code>primesim_resmax</code>	Sets maximum value for a resistor to be considered in simulation.
<code>primesim_resmin</code>	Sets minimum value for a resistor to be considered in simulation.
<code>primesim_reuse_mos_model</code>	Reuses MOSFET models built in the previous iteration.
<code>primesim_reuse_op</code>	Enables the PrimeSim SPICE tool to reuse the operating point solution from a previous analysis as the initial guess for subsequent analyses, when there are multiple analyses, or for Monte Carlo sweep simulation.
<code>primesim_sparse_fcap</code>	Sets the absolute threshold for marking sparse floating capacitors.

Table 20 Accuracy and Speed Options (Continued)

Option	Description
<code>primesim_speed</code>	Sets the speed level of the simulation.
<code>primesim_spred</code>	Determines the type and level of RC reduction.
<code>primesim_spredtc</code>	Sets the time constant value used by the RC reduction algorithm.
<code>primesim_tolscale</code>	Sets a multiplier applied to all internal tolerance values.
<code>primesim_tunit</code>	Defines the base time unit of the simulation.
<code>primesim_vdd</code>	Sets the VDD value used for MOS table generation.

Back-Annotation Options

The PrimeSim tool commands for back-annotation are divided into DSPF annotation, DPF annotation, and non-ideal power analysis. See [Back-Annotation](#), for more details about post-layout simulation with back-annotation.

Table 21 *DSPF Annotation Options*

Option	Description
<code>primesim_enprefix</code>	Specifies the prefix characters in a pre-layout netlist that should be removed to match with the DSPF instance name during back-annotation.
<code>primesim_eqr2spres</code> , <code>primesim_spresout</code>	Enables and specifies the output format for the SPRES solver. The SPRES solver is a faster means of calculating equivalent resistance (EQR) of resistor networks from the SPF file and is used in conjunction with: <code>primesim_spfeqr</code> , <code>primesim_spf2eqr</code> , <code>primesim_spfeqrfile</code> , <code>primesim_spfeqr_only</code> , <code>primesim_spf2eqr_select</code> .
<code>primesim_gpd</code>	Back-annotates a corner used in the Galaxy Parasitic Database (GPD).
<code>primesim_repdot</code>	Replaces a period (.) with another character.
<code>primesim_simple_em_naming</code>	Uses the period (.) as single hierarchical delimiter when period (.) and slash (/) characters are mixed in signal names. This is used in EM analysis.
<code>primesim_spf</code>	Specifies the DSPF file that contains SPF data extracted from the layout interconnects.
<code>primesim_spf_acc_irem</code>	Performs dynamic EM simulation of a huge back-annotated circuit with better performance than standalone IR/EM.
<code>primesim_spf_add_netpin</code>	Allows you to add a small number of netpins in a DSPF file.
<code>primesim_spf_add_netpin_file</code>	Allows you to add a large number of netpins in a DSPF file.
<code>primesim_spf_add_irem_window</code>	Specifies analysis time range for the IR/EM .em file.
<code>primesim_spf_allow_rc_parasitic_instance</code>	Allows you to program the RC net of a Standard Parasitic Format (SPF) file with a custom subckt module in order to estimate the RC effect of that SPF NET.
<code>primesim_spf_bus_char</code>	Used when extracted SPEF file bus notation is different from schematic netlist bus notation.
<code>primesim_spf_cc_parse</code>	Specifies whether to parse all coupling capacitor NETs even if one of the NETs is not found in the schematic netlist.
<code>primesim_spf_delete_netpin</code>	Allows you to delete a small number of netpins in a DSPF file.
<code>primesim_spf_delete_netpin_file</code>	Allows you to delete a large number of netpins in a DSPF file.

Table 21 *DSPF Annotation Options (Continued)*

Option	Description
<code>primesim_spf_exclude</code>	Excludes the specified instances from SPF back-annotation.
<code>primesim_spf_fingerchars</code>	Translates the special SPF finger delimiter from the specified string to an at-sign (@) character.
<code>primesim_spf_finger_res_policy</code>	Specifies whether finger resistors must be in parallel to the base resistor or are allowed to be a series or a mix of parallel and sequential order.
<code>primesim_spf_histogram</code>	Specifies whether to print an RC Histogram for each DSPF file during back-annotation.
<code>primesim_spf_include</code>	Includes the specified instances from SPF back-annotation.
<code>primesim_spf_layout_only_device_models</code>	Specifies the device model names in the DSPF file to be added to the back-annotation simulation.
<code>primesim_spf_layout_only_net</code>	Use this option to specify one or more NETs which exist only in layout so that you can apply RC annotation options to them.
<code>primesim_spf_map_subckt_port</code>	Sets mapping between the port names of the schematic file and the pin names of the back-annotation file.
<code>primesim_spf_matcheffort</code>	Performs more rigorous naming matching for SPF net and devices.
<code>primesim_spf_port_name_check</code>	Specifies whether to check for multiple instances of port names connecting to another NET.
<code>primesim_spf_removetoprc</code>	Removes the top-level parasitics in the <code>primesim_spfinst=2</code> back-annotation flow.
<code>primesim_spf_sch_check_mode</code>	Turn this option on in back-annotation simulations when the schematic netlist changes faster than layout and DSPF files can be generated.
<code>primesim_spf_sensitive</code>	Supports case sensitivity for netlist back-annotation flow.
<code>primesim_spf_skip_res_for_ivsrc</code>	Tests to see whether any SPF nets for RC annotation are connected to any current sources or voltage sources. If so, all of the resistors for those nets are shorted automatically.
<code>primesim_spf_spice_names</code>	Specifies whether the instance names in the DSPF file include SPICE type prefix.
<code>primesim_spf_user_defined_gnd_list</code>	Specifies the node list that should be treated as ideal ground in SPF back-annotation.
<code>primesim_spfallowerror</code>	Continues simulation even when there is a parsing error of SPF.
<code>primesim_spfcnet</code>	Specifies the net names to be annotated with lumped capacitance only.

Table 21 *DSPF Annotation Options (Continued)*

Option	Description
<code>primesim_spfeqr</code> , <code>primesim_spf2eqr</code> , <code>primesim_spfeqrfile</code> , <code>primesim_spfeqr_only</code> , <code>primesim_spf2eqr_select</code>	Controls equivalent resistance (EQR) analysis of a DSPF file. To use the SPRES solver, see <code>primesim_eqr2spres</code> , <code>primesim_spresout</code> .
<code>primesim_spffcmin</code>	Sets the minimum floating capacitor value allowed for back-annotation.
<code>primesim_spffcnet</code>	Keeps the floating capacitance for C-only back-annotation.
<code>primesim_spfhdiv</code>	Sets the divider character for hierarchical names in DSPF files.
<code>primesim_spfinst</code>	Controls whether the instance section of DSPF file is used for back-annotation.
<code>primesim_spfkeepnodecap</code>	Controls whether to keep or remove the pre-layout ground capacitance of back-annotated nets.
<code>primesim_spfmergeport</code>	Merges ports in a DSPF net.
<code>primesim_spfmodule</code>	Specifies the module for SPF back-annotation.
<code>primesim_spfnet_wildcard_mode</code>	Specifies whether the wildcard character (*) when used in back-annotation options refers to all levels of hierarchy or just one level of hierarchy.
<code>primesim_spfnonet</code>	Specifies the nets that are not going to be back-annotated by DSPF data.
<code>primesim_spfpost</code>	Sets the node names for post analysis of DSPF back-annotated simulation.
<code>primesim_spfpost_out_only</code>	Probes the internal nodes of power/ground during EM simulation.
<code>primesim_spfprb</code>	Sets the probe mode for DSPF nodes.
<code>primesim_spfprb_finger</code>	Specifies whether to probe fingered device pins based on the <code>primesim_spfprb</code> option setting.
<code>primesim_spfprb_mode</code>	Specifies whether to flatten an SPF-annotated node name.
<code>primesim_spfprb_sync</code>	Specifies whether the probed NET's device's pin nodes or internal subnodes are probed.
<code>primesim_spfprefix</code>	Lists the prefixes that should be removed from device names.
<code>primesim_spfrc_ctype</code>	Specifies whether and how to filter grounded capacitors (gcap) and/or coupling capacitors (fcap) for option <code>primesim_spfrcnet</code> .
<code>primesim_spfrcnet</code>	Specifies the net names that are annotated with RC trees.
<code>primesim_spfrepgate</code>	Picks the first GATE pin defined at (* I)/(* NET) from the DSPF file, only if the GATE pin exists under (* NET) from the DSPF file.

Table 21 *DSPF Annotation Options (Continued)*

Option	Description
<code>primesim_spfreplast</code>	Controls how the representative node is chosen when annotating DSPF.
<code>primesim_spfrgcnct</code>	Specifies the net names that are annotated with parasitic resistors and ground capacitors.
<code>primesim_spfrmax</code>	Sets the maximum resistor value allowed for back-annotation.
<code>primesim_spfrmin</code>	Sets the minimum resistor value allowed for back-annotation.
<code>primesim_spfrnet</code>	Specifies the net names to be annotated with parasitic resistor only.
<code>primesim_spfrptrmax</code>	Sets the warning threshold number for SPF resistors.
<code>primesim_spfscale</code>	Sets the scale for devices in the DSPF file.
<code>primesim_spfspef</code>	Sets <code>primesim_spf</code> input to be DSPF or SPEF format.
<code>primesim_spfsplitnet</code>	Splits nets into multiple nets after layout is complete.
<code>primesim_spfsuffix</code>	Defines the suffix in an SPF file.
<code>primesim_spfwarn</code>	Controls warning message output when performing and debugging back-annotation.

Table 22 *DPF Annotation Commands*

Command	Description
<code>primesim_dpf</code>	Specifies the DPF file that contains the extracted device parameter data.
<code>primesim_dpf_mos_only</code>	Specifies whether to back-annotate all devices or only MOSFET devices.
<code>primesim_dpfaadddev</code>	Controls whether DPF devices that are not in the netlist are added.
<code>primesim_dpfhdiv</code>	Sets the divider character for hierarchical names in DPF files.
<code>primesim_dpftime</code>	Specifies the module for DPF back-annotation.
<code>primesim_dpfprefix</code>	Lists the prefixes that should be removed from DPF device names.
<code>primesim_dpfscale</code>	Sets the scale for devices in the DPF file.
<code>primesim_dpfsuffix</code>	Defines the suffix in a DPF file.

Table 23 IR/EM Commands

Command	Description
<code>primesim_ra_cmd</code>	Specifies the output formats to be reported and provides backward compatibility to read the command format supported by Titan/ <code>fscript</code> .
<code>primesim_ra_net</code>	Specifies the nets to report for IR Drop output.
<code>primesim_ra_tcl</code>	Reads a PrimeSim XA RA Tcl file.
<code>primesim_spf_em_version</code>	Sets the EM database version.
<code>primesim_spfdb_version</code>	Sets the IR drop database version.

PrimeSim SPICE Accuracy and Speed Options (-runlvl and .option runlvl)

The PrimeSim SPICE tool supports the following option for easier performance and accuracy tuning:

- `runlvl`: Specifies the accuracy level setting for transient analog simulation where 1 is the best performance and 6 is the best accuracy. The default if not specified on the command line is 4.

This option can be specified in the command line or the netlist, for example:

```
primesim -spice netlist.sp -runlvl 5
```

or

```
.option runlvl=5
```

Note:

- Command-line options take precedence over netlist options.
- If no command-line option is specified, then netlist options are used.
- The PrimeSim HSPICE `accurate` option is not supported by the PrimeSim SPICE tool.

Figure 11 lists how performance and accuracy PrimeSim options are preset according to the `runlvl` setting.

Figure 11 Ease-of-Use Settings for `runlvl`

runlvl	6	5	4	3	2	1
primesim_tolscale	1	5	10	50	100	150
primesim_rcred (for all except AC small-signal analysis)	0	Conservative RC Reduction on Aggressive →				
primesim_rcred (for AC small-signal analysis)	0	0	0	0	0	0
primesim_model	4.5	4.5	4.5	4	4	3
primesim_indn	0	0	0	1	1	1
primesim_tunit	1e-14	1e-14	1e-14	1e-14	1e-14	1e-14
primesim_method	egear	egear	egear	egear	egear	egear

It is recommended that you use the `runlvl` option for high performance transient analysis with built-in RC reduction.

[Table 24](#) lists recommended `runlvl` settings for different kinds of analog circuits.

Table 24 General Guidance

Circuit	PLL (<3GHz)	PLL (>3GHz)	ADC (<12bits)	ADC (>12bits)	TX/RX (<5 GHz)	TX/RX (>5GHz)	PMIC
<code>runlvl</code>	3	4	4	5	3	4	2

PrimeSim Option Reference (Alphabetical List)

This section lists the PrimeSim options in alphabetical order.

primesim_ac_at_diff_t

Description

Note:

This option is for the PrimeSim SPICE tool only.

Controls whether the starting point of an AC analysis is a user-specified "non-zero" transient analysis timepoint (known as an "AC at Diff T" simulation).

If this option is set to 1 (default) and a transient analysis is run, when the user-specified transient timepoint is reached, an AC analysis is run concurrently with the transient analysis, with the AC analysis starting point being the node voltages of the user-specified transient timepoint. Multiple timepoints may be specified for multiple AC analyses.

However, If this option is set to 2, it allows multiple small-signal analyses to perform at specified transient time, including .AC, .LIN, .LSTB, and .NOISE. It also supports multiple parameter and monte-carlo sweeps. Note that .ALTER sweep is not supported.

Note:

- Noise, stability, and linear analyses are supported, as well as 1CPU or MCPU.
- Output filenames contain the timestamp in the suffix. Whether the analysis generates operating point (.op) or initial condition (.ic) files is determined using [primesim_gen_ac_op](#).
- Monte Carlo and sweep analyses are not supported with "AC at Diff T" simulations.
- Command-Line Option `-map_op` should not be used in an "AC at Diff T" simulation.

Syntax

```
.option primesim_ac_at_diff_t=[0|1|2]
```

Arguments

Argument	Description
0	Disables this feature.

Argument	Description
1	(default) Runs an AC analysis concurrently with a transient analysis at the user-specified transient timepoint.
2	Allows multiple small-signal analyses to perform at specified transient time, including <code>.AC</code> , <code>.LIN</code> , <code>.LSTB</code> , and <code>.NOISE</code> .

Examples

As shown in the following examples, user-specified transient timepoints are specified on one or more `.op` lines in SPICE syntax, or as parameter `actimes` in a 3rd-party netlist.

SPICE Syntax

```
.op t1 t2
.op tn
.tran tstep tstop
.ac type np fstart fstop
```

primesim_ac_flow

Description

Controls whether small signal AC analyses of 3rd-party netlists are run in a PrimeSim HSPICE-compatible flow or in a 3rd-party compatible flow (default). This option is ignored in small signal AC analyses of PrimeSim HSPICE netlists.

In a PrimeSim HSPICE-compatible flow, any small signal analysis requires an `.AC` statement which is the primary analysis. Other small signal analyses (for example, `.NOISE`, `.LSTB`) are considered secondary and run in conjunction with `.AC` for the frequency sweeps given in AC analysis.

In a 3rd-party compatible flow, each small signal analysis is considered an independent primary analysis with its own frequency sweep.

Depending on the types and number of AC analyses requested, overall simulation runtime between the flows may differ. If runtime is a concern, you can experiment with both flows to find out which best suits your needs.

Syntax

```
.option primesim_ac_flow=[0|1]
```

Arguments

Argument	Description
0	When AC and/or NOISE and/or LSTB analyses are run, the PrimeSim tool runs in PrimeSim HSPICE-compatible mode, which runs an AC analysis for each NOISE and LSTB analysis, and generates output files comparable to those produced by PrimeSim HSPICE.
1	(default) When AC and/or NOISE and/or LSTB analyses are run, the PrimeSim tool runs the analyses in 3rd-party compatible mode, which does <i>not</i> run an AC analysis for each NOISE and LSTB analysis, and generates output files comparable to those produced by the 3rd-party tool.

primesim_acc_sparse_fcap

Description

Specifies whether to selectively add back sparse coupling capacitors between the threshold specified by the `primesim_sparse_fcap` option and 1e-18.

Note:

If `primesim_sparse_fcap=1e-18` (or smaller), then this option does nothing.

Syntax

```
.option primesim_acc_sparse_fcap=[0|1|2]
```

Arguments

Argument	Description
0	Use the threshold specified by the <code>primesim_sparse_fcap</code> option and do not add back any sparse coupling capacitors. This setting has the best performance with a slight cost in accuracy. 0 is the default for PrimeSim Pro.
1	(default) Selectively add back sparse coupling capacitors between the threshold specified by the <code>primesim_sparse_fcap</code> option and 1e-18. This setting has the best accuracy with a slight cost in performance. 1 is the default for PrimeSim SPICE.
2	Selectively add back sparse coupling capacitors (fewer than setting 1) between the threshold specified by the <code>primesim_sparse_fcap</code> option and 1e-18. This setting has better accuracy than setting 0 and better performance than setting 1.

primesim_adaptive_mt

Description

Enables an optimized method for handling multi-threading on an overloaded machine.

Note:

This option is only supported by the PrimeSim SPICE tool.

Syntax

```
.option primesim_adaptive_mt=[0|1]
```

Arguments

Argument	Description
0	(default) A PrimeSim thread that completes its task actively checks and waits for its next task.
1	A PrimeSim thread that completes its task sleeps until awakened for its next task.

primesim_add_divider

Description

Defines a character to use as an additional hierarchy divider.

Syntax

```
.option primesim_add_divider=divider_character
```

Examples

This option is typically used when a flat DSPF with a different hierarchy separator than a period (.) is directly included as part of the netlist.

Consider the net name:

```
x1.x2.x3.x4/x5/net1
```

By default, the PrimeSim tool builds the hierarchy structure with three hierarchy levels:

```
x1 ® x2 ® x3 ® "x4/x5/net1"
```

If the following is specified:

```
.option primesim_add_divider="/"
```


Then the PrimeSim Pro tool considers the slash (/) character an additional hierarchy divider and builds the hierarchy structure with five hierarchy levels:

```
x1 @ x2 @ x3 @ x4 @ x5 @ "net1"
```

Note:

By default, there is no net named `net1`.

primesim_add_instance

Description

Automatically creates a subcircuit call line. If you do not specify *instance_name*, the PrimeSim tool automatically names it `x1`.

Syntax

```
.option primesim_add_instance=[instance_name:]subckt_name
```

Examples

```
.option primesim_add_instance="x1:inv"
```

or

```
.option primesim_add_instance="inv"
```

In the previous example, the PrimeSim tool makes a top-level instance `x1`, which comes from subcircuit `inv`.

primesim_aging_spfdivider

Description

Lets aging analysis provide a hierarchy divider in the `stressvec` SPF file.

Syntax

```
.option primesim_aging_spfdivider="character"
```

Examples

```
.option primesim_aging_spfdivider="."
```

Changes the divider from / to .

primesim_aginglib

Description

Loads an aging model shared library into the PrimeSim tool.

Syntax

```
.option primesim_aginglib="filename"
```

primesim_allow_3t_mosfet

Description

Allows three-terminal MOSFETs. The default value is 1. When `primesim_allow_3t_mosfet=1`, the PrimeSim tool continues simulation regardless of any MOSFETs with three terminals. When you set this option to 0, the PrimeSim tool stops the simulation when there is a MOSFET with three terminals.

Syntax

```
.option primesim_allow_3t_mosfet=[0|1]
```

primesim_allow_dup_element

Specifies whether duplicate instance definitions are allowed.

Syntax

```
primesim_allow_dup_element=[0|1]
```

Arguments

Argument	Description
0	Default value; errors out when instance or device is duplicated.
1	Displays warning when instance or device is duplicated and takes the last one.

Description

When set to 1, the simulator allows duplicate instance or device or any element definitions. However, it honors only the last one and prints the warning message in the log file.

primesim_allow_dup_port

Description

Allows duplicated ports. The default value is 0 and the PrimeSim tool stops the simulation when there is a duplicated port. When you set this option to 1, the PrimeSim tool continues simulation regardless of duplicate ports.

Syntax

```
.option primesim_allow_dup_port=[0|1]
```

primesim_analog

Provides a turnkey accuracy setting to run large post-layout analog centric circuits in PrimeSim Pro.

Note:

This option is supported only in `.option primesim_mode=prohd`.

Syntax

```
.option primesim_analog=[0|1]
```

Arguments

Argument	Description
0	(default) This option is not enabled and has no effect on the simulation.
1	Enables the accuracy setting for large postlayout analog centric circuit simulations in <code>prohd</code> mode.

primesim_auto_gmin_scale

Description

When set to 1 (default) the PrimeSim tool automatically scales down by 1e-3 any values for `gmin` and `gmindc` when simulating FinFET designs. For example, if `gmin=1e-12`, then FinFET simulation uses `gmin=1e-15`.

When set to 0, `gmin` auto-scaling is disabled and the user-defined `gmin/gmindc` values are kept as-is for FinFET simulation.

Syntax

```
.option primesim_auto_gmin_scale=[0|1]
```

Arguments

Argument	Description
0	Disables automatic scaling of gmin and gmindc for FinFET models.
1	(default) gmin/gmindc is scaled down by 1e-3 for FinFET models.

primesim_auto_restart

Description

Allows you to create snapshots during transient simulation that can be used in conjunction with the `-auto_restart` command-line option (see [Command-Line Options](#)) to skip all parsing and elaboration when:

- Restarting a simulation job that ended unexpectedly
- Extending a simulation job beyond the originally defined stop time (`-stop tstop`)

For more information, see [Restarting Transient Simulations From Snapshots](#).

Syntax

```
.option primesim_auto_restart="mode percent"  
.option primesim_auto_restart="mode elapsed_time"
```

Arguments

Argument	Description
<i>mode</i>	Specifies the mode for creating simulation snapshots. Valid values are: • 1: Creates OP-based snapshots. When you restore using mode 1 snapshots, the PrimeSim tool still goes through input parsing. • 2: Saves the entire simulation state in memory as snapshots. With this mode, when you restore the simulation, the PrimeSim tool skips all parsing and elaboration and starts directly from the last saved state during transient. • 3: Same as mode 2 except generates another snapshot at the end of simulation which you can use to extend the simulation <i>tstop</i> time. • 4: Same as mode 2 except generates snapshots at the specified <i>elapsed_time</i> interval.
<i>percent</i>	Used with modes 1, 2, and 3 only. Specifies how often the snapshots are saved as a percentage of <i>tstop</i>. For example, a value of 20 saves the simulation state at every 20% of <i>tstop</i>. Note: Whenever the tool saves a snapshot, it removes any previous snapshots so that only the latest one is available for restoring.

Argument	Description
<i>elapsed_time</i>	Used with mode 4 only. Similar to <i>percent</i> , but generates snapshots at the elapsed time (or wall/real time) interval instead. Time can be specified with s/m/h as unit.

primesim_bisection_concurrent_mode

Description

Controls the behavior of bisection optimization when performing [Concurrent Bisection for Independent Circuit Blocks](#).

Syntax

```
.option primesim_bisection_concurrent_mode=[0|1]
```

Arguments

Argument	Description
0	(default) Does not treat bisections independently; that is, a successful iteration requires that all bisections are successful.
1	Treats bisections independently; that is, only the last iteration must ensure that all bisections are successful for all measurements and criteria. More iterations may be added for this purpose.

primesim_bisection_fail_continue

Description

When set to 1, this option allows the simulation to continue to other sweeps outside the bisection in the event that the bisection sweep fails.

The default is 0, which ends the simulation if the bisection sweep fails.

Syntax

```
.option primesim_bisection_fail_continue=[0|1]
```

primesim_bisection_output

Description

Controls the measure file and waveform database output for each bisection sweep. The default value is 1.

Syntax

```
.option primesim_bisection_output=[0|1|2]
```

Arguments

Argument	Description
0	No bisection sweep measurement is output. Only the final waveform is saved.
1	(default) Each bisection sweep measurement is lumped into a single mt0 file. Only the final waveform is saved.
2	Each bisection sweep measurement is lumped into a single mt0 file. Each bisection sweep waveform is saved with the prefix extension <code>_bsX</code> .

primesim_bisection_summary

Description

Controls the bisection summary file `.bisect_mt0`, which contains the final measurement data of the bisection sweep.

Syntax

```
.option primesim_bisection_summary=[0|1|2|3]
```

Arguments

Argument	Description
0	Does not generate a bisection summary.
1	(default) Generates one <code>bisec_mt0</code> file for the whole simulation (<code>alter/temp/sweep</code>).
2	Generates one summary <code>bisec_mt0</code> file for each <code>alter/temp</code> .
3	Generates one summary <code>bisec_mt0</code> file for each <code>sweep</code> .

primesim_bsim3gate_leakage

Description

Applies the `gidl/gisl/gate` tunneling current model from the BSIM4 implementation into BSIM3. By default (0), the tunneling current model is disabled. When set to 1, the PrimeSim tool uses the BSIM4 equation for `gidl/gisl/gate` tunneling current and applies it to the BSIM3 model.

Syntax

```
.option primesim_bsim3gate_leakage=[0|1]
```

primesim_bytol

Sets the voltage value to use in determining whether a MOS is latent.

Syntax

```
.option primesim_bytol=voltage_value
```

Description

A MOS transistor is latent when its terminal voltages are stable within a defined limit. Model evaluation can be skipped for these transistors. The default for the PrimeSim SPICE tool and the PrimeSim Pro tool's `spicehd` mode is 0.1mV. The default of the other SPICE and Pro modes is 1mV.

Model evaluation bypass is disabled for DC/AC analyses. It is enabled only during transient analysis.

Examples

```
.option primesim_bytol=0.05mV  
.option primesim_bytol=0
```

In the first example, a MOS transistor is latent if its terminal voltages vary less than 0.05mV and no model evaluation is performed under this voltage limit.

In the second example, no MOSFETs are latent because no voltage variation is smaller than 0. This means that all models are reevaluated.

primesim_c_model

Description

The C-model code is a set of C-function APIs which allows you to code-up behavioral models using provided types and functions. It allows you to define a model with ports and assign/obtain analog voltages or digital values to those ports.

Syntax

```
.option primesim_c_model="model_file_name"
```

primesim_c_model_exclude

Description

Excludes some instances of C-models from being replaced with the model that was created. This option is intended to be used where multiple instances of the same C-model would normally all be replaced with the model. Using the exclusion option allows some instances to remain as transistor-level instances.

Syntax

```
.option primesim_c_model_exclude="inst1 inst2 ..."
```

Examples

```
.option primesim_c_model_exclude="x1 x3"
```

In this example, the instances `x1` and `x3` remain at the transistor level and are not replaced with the C-model.

primesim_capmin

Description

Specifies a threshold at which linear capacitors are ignored. The default is 0 (disabled).

Syntax

```
.option primesim_capmin=value
```

Examples

In the following example, capacitors less than 10fF are removed from the circuit.

```
.option primesim_capmin=10fF
```

primesim_case_sync_in_subckt

Description

Resolves potential error for mismatched node names caused by mixed case-sensitivity within subcircuit.

With this option enabled, if the subcircuit is case-sensitive and some parts of the subcircuit are insensitive, all insensitive parts are changed to case-sensitive. Conversely, if the subcircuit is case-insensitive and some parts of the subcircuit are sensitive, all sensitive parts are changed to case-insensitive.

Syntax

```
.option primesim_case_sync_in_subckt=[0|1]
```

Arguments

Argument	Description
0	(default) Disables the option.
1	Enables the option.

Examples

For example, with the `primesim_case_sync_in_subckt` option enabled, statement (3) moves to the "ends", and the entire subcircuit becomes case-sensitive, given in statement (1).

```
(1) simulator lang=spectre insensitive=no
(2) subckt sub (a b c)
....
(3) simulator lang=spectre insensitive=yes
....
(4) ends
```

primesim_cbmodel

Description

Controls PrimeSim behavior at the channel boundaries of partitions.

Syntax

```
.option primesim_cbmodel=[1|2]
```

Arguments

Argument	Description
1	(default) Specifies the default way of handling Miller capacitance effects at the partition boundary.
2	Detects events more effectively at channel boundaries, especially when the driving strength of a MOSFET is weak because of low VDD.

primesim_cck_post

Description

Sets all three post-processing options—[primesim_chkdevop_post](#), [primesim_chkrftime_post](#), and [primesim_chktiming_post](#)—with a single command.

Note:

The individual post-processing options can override this option.

Syntax

```
.option primesim_cck_post=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables post-processing and performs on-the-fly analysis.
1	Performs post-processing right after finishing transient analysis.
2	Creates a waveform for post-processing, then terminates with a normal completion message.

primesim_cck_remove_result

Description

When set to 1, the PrimeSim tool removes the circuit check result file when there are no errors. The default setting is 0.

The following circuit checks are supported:

- [Check DC Path \(.CHKDCPATH\)](#)
- [Check Device Current \(.CHKDEVCUR\)](#)
- [Check Device Operation Point \(.CHKDEVOP\)](#)
- [Check Expression \(.CHKEXPR\)](#)
- [Check OFF Device Leakage Current \(.CHKOFFDEVLEAK\)](#)
- [Check Rise/Fall Transition Time \(.CHKRFTIME\)](#)
- [Check Signal Voltage Difference \(.CHKSIGDIFF\)](#)
- [Static Circuit Checks \(.CHKSTATICERC\)](#).

- [Check Timing Setup/Hold/Delay/Width \(.CHKTIMING\)](#)
- [Check High Impedance State Node \(.CHKZNODE\)](#)

Syntax

```
.option primesim_cck_remove_result=[0|1]
```

primesim_cck_report_limit

Controls the maximum number of error reports generated for each file.chkdevop file during simulation.

Syntax

```
.option primesim_cck_report_limit=value
```

Description

If the number of errors exceeds the specified limit, subsequent errors are not reported and a message indicating the limit set by the option is added to the file.chkdevop file.

Examples

```
primesim_cck_report_limit=5
```

This setting limits the number of error reports for each file.chkdevop file to 5. If more than 5 errors are detected, only the first 5 are reported, and a message indicating the limit being reached is added to the file.

primesim_ccmodel

Controls the modeling of coupling capacitors based on the total coupling capacitance ratio.

Syntax

```
.option primesim_ccmodel=[0|value]
```

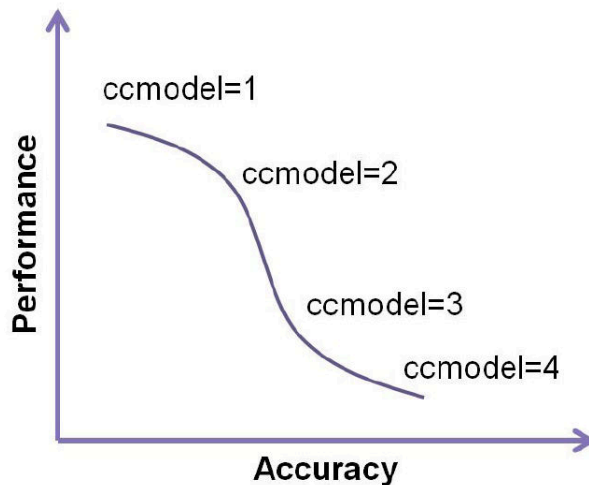
Description

You can adjust this setting to gain a speed-up for circuits with a large number of capacitors. Setting value to 1 yields the best performance, while 4 gives the best accuracy. The default is 4 in PrimeSim Pro and value 0 disables the option.

By default, the existing option [primesim_evlevel](#) is ignored and this option will be obsoleted in the future release. If the option [primesim_fcmodel](#) is given specifically, then the default [primesim_ccmodel](#) is turned off. However, when you set value to any number between 1 and 4 specifically, then the existing [primesim_fcmodel](#) is ignored.

However, the performance/accuracy trade-off in this range is not always linear. For example, while setting 4 gives the best accuracy, the performance may be close to setting 3. The same thing happens with the performance: setting 2 may give the same performance but with better accuracy compared to setting 1.

This relationship is illustrated in the following figure.



Furthermore, *value* is not limited to an integer; you can set a floating number such as 3.5 to get the best optimization in your design. Setting *value* to 2.5 or 3 is a good starting point to apply.

The existing `primesim_fcmodel` and `primesim_evlevel` is based on an individual capacitor's value which is affected by the extraction setting or RC reduction. The effect of coupling capacitors should be considered as its importance is not degraded even though split to smaller capacitors.

The `primesim_ccmodel` option considers the total coupling capacitance between NETs and also the ratio of total NET capacitance including a device's pin capacitance. Because even when total coupling capacitance is small, if the ratio is high, then the coupling effect is more significant.

Note:

The option is applicable for PrimeSim Pro mode only.

primesim_cell_char

Enhances performance when using the PrimeSim tool for cell characterization.

Syntax

```
.option primesim_cell_char=[0|1|2]
```

Arguments

Argument	Description
0	(default) No enhanced cell characterization support.
1	Suppresses printing the sweep parameters (input values) to the <code>.mt0</code> file. In addition, the PrimeSim tool does not output any data to the log file after the first DC and then only updates the log file at the end of the simulation.
2	Same as 1, but also does some additional performance optimization in the circuit elaboration and the matrix solver for transient sweep or data sweep analysis.

Description

An additional accuracy level option, `runlvl`, is supported for use with the `primesim_cell_char` option.

primesim_change_r3cmc_pin_order

Description

Specifies the pin order to use when assigning pins for the CMC R3 model. Typically, the CMC R3 model definition pin order is as follows:

```
Rxxx n1 nc n2 mname [M=val] instanceParameters
.MODEL mname r LEVEL=5 [keyword=val] ...
```

However, the StarRC and IC Validator tools allow for a swapped pin order in the extracted netlist by the equivalence of resistor pins for LVS, so the resistors are defined as follows:

```
Rxxx n1 n2 nc mname
```

By default (0), the PrimeSim tool uses the `n1 nc n2` pin order. If this option is set to 1, the PrimeSim tool assigns the pins using the `n1 n2 nc` pin order.

Syntax

```
.option primesim_change_r3cmc_pin_order=[0|1]
```

primesim_check_dummy_mos_size

Description

When set to 1, the PrimeSim tool skips shorting dummy MOSFETs during parsing and checks their size during elaboration for violations. This option can be useful when you

want to check the size of spare devices that can be used later in the design cycle. The default is 0.

Syntax

```
.option primesim_check_dummy_mos_size=[0|1]
```

primesim_check_model

Description

Determines whether RC models exist. The default value is 0, which means the PrimeSim tool does nothing for "model not found RC" devices. If it is set to 1, then the PrimeSim tool checks the proper model for RLC and terminates if it fails.

Syntax

```
.option primesim_check_model=[0|1]
```

primesim_check_undefined_param

Description

Specifies whether to check for undefined parameters in the netlist and then, if found, exit the simulation before circuit elaboration or SPF back-annotation.

Syntax

```
.option primesim_check_undefined_param=[0|1]
```

Arguments

Argument	Description
0	(default) Does not check for undefined parameters in the netlist before circuit elaboration or SPF back-annotation.
1	Checks for undefined parameters in the netlist and then, if found, exit the simulation before circuit elaboration or SPF back-annotation.

primesim_check_vth

Description

This option is used for [Check High Impedance State Node \(.CHKZNODE\)](#) to determine whether `primesim_goff`, Vth or operating region is used for determining the ON/OFF state of the transistor. The default is 1, which uses the Vth method.

Syntax

```
.option primesim_check_vth=[0|1|2]
```

Arguments

Argument	Description
0	primesim_goff (Off : Gds < primesim_goff).
1	(default) Vth (Determines the state based on primesim_chkznnode_rule).
2	Determines the off state only when the MOSFET is in the cut-off region.

primesim_checksum_error_exit

Description

If MD5 checksum error checking is enabled for input files (see [primesim_checksum_on](#)), this option specifies whether to exit simulation in the event of a checksum error.

Syntax

```
.option primesim_checksum_error_exit=[0|1]
```

Arguments

Argument	Description
0	If a checksum error occurs, the simulation run continues and ignores any changed vectors.
1	(default) If a checksum error occurs, the simulation exits with a checksum error message.

primesim_checksum_on

Description

Specifies whether to enable MD5 checksum error checking for input files.

Note:

This option must be enabled during an initial `.restart_db` file generation run for it to be valid during restart runs. See [Restarting Simulations With Vector Changes](#) for more information.

Syntax

```
.option primesim_checksum_on=[0|1]
```

Arguments

Argument	Description
0	(default) Disables MD5 checksum error checking.
1	Enables MD5 checksum error checking.

primesim_chk_devport

Checks the device port name during wildcard matching in the `.PROBE` statement.

Syntax

```
.option primesim_chk_devport=[0|1]
```

Description

When this option is set to 0 (which is the default), the PrimeSim Pro tool disables wildcard matching for device port names and considers the port names which have been specifically named. When this option is set to 1, the PrimeSim Pro tool allows wildcard matching.

Examples

```
.option primesim_chk_devport=1
.probe v(xil.xg.xgperm_top.xpd38<0>.xpdlat<6>.xmm7.mtp:g)
```

Where `g` means the second terminal of the MOSFET, regardless of the actual pin name.

primesim_chk_disk_space

Description

Set this option to check available disk space at regular intervals.

If the PrimeSim Pro tool detects that there is not sufficient disk space to write waveform data, it terminates the simulation with a warning message.

Syntax

```
.option primesim_chk_disk_space=[0|1|2|3]
```


Arguments

Argument	Description
0	(default) No disk space checking.
1	Checks available disk space every 10% of transient by default or every <code>primesim_tflush</code> period. If the available disk space is less than 10MB or the space decreased by the previous step, the simulator exits.
2	Checks available disk space once per minute (required free disk space is 1GB). If the available disk space is less than 1GB, the simulator waits until at least 1GB is available. While waiting, the simulator releases its license and checks available disk space every two minutes.
3	Checks available disk space once per minute against a user-specified minimum. If the available disk space is less than user-specified minimum, the simulator waits until at least the user-specified minimum is available. While waiting, the simulator releases its license and checks available disk space at a user-specified interval. For option 3, the user-specified minimum disk space is set with the <code>primesim_chk_disk_min_space=value</code> option (default: 1 GB) and the user-specified wait interval is set with the <code>primesim_chk_disk_sleep_time=value</code> option (default: 5 minutes) Note: When <code>primesim_chk_disk_space=3</code> , the default for <code>primesim_chk_disk_sleep_time</code> should be 3 minutes.

primesim_chk_format

When set to 1, PrimeSim `.CHKDEVOP` commands use the PrimeSim XA Tcl-CCK output format. CustomExplorer Ultra can then be used to analyze the report. The default is 0.

Syntax

```
.option primesim_chk_format=[0|1]
```

Description

The following examples show the output of a `.CHKDEVOP` command:

- When `.option primesim_chk_format=0`, the output is:

```
*[device-name] [tag/type] [begin_time] [end_time] [period] [min/max/terminal V]
ma_n_1         vgs          0.0000e+00  1.0000e-09  1.0000e-09  vgs=4 (at=150ps)
```

- When `.option primesim_chk_format=1`, the output is:

```
Dev #1: ma_n_1, in [0, 1n]
Attributes: L=2e-06, W=3e-06, Peak terminal voltage = 4V ( vgs at 150ps)
Connectivity:
D: vdd3
G: vdd7
```

```
S: nal  
B: vss
```

primesim_chk_fsdb

Description

This option prevents overwriting FSDB files. The default value is 0. If set to 1, the PrimeSim tool generates an `input_machineName_processID.fsdb` file.

Note:

This option is only usable when the command-line option `-lock` is set to 0. If `-lock` is set to 1 (default) or 2, the `primesim_chk_fsdb` option is ignored. See [Table 2](#) for more information.

Syntax

```
.option primesim_chk_fsdb=[0|1]
```

primesim_chkblkpwr_pwrnode

Description

Marks all specified nodes as supply nodes in the power report (see [Check Block Power \(.CHKBLKPWR\)](#)). This option supports wildcards.

Syntax

```
.option primesim_chkblkpwr_pwrnode="node1 node2 ..."
```

primesim_chkblkpwr_pwrport

Defines the power port of all subcircuits (see [Check Block Power \(.CHKBLKPWR\)](#)).

Syntax

```
.option primesim_chkblkpwr_pwrport="port1 port2 ..."
```

Description

This is a global option that applies to all `.chkblkpwr` commands. The defined power port is treated as a supply node. Wildcards are also supported for defining power ports.

Examples

```
.option primesim_chkblkpwr_pwrport="port1 port2 ..."
```

The previous example marks all port names matching these patterns as supply nodes for all subcircuits.

primesim_chkblkpwr_specified_only

Description

The `.CHKBLKPWR` command reports block power based on auto-detection of power/ground sources (see [Check Block Power \(.CHKBLKPWR\)](#)). If there is an internal power/ground or port, then you can define that using the `primesim_chkblkpwr_pwrnode` or `primesim_chkblkpwr_pwrport` options, respectively.

If you define some node using the `primesim_chkblkpwr_pwrnode` option and want to get the result only from that node, then set the `primesim_chkblkpwr_specified_only` option to 1. The PrimeSim tool reports block power/ground only for the specified node. The default is 0.

Syntax

```
.option primesim_chkblkpwr_specified_only=[0|1]
```

primesim_chkdcpath_gndgroup

Description

If set to 1, any node connected to an ideal ground (zero) voltage source through a zero voltage source is treated as the same ground group by the `.CHKDCPATH` command (see [Check DC Path \(.CHKDCPATH\)](#)).

By default (0), such nodes are treated separately in DC Path searching.

Syntax

```
.option primesim_chkdcpath_gndgroup=[0|1]
```

Examples

See the detailed example for `primesim_chkdcpath_pwrgroup`.

primesim_chkdcpath_pwrgroup

Description

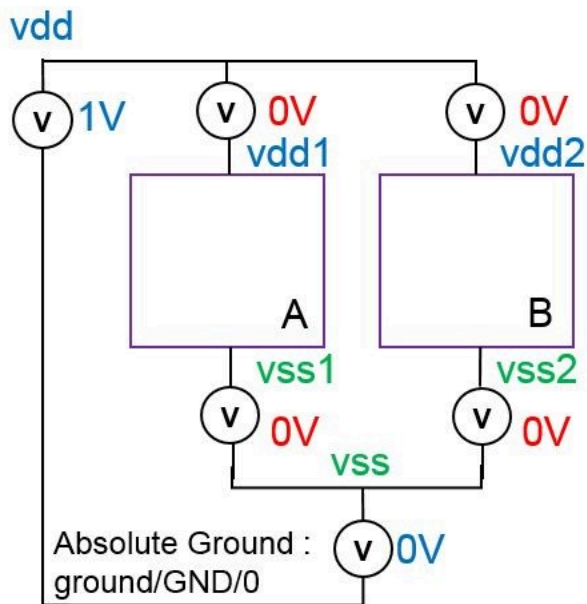
If set to 1, any node connected to an ideal power (non-zero) voltage source through a zero voltage source is treated as the same power group by the `.CHKDCPATH` command (see [Check DC Path \(.CHKDCPATH\)](#)).

By default (0), such nodes are treated separately in DC Path searching.

Syntax

```
.option primesim_chkdcpath_pwrgroup=[0|1]
```

Examples



In this example, if you set `primesim_chkdcpath_pwrgroup=1`, then you do not have to define `vdd1` and `vdd2` for the `.CHKDCPATH` command and can just use `vdd`.

```
.chkdcpath name="vdd vss1" ...  
.chkdcpath name="vdd vss2" ...
```

Similarly, if you also set `primesim_chkdcpath_gndgroup=1`, then you do not have to define `vss1` and `vss2` for the `.CHKDCPATH` command and can just use `vss`.

```
.chkdcpath name="vdd vss"
```

primesim_chkdcpath_rth

Description

If a resistor value is less than the specified value (default=1k), it is considered a parasitic resistor and part of the resistor network. That individual resistor is not printed in the DC Path result (see [Check DC Path \(.CHKDCPATH\)](#)). Instead, the PrimeSim tool prints out:

```
----R Network----
```

If it is connected to an ideal power/ground which is the end point in the DC Path, then the PrimeSim tool treats the resistor network as the end point, that is, the same as Power/Ground in DC Path searching.

If a resistor value is greater than or equal to the specified value, that resistor is treated as a normal device and each side node is treated as a different resistor network. That resistor is printed in the DC Path result, and not as a part of the resistor network.

Syntax

```
.option primesim_chkdcpath_rth=value
```

primesim_chkdevcur_report

Description

If set to 1 (default), the `.CHKDEVCUR` command generates two log files: the main file that reports violations on every time step, and the summary file that reports the average (by default) or RMS (root-mean-square) of the window-based violations per device (see [Check Device Current \(.CHKDEVCUR\)](#)).

The main file can get very large, so you can set this option to 0 to generate only the summary file.

Syntax

```
.option primesim_chkdevcur_report=[0|1]
```

primesim_chkdevop_disable

Description

Disables `.chkdevop` for those circuits identified with the specified `tag=tag_name` arguments (see [Check Device Operation Point \(.CHKDEVOP\)](#)) and enables `.CHKDEVOP` for all other circuits including those without a `tag` argument.

Note:

Using this option and `primesim_chkdevop_enable` results in an error.

Syntax

```
.option primesim_chkdevop_disable="tag_name1 tag_name2 ..."
```

primesim_chkdevop_enable

Description

Enables `.chkdevop` for those circuits identified with the specified `tag=tag_name` arguments (see [Check Device Operation Point \(.CHKDEVOP\)](#)) and disables `.CHKDEVOP` for all other circuits including those without a `tag` argument.

Note:

Using this option and `primesim_chkdevop_disable` results in an error.

Syntax

```
.option primesim_chkdevop_enable="tag_name1 tag_name2 ..."
```

primesim_chkdevop_filter_in

Description

Controls which devices to include under the specified subcircuit's instances and the specified instances when for checking device operation condition (see [Check Device Operation Point \(.CHKDEVOP\)](#)).

Syntax

```
.option primesim_chkdevop_filter_in="subckt=sname instance=iname  
device=dname"
```

Arguments

Argument	Description
<code>subckt=sname</code>	Specifies the subcircuits to include.
<code>instance=iname</code>	Specifies the instances to include.
<code>device=dname</code>	Specifies the devices to include.

primesim_chkdevop_filter_out

Description

Controls which devices to exclude under the specified subcircuit's instances and the specified instances when for checking device operation condition (see [Check Device Operation Point \(.CHKDEVOP\)](#)).

Syntax

```
.option primesim_chkdevop_filter_out="subckt=sname instance=iname  
device=dname"
```

Arguments

Argument	Description
<code>subckt=sname</code>	Specifies the subcircuits to exclude.
<code>instance=in ame</code>	Specifies the instances to exclude.
<code>device=dname</code>	Specifies the devices to exclude.

primesim_chkdevop_mode

Description

By default (1), when `subckt=subckt_name` is specified in `.chkdevop` (see [Check Device Operation Point \(.CHKDEVOP\)](#)), the PrimeSim tool checks devices below the hierarchy of the subckt.

Set this option to 0 to not check lower hierarchies of the subckt for `.CHKDEVOP` when `subckt=subckt_name` is specified. That is, the PrimeSim tool checks only the specified scope of the subckt.

Note:

The default for this option was 0 prior to the P-2019.06 version, and then removed and the behavior was the same with `primesim_chkdevop_mode=1`. As of the Q-2020.03 version, this option is re-activated and the default is 1.

Syntax

```
.option primesim_chkdevop_mode=[0|1]
```

primesim_chkdevop_post

Description

Allows you to post-process the `.CHKDEVOP` command (see [Check Device Operation Point \(.CHKDEVOP\)](#)). Through this feature, you get some performance enhancement of the transient simulation and some level of freedom to modify the condition of the `.CHKDEVOP` command.

Syntax

```
.option primesim_chkdevop_post=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables post-processing and performs on-the-fly analysis.
1	The PrimeSim tool performs post-processing right after finishing transient analysis.
2	The PrimeSim tool creates a waveform for post-processing, then terminates with a normal completion message.

Examples

```
.option primesim_chkdevop_post=1
```

In this example, the PrimeSim tool runs the simulation without performing `.CHKDEVOP` analysis during the transient simulation. After the transient simulation is finished, the PrimeSim tool starts post-processing for the `.CHKDEVOP` command.

Note:

The netlist must include a `.CHKDEVOP` statement for this option to have any effect.

```
.option primesim_chkdevop_post=2
```

In this example, the PrimeSim tool runs the simulation without performing `.CHKDEVOP` analysis during the transient simulation and has normal termination without performing `.CHKDEVOP`, then outputs the `.CHKDEVOP` data into a waveform file.

Note:

The netlist must include a `.CHKDEVOP` statement for this option to have any effect.

You can recheck the `.CHKDEVOP` command with the `-rchk` command (see [Table 2](#)).

primesim_chkdevop_print_info

Description

When set to 1, the PrimeSim tool appends W,L,m device information to the `.CHKDEVOP` violations report. The default is 0.

Syntax

```
.option primesim_chkdevop_print_info=[0|1]
```

primesim_chkdgt

Description

Sets the number of digits displayed after the decimal by the `.CHKRFTIME` and `.CHKTIMING` commands. The default is 4.

Syntax

```
.option primesim_chkdgt=numdgt
```

Examples

For example, the following output displays the default four digits after the decimal:

```
sig=v(out) Tf = (1.3897e-08 - 1.1463e-08) = 2.434e-09, limit = 1e-10
```

If the following option is set:

```
.option primesim_chkdgt=8
```

Then in this example, the output displays eight digits after the decimal:

```
sig=v(out) Tf = (1.38970000e-08 - 1.14630000e-08) = 2.43400000e-09, limit  
= 1.00000000e-10
```

primesim_chkexpr_disable

Description

Disables `.CHKEXPR` for those circuits identified with the specified `tag=tag_name` arguments (see [Check Expression \(.CHKEXPR\)](#)) and enables `.CHKEXPR` for all other circuits including those without a `tag` argument.

Wildcard expansion is not supported so you must fully specify each `tag_name`. If you use this option multiple times, the `tag_names` are appended.

Note:

Using this option and `primesim_chkexpr_enable` results in an error.

Syntax

```
.option primesim_chkexpr_disable="tag_name1 tag_name2 ..."
```

primesim_chkexpr_enable

Description

Enables `.CHKEXPR` for those circuits identified with the specified `tag=tag_name` arguments (see [Check Expression \(.CHKEXPR\)](#)) and disables `.CHKEXPR` for all other circuits including those without a `tag` argument.

Wildcard expansion is not supported so you must fully specify each `tag_name`. If you use this option multiple times, the `tag_names` are appended.

Note:

Using this option and `primesim_chkexpr_disable` results in an error.

Syntax

```
.option primesim_chkexpr_enable="tag_name1 tag_name2 ..."
```

primesim_chkrftime_post

Description

Allows you to post-process `.CHKRFTIME` (see [Check Rise/Fall Transition Time \(.CHKRFTIME\)](#)). Through this feature, you get some performance enhancement of the transient simulation and some level of freedom to modify the condition of `.CHKRFTIME`.

Syntax

```
.option primesim_chkrftime_post=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables post-processing and performs on-the-fly analysis.
1	The PrimeSim tool performs post-processing right after finishing transient analysis.
2	The PrimeSim tool creates a waveform for post-processing, then terminates with a normal completion message.

Examples

```
.option primesim_chkrftime_post=1
```

In this example, the PrimeSim tool runs the simulation without performing `.CHKRFTIME` analysis during the transient simulation. After the transient simulation is finished, the PrimeSim tool starts post-processing for the `.CHKRFTIME` command.

Note:

The netlist must include a `.CHKRFTIME` statement for this option to have any effect.

```
.option primesim_chkrftime_post=2
```

In this example, the PrimeSim tool runs the simulation without performing `.CHKRFTIME` analysis during the transient simulation and has normal termination without performing `.CHKRFTIME`, then dumps out the `.CHKRFTIME` data into a waveform file.

Note:

The netlist must include a `.CHKRFTIME` statement for this option to have any effect.

You can re-check the `.CHKRFTIME` command with the `-rchk` command (see [Table 2](#)).

primesim_chktiming_post

Description

Allows you to post-process `.CHKTIMING` (see [Check Timing Setup/Hold/Delay/Width \(.CHKTIMING\)](#)). Through this feature, you get some performance enhancement of the transient simulation and some level of freedom to modify the condition of `.CHKTIMING`.

Syntax

```
.option primesim_chktiming_post=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables post-processing and performs on-the-fly analysis.
1	The PrimeSim tool performs post-processing right after finishing transient analysis.
2	The PrimeSim tool creates a waveform for post-processing, then terminates with a normal completion message.

Examples

```
.option primesim_chktiming_post=1
```

In this example, the PrimeSim tool runs the simulation without performing `.CHKTIMING` analysis during the transient simulation. After the transient simulation is finished, the PrimeSim tool starts post-processing for the `.CHKTIMING` command.

Note:

The netlist must include a `.CHKTIMING` statement for this option to have any effect.

```
.option primesim_chktiming_post=2
```

In this example, the PrimeSim tool runs the simulation without performing `.CHKTIMING` analysis during the transient simulation and has normal termination without performing `.CHKTIMING`, then outputs the `.CHKTIMING` data into a waveform file.

Note:

The netlist must include a `.CHKTIMING` statement for this option to have any effect.

You can re-check the `.CHKTIMING` command with the `-rchk` command (see [Table 2](#)).

primesim_chkznnode_debug_out

Description

Specifies that device turn-on/off information be printed out for Hi-Z node debugging (see [Check High Impedance State Node \(.CHKZNODE\)](#)).

The output file name is `output_prefix.chkzdebug`.

Syntax

```
.option  
primesim_chkznnode_debug_out="subckt_name1 subckt_name2 instance_name1 in  
stance_name2 ..."
```

primesim_chkznnode_rule

Description

Specifies a global option for checking a Hi-Z node (see [Check High Impedance State Node \(.CHKZNODE\)](#)). This rule is based on the conductive state of the device and has been enhanced to reduce misjudgments and detect potential issues.

Syntax

```
.option primesim_chkznnode_rule=[0|1|2]
```

Arguments

Argument	Description
0	ON state @ NMOS: $V_{gs} > V_{th}$ ON state @ PMOS: $V_{gs} < V_{th}$
1	ON state @ NMOS: $(V_{gs} > V_{th}) \parallel ((V_{ds} < V_{th}) \&\& (V_g > V_{th}))$ ON state @ PMOS: $(V_{gs} < V_{th}) \parallel ((V_{ds} < -V_{th}) \&\& (V_g < -V_{th}))$
2	(default) DC voltage comparison based on circuit topology applied to diode-connected MOS. <pre> if (bias can be determined based on topology @DioConMos) Forward bias: ON Reverse bias: OFF else ON state @ NMOS: $(V_{gs} > V_{th}) \parallel ((V_{ds} < V_{th}) \&\& (V_g > V_{th})) \parallel (V_d \geq V_s \text{ @DioConMos})$ ON state @ PMOS: $(V_{gs} < V_{th}) \parallel ((V_{ds} < -V_{th}) \&\& (V_g < -V_{th})) \parallel (V_d \leq V_s \text{ @DioConMos})$ </pre>

primesim_chkznnode_vth

Used for [Check High Impedance State Node \(.CHKZNODE\)](#) to set the V_{th} value in calculating the ON/OFF state of MOSFET.

Syntax

```
.option primesim_chkznnode_vth=value
```

Description

By default, the PrimeSim tool automatically determines this value by calculating the V_{th} of the device. When you specify this option, MOSFET is considered off when $|V_{gs}|$ is below the `primesim_chkznnode_vth` value.

Examples

```
.option primesim_chkznnode_vth=0.3
```

The previous example sets the V_{th} value to 0.3.

primesim_cktpt

Description

Applies circuit optimization.

Note:

Setting `primesim_cktopt=0` slows down performance.

Syntax

```
.option primesim_cktopt=[0|1]
```

Arguments

Argument	Description
0	Disables circuit optimization.
1	(default) Enables circuit optimization.

primesim_clampVerilog

When you enable conductance clamping by setting the option `primesim_clampVerilog=1` the PrimeSim Pro tool clamps the stamped conductance value (dI/dV) to `primesim_gmax` (which defaults to 100).

Syntax

```
.option primesim_clampVerilog=[0|1]
```

Description

The PrimeSim Pro tool issues a warning message if the conductance value exceeds `primesim_gmax`. By default, the `primesim_clampVerilog` option is enabled (1) and the PrimeSim Pro tool always clamps `g`, even when it does not issue the warning message.

Given the flexibility provided by Verilog-A, it is possible to create models containing terms whose derivatives are singular (infinite), and used as conductance values. the PrimeSim Pro tool detects this and may issue a warning, as follows:

Examples

```
WARNING! g(1.000000e+12) across nodes: (abc, GND) is larger than
gmax(1.000000e+02), in instance 'x' of model 'my_model'
-----> Port #1= 0.000000e+00
-----> Port #2= 6.605598e-08
-----> Port #3= 6.605598e-08
-----> Port #4= 1.981679e-07
-----> Port #5= 1.981679e-07
-----> Port #6= 1.981679e-07
```

Note:

The PrimeSim Pro tool only prints a few such warnings, to prevent unreasonable growth of the log file.

Note:

The `primesim_clampVerilog` option is only available in cVA flow. Set `.option primesim_pva_flow = 0` to enable the cVA flow.

primesim_connect_mode

Description

Specifies what the `.CONNECT` PrimeSim HSPICE-compatible command uses to connect two nodes.

Syntax

```
.option primesim_connect_mode=[0|1]
```

Arguments

Argument	Description
0	<code>.CONNECT</code> connects two nodes with a 0-ohm resistor.
1	(default) <code>.CONNECT</code> connects two nodes with a 0-volt DC source.

primesim_consistent_vloop_check

Description

Enables to configure how the simulation behaves when PrimeSim encounters a consistent voltage loop.

Syntax

```
.option primesim_consistent_vloop_check=[0|1]
```

Arguments

Argument	Description
0	When set to default 0, PrimeSim issues warning message and proceeds with the simulation. For example, if the sum of all voltage sources in a loop adds up to 0V, PrimeSim issues a warning and continue with the simulation.

Argument	Description
1	Forces PrimeSim to stop the simulation when a consistent voltage loop is detected.

primesim_cont_measout

Description

Controls the *continuous* measurement file format.

Note:

For the measurement file format, see [primesim_measout](#).

Syntax

```
.option primesim_cont_measout=[0|1]
```

Arguments

Argument	Description
0	(default) Use PrimeSim format.
1	Use PrimeSim HSPICE format.

primesim_convlevel

Determines how vigorously the PrimeSim tool attempts to solve the `time step too small error`.

Syntax

```
.option primesim_convlevel=[0|0.5|1|2|3|retry]
```

Arguments

Argument	Description
0	(default) Disables the option.
0.5	Helps with convergence difficulties caused by parallel simulation (<code>-np N</code> with $N > 1$).

Argument	Description
1	• Similar to 0.5. • No time step prediction. • Waveforms are smoothed with bigger time stepping. • GEAR 2nd order integration (see primesim_method). • Bigger pivot (easier pivoting, but slower and more accurate).
2	• Similar to 1. • Charge-based calculation is turned off. • Turn off incremental stamping to have a fresh matrix every time (less numerical error to help convergence, but slower). • Turn off LUT.
3	• Similar to 2. • <code>imax</code> (or <code>itl4</code>) = 32. • Backward Euler (BE) integration (see primesim_method).
<code>retry</code>	Automatically switches from 0 (default) to 1 in case of convergence difficulty.

Description

For transient analysis, as well as in DC analysis, you sometimes run into a `time step too small` error. This error indicates that the numerical iteration routines have failed and cannot reach convergence. To solve this problem, the PrimeSim tool includes an effort-level parameter, `primesim_convlevel`.

Possible values are 0, 0.5, 1, 2, 3, and `retry`. The default value is 0. Generally, you need not set this option to a non-zero value unless the PrimeSim tool has reported the following error in your log file:

```
ERROR! time step too small (diverged). Setting the
option 'primesim_convlevel=<1 or 2>' may help.
```

Note:

Simulation speed is impacted by the value of this option, with `primesim_convlevel=0` requiring no additional time and `primesim_convlevel=2` being the most time-consuming.

`primesim_convlevel` helps with convergence in the case of circuits which give rise to “difficult” matrices. Said difficulties can arise from unusual circuit topology, model parameters, or model behavior. The various levels of `convlevel` use different techniques to get around these difficulties and try to force convergence despite them.

While this option can sometimes very significantly reduce the runtime for runs that need it, it can also increase the runtime for circuits that do not by as much as 50%. Also, the “effort” put into forcing convergence and the ensuing runtime overhead generally increases with the `convlevel` value. For these reasons, a blanket application of the

`primesim_convlevel` option is not recommended. Instead, start with the lowest values of `convlevel` first. The techniques used by `convlevel=2` are a superset of those used by `convlevel=1`. `convlevel=3` uses mostly different techniques relative to those of 1 and 2.

If a circuit is found to require `convlevel` to converge or run in reasonable time, the best option is to check the circuit topology and element values (that is, very small/large resistors, very large capacitors, FET instance parameters). Another option that can be useful in that circumstance is `primesim_mparcheck` which checks the parameter values passed down to BSIM models and flags any unusual model parameter values.

primesim_cprbtol

Description

Sets the capacitance tolerance for capacitance print out. The default value is `1e-15F`. You can use this option to control the resolution of output waveforms in `fsdb/wdf`.

Syntax

```
.option primesim_cprbtol=value
```

primesim_cshpt

Description

Adds a grounded capacitor with the specified value to each node during pseudo-transient in DC. The default value is 0.

Syntax

```
.option primesim_cshpt=value
```

Examples

The following example adds a 1fF capacitor from each node to ground.

```
.option primesim_cshpt=1e-15
```

primesim_cutnode

Use this option to reduce partition size for signal nodes.

Syntax

```
.option primesim_cutnode=value
```

Description

The `primesim_cutnode` option can be assigned a *value* consisting of either:

- A numerical expression representing the number of connections that serve as a threshold for determining which nodes are cutnodes
- Alphanumerical expressions (that is, actual node names) that specify which nodes to cut

Examples

In the following example, any nodes having 500 or more device connections are selected as a cutnode.

```
.option primesim_cutnode=500
```

Note:

If a node is labeled "500" and does not have 500 or more device connections, it is not cut. This option may be useful for debug, but is not intended for accuracy or performance enhancement.

In the following example, nodes that are labeled `vdda`, `vpp`, and `500x` are cut.

```
.option primesim_cutnode="vdda vpp 500x"
```

The default value is 0 (disabled) in all Pro (FastSPICE) modes. This option is ignored in SPICE modes.

primesim_dcalg

Use this option to set the algorithm for DC convergence in the PrimeSim tool for SPICE modes only. Possible values for this parameter are 0, 1, 2, 3, and 4.

Syntax

```
.option primesim_dcalg=[0|1|2|3|4]
```

Description

Note: The `primesim_dcalg` option is for SPICE mode only. For Pro mode, see [primesim_pro_dcalg](#).

The default value for `primesim_dcalg` is 0.

- In default mode, the PrimeSim tool in SPICE mode runs through algorithms 1, 2, 3, and 4 for DC convergence:
 1. Damped Newton-Raphson (gmin ramping)
 2. Stepping gmin/src (DC source ramping)
 3. Pseudo-transient (Ramping up from 0 to condition $t=0$)
 4. Damped pseudo-transient

If DC converges in algorithm 1, it moves to the next analysis but if DC does not converge with algorithm 1, it tries algorithm 2, and so on.

If DC does not converge after going through all the algorithms sequentially then “DC not converged” is reported in the log file and simulation moves to next phase.

Note:

By default, simulation can continue even if DC does not converge, but you can force the PrimeSim tool to exit by using `primesim_exitwarn`.

- If `primesim_dcalg` is set to [1-4], only the specified algorithm is used for DC convergence. This allows you to improve performance by skipping algorithms that you already know do not converge.
- If an invalid number is specified (such as `primesim_dcalg=8`), a warning is issued in the log file and the PrimeSim tool uses the default value (0).

The following PrimeSim options can be useful when trying to achieve DC convergence:

- `primesim_convlevel` – Determines how vigorously the PrimeSim tool attempts to solve the `time step too small` error.
- `primesim_dceffort` – Controls the number of iterations to run DC convergence.
- `primesim_gic` – Set a bigger value to make it a stronger driver for `.IC` and `.NODESET`.

The following PrimeSim HSPICE-compatible commands and control options can also be useful when trying to achieve DC convergence:

- `.IC` – Attach a voltage source to drive the node.
- `.NODESET` – Preassign a node value for DC convergence.
- `.OPTION GMIN` – Set this to a large value to help convergence.
- `.OPTION GMINDC` – Specifies conductance across Drain and Source of MOS devices.
- `.OPTION GSHUNT` – Adds conductance to each node. Setting to greater than $1e-12$ can change the simulation result.

primesim_dceffort

Description

Use this option to control the maximum number of iterations to perform DC convergence for the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE mode.

Set this option to 2 or 3 to progressively increase the number of iterations. If an invalid number is given (such as `primesim_dceffort=5`) a warning is issued and the tool uses the default behavior. The default value is 1.

Syntax

```
.option primesim_dceffort=[1|2|3]
```

Arguments

Argument	Description
1	(default) Uses the default (200) or user-specified iteration limit.
2	Increases the number of convergence iterations for DC analysis by multiplying by: • 4 for Newton-Raphson, DC algorithms 1 and 2 • 2 for Pseudo-transient, DC algorithms 3 and 4
3	Increases the number of convergence iterations for DC analysis by multiplying by: • 16 for Newton-Raphson, DC algorithms 1 and 2 • 4 for Pseudo-transient, DC algorithms 3 and 4

primesim_dcopt

Description

Applies DC performance optimization.

Syntax

```
.option primesim_dcopt=[1|2]
```

Arguments

Argument	Description
1	Original default settings for DC convergence and performance.
2	Enables DC performance optimization for large circuit or post layout circuit.

Note:

The option is applicable for PrimeSim SPICE mode only.

primesim_delmax

Sets the maximum value of the internal time step in simulation.

Syntax

```
.option primesim_delmax=value
```

Description

You can have different values for different simulation windows. Although a smaller maximum time step improves accuracy for some kinds of circuits (for example a source-less ring oscillator), it slows the simulation down.

If not specified, the PrimeSim tool uses dynamic `delmax` value, and there is no `delmax` information in the log file. However, if you still want to use the previous method to calculate `delmax`'s value based on the `tstep` and `tstop` values of the `.TRAN` statement, as well as the input vectors, then set this option to 0, for example:

```
.option primesim_delmax=0
```

Then, in your `.log` file, you can find the internally calculated `delmax` value, as shown in the following example:

```
Initializing ...  
delmax : 1e-08
```

Based on this value, you might need to manually define a different `delmax` with the `primesim_delmax` option to, for example, speed up your simulation. In the first simulation time slot (from `t=0` to `t=time1`), `primesim_delmax=default`. In the second simulation time slot (from `t=time1` to `t=time2`), `primesim_delmax=value1`. In the last simulation time slot (from `t=time2` to `t=end of simulation`), `primesim_delmax=value2`.

```
.option primesim_delmax="time1:value1 time2:value2"  
.option primesim_delmax="2.2e-6:1e-10 3.8e-06:1e-9"
```

In the previous example, the PrimeSim tool makes the following settings for `primesim_delmax`:

- From 0 to 2.2us, `primesim_delmax=default`
- From 2.2us to 3.8us, `primesim_delmax=100ps`
- From 3.8us to the end of the `.tran` window, `primesim_delmax=1ns`

Expressions

The `primesim_delmax` parameter could also be used with an expression, but the usage for a space sign without a bracket is limited and an error message is issued. For example:

```
.param stoptime=1n
.option primesim_delmax="1n+stoptime/10"
.option primesim_delmax="1n + stoptime/10"
.option primesim_delmax="1n+( stoptime/10 )"
```

In the above examples, both the first and the last can work well. However, the middle one gets a parser error due to the limited space sign.

primesim_delmax_level

Note:

This option is for the PrimeSim Pro tool only.

Controls whether to use synchronous or asynchronous variable `delmax`.

Syntax

```
.option primesim_delmax_level=[1|2]
```

Description

By default (2), the PrimeSim Pro tool applies an asynchronous variable `delmax` such that each part of the circuit can have an independent `delmax` progression. When set to 1, the PrimeSim Pro tool uses synchronous variable `delmax` where all parts of the circuit share the same `delmax` progression.

Using the asynchronous method can help speed up simulation where some parts of the circuit are inactive for a long period of time. Note this option is only applicable to the PrimeSim Pro tool's Pro modes (`prohd`, `promd`, and `proxd`).

primesim_delmin

Specifies the minimum time step in a simulation.

Syntax

```
.option primesim_delmin="value"
```

Description

You can use this option to specify the minimum time step in a simulation. However, if there is a convergence problem at a time point then the PrimeSim tool scales back the `delmin` until convergence is achieved, or it reaches the `tunit`. The default value is 0 for both the PrimeSim Pro and PrimeSim SPICE tools.

Setting this option to a value equal to or smaller than `primesim_tunit` has no effect.

Examples

```
.option primesim_delmin="10p"
```

primesim_delvto

Sets options for the flash core cell.

Syntax

```
.option primesim_delvto=["value"] ["device_name1:value1 [...  
"device_nameN:valueN"]:device] ["subckt_name1:value1 [...  
"subckt_nameN:valueN"]:subckt]
```

Arguments

Argument	Description
<i>value</i>	Specifies the change in threshold voltage (vth).
<i>device_name</i>	Selects flash cells for initialization by device name. This argument supports wildcards.
<i>subckt_name</i>	Selects flash cells for initialization by subcircuit name. This argument supports wildcards.

Description

This command specifies the initial change in threshold voltage (vth) to be applied to flash cells. Multiple devices or subcircuits can be specified in a single command. If `:device` or `:subckt` is not used to specify specific instances, the command applies to all flash cells in the netlist.

Examples

The following example sets an initial vth change of -0.5V on all flash cells in the netlist.

```
.option primesim_delvto="-0.5"
```

The following example sets an initial vth change of +0.3V on the `xtop.x1.x2.mn` flash cell and +0.4V on the `xtop.x1.x3.mn` flash cell.

```
.option primesim_delvto="xtop.x1.x2.mn:0.3 xtop.x1.x3.mn:0.4":device
```

The following example sets an initial vth change of +0.2V on all flash cells in the `inv1` subcircuit and -0.3V on all flash cells in the `nor1` subcircuit.

```
.option primesim_delvto="inv1:0.2 nor1:-0.3":subckt
```

primesim_detect_lvdd

Provides auto-detection of the VDD level for `lprobe`. The default value of this option is 0.

Syntax

```
.option primesim_detect_lvdd=[0|1]
```

Description

When set to 0, the user should specify low/high threshold voltage for `lprobe`. If not set, the PrimeSim tool stops parsing with an error message. If the option is set to 1, the PrimeSim tool automatically detects the VDD level of the `lprobed` node.

primesim_detect_lvdd_static

Description

Sets logic probes for internal voltage sources generated, non-ideal sources, such as voltage regulator.

Syntax

```
.option primesim_detect_lvdd_static="nodename:value ..."
```

Examples

```
.option primesim_detect_lvdd_static="VINT1:1.2 VINT2:1.5 VINT3:2.0"
```

primesim_detect_vcvs_loop

Description

During partitioning (PrimeSim Pro mode only), loops between VCVS sources and Verilog-A may be broken causing "time step too small" errors in transient analysis.

Set this option to 1 to enable the PrimeSim Pro tool to first detect the loop between the VCVS source and Verilog-A and then keep the partitioning from breaking the loop. The default value is 0.

Syntax

```
.option primesim_detect_vcvs_loop=[0|1]
```

primesim_double_precision_output

Description

For output files, the PrimeSim tool uses 4 bytes of memory for each output value. If this option is set to 1, then the PrimeSim tool uses 8 bytes of memory for each output value, which means it could give more precise results, though the file size would be bigger. The default value is 0.

Syntax

```
.option primesim_double_precision_output=[0|1]
```

primesim_dpf

Description

Specifies the Device Parameter Format (DPF) file that contains the device parameter data extracted from the layout.

Note:

If both the DPF and SPF files try to annotate the same element, the DPF device is annotated and the SPF device is skipped. A related warning such as the following is output in the log file.

```
WARNING! add4.spf:10481: Element 'x0/x1/M1' is already
annotated by DPF. Skipped
```

Syntax

```
.option primesim_dpf="file_name"
```

Examples

```
.option primesim_dpf="ddd.dpf"
.option primesim_dpf="eee.dpf"
```

The DPF files `ddd.dpf` and `eee.dpf` are used for back-annotation.

Note:

Unlike the [primesim_spf](#) option, this option does not support `-renaming` to rename the subcircuit.

primesim_dpf_mos_only

Description

Specifies whether to back-annotate all devices or only MOSFET devices.

Note:

If this option is defined before the `primesim_dpf` option, then it is applied to all DPF files.

If this option is defined after the `primesim_dpf` option, then it is applied only to that DPF file.

Syntax

```
.option primesim_dpf_mos_only=[0|1]
```

Arguments

Argument	Description
0	(default) Back-annotate all devices.
1	Back-annotate only MOSFET.

primesim_dpfadddev

Controls whether DPF devices that are not in the netlist are added.

Syntax

```
.option primesim_dpfadddev=[0|1]
```

Description

By default (0), when the PrimeSim Pro tool finds a device in a DPF file that was not in the original netlist, it issues a warning and skips the device. For example, if the DPF file contains `xinv1/mn1` and `xinv1/mn1@2`, but there is no `xinv1.mn1` in the design, these devices would normally be skipped. Setting `primesim_dpfadddev` to 1 causes these devices to be added.

Examples

```
.option primesim_dpf="file1.dpf"  
.option primesim_dpfadddev=1 $ add new dpf devices
```

primesim_dpfhdiv

Description

Sets the divider character for hierarchical names in DPF files. The default value is `/`.

Syntax

```
.option primesim_dpfhdiv="character"
```

Examples

```
.option primesim_dpfb="file1.dpf"  
.option primesim_dpfhdiv="." $ handle names like xiv1.m32
```

primesim_dpfbmodule

Description

Specifies the module for DPF back-annotation. The `primesim_dpfb` option should be declared in front of this command.

Syntax

```
.option primesim_dpfbmodule=module_name
```

primesim_dpfbprefix

Description

Lists the prefixes that should be removed from DPF device names. Extraction tools sometimes add an additional type prefix on extracted devices in the DPF file.

Syntax

```
.option primesim_dpfbprefix="prefix"
```

Examples

```
.option primesim_dpfbprefix="m r c q"
```

`xinv1.mn2` might be listed in the DPF file as `mxinv1/mn2`. The option would remove the leading `m` in `mxinv1/mn2`.

primesim_dpfbscale

Description

Sets the scale for devices in the DPF file. The default behavior is to use the same scale as in the netlist. This option only needs to be used when the DPF scale differs from the netlist scale.

Syntax

```
.option primesim_dpfb="scale_value"
```

Examples

```
.option primesim_dpfb="file1.dpf"  
.option primesim_dpfscale=1 $ DPF devices should not be scaled.
```

primesim_dpfsuffix

Defines the suffix in a DPF file.

Syntax

```
.option primesim_dpfsuffix="string"
```

Description

In DPF files, when a device has been broken up into fingers, the extra devices and nodes have a suffix added to them. By default, the PrimeSim Pro tool assumes the suffix is @<number>. But the suffix can be other characters too. For example, consider the following excerpt from a DPF file:

```
M102 ...  
M102#1 ...  
M102#2 ...  
M102#3 ...
```

The PrimeSim Pro tool includes this option to define the suffix in a DPF file:

```
.option primesim_dpfsuffix="string"
```

Examples

```
.option primesim_dpfsuffix="#"
```

The PrimeSim Pro tool can handle the device with a hash (#) character as the suffix in a DPF file.

primesim_dvmax

Sets the maximum voltage change of a node in a single time step. Its default value is set by [primesim_mode](#).

Syntax

```
.option primesim_dvmax=value
```

Description

The value of `primesim_dvmax` is used as an upper bound and the actual value is calculated from the VDD value, which is automatically detected or can be calculated according to `primesim_vdd`.

Examples

```
.option primesim_dvmax=0.05m
```

In this example, the maximum voltage change on any node is limited to 0.05mV.

primesim_elapsed_tflush

Description

If *value*>0, sets the PrimeSim tool to flush output files (fsdb/tr0/wdf) after the specified number of seconds in elapsed runtime since the previous flush. If *value*≤0, this option has no effect. The default value is 0.

Note:

Use [primesim_tflush](#) to flush output files at a specified time interval of analysis.

Syntax

```
.option primesim_elapsed_tflush=value
```

Examples

```
.option primesim_elapsed_tflush=600
```

In this example, the PrimeSim tool flushes output files after every ten minutes (that is, 600 seconds) of runtime.

primesim_enhanced_tcl_mode

Description

This option is required in order for Tcl to perform model name matching. If interactive mode (-istop) is activated or [primesim_write_mcpaam](#) is specified, this option automatically turns on (or set to 1). Note that setting this option increases memory usage for the PrimeSim tool. The default value is 0.

Syntax

```
.option primesim_enhanced_tcl_mode=[0|1]
```

primesim_enprefix

Description

Specifies the prefix characters in a pre-layout netlist that should be removed to match with the DSPF instance name during back-annotation.

Sometimes when using an extracted device netlist as a pre-layout netlist, there may be additional prefix characters that are unnecessary when back-annotating parasitic RCs on that netlist. Use this option to remove those additional characters during DSPF back-annotation.

Syntax

```
.option primesim_enprefix="prefix1 prefix2 ..."
```

Examples

In this example, the following netlist is being used for back-annotation:

```
XxU6 up net181 clki Reset CntOut<6> c7 c6 dUDcellA
XxU7 up net181 clki Reset CntOut<7> c8 c7 dUDcellA
MxU9\mxN10 vssA net282 CEn vssA nmos ad=1.188p
+ as=0.858p l=0.3u pd=4.4u ps=2.462u w=1.25u
MxU9\mxP13 vddA net282 CEn vddA pmos ad=1.938p
+ as=1.729p l=0.3u pd=5.067u ps=4.222u w=2.5u
```

Using the following option, the leading **x** and **M** are removed during back-annotation.

```
.option primesim_enprefix="x m"
```

For example, **XxU6** is changed to **xU6** and **MxU9\mxN10** is changed to **xU9\mxN10** during back-annotation.

Note:

The prefix characters specified in this option are not case-sensitive.

primesim_eqr2spres, primesim_spresout

The SPRES solver is a faster means of calculating equivalent resistance (EQR) of resistor networks from the SPF file and is used in conjunction with: [primesim_spfeqr](#), [primesim_spf2eqr](#), [primesim_spfeqrfile](#), [primesim_spfeqr_only](#), [primesim_spf2eqr_select](#).

Syntax

```
.option primesim_eqr2spres=[0|1]
.option primesim_spresout=[1|2|3]
```

Description

Option `primesim_eqr2spres` activates the SPRES solver when set to 1. Setting this to 0 activates the original EQR solver. The default is 1.

Option `primesim_spresout` specifies the output format (1 is the default):

- 1: (default) Prints the output in the same previous format as EQR (*.eqr).
- 2: Prints the output sorted by the value of resistances (*.rout).
- 3: Prints the output as both options 1 and 2.

Running SPRES From the Command Line

Another way to specify input data for the SPRES solver is to prepare a separate Tcl command file which contains all of the necessary information. The main netlist is not used in this case. Input data are placed into file `input.tcl`. The simulation is launched by running the PrimeSim tool with the following arguments:

```
primesim -r input.tcl -o output
```

The `input.tcl` file uses commands equivalent to PrimeSim options for specifying input data, as follows:

```
file name_of_spf_file
net NetName1 NetName2 ...
rmin value
rmax value
```

Note:

Only the first two commands, `file` and `net`, are required.

Note:

For a list of supported Tcl commands, see the "Net Resistance Calculator Commands" section in the "Static Resistance Check" chapter of the *PrimeSim EMIR Reference Manual*.

Files with sorted resistances (*.rout) can be post-processed to print only resistances for selected pins:

```
primesim -rout rout_file [filter1 [filter2 [filter3 ...]]]
```

The following filters are examples:

Filter	Description
-minr minvalue	Prints only resistances larger than minvalue.
-maxr maxvalue	Prints only resistances smaller than maxvalue.
-pat pattern	Prints only resistances for pins which match pattern. pattern may contain * or ? (* matches a sequence of characters, while ? matches a single character).

Filter	Description
<code>-patskip <i>pattern</i></code>	Does not print resistances for pins which match <i>pattern</i> . <i>pattern</i> may contain * or ? (* matches a sequence of characters, while ? matches a single character).

Note:

For a list of supported filters, see the "Report Generation Commands and Options" section in the "Static Resistance Check" chapter of the *PrimeSim EMIR Reference Manual*.

The results go to file `route_file.report`.

Examples

```
.option primesim_spfeqr="net1 net2"
.option primesim_spfeqr_only=1
.option primesim_eqr2spres=1
```

In the preceding example, the PrimeSim tool extracts equivalent resistor values for `net1` and `net2`, specifies that EQR analysis should be performed without running back-annotation or full simulation, then activates the SPRES solver.

To run from the command line using `-r input.tcl`, prepare file `test.tcl` which contains the following commands:

```
file example.spf
net vdd vss
```

To run the simulation, use the command:

```
primesim -r test.tcl -o out/test
```

The results are in files `test.eqr`, `test.vdd.rout`, and `test.vss.rout`. Examples of runs for post-processing:

```
primesim -rout test.vdd -minr 10 -maxr 1000 -pat "*:src"
primesim -rout test.vss -minr 100 -pat "*:src" -patskip "xblk1*"
```

The results are in files `test.vdd.report` and `test.vss.report`.

primesim_evlevel

Description

Improves performance especially in post-layout circuits with many coupling capacitors such as SRAM/DRAM.

Syntax

```
.option primesim_evlevel=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables <code>primesim_evlevel</code> .
1	Enables optimized event control for capacitive coupling.
2	Enables optimized event control for both capacitive and non-capacitive coupling.

primesim_exit

Description

Forces the PrimeSim tool to terminate after parsing ("`parser`"), SPF back-annotation ("`spf`"), or elaboration ("`elab`"), respectively.

Syntax

```
.option primesim_exit="parser"|"spf"|"elab"
```

primesim_exit_dc_fail

Description

Specifies what to do if DC does not converge. By default, 0, the PrimeSim tool issues a warning message and then continues the simulation. If set to 1, the PrimeSim tool exits the simulation (see [Exit Codes](#)).

Syntax

```
.option primesim_exit_dc_fail=[0|1]
```

primesim_exit_invopt

Description

Forces the PrimeSim tool to exit if an invalid `primesim` option is used. By default, 0, the PrimeSim tool prints a warning if any `.option primesim_xxx` does not exist. Set this option to 1 to force an exit if an option is invalid.

Syntax

```
.option primesim_exit_invopt=[0|1]
```

primesim_exitwarn

Description

You can change any PrimeSim Pro warning to an error by defining the specific warning strings with the `primesim_exitwarn` option. Only one string is supported per option instance. A notification is issued in the log file if the simulation terminates due to `primesim_exitwarn`.

Optionally, you can specify the number of times, n , that a message can be output before it is treated as an error.

When a job exit is triggered by this option, the UNIX process return code is non-zero.

To clear all previously set `primesim_exitwarn` options, set this option with a null string:

```
.option primesim_exitwarn=""
```

Syntax

```
.option primesim_exitwarn="warning_text"[:n]
```

Examples

The following example changes the specified warning into an error, exiting simulation as soon as it occurs:

```
.option primesim_exitwarn="Following MOS() 's operating voltages exceed  
cache range"
```

The following example changes the specified warning into an error and exits simulation only upon the third time it occurs (that is, the first and second occurrences are still just warnings):

```
.option primesim_exitwarn="Following MOS() 's operating voltages exceed  
cache range":3
```

primesim_expand_instance_mfactor

Expands subcircuit instances based on the instance's multiplier parameter (M) and assigns a starting index.

Syntax

```
.option primesim_expand_instance_mfactor=value
```

Description

By default (0), instances are not expanded. Setting a value of 1 or more expands the instances and determines the starting index.

See also [primesim_fingerchar](#) and [primesim_spf_fingerchars](#).

Note:

This option is interchangeable with the PrimeSim HSPICE-compatible option, `.OPTION SPLIT_XMULT_MC`.

Examples

```
.option primesim_expand_instance_mfactor=2
```

In this example, the following instance:

```
x1 a b inv m=3
```

is expanded as follows:

```
x1 a b inv m=1  
x1@2 a b inv m=1  
x1@3 a b inv m=1
```

primesim_extended_pwrblock

Description

Enables power block detection for topologies that have feedback and/or circuits that must be in the same partition (for example, charge pumps, flash, and DRAM designs).

When this option is set to 1, the power block algorithm (see [primesim_pwrblock](#)) uses a more conservative partitioning approach at the power port boundary. This is needed for circuits with internally generated power supplies with feedback loops like charge pumps and regulators. The default value is 0.

Syntax

```
.option primesim_extended_pwrblock=[0|1]
```

primesim_fast_corner

Description

Enables fast multi-corner simulation to enhance throughput when you use the same input netlist and input vector with variations of PVT (Process, Voltage and Temperature) using `.ALTER` statements.

Note:

If you are using `.DATA SWEEP` simulation, use option [primesim_fast_corner_sweep](#).

When enabled, this option sets options `primesim_reuse_mos_model=2` and `primesim_inactive_chkanode_dv` as shown below.

Syntax

```
.option primesim_fast_corner=[0|1|2|3|4]
```

Arguments

Argument	Description
0	(default) Do not enable fast corner simulation.
1	Slowest, but most accurate.
2	Moderate speed, moderate accuracy. .option primesim_inactive_chkanode_dv=10mV
3	Faster, but less accurate. .option primesim_inactive_chkanode_dv=50mV
4	Fastest, but least accurate, .option primesim_inactive_chkanode_dv=100mV

primesim_fast_corner_sweep

Description

Enables fast multi-corner simulation to enhance throughput when you use the same input netlist and input vector with variations of PVT (Process, Voltage and Temperature) using `.DATA SWEEP` simulation. This option is also useful for characterizing standard I/O cells, SRAM changing load capacitors, input slew, and design parameters such as width and length using `.DATA SWEEP` simulation.

Note:

If you are using `.ALTER` statements, use option `primesim_fast_corner`.

When enabled, this option sets options `primesim_reuse_mos_model=2` and `primesim_inactive_chkanode_dv` as shown below.

Syntax

```
.option primesim_fast_corner_sweep=[0|1|2|3|4]
```

Arguments

Argument	Description
0	(default) Do not enable fast corner simulation.
1	Slowest, but most accurate.
2	Moderate speed, moderate accuracy. <code>.option primesim_inactive_chkanode_dv=10mV</code>
3	Faster, but less accurate. <code>.option primesim_inactive_chkanode_dv=50mV</code>
4	Fastest, but least accurate, <code>.option primesim_inactive_chkanode_dv=100mV</code>

primesim_fcamin

Sets the minimum value of floating (cross-coupling) capacitance.

Syntax

```
.option primesim_fcamin=value
```

Description

All floating capacitors smaller than this value are split into grounded capacitors. This prevents unnecessarily large partitions due to small coupling capacitance between signal nets and/or power.

For the PrimeSim Pro tool, the default value of `primesim_fcamin` is determined by the `primesim_spred` setting as follows:

- `primesim_spred=0|1|2`: 1e-20
- `primesim_spred=3`: 1e-15

For the PrimeSim SPICE tool, the default value of `primesim_fcamin` is 0.

Examples

```
.option primesim_fcamin=10fF
```

In the previous example, all floating capacitors less than 10fF become grounded capacitors.

primesim_fcapmodel

Controls the modeling of floating capacitances.

Syntax

```
.option primesim_fcapmodel=[1|2|3|4]
```

Arguments

Argument	Description
1	Specifies loose tolerances for all floating capacitors.
2	Specifies loose tolerances for small floating capacitors (default for <code>promd/proxd</code>).
3	Specifies standard tolerances for all floating capacitors (default for <code>prohd/legacy</code> FastSPICE mode).
4	Specifies tightening tolerances for all floating capacitors.

Description

You can adjust the `fcapmodel` setting to gain speed-up for circuits with a large number of floating capacitances. This option only applies to Pro (FastSPICE) mode. Setting to 1 yields the best performance, while 4 gives the best accuracy.

Note:

This option only works when `primesim_ccmodel=0`. Currently the `ccmodel` option is set default and this option is reset to 0.

primesim_fflush

Description

Sets the PrimeSim tool to output simulation data to a file based on elapsed time or simulation progress for the following analyses:

- `.AC` (PrimeSim Pro) / `.AC` (PrimeSim SPICE)
- `.LIN` (PrimeSim Pro) / `.LIN` (PrimeSim SPICE)
- `.LSTB` (PrimeSim Pro) / `.LSTB` (PrimeSim SPICE)
- `.NOISE` (PrimeSim Pro) / `.NOISE` (PrimeSim SPICE)

Syntax

```
.option primesim_fflush=time_interval
```

Arguments

Argument	Description
<code>time_interval</code>	Can be specified in time units or as a percentage of the total frequency points swept. Default is 10%. If specified in time units, for example, <code>10n</code> , the PrimeSim tool outputs simulation data every 10 nanoseconds. If specified as a percentage, for example, <code>25%</code> and 100 frequency points are swept, the PrimeSim tool outputs simulation data every 25 frequency points.

primesim_finger_current

Description

By default (0) the PrimeSim tool reports the total device current after back-annotation with syntax such as `i(m1)`. When set to 1, this option directs the PrimeSim tool to report only the main device's finger current contribution.

Syntax

```
.option primesim_finger_current=[0|1]
```

primesim_fingerchar

Description

Sets the finger delimiter character for devices and instances. The default is an at-sign (@) character.

See also [primesim_expand_instance_mfactor](#) and [primesim_spf_fingerchars](#).

Syntax

```
.option primesim_fingerchar=character
```

Examples

```
.option primesim_fingerchar=@  
x1 a b inv m=1  
x1@2 a b inv m=1
```

In this example, the PrimeSim tool interprets `x1` as an instance with two fingers: `x1` and `x1@2`.

primesim_flash

Description

Provides auto-detection and partitioning of internally-generated power supplies in flash design simulations, and enables the advanced algorithm of massive array detection and partitioning, which is critical for simulation performance.

Note:

When enabling this option, the recommended value is 2; only try a value of 1 if your simulation results in an accuracy issue.

Syntax

```
.option primesim_flash=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables this option so that it has no effect on the simulation.
1	Enables this option for better accuracy.
2	Enables this option for better performance.

primesim_flashcell_erase_update

Description

Specifies `flashlevel=2` erase behavior (see [Flash Cell Model](#)).

Syntax

```
.option primesim_flashcell_erase_update=[0|1]
```

Arguments

Argument	Description
0	(default) Specifies that the minimum bound for Vth is now the maximum of the following: • Vg • Vth_init - abs(Vtersmaxshift) • Vtlow + Vterssat

Argument	Description
1	Specifies that the minimum bound for Vth is now the maximum of the following: - Vg + Vterssat - Vth_init - abs(Vtersmaxshift) - Vtlow + Vterssat

primesim_flatsize

Sets a threshold such that any subcircuit with a transistor count under it is flattened before partitioning starts.

Syntax

```
.option primesim_flatsize=value
```

Description

A small partition, due to its strong coupling, can increase runtime without any accuracy improvement. This option merges small subcircuit partitions into a larger one to speed up the simulation run. Its default is 7.

Examples

```
.option primesim_flatsize=50
```

In this example, a subcircuit with fewer than 50 transistors is flattened before partitioning.

primesim_floating_gate_gshunt

Description

A conductance defined by this option is added between the floating gate node and ground. This option includes any floating gate that has RCs connected to it. The default is 0.

Syntax

```
.option primesim_floating_gate_gshunt=value
```

Examples

```
.option primesim_floating_gate_gshunt=1e-9
```

The PrimeSim Pro tool inserts a 1e-9 conductance between any floating gate and ground.

primesim_fmiflag

Description

Lets you disable the CMI or FMI interface. The default is -1.

Syntax

```
.option primesim_fmiflag=[-1|0|1]
```

Arguments

Argument	Description
-1	(default) Errors out if both FMI and CMI models are used in a single simulation.
0	Disables the FMI flow. Only CMI models are used.
1	Disables CMI when the cmiflag is set to 0. Only FMI models are used.

primesim_fsc_auto_detect

Description

Voltage levels corresponding to logic levels or setting the VDD for a C-model are handled automatically using this option. Specific unique voltage levels can still be set from within the C-model itself. The default value is 0.

Syntax

```
.option primesim_fsc_auto_detect=[0|1]
```

primesim_fsc_vdd

Description

Specifies the supply voltage for the C-model interface. Alternatively, you can use `primesim_fsc_auto_detect=1` for automatic detection of supply voltage.

If `primesim_fsc_vdd` is not defined, then the PrimeSim tool uses the propagated vdd and if that cannot be determined, it uses `primesim_vdd` which is the maximum vdd found from all the voltage sources and input vectors.

Syntax

```
.option primesim_fsc_vdd=value
```

primesim_fsdb_limit

Specifies a size limit for FSDB files, resulting in the generation of multiple FSDB files of a more manageable size.

Syntax

```
.option primesim_fsdb_limit=value
```

Description

This option accepts integer values representing megabytes. The default value is 0, resulting in one FSDB file without a size limit.

Examples

```
.option primesim_fsdb_limit=1500
```

In the previous example, the limit is set at an integer value of 1500 megabytes or 1.5 gigabytes. In the case of 3GB of output, the PrimeSim Pro tool creates two 1.5GB .fsdb files.

When `primesim_fsdb_limit` is set, `primesim_output_fname_type=2` is also set automatically. This directs the PrimeSim tool to generate the multiple files with built-in instructions for PrimeWave Design Environment WaveView which reads all of the multiple FSDB files when the main FSDB file is read.

The default hierarchy delimiter in FSDB output files is ". ". Take care as this can affect PrimeSim Pro users with custom FSDB readers.

primesim_fsdb_max_size

Terminates the simulation when the waveform size reaches a specified limit.

Syntax

```
.option primesim_fsdb_max_size=value
```

Description

The *value* for `primesim_fsdb_max_size` is in MB. The default is unlimited. Once this option is set, all other size-affected options (`primesim_fsdb_limit` and `primesim_fsdb_split_time`) are ignored. Note that the size of the FSDB can be slightly larger than the specified limit to complete writing the buffer in memory.

primesim_fsdb_split_pg

Description

Specifies to split Power/Ground R-Network nodes in non-ideal Power/Ground simulation from all other signals and then output them into a separate FSDB file.

The split FSDB file name is: *output_prefix.pg.fsdb*

Note:

The following options cannot be used with this option:

- `primesim_fsdb_limit`
- `primesim_fsdb_split_time`
- `primesim_output_range`

If used with option `primesim_fsdb_split_pg`, these options are ignored with a warning message.

Syntax

```
.option primesim_fsdb_split_pg="net_name1 net_name2 ..."
```

primesim_fsdb_split_time

Description

Specifies to split FSDB based on the time values *time1*, *time2*, *time3*, and so on.

When `primesim_fsdb_split_time` is set, `primesim_output_fname_type=2` is also set automatically. This directs the PrimeSim tool to generate the multiple files with built-in instructions for PrimeWave Design Environment WaveView which reads all of the multiple FSDB files when the main FSDB file is read.

Syntax

```
.option primesim_fsdb_split_time="time1 [time2 time3 ...]"
```

primesim_fsdb_term_var_hier

Description

Changes the terminal delimiter, in hierarchical signal names, from a colon (:) to a period (.).

By default (`primesim_fsdb_term_var_hier=3`), both Verilog-A variables and device terminals for a 3rd-party tool are changed from a colon (:) to a period (.).

Syntax

```
.option primesim_fsdb_term_var_hier=[0|1|2|3]
```

Arguments

Argument	Description
0	Do not change the terminal delimiter for Verilog-A variables and device terminals for a 3rd-party tool; that is, keep a colon (:) in the name as is.
1	Change the terminal delimiter for Verilog-A variables (for example, <code>vamod:var</code> becomes <code>varmod.var</code>).
2	Change the terminal delimiter for device terminals for a 3rd-party tool (for example, <code>R1:p</code> becomes <code>R1.p</code>).
3	(default) Change the terminal delimiter for both Verilog-A variables and device terminals for a 3rd-party tool.

primesim_fsdb_version

Description

Specifies what version of FSDB writer the PrimeSim tool should use, in order to support backward compatibility with some third-party tools. The default is 6.0.

Note:

FSDB versions 5.8 and below are no longer supported.

Syntax

```
.option primesim_fsdb_version=[5.9|6.0]
```

primesim_gdcpath

Description

Adds gshunts to nodes having no DC path to ground.

Syntax

```
.option primesim_gdcpath=value
```

Examples

```
.option primesim_gdcpath=1e-12
```

In Transient Analysis, this adds 1e-12 mho gshunts to nodes having no DC path to ground. The default value is gmin (if defined) or else 1e-12.

primesim_gen_ac_op

Description

Controls whether operating point files (for example, .op0@1.0000e-07) or initial condition files (for example, .ic0@1.0000e-07) are generated during an "AC at Diff T" simulation (see [primesim_ac_at_diff_t](#)).

- Operating point files contain initial condition data and element information at the transient timepoint.
- Initial condition files are smaller, as they just contain initial condition data at the transient timepoint.

Syntax

```
.option primesim_gen_ac_op=[0|1]
```

Arguments

Argument	Description
0	Generates initial condition (.ic) files during an "AC at Diff T" simulation.
1	(default) Generates operating point (.op) files during an "AC at Diff T" simulation.

primesim_gen_ic_op

Generates a .op file.

Syntax

```
.option primesim_gen_ic_op=[0|1]
```

Description

When a .op card is included in a simulation deck, the DC operating point is calculated and an output file .op is created. But if the keyword UIC is specified for .tran analysis, this time=0 operation point calculation is not used in the simulation.

If an analysis such as small signal .ac or .net requires an operating point to begin with, the .op card does not have to be specified in the deck and the PrimeSim Pro tool runs it

automatically. The `.op` file by default (0) is not created for such analysis. To generate this file, use the following option:

```
.option primesim_gen_ic_op=1
```

where 1 sets `.op` file generation.

The `.op` command also can be used at different time points along a `.tran` analysis. Consider the following example:

```
.OP 0n 10n 20n 30n
```

This creates four `.OP` files: `.op0`, `.op1`, `.op2`, and `.op3`.

Multi `.op` cards, however, are not allowed in a simulation deck.

primesim_gen_mc0

Description

Specifies whether to generate the `*.mc0` file for Monte Carlo analysis. The `*.mc0` file includes the independent random variable values for each sample.

Syntax

```
.option primesim_gen_mc0=[0|1]
```

Arguments

Argument	Description
0	(default) Do not generate the <code>*.mc0</code> file.
1	Generates the <code>*.mc0</code> file.

primesim_gen_op_captab

Description

Generates node capacitance information in the `.op` file.

Syntax

```
.option primesim_gen_op_captab=[0|1|2]
```


Arguments

Argument	Description
0	(default) Do not generate node capacitance information in the .op file.
1	Generate node capacitance information in the .op file with .OPTION CAPTAB=1.
2	Generate node capacitance information in the .op file with .OPTION CAPTAB=2.

primesim_gen_pn_c

Description

Enables output to the *.pn_c0 file in .NOISE analysis. By default, it is set to 1 and the pn_c0 file is generated. If set to 0, then the pn_c0 file is not generated.

Syntax

```
.option primesim_gen_pn_c=[0|1]
```

primesim_gic

Sets the conductance value of the resistor in the Norton-type voltage source that is tied to a node.

Syntax

```
.option primesim_gic=value
```

Description

The recommended value range for primesim_gic is from 1 to 1e10, with its default being 1e10. If primesim_gic is not defined, then no Norton voltage source is applied.

A Norton voltage source, which has a resistor in parallel to a current source, is used to maintain a node voltage for .ic conditions. The larger this conductance, the stronger the node voltage it can maintain.

Use this option when you have set .ic conditions but final DC initialization does not converge. Some of the nodes in the .ic file might have inaccurate initial voltages. With a new gic value, usually smaller than its default, the final node voltage drifts to the correct value – for instance, from 0.8v to 0.65v.

Examples

```
.option primesim_gic=100
```

In this example, suppose you have a node `.ic` value of $V(a)=1$. This example sets the Norton voltage source to `conductance=100mho` and `I=100A`.

primesim_gmax

Description

The PrimeSim tool clamps the stamped conductance value (dI/dV) to `primesim_gmax` (which defaults to 100 for PrimeSim Pro and 1000 for PrimeSim SPICE).

Syntax

```
.option primesim_gmax=value
```

primesim_gmi_telem

Description

As of release L-2016.03, the PrimeSim tool uses the PrimeSim HSPICE implementation for [Lossless Transmission Lines \(T-element\)](#) by default (1), but you can specify 0 for backward compatibility.

Syntax

```
.option primesim_gmi_telem=[0|1]
```

primesim_goff

Description

Works with the `.CHKZNODE` statement to determine whether a MOSFET is in the `on` state or `off` state. When the Gds of the device is greater than `primesim_goff`, it is considered to be turned on. If `primesim_goff` is not defined, then `.CHKZNODE` determination is performed based on `vth`.

Syntax

```
.option primesim_goff=value
```

primesim_gpd

Description

Back-annotates a corner used in the Galaxy Parasitic Database (GPD).

Syntax

```
.option primesim_gpd="[gpd_path] [corner] -top 0|1 -dump_spf 0|1"
```

Arguments

Argument	Description
<i>gpd_path</i>	Path to the design GPD.
<i>corner_name</i>	Name of the corner to simulate. Default is the first corner inside the GPD.
-top	Specifies how to process <code>.SUBCKT</code> and <code>.ENDS</code> while reading the GPD: • 0 – Use <code>.SUBCKT</code> and <code>.ENDS</code> while reading the GPD. • 1 – Ignore <code>.SUBCKT</code> and <code>.ENDS</code> while reading the GPD. Default is as set in the GPD.
-dump_spf	Specifies whether to output an SPF format file in the current working directory: • 0 – (default) Do not output an SPF file. • 1 – Output an SPF file, <i>gpdlib_name.corner_name.spf</i>.

Examples

```
.option primesim_gpd="dir/path_to_gpd FF -top 0"
```

In this example, the PrimeSim tool back-annotates the `FF` corner used in the GPD, including `.SUBCKT` and `.ENDS` statements.

primesim_gpuid

Description

Specifies the GPU card to use for the current process. If not specified using this option, the PrimeSim tool automatically selects the most suitable GPU card based on the status of all available GPU cards.

For more information, see [GPU Card Selection](#).

Syntax

```
.option primesim_gpuid=[gpu_card_id]
```

primesim_gpu_mode

Description

Specifies the mode of operation when using GPU simulation.

For more information, see [Running GPU Simulation](#).

Syntax

```
.option primesim_gpu_mode=[0|1|2]
```

Arguments

Argument	Description
0	Enables all the necessary and essential GPU features to provide basic GPU simulation.
1	Enables enhanced matrix acceleration algorithms that are device model independent. Note: This is the default setting for the PrimeSim Pro tool's Pro modes (<code>prohd</code> , <code>promd</code> , and <code>proxd</code>).
2	Moves device model evaluations from CPU to GPU. Recommended for performance. Note: This is the default setting for the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE modes (<code>spicehd</code> , <code>spicemd</code> , and <code>spicexd</code>).

primesim_gshpt

Description

Adds conductance of *value* from each node to ground during pseudo-transient simulation. After pseudo-transient converges, the conductance value is ramped down to achieve better DC accuracy.

Recommended values are between $1e-12$ (`.OPTION GMIN`) and $1e-9$. Larger values, for example from $1e-9$ to $1e-3$, improve convergence, but could also reduce accuracy if the trying of smaller values stops at a large value.

Syntax

```
.option primesim_gshpt=value
```

Examples

```
.option primesim_gshpt=1e-8
```

In this example, the conductance value is $1e-8$. After pseudo-transient converges, the PrimeSim tool continues to try smaller conductance values $1e-9$, $1e-10$, ..., $1e-18$ to achieve better and better DC accuracy while the iteration can still converge.

primesim_hb_autoharm_tol

Description

When used together with the `AUTOHARMS` argument (see `.HB`), this option controls the tolerance when choosing the number of harmonics to be used by Harmonic Balance analysis. The default value is $1e-4$.

Syntax

```
.option primesim_hb_autoharm_tol=value
```

Examples

```
.option primesim_hb_autoharm_tol=1e-5
```

In this example, the tolerance when choosing the number of harmonics to be used by Harmonic Balance analysis is set to 1e-5.

primesim_hier_delimiter

Description

Changes the hierarchical delimiter from a period (.) to another character.

Syntax

```
.option primesim_hier_delimiter="char"
```

Examples

```
.option primesim_hier_delimiter="/"
```

Specifies a slash (/) as the hierarchical delimiter instead of a period (.)

primesim_hiersim

Selects hierarchical simulation mode.

Syntax

```
.option primesim_hiersim=[0|0.5|1]
```

Arguments

Argument	Description
0	Disables the hierarchical detection algorithm.
0.5	(default) Enables a conservative hierarchical detection algorithm.
1	Enables an aggressive hierarchical detection algorithm.

Description

The PrimeSim Pro tool automatically detects hierarchical structures, such as memory arrays, in a design. However, you can also selectively apply this adaptive hierarchical

simulation approach to different portions of a design or disable it completely by using the option `primesim_hiersim`.

The PrimeSim Pro tool automatically searches for the best candidate for hierarchical simulation.

Note that 1 can slow down non-memory simulations because this option works well with array structures but not random control logic.

Examples

```
.option primesim_hiersim="subckt1 subckt2 ..."
```

In this example, the PrimeSim Pro tool selectively applies hierarchical simulation to `subckt1`, `subckt2`, ..., and runs the rest of the circuit in flattened mode.

primesim_hstolscale

Description

Sets a multiplier applied to all internal tolerance values for subcircuits that are simulated with the `hiersim` method. The smaller is the value, the more accurate the simulation is for those circuit parts. This option is similar to `primesim_tolscale`, which sets the tolerance scale globally.

Syntax

```
.option primesim_hstolscale=value
```

Examples

```
.option primesim_tolscale=1  
.option primesim_hstolscale=0.01
```

The subcircuits that are simulated with hierarchy method have more strict tolerance with multiplier 0.01, while other parts still use the default tolerance for that simulation mode.

The default value for `primesim_hstolscale` is 1. It only supports globally setting, and cannot be used for a specific subcircuit.

primesim_ic_priority

Description

If `.IC` or `.NODESET` values are defined on the same node, by default (0), the PrimeSim tool takes the later one. If you set this option to 1, the PrimeSim tool takes the `.IC` value even if there is a later defined `.NODESET` value.

Syntax

```
.option primesim_ic_priority=[0|1]
```

primesim_icsweep

Note:

This option is for the PrimeSim Pro tool only.

Description

Controls initial condition reuse under multiple sweep(s) or analyses scenarios. This option is helpful to get consistent DC results between sweeps and speedup DC convergence.

Syntax

```
.option primesim_icsweep="<0|1|2|3|4> \  
  [-at=<ALTER|TEMP|ANA|outer|MC|DATA|BISEC|inner>] \  
  [-iclevel=<initval|nodeset|ic>] \  
  [-dp=<0|1>]"
```

Arguments

Argument	Description
0	Fully turn off any reuse.
1	Use previous iteration's operating point as initial condition for DC convergence.
2	Use first iteration's operating point as initial condition for DC convergence.
3	Similar to argument 2. For Monte Carlo sweeps - Use nominal iteration's operating point as initial condition for DC convergence.
4	(default) Auto selection of reuse mechanism depending on type of sweep(s) in the netlist. • If Monte Carlo sweep is detected – it uses <code>mode=3</code> and reuse operating point at nominal. • If <code>DATA BISEC</code> or other simple sweep selected, it uses <code>mode=1</code> where previous iteration's operating point is used as initial condition.

Argument	Description
-at	Fine tunes at which sweep/ analyses to apply reuse. <i>ALTER</i>: Sets reuse at <i>.ALTER</i>. <i>TEMP</i>: Sets reuse at <i>.TEMP</i>. <i>ANA</i>: Sets reuse at analyses. <i>outer</i>: (default) Indicates the first loop from the analyses statement sweep setting. <i>MC</i>: Sets reuse at Monte Carlo sweep <i>DATA</i>: Sets reuse at <i>.DATA</i> sweep. <i>BISEC</i>: Sets reuse at Bisection sweep. <i>inner</i>: Indicates the most inner layer of the loops from the analyses statement.
-iclevel	Strength of initial condition for operating point reuse. <i>initval</i>: (default) Lowest strength for initial condition. <i>nodeset</i>: Higher strength for initial condition. <i>ic</i>: Highest strength for initial condition for operating point reuse.
-dp	<i>0</i>: Distribute without matching first sweep among workers in distributed mode. <i>1</i>: (default) Distribute sweeps by matching the first iteration of the to be reused over all workers in distributed mode.

Note:

`.option icsweep=0|1` now maps to `.option primesim_icsweep=0|1` respectively.

Note:

`.option icsweep=2` is ignored and the default for the `primesim_icsweep` is maintained.

Note:

This option `primesim_icsweep` is not applicable in CoSim-DKI flow.

Examples

```
.option primesim_icsweep=3
```

When analysis statement is `.tran 1p 100p sweep data=arc_data monte=5`, this reuses the operating point at the nominal Monte Carlo run for every monte or data loop.

primesim_ichier

Description

When multiple `.IC` commands are given for a node through subcircuit ports hierarchically, the PrimeSim Pro tool selects the outermost `.IC` by default. You can control the selection with `primesim_ichier`.

Syntax

```
.option primesim_ichier=[local|global]
```

Examples

```
.option primesim_ichier=local
.option primesim_ichier=global (default)

.subckt subl a
c1 a 0 1f
.ic v(a)=2v
.ends
x1 a subl
.ic v(a)=1v
```

In the previous example above, when `primesim_ichier=global` is set, 1v is selected. When `primesim_ichier=local` is set, 2v is selected.

primesim_identical_mc_instance_file

Description

When the option is given with an `imc_file` as the described syntax, the PrimeSim tool generates the same random number sequence for the Monte Carlo simulation for those subcircuit instances within a parenthesis.

Syntax

```
.option primesim_identical_mc_instance_file=imc_file_name imc_file
( instance_0_1 instance_0_2 instance_0_3 ...)
( instance_1_1 instance_1_2 instance_1_3 ...) ...
```

Examples

```
.option primesim_identical_mc_instance_file=imc_file_name
imc_file
( instance_0_1 instance_0_2 instance_0_3 ...)
( instance_1_1 instance_1_2 instance_1_3 ...)
...
```

where `instance_x_x` is a full hierarchical subcircuit instance name. In the previous syntax, any random number generation within `instance_0_x` has the same value sequence as `instance_1_x`.

```
.tran .1n 40n sweep monte=10

.subckt sss 1 2
.param rval=agauss(50,1,1)
R1 1 2 R=rval
R2 1 2 R=rval
.ends

x1 1 0 sss
x2 2 0 sss
x3 3 0 sss
v1 1 0 1
v2 2 0 1
v3 3 0 1
.print i(v1) i(v2) i(v3)
.option primesim_identical_mc_instance_file="a.imc"
.END

-----
"a.imc"
(x1 x3)
-----
```

In the previous example, the result shows the same values for `i(v1)` and `i(v3)`, but not for `i(v2)`.

primesim_ignore

Description

Use this option to get rid of instances or subcircuits from simulation to achieve higher performance.

Syntax

```
.option primesim_ignore="instance subcircuit"
```

Examples

```
.option primesim_ignore="x1 x2.xinv"
.option primesim_ignore="nand"
```

In the first example, the PrimeSim Pro tool ignores the `x1` and `x2.xinv` instances in the SPICE netlist. In the second example, the PrimeSim Pro tool ignores the `nand` subcircuit in the SPICE netlist.

The PrimeSim Pro tool supports wildcards when using the ignore feature:

```
.option primesim_ignore="x101.xj*.x11"
```

primesim_ignore_chkfunc_error

Specifies to ignore error messages related to [Circuit Checks](#) (.chkxxx) commands.

Syntax

```
.option primesim_ignore_chkfunc_error=[0|1]
```

Description

By default (0), the PrimeSim Pro tool errors out if the circuit check is performed on a floating node, dangling node, and/or a node that does not exist in the netlist. When this option is set to 1, the PrimeSim Pro tool continues the simulation and bypass the error. You still see the error message in the log file.

Note that this option only ignores error messages related to Circuit Checks (.chkxxx) commands. It does not apply to other simulation errors.

primesim_ignore_device

Description

Use this option to make quick netlist changes by ignoring specified devices during the simulation. This option can be used to target a single device or a few devices that you want to ignore in a subcircuit.

Note:

Wildcards are not supported with this option.

Syntax

```
.option primesim_ignore_device="subckt_name::device_name"
```

Examples

```
.option primesim_ignore_device="nand::M2"
```

In this example, the MOSFET `M2` is ignored from all instantiations of the subcircuit `nand` in the simulation.

primesim_ignore_flat_device

Description

Use this option to ignore one or more specified flattened devices (that is, devices with no top-level subcircuit) during the simulation.

Note:

Wildcards are supported with this option.

Syntax

```
.option primesim_ignore_flat_device="device_names"
```

primesim_ignore_floating_node

Suppresses WARNINGS about "No DC path to ground on MOSFET...".

Syntax

```
.option primesim_ignore_floating_node="string"
```

Description

If you set this option, the PrimeSim tool suppresses WARNINGS about "No DC path to ground on MOSFET...". Wildcards are accepted in the syntax.

Examples

```
.option primesim_ignore_floating node="*ld_*"  
.option primesim_ignore_floating_node="*"   
.option primesim_ignore_floating_node="xdut.ld_x221:g"
```

The first example suppresses all WARNINGS about "No DC path to ground on MOSFET...." where the floating gate node name contains "ld_".

The second example suppresses all WARNINGS about "No DC path to ground on MOSFET" on all floating gates.

The third example suppresses all WARNINGS about "No DC path to ground on MOSFET...." where the floating gate node name is "xdut.ld_x221:g".

primesim_ignore_layout_only_device

Description

Skips the specified devices for the include flow.

Syntax

```
.option primesim_ignore_layout_only_device="+name|-name pattern ..."
```

Examples

```
.model dmodel1 d ...
.model dmodel2 d ...
.subckt dmodel3 p n ...
...
.ends

d1 1 2 dmodel1 ...
d2 1 2 dmodel2 ...
x1 1 2 dmodel3 ...

.option primesim_ignore_layout_only_device="+name dmodel* -name dmodel2"
```

This example removes all diodes and instances of diodes that contain the model name `dmodel*` except for `dmodel2`. So in this example, `d1` and `x1` are deleted and `d2` is kept.

primesim_ignore_no_vec_tdata_error

Description

When set to 1, an error for missing tabular data in the I/O vector file is ignored and the simulation continues. The default value is 0, so that an error for missing tabular data in the I/O vector file ends the simulation. See [Vector File Format](#) for more information.

Syntax

```
.option primesim_ignore_no_vec_tdata_error=[0|1]
```

primesim_ignore_op_analysis

Description

Specifies whether to ignore `.OP` analysis in the netlist.

Syntax

```
.option primesim_ignore_op_analysis=[0|1]
```

Arguments

Argument	Description
0	<code>.OP</code> analysis, if defined in the netlist, is not ignored. Note: This is the default setting for the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE modes (<code>spicehd</code> , <code>spicemd</code> , and <code>spicexd</code>).

Argument	Description
1	.OP analysis, if defined in the netlist, is ignored. Note: This is the default setting for the PrimeSim Pro tool's Pro modes (<code>prohd</code> , <code>promd</code> , and <code>proxd</code>).

primesim_ignore_option_error

Description

Ignores incorrect input for the `primesim_hiersim` and `primesim_pwrblock` options. The default is 0. When set to 1, the PrimeSim tool continues a simulation even if those options contain an error.

Syntax

```
.option primesim_ignore_option_error=[0|1]
```

primesim_ignore_subblk_option_error

Specifies whether to terminate a simulation if a user-specified block-level option exists but the subcircuit does not exist in the design.

Syntax

```
.option primesim_ignore_subblk_option_error=[0|1]
```

Description

The PrimeSim Pro tool normally terminates with an error message if the user-specified block-level option exists but the subcircuit does not exist in the design. When set to 1, the PrimeSim Pro tool does not error out. Instead, it prints a warning message and continues the simulation. The default value is 0.

Currently, this option supports ignoring the incorrect block level option value for following options:

- `primesim_mode`
- `primesim_model`
- `primesim_partition`
- `primesim_speed`

primesim_ignore_va

Description

When set to 1, the PrimeSim tool ignores Verilog-A file parsing which improves front-end performance. The default setting is 0.

Note:

This option should be used only when Verilog-A instances are not present in the circuit. In addition, this option works only when `primesim_pva_flow=0`.

Syntax

```
.option primesim_ignore_va=[0|1]
```

primesim_inactive_chkanode_dv

Description

When performing fast multi-corner simulation (see [primesim_fast_corner / primesim_fast_corner_sweep](#)), this option inserts a `.CHKANODE` option internally in the input run file to detect information on inactive nodes from the first run. See [Check Active/Inactive Nodes \(.CHKANODE\)](#) for more information.

Set this option to the *value* in mV under which a node should be considered inactive. This *value* represents a trade-off in performance versus accuracy as a higher value results in more inactive nodes.

Syntax

```
.option primesim_inactive_chkanode_dv=value
```

Examples

In this example, any nodes below 10mV are considered inactive when performing fast multi-corner simulation.

```
.option primesim_inactive_chkanode_dv=10mV
```

primesim_include_instance

Description

Use this option to include the instances needed in the simulation. All other instances are ignored.

Note:

Wildcards are not supported with this option. Multiple instances cannot be specified in one command.

Syntax

```
.option primesim_include_instance="instance"
```

Examples

```
.option primesim_include_instance="X1.X2"  
.option primesim_include_instance="X1.X3"  
.option primesim_include_instance="X1.X4"
```

Assuming there is one top-level instance `x1` that has ten instances under it from `x1`, `x2`, ..., to `x10`, then this example includes in the simulation only the three instances specified in the options, while the other seven instances are ignored.

primesim_indn

Description

Specifies whether the PrimeSim tool uses a Norton equivalent or a more accurate formulation to model a simple inductor.

Syntax

```
.option primesim_indn=[0|1]
```

Arguments

Argument	Description
0	Use a more accurate formulation to model a simple inductor (default for the PrimeSim SPICE tool when <code>runlvl=4 5 6</code>).
1	Use the Norton equivalent to model a simple inductor (default for the PrimeSim Pro tool and for the PrimeSim SPICE tool when <code>runlvl=1 2 3</code>).

primesim_ind_matrix_passivity

Description

Checks and handles passivity of inductance matrices. The default is `1`.

Syntax

```
.option primesim_ind_matrix_passivity=[0|1|2]
```


Arguments

Argument	Description
0	Does not check the passivity for all inductance matrices.
1	Checks the passivity for all inductance matrices whose dimension is not bigger than value of <code>option primesim_ind_matrix_passivity_dim_limit</code> . Reports the information of related inductors and mutual inductors if non-passivity is found.
2	Checks the passivity for all inductance matrices whose dimension is not bigger than value of <code>option primesim_ind_matrix_passivity_dim_limit</code> . If non-passivity is found, the information of related inductors and mutual inductors is reported and all mutual inductance in the group are scaled by 1e-5 to make the matrix passive.

primesim_ind_matrix_passivity_dim_limit

Description

Used along with `primesim_ind_matrix_passivity=1` or `2`, to set the matrix dimension limit for inductance matrices passivity check. The default is `1000`.

Syntax

```
.option primesim_ind_matrix_passivity_dim_limit=VALUE
```

primesim_info_limit

Sets the number of "INFO:" messages to a limited number of user-specified lines.

Syntax

```
.option primesim_info_limit=value
```

Description

If you set this option to an integer value, the PrimeSim tool prints the specified number of "INFO:" messages for every INFO type. After simulation, the PrimeSim tool does not report how many "INFO:" messages have been suppressed.

The default value is `10`, which means the PrimeSim tool reports ten "INFO:" messages and suppresses the rest.

Note:

"INFO!" messages are not limited by this option.

primesim_iovec_abs_value

Description

By default (0), the PrimeSim tool treats the time value in the vector file as incremental from the last time value. If set to 1, the PrimeSim tool treats all the time values in the vector file as absolute time.

Syntax

```
.option primesim_iovec_abs_value=[0|1]
```

Examples

```
signal a b
radix 11
io      ii
logichv LOGICLV_PARAM
logiclv LOGICLV_PARAM
period '2*tCK'
slope tt
#0      00
#10     10
#30     01
```

By default, signal a is high at 10ns and signal b is high at 40ns (10n + 30n). With this option, signal b is instead high at 30ns.

primesim_iovec_option

Description

When set to 1, specifies that PrimeSim accuracy options such as [primesim_mode](#) can be used inside a vector file using the `fscript` utility `fset` command. The default is 0.

Note:

This option must be set before `.VEC` or [primesim_vector](#) is used to specify the vector file.

Syntax

```
.option primesim_iovec_option=[0|1]
```

primesim_iovec_vih

Sets the `logichv` outside of a vector file.

Syntax

```
.option primesim_iovec_vih="signal_name:vih_level"
```

Description

The PrimeSim tool supports setting the voltage threshold of vector files on individual pins using `.option primesim_iovec_vih="signal_name:vih_level"`.

For example:

- `primesim_iovec_vih="level"` – Sets vih level.
- `primesim_iovec_vih="signal:level"` – Sets vih level for individual signal.
- `primesim_iovec_vih="level signal:level ..."` – Sets level 1 for global vih and sets vih level 2 for signal vih.

primesim_iovec_vil

Sets the `logiclv` outside of a vector file.

Syntax

```
.option primesim_iovec_vil="signal_name:vil_level"
```

Description

The PrimeSim tool supports setting the voltage threshold of vector files on individual pins using `.option primesim_iovec_vil="signal_name:vil_level"`.

primesim_iovec_voh

Description

Sets the default `voh` value for the vector file.

Note:

There is no default value for this option; failure to specify a value generates an error.

Syntax

```
.option primesim_iovec_voh=value
```

primesim_iovec_vol

Description

Sets the default `vol` value for the vector file.

Note:

There is no default value for this option; failure to specify a value generates an error.

Syntax

```
.option primesim_iovec_vol=value
```

primesim_iprbtol

Description

Sets the current tolerance for the current print out. You can use this option to control the resolution of output waveforms in `fsdb` or `wdf`.

- For the PrimeSim Pro tool, the default is `1e-12amp`.
- For the PrimeSim SPICE tool, the default is `1e-15amp`.

Syntax

```
.option primesim_iprbtol=value
```

primesim_iprobe_leaf_level

Description

When set to `1`, the PrimeSim tool outputs device currents at the lowest level of hierarchy in the design.

Note:

There may be multiple lowest levels of hierarchy in a design. Devices at higher levels of hierarchy are not probed.

A current probe statement (for example, `probe i(*)`) must reside in the netlist. The default is `0`.

Syntax

```
.option primesim_iprobe_leaf_level=[0|1]
```

Examples

In the following example, the PrimeSim tool probes the drain currents of transistors in the leaf cell instances located at the lowest level of hierarchy in the design.

```
.option primesim_iprobe_leaf_level=1  
.probe i(*)
```

In the following example, the PrimeSim tool probes the drain/gate/source/bulk currents of transistors in the leaf cell instances located at the lowest level of hierarchy in the design.

```
.option primesim_iprobe_leaf_level=1  
.probe iall(*)
```

In the following example, the PrimeSim tool probes the drain currents of transistors, as well as the passive devices (resistors, capacitors, and so on) in the leaf cell instances located at the lowest level of hierarchy in the design.

```
.option primesim_iprobe_leaf_level=1  
.option primesim_probe_passive_device=1  
.probe i(*)
```

In the following example, the PrimeSim tool probes the drain currents of transistors and sources (for example, power supply, input voltage source) at all levels of hierarchy.

```
.option primesim_iprobe_leaf_level=0 (default)  
.probe i(*)
```

primesim_isolated_vsrc_res

Description

When set to 1, the PrimeSim tool detects floating voltage sources which are referenced by other current controlled sources and adds a shunt resistance of 1M to them. The default value is 0.

Syntax

```
.option primesim_isolated_vsrc_res=[0|1]
```

primesim_keep_0v_dcvs

Description

This option keeps zero voltage DC sources in the simulation database. The default is 0.

Note:

The PrimeSim tool also accepts the `keep_0v_dcvs` option.

Syntax

```
.option primesim_keep_0v_dcvs=[0|1]
```

primesim_keepzeroparms

Description

Sometimes an instance call in a netlist passes zero values for PS, PD, AS, and AD parameters in BSIM MOSFETs. Because these parameters are not physical and can cause convergence problems, the PrimeSim tool has an option to ignore these zero-specified values and substitute its own calculated values as it would normally do if these parameters were omitted from the netlist.

The default value is 1, where zero parameters are kept. 0 substitutes PS, PD, AS, and AD with calculated values from BSIM equations.

Syntax

```
.option primesim_keepzeroparms=[0|1]
```

Examples

```
Mxxx D G S B l=length w=width as=0 ad=0 ps=0 pd=0  
.option primesim_keepzeroparms=0
```

The above example ignores the `as=0`, `ad=0`, `ps=0`, `pd=0` and recalculates from the BSIM equation.

primesim_leakage_mode

Description

This option is used for better accuracy for leakage measurement. The default value is 0. When set to 1, the PrimeSim Pro tool uses more conservative tolerances where accurate leakage currents are needed to be measured.

Syntax

```
.option primesim_leakage_mode=[0|1]
```

Examples

```
.option primesim_leakage_mode=1
```

primesim_list_noisesources

Description

Note:

This option is for the PrimeSim SPICE tool only.

Specifies whether to include noise contributors in the output waveform file from the following analyses:

- [.HBNOISE](#)
- [.LIN](#)
- [.NOISE](#)
- [.PHASENOISE](#)
- [.SNNOISE](#)

The default value is 1 which includes noise contributors in the output. When set to 0, the PrimeSim SPICE tool includes only the probed output noise.

Syntax

```
.option primesim_list_noisesources=[0|1]
```

Examples

```
.option primesim_list_noisesources=0
```

primesim_loadmodel

Description

Sets the capacitive load model that a partition output drives. Its default value is determined by the [primesim_mode](#) setting.

Syntax

```
.option primesim_loadmodel=[1|2|3|4]
```

Arguments

Argument	Description
1	Specifies the average gate capacitance.
2	Specifies either 1 or 3 for a load.

Argument	Description
3	Specifies an efficiently calculated Miller effect and voltage dependent capacitance load.
4	Specifies an accurate Miller effect and voltage dependent capacitance load.

Examples

```
.option primesim_loadmodel=3
.option primesim_mode=promd
```

In this example, `loadmodel` is set to 3 for `promd` mode.

You can specify a subcircuit, instance or device-based local setting. For example:

```
.option primesim_loadmodel=2
.option primesim_loadmodel="subcktA:2 subcktB:3":subckt
.option primesim_loadmodel="XA.XB:2 XA.XC:3":instance
.option primesim_loadmodel="XA.m1:2 XA.m2:3":device
```

primesim_lprobe_hiz

Description

When set to 1, the PrimeSim tool checks the Hi-Z node and displays the "Z" state in the waveform. The default is 0.

Syntax

```
.option primesim_lprobe_hiz=[0|1]
```

primesim_lprobe_vh

Sets the high threshold value for signals with `.LPROBE` statement.

Syntax

```
.option primesim_lprobe_vh=value
```

Description

Any value that is greater than this threshold is probed as logically one (1). This option is for the global setting; the `high` definition in `.lprobe` overrides it for corresponding signals. If there is no `primesim_lprobe_vh` definition and also no `high` definition in the `.lprobe` statement, the PrimeSim Pro tool issues an error message and stops the simulation.

The default value for this option is 80% of the highest voltage detected when looking at voltage sources and input vectors.

Examples

```
.option primesim_lprobe_vh=2.5  
.lprobe v(a) high=2.3  
.lprobe v(b)
```

The high threshold for signal *a* is 2.3v, while the high threshold for signal *b* is 2.5v.

primesim_lprobe_vl

Sets the low threshold value for signals with `.LPROBE` statement.

Syntax

```
.option primesim_lprobe_vl=value
```

Description

Any value that is less than this threshold is probed as logically zero (0). This option is for the global setting; the `low` definition in `.lprobe` overrides it for corresponding signals. If there is no `primesim_lprobe_vl` definition and also no `low` definition in the `.lprobe` statement, the PrimeSim Pro tool issues an error message and stops the simulation.

The default value for this option is 20% of the highest voltage detected when looking at voltage sources and input vectors.

Examples

```
.option primesim_lprobe_vl=0.5  
.lprobe v(a) low=0.3  
.lprobe v(b)
```

The low threshold for signal *a* is 0.3v, while the low threshold for signal *b* is 0.5v.

primesim_lsf_format_chars

Allows usage of 3rd-party Verilog-A special characters.

Syntax

```
.option primesim_lsf_format_chars=[0|1]
```

Description

This command allows usage of 3rd-party Verilog-A special characters, listed below:

- `%P` – LSF job process ID
- `%T` – LSF job starting time

- %D – LSF job starting date
- %H – LSF host name

Examples

```
primesim_lsf_format_chars=[0|1]
```

The default value is 0. Changing it to 1 changes the definition of %P, %T, %D, %H to LSF parameters.

primesim_map_prbport

Description

Maps probe statements from : [dgsb] to v[1234] syntax. For example:

```
.probe v(*.*:g)
```

is ported to:

```
.probe v2(*.*)
```

Note:

Using .probe v(*.*:g) in a simulation can cause a huge increase in memory consumption. Changing the statement to v2(*.*) does not increase the memory consumption.

Syntax

```
.option primesim_map_prbport=[0|1]
```

Arguments

Argument	Description
0	(default) Does not map v(*.*:[dgsb]) to v[1234](*.*)
1	Maps v(*.*:[dgsb]) to v[1234](*.*)

primesim_max_parallel

Description

When you run a parallel PrimeSim simulation on a set number of CPUs, the results may vary from run to run due to the slightly different ordering of events in solving the partitions.

When this option is set to 0, then for a run on the same number of CPUs, the ordering of the internal evaluations is maintained. Note that using this option can result in a significant runtime penalty.

The default setting is 1.

Syntax

```
.option primesim_max_parallel=[0|1]
```

primesim_max_swap_allowed

Description

Controls the maximum amount of swap memory that can be used during a simulation. If the simulation requires more than the given value, then it exits.

Note:

This option only applies if `primesim_no_swap` is set to 1. If `primesim_no_swap` is set to 0, the operating system controls swap memory and this option is ignored.

Syntax

```
.option primesim_max_swap_allowed=value_in_MB
```

primesim_max_width_tol

Description

For the `wmax` binning check, using this option can allow the PrimeSim tool to add a factor of `wmax` and use the new effective `wmax` $[(1+\text{primesim_max_width_tol}) \times wmax]$ to check for model binning. The default value is 0.

Syntax

```
.option primesim_max_width_tol=extension_value
```

Examples

```
.option primesim_max_width_tol=0.1
```

The above example extends the maximum width allowed by an additional 10%.

primesim_maxicout

Description

Sets the maximum number of node voltages included in the `.ic` (Initial Condition) file. The default value is 2,147,483,647, which is also the upper limit of the value possible for this option.

Syntax

```
.option primesim_maxicout=value
```

Examples

```
.option primesim_maxicout=500,000  
.option primesim_maxicout=0
```

In the first example, 500,000 nodes are saved in the `.ic` file.

In the second example, no `.ic` file is created. This is useful when you run a large simulation and you do not need your `.ic` file for future use.

primesim_mc_sigma_scale

Description

Scales the Monte Carlo distribution. When *value* is greater than 0, the sample points are generated such that the standard deviation of the Gaussian distribution is multiplied by *value*. The default value is 1.

Syntax

```
.option primesim_mc_sigma_scale=value
```

primesim_mc_stats_report

Controls printing of the Monte Carlo statistics.

Syntax

```
.option primesim_mc_stats_report=[0|1]
```

Description

By default, this option is set to 1. The PrimeSim tool prints out the avg/std/min/max information at the end of the `.mt0` file, excluding any "error" measurements.

Set this option to 0 to disable printing of the Monte Carlo statistics.

The following example shows the default behavior.

```
$DATA1 SOURCE='PrimeSimPro' VERSION='2020.12-SP1'
.TITLE '* Monte Carlo tutorial - (c) Synopsys'
monte          xl          temper          alter#
  1.0000e+00    1.6184e-09    2.7000e+01    1.0000e+00
  2.0000e+00    1.6211e-09    2.7000e+01    1.0000e+00
.....
          avg    1.7319e-09    2.7000e+01    1.0000e+00
          std    4.0956e-10    2.7000e+01    1.0000e+00
          min    1.5990e-09    2.7000e+01    1.0000e+00
          max    4.2459e-09    2.7000e+01    1.0000e+00
```

primesim_mcbrief

Limits the Monte Carlo information written to the log file.

Syntax

```
.option primesim_mcbrief=[0|1]
```

Description

By default (1), the PrimeSim tool abbreviates the log file information for Monte Carlo after the first sweep. Set this option to 0 to enable printing all the Monte Carlo sweep information into the log file.

The log shows:

```
SWEEP #1.
Ignore writing out the details for the rest of the sweep...
```

primesim_mcseed

Description

Provides a seed to the random number generator that produces the same sequence of random numbers in Monte Carlo simulation.

The value can be set to any integer ≥ 0 (0 is the default). Specify a value of -1 to use a random seed.

Syntax

```
.option primesim_mcseed=value
```

primesim_meas_from_to

Description

When set to 1, the `FROM` and `TO` parameters for the `TARG` parameter of a measurement are not applied to the `TRIG` parameter of the measurement (see `.MEASURE` / `MEAS` statement). The default is 0.

Syntax

```
.option primesim_meas_from_to=[0|1]
```

primesim_measout

Controls the measurement file format.

Syntax

```
.option primesim_measout=[0|1|2|3|4]
```

Arguments

Argument	Description
0	(default) Concatenates all measurement values and outputs them on one line (as with PrimeSim HSPICE).
1	Prints each measurement value on a separate line (as with legacy HSPICE). Note: Multiple <code>.mt#</code> files are generated for sweep analysis.
2	Prints each measurement value on a separate line (as with legacy HSPICE). Note: Only one <code>.mt0</code> file is generated for sweep analysis.
3	Generates the <code>.mt0.csv</code> file in a Comma Separated Value (CSV) format.
4	Prints the output <code>trig/targ</code> , <code>from/to</code> , and <code>error/failed</code> information to a <code>.meas</code> file (as with PrimeSim XA). Note: Multiple <code>.meas#</code> files are generated for sweep analysis.

Description

When using bisection, if `primesim_measout=1`, the PrimeSim Pro tool generates one file for each `.ALTER` loop. This is the same behavior as `primesim_measout=0`, but the file format is different. The final result is now in one separate file `xxx.bisec_mt0`.

Similarly, during Monte Carlo analysis, if `primesim_measout=1`, the PrimeSim tool generates one measure output file for each sample, but generates a single file for all samples when `primesim_measout=0` (default).

Note:

For the *continuous* measurement file format, see [primesim_cont_measout](#).

Examples

In this example, multiple sweeps are specified.

If `.option primesim_measout=1`, then the output files follow the naming convention:
`*.mt#`.

If `.option primesim_measout=4`, then the output files follow the naming convention:
`*.meas#`.

primesim_meastolscale

Specifies the compression factor of internally saved data when performing runtime measurement.

Syntax

```
.option primesim_meastolscale=value
```

Description

In runtime measurement, if one measurement, M2, depends on another measurement, M1, the signal of M2 is internally saved and the measurement, M2, is executed with this saved data after M1 is resolved.

The default value for this option is `1.0` and the internally saved data is compressed. Specify a value of `0` to disable compression of the saved signal.

primesim_merged_meas_file

Description

Specifies whether to output separate `.mt0` files or a single merged `.mt0` file for sweep and bisection.

Syntax

```
.option primesim_merged_meas_file=[0|1]
```

Arguments

Argument	Description
0	(default) Multiple <code>.mt0</code> files are output for each sweep, and each file has intermediate bisection data.
1	Only one <code>.mt0</code> file is generated for each sweep and bisection.

primesim_merge_fsdb

Description

Specifies whether to merge the PrimeSim tool's FSDB output (see [primesim_output](#)) with a digital FSDB file for mixed-signal simulation using the VCS PrimeSim AMS tool.

Syntax

```
.option primesim_merge_fsdb=[0|1]
```

Arguments

Argument	Description
0	(default) Do not merge the PrimeSim output with a digital FSDB file for cosimulation.
1	Merge the PrimeSim output with a digital FSDB file for cosimulation.

Examples

```
.option primesim_output=fsdb  
.option primesim_merge_fsdb=1
```

The preceding example outputs in FSDB format and merges it with a digital FSDB file for cosimulation.

primesim_merge_parmosdio

Description

Optimizes MOSFET device instances that are in parallel.

Syntax

```
.option primesim_merge_parmosdio=[0|1]
```


Arguments

Argument	Description
0	Disables parallel MOSFET device instance optimization.
1	(default) Enables parallel MOSFET device instance optimization.

primesim_merge_pinstance

Description

Optimizes subcircuit instances that are in parallel.

Syntax

```
.option primesim_merge_pinstance=[0|1]
```

Arguments

Argument	Description
0	Disables parallel subcircuit instance optimization. Note: This is the default setting for the PrimeSim Pro tool's Pro modes (<code>prohd</code> , <code>promd</code> , and <code>proxd</code>).
1	Enables parallel subcircuit instance optimization. Note: This is the default setting for the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE modes (<code>spicehd</code> , <code>spicemd</code> , and <code>spicexd</code>).

primesim_merge_vector_errmsg

Description

Specifies whether to merge the `.err` files from multiple runs into a single `.err` file.

Syntax

```
.option primesim_merge_vector_errmsg=[0|1]
```

Arguments

Argument	Description
0	(default) Generate separate <code>.err</code> files for each run.
1	Generate a single merged <code>.err</code> file.

primesim_method

Sets the numerical integration method used during a transient analysis.

Syntax

```
.option primesim_method=integ_method
```

Description

This option defaults to either `TRAP` or `GEAR` based on the presence of voltage sources, current sources, and other circuit conditions. For example, if `VCCS` is present, then the default is `GEAR`.

The possible selections are:

- `primesim_method=TRAP` (Trapezoidal)
- `primesim_method=BE` (Backward Euler)
- `primesim_method=GEAR` (2nd order of Gear)
- `primesim_method=EGEAR` (Enhanced 2nd order of Gear)

primesim_mode

Note:

This option is for the PrimeSim Pro tool only.

Specifies the targeted circuit type.

Syntax

```
.option primesim_mode=mode
```

Description

Each of the PrimeSim Pro modes has a targeted circuit type. The following is a guideline on when to use each mode.

primesim_mode Usage Guideline

[Table 25](#) lists the mode guidelines.

Table 25 Mode Guidelines

Mode	Target
spicehd	Highly sensitive analog circuits.

Table 25 Mode Guidelines (Continued)

Mode	Target
spicemd	General SPICE mode for all circuit types.
spicexd	Mixed signal design/Extracted Post Layout.
prohd	Large mixed signal design/leakage/power simulations.
promd	Timing simulations.
proxd	Functional verifications.

Setting Local Options for Instance, Node, or Device

The PrimeSim Pro tool provides a subcircuit, instance, node, or device-based local option setting for the `primesim_mode` option.

Global `primesim_mode` option setting:

```
.option primesim_mode=mode
```

Local `primesim_mode` option setting:

```
.option primesim_mode="pattern:mode"[:subckt|:instance|:node|:device]
```

As shown in the preceding expression, when setting a local option, the PrimeSim Pro tool applies mode with matched patterns for a given type (the default is `subckt`).

In the following example, all devices under instance `x1` are simulated with `spicemd` mode.

```
.option primesim_mode="x1.*:spicemd":device
```

In the following example, all subcircuits which begin with "inv" are simulated with `promd` mode, because the default type of pattern is `subckt`.

```
.option primesim_mode="inv*:promd"
```

This usage is identical to the following:

```
.option primesim_mode="inv*:promd":subckt
```

In the following example, all instances under `xpll` which begin with "xj" are simulated with `spicehd` mode.

```
.option primesim_mode="xpll.xj*:spicehd":instance
```

Note:

The local setting affects only the specified block (subcircuit, instance, node, or device) and any underlying blocks contained by it. If any of the underlying

blocks are also part of a different block that is not affected by the local setting, then those underlying blocks are instead affected by the global setting.

.opset Option

This option applies specified option sets when the value of `primesim_mode`, as specified in the netlist, is matched. the PrimeSim Pro tool also supports wildcards for the value of `primesim_mode`.

```
.opset (primesim_mode=mode) option_name[=option_value] ....
```

Example:

```
.opset (primesim_mode=spicemd) primesim_method=gear
.opset (primesim_mode=prohd) primesim_method=gear
.opset (primesim_mode=default) primesim_method=be
.opset (primesim_mode=pro*) primesim_model=3
```

In this example, when `primesim_mode` is set to `spicemd` or `prohd` mode, apply the `primesim_method` option to `gear`. When `primesim_mode` is `default`, apply the `primesim_method` option to `be`. When `primesim_mode` is set to any of the three PrimeSim Pro modes (that is: `prohd`, `promd`, or `proxd`), apply the `primesim_model` option to 3.

Setting the Simulation Mode

This is the most important option in the PrimeSim Pro tool because it automatically selects a subset of options that are crucial to simulation accuracy, performance, and capacity.

The default is `promd` in the PrimeSim Pro tool's Pro modes and `spicemd` in the PrimeSim Pro tool's SPICE modes.

Table 26 lists the corresponding values for `primesim_model`, `primesim_speed`, `primesim_spred`, `primesim_partition`, `primesim_loadmodel`, and `primesim_tunit` for each mode.

Table 26 *primesim_mode Options*

	model	speed	spred	partition	loadmodel	tunit
spicehd	4	0.5	0	0	N/A	0.01p
spicemd	4	1	0	0	N/A	0.1p
spicexd	3	1	1	0	N/A	1p
prohd	3	1	1	1	4	1p
promd	2	2	2	1	2	1p
proxd	2	3	3	1	1	1p

`spicehd` is the most accurate mode and `proxd` is the least accurate mode. All of the PrimeSim Pro tool's SPICE modes are single matrix solving without any partitioning being performed. If you want to run in SPICE mode, you must add `primesim_mode=spice` to your netlist before invoking the PrimeSim Pro tool with the PrimeSim Pro script, or use the command-line argument `-spice_lic`.

Examples

```
.option primesim_mode=spice (or spicemd)
.option primesim_mode="PLL:spicehd ADC:spicemd CTRL:proxd"
```

In the first example, the simulation runs in `spicemd` mode.

In the second example, the simulation runs in different modes on different parts of the design. PLL is in `spicehd`, ADC is in `spicemd`, and CTRL is in `proxd`. The rest of the design is in the default mode, `promd`.

primesim_model

Sets the modeling method of the MOSFETs.

Syntax

```
.option primesim_model=[1|2|3|4|4.5|5]
```

Arguments

Argument	Description
1	Reduced MOS model that includes efficient Ids, capacitance, and junction diodes.
2	Same as 1 plus Miller feedback, and feedforward capacitance.
3	Same as 2 plus accurate Ids, capacitance, and tunneling current.
4	Exact MOS model that includes all components.
4.5	Same as 4 plus quadratic interpolation. Can use direct equations if not within confidence interval.
5	Direct equation evaluation.
Note: This setting is recommended for model validation and verification; however, it comes with some trade off in performance.	

Description

Each `primesim_mode` definition uses one of these model definitions by default (see [Table 26](#)).

Also, different models can be assigned to different subcircuits in one simulation run.

Note:

Settings of 2 and below ignore leakage current such as igs/igd/igb/gidl/gisl/isub.

Examples

```
.option primesim_mode=prohd  
.option primesim_model="PLL:4 CTRL:2"
```

In this example, the MOSFETs in PLL and CTRL use model 4 and model 2, respectively, while all others use model 3 (default for prohd, see [Table 26](#)).

primesim_model_cache_dc

Description

Controls whether to use model caching for DC analysis. When modeling highly nonlinear models, like SOI models, sometimes disabling model caching for DC can improve convergence. `primesim_model_cache_dc` enables/disables model caching. By default (1), model caching is turned on. Setting it to 0 disables model caching during DC analysis.

Syntax

```
.option primesim_model_cache_dc=[0|1]
```

primesim_model_dup_rule

Description

Specifies how to handle duplicate `.MODEL` definitions.

Note:

Regardless of how this option is set, if you define duplicate model names, a warning is posted to the log file.

Syntax

```
.option primesim_model_dup_rule=[0|1|2]
```

Arguments

Argument	Description
0	(default) If there are duplicate <code>.MODEL</code> definitions, select the last one.
1	If there are duplicate <code>.MODEL</code> definitions, select the first one.

Argument	Description
2	Error out for duplicate <code>.MODEL</code> definitions.

primesim_model_verification_mode

Runs model qualification.

Syntax

```
.option primesim_model_verification_mode=[0|1]
```

Description

This option is for simulations involving model qualification. Setting the option to 1 uses one of the tightest simulation settings to generate the most accurate result. Note this option is only recommended for qualification involving few devices, and not recommended for general use. The default value is 0.

primesim_module_instance

Applies a 3DIC module for a regular netlist without modifying the input netlist.

Syntax

```
.option primesim_module_instance="instance_name=module_name::subckt_name"
```

Description

You can specify the instance name (*instance_name*) to apply to a 3DIC module (*module_name*) and subcircuit (*subckt_name*) without any netlist modification.

Note:

This option is recommended only for simple 3DIC application structures when running the PrimeSim tool standalone. Applying a donut style structure is not supported.

Examples

```
.option primesim_module_instance="xi1=SS::inv"
.option primesim_module_instance="xi2=FF::inv"

xi0      in      out1    vdd      inv
xi1      in      out2    vdd      inv
xi2      in      out3    vdd      inv

.module SS
. lib './model.lib'  slow
```

```
.endmodule SS

.module FF
  .lib './model.lib' fast
.endmodule FF
```

primesim_monitor

Description

Provides CPU monitoring functionality when the PrimeSim tool is running on a server. By default, CPU usage and machine loading messages are printed in an extended .elog file.

The CPU monitor also issues a warning message when the PrimeSim tool is running on an overloaded machine. By default, option primesim_adaptive_mt=1 is invoked when the CPU usage is lower than the specified threshold.

Note:

This option only works with .TRAN analysis; it issues a warning message for AC/DC/OP or other analyses.

Syntax

```
.option primesim_monitor="t1 [-log=0|1 -threshold=n1 -adaptive=0|1  
-swap=-1|0|n2 -swap_exit=n3]"
```

Arguments

Argument	Description
t1	(default: 600) Specifies the time interval (in seconds) at which CPU messages are printed out. The minimum value for t1 is 60; any non-zero value smaller than that is replaced with 60 and a warning is issued. Note: Setting this argument to 0 disables the CPU monitoring feature.
-log=0 1	Specifies whether CPU messages are printed to a .log file or a .elog file: 0: Prints all CPU messages into a .log file. 1: (default) Prints all CPU messages into a .elog file.

Argument	Description
<code>-threshold=<i>n1</i></code>	(default 80) Specifies the threshold (as a percentage) for invoking option <code>primesim_adaptive_mt=1</code>. The maximum value for <i>n1</i> is 95; any value larger than that is replaced with 95 and a warning is issued. If PrimeSim CPU usage is lower than (<i>n1</i> * number of threads), an overloading warning message is printed to the output file and then option <code>primesim_adaptive_mt=1</code> may be invoked depending on <code>-adaptive=0 1</code>. For example, when <i>n1</i> is set to 80 and the CPU usage for an 8-thread PrimeSim job is smaller than 80%*8=640%, it is defined as overloading.
<code>-adaptive=0 1</code>	Specifies whether <code>primesim_adaptive_mt=1</code> is invoked automatically on an overloaded machine: • 0: Does not invoke <code>primesim_adaptive_mt=1</code> automatically. • 1: (default) Invokes <code>primesim_adaptive_mt=1</code> when CPU usage is lower than specified by <code>-threshold=<i>n1</i></code>. Note: Option <code>primesim_adaptive_mt=1</code> improves performance when multi-threading but has no benefit when running on a single CPU. In addition, option <code>primesim_adaptive_mt=1</code> is only supported by PrimeSim SPICE modes.
<code>-swap=-1 0 <i>n2</i></code>	Controls swap monitoring. By default, the PrimeSim tool does not exit even if using disk swap. However, you can change this behavior by setting the <code>-swap</code> argument as follows: • -1: The PrimeSim tool does not control swap and gives control to the operating system (equivalent to <code>primesim_no_swap=0</code>). The simulation exits if it uses disk swap for <code>-swap_exit=<i>n3</i></code> periods. • 0: The PrimeSim tool avoids using disk swap, and if the simulation runs out of physical memory, the simulation errors out (equivalent to <code>primesim_no_swap=1</code>). • <i>n2</i>: The PrimeSim tool can use a maximum of <i>n2</i> MB of swap memory during the simulation. The simulation exits if the size of swap memory exceeds <i>n2</i> and this state lasts for <code>-swap_exit=<i>n3</i></code> periods. Note: Setting the <code>primesim_no_swap</code> and/or <code>primesim_max_swap_allowed</code> options overrides the <code>-swap</code> argument.
<code>-swap_exit=<i>n3</i></code>	(default: 6) Because the <code>primesim_monitor</code> option flushes the monitoring result periodically (see argument <code>t1</code>), the value <i>n3</i> specifies the number of periods before the simulation exits due to low performance introduced by memory swapping. For example, for <code>primesim_monitor=600 -swap_exit=6</code>, the PrimeSim tool exits after six reports (that is, one hour) due to having high amount of swap and low runtime performance.

primesim_monte_cmd_overwrite

Description

Overwrites the `SWEEP MONTE=MCcommand` section of the analysis statements without having to modify the netlist.

Syntax

```
.option primesim_monte_cmd_overwrite="MCcommand"
```

Arguments

Argument	Description
<i>MCcommand</i>	See Input Syntax under Monte Carlo Analysis in the PrimeSim Tool .

Examples

The following example overwrites the analysis statement in the netlist and runs Monte Carlo analysis on indexes 5, 6, 7, and 10.

```
.option primesim_monte_cmd_overwrite="list (5:7 10)"
```

primesim_moscapcut

Description

Specifies when the MOS capacitor should be cut for partitioning based on its estimated capacitance value.

The default is `1e-16`. For advanced technologies, it is recommended that you set *value* to be smaller than the default in order to get larger partitions and better accuracy.

Syntax

```
.option primesim_moscapcut=value
```

primesim_mosfet_fast_probe_lx34

Description

When set to 1, this option speeds up probing the LX34 MOS parameter (see [Probing Element Parameters](#)) for the level=129 CMI model. The default is 0.

Syntax

```
.option primesim_mosfet_fast_probe_lx34=[0|1]
```

primesim_mparcheck

Description

Determines whether MOSFET model parameters are checked.

Syntax

```
.option primesim_mparcheck=[0|1]
```

Arguments

Argument	Description
0	(default) No parameter checking.
1	Check and print out warning/error messages.

primesim_negcap

Description

When set to 0, the PrimeSim tool removes negative capacitances. The default value is 1.

Syntax

```
.option primesim_negcap=[0|1]
```

primesim_negres

Controls handling of negative resistors.

Syntax

```
.option primesim_negres=[0|1]
```

Description

Some RC extraction tools extract negative resistors for MOSFET devices. By default (1), the PrimeSim Pro tool keeps such negative resistors and also keeps them from being reduced. If this option is set to 0, such negative resistors are ignored and shorted by the PrimeSim Pro tool.

primesim_no_swap

Description

Controls whether disk swap can be used when the simulation runs out of physical memory.

Note:

Using disk swap slows down performance.

Syntax

```
.option primesim_no_swap=[0|1]
```

Arguments

Argument	Description
0	(default) Allows the PrimeSim tool to use disk swap.
1	Avoids using disk swap; if the simulation runs out of physical memory, the simulation errors out.

Examples

With a default of `primesim_no_swap=0`, if the simulation runs out of physical memory, then the following warning message is issued and the simulation continues:

```
WARNING! Out of Physical Memory. (Total Physical RAM: 16418900 kB; Free: 66024 kB)
```

The simulation may become very slow due to disk swapping.

But if set `primesim_no_swap=1`, then the simulation terminates with the following error message:

```
ERROR! Out of Physical Memory. (Total Physical RAM: 16418900 kB; Free: 43668 kB)
```

primesim_noise_format

Description

When running noise analysis in the PrimeSim tool, set this option to 2 to switch the format of `.pn0` output file to be compatible with PrimeWave Design Environment WaveView. The default value is 1.

Syntax

```
.option primesim_noise_format=[1|2]
```

primesim_noise_off

Description

Disables noise generation only for noise sources of the specified `type`. Noise of all other types is enabled. This option is supported for both AC noise and transient noise analyses.

One keyword, `instance` | `subckt` | `device`, should be specified along with a `type`. If more than one keyword is specified in the same statement, the last keyword is the one honored. If multiple statements—each with one `instance` | `subckt` | `device` keyword—are specified, then all statements are honored. If a keyword or `type` is not specified, a warning is generated and the option is ignored.

If the `primesim_noise_on|off` options are not used, all noise types are enabled for all devices.

Note:

Noise control for parasitic resistors is applicable only in post-layout circuits with included SPF files flow.

If this option is used multiple times for the same instance, subcircuit, or device with different noise types, then only the last option sequentially in the netlist is used.

Note:

You can also define `noise=0|1` for an instance, subcircuit, or device directly in the netlist; however, if there is a conflict between that direct definition and this option for a specific instance, subcircuit, or device, then this option takes precedence.

Syntax

```
.option primesim_noise_off="[instance|subckt|device=name1 [, ...]]  
[type=thermal|flicker|shot|ign|all|none]"
```

Arguments

Argument	Description
<code>instance</code>	Specifies the name of the instances.
<code>subckt</code>	Specifies the name of the subcircuits.
<code>device</code>	Specifies the name of the devices.

Argument	Description
<code>type</code>	• <code>thermal</code>: Thermal noise. • <code>flicker</code>: Flicker noise. • <code>shot</code>: Channel shot noise in FETs and MOSFETs; junction shot noise in diodes. • <code>ign</code>: Gate shot noise in MOSFETs. • <code>all</code>: All noise inject. • <code>none</code>: No noise inject.

Examples

In the following example, thermal and flicker noise only are disabled for instance `I1`, while all other noise types are enabled for it and all other devices.

```
.option primesim_noise_off="instance=I1 type=thermal,flicker"
```

primesim_noise_on

Description

Enables noise generation only for noise sources of the specified `type`. Noise of all other types is disabled. This option is supported for both AC noise and transient noise analyses.

One keyword, `instance` | `subckt` | `device`, should be specified along with a `type`. If more than one keyword is specified in the same statement, the last keyword is the one honored. If multiple statements—each with one `instance` | `subckt` | `device` keyword—are specified, then all statements are honored. If a keyword or `type` is not specified, a warning is generated and the option is ignored.

If the `primesim_noise_on|off` options are not used, all noise types are enabled for all devices.

Note:

Noise control for parasitic resistors is applicable only in post-layout circuits with included SPF files flow.

If this option is used multiple times for the same instance, subcircuit, or device with different noise types, then only the last option sequentially in the netlist is used.

Note:

You can also define `noise=0|1` for an instance, subcircuit, or device directly in the netlist; however, if there is a conflict between that direct definition and this option for a specific instance, subcircuit, or device, then this option takes precedence.

Syntax

```
.option primesim_noise_on="[instance|subckt|device=name1 [, ...]]  
[type=thermal|flicker|shot|ign|all|none]"
```

Arguments

Argument	Description
instance	Specifies the name of the instances.
subckt	Specifies the name of the subcircuits.
device	Specifies the name of the devices.
type	- thermal: Thermal noise. - flicker: Flicker noise. - shot: Channel shot noise in FETs and MOSFETs; junction shot noise in diodes. - ign: Gate shot noise in MOSFETs. - all: All noise inject. - none: No noise inject.

Examples

In the following example, thermal and flicker noise only are enabled for instance `I1`, while all other noise types are disabled for it and all other devices.

```
.option primesim_noise_on="instance=I1 type=thermal,flicker"
```

primesim_npc_max

Description

Specifies that the PrimeSim SPICE tool should take extra effort to ensure that simulation results are consistent across a different number of threads (see command-line option `-np` `x` in [Table 2](#)), especially for large post-layout simulation. Simulation run-time may increase as a trade-off.

Syntax

```
.option primesim_npc_max=[0|num]
```

Arguments

Argument	Description
0	(default) Disables the option.
num	A positive integer up to 128 representing the maximum number of processors.

Examples

For example, to get consistent simulation results for up to 8 threads, set:

```
.option primesim_npc_max=8
```

for the following set of simulations:

- `primesim input.sp -o`
- `primesim input.sp -o -np 2`
- `primesim input.sp -o -np 4`
- `primesim input.sp -o -np 8`

Note:

For two parallel simulations to be consistent, they must use the same `primesim_npc_max` option value. Otherwise, the following two simulations are not guaranteed to be consistent:

- `-np 2 + primesim_npc_max=8`
- `-np 4 + primesim_npc_max=16`

primesim_num_meas_log

Description

Limits the number of measurement results printed to the screen and written to the log file. If `primesim_num_meas_log` is not defined, the PrimeSim tool prints and writes all of the measurement data.

Syntax

```
.option primesim_num_meas_log=number
```

primesim_num_meas_per_line

Outputs measurement values to multiple lines.

Syntax

```
.option primesim_num_meas_per_line=number
```

Description

When a simulation has large number of `.meas` statements, the PrimeSim tool outputs all the measurement values into one single line in the `.mt0` file by default. This very long line

structure sometimes made the `.mt0` file unreadable or difficult to post-process with scripts. Set this option to output measurement values to multiple lines:

```
.option primesim_num_meas_per_line=number
```

where *number* can be any integer larger than 0, which is the default value.

```
.option primesim_num_meas_per_line=2
```

This option outputs two measurement values per line as follows:

```
rmvariable_i(x3.x2.r1) rmvariable_i(x3.x2.r2)
rmvariable_i(x3.x1.r1) rmvariable_i(x3.x1.r2)
rmvariable_i(x3.x6.r1) rmvariable_i(x3.x6.r2)
```

primesim_op_compatible

Description

Specifies which operating point (`.OP`) analysis output format to use.

Syntax

```
.option primesim_op_compatible=[0|1]
```

Arguments

Argument	Description
0	(default) Use the standard PrimeSim <code>.OP</code> output format which has some differences with both PrimeSim HSPICE and 3rd-party formats.
1	Use the common <code>.OP</code> output format for compatibility with PrimeSim HSPICE and 3rd-party formats. Note: Supported models: BSIM3, BSIM4, BSIM-CMG, and BSIM-BULK (Resistor, Capacitor, and Vsource).

primesim_op_parasitic_capacitance

Description

Specifies whether to output parasitic passive capacitors into the OP result file.

Syntax

```
.option primesim_op_parasitic_capacitance=[0|1]
```

Arguments

Argument	Description
0	(default) Do not output parasitic passive capacitors into the OP result file.
1	Output parasitic passive capacitors into the OP result file.

primesim_op_region_display

Description

Specifies which values to use for different regions of MOS operation when using waveform display tools such as PrimeWave Design Environment WaveView.

Syntax

```
.option primesim_op_region_display=[0|1]
```

Arguments

Argument	Description
0	(default) Displays the following: • 0 – Cutoff region • 0.05 – Subthreshold region • 0.5 – Linear region • 1 – Saturation region
1	Displays the following: • 0 – Cutoff region • 1 – Linear region • 2 – Saturation region • 3 – Subthreshold region Note: This value is the default if using a 3rd-party netlist.

primesim_output

Sets the output file format.

Syntax

```
.option primesim_output=[fsdb|wdf|psf|psfx1|tr0|psfascii|none|out]
```

Description

Sets the output file format for saving transient analysis results, so waveform display tools, such as PrimeWave Design Environment WaveView, can display the result. The supported formats are FSDB, WDF, PSF, PSFXL, TR0, PSFASCII, and OUT. The default format is FSDB.

For `primesim_output=none`, the PrimeSim tool keeps measuring data, while ignoring all commands related with `.PROBE` and `.OPTION POST`.

Examples

Multiple values can be given to `primesim_output` and PrimeSim generates both output formats at the same time. Note that multiple outputs is only supported for the following formats: FSDB and WDF. Currently, TR0 and PSF are not supported.

```
.option primesim_output="fsdb wdf"
```

The preceding example outputs both FSDB and WDF files during the simulation.

primesim_output_ac

Sets the AC output file format as `tr0`.

Syntax

```
.option primesim_output_ac=[tr0]
```

Description

Setting this option as `tr0` can get AC related output files of `tr0` format.

Examples

```
.option primesim_output_ac=tr0
```

For AC and LSTB analysis, this generates `.ac0` and `.cx0` files.

primesim_output_fname_type

Description

Defines the naming convention for output files when the netlist has a `.ALTER` and/or `.TEMP / TEMPERATURE` sweep parameter.

Syntax

```
.option primesim_output_fname_type=[0|1|2]
```

Arguments

Argument	Description
0	(default) Use the PrimeSim output file naming convention.
1	Use the PrimeSim HSPICE output file naming convention.
2	Use the legacy HSPICE output file naming convention.
Note: If either <code>primesim_fsdb_limit</code> or <code>primesim_fsdb_split_time</code> are set, <code>primesim_output_fname_type=2</code> is set automatically.	

Examples

If the netlist has a `.alter` statement and two *data* sweeps, the output file names are generated as follows.

- PrimeSim file naming convention (`primesim_output_fname_type=0`):

```
netlist_file_prefix_s0.fsdb
netlist_file_prefix_s1.fsdb
netlist_file_prefix#1_s0.fsdb
netlist_file_prefix#1_s1.fsdb
netlist_file_prefix.fsdb.grp
```

- PrimeSim HSPICE file naming convention (`primesim_output_fname_type=1`):

```
netlist_file_prefix.a0.s0.fsdb
netlist_file_prefix.a0.s1.fsdb
netlist_file_prefix.a1.s0.fsdb
netlist_file_prefix.a1.s1.fsdb
netlist_file_prefix.fsdb.grp
```

- Legacy HSPICE file naming convention (`primesim_output_fname_type=2`):

```
netlist_file_prefix.a0.s0.fsdb
netlist_file_prefix.a0.s1.fsdb
netlist_file_prefix.a1.s0.fsdb
netlist_file_prefix.a1.s1.fsdb
netlist_file_prefix.fsdb.grp
```

If the netlist has a `.alter` statement and two *temperature* sweeps, the output file names are generated as follows:

- PrimeSim file naming convention (`primesim_output_fname_type=0`):

```
netlist_file_prefix.fsdb
netlist_file_prefix_t1.fsdb
netlist_file_prefix#1.fsdb
```

```
netlist_file_prefix#1_t1.fsdb  
netlist_file_prefix.fsdb.grp
```

- PrimeSim HSPICE file naming convention (`primesim_output_fname_type=1`):

```
netlist_file_prefix.a0.t0.fsdb  
netlist_file_prefix.a0.t1.fsdb  
netlist_file_prefix.a1.t0.fsdb  
netlist_file_prefix.a1.t1.fsdb  
netlist_file_prefix.fsdb.grp
```

- Legacy HSPICE file naming convention (`primesim_output_fname_type=2`):

```
netlist_file_prefix.a0.t0.fsdb  
netlist_file_prefix.a0.t1.fsdb  
netlist_file_prefix.a1.t0.fsdb  
netlist_file_prefix.a1.t1.fsdb  
netlist_file_prefix.fsdb.grp
```

Note:

This option sets the output file *naming convention* only; use the `primesim_output` option to specify the output file *format*.

The default output format for the PrimeSim tool is FSDB and the default output format for the PrimeSim HSPICE tool is TR0 (that is, `.option primesim_output=tr0`).

primesim_output_gzip

Description

Enables `gzip` file compression of the output files for the following:

- `.IC` results
- `.OP` results
- `.AC` / `.DC` / `.NOISE` results
- [Circuit Checks](#) commands results
- `.MEASURE` / `MEAS` results
- `.PRINT` results
- `.ALTER` results (`.tr#` files)

Note:

`gzip` file compression of the log file is not supported.

Syntax

```
.option primesim_output_gzip=[0|1]
```

Arguments

Argument	Description
0	(default) No <code>gzip</code> file compression.
1	Output files are compressed as <code>.gzip</code> files.

primesim_output_op

Description

Sets the output format for the operating point (.op) analysis.

Syntax

```
.option primesim_output_op = [ascii | wdf]
```

Arguments

Sets the output format for .op analysis, it can only be either *ascii* or *wdf*.

Argument	Description
<i>ascii</i>	Generates a text-based OP report.
<i>wdf</i>	Generates a <i>wdf</i> format OP report.

primesim_output_range

Limits output of transient results of waveforms to the waveform file. The default range is 0 to the end of the user-defined simulation time.

Syntax

```
.option primesim_output_range="start_time:end_time"  
.option primesim_output_range="start_time1[:end_time1] ... [-reset] ...  
[start_timeN[:end_timeN]]"
```

Description

This option can limit output of transient results of waveforms to the waveform file. All time variables can be parameterized.

The optional `-reset` keyword clears all prior variables and is used when you want the new output range to supersede all previous output ranges for a `.ALTER` rerun of the simulation.

For example:

```
.option primesim_output_range="t1:t2 t3:t4 t5"  
.option primesim_output_range="2n:20n"
```

This option outputs waveforms to the waveform file from 2n to 20n.

```
.option primesim_output_range="2n:20n 30n:40n"
```

This option outputs waveforms to the waveform file from 2n to 20n and again from 30n to 40n.

```
.option primesim_output_range="1n:5n"  
.option primesim_output_range="10n:15n"  
.alter  
.option primesim_output_range="-reset 5n:10n"
```

In this example, the first run generates output with a range from 1n to 5n and then from 10n to 15n, while the second `.ALTER` run generates output with a range of from 5n to 10n.

primesim_parihier_mode

Description

Specifies how the SPICE-compatible `.OPTION PARHIER` functions when set to `LOCAL`.

Syntax

```
.option primesim_parihier_mode=[0|1]
```

Arguments

Argument	Description
0	(default) When <code>.option parhier=LOCAL</code> , follow the PrimeSim description: During analysis of a subcircuit, a parameter name specified in the subcircuit prevails over the same parameter name specified at a higher hierarchical level.
1	When <code>.option parhier=LOCAL</code> , follow the PrimeSim HSPICE description: All parameters defined in the context of a subcircuit definition remain local to that subcircuit (the lines between <code>.SUBCKT</code> and <code>.ENDS</code>). This applies to any parameters defined in <code>.INCLUDE</code> or <code>.LIB</code> commands referenced within the subcircuit name space.

primesim_partition

Specifies how partitioning is done.

Syntax

```
.option primesim_partition=[0|1|2]
```

Arguments

Argument	Description
0	No partition.
1	Conservative partitioning.
2	Moderate partitioning.

Description

The higher the value specified, the more aggressive the partitioning.

The default value is determined by the [primesim_mode](#) option (see [Table 26](#)).

If a netlist is flat, 1 and 2 are equivalent because no hierarchy boundary exists.

In the PrimeSim Pro tool non-ideal power analysis feature, the partition is further divided into power rail RC partitions and MOSFET partitions. Refer to [Back-Annotation](#) for details on non-ideal power analysis.

Examples

```
.option primesim_partition=SUBCKT1:0
.option primesim_partition="1 SUBCKT1:0"
.option primesim_partition="1 SUBCKT1:0 SUBCKT2:0"
.option primesim_partition="0 SUBCKT1:1 SUBCKT2:2"
```

In the first example, SUBCKT1 has no partition but the rest of the circuit is partitioned by [primesim_mode](#).

In the second example, SUBCKT1 has no partition while the rest of the circuit is partitioned by 1, which overrides the [primesim_mode](#) partition setting.

In the third example, SUBCKT1 and SUBCKT2 have no partitions while the rest of the circuit is partitioned by option 1.

In the last example, suppose SUBCKT2 is within SUBCKT1. SUBCKT2 is partitioned by 2 while the rest of SUBCKT1 is partitioned by 1. The rest of the circuit is partitioned by 0.

In SPICE modes, the PrimeSim tool does not partition the design. If you use `primesim_partition=1|2` in SPICE modes, then the design is partitioned and promoted to a Pro mode simulation requiring Pro mode license keys. However, if `-spice` is specified on the PrimeSim command line, then the `primesim_partition` option is ignored and the simulation remains as a SPICE mode simulation.

primesim_pdn

Note:

This option is for the PrimeSim Pro tool only.

Description

When enabled, this option provides autodetection of power and ground nets and optimized RC reduction in PDN (Power Distribution Network) design simulations, and enables advanced event control between circuit and power networks automatically.

Syntax

```
.option primesim_pdn=[0|1|2]
```

Arguments

Argument	Description
0	(default) Disables this option so that it has no effect on the simulation.
1	Enables this option for better accuracy.
2	Enables this option for better performance.

primesim_phasenoise_lorentz

Description

Note:

This option is for the PrimeSim SPICE tool only.

Used with `.PHASENOISE` analysis, specifies whether to calculate and output the Lorentzian corner frequency and apply the Lorentzian model on phase noise at a low frequency offset.

The Lorentzian model of signal sided band PSD of normalized phase noise is given by the following equation:

$$L(f) = \frac{f_c}{\pi(f_c^2 + f^2)}$$

where f_c denotes the Lorentzian corner frequency. The Lorentzian model is applied at $f < 3f_c$ if this option is specified as 1.

Note:

If keyword LORENTZ is specified in the .PHASENOISE command, it overrides this option.

Syntax

```
.option primesim_phasenoise_lorentz=[0|1]
```

Arguments

Argument	Description
0	Neither the Lorentzian corner frequency nor the Lorentzian model are calculated/applied.
1	The Lorentzian corner frequency is calculated and output, and the Lorentzian model is applied on the phase noise result at a low frequency offset.
Unspecified	The Lorentzian corner frequency is calculated and output, but the Lorentzian model is not applied on the phase noise result.

primesim_phasenoise_min_harmfreq_offset

Description

Note:

This option is for the PrimeSim SPICE tool only.

In phase noise analysis, the output noise approaches infinity when the output frequency gets close to harmonic frequencies. Use this option to skip frequency points that are too close to harmonic frequencies.

When used with the .PHASENOISE command to perform phase noise analysis, this option skips output frequency points that satisfy the following criteria:

$$|freq - N * fund| < value, \quad N=1, 2, 3, \dots$$

where *freq* stands for the output frequency, *fund* stands for the fundamental frequency, and *value* is specified with this option.

Syntax

```
.option primesim_phasenoise_min_harmfreq_offset=value
```

Examples

In the following example, the fundamental frequency *fund* is 10M, and the offset specified by this option is 1M (or *fund*/10), so all frequency points are skipped at 9M-11M, 19M-21M, 29M-31M, and so on.

```
.option primesim_phasenoise_min_harmfreq_offset=fund/10
```

primesim_pinst_in_p

Description

Specifies how the `p()` power reporting function works when [Probing and Measuring](#).

Syntax

```
.option primesim_pinst_in_p=[0|1]
```

Arguments

Argument	Description
0	(default) <code>p(device_name)</code> is used to report power for devices, while <code>pinst(instance_name)</code> is used to report power for instances. See Support for Power Reporting for more information.
1	The <code>p()</code> function can be used to report power for both devices and instances. Note: Setting this option can result in some performance loss.

primesim_power_mode

Description

Use this option when an accurate current measurement is required, especially at the power block partition. The default is 0.

Syntax

```
.option primesim_power_mode=[0|1]
```

Examples

```
.option primesim_power_mode=1
```

primesim_prb_skipmeas

Description

Use this option to skip saving probed signals from `.MEASURE` statements in the FSDB file. The default is 0.

Syntax

```
.option primesim_prb_skipmeas=[0|1]
```

primesim_prb_tmin

Description

Set this option if you want to skip saving data points too frequently. The waveform file records data points only after the minimum transient time set by this option has passed. The default behavior is to output all points.

Note:

Setting a *value* smaller than the minimum time step (based on the PrimeSim mode) results in a warning message.

Syntax

```
.option primesim_prb_tmin=value
```

primesim_prbexprvar

Description

Determines whether to save any signals in the expression of a `.PROBE` statement.

Syntax

```
.option primesim_prbexprvar=[0|1]
```

Arguments

Argument	Description
0	PrimeSim tool does not save any expressed signals.
Note: This value is the default if using a 3rd-party netlist.	

Argument	Description
1	(default) PrimeSim Pro tool saves all specified signals.

Examples

```
.probe prb_diff=par('v(n1)-v(n2)')  
.option primesim_prbexprvar=0
```

In this case, the PrimeSim tool saves only the `prb_diff` signal into output file. But if the option is set to 1, the PrimeSim tool saves `prb_diff` and `v(n1)`, `v(n2)` as well.

primesim_prbport

Description

Determines whether the port voltages is probed, provided the voltage change is larger than the tolerance in [primesim_vprbtol](#).

Syntax

```
.option primesim_prbport=[0|1|2]
```

Arguments

Argument	Description
0	No print out of port voltages.
1	(default) Prints out the port voltage of all subcircuit instances if its variation is larger than the tolerance defined by primesim_vprbtol . This option prints the port voltages only when <code>.probe v(*)</code> or <code>post=1</code> is set.
2	Similar to 1, but it does not require <code>.probe v(*)</code> or <code>post=1</code> and they are ignored. This option also prints global signals.

primesim_prbtol_disable_visrc

Description

When set to 1, this option disables [primesim_vprbtol](#) or [primesim_iprbtol](#) for any independent voltage source(V)'s node voltage and current source(I)'s current, respectively. These ideal sources should be connected with absolute ground node.

The default is 0.

Syntax

```
.option primesim_prbtol_disable_vsrc=[0|1]
```

primesim_precision

Note:

This option is for the PrimeSim SPICE tool only.

Specifies whether to serve as the reference with the highest precision and ignore the `runlvl` option.

Syntax

```
.option primesim_precision=0|1
```

Description

When set to 1, this option serves as the reference with the highest precision (see [Table 27](#)) and the `runlvl` option is ignored.

When this option is set to 0, the `runlvl` option is used as normal (see Table in [PrimeSim SPICE Accuracy and Speed Options \(-runlvl and .option runlvl\)](#)).

Note:

If no value specified (`.option primesim_precision`), the PrimeSim SPICE tool issues the following message:

```
WARNING! value should be specified for option primesim_precision  
and the option will be ignored.
```

[Table 27](#) lists how performance and accuracy PrimeSim options are preset when `primesim_precision=1`.

Table 27 *Ease-of-Use Settings for primesim_precision=1*

	<code>primesim_precision=1</code>
<code>primesim_tolscale</code>	1
<code>primesim_rcred</code>	0
<code>primesim_model</code>	4.5
<code>primesim_tunit</code>	1e-16
<code>primesim_sparse_fcap</code>	1e-18
<code>primesim_cktopt</code>	0

Table 27 Ease-of-Use Settings for `primesim_precision=1` (Continued)

	<code>primesim_precision=1</code>
<code>primesim_resmin</code>	1e-6

primesim_prelayout_models

Sets which model the PrimeSim Pro tool uses.

Syntax

```
.option primesim_prelayout_models=[0|1]
```

Description

Sets which model the PrimeSim Pro tool uses: the model in the pre-layout netlist or in the SPF file. Usually, different models are used for the pre-layout netlist and post-layout netlist. When running simulation with the pre-layout netlist and back-annotation with SPF extracted from layout, the value for this option can decide which model should be used.

The default value for `primesim_prelayout_models` is 0.

Note:

This option applies only to SPF back-annotation (`primesim_spf`) and not DPF back-annotation (`primesim_dpf`).

Examples

```
schematic netlist:
M1 a b c d NCH l=45e-9 w=180e-9
```

```
SPF:
M1 a b c d NMOS l=45e-9 w=90e-9 ...
M1@2 a b c d NMOS l=45e-9 w=90e-9 .....
```

```
.option primesim_prelayout_models=1
```

The PrimeSim Pro tool back-annotates M1 and M1@2 with NCH instead of NMOS.

```
.option primesim_prelayout_models=0
```

The PrimeSim Pro tool back-annotates M1 and M1@2 with NMOS instead of NCH.

primesim_print_global_param

Prints global parameter names and resolved values.

Syntax

```
.option primesim_print_global_param=[0|1]
```

Description

When set to 1, the `output_prefix.globalparam` file is generated, and prints global parameter variable names and resolved values in line by line. The default value is 0.

Examples

```
.option primesim_print_global_param=1
```

The PrimeSim tool generates a file, `output.globalparam`, which contains the following:

```
vdd_val = 2.5  
vss_val = 0
```

primesim_print_max_con_node

Identifies nodes with the greatest number of connections.

Syntax

```
.option primesim_print_max_con_node=value
```

Description

With the `primesim_print_max_con_node` option, you can identify nodes with the greatest number of connections for which a connectivity report is desired. Then you can use this information with the `primesim_cutnode` option to reduce partition size for signal nodes.

The value of `primesim_print_max_con_node` is an integer identifying the number of nodes that you want to report.

The default value is 20 for multi-partition modes.

Examples

```
.option primesim_print_max_con_node=5
```

In this example, the PrimeSim Pro tool prints out the top five nodes with maximum connections in the largest partition. The node names output by this option correspond to nodes that are connected to many devices. Reducing the size of the largest partition can reduce the maximum partition size and allow more efficient simulation.

primesim_print_period

Defines the fixed interval at which the `.PRINT` command prints to the output file.

Syntax

```
.option primesim_print_period=time_period
```

Description

The default value is set to print per each time point. If you set this option to a small value, it can result in additional time points being created.

Examples

```
.option primesim_print_period=10p
```

In the preceding example, the PrimeSim tool outputs the `.print` file every 10p seconds.

primesim_print_rchk_resource

Description

When set to 1, this option reports the detailed time and memory usage for the `-rchk` command-line option when rechecking the database (see [Command-Line Options](#)). The default is 0.

Syntax

```
.option primesim_print_rchk_resource=[0|1]
```

primesim_print_to_probe

Converts all `.PRINT` statements to `.PROBE` statements.

Syntax

```
.option primesim_print_to_probe=[0|1]
```

Description

When set to 1, all `.print` statements automatically convert to `.probe` statements.

For the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE modes (`spicehd`, `spicemd`, and `spicexd`), the default is 0, which keeps all `.print` statements and outputs `.pt#` files.

For the PrimeSim Pro tool's Pro modes (`prohd`, `promd`, and `proxd`), the default is 1.

primesim_prll_opt

Description

Controls how the `-np x` command line option functions (see [Command-Line Options for Running Multi-CPU Simulations](#)).

Syntax

```
.option primesim_prll_opt=[0|1]
```

Arguments

Argument	Description
0	(default) If the simulator cannot find a parallelism with the specified number of CPUs (<code>-np x</code>) in SPICE mode or single partitioned Pro mode, then it uses one CPU.
1	The simulator attempts to find a more optimized number of CPUs and then uses it.

primesim_pro_dcalg

Description

Sets the approach for DC convergence in the PrimeSim Pro tool for Pro (FastSPICE) modes only.

Note:

This option is for the PrimeSim Pro tool's Pro modes only. For SPICE modes, see [primesim_dcalg](#).

Syntax

```
.option primesim_pro_dcalg=[1|2]
```

Arguments

Argument	Description
1	Specifies that DC convergence proceeds as follows: 1.Attempt an event-based pre-initialization step. 2.If convergence is not achieved, then use a global pseudo-transient simulation (GPT) ramping down <code>cshunt</code> . Note: This is the recommended setting for power-up cases.

Argument	Description
2	(default) Specifies that DC convergence proceeds as follows: 1.Use a global pseudo-transient (GPT) simulation ramping down <code>cshunt</code> . 2.Perform a finalization step to correct nodes with large time constants. Note: This is both the default and recommended setting for general use.

primesim_probe_passive_device

Description

Specifies whether to probe passive devices.

Syntax

```
.option primesim_probe_passive_device=[0|1|2]
```

Arguments

Argument	Description
0	(default) Does not probe passive devices.
1	Probes all passive devices.
2	Probes all passive devices, except for parasitic RCs. Note: This setting is the default in the PrimeWave Design Environment if the device port current is probed (that is, <code>.probe iall(*)</code>).

primesim_probe_protected

Description

Specifies whether to allow the probing of subcircuits within protected or encrypted blocks.

Syntax

```
.option primesim_probe_protected=[0|1]
```

Arguments

Argument	Description
0	Do not allow the probing of subcircuits within protected or encrypted blocks. This corresponds to PrimeSim HSPICE behavior.
1	(default) Allow the probing of subcircuits within protected or encrypted blocks.

primesim_probe_resource_usage

Description

Probes the CPU time and physical memory usage as a function of the transient simulation time. This option is supported only when `primesim_output` is set to `fsdb` or `wdf`. This command helps you detect bottlenecks when the CPU time is increasing faster than the transient simulation time.

Syntax

Syntax

```
.option primesim_probe_resource_usage="0|1 [-cpu=0|1] [-wall=0|1]
[-vm=0|1] [-pm=0|1] [-cpu_load=0|1] [-remaining=0|1] [-all=0|1]"
```

Arguments

Argument	Description
-cpu=0 1	Control to probe the CPU time versus the transient simulation time in the waveform file.
-wall=0 1	Control to probe the wall time versus the transient simulation time in the waveform file.
-vm=0 1	Control to probe the virtual memory usage versus the transient simulation time in the waveform file.
-pm=0 1	Control to probe the peak physical memory (which resides in core memory) versus the transient simulation time in the waveform file.
-cpu_load=0 1	Control to probe the CPU load in the waveform file.
-remaining=0 1	Control to probe the estimated remaining time in the waveform file.
-all=0 1	Control to probe all of the above resources.

Examples

The following syntax probes the CPU time versus transient time and the physical memory used versus transient time.

```
.option primesim_probe_resource_usage=1
```

The following syntax probes CPU time, physical memory used, virtual memory used, and CPU load during transient time of the simulation.

```
.option primesim_probe_resource_usage="1 -vm=1 -cpu_load=1"
```

primesim_profile

Controls profiling analysis output for chip-level simulation. If `.option primesim_profile=[1|2]` is given, profile data is generated in the file `header.prof` after transient analysis. If `primesim_profile` is not defined, then no profile data is generated.

Syntax

```
.option primesim_profile=[1|2]
```

Arguments

Argument	Description
1	Prints the summary of simulation cost for all devices in the subcircuit.
2	Prints the simulation cost for each device.

Description

The profile data is formatted as follows:

```
.subckt subckt_name number_of_usage evaluation_cost solving_cost  
device_name number_of_evaluation evaluation_cost solving_cost  
instance_name subckt_name evaluation_cost solving_cost  
.ends
```

The profile report does not show all of the instances because many of them are the same subckt and are grouped and may use a pre-existing solution.

In the report, the `evaluation_cost` of a device/instance is the number of matrix evaluations performed in the simulation, as a percentage of total matrix evaluations.

The `solving_cost` of a device/instance is a percentage in terms of solving the complexity of the matrix. It is a relative number.

Both of these numbers are provided to give you a guide as to where the simulator is using most of its resources in computation.

primesim_pseudotrandbg

Controls whether an FSDB file is generated from pseudo-transient analysis.

Syntax

```
.option primesim_pseudotrandbg=[0|1]
```

Description

When set to 1, the PrimeSim Pro tool generates an FSDB file with waveforms from the pseudo-transient analysis step. This waveform file may be helpful for debugging DC convergence issues.

The default value is 0.

Note:

This option is available for use in PrimeSim Pro tool's Pro mode simulations only (that is, `prohd`, `promd`, and `proxd`). It is not available in the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE mode simulations (that is, `spicehd`, `spicemd`, and `spicexd`).

In addition, this option is only available when the output format is `fsdb`. It is not available for `tr0`, `psf`, or `wdf` formats.

primesim_psfdir

Description

Specifies the directory in which to write the PSF- formatted output data files. You can also specify this directory with a command line option:

```
... -psfdir ../PSF_RESULTS
```

Syntax

```
.option primesim_psfdir target_psf_directory
```

Examples

```
.option primesim_psfdir="PSF_RESULTS"
```

In the first example, `.option primesim_psfdir` is either in the netlist or a `primesim.cfg` file. The PSF output files are dumped to the `../PSF_RESULTS` directory.

```
> primesim -format psf -psfdir ../psf input.sp
```

In the second example, `-psfdir` is a command-line option. The PSF output files are dumped to the `../psf` directory.

primesim_pt0_format

Description

By default, the `.PRINT` outputs a file in table format `[time point\node]`. This can be changed into a list format where it prints the time point value for each node.

Syntax

```
.option primesim_pt0_format=[table|list]
```

Examples

```
.option primesim_pt0_format=[table|list] (default is table)
```

Table

time	v(out)	v(in)
0.0000e+00	3.6729e-03	0.0000e+00
1.0000e-11	3.6735e-03	0.0000e+00

List

```

    TIME v(in)
      0.0  0.0
    5.0000n 0.0
...
    TIME v(out)
      0.0  3.6729m
    10.0000p 3.6729m
...

```

primesim_pva_cross_ttol

Description

Specifies the PrimeSim / Portable Verilog-A Engine timing accuracy for the `CROSS` and `ABOVE` functions.

Syntax

```
.option primesim_pva_cross_ttol=[0|1|2]
```

Arguments

Argument	Description
0	(default) Timing accuracy is determined by an internal formula.
1	For accurate simulation, the timing error is the value of <code>primesim_tunit</code> .
2	For better performance with good accuracy, the timing error is the value of (2 * <code>primesim_tunit</code>).

Examples

```
.option primesim_pva_cross_ttol=2
```

primesim_pva_flow

Description

Simulates Verilog-A elements using the Portable Verilog-A Engine compatible with PrimeSim HSPICE.

Syntax

```
.option primesim_pva_flow=[0|1]
```

Arguments

Argument	Description
0	Use the PrimeSim native Verilog-A simulation.
1	Use the Portable Verilog-A simulation compatible with PrimeSim HSPICE. Note: This is the default setting for both the PrimeSim SPICE and Pro tools.

primesim_pva_opt_port

Description

When set to 1, the PrimeSim tool treats the Verilog-A port as non-voltage, and so not subject to per-iteration limiting. This may help with Verilog-A accuracy issues. The default is 0.

Note:

This option is only available when `primesim_pva_flow=1`.

Syntax

```
.option primesim_pva_opt_port=[0|1]
```

primesim_pwl_r_end

Changes the meaning of parameter R for PWL source.

Syntax

```
.option primesim_pwl_r_end=[0|1]
```

Description

By default, `primesim_pwl_r_end=0`, R specifies the starting point of the waveform to be repeated.

With `primesim_pwl_r_end=1`, R specifies the time to be subtracted from the last break point. The difference computed is the starting point of the waveform to be repeated. This usage is equivalent to the parameter, `pwlperiod`, for 3rd-party PWL source.

Examples

```
v0 in 0 dc=0 pwl (5n 0.0 6n 3.0 10n 3.0 11n 0.0 r=2n td=0 )
```

In this example, if `primesim_pwl_r_end=1`, with `R=2n`, the last 2ns of the PWL waveform ending at 11ns is repeated.

However, if `primesim_pwl_r_end=0`, the PWL waveform between 2ns to 11ns is repeated starting at 11ns.

primesim_pwrblock

Detects the parasitics in the power network.

Syntax

```
.option primesim_pwrblock="[0|1] net/nodename[:0|1] subckt(port) "
```

Description

This option is used to detect the parasitics in the power network, as well as the transistors that control or regulate the power domains. You can use this option when you have, for example, headers and footers that are used for power management in a design. The default value is 1 in Pro modes; this option does not apply to SPICE modes.

The PrimeSim Pro tool automatically traces ideal and non-ideal power supplies to determine if the network is a power candidate, and treat it accordingly. You can also manually specify the port of non-ideal voltage output or the net/node that should be treated as power.

```
.option primesim_pwrblock="[0|1] net/nodename[:0|1] subckt(port) "
```

When this option is set to 0, automatic power block detection is turned off. When set to 1, automatic detection is turned on. A net name or node name can be given as a potential power block candidate. You can similarly turn off the detection for a net name or node name. You can also give the *subckt(port)* specification for power block detection, where the *subckt* is the block generating the power supply and the *port* is the driving port from the voltage generation subcircuit.

Note:

When a particular node/net is set with this option, the node/net is considered to be a *potential* power block candidate. However, if after internal partitioning, the PrimeSim Pro tool finds that the partitioning for the user-defined candidate is making very small partitions, then it may be canceled.

Examples

```
.option primesim_pwrblock="1 subckt1(port1,port2) "
```

In the first example, *subckt1* is an internally-generating power supply subcircuit, not an instance. In addition, *port1*, *port2* are the driving ports of the subcircuit.

```
.option primesim_pwrblock="1 subckt1(port1) subckt2(port2) "
```

In the second example, *subckt1* and *subckt2* are two internally-generating power blocks, and *port1*, *port2* are their respective driving ports.

primesim_pwrloadmodel

Enables a power load model especially well-suited for power supply networks.

Syntax

```
.option primesim_pwrloadmodel=[1|2|3]
```

Arguments

Argument	Description
1	Standard power load model for quick functional simulation.
2	More thorough power load model at the power partition boundary that uses capacitance and current information to model the currents more accurately.
3	(default) More accurate and efficient power load model (compared to 2) which considers more coupling effects.

Description

Power supply networks are becoming larger and more complex, while the need for power-gated designs is on the increase due to demand for lower power requirements. The detection of these power networks has become important so that they can be properly partitioned to get runtime benefits with negligible loss in accuracy. This also applies to internally-generated power supplies such as those in flash designs.

primesim_pwrtol

Description

Specifies the voltage tolerance setting used to control an event between a power block partition and the other partitions. A smaller value generally makes the non-ideal power signal waveforms with more detail, sacrificing runtime. The default value is 3mv.

Syntax

```
.option primesim_pwrtol=value
```

primesim_ra_cmd

Description

Gets output format preferences. Titan IR/EM users can also use this option to provide the old command format supported by Titan/fscript.

Syntax

```
.option primesim_ra_cmd="key:value" [key2:value2]
```

Arguments

Argument	Description
Out	Output format specifier.
CMD	fscript/Titan command file.

Examples

```
option primesim_ra_cmd="out:text"
```

Specifies text output of Vmax and Vavg (IR drop information).

```
.option primesim_ra_cmd="cmd:old.tcl"
```

Specifies the Titan/fscript command file for IR/EM. You should not use this option with `primesim_ra_tcl`.

primesim_ra_net

Description

Specifies power nets for IR Drop analysis only. Power nets and their reference information are provided in this command to run IR Drop analysis.

Syntax

```
.option primesim_ra_net="net_name:level"
```

Examples

```
.option primesim_ra_net="vdd:2.5"
```

This example chooses the VDD net with a 2.5V reference.

primesim_ra_tcl

Description

Reads a PrimeSim XA RA Tcl file to run and create an EM analysis report. The PrimeSim tool can directly read a PrimeSim XA RA Tcl file and report IR/EM results.

Syntax

```
.option primesim_ra_tcl="ra_tcl_file"
```

Examples

```
.option primesim_ra_tcl="ra.tcl"
```

primesim_rclamp

Description

The PrimeSim tool ignores (that is, considers as short nodes) those resistors where $r < \text{the value}$ set by the `primesim_resmin` option, except for some important resistors such as a resistor in a V-R loop, a resistor connected to an inductor, and so on.

These remaining resistors are clamped to the *value* set by option `primesim_rclamp` if $r < \text{that value}$.

The default is $1e-5$ for the PrimeSim SPICE tool with PrimeSim HSPICE format, and $1e-3$ for the PrimeSim Pro tool's SPICE modes (`spicehd`, `spicemd`, and `spicexd`).

Syntax

```
.option primesim_rclamp=value
```

Examples

```
.option primesim_rclamp=1e-5
```

primesim_rcred

Description

Sets the level of acceleration to use when performing post-layout analog simulation with RC reduction. The default is 0.

Note:

This option should be used with the PrimeSim SPICE tool and the PrimeSim Pro tool's SPICE mode simulations only (that is, `spicehd`, `spicemd`, and `spicexd`).

This option is suitable for post-layout analog circuits in advanced nodes and supports back-annotated netlists, flattened netlists, and netlists with an included SPF file.

Syntax

```
.option primesim_rcred=[0|1|2|3]
```

Arguments

Argument	Description
0	(default) No post-layout analog simulation with RC reduction.
1	Conservative post-layout analog simulation with RC reduction.
2	Moderate post-layout analog simulation with RC reduction.
3	Aggressive post-layout analog simulation with RC reduction.

primesim_rcredscale

Description

Sets a scaling factor applied for RC reduction performed using the `primesim_rcred` command. The higher the value, the more aggressive the RC reduction applied.

Note:

If the netlist includes multiple `primesim_rcredscale` options, the last value overrides any previous ones.

Syntax

```
.option primesim_rcredscale=value
```

Arguments

Argument	Description
<i>value</i>	RC reduction scaling factor larger than 1.0.

Examples

```
.option primesim_rcred=3 primesim_rcredscale = 3
```

primesim_remove_floating_vsrc

Description

When set to 1, the PrimeSim tool removes any floating voltage sources and issues a warning message. The default setting is 0.

Syntax

```
.option primesim_remove_floating_vsrc=[0|1]
```

primesim_remove_probe_prefix

Description

When this option is set to 1, signals from `.PROBE` statements do not have a probe prefix. This option works on FSDB and WDF formats only. The default value is 0.

This option does not affect prefixes for signals saved with third-party format `save` statements.

Note:

The `fscript` utility (see [Using fscript](#)) cannot perform post-simulation measurements when waveform output is generated without a prefix (that is, when `primesim_remove_probe_prefix=1`).

Syntax

```
.option primesim_remove_probe_prefix=[0|1]
```

primesim_remove_va_so_files

Controls keeping or removing .so files after a simulation.

Syntax

```
.option primesim_remove_va_so_files=[0|1]
```

Description

When `primesim_remove_va_so_files=1` is set, all the temporary Verilog-A files are removed; in addition, the same Verilog-A file does not share the .so with other processes. Instead, each process compiles and uses its own Verilog-A library and is deleted after the simulation completes.

The default value is 0, which keeps all .so files after the simulation. When set to 1, it removes the .so files generated from Verilog-A files after being loaded for further simulation.

```
.option primesim_remove_va_so_files=1
```

The use of this option forces the regeneration of the shared-object (.so) file for every run and thus incurs a time penalty.

primesim_repdot

Description

Replaces any periods (.) with another character during back-annotation.

Syntax

```
.option primesim_repdot=character
```

Examples

In the following example, the PrimeSim parser replaces periods (.) with underscore (_) characters during SPF annotation.

```
.option primesim_repdot="_"
```

primesim_report_large_no_instances

Description

Reports subckt instances that contain *value* or more devices, where *value* is a large number (for example, 10000) and is the minimum number of devices contained in the subckt instance.

Note:

The count is totaled from a recursive search.

Syntax

```
.option primesim_report_large_no_instances=value
```

Examples

```
.option primesim_report_large_no_instances=10000
```

primesim_resmax

Description

Sets the maximum value for a resistor to be considered in simulation. Resistors with a larger value are ignored and considered open nodes. The defaults are as follows:

- PrimeSim Pro: 100G (1e11) ohms
- PrimeSim SPICE: 10T (1e13) ohms

Syntax

```
.option primesim_resmax=value
```

Examples

```
.option primesim_resmax=0.5T
```

In this example, resistors larger than 0.5T ohms are considered open nodes.

primesim_resmin

Description

Sets the minimum value for a resistor to be considered in simulation. The default is 0.1 ohm for Pro (FastSPICE) modes and 0.001 for all SPICE modes, except when `primesim_precision=1`, in which case, the `primesim_resmin` default is 1e-6.

Resistors in the netlist with a smaller value are ignored and considered short nodes. Resistors in the SPF or DSPF file, with an equal or smaller value, are ignored and considered short nodes.

Note:

Because the `primesim_negres` option is set to 1 by default, resistors with values between 0 and the negative `primesim_resmin` value are also ignored and shorted. If `primesim_negres=0` is set, all negative resistors are ignored and shorted.

Syntax

```
.option primesim_resmin=value
```

primesim_restore (.SNAPSHOT)

The `.snapshot` option provides a way to capture a snapshot of the dynamic state of the circuit in motion for later use with the `primesim_restore` option.

Syntax

```
.option primesim_restore=file_name snapshot
```

Description

The `.snapshot` and `primesim_restore` options together provide a way to capture a snapshot of the dynamic state of the circuit in motion for later use with the `primesim_restore` option, which resumes simulations where the snapshot was taken. This allows you to fast-forward to an interesting portion of a simulation or record checkpoints to guard against machine crashes, power outages, or other unexpected termination of simulation in cases in which the runtime is very long.

```
.snapshot [file=aaa] time=(10ns,20ns,30ns)
.snapshot [file=aaa] elapsed=seconds
.snapshot [file=aaa] [start=0ns] [stop=100ns] period=10ns
.snapshot ... keep=value
```

The `keep` option specifies the number of snapshot files you want to keep during the simulation. Without the `keep` option, `.snapshot` creates 10 snapshot files. Using `keep=1` keeps the last snapshot. This option helps in case a simulation is killed inadvertently as only the last snapshot is needed to restore the simulation.

The default file name is the prefix of the input netlist file just like the other output files.

The default values of `start` and `stop` are 0 and `tstop` of `.tran`, respectively.

The snapshot of the dynamic state can be restored using the `primesim_restore` option:

```
.option primesim_restore=aaa_10ns.snapshot
```

Note:

`.snapshot` is not supported for frequency-dependent elements. It also may not store internal states of Verilog-A/B-elements.

If there are frequency-dependent elements, then the PrimeSim tool ignores the `.snapshot` and continues the simulation. If there are frequency-dependent elements and there is a `primesim_restore` or `simstart` statement, the PrimeSim tool exits with an appropriate warning.

primesim_reuse_mos_model

Description

Specifies whether to reuse MOSFET models. This option is applied to simulation with sweep, such as DC sweep.

Syntax

```
.option primesim_reuse_mos_model=[0|1|2]
```

Arguments

Argument	Description
0	The PrimeSim Pro tool builds a MOSFET model in each sweep, regardless of the result in previous iterations. Rebuilding MOSFET models can result in a little longer runtime, while saving memory usage.
1	(default) The PrimeSim Pro tool reuses MOSFET models built in previous iterations in order to save time between DC and tran.
2	The option is applied to simulation with <code>.ALTER</code> statements, as well as sweep. Note: This is set automatically when using either <code>primesim_fast_corner</code> and <code>primesim_fast_corner_sweep</code> .

primesim_reuse_op

Description

Enables the PrimeSim SPICE tool to reuse the operating point solution from a previous analysis as the initial guess for subsequent analyses, when there are multiple analyses, or for Monte Carlo sweep simulation.

For example, when there are two analyses on the input netlist running Monte Carlo simulation, with an AC analysis followed by a transient analysis, the PrimeSim tool first solves for the operating point in the AC analysis, then it reuses the operating point solution from the AC analysis as an initial guess in the subsequent transient analysis. As a result, simulation performance can be improved.

Note:

This option has no effect on RF analyses.

Syntax

```
.option primesim_reuse_op=[0|1|2]
```

Arguments

Argument	Description
0	Disable cross-analyses operating point reuse.
1	(default) Reuse the operating point from the previous analysis <i>if it is suitable</i> . That is, the simulator may skip the operating point reuse if it is not a good choice, for example, when sources have different operating point values for DC analysis and transient analysis.
2	Force reuse of the operating point from the previous analysis.

primesim_rng

Description

Specifies the random number generator (RNG) used by the simulator.

Syntax

```
.option primesim_rng=[0|1]
```

Arguments

Argument	Description
0	Use the PrimeSim random number generator.
1	(default) Use the PrimeSim HSPICE random number generator.

primesim_rpitft_mode

Description

This option, set to 1, enables PrimeSim tool support for SmartSpice TFT Model correlation. The default is 0. In a PrimeSim HSPICE format netlist and model, the PrimeSim tool matches with the PrimeSim HSPICE result for RPI TFT model (level=62, version=2). In a 3rd-party format netlist and model, the PrimeSim tool matches with the 3rd-party result.

Note:

If temperature is not given in the input netlist, use `.temp 27` to match with the SmartSpice result.

Syntax

```
.option primesim_rpitft_mode=[0|1]
```

primesim_runtime_meas

Description

Set this option to process measurements during runtime instead of post-processing them. The measures then use the simulation data directly rather than using data from the output file. This makes the results independent of any tolerances or resolutions set for the output through options such as `primesim_vprbtol`, `primesim_iprbtol`, or `.OPTION INTERP`.

Note:

This option does not affect measurements related to `.FFT`, `.FOUR`, or [Continuous Measure](#). Such measurements are always computed by the PrimeSim tool after simulation is finished.

Syntax

```
.option primesim_runtime_meas=[0|1]
```

Arguments

Argument	Description
0	Run measure commands on output data.
1	(default) Use runtime measurement instead of post measurement.

primesim_scale

Applies the scale factor for subcircuits.

Syntax

```
.option primesim_scale="subckt_name:scale_factor ..."
```

Description

The global scale factor is controlled by `.option scale`. This option applies to subcircuits only. For example:

```
.option primesim_scale="inv:1u inv2:1.0"
```

primesim_set_cpu_time

Description

When specified, the PrimeSim tool errors out if the simulation time exceeds the specified threshold. The default time value is in minutes. If `primesim_set_cpu_time` is not defined, there is no limit to the simulation time.

Syntax

```
.option primesim_set_cpu_time=time
```

primesim_set_floating_node

Description

Sets a user-specified voltage value for either all floating nodes or floating gate nodes only (see [primesim_set_floating_node_type](#)). They can be set at either the DC operating point or also throughout the transient simulation (see [primesim_set_floating_node_format](#)).

Syntax

```
.option primesim_set_floating_node="value  
[-n_gate=ngate_value] [-p_gate=pgate_value]  
[-type=0|1] [-format=0|1] [-outfile=0|1] [-power_auto_detect=0|1]"
```

Arguments

Argument	Description
<i>value</i>	Value for all floating nodes/gates depending on <code>-type</code> .
<code>-n_gate=ngate_value</code>	Sets <i>ngate_value</i> on NMOS device gates. Note: If the NMOS/PMOS gate is tied to the same net, the NMOS gate value takes priority.
<code>-p_gate=pgate_value</code>	Sets <i>pgate_value</i> on PMOS device gates. Note: If the NMOS/PMOS gate is tied to the same net, the NMOS gate value takes priority.
<code>-type=0 1</code>	The same as primesim_set_floating_node_type : 0: (default) Sets all floating nodes. 1: Sets only MOSFET gates. Note: This argument is not applicable when using the <code>-n_gate</code> and <code>-p_gate</code> options.

Argument	Description
<code>-format=0 1</code>	The same as <code>primesim_set_floating_node_format</code> : 0: (default) Voltage value set for floating nodes is applied in DC initialization only. 1: Voltage value set for floating nodes is applied in DC initialization and also throughout the transient simulation.
<code>-outfile=0 1</code>	0: (default) No additional output file. 1: Produces a file <code>*.ic_floating</code> with the <code>.IC</code> statements set with the command.
<code>-power_auto_detect=0 1</code>	Value for PMOS floating gate is set automatically by finding its power domain. This is useful when multiple power domains exist. 0: (default) Use <code>p_gate_value</code> setting. 1: Auto-detects supply value for PMOS and set the value for the PMOS floating gate. When this argument is used , <code>-p_gate</code> argument should not be used.

Examples

The following example sets all floating nodes in the design to 1 volt in DC initialization.

```
.option primesim_set_floating_node=1
```

The following example sets NMOS device floating gates to 0 V, PMOS device floating gates to 1.0 V, and all other floating nodes (excluding bulks) to 0 V.

```
.option primesim_set_floating_node="0 -n_gate=0 -p_gate=1.0"
```

primesim_set_floating_node_format

Description

Determines whether the floating node value specified by `primesim_set_floating_node` is applied in DC initialization only or also throughout the transient simulation.

Syntax

```
.option primesim_set_floating_node_format=[0|1]
```

Arguments

Argument	Description
0	(default) Voltage value set for floating nodes is applied in DC initialization only.
1	Voltage value set for floating nodes is applied in DC initialization and also throughout the transient simulation.

Examples

```
.option primesim_set_floating_node_format=1
```

This example applies the floating node voltage value set by [primesim_set_floating_node](#) to both DC initialization and throughout the transient simulation.

primesim_set_floating_node_type

Description

Determines whether the floating node value specified by [primesim_set_floating_node](#) is applied to all floating nodes or to floating gate nodes only.

Syntax

```
.option primesim_set_floating_node_type=[0|1]
```

Arguments

Argument	Description
0	(default) Voltage value set for floating nodes is applied to all floating nodes.
1	Voltage value set for floating nodes is applied to floating gate nodes only.

Examples

```
.option primesim_set_floating_node_type=1
```

This example applies the floating node voltage value set by [primesim_set_floating_node](#) to floating gate nodes only.

primesim_set_special_char

Description

Appends a backslash (\) before special characters specified during PSF output.

Syntax

```
.option primesim_set_special_char="char1 cha2 ..."
```

Examples

Netlist signal: a\<2\>

Previous PSF output name: a<2>

```
.option primesim_set_special_char="< >"
```

New PSF output name: a\<2\>

primesim_setenv

Description

Specifies an environment variable *name* and sets it to *value*. This option can also be used in the `primesim.cfg` file.

Note:

This option cannot be used to set the license server.

Syntax

```
.option primesim_setenv="name value"
```

Examples

The following example sets the `$MODEL_DIR` environment variable to the specified directory.

```
.option primesim_setenv="$MODEL_DIR /usr/models"
```

primesim_short_rnet

Description

Shorts the resistors in an RC network that includes the specified node name.

Syntax

```
.option primesim_short_rnet="node_name1 node_name2 ..."
```

Examples

In this example, the resistors r1~r5 are shorted.

```
.option primesim_short_rnet="clk"
```

```
vclk clk 0 pu ( 0 1v 2n 0.5n 0.5n 2.0n 5n )
* RC-network of clk
r1 clk clk1 10
c1 clk1 0 10f
r2 clk1 clk2 15
c2 clk2 0 15f
r3 clk1 clk 5 5
c3 clk 0 10f
r4 clk2 clk3 20
c4 clk3 0 20f
```



```
r5 clk3 clk4 1e-5
c5 clk4 0 50f
xinv clk4 clkout inv
```

primesim_simple_em_naming

Description

Some extraction tools mix SPICE hierarchical delimiters, periods (.) and slash (/) characters, as part of the name, which can result in EM net mismatches. Setting this option to 1 forces the output file to store everything using period (.) chaacters to avoid EM analysis being unable to find the signal nets. The default is 0.

Syntax

```
.option primesim_simple_em_naming=[0|1]
```

primesim_single_bin_model_check

Description

Specifies whether to apply a similar model binning check for a model card instance without binning.

Syntax

```
.option primesim_single_bin_model_check=[0|1|2]
```

Arguments

Argument	Description
0	(Default) Does not check the user-defined parameter value against the model card specified ranges.
1	Checks the user-defined parameter value against the width and length range for a single binning model and generates an error if the value is outside the range.
2	Checks the user-defined parameter value against the width and length range for one or more binning models. If the value is outside the range, selects the closest binning available and continues the simulation.

Examples

If w_{max} is defined as 900u in the model card and the user-defined w=901u:

```
.option primesim_single_bin_model_check=1
```

The above example generates an error in the PrimeSim tool:

```
ERROR! w+xw(0.000901) of mn is larger than wmax+xwref(0.0009)
```

primesim_skip_flat_pl_node

Description

SPF files contain subnodes with names ending in ": [number]". Non-SPF files may contain similarly named nodes. In cases with a `probe v(*)` command, FSDB file size can be quite large. This function reduces FSDB file size by reducing the number of nodes probed.

Syntax

```
.option primesim_skip_flat_pl_node=[0|1|2]
```

Arguments

Argument	Description
0	(Default in SPICE modes) Probe all nodes.
1	Skip probing of all SPF nodes.
2	(Default in Pro modes) Skip probing of all SPF subnodes and/or any node with a ": [number]" suffix.

primesim_skip_meas_error

Description

Specifies whether simulation should error out or continue with a warning if a `.MEASURE / MEAS` statement is missing a parameter.

Syntax

```
.option primesim_skip_meas_error=[0|1]
```

Arguments

Argument	Description
0	(default) The PrimeSim tool ERRORS out if a parameter is missing from a <code>.MEAS</code> statement.
1	The PrimeSim tool skips a <code>.MEAS</code> statement missing a parameter and generates a WARNING in the log file.

primesim_skip_unused_param

Description

When set to 1, ignores legacy HSIM-specific parameters to avoid ineffective warning messages and also ignores unused instance parameters. The default value is 0.

Syntax

```
.option primesim_skip_unused_param=[0|1]
```

primesim_skipwarn

Description

Determines the amount and the type of information the PrimeSim tool outputs to the log file.

By default, the PrimeSim tool outputs all information to the log file. Use this option to limit the number and type of warnings the PrimeSim tool reports.

To clear all previously set `primesim_skipwarn` options, set this option with a null string:

```
.option primesim_skipwarn=""
```

Syntax

```
.option primesim_skipwarn="string"[:number]
```

Examples

This option is based on matching an expression. Consider the following example:

```
.option primesim_skipwarn="smaller than PRIMESIM_RESMIN"
```

When you set this option, the PrimeSim tool skips all warnings related to `PRIMESIM_RESMIN`.

You also can use the following command:

```
.option primesim_skipwarn="smaller than PRIMESIM_RESMIN":10
```

When you set this option, the PrimeSim tool limits the number of warnings issued to 10 per log file.

primesim_soa_check_on_dc_init

Description

The PrimeSim Pro tool normally runs Safe Operating Area (SOA) checks throughout the simulation. Enabling this option replaces the `.tran` line in the netlist with `.tran tunit tunit`, then runs SOA checks in Transient at the first two timepoints, and then exits.

Note:

This option is for Model parameter-based SOA checking only. It does not support the `.CHKEXPR` or `.CHKDEVOP` circuit check commands.

This option can be set in the netlist or can be run from the command line as follows:

```
> primesim -soacheck input.sp
```

Syntax

```
.option primesim_soa_check_on_dc_init=[0|1]
```

Arguments

Argument	Description
0	(default) Perform SOA checks throughout the simulation.
1	Perform SOA checks at the first two <i>tunit</i> time steps of a simulation and then exit.

primesim_soa_maxwarns

Description

Sets the maximum number of warnings for `primesim_soa_warn`. The default is 5.

Note:

This option is for Model parameter-based SOA checking only. It does not support the `.CHKEXPR` or `.CHKDEVOP` circuit check commands.

Syntax

```
.option primesim_soa_maxwarns=value
```

primesim_soa_warn

Description

Specifies whether to enable Safe Operating Area (SOA) checking. The PrimeSim tool supports TSMC SOA (version 0.4), which checks for devices safe operating areas (predefined in the model).

Note:

This option is for Model parameter-based SOA checking only. It does not support the `.CHKEXP` or `.CHKDEVOP` circuit check commands.

Syntax

```
.option primesim_soa_warn=[0|1]
```

Arguments

Argument	Description
0	(default) Disable Safe Operating Area (SOA) checking.
1	Enable Safe Operating Area (SOA) checking.

primesim_soa_warn_format

Description

Specifies time window-based SOA reporting.

Note:

This option is for Model parameter-based SOA checking only. It does not support the `.CHKEXP` or `.CHKDEVOP` circuit check commands.

Syntax

```
.option primesim_soa_warn_format=[0|1|2]
```

Arguments

Argument	Description
0	Specifies the time point-based SOA warning format (default).
1	Specifies the brief time window-based SOA warning format.
2	Specifies the verbose time window-based SOA warning format.

Examples

```
primesim_soa_warn_format=0

@0.010000n: Vgs (=1) of mp1 has exceeded Vgs_max (=0.95)
@0.030000n: Vbs (=1) of mp8 has exceeded Vbd_max (=0.6)
@0.030000n: Vbs (=1) of mp7 has exceeded Vbs_max (=0.65)

primesim_soa_warn_format=1

Device #1: mn1
  Terminal: gs (|V(gs)| <= 0.95)
    Violation Window #1: [5.1nS      --> 10nS] (duration: 4.9nS)
    Violation Window #2: [15.2nS     --> 20.1nS] (duration: 4.9nS)
    Violation Window #3: [25.3nS     --> 30.2nS] (duration: 4.9nS)
    Violation Window #4: [35.4nS     --> 39.955nS] (duration:
4.555nS)

primesim_soa_warn_format=2

Device #1: mn1
  Terminal: gs (|V(gs)| <= 0.95)
    Violation Window #1: [5.1nS      --> 10nS] (duration: 4.9nS)
    Start: V(gs)=1 @ t=5.1nS
  Stop: V(gs)=1 @ t=10nS
    Minimum: V(gs)=1 @ t=5.1nS
  Maximum: V(gs)=1 @ t=5.1nS
    Violation Window #2: [15.2nS     --> 20.1nS] (duration: 4.9nS)
    Start: V(gs)=1 @ t=15.2nS
  Stop: V(gs)=1 @ t=20.1nS
    Minimum: V(gs)=1 @ t=15.2nS
  Maximum: V(gs)=1 @ t=15.2nS
    Violation Window #3: [25.3nS     --> 30.2nS] (duration: 4.9nS)
    Start: V(gs)=1 @ t=25.3nS
  Stop: V(gs)=1 @ t=30.2nS
    Minimum: V(gs)=1 @ t=25.3nS
  Maximum: V(gs)=1 @ t=25.3nS
    Violation Window #4: [35.4nS     --> 39.955nS] (duration:
4.555nS)
    Start: V(gs)=1 @ t=35.4nS
  Stop: V(gs)=1 @ t=39.955nS
    Minimum: V(gs)=1 @ t=35.4nS
  Maximum: V(gs)=1 @ t=35.4nS
```

primesim_soa_warn_range

Description

Limits SOA checking to one or more user-specified time ranges. As shown below, SOA checking occurs from t_1 to t_2 , and then from t_3 to t_4 , if defined. The default is 0:1.

Note:

This option is for Model parameter-based SOA checking only. It does not support the .CHKEXPR or .CHKDEVOP circuit check commands.

Syntax

```
.option primesim_soa_warn_range="t1:t2 [t3:t4] ..."
```

primesim_soa_warn_skipwindow

Description

Controls whether to skip the SOA warning report if the violation time window is less than *value*. The default is 0.

Note:

This option is for Model parameter-based SOA checking only. It does not support the `.CHKEXP` or `.CHKDEVOP` circuit check commands.

Note:

This is applied for `primesim_soa_warn_format=1|2`.

Syntax

```
.option primesim_soa_warn_skipwindow=value
```

primesim_soa_warn_to_file

Description

Controls whether to output the SOA warnings in the log file or separate file. By default (0), the PrimeSim tool outputs SOA warnings to log file. When set to 1, it redirects all SOA warnings to a separate `.soa` file (prefix.soa).

Note:

This option is for Model parameter-based SOA checking only. It does not support the `.CHKEXP` or `.CHKDEVOP` circuit check commands.

Syntax

```
.option primesim_soa_warn_to_file=[0|1]
```

Examples

The `.soa` file contains all the warnings and total number of warnings. For example:

```
.option primesim_soa_warn_to_file=1
```

In the previous example all the SOA warnings are output to `prefix.soa`.

primesim_sparse_fcap

Description

Sets the absolute threshold for marking sparse floating capacitors. The default setting varies as follows:

- 1e-18: PrimeSim SPICE tool (with `primesim_precision=1`)
- 1e-16: PrimeSim SPICE tool (with `runlvl=2-6`) and the PrimeSim Pro tool's `spicehd` and `spicemd` modes
- 1e-15: PrimeSim SPICE tool (with `runlvl=1`) and the PrimeSim Pro tool's `spicexd`, `prohd`, `promd`, and `proxd` modes

Syntax

```
.option primesim_sparse_fcap=value
```

primesim_speed

Note:

This option is for the PrimeSim Pro tool only.

Sets the speed level of the simulation: globally, for a specified time period, for a specified subcircuit, or for a specified instance. Values are: 0, 0.5, 1, 2, 3, 4, and 5.

Syntax

```
.option primesim_speed="{t1:value1 [t2:value2] ...}"  
.option primesim_speed="subckt_name1{t1:value1  
[t2:value2]} subckt_name2{t3:value3 [t4:value4]}...":subckt  
.option primesim_speed="instance_name1{t1:value1  
[t2:value2]} instance_name2{t3:value3 [t4:value4]}...":instance
```

Description

The PrimeSim Pro tool has a unique algorithm for controlling tolerances for time step, event signal, and Newton-Raphson procedures. This option sets the combination of the tolerance allowances internally from 0, the slowest and most accurate, to 5, the fastest but least accurate.

Note:

Each `primesim_mode` has a default speed. You can override that default by explicitly setting a value.

The speed level of the simulation is set as follows:

1. If `primesim_speed` is not set, its value is determined by `primesim_mode`.
2. If both `primesim_speed` and a time-dependent speed are set, the final speed value is determined by the time-dependent speed setting.
3. If both a time-dependent, global speed and a subcircuit/instance-based speed are set, the final speed value is determined by the subcircuit/instance-based speed setting.
4. If `t1` is greater than 0, then the default speed is applied from `time=0` to `time=t1`.
5. When multiple `primesim_speed` option statements are set for the same subcircuit or instance, the last option statement is taken.
6. The `primesim_speed` setting applied on an instance has higher priority than the `primesim_speed` setting applied to the subcircuit definition.
7. The lower level hierarchy instance/subcircuit has the higher priority.
8. The `-speed` command-line option (see [Table 2](#)) overrides any `primesim_speed` setting on the netlist.

Examples

```
.option primesim_mode=prohd  
.option primesim_speed="{0:0}"
```

In this example, the default speed (1) in PrimeSim Pro mode `prohd` (see [Table 26](#)) is changed to 0 from `time=0` to the end of transient time.

```
.option primesim_speed="{0:2 300n:1 1u:0.5}"
```

In this example, the PrimeSim Pro tool uses `primesim_speed=2` from `time=0` to before `time=300ns`, then uses `primesim_speed=1` from `time=300ns` to before `time=1us`, and then uses `primesim_speed=0.5` from `time=1us` to the end of transient time.

```
.option primesim_speed="diffamp{0:0.5 90n:0 200n:0.5 0.5u:1}":subckt
```

In this example, for the subcircuit `diffamp`, the PrimeSim Pro tool applies `primesim_speed=0.5` from `time=0` to before `time=90ns`, then `primesim_speed=0` from `time=90ns` to before `time=200ns`, then `primesim_speed=0.5` from `time=200ns` to before `time=0.5us`, and then `primesim_speed=1` from `time=0.5us` to the end of transient time.

```
.option primesim_speed="xbandgap{0:0.5 90n:0 200n:0.5 0.5u:1} xbias{0:0.5 1u:1}":instance
```

In this example, for instance `xbandgap`, the PrimeSim tool applies `primesim_speed=0.5` from `time=0` to before `time=90ns`, then `primesim_speed=0` from `time=90ns` to before `time=200ns`, the `primesim_speed=0.5` from `time=200ns` to before `time=0.5us`, and then `primesim_speed=1` from `time=0.5us` to the end of transient time.

However, for instance `xbias`, the PrimeSim Pro tool applies `primesim_speed=0.5` from `time=0` to before `time=1us`, then `primesim_speed=1` from `time=1us` to the end of transient time.

```
.option primesim_speed="xmy_comp{0:1}":instance
+ primesim_speed="my_comp{0:2 400n:0}":subckt
```

In this example, `xmy_comp` is an instance of subcircuit `my_comp` and the `primesim_speed` setting is determined by the time-value pair from the instance statement because the instance has a higher priority than the subcircuit.

```
.option primesim_speed="xamp{0:1 400n:1}":instance
.option primesim_speed="xamp{0:2 400n:2}":instance
```

In this example, the second `primesim_speed` option statement determines the `primesim_speed` value.

primesim_spf

Description

Specifies the DSPF file that contains Standard Parasitic Format (SPF) data extracted from the layout interconnects. You can rename the subcircuit name with `-renaming`. Also specifies the file that contains multiple SPF options are present.

Note:

- By default, the post-layout device models found in the SPF file are used in back-annotation. To use the different models from the pre-layout netlist, specify `primesim_prelayout_models=1`.
- When multiple `primesim_spf` options are present, each SPF file can have its own annotation policies/ settings. In general, `primesim_spf` options are order dependent. The `primesim_spf` options that come before the first `.option primesim_spf` apply to all SPF files specified with `primesim_spf`. The `primesim_spf` options that come after `primesim_spf` option only apply to that particular `primesim_spf` file and can overwrite the previous setting. See [Chapter 14, Back-Annotation](#) Chapter for examples.

Syntax

```
.option primesim_spf="spf_file [-renaming subckt_original subckt_rename]"
```

Examples

```
.option primesim_spf="aaa.spf"
.option primesim_spf="bbb.spf"
```

In the above example, the DSPF files `aaa.spf` and `bbb.spf` are read in and back-annotated to the design.

To rename a subcircuit:

```
.option primesim_spf="inv.spf -renaming INV_POST INV_PRE"
```

In the above example, the DSPF file `inv.spf` is read in and back-annotated. The subcircuit name `INV_POST` inside the SPF is renamed to `INV_PRE`.

primesim_spf_acc_irem

Description

Performs dynamic EM simulation of a large back-annotated circuit with better performance than standalone IR/EM.

Note:

Option `primesim_spfpost_out=1` is enabled by default when accelerated IR/EM mode is enabled with option `primesim_spf_acc_irem`.

Syntax

```
.option primesim_spf_acc_irem="mode"
```

Arguments

Argument	Description
hd	Most accurate mode.
md	Well-tuned mode to pursue both performance and accuracy.
xd	Fastest mode.

Examples

```
.option primesim_spf_acc_irem="md"
```

In the above example, the PrimeSim tool uses `md` mode, which balances performance and accuracy.

primesim_spf_add_irem_window

Specifies the analysis time range for the IR/EM `.em` file.

Syntax

```
.option primesim_spf_add_irem_window="start_time end_time"
```

Description

If multiple instances of the `primesim_spf_add_irem_window` command are specified, multiple `.em` files are created. However, multiple `primesim_spf_add_irem_window` instances cannot overlap. If overlap occurs, the latter defined `primesim_spf_add_irem_window` is ignored.

Examples

```
.option primesim_spf_add_irem_window="1e-9 2e-9"  
.option primesim_spf_add_irem_window="3e-9 4e-9"
```

In the above example, the PrimeSim tool generates the following `.em` files with time windows string embedded in the file names:

```
output.em.1e-9_2e-9  
output.em.3e-9_4e-9
```

primesim_spf_add_netpin

Description

Allows you to add netpins in a DSPF file.

Note:

Use this command to add a small number of netpins; to add a large number of netpins, use [primesim_spf_add_netpin_file](#).

Syntax

```
.option primesim_spf_add_netpin="net node1 node2 ..."
```

Arguments

Argument	Description
<i>net</i>	DSPF net name
<i>node1 node2</i>	Target pin names in DSPF.
...	Note: The node(s) must be in * S, otherwise the PrimeSim tool ignores this command and outputs either a Warning or Error based on the setting of primesim_exitwarn .

Examples

In this example, the following option is used to add netpin `vdd:10`:

```
.option primesim_spf_add_netpin="vdd vdd:10"
```

As a result, in the following netlist:

```
*|NET VDD 1p
*|S (VDD:10 B 0 16.0925 161.3080) //
```

The netpin `*|S (VDD:10` is changed to `*|P (VDD:10.`

primesim_spf_add_netpin_file

Description

Allows you to add netpins in a DSPF file.

Note:

Use this command to add a large number of netpins; to add a small number of netpins, use [primesim_spf_add_netpin](#).

Syntax

```
.option primesim_spf_add_netpin_file=netpin_file
```

Arguments

Argument	Description
<i>netpin_file</i>	Name of file containing netpin information. File format and contents must follow that of the <i>node</i> argument of primesim_spf_add_netpin .
Note: The node(s) must be in <code>* S</code> , otherwise the PrimeSim tool ignores this command and outputs either a Warning or Error based on the setting of primesim_exitwarn .	

Examples

In this example, the following option is used to add netpins using file `vdd_list`:

```
.option primesim_spf_add_netpin_file ="vdd_list"
```

The file `vdd_list` contains the following subnodes:

```
vdd vdd:1 vdd:2 vdd:3
```

As a result, subnodes `vdd:1`, `vdd:2`, and `vdd:3` are changed to `*|P` (that is, they are changed to netpins).

However, if the file `vdd_list` contains the following netpins:

```
vdd vdd_1 vdd_2 vdd_3
```

Then the PrimeSim tool outputs either a Warning or Error based on the setting of `primesim_exitwarn`.

primesim_spf_allow_rc_parasitic_instance

Description

This option allows you to program the RC net of a Standard Parasitic Format (SPF) file with a custom subckt module in order to estimate the RC effect of that SPF NET.

To achieve this task, create a subckt module and manually edit the SPF file with custom subckt, Xinstance, then set this option to 1. The default is 0.

Other rules:

- Instance name within the RC net section should always start as: `Xrc ...`
- Port number of the Xinstance/subckt cannot be more than 6.
- Parameters of the Xinstance must be all numbers.
- PrimeSim Back-Annotation mode for the nets with those Xinstances must be RC nets, and cannot be FC or others.

Syntax

```
.option primesim_spf_allow_rc_parasitic_instance=[0|1]
```

primesim_spf_bus_char

Description

This option is used when extracted SPEF or SPF file bus notation is different from schematic netlist bus notation.

Syntax

```
.option primesim_spf_bus_char="schematic_delimiters  
[parasitic_delimiters]"
```

Arguments

Argument	Description
<i>schematic_delimiters</i>	Character(s) used for schematic netlist bus notation. In the SPEF flow, this character is replaced with square brackets ([]).
<i>parasitic_delimiters</i>	In the SPF flow, these are the SPF file bus notation characters to be substituted for the schematic netlist bus notation characters specified as <i>schematic_delimiters</i> .

Examples

In this SPEF flow example, the schematic netlist uses bus notation with an underscore (_) character such as A_6, while the SPEF file uses bus notation with square brackets ([]) such as A[6]. After specifying this option:

```
.option primesim_spf_bus_char="_"
```

the SPEF bus notation is mapped from square brackets ([]) to an underscore (_) character.

In this SPF flow example, A<1> in the schematic netlist is mapped to A[1] in the DSPF file:

```
.option primesim_spf_bus_char="<> []"
```

Note:

This option is currently supported in the SPEF and SPF flows only.

primesim_spf_cc_parse

Description

When set to 1, this option parses all coupling capacitor NETs even if one of the NETs is not found in the schematic netlist. The coupling capacitor with a missing NET is converted to a ground capacitor and back-annotated.

The default setting is 0 meaning that a coupling capacitor under a NET is back-annotated only when that NET is found in the schematic netlist.

Syntax

```
.option primesim_spf_cc_parse=[0|1]
```

Examples

```
*|NET A 0.01PF
*|P (A A 0 0.565 9.685)
*|I (XND5/MXM4:GATE XND5/MXM4 GATE I 2e-16 0 0)
C1 XND5/MXM2:GATE B 1e-16
```

In this example, when `primesim_spf_cc_parse=1`, if NET A is not found and NET B exists in the schematic netlist, then C1 is converted to a ground capacitor as follows:

```
C1 0 B 1e-16
```

By default (`primesim_spf_cc_parse=0`), C1 is ignored because NET A does not exist in the schematic netlist.

If NET _A is in the schematic netlist and NET _B is not, then C1 is converted to a ground capacitor regardless this option setting:

```
C1 XND5/MXM2:GATE 0 1e-16
```

primesim_spf_delete_netpin

Description

Allows you to delete netpins in a DSPF file.

Note:

Use this command to delete a small number of netpins; to delete a large number of netpins, use [primesim_spf_delete_netpin_file](#).

Syntax

```
.option primesim_spf_delete_netpin="net node1 node2 ..."
```

Arguments

Argument	Description
<i>net</i>	DSPF net name
<i>node1 node2</i>	Target pin names in DSPF.
...	Note: The node(s) must be in * P, otherwise the PrimeSim tool ignores this command and outputs either a Warning or Error based on the setting of primesim_exitwarn .

Examples

In this example, the following option is used to delete netpin *vdd*:

```
.option primesim_spf_delete_netpin="vdd vdd"
```

As a result, in the following netlist:

```
*|NET VDD 1p
*|P (VDD B 0 16.0925 161.3080) //
```

The netpin *|P (VDD is changed to *|S (VDD.

primesim_spf_delete_netpin_file

Description

Allows you to delete netpins in a DSPF file.

Note:

Use this command to delete a large number of netpins; to delete a smaller number of netpins, use [primesim_spf_delete_netpin](#).

Syntax

```
.option primesim_spf_delete_netpin_file=netpin_file
```

Arguments

Argument	Description
<i>netpin_file</i>	Name of file containing netpin information. File format and contents must follow that of the <i>node</i> argument of primesim_spf_delete_netpin .
	Note: The node(s) must be in * P, otherwise the PrimeSim tool ignores this command and outputs either a Warning or Error based on the setting of primesim_exitwarn .

Examples

In this example, the following option is used to delete netpins using the file `vdd_list`:

```
.option primesim_spf_add_netpin_file ="vdd_list"
```

The file `vdd_list` contains the following netpins:

```
vdd vdd_1 vdd_2 vdd_3
```

As a result, netpins `vdd_1`, `vdd_2`, and `vdd_3` are changed to *|S (that is, they are changed to subnodes).

However, if the file `vdd_list` contains the following subnodes:

```
vdd vdd:1 vdd:2 vdd:3
```

Then the PrimeSim tool outputs either a Warning or Error based on the setting of [primesim_exitwarn](#).

primesim_spf_em_version

Description

Sets the EM database version.

Note:

This option is required only when generating a `.em` file for legacy usage. See also [primesim_spfdb_version](#).

Syntax

```
.option primesim_spf_em_version=version_number
```

Arguments

Argument	Description
1.1	Backward compatibility with Titan IR/EM.
1.3	(default) The current EM database version.

primesim_spf_exclude

Description

Excludes the specified instances from SPF back-annotation.

Syntax

```
.option primesim_spf_exclude="instance_name1 instance_name2 ...  
-subckt subckt_name"
```

Examples

In this example, the SPF back-annotation step is skipped for instance `xtop.xblock_inst1` and all `inv` instances under hierarchy `xtop2.xblock2`.

```
.option primesim_spf_exclude="xtop.xblock_inst1 xtop2.xblock2.* -subckt inv"  
.option primesim_spf=inv.spf
```

primesim_spf_include

Description

Includes only the specified instances from SPF back-annotation and while ignoring the rest of the instances.

Syntax

```
.option primesim_spf_include="instance_name1 instance_name2 ...  
-subckt subckt_name"
```

Examples

In this example, the SPF back-annotation step only annotates for instance `xtop.xblock_inst1` and all sub-instances under hierarchy `xtop2.xblock2`.

```
.option primesim_spf_include="xtop.xblock_inst1 xtop2.xblock2.* -subckt  
inv"  
.option primesim_spf=inv.spf
```

primesim_spf_fingerchars

Description

Translates the special SPF finger delimiter from the specified string to an at-sign (@) character.

See also [primesim_fingerchar](#) and [primesim_expand_instance_mfactor](#).

Syntax

```
.option primesim_spf_fingerchars="string"
```

Examples

```
.option primesim_spf_fingerchars="_NETTRAN_"
```

primesim_spf_finger_res_policy

When set to 1, this option allows for the successful back-annotation of the series-connected resistor fingers in the `primesim_spfinst=1` back-annotation flow (that is, fingered resistors in the SPF file may be a series or parallel-connected). The default value is 0.

Syntax

```
.option primesim_spf_finger_res_policy=[0|1]
```

Description

For the default value, `primesim_spf_finger_res_policy=0`, finger resistors must be in parallel to the base resistor in the schematic. In other words, they are added between the same nets as the base resistor.

When `primesim_spf_finger_res_policy=1`, additional searching and processing is performed so that finger resistors are allowed to be a series or a mix of parallel and sequential order.

primesim_spf_histogram

Description

Specifies whether to print an RC Histogram for each DSPF file during back-annotation.

Syntax

```
.option primesim_spf_histogram=[0|1]
```

Arguments

Argument	Description
0	(default) No RC Histogram for each DSPF file.
1	Prints an RC Histogram for each DSPF file during back-annotation.

primesim_spf_layout_only_device_models

Specifies the device model names in the DSPF file to be added to the back-annotation simulation. The model name may point to a device model or a subcircuit model definition. This option supports wildcards.

Syntax

```
.option primesim_spf_layout_only_device_models=  
"device_model_name1 device_model_name2 ..."
```

Description

The purpose of this option is to add devices, which are in the DSPF file but not in the schematic netlist, into the back-annotation simulation. When you use this option, devices in the DSPF file with the given model names are included in the simulation. This option affects devices with the model names that are in the DSPF file, but are not in the schematic netlist.

Examples

```
.option primesim_spf_layout_only_device_models="MOSmodelA"  
.option primesim_spf_layout_only_device_models="DIODEmodelB"
```

Adds devices from the DSPF file, with model names `MOSmodelA` and `DIODEmodelB`, into the back-annotation simulation.

```
.option primesim_spf_layout_only_device_models="*model*"
```

Adds devices from the DSPF file, with model names containing the string `model`, such as `MOSmodelA`, `MOSmodelB`, `DIODEmodelA`, `DIODEmodelB`, and so on, into the back-annotation simulation.

primesim_spf_layout_only_net

Description

Use this option to specify one or more NETs which exist only in layout so that you can apply RC annotation options to them.

By default, the PrimeSim tool does not perform RC annotation on a NET if it does not exist in the pre-layout netlist, so if you have a NET which exists only in layout, the PrimeSim tool does not parse the NET name and cannot apply `primesim_spffcnnet` or other similar options to it.

Syntax

```
.option primesim_spf_layout_only_net="NET_name"
```

Examples

In this example, the PrimeSim tool annotates parasitic coupling and ground capacitors on `LN_*` NETs in the SPF file.

```
.option primesim_spf_layout_only_net="LN_*"
.option primesim_spffcnnet="LN_*
```

primesim_spf_map_subckt_port

Description

Use this option to set mapping between the port names of the schematic file and the pin names of the back-annotation file.

Syntax

```
.option
primesim_spf_map_subckt_port="subckt_name port_name1 ... port_nameN ...
pin_name1 ... pin_nameN"
```

Examples

In this example, the schematic file has `.subckt nch n1 n2 n3 n4`, while the SPF instantiation is `"XM1 XM1:d XM1:g XM1:s XM1:b nch"`.

```
.option primesim_spf_map_subckt_port="nch n1 n2 n3 n4 d g s b"
```

This maps the port names of subcircuit `nch` from `n1`, `n2`, `n3`, `n4` in the schematic to `d`, `g`, `s`, and `b`, respectively, in the SPF post-layout.

In some SPF files which allow layout flexibility, there is a need to match a symmetric custom block where a pre-layout port can be mapped to any of the symmetric ports. This option can also be used for this purpose to map one port (in the schematic) to multiple ports (in post-layout).

For instance, subcircuit `custom_block` port `A` in pre-layout might be mapped to port `A1` or `A2` in post-layout. This example shows how you can swap two ports using the following options together:

```
.option primesim_spf_map_subckt_port="custom_block A B C A1 A2 C"  
.option primesim_spf_map_subckt_port="custom_block A B C A2 A1 C"
```

primesim_spf_matcheffort

Controls SPF matching for nets and devices.

Syntax

```
.option primesim_spf_matcheffort=[0|1]
```

Description

This option is for SPF back-annotation. Sometimes the extraction tool uses the hierarchical divider for both the flattened portion of the netlist, as well as the hierarchical portion of the netlist.

When this option is set to `1`, the PrimeSim tool performs more rigorous naming matching for the SPF net and devices in order to achieve a higher percentage of matched net/devices.

This option can also be used when back-annotating macro model devices that have a leading `x` in SPF to those without a leading `x` in the pre-layout netlist.

The default value is `0`, and setting it to `1` results in runtime penalties.

primesim_spf_port_name_check

Description

Specifies whether to check for multiple instances of port names connecting to another NET.

Syntax

```
.option primesim_spf_port_name_check=[0|1]
```

Arguments

Argument	Description
0	(default) Do not check for multi-port names.
1	Compare multi-port names with all NET names; if matched, then error out.

Examples

For example, suppose an SPF file contains a B_1 NET name with a B_1 Port name, but there is also a B_1 Port name in NET B:

```
*|NET B_1 0.00287703PF
*|I (XND5/MXM0:GATE XND5/MXM0 GATE I 1.37732e-15 1.425 17.989)
*|P (B_1 B 0 1.705 10.825)
*|I (XND5/MXM4:GATE XND5/MXM4 GATE I 2.6e-16 0.955 5.33)
Cg1 XND5/MXM2:GATE GND 7.27947e-16
R1 XND5/MXM2:GATE B 214.925
R2 XND5/MXM2:GATE XND5/MXM4:GATE 1050.35
R3 B XND5/MXM4:GATE 205.296
R4 B B_1 10
```

NET B has two ports, including a B_1 Port name:

```
*|NET B 0.00269325PF
*|I (XND5/MXM2:GATE XND5/MXM2 GATE I 1.37732e-15 0.955 17.989)
*|P (B B 0 0.565 9.685)
*|P (B_1 B 0 0.0 9.685)
```

In this case, the PrimeSim tool keeps the original node name and then the B_1 Port node of NET B_1 and the B_1 Port node of NET B are shorted automatically. If this happens and `primesim_spf_port_name_check=1`, then the PrimeSim tool exits with an error message.

primesim_spf_removetoprc

Description

Remove the top-level parasitics in the `primesim_spfinst=2` back-annotation flow. Setting this option to 1 removes the RC. The default is 0 (disabled).

Syntax

```
.option primesim_spf_removetoprc=[0|1]
```

primesim_spf_sch_check_mode

Turn this option on (1) in back-annotation simulations when the schematic netlist changes faster than layout and DSPF files can be generated. The default is 0.

Syntax

```
.option primesim_spf_sch_check_mode=[0|1]
```

Description

- When `.option primesim_spf_sch_check_mode=0`: It is a default value.
- When `.option primesim_spf_sch_check_mode=1`: Prints an information for the instances or transistors that are in the ideal netlist but missing in the SPF.

This option is valid only in back-annotation simulations. It is useful for when the schematic netlist changes faster than the layout and DSPF files can be generated. It is highly recommended that you start with a matching (clean) netlist and DSPF files. Subsequent back-annotation simulations of the original DSPF file and the modified schematic netlist file generate warnings in the log file.

Transistors are the only devices checked. Resistors, capacitors, and diodes are not checked. For example, if transistors are added to the schematic, those are printed in the log file.

Examples

```
**** Information on mismatched elements (exist only in schematic, not dspf)
```

```
SUBCKT ELEMENT LOCATION
```

```
buf xinv3 (in.sp:26)
```

```
buf mdummy (in.sp:27)
```

```
**** total : 2 devices mismatched
```

primesim_spf_sensitive

Description

Supports case sensitivity for the 3rd-party netlist back-annotation flow. The default is 0. If the SPF file is case-sensitive, the option must be set to 1.

Syntax

```
.option primesim_spf_sensitive=[0|1]
```

primesim_spf_skip_res_for_ivsrc

Description

When this option is set to 1, any SPF nets for RC annotation are tested to see whether they are connected to any current sources or voltage sources. If found to be connected to any ISRC or VSRC, all of the resistors for those nets are shorted automatically. The default is 0.

This option supports the following order dependency:

- If defined before any `primesim_spf` option, then it applies to all SPF files as a global option.
- If defined after a `primesim_spf` option, then it applies only to that SPF file like a local option. This localized option is a higher priority than the global option.

Note:

Sources defined by the vector file are not affected by this option; the PrimeSim tool keeps those resistors in RC annotation.

Syntax

```
.option primesim_spf_skip_res_for_ivsrc=[0|1]
```

primesim_spf_spice_names

Specifies whether the instance names in the DSPF file include the SPICE type prefix.

Syntax

```
.option primesim_spf_spice_names=[0|1]
```

Arguments

Argument	Description
0	Names have no SPICE prefix.
1	(default) Names have SPICE prefix.

Description

It is common for hierarchical DSPF files not to include the SPICE prefix. For example, a net in the DSPF file might be `u0/u1/net10` instead of the standard SPICE format with prefix of `x` as in `xu0/xu1/net10`. This option restores the SPICE-like names to back-annotate the pre-layout netlist.

primesim_spf_user_defined_gnd_list

Description

Specifies the node list that should be treated as ideal ground in SPF back-annotation (see also, [primesim_spfkeepnodecap](#)).

Syntax

```
.option primesim_spf_user_defined_gnd_list="node_list"
```

primesim_spfallowerror

Description

When this option is set to 1, you can continue the simulation even though there is a parsing error of SPF. The default value is 0.

Syntax

```
.option primesim_spfallowerror=[0|1]
```

primesim_spfcnet

Description

Specifies the net names to be annotated with lumped capacitance only. The names must be quoted if wildcard characters or multiple net names are used. For other (unspecified) nets, by default, RC trees is annotated.

Syntax

```
.option primesim_spfcnet="net_name1 net_name2 ..."
```

Examples

```
.option primesim_spfcnet="a.* b.*"  
.option primesim_spfcnet=" "
```

In the first example, all the nets that match `a.*` or `b.*` are back-annotated with lumped capacitance. All other nets use RC back-annotation.

In the second example, none of the nets are annotated even though the DSPF is read.

primesim_spfdb_version

Description

Sets the IR drop database version.

Note:

This option is required only when generating an `irdrop.db` file for legacy usage. See also [primesim_spf_em_version](#).

Syntax

```
.option primesim_spfdb_version=version_number
```

Arguments

Argument	Description
1.2	Backward compatibility with Titan IR/EM.
1.4	(default) The current IR drop database version.

primesim_spfeqr, primesim_spf2eqr, primesim_spfeqrfile, primesim_spfeqr_only, primesim_spf2eqr_select

The equivalent resistance (EQR) of a DSPF file is calculated by extracting the equivalent resistors from the port `(*|P)` to pin `(*|I)` in the DSPF file.

Syntax

```
.option primesim_spfeqr="net1 [net2 ...] [-no_pin_merging]
+ [-add_subnode] [-src_nodename_file='filename']"
.option primesim_spf2eqr="net1 [net2 ...]"
.option primesim_spfeqrfile="output_file_name"
.option primesim_spfeqr_only=[0|1]
.option primesim_spf2eqr_select=value
```

Description

- Option `primesim_spfeqr` activates the EQR analysis and specifies the net names to process. If there are multiple ports `(*|P)` in the SPF file, you can use the `-no_pin_merging` option so that the ports are not merged.

The `-add_subnode` argument enables additional EQR calculation from port `(*|P)` to subnode.

The `-src_node_name` argument specifies a file that defines one or more SRC node names. If multiple SRC nodes are defined, the option writes node names line by line, merges them into one node (virtually all connected), and then calculates EQR from SRC to DEST. DEST is for each instance pin `(*|I)`, as well as subnode if the `-add_subnode` argument is used.

Note:

If multiple SRC node names are used, then the `-no_pin_merging` argument is ignored.

- Option `primesim_spf2eqr` extracts the EQR from port-to-port, in addition to port-to-pin.
- Option `primesim_spfeqrfile` specifies an optional output file name.

To run the EQR analysis without running back-annotation or full simulation, set the option `primesim_spfeqr_only` to 1. The default is 0.

If you set `primesim_spfeqr_only=1`, then you can use multiple CPUs and the EQR calculation can be performed using each. The usage is the same as with normal parallel run arguments, but the input file should have the proper options set:

```
$> primesim -np 4 input.sp
```

- Option `primesim_spf2eqr_select` extracts the EQR from specified instance pin combinations, where *value* is a number (or sum of numbers) from the following table:

Value	Instance Pins
1	GATE–GATE
2	DRN–GATE
4	DRN–DRN
8	SRC–GATE
16	SRC–DRN
32	SRC–SRC

For example, If you want calculate the EQR for DRN–GATE (*value*=2) and SRC–GATE (*value*=8), then set the option as 2+8=10:

```
.option primesim_spf2eqr_select=10
```

Note:

The default SPF EQR resistors between port and instance pins are not affected by the `primesim_spf2eqr_select` option.

To use the SPRES solver, see [primesim_eqr2spres](#), [primesim_spresout](#).

Examples

```
.option primesim_spfeqr="vdd vss"  
.option primesim_spfeqrfile=test.eqr
```

In the preceding example, the PrimeSim tool extracts equivalent resistor values for `vdd` and `vss` and outputs the result to the `test.eqr` file.

primesim_spffcmin

Sets the minimum floating (cross coupling) capacitor value allowed for back-annotation. All floating capacitors with a smaller value are split into two grounded capacitors for back-annotation.

Syntax

```
.option primesim_spffcmin=value
```

Description

This option can be applied on a net by net basis, and applies only to the DSPF file in the preceding `primesim_spf` option. The default value is 0.

Examples

```
.option primesim_spffcmin=2fF
```

If RC reduction is also requested, floating capacitors smaller than this value both before and after the reduction are split into two grounded capacitors.

```
.option primesim_spfrcnet=netB  
.option primesim_spffcnet=netA  
.option primesim_spffcmin=1f
```

In this example, the `primesim_spffcmin` option is applied only to `netA` and `netB`.

primesim_spffcnet

Keeps the floating (coupling) capacitance for C-only back-annotation.

Syntax

```
.option primesim_spffcnet="net_names"
```

Description

The difference between this option and the `primesim_spfcnet` option is that the latter keeps only the grounded capacitance for annotation. The `primesim_spffcnet` option keeps both grounded and coupling capacitances.

Examples

```
.option primesim_spffcnnet="A B"
```

Nets A and B are back-annotated with grounded capacitances as well as coupling capacitances.

primesim_spfhdiv

Description

Sets the divider character for hierarchical names in DSPF files. The default value is a slash (/) character.

Syntax

```
.option primesim_spfhdiv="character"
```

Examples

```
.option primesim_spf="file1.spf"  
.option primesim_spfhdiv="." $ handle names like xiv1.m32
```

primesim_spfinst

Controls whether the instance section of the DSPF file is used for annotation.

Syntax

```
.option primesim_spfinst=[0|1|2]
```

Description

Set this option to 0 if you want the PrimeSim Pro tool to ignore the instance section of the DSPF file.

By default, this option is set to 1 and the devices in the instance section are annotated. New devices are added to the netlist and any device parameters given in the instance section are used. This option causes the instance section to be annotated as if it were in a DPF file:

```
.option primesim_spfinst=1
```

If this option is set to 2, a new parsing flow is applied. This flow can only be applied when the SPF is self-contained with complete netlist and connectivity information. The new flow uses the SPF netlist to replace the schematic netlist (as opposed to annotating the SPF netlist to the existing one). All the device's instance and subcircuit instances are removed, except for the top-level parasitic and stimulus sources, and replaced with the SPF instance section. The PrimeSim tool back-annotates the SPF net section to run the simulation.

To enable the new flow, set the `primesim_spfirst` as shown below.

Examples

```
.option primesim_spfirst=2
```

By default (1), this new flow feature is disabled.

To remove the top-level parasitic, see option [primesim_spf_removetoprc](#).

primesim_spfkeepnodecap

Description

Controls whether to keep or remove the pre-layout ground capacitance of back-annotated nets (see also, [primesim_spf_user_defined_gnd_list](#)).

Syntax

```
.option primesim_spfkeepnodecap=[0|1]
```

Arguments

Argument	Description
0	Remove the pre-layout ground capacitance of back-annotated nets.
1	(default) Keep the pre-layout ground capacitance of back-annotated nets.

primesim_spfmergeport

Allows you to merge ports in a DSPF net.

Syntax

```
.option primesim_spfmergeport=[0|1]
```

Arguments

Argument	Description
0	(default) No ports merged.
1	All ports in a DSPF net are merged.

Description

In some cases, networks in the DSPF file have more than one port for a net. For example, the VDD net in the ideal netlist, which is a single port, may be annotated as a network with ports VDD, VDD_1 and VDD_2, and so on, in the physical data of DSPF. It is sometimes convenient to merge these into a single port so that all ports are driven by the same voltage source for simulation.

primesim_spfmodule

Description

Specifies the module for SPF back-annotation. The `primesim_spf` option should be declared in front of this command.

Syntax

```
.option primesim_spfmodule=module_name
```

primesim_spfnet_wildcard_mode

Description

Specifies whether the wildcard character (*) when used in back-annotation options refers to all levels of hierarchy or just one level of hierarchy.

Syntax

```
.option primesim_spfnet_wildcard_mode=[0|1]
```

Arguments

Argument	Description
0	(default) Global expression style (that is, wildcard characters refer to all levels of hierarchy).
1	Wildcard expansion is only allowed for one level of hierarchy.

Examples

In the following example, the wildcard character (*) is applied to only one level of hierarchy.

```
.option primesim_spfnet_wildcard_mode=1
.option primesim_spffcnnet="x1.^*.x2.net1"
```

primesim_spfnonet

Description

Specifies the nets that are not going to be back-annotated by DSPF data.

Syntax

```
.option primesim_spfnonet="net_names"
```

Examples

```
.option primesim_spfnonet="vss1 vss2"
```

In this example, the ground nets `vss1` and `vss2` are not annotated with DSPF data.

primesim_spfpost

Sets the node names for post analysis of DSPF back-annotated simulation.

Syntax

```
.option primesim_spfpost="node_name1 node_name2 ..."
```

Description

The PrimeSim tool supports wildcard usage for `primesim_spfpost`. This can be used to find the worst resistor and net (in EM analysis, for example).

The purpose of this option is to record the simulation results along with physical information within the DSPF data for non-ideal power analysis for dynamic voltage drop and EM violations.

When the option is used, the DSPF information for the given nodes is saved into the file `XXX_irdrop.db`, and if the nodes are probed, the internal DSPF nodes are also probed for post analysis.

Examples

```
.option primesim_spfpost="AVDD AVSS"
```

primesim_spfpost_out_only

Description

Probes the internal nodes of power/ground during EM simulation. By default, the PrimeSim Pro tool did not save those node's waveforms into FSDB. This option lets the user save and see those waveforms. The default value is 1.

Syntax

```
.option primesim_spfpost_out_only=[0|1]
```

primesim_spfprb

Sets the probe mode for DSPF nodes.

Syntax

```
.option primesim_spfprb="[0|0.5|1|2][net_name:[0|0.5|1|2]]"
```

Arguments

Argument	Description
0	(default) None of the new nodes are probed.
0.5	Only GATE pin nodes are probed.
1	Only the new instance pin nodes are probed. These nodes form the boundary of the annotated network.
2	All new nodes, including internal nodes, are probed.

Description

After layout interconnect is extracted in DSPF format, new subnodes are usually generated to connect all resistors and capacitors, and the primary nodes defined in the pre-layout netlist. This option specifies whether these newly created nodes within the DSPF are probed.

Examples

```
.option primesim_spfprb="0 NET1:1"
```

This example probes SPF `NET1`, and the instance pins of `NET1`.

primesim_spfprb_finger

Description

Specifies whether to probe fingered device pins based on the `primesim_spfprb` option setting.

Syntax

```
.option primesim_spfprb_finger=[0|1]
```

Arguments

Argument	Description
0	Do not probe fingered device pins based on the <code>primesim_spfprb</code> option setting.
1	(default) Probe all fingered device pins based on the <code>primesim_spfprb</code> option setting.

primesim_spfprb_mode

Description

Ordinarily, an SPF-annotated node name is flattened as `"v(netname#flattened_nodename)"`. When this option is set to 1, the node is put into the proper hierarchy. The default value is 0.

Syntax

```
.option primesim_spfprb_mode=[0|1]
```

Examples

SPF net name is `net074` and SPF file is annotated to instance `xi0`, and SPF node name is `xi0/xi0/xgil/mna:g`.

```
.option primesim_spfprb_mode=0
```

Output node name is `v(net074#xi0/xi0/xgil/mna:g)` under hierarchy `xi0`.

```
.option primesim_spfprb_mode=1
```

Output node name is `v(mna:g)` under hierarchy `xi0/xi0/xi0/xgil`.

primesim_spfprb_sync

Description

Specifies that only the probed NET's device's pin nodes or internal subnodes are probed. If the NET is not probed, those nodes are also not probed.

Syntax

```
.option primesim_spfprb_sync=[0|1]
```

Arguments

Argument	Description
0	(default) Probe the device's pin nodes or internal subnode inside the SPF file based on the primesim_spfprb option setting.
1	Probe those additional nodes only when the back-annotated NET name is probed.

primesim_spfprefix

Lists the prefixes that should be removed from device names.

Syntax

```
.option primesim_spfprefix="prefix1 prefix2 ..."
```

Description

The `primesim_spfprefix` option lists the prefixes that should be removed from device names. Extraction tools sometimes add an additional type prefix on extracted devices in the DSPF file.

When `primesim_spfinst=1` (default), the PrimeSim Pro tool annotates devices from the instance section of a DSPF file much like it annotates devices in a DSPF file. The `primesim_spfprefix` option is the equivalent of the `primesim_dpfprefix` option for these devices.

Examples

```
.option primesim_dpfprefix="m r c q"
```

`xinv1.mn2` might be listed in the DSPF file as `mxinv1/mn2`. The option would remove the leading `m` in `mxinv1/mn2`.

primesim_spfrc_ctype

Description

Specifies whether and how to filter grounded capacitors (`gcap`) and/or coupling capacitors (`fcap`) for option [primesim_spfrcnet](#).

Syntax

```
.option primesim_spfrc_ctype=[0|1|2|3]
```

Arguments

Argument	Description
0	(default) No filter; <code>primesim_spfrcnet</code> annotates resistors, gcap, and fcap.
1	Filters gcap; <code>primesim_spfrcnet</code> annotates resistors and fcap.
2	Filters fcap; <code>primesim_spfrcnet</code> annotates resistors and gcap.
3	Filters gcap and fcap; <code>primesim_spfrcnet</code> annotates resistors only.

primesim_spfrcnet

Specifies the net names that are annotated with RC trees. The names must be quoted if wildcard characters or multiple net names are used. The default value is "*", RC tree back-annotation for all nets.

Syntax

```
.option primesim_spfrcnet="net_name1 net_name2 ..."  
.option primesim_spfrcnet="net_name1 [-lvl layer_name1 layer_name2 ...]  
[-exclude layer_name3] "
```

Description

This option is usually used in conjunction with the `primesim_spfcnet` option. If only `primesim_spfrcnet` is specified for specific nets, the PrimeSim Pro tool back-annotates these nets with RC trees and leave all the other nets untouched. So, normally, you should not use this option without `primesim_spfcnet` to cover the other nets unless it is intended for cases such as critical path timing analysis.

If `primesim_spfrcnet` and `primesim_spfcnet` are given the same nets, higher priority is given to `primesim_spfrcnet`. If `primesim_spfcnet` is specified explicitly for a net, lumped capacitance annotation is applied instead of the default.

The default value is "*".

You can use the `-lvl` argument to selectively annotate resistors from the specified layer(s) for any given net(s). The resistors from layers not annotated are shorted. Wildcards can be used for the `net_name` and `layer_name`. To use this feature, the DSPF should contain the `*LAYER_MAP` section and the `$lvl` information for the resistors.

The `layer_name` is based on the Interconnect Technology File (ITF) layer name. The layer names and the mapping can be found from the SPF file as shown in the following example:

```
Example SPF section:  
*LAYER_MAP
```

```
*0 SUBSTRATE
*1 M2
*2 M1

*|NET SUM0 0.014152PF
...
R8 SUM0:5 SUM0:6 0.0310001 $l=1.5 $w=3 $lvl=1 $llx=-461 $lly=26.5
  $urx=-458 $ury=28 $dir=1
...
```

You can use the `-exclude` argument to exclude certain layers that may have been set with wildcards in the `-lvl` argument.

Note:

See [primesim_spfrc_ctype](#) for additional filtering options.

Examples

Example 1

```
.option primesim_spfrcnet="c.* d.e.*"
```

In this example, all nets that match `c.*` or `d.e.*` are RC tree back-annotated. All other nets are not annotated, which is the potential danger of using this option alone. A solution is to add `primesim_spfcnet` for all other nets:

```
.option primesim_spfrcnet="c.* d.e.*" primesim_spfcnet="*"
```

This option applies lumped capacitance back-annotation to all nets not specified by `primesim_spfrcnet`.

Example 2

```
.option primesim_spfrcnet=a primesim_spfcnet=b
```

In this example, only net `a` and `b` are annotated with RC tree and lumped capacitance, respectively. All other nets are not annotated.

Example 3

```
.option primesim_spfrcnet="aa bb" primesim_spfcnet="bb cc"
```

In this example, the nets `aa` and `bb` are annotated in RC-tree, and net `cc` in lumped capacitance. All other nets are not annotated.

Example 4

```
primesim_spfrcnet="NET* -lvl M1 M2 M3"
primesim_spfrcnet="NET* -lvl * -exclude VIA"
primesim_spfrcnet="NET* -lvl M* -exclude M2"
```

In this example, the first command annotates resistors from layers `M1`, `M2`, and `M3`. The other layers are shorted.

The second command annotates resistors from all layers except for the `VIA` layer. The `VIA` layer is shorted.

The third command annotates resistors from all `M*` layers except for the `M2` layer. The `M2` layer is shorted. Resistors from any other layer are also shorted.

primesim_spfrepgate

Description

Picks the first GATE pin defined at `(*I)/(*|NET)` from the DSPF file, only if the GATE pin exists under `(*|NET)` from the DSPF file. This allows you to get a more accurate result when probing on timing by picking the first GATE pin from DSPF.

Note:

This option is only for the first GATE instance pin selection.

This option can be used together with `primesim_spfreplast` and has precedence over it.

This option has no impact for NETs with a `*|P` statement.

Syntax

```
.option primesim_spfrepgate=[0|1]
```

Arguments

Argument	Description
0	(default) The rule follows the <code>primesim_replast</code> option.
1	Chooses the first GATE pin as the representative node of <code>(* I)</code> . If the instance pin GATE does not exist on <code>(* I)</code> , this option is ignored and the rule follows the <code>primesim_replast</code> option.

primesim_spfreplast

Controls how the representative node is chosen when annotating DSPF.

Syntax

```
.option primesim_spfreplast=[0|1|2|3]
```

Arguments

Argument	Description
0	(default) Choose the first instance pin as the representative node.
1	Choose the last instance pin as the representative node.
2	Choose the first instance pin as the representative node after sorting of (* I).
3	Choose the last instance pin as the representative node after sorting of (* I).

Description

When DSPF is back-annotated onto a design, what was one node in the original design is replaced with an RC network. When the annotation is done, the PrimeSim Pro tool chooses one node in the RC network as the representative node. This node keeps the original node name, so any measurements involving the original node are actually made on the representative node.

If port (*|P) or a node of the same name is defined, and the port definition is the represented node, and this option is not applied. However, if the port is floating, then this option's setting applies.

For example, in the SPF file, there is NET21 as follows:

```
*|NET NET21 0.00419788PF
*|I (XND5/MXM0:DRN XND5/MXM0 DRN B 0 1.495 17.989)
*|I (XIV0/MXM1:GATE XIV0/MXM1 GATE I 7e-16 2.365 15.57)
*|I (XND5/MXM1:DRN XND5/MXM1 DRN B 0 1.36 5.33)
*|I (XND5/MXM4:DRN XND5/MXM4 DRN B 0 1.02 5.33)
*|I (XIV0/MXM2:GATE XIV0/MXM2 GATE I 2.6e-16 2.365 5.33)
```

By default, the PrimeSim Pro tool chooses the first instance pin XND5/MXM0:DRN as representative node.

When the `primesim_spfreplast` option is set to 1, then the last instance pin XIV0/MXM2:GATE is chosen.

When the `primesim_spfreplast` option is set to 2, then all names of *|I is sorted first, and then the first instance pin XND5/MXM4:DRN is chosen.

Similarly, when the `primesim_spfreplast` option is set to 3, then the last instance pin XIV0/MXM1:GATE is chosen after sorting.

primesim_spfrcnet

Description

Specifies the net names that are annotated with parasitic resistors and ground capacitors. All coupling capacitors under this NET section are changed to ground capacitors. Any coupling capacitors under the other NET sections are also changed to ground capacitors.

Syntax

```
.option primesim_spfrcnet="net_name1 net_name2 ..."
```

Examples

In the following example, `vdd_ext` and `vdd_int` are annotated with parasitic resistors and ground capacitors.

```
.option primesim_spfrcnet="vdd_ext vdd_int"
```

primesim_spfmax

Sets the maximum resistor value allowed for back-annotation. All resistors with a greater value are ignored as an open circuit.

Syntax

```
.option primesim_spfmax=value
```

Description

This option can be applied on a net by net basis, and applies only to the DSPF file in the preceding `primesim_spf` option.

If RC reduction is also requested, resistors larger than this value both before and after the reduction are eliminated.

The default value is `1.0e8 ohm`.

Examples

```
.option primesim_spf="a.spf"  
.option primesim_spfmax="1e6 clk:10000 vdd:1000"  
.option primesim_spf="b.spf"  
.option primesim_spfmax=100k
```

In the first example, the maximum resistor value is 10000 for `clk`, 1000 for `vdd`, and 1e6 for all other nets in the `a.spf` file.

In the second example, the maximum resistor value is 100,000 for all nets in the `b.spf` file.

primesim_spfrmin

Sets the minimum resistor value allowed for back-annotation. All resistors with a smaller value are treated as short circuits.

Syntax

```
.option primesim_spfrmin=value
```

Description

This option can be applied on a net by net basis, and applies only to the DSPF file given by the preceding `primesim_spf` option.

If RC reduction is also requested, resistors smaller than this value both before and after reduction are eliminated.

The default value is 0.01 ohm.

Examples

```
.option primesim_spf="a.spf"  
.option primesim_spfrmin="1e-5  clk:0.001 vdd:0.0001"  
.option primesim_spf="b.spf"  
.option primesim_spfrmin=1e-4
```

In the first example, the minimum resistor value is 0.001 for `clk`, 0.0001 for `vdd`, and 1e-5 for all other nets in the `a.spf` file.

In the second example, the minimum resistor value is 1e-4 for all the nets in the `b.spf` file.

primesim_spfrnet

Description

Specifies the net names to be annotated with parasitic resistor only. All ground capacitors under this NET section are removed and all coupling capacitors under this NET section are changed to ground capacitors which are added to the other side NET. Any coupling capacitors under the other NET sections are also changed to ground capacitors.

Syntax

```
.option primesim_spfrnet="net_name1 net_name2 ..."
```

Examples

In the following example, `vdd_int` is annotated with parasitic resistors only.

```
.option primesim_spfrnet="vdd_int"
```

primesim_spfrptrmax

Description

Sets the warning threshold number (per net) for SPF resistors. The default value is 2000.

Syntax

```
.option primesim_spfrptrmax=value
```

Examples

```
.option primesim_spfrptrmax=300
```

This generates the following warnings in the log file:

```
WARNING! Net detected with unexpectedly big number of elements: Res#=323  
FCap#=0 GCap#=1062 Pin#=262 <=== xiblock[7].xisubblock_left.bl3[9]
```

primesim_spfscale

Sets the scale for devices in the DSPF file.

Syntax

```
.option primesim_spfscale=[0|1]
```

Description

The default behavior (0) is to use the same scale as in the netlist. This option only needs to be used when the DSPF scale differs from the netlist scale.

This option supports macro models in the SPF file.

When `primesim_spfinst=1` (default), the PrimeSim Pro tool annotates devices from the instance section of a DSPF file much like it annotates devices in a DPF file. The `primesim_spfscale` option is the equivalent of the `primesim_dpfscale` option for these devices.

Examples

```
.option primesim_spf="file1.spf"  
.option primesim_spfscale=1 $ DSPF devices should not be scaled.
```

primesim_spfspef

Description

By default (0), `primesim_spf` input is assumed to be DSPF format. When you set this option to 1, the `primesim_spf` file is assumed to be SPEF format instead of DSPF.

Syntax

```
.option primesim_spfspef=[0|1]
```

primesim_spfsplitnet

Controls how to split nets in DSPF files.

Syntax

```
.option primesim_spfsplitnet=[0|1]
```

Description

The PrimeSim Pro tool includes split nets in DSPF files. A single net in the schematic netlist can be split into multiple nets, or *fingers*, after layout is complete. These split nets are extracted into DSPF format in the form of nets, for example, A, A@1, and A@2, and so on, where net A is the original net in the schematic netlist and A@*number* is the newly-added net bound for layout.

The PrimeSim Pro tool treats any net in the format *net@number* as a split net. You can control this functionality with the `primesim_spfsplitnet` option.

- When `primesim_spfsplitnet` is set to 1, the default value, all nets for that split net are back-annotated.
 - When `primesim_spfsplitnet` is set to 0, only the last net (for example, A@2 in the previous example) is back-annotated.
-

primesim_spfsuffix

Defines the suffix in an SPF file.

Syntax

```
.option primesim_spfsuffix="string"
```

Description

In SPF files, when a device has been broken up into fingers, the extra devices and nodes have a suffix added to them. By default, the PrimeSim Pro tool assumes the suffix is @<*number*>. But the suffix can be other characters too. For example, consider the following excerpt from an SPF file:

```
M102 ...  
M102#1 ...  
M102#2 ...  
M102#3 ...
```

The PrimeSim Pro tool includes this option to define the suffix in a DSPF file:

```
.option primesim_spfsuffix="#"
```

The PrimeSim Pro tool can handle the device with a hash (#) character as a suffix in a DSPF file.

primesim_spfwarn

Description

Controls warning message output when performing and debugging back-annotation.

Syntax

```
.option primesim_spfwarn=[0|1]
```

Arguments

Argument	Description
0	(default) Prints a warning message for each unmatched net/device.
1	Prints a simplified warning message that skips individual unmatched hierarchy names in the net/device. If the full hierarchy name is matched and the last net/device is not matched, then it prints the warning message.

primesim_spred

Determines the level of RC reduction to perform on the circuit.

Syntax

```
.option primesim_spred="[0|1|2|3|4|5] [netname1:[0|1|2|3|4|5] netname2:[0|1|2|3|4|5] ...]"
```

Arguments

Argument	Description
0	Specifies no reduction.
1	Specifies conservative reduction, which can be used on most circuits with minimal accuracy loss.
2	Specifies a level between conservative and moderate reduction, which can be used on most circuits with great performance and accuracy.

Argument	Description
3	Specifies moderate reduction, which is recommended for timing or post-layout simulations.
4	Specifies aggressive reduction, which is recommended for functional verification or higher process technology nodes.
5	Specifies the most aggressive reduction, which is recommended for the most possible speedup

Description

RC reduction is recommended for extracted netlists to reduce the runtime penalty. You can also enable conservative or aggressive reduction for individual nets.

- For the PrimeSim Pro tool, the default value is determined by the [primesim_mode](#) setting (see [Table 26](#)).
- For the PrimeSim SPICE tool, the default value is 0.

RC reduction is only performed on nets with a time constant value smaller than [primesim_spredtc](#); however, it is recommended to leave the default set by the PrimeSim tool.

Examples

```
.option primesim_spred="2 netA:0"
```

This example sets reduction level 2 for the whole circuit, and disables RC reduction for the `netA` network.

primesim_spredtc

Sets the time constant value used by the PrimeSim Pro tool RC reduction algorithm.

Syntax

```
.option primesim_spredtc="value netname1:value netname2:value ..."
```

Description

The RC reduction algorithm ignores any parasitic network frequencies higher than the given value.

The default value is preset by the [primesim_spred](#) option as follows:

- `.option primesim_spred=1` – Time constant value: 1e-13
- `.option primesim_spred=2` – Time constant value: 2e-13

- `.option primesim_spred=3` – Time constant value: 3e-13
- `.option primesim_spred=4` – Time constant value: 2e-12
- `.option primesim_spred=5` – Time constant value: 1e-9

Examples

```
.option primesim_spredtc="1.2p netA:10p"
```

This example sets the time constant value to 1.2ps for all nets, and to 10ps for the `netA` network.

primesim_subckt_dup_rule

Description

Selects which subcircuit definition to use in simulation when a simulation deck has multiple subckt definitions.

Note:

When the `-top` command-line option (see [Table 2](#)) is used to specify the top-level subcircuit name, the top subcircuit selects the first subckt definition regardless of how the `primesim_subckt_dup_rule` option is set.

Syntax

```
.option primesim_subckt_dup_rule=[0|1|2]
```

Arguments

Argument	Description
0	(default) Selects the last subckt definition.
1	Selects the first subckt definition.
2	Errors out for multiple subckt definitions.

primesim_sweep_brief

Limits the parametric sweep information written to the log file.

Syntax

```
.option primesim_sweep_brief=[0|1]
```

Description

By default (0), the PrimeSim tool outputs all parametric sweep information into the log file. Set this option to 1 to enable abbreviating the log file information for parametric sweep after the first sweep.

The log shows:

```
SWEEP #1.  
Ignore writing out the details for the rest of the sweep...
```

primesim_syntax_check

Description

When enabling this option, PrimeSim simulation errors out if any parameter that cannot be resolved due to undefined parameter.

Syntax

```
.option primesim_syntax_check[=1|strict|0]
```

Arguments

Argument	Description
1 or strict	Errors out if any parameter that cannot be resolved due to undefined parameter.
0	The option is turned off.

Note:

If this option is in the netlist without a value, it is equivalent to
`primesim_syntax_check=1`

If the option is not in the netlist, it is equivalent to `primesim_syntax_check=0`.

primesim_tcl_init_file

Description

Specifies the Tcl file for the PrimeSim tool to source for the simulation. For more information regarding support Tcl commands, refer to [Tcl Interactive Mode](#).

Syntax

```
.option primesim_tcl_init_file="file"
```


Examples

```
.option primesim_tcl_init_file="check_voltage.tcl"
```

The PrimeSim tool sources in `check_voltage.tcl` to execute the specified Tcl function.

primesim_tflush

Description

Sets the PrimeSim tool to flush output waveform files (`fsdb/tr0/wdf`) at the specified time interval of analysis. The default is `tstop/10`, or every 10% of the total transient analysis simulation time (see `.TRAN / TR` in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*).

Note:

Use `primesim_elapsed_tflush` to flush output files after a specified number of seconds in elapsed runtime since the previous flush.

Syntax

```
.option primesim_tflush=time_interval
```

Arguments

Argument	Description
<code>time_interval</code>	Can be specified in time units or as a percentage of the total transient analysis simulation time (<code>tstop</code>). Default is 10%. If specified in time units, for example, <code>10n</code> , the PrimeSim tool flushes output waveform files every 10 nanoseconds, regardless of <code>tstop</code> . If specified as a percentage, for example, <code>25%</code> , the PrimeSim tool flushes output waveform files every <code>tstop/4</code> .

Examples

Given the following:

```
.tran 1n 1000n
```

By default, the PrimeSim tool flushes output waveform files every `1000ns/10=100ns` (that is, `tstop/10`), or every 10% of the simulation.

If the following statement is added:

```
.option primesim_tflush=10n
```

Then in this example, the PrimeSim tool flushes output waveform files every 10ns, that is, for each 1% of the simulation.

However, if instead the following statement is added:

```
.option primesim_tflush=20%
```

Then in this example, the PrimeSim tool flushes output waveform files every 1000ns/5=200ns (that is, $t_{stop}/5$), or every 20% of the simulation.

primesim_title_in_alter_filenames

Description

Specifies whether to append the *title_string* from `.ALTER` statements to the output file name prefix.

Syntax

```
.option primesim_title_in_alter_filenames=[0|1]
```

Arguments

Argument	Description
0	(default) Does not append the <code>.ALTER title_string</code> to the output filename prefix.
1	Appends the <code>.ALTER title_string</code> to the output filename prefix. Note: Any blank spaces in <i>title_string</i> are converted to underscore (<code>_</code>) characters.

Examples

In the following example, the simulation is run for two different operating temperatures:

```
.option primesim_title_in_alter_filenames=1
...
.alter temp25
.temp 25
.alter temp 125
.temp 125
```

This produces the following output files:

```
spicemd#1.im
spicemd#1_temp25.fsdb
spicemd#1_temp25.ic
spicemd#1_temp25.mt0
spicemd#2.im
spicemd#2_temp_125.fsdb
```

```
spicemd#2_temp_125.ic  
spicemd#2_temp_125.mt0
```

primesim_tmiage_reltol

Description

For TMI aging simulation, devices can share the same table if the difference between their model and instance parameters is less than the relative tolerance specified by this option. The default is 0.1%.

Syntax

```
.option primesim_tmiage_reltol=value
```

Examples

```
.option primesim_tmiage_reltol=0.005
```

In this example, the acceptable relative tolerance is set to 0.5%.

primesim_tmishe_abstol

Description

For TMI self-heating simulation, devices can share the same table if the difference between their temperatures is less than the absolute tolerance specified by this option. The default is 1°C.

Syntax

```
.option primesim_tmishe_abstol=value
```

Examples

```
.option primesim_tmishe_abstol=2
```

In this example, the acceptable absolute tolerance is set to 2°C.

primesim_tolscale

Sets a multiplier applied to all internal tolerance values.

Syntax

```
.option primesim_tolscale="value [subckt1:value1 subckt2:value2 ...]  
[instance1:value1 instance2:value2 ...] [node1:value1 node2:value2  
...]" : [subckt|instance|node]
```

Time-varying tolerance values are also supported:

```
.option primesim_tolscale="{time1:value1 [time2:value2 ...]}"
```

Description

You determine the PrimeSim tool internal tolerances (such as for event signal) when you select a simulation mode (see [primesim_mode](#)). This option gives you the flexibility of trading off between accuracy and speed.

- For the PrimeSim Pro tool's modes (that is, `spicehd`, `spicemd`, `spicexd`, `prohd`, `promd`, and `proxd`), the recommended value range is from $1e-3$ to 1, with 1 being the default. The smaller the value, the more accurate the simulation.
- For the PrimeSim SPICE tool, if speed is desired over accuracy, you can set the value greater than 1 and as high as 150. Setting the value to less than 1 (as low as $1e-3$) tightens the simulation tolerance to achieve greater accuracy at the expense of speed.

The `primesim_tolscale` option is not only a global parameter, but can be extended to any hierarchy level `subckt` (default), `instance`, or `node` you choose. You can control this functionality as shown in the following examples, where `subckt1` equals `value1`, `subckt2` equals `value2`, or `instance1` equals `value1`, `instance2` equals `value2`, or `node1` equals `value1`, `node2` equals `value2`, respectively.

Note:

Time-varying tolerance values must be applied globally.

If the netlist includes multiple `primesim_tolscale` options, the last global tolerance values set override any previous ones.

Examples

In the following *subcircuit*-based example, the simulation tolerance is globally set to 0.1 except for digital control block `dig_cntl` which has the default value of 1.0 and `PLL` which is tightened by a factor of 0.01.

```
.option primesim_tolscale="0.1 dig_cntl:1.0 PLL:0.01"
```

In the following *node*-based example, the simulation tolerance for nodes in `xvreg.x_op_biasing_block*` is tightened by a factor of 0.2 and for nodes in `xvreg.x_op_vreg*` is tightened by a factor of 0.5.

```
.option primesim_tolscale="xvreg.x_op_biasing_block*:0.2  
xvreg.x_op_vreg*:0.5":node
```

In the following *instance*-based example, the simulation tolerance for instances in `xvreg.x_op_biasing_block*` is tightened by a factor of 0.2 and for instances in `xvreg.x_op_vreg*` is tightened by a factor of 0.5.

```
.option primesim_tolscale="xvreg.x_op_biasing_block*:0.2  
xvreg.x_op_vreg*:0.5":instance
```

In the following global example, the simulation tolerance is globally tightened by a factor of 0.5 for a more accurate result.

```
.option primesim_tolscale=0.5
```

In the following global example, the simulation tolerance is globally set to 30 to speed up the simulation with the PrimeSim SPICE tool.

```
.option primesim_tolscale=30
```

In the following time-varying example, the simulation tolerance is first globally set to 1 then is changed to 100 after 500 nanoseconds to speed up the simulation with the PrimeSim SPICE tool.

```
.option primesim_tolscale="{0:1 500n:100}"
```

In the following example, the global values of 10 from the first line and 0.1 from the second line are overridden by the time-varying global values of 1 and 100 from the third line. The local values of 1.0 for `dig_cnt1` and 0.01 for `PLL` are still recognized.

```
.option primesim_tolscale=10  
.option primesim_tolscale="0.1 dig_cnt1:1.0 PLL:0.01"  
.option primesim_tolscale="{0:1 500n:100}"
```

In the following example, the global value of 10 from the first line and the time-varying global values of 1 and 100 from the second line are overridden by the global value of 0.1 from the third line. The local values of 1.0 for `dig_cnt1` and 0.01 for `PLL` are also recognized.

```
.option finsim_tolscal=10  
.option primesim_tolscale="{0:1 500n:100}"  
.option primesim_tolscale="0.1 dig_cnt1:1.0 PLL:0.01"
```

primesim_tran_conv_retry

Description

When set to 1 and transient simulation fails to converge the first time, the PrimeSim tool reruns transient simulation a second time with `primesim_convlevel=1` set. If transient convergence has no problems in the first simulation, this option does nothing. The default is 0.

Syntax

```
.option primesim_tran_conv_retry=[0|1]
```

primesim_tsc

Sets the algorithm mode to be used for time step control during transient analysis.

Syntax

```
.option primesim_tsc=lte1|lte2
```

Description

This option sets the algorithm mode to be used for time step control during transient analysis. Its values are defined as follows, where LTE means Local Truncation Error:

- `primesim_tsc=lte1` (voltage-bound LTE)
- `primesim_tsc=lte2` (voltage- and current-based LTE)

While `lte1` is a general purpose approach, `lte2` is designed for analog circuits and charge-sensitive simulations. Though these two algorithms are analogous in core behavior, they have unique proprietary strategies for simulation convergence.

Note:

PrimeSim Pro tool only supports `lte1`. `lte2` is the default time step control in the PrimeSim SPICE tool.

primesim_tstop

Description

Modifies the `tstop` specified in the `.tran` command line to shorten/extend simulation length. It has the same functionality as `-stop` in the command line. If `primesim_tstop` is not defined, the PrimeSim tool uses the unmodified `tstop` specified in the `.tran` command line.

Syntax

```
.option primesim_tstop=value
```

Examples

```
.option primesim_tstop=100u
```

This example changes the simulation duration to `100u`.

primesim_tunit

Description

Defines the base time unit of the simulation. Internal simulation time is always represented by an integer multiple of this value. The defaults are 0.01ps for the PrimeSim SPICE tool and the PrimeSim Pro tool's `spicehd` mode, 0.1ps for `spicemd` mode, and 1ps for `spicexd` and Pro modes.

Syntax

```
.option primesim_tunit=time_unit
```

Examples

```
.option primesim_mode=proxd  
.option primesim_tunit=0.5ps
```

In this example, `proxd` simulation has a `tunit` of 0.5ps.

primesim_use_old_trout

Description

Determines whether to use the previous output file name extensions (`*.prt` and `*.msr`) or the newer ones (`*.pt0` and `*.mt0`) for `.print` and `.measure` during transient analysis. The default value is 0, which enables the newer extensions. If you want to use the previous extensions, set `primesim_use_old_trout=1`.

Syntax

```
.option primesim_old_trout=[0|1]
```

primesim_vdd

Sets the VDD value used for MOS table generation.

Syntax

```
.option primesim_vdd=value
```

Description

This option is usually used in high voltage generation circuitry, such as charge pumping. For such circuit simulation, higher voltage than the primary power supply is generated. To cover the wider range of voltage values, MOSFETs must be modeled to that generated voltage level.

The default value is the largest auto-detected voltage source value.

Note:

The maximum possible value is 30V. If the detected value is larger than 30V, the PrimeSim tool uses 30V.

Examples

```
.option primesim_vdd=20
```

In this example, the MOS table is modeled to 20V.

primesim_vector

Specifies a vector file that contains an input stimulus signal pattern and an expected output pattern for comparison with simulation results. The support vector data types are input, output, and bidirectional.

Syntax

```
.option primesim_vector="vector_file"
```

Description

Prior to version J-2014.09-SP2, when using a vector file, the PrimeSim tool would print the difference in the expected state and the simulated state to the log file, as follows:

```
*****
WARNING! vector output comparison error
time= 172ns, node= s[3], state= M, window= (172ns ~ 172ns)
      expect= 0, vec.time= 172ns (delay= 0s, steady= 0)
WARNING! vector output comparison error
time= 172ns, node= s[2], state= M, window= (172ns ~ 172ns)
      expect= 0, vec.time= 172ns (delay= 0s, steady= 0)
*****
```

As of version J-2014.09-SP2, the PrimeSim tool generates an `.err` file when comparing the simulated vs. expected states from a vector file, as follows:

```
*****
Time      Signal      Simulated      Expected
=====
172ns     s[3]         M              0
172ns     s[2]         M              0
*****
```

In addition, the PrimeSim tool prints a message to the log file, as follows:

```
*****
35 errors in vector output comparison
*****
```

See [Digital I/O Vectors](#) for more details.

primesim_vector_errmsg

Description

Specifies how errors are handled during vector comparison.

Syntax

```
.option primesim_vector_errmsg=[0|1|2]
```

Arguments

Argument	Description
0	Reports errors in the <code>.err0</code> file as set by <code>primesim_merge_vector_errmsg</code> and displays the total error messages at the end of simulation.
1	Reports errors in the log file and displays the total error messages at the end of simulation.
2	Reports errors in the log file and exits simulation after encountering the first error message.

primesim_vector_mode

Description

Sets the I/O vector file format for bus signals, 0 means it follows the PrimeSim HSPICE rule, while 1 means it follows the legacy HSPICE rule. Both `.VEC` and `primesim_vector` process vector file with this definition. If this option is absent, `.vec` processes bus signals as the PrimeSim HSPICE rule, while `primesim_vector` processes as the legacy HSPICE rule.

Syntax

```
.option primesim_vector_mode=[0|1]
```

Examples

```
.option primesim_vector_mode=0
```

Then the PrimeSim Pro tool expands `a[1:0]` in vector file to `a0` and `a1` for both `.vec` and `primesim_vector` commands.

```
.option primesim_vector_mode=1
```

Then the PrimeSim Pro tool expands `a[1:0]` in vector file to `a[0]` and `a[1]` for both `.vec` and `primesim_vector` commands.

If this option is absent, then the PrimeSim Pro tool expands `a[1:0]` in vector file to `a0` and `a1` for `.vec` command, and expands to `a[0]` and `a[1]` for `primesim_vector` command.

The default value of `primesim_vector_mode` is set to `1` for `.vcd2vec` and `.vec` when `-spectre` is given as a PrimeSim argument.

The `primesim_vector_mode=1` is used as default for `.option primesim_vector` all the time.

primesim_veriloga (.hdl)

Description

Includes a Verilog-A file in the netlist.

Syntax

```
.option primesim_veriloga=veriloga_file.va
```

primesim_veriloga_bypass

Enables the model bypass of entire Verilog-A models in the design. The default value is `1`. The use of this option can result in significant speed-up of Verilog-A simulations. However, you should not use this feature when a high level of accuracy is required.

Syntax

```
.option primesim_veriloga_bypass=[1|0]
```

Description

You can use this command with a selective list of model names, as in the following example:

```
.option primesim_veriloga_bypass=" 1 <model 1>:0 <model 2>:0"
```

The previous command disables model evaluation bypass for model 1 and model 2 but leaves bypass on for all other models. Another selective example is as follows:

```
.option primesim_veriloga_bypass=" 0 <model 1>:1 <model 2>:1"
```

The previous command disables model evaluation bypass for all models except model 1 and model 2.

primesim_veriloga_cross_method

Description

Specifies what value (`true|false`) the Verilog-A `cross(x)` function returns when `x` is exactly zero.

Syntax

```
.option primesim_veriloga_cross_method=[0|1]
```

Arguments

Argument	Description
0	The Verilog-A <code>cross(x)</code> function returns <code>true</code> when there is a timepoint <i>after</i> the crossing event (that is, <code>x>0</code>). If there is a timepoint exactly at the crossing event (that is, <code>x=0</code>), the <code>cross(x)</code> function returns <code>false</code> .
1	(default) The Verilog-A <code>cross(x)</code> function returns <code>true</code> when there is a timepoint exactly at the crossing event (that is, <code>x=0</code>).

primesim_veriloga_dkind_def

Description

The Verilog-A model is a "black box" model, so for Verilog-A elements, the PrimeSim tool cannot normally traverse the DC path or Hi-Z node using the dynamic circuit checker (see [Circuit Checks](#)).

This option defines the device type for the Verilog-A device model, allowing the dynamic circuit checker to operate normally on the Verilog-A device model just as with normal primitive devices. Undefined Verilog-A elements are processed as an open device.

The allowed device types are: R, L, C, PMOS, NMOS, PNP, NPN, NJET, PJET.

Note:

This option must be enabled using the `primesim_veriloga_dkind_mode` option.

Syntax

```
.option primesim_veriloga_dkind_def =  
"veriloga_model_name1:device_type1 veriloga_model_name2:device_type2 .  
.."
```

primesim_veriloga_dkind_mode

Description

Specifies whether to enable (1) or disable (0) the `primesim_veriloga_dkind_def` option. The default value is 0.

Syntax

```
.option primesim_veriloga_dkind_mode=[0|1]
```

primesim_veriloga_table_fill_in_policy

Description

Specifies the direction to start extrapolation if there are missing indexed entries when using the `table_param` built-in function (see [External Table Parameter File Interface](#)) to specify a file that contains a parameter values table and return the linear interpolated value from that table. This option is also used in the PrimeSim tool's native Verilog-A simulation engine's implementation of `$table_model()`.

Note:

The best approach is to provide a complete table, so there is no ambiguity as to what interpolated value to use.

Syntax

```
.option primesim_veriloga_table_fill_in_policy=[0|1|2|3]
```

Arguments

Argument	Description
0	(default) End simulation with an error message.
1	Average the values obtained with the leftmost (2) and rightmost (3) approaches.
2	Start with the leftmost value to extrapolate missing indexed entries from the table.
3	Start with the rightmost value to extrapolate missing indexed entries from the table.

primesim_veriloga_var_opt

Description

Sets some local variables within Verilog-A modules in order to have the variables shared among instances.

Note:

This option should be used only for local time-independent variables. If it is used for time-dependent variables, the results are wrong.

The multiple models and variables can be described in one option or multiple options. Wild cards (*|?) are allowed in the variable names.

Syntax

```
.option primesim_veriloga_var_opt="model_name={var1 var2 ...} ..."
.option primesim_veriloga_var_opt="model_name=(var1 var2 ...) ..."
.option
primesim_veriloga_var_opt="model_name=var1 var2; model_name=v1 v2; ..."
```

primesim_vprbtol

Description

Sets the voltage tolerance for voltage print out. You can use this option to control the resolution of output waveforms in fsdb or wdf.

- For the PrimeSim Pro tool, the default is 0.1mV.
- For the PrimeSim SPICE tool, the default is 1uV.

Syntax

```
.option primesim_vprbtol=value
```

primesim_vpwltol

Removes small changes in the voltage input waveform in PWL format.

Syntax

```
.option primesim_vpwltol=value
```

Description

If the voltage difference of two consecutive points of PWL waveform is less than the value given by this option, the PrimeSim tool removes one of the two points to smooth the waveform. The default value is 1e-4.

Examples

```
.option primesim_vpwltol=1m  
.option primesim_vpwltol=0 (all the points are kept)
```

primesim_warn_limit

Sets the number of all warning messages to a limited number of user-specified lines.

Syntax

```
.option primesim_warn_limit=value
```

Description

If you set this option to an integer value, the PrimeSim tool prints the specified number of warning messages for every warning type. After simulation, the PrimeSim tool reports how many warnings have been suppressed.

The default value is 10, which means the PrimeSim tool reports ten warning messages and suppresses the rest.

primesim_wdf_limit

Description

Splits wdf output files that exceed a user-specified value. The value specified places an upper limit on the file size before splitting. The units are in Mb. If `primesim_wdf_limit` is not defined, then no splitting is performed.

Syntax

```
.option primesim_wdf_limit=value
```

primesim_wdf_mode

Description

Controls the amount of compression for the wdf output file. The available options are `none`, `lossless`, `fast`, `max`. The default value is `lossless`.

Syntax

```
.option primesim_wdf_mode=[none|lossless|fast|max]
```

primesim_write_instance_param

Description

Dumps out a list of resolved instance parameters with full scope names into a `.iparam` file when this option is set to 1. The default is 0 where no `.iparam` file is created.

Syntax

```
.option primesim_write_instance_param=[0|1|2]
```

Examples

The output file format is as follows:

```
"instance_name" (subckt_name)
  parameter1 = value1
  parameter2 = value2
:

"instance_name" (subckt_name)
  parameter1 = value1
  parameter2 = value2
:
```

Example 1

```
"x1" (buf) (in.sp:21)

"x1.x1" (inv) (in.sp:15)
  A = 1

"x1.x2" (inv) (in.sp:15)
  A = 2
```

Example 2

```
.iparam file content with primesim_write_instance_param=1:

circuit name directory and parameter definitions
  "m1" (dgxnfet) (/file_path/input.sp:4)
    w = 1u
    l = 0.18u
```

Example 3

```
.iparam file content with primesim_write_instance_param=2:

circuit name directory and parameter definitions
  "m1" (dgxnfet) (1:4)
    w = 1u
    l = 0.18u
    par = 1
```

```
dtemp = 0
as = 0
ad = 0
ps = 0
pd = 0
sa = 0
sb = 0
sd = 0
nf = 1
tw = 0
```

primesim_write_instance_table

Description

Generates an instance list (`.ins`) file when this option is set to 1. The instance list has information about which instance corresponds to which subcircuit. The default is 0 where no instance list is created.

Syntax

```
.option primesim_write_instance_table=[0|1]
```

primesim_write_mcparam

Description

When `primesim_write_mcparam` is set to a non-zero value during Monte Carlo analysis, the PrimeSim Pro tool saves the specified random parameters generated in each run to a file. This file is located in the output directory with the following naming convention:

inputprefix.mcpanlysisN

For example, `input.mcpt0` is the file name for a transient analysis MC without sweep.

The contents of this file is similar to a measure file in ASCII format and can be loaded into the PrimeWave Design Environment WaveView tool for plotting distributions and scatter plots.

The default is 0 where no extra Monte Carlo data is saved.

Syntax

```
.option primesim_write_mcparam=[0|1|d,m,p]
```

Arguments

Argument	Description
0	(default) Do not save Monte Carlo data.

Argument	Description
1	Save all Monte Carlo local, global, model, and device parameters to a file.
d	Save Monte Carlo device parameters to a file.
m	Save Monte Carlo model parameters to a file.
p	Save Monte Carlo local and global parameters to a file.

Examples

```
.option primesim_write_mcparam=1
```

This example saves all Monte Carlo random parameters to a file.

```
.option primesim_write_mcparam="m,d"
```

This example saves Monte Carlo model and device parameters to a file, but does not save local and global parameters.

pva_genso_filebase

Description

Automatically generates the `.so` files per VA files based on all `.hdl` VA files or reuse `.so` files during simulation. The default is 0.

Syntax

```
.option pva_genso_filebase=[1 | 0]  
Or  
-aopt pva_genso_filebase=[1 | 0]
```

Examples

Example of a simple flow to generate and use the `.so`

```
.hdl resistor.va  
.hdl capacitor.va
```

First run

```
Generated ./  
mylib.pvadir/pvaRTL_619317473fbd3044878ff78200006b2fLN_resistor.so  
Generated ./  
myliboutf.pvadir/pvaRTL_619317473fbd3044878ff78200006b2fLN_capacitor.so
```

Second run

```
./mylib.pvalib/pvaRTL_619317473fbd3044878ff78200006b2fLN_resistor.so is reused
./mylib.pvadir/pvaRTL_619317473fbd3044878ff78200006b2fLN_capacitor.so is reused
```

Internally determines whether to generate `.so` or reuse the `.so` by checksum not included CL#.

The `pva_genso_filebase` can be used with `pva_output_dir` to specify the output directory.

pva_output_dir

Description

Controls the cache directory or sets the output directory of the Portable Verilog-A Engine (pVA) to the desired location. It is very useful for reusing `.so` files.

Syntax

```
.option pva_output_dir=<path>
Or
-aopt pva_output_dir=<path>
```

Examples

```
.option pva_output_dir="./mylib"
Or
-aopt pva_output_dir="./mylib"
```

senssort

Used to sort the sensitivity results from big to small for DC sensitivity or AC sensitivity if its value is 1. If its value is 0 (as default), sensitivity results are not sorted.

Syntax

```
.option senssort=0|1 (default 0)
```

Description

For DC sweep, `senssort=1` sorts the sensitivity (absolute value) for all sweep points. But the contribution percentage is only for one sweep point.

For AC sweep, `senssort=1` sorts the sensitivity results according to the magnitude.

```
sqrt (Sensitivity.Real^2+Sensitivity.Imag^2)
```

s_cache_reuse

Global option to turn on and off S parameter cache reuse.

Syntax

```
.option s_cache_reuse=[0|1]
```

Description

Use to turn on and off S parameter cache reuse. It can be overridden by that in instance or model specification.

This option may influence the cache reuse in IFFT (impulse response), rational fitting results and prune or compress S model.

For rational model cache reuse, option `s_rational_func_reuse` has higher priority and may overwrite `s_cache_reuse`.

- When `.option s_cache_reuse=0`: No cache reuse
- When `.option s_cache_reuse=1`: Cache reuse

PrimeSim Commands

This section lists the following commonly used PrimeSim-compatible commands:

- [.AUTOSETTLE](#)
- [.FORCE](#)
- [.RELEASE](#)

.AUTOSETTLE

Description

The `.AUTOSETTLE` statement checks the settling time to determine whether the voltage signals meet a specified type of settling analysis within a tolerance value. If the voltage signals meet the condition, the simulation ends before any user-defined `tstop`. The command also checks whether the settling time, under the conditions of the observed signal, meets the tolerance for each repeating period.

You can use multiple commands to consider multiple signals simultaneously. Simulation ends when all of the multiple commands meet the constraints. If one of the signals does not meet the constraints for multiple signals, the repeating period is reset and begins again at the next period.

If this command conflicts with another measurement, the simulation ends when both `.AUTOSETTLE` and the other measurement have completed.

The observed signals that you specify are added to the probed signal list automatically to make the output waveform.

When the settling times for the observed signals are detected, this command records the settling time information in the log file.

Syntax

```
.AUTOSETTLE signal=sig_name1 [sig_name2 sig_name3 ...]  
+ type=[min|max|avg|all] period=period_time  
+ [tolerance=tol_value] [start=start_time]  
+ [repeat=repeat_number_of_period]
```

Arguments

Argument	Description
<i>sig_name1, sig_name2, ...</i>	Signals to be observed for settling. Note: Currently, only voltage signals are supported.
<i>type</i>	Type of settling analysis to be performed: min, max, avg, or all.
<i>period_time</i>	Timing period for measurement of minimum (min), maximum (max), and average (avg) values of the observed signals.
<i>tol_value</i>	Tolerance value for determining whether the observed signals settle within the specified repeat period. Default is 2mV.
<i>start_time</i>	Start time to begin checking the settling time of the observed signals. Default is 0ns.
<i>repeat_number_of_period</i>	Number of repetitions of the period during which the signals are observed. The default value is calculated by: <code>int((tstop-start)/period).</code>

Examples

```
.AUTOSETTLE signal=bl_in type=avg tolerance=10u start=400p period=1p repeat=2  
.AUTOSETTLE signal=blb_in type=avg tolerance=1m start=400p period=1p repeat=3
```

In this example, the PrimeSim tool observes the settling times for the voltage signals, `v(bl_in)` and `v(blb_in)` based on their average values. Voltage signal `v(bl_in)` is observed for two repetitions with a tolerance value of 10uV, while signal `v(blb_in)` is observed for three repetitions with a tolerance value of 1mV.

.FORCE

Description

The `.FORCE` and `.RELEASE` commands are equivalent to the interactive mode commands, `fn` and `rn`, respectively, but with different syntax.

The `.FORCE` command forces the specified nodes or data bus lines to keep a constant voltage. To avoid abrupt changes of voltage, the voltages of the forced nodes change from their current voltage to the specified voltage at the given slope.

Syntax

For regular signals:

```
.FORCE [-time t0] [-slope slope0] nodename0=val0 [[-time t1]
[-slope slope1] nodename1=val1 ...]
```

For bus signals:

```
.FORCE [-time t0] [-slope slope0] -bus bus_name -vh high_value
-vl low_value logical_value_for_bus
```

Arguments

Argument	Description
<i>t0 t1 ...</i>	Specifies the time at which to force a node or data bus from its current voltage to the specified voltage.
<i>slope0 slope1 ...</i>	Specifies the slope of the voltage change. Units are ps/V, and the default slope is 10 ps/V.
<i>nodename0=val0 nodename1=val1 ...</i>	For regular signals, specifies one or more nodes to force to a constant voltage.
<i>bus_name</i>	For bus signals, specifies the range of lines in a data bus to force to a constant voltage, at either <i>high_value</i> or <i>low_value</i> , depending on the signal mask specified by <i>logical_value_for_bus</i> .
<i>high_value</i>	For bus signals, specifies the high voltage forced on lines in a data bus marked by a 1 in <i>logical_value_for_bus</i> .
<i>low_value</i>	For bus signals, specifies the low voltage forced on lines in a data bus marked by a 0 in <i>logical_value_for_bus</i> .
<i>logical_value_for_bus</i>	For bus signals, acts as a signal mask to determine which lines in a data bus are forced high (1), low (0), or left unchanged (x).

Examples

For regular signals:

```
.FORCE -time 5n -slope 0.1n sig_a=0.3
.RELEASE -time 7n sig_a
.FORCE -time 8n -slope 0.1n sig_a=0.5 -time 8.5n -slope 0.2n sig_a=0.6
-time 9n -slope 0.3n sig_b=0.7

.FORCE -time 7n -slope 0.1n a[0]=0.5
.FORCE -time 8n -slope 0.1n a[0]=0.5 -time 8.5n -slope 0.2n a[0]=0.6
-time 9n -slope 0.3n a[0]=0.7
.RELEASE -time 10n a[0]
```

For BUS signals:

```
.FORCE -time 0n -slope 0.1n -bus a[3:0] -vh 1.5 -vl 0 1010
.RELEASE -time 5n -bus a[3:0]
.FORCE -time 6n -slope 0.1n -bus a[3:0] -vh 1.6 -vl 0 0110
```

.RELEASE

Description

The **.FORCE** and **.RELEASE** commands are equivalent to the interactive mode commands, **fn** and **rn**, respectively, but with different syntax.

The **.RELEASE** command releases the constant voltage set by the **.FORCE** command for the specified nodes or data bus. The node voltages are therefore determined by normal simulation result.

Syntax

For regular signals:

```
.RELEASE -time=t0 nodename0 ... | -all | -alltime [-time=t1 nodename1 ... |
-all | -alltime]
```

For BUS signals:

```
.RELEASE -time=t0 -bus bus_name | -all | -alltime
```

Arguments

Argument	Description
<i>t0 t1 ...</i>	Specifies the time at which to release a node or data bus from its forced voltage to its normal simulated voltage.
<i>nodename0 nodename1 ...</i>	For regular signals, specifies one or more nodes to release from a forced voltage.

Argument	Description
<i>bus_name</i>	For bus signals, specifies the range of lines in a data bus to release from a forced voltage.
-all	Specifies that all currently forced nodes and data bus lines be released.
-alltime	Specifies that all currently forced nodes and data bus lines be released, as well as those still to be forced even if their <i>time</i> has not yet been reached.

Examples

For regular signals:

```
.FORCE -time 5n -slope 0.1n sig_a=0.3
.RELEASE -time 7n sig_a
.FORCE -time 8n -slope 0.1n sig_a=0.5 -time 8.5n -slope 0.2n sig_a=0.6
-time 9n -slope 0.3n sig_b=0.7

.FORCE -time 7n -slope 0.1n a[0]=0.5
.FORCE -time 8n -slope 0.1n a[0]=0.5 -time 8.5n -slope 0.2n a[0]=0.6
-time 9n -slope 0.3n a[0]=0.7
.RELEASE -time 10n a[0]
```

For BUS signals:

```
.FORCE -time 0n -slope 0.1n -bus a[3:0] -vh 1.5 -vl 0 1010
.RELEASE -time 5n -bus a[3:0]
.FORCE -time 6n -slope 0.1n -bus a[3:0] -vh 1.6 -vl 0 0110
```

10

PrimeSim HSPICE-Compatible Commands and Control Options

This chapter is a reference for commonly used PrimeSim HSPICE-compatible commands and control options.

This chapter covers the following topics:

- [PrimeSim HSPICE-Compatible Commands](#)
- [PrimeSim HSPICE-Compatible Control Options](#)
- [Optical Port Mapping](#)
- [Bisection Optimization](#)

PrimeSim HSPICE-Compatible Commands

Note:

PrimeSim HSPICE-compatible commands are documented in the PrimeSim HSPICE Simulation Command Reference chapter in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The PrimeSim tool supports the following PrimeSim HSPICE-compatible commands:

- .AC
- .ALTER
- .BIASCHK
- .CONNECT
- .DATA
- .DC
- .DCVOLT
- .DEL LIB

Chapter 10: PrimeSim HSPICE-Compatible Commands and Control Options

PrimeSim HSPICE-Compatible Commands

- .DOUT
- .DTEMP_LIST
- .END
- .ENDDATA
- .ENDL
- .ENDS
- .EOM
- .FFT
- .FLAT
- .FOUR
- .FSOPTIONS
- .GLOBAL
- .GPD_INC
- .HB
- .HBAC
- .HBNOISE
- .HBOSC
- .HBXF
- .IBIS
- .IC
- .IF / .ELSE
- .INCLUDE / INC / INCL
- .INSTVALID
- .IVTH
- .LAYERSTACK
- .LIB
- .LIN

- `.LOAD`
- `.LSTB`
- `.MACRO`
- `.MALIAS`
- `.MATERIAL`
- `.MEASURE` / `MEAS` (see also [.MEASURE](#) in this manual)
- `.MODEL`
- `.MODEL_INFO`

Note:

The `*.model_info` file does not have the same output format as with the PrimeSim HSPICE command; every instance has the model information reported.

- `.NODESET`
- `.NOISE`
- `.OP`
- `.OPTION` / `OPTIONS`
- `.PARAM` / `PARAMETER` / `PARAMETERS`
- `.PAT`
- `.PHASENOISE`
- `.PKG`
- `.PRINT`
- `.PROBE`
- `.PROTECT` / `PROT`
- `.PZ`
- `.SAVE`
- `.SHAPE`
- `.SN`
- `.SNAC`

- `.SNNOISE`
- `.SNOSC`
- `.SNXF`
- `.SUBCKT`
- `.TEMP / TEMPERATURE`
- `.TF`
- `.TRAN / TR`
- `.UNPROTECT / UNPROT`
- `.VEC`

The following commands are not documented in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*, so they are included in this manual:

- `.NET`
- `.XF`

.NET

The `.NET` option computes the parameters of the impedance, admittance, scattering, and hybrid matrices. This is done as part of the AC analysis.

Syntax

```
.NET output input-source [ROUT=xxx] [RIN=yyy]
```

Description

The input can be voltage or current source with AC input.

The output can be voltage (for example, `V(N1)`).

The input/output impedances default to 1 Ohm.

Parameters of the various matrices can be printed using the following syntax, where *i* and *j* are 1 or 2, indicating the element of the 2x2 matrix to be printed and *x* is one of: `M` (magnitude, default if *x* omitted), `P` (phase), `R` (real part), `I` (imaginary part), `DB` (decibel).

Examples

```
ZIN(x), ZOUT(x), YIN(x), YOUT(x), Sij(x), Zij(x), Hij(x), Yij(x)
```

```
.ac lin 100 100meg 200meg
```

```
.net v(out) vin rout=50 rin=50
.print ac zin(m) yout(p) s11(m) s21(db) z11(r)
```

In this example, the PrimeSim Pro tool performs a network analysis by using the `.ac` and `.net` analysis statements, and prints the magnitude of the input impedance, the phase of the output admittance and several S and Z parameters.

.XF

Description

Command for AC small signal transfer functions.

Syntax

```
.XF ov type np fstart fstop
```

Arguments

Argument	Description
ov	Small signal output variable.
type np fstart fstop	Refer to <code>.AC</code> in the <i>PrimeSim™ Continuum Reference Manual: Commands and Control Options</i> .

Examples

```
.xf v(1,2) dec 10 1e2 1e9
```

PrimeSim HSPICE-Compatible Control Options

Note:

PrimeSim HSPICE-compatible control options are documented in the PrimeSim HSPICE Simulation Control Options Reference chapter in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

The PrimeSim tool supports the following PrimeSim HSPICE-compatible control options:

- `.OPTION AC_INIT`
- `.OPTION ALTER_SELECT`
- `.OPTION ASPEC`
- `.OPTION AUTOSTOP`
- `.OPTION CAPTAB`

Chapter 10: PrimeSim HSPICE-Compatible Commands and Control Options

PrimeSim HSPICE-Compatible Control Options

- .OPTION CHECK_TABLE_PARAM_FILE
- .OPTION CMIFLAG
- .OPTION CMIMULTILIB
- .OPTION CMIPATH
- .OPTION CMIUSRFLAG
- .OPTION CMIVTH
- .OPTION CSHUNT
- .OPTION DCAP
- .OPTION DCIC
- .OPTION DEFAD
- .OPTION DEFAS
- .OPTION DEFL
- .OPTION DEFNRD
- .OPTION DEFNRS
- .OPTION DEFPPD
- .OPTION DEFPS
- .OPTION DEFW
- .OPTION DP_DISKFREE
- .OPTION DP_LOADAVG
- .OPTION DP_MASTER_WAITTIME
- .OPTION DP_MEMFREE
- .OPTION DP_TASK_MAXATTEMPT
- .OPTION EXPMAX
- .OPTION FAST_SENS_METHOD
- .OPTION GDCPATH
- .OPTION GEN_CUR_POL
- .OPTION GEOSHRINK

Chapter 10: PrimeSim HSPICE-Compatible Commands and Control Options

PrimeSim HSPICE-Compatible Control Options

- `.OPTION GMIN`
- `.OPTION GMINDC`
- `.OPTION GRAMP`
- `.OPTION GSHUNT`
- `.OPTION HIER_SCALE`
- `.OPTION ICSWEEP`
- `.OPTION IGNORE_MALIAS`
- `.OPTION INGOLD`
- `.OPTION INTERP`
- `.OPTION ITL1`
- `.OPTION IMIN / ITL3` (see [Obsolete Options–Transient Time Step and Accuracy](#))
- `.OPTION IMAX / ITL4` (see [Obsolete Options–Transient Time Step and Accuracy](#))
- `.OPTION LOADSNINIT`
- `.OPTION MACMOD`
- `.OPTION MEASDGT`
- `.OPTION MEASFILE`
- `.OPTION MEASLABEL`
- `.OPTION MONTECON`
- `.OPTION XMULT_IN_EXP / M_IN_EXP`
- `.OPTION NUMDGT`
- `.OPTION PARHIER`
- `.OPTION POST [PROBE]`
- `.OPTION POST_VERSION`
- `.OPTION PUTMEAS`
- `.OPTION REPLICATES`
- `.OPTION RISETIME / RISETI`
- `.OPTION RM_TMIAGE_FILE`

Chapter 10: PrimeSim HSPICE-Compatible Commands and Control Options

PrimeSim HSPICE-Compatible Control Options

- .OPTION S_CAUSALITY
- .OPTION S_CAUSALITY_FMAX_SCALE
- .OPTION S_ELEM_CACHE_DIR=dir_str
- .OPTION S_RATIONAL_FUNC
- .OPTION S_RATIONAL_FUNC_WINDOWING
- .OPTION S_RATIONAL_FUNC_DC_ACC
- .OPTION S_RATIONAL_WRITE_FITTED_FILE
- .OPTION S_WRITE_CAUSALITY_ENFORCED_FILE
- .OPTION SAMPLING_METHOD
- .OPTION SAVESNINIT
- .OPTION SCALE
- .OPTION SCALM
- .OPTION SEARCH
- .OPTION SSTAMP
- .OPTION TMIAGE
- .OPTION TMIAGEMODE
- .OPTION TMIFLAG
- .OPTION TMIINPUT
- .OPTION TMIPATH (see **Obsolete Options—Transient Time Step and Accuracy**)
- .OPTION TMISAVE
- .OPTION TMPLT_POL
- .OPTION TNOM
- .OPTION UNWRAP
- .OPTION WARNLIMIT
- .OPTION XDTEMP

The following control options are not documented in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*, so they are included in this manual:

- [ge_rational_func](#)
- [hisimhv_version_update](#)
- [s_rational_func_preset](#)
- [stress_input](#)

ge_rational_func

Description

Specifies the method used for Laplace transform source convolution.

Syntax

```
.option ge_rational_func=[0|1]
```

Arguments

Argument	Description
0	State variable method. This is the default method for the PrimeSim HSPICE tool.
1	(default) Recursive convolution based on pole/residue expression.

hisimhv_version_update

Description

Specifies whether to change the version of HiSIM_HV being used due to problems in the version 1.22 model equation that could result in convergence issues.

Syntax

```
.option hisimhv_version_update=[0|1]
```

Arguments

Argument	Description
0	(default) Use the given version of HiSIM_HV.
1	Change HiSIM_HV version 1.22 to 1.23 which fixes problems in the 1.22 model equation.

s_rational_func_preset

Description

Specifies whether to enforce passivity when using rational function approximation for all S-elements in a netlist.

When passivity is enforced, the S-element checks the passivity of all the frequency sampling points and if violations are found, the S-element seeks a minimum amount of loss property, which restores the passivity of all the violated points. The S-element then adds the loss to all the given frequency points.

Syntax

```
.option s_rational_func_preset=[rf|si]
```

Arguments

Argument	Description
rf	(default) Enforces passivity, such as for RF simulation.
si	Does not enforce passivity, such as for signal integrity (SI) simulation.

stress_input

Description

PrimeSim SPICE and Pro supports FSDB format file-based Open Model Interface (OMI) aging computation. When this option is set, the transient simulation is skipped, and device stress information (node voltage) is retrieved from the input file instead of transient simulation results. This option only applies within the OMI aging flow.

Syntax

```
.option stress_input='filename'
```

Arguments

Argument	Description
<i>filename</i>	FSDB file name.

Optical Port Mapping

This section lists the following PrimeSim HSPICE-compatible commands and control options for handling optical port mapping:

- [.Convert_Port](#)
- [.OPTION OPTICAL_PORT](#)
- [.PortMapping](#)
- [Port Mapping Example](#)

Note:

The `optical_port` control option, the `.PortMapping` and `.Convert_Port` commands, and a reserved keyword in a `.SUBCKT` declaration, `SUB_ALIAS`, that defines the new subcircuit name after port conversion, must all be used together, as shown in [Port Mapping Example](#).

.Convert_Port

Description

Specifies one or more subcircuit ports to be expanded based on the port mapping rule defined by the `.PortMapping` command.

Syntax

```
.Convert_Port subckt_node1=PortMapping_rule1  
[subckt_node2=PortMapping_rule2] ... [subckt_port_nodeN=PortMapping_ruleN]
```

Note:

- The `.Convert_Port` command must be placed inside a `.SUBCKT` statement.
- One `.Convert_Port` command can define multiple subcircuit port expansions.
- Multiple `.Convert_Port` commands can be used in one subcircuit declaration.

Examples

The following example defines that the subcircuit ports, `p1` and `p2`, are expanded using port mapping rule `optical4`.

```
.PortMapping port=optical4 struct=ND1,ND2,ND3,ND4  
.SUBCKT dev p1 p2 plus minus Sub_Alias=dev_PortMapping length="10e-6"  
.Convert_Port p1="optical4" p2="optical4"
```

```
.ends
Xm1 NET1 NET2 P M dev length=1e-6
```

After the PrimeSim tool resolves the `.PortMapping` and `.Covert_Port` commands, new nodes in the simulation are `NET1_ND1`, `NET1_ND2`, `NET1_ND3`, `NET1_ND4`, `NET2_ND1`, `NET2_ND2`, `NET2_ND3`, and `NET2_ND4`.

.OPTION OPTICAL_PORT

Description

Expands one optical port to multiple electrical nodes operation. If the option is not specified in the netlist, the value defaults to 0; if the option is specified in the netlist without a corresponding value, it defaults to 1.

When this option is set to 1, the PrimeSim tool converts an optical port to electrical nodes based on definitions in the `.PortMapping` and `.Convert_Port` commands.

Syntax

```
.option optical_port=[0|1]
```

Examples

```
.option optical_port=1
```

.PortMapping

Description

Defines the port mapping rule used by the `.Convert_Port` command.

Syntax

```
.PortMapping port=[rule_name struct=node1,node2,... nodeN]
```

Arguments

Argument	Description
<i>rule_name</i>	Defines the name of a port mapping rule.
<i>node1,node2,... nodeN</i>	Defines one or more node names for a port to be expanded.

Examples

The following example defines a port mapping rule named `optical4`, that expands one optical port to four nodes named `port_ND1`, `port_ND2`, `port_ND3`, and `port_ND4`, where

the prefix `port` refers to an X element node that is instantiated from a `.SUBCKT` declaration that contains a `.Convert_Port` command.

```
.PortMapping port=optical4 struct=ND1,ND2,ND3,ND4
```

Port Mapping Example

In the following example, port expansion is performed to `Xm1` instantiated from the `.SUBCKT dev` declaration.

```
.option optical_port=1
**** Define the port expansion rule
.PortMapping port=optical4 struct=ND1,ND2,ND3,ND4
**** After port expansion, a new subcircuit, dev_PortMapping, is used for
dev instances in simulation
.SUBCKT dev p1 p2 plus minus SUB_ALIAS=dev_PortMapping length="10e-6"
**** Subcircuit ports p1 and p2 use the optical4 rule to expand
.Convert_Port p1=optical4 p2=optical4
**** In the wrapper subcircuit, use "." in the expanded node name to
specify port-node hierarchy
xdev p1.ND1 p1.ND2 p1.ND3 p1.ND4 p2.ND1 p2.ND2 p2.ND3 p2.ND4 plus minus
dev_PortMapping length="10e-6"
.ends

.SUBCKT dev_PortMapping p1_ND1 p1_ND2 p1_ND3 p1_ND4 p2_ND1 p2_ND2 p2_ND3
p2_ND4 plus minus length="a+b"
.param a=1e-6 b=2e-6
**** In the content subcircuit, use "_" in the expanded node name
xin1 p1_ND1 p1_ND2 p1_ND3 p1_ND4 in_1 ... internal1
xin2 p2_ND1 p2_ND2 p2_ND3 p2_ND4 in_2 ... internal2
.ends
Xm1 NET1 NET2 P M dev length=1e-6
```

After port expansion, `Xm1` uses the `.SUBCKT dev_PortMapping` declaration for simulation, with nodes `NET1` and `NET2` being expanded to nodes `NET1_ND1`, `NET1_ND2`, `NET1_ND3`, `NET1_ND4`, `NET2_ND1`, `NET2_ND2`, `NET2_ND3`, and `NET2_ND4`, that is:

```
Xm1 NET1_ND1 NET1_ND2 NET1_ND3 NET1_ND4 NET2_ND1 NET2_ND2 NET2_ND3
NET2_ND4 P M dev_PortMapping length=1e-6
```

Bisection Optimization

Bisection automates the characterization of a design for setup, hold, or minimum pulse width. The PrimeSim tool provides two different methods for defining the exit criteria:

- BISECTION and PASSFAIL. Both methods use a binary search method varying the input parameters to optimize. The difference is in the definition of the goal:
- PASSFAIL exits the binary search when the difference between the two latest test input values is within the error tolerance and one of the values passes the goal with the other failing.
 - BISECTION works in a similar fashion except that the last two test input values are within the error tolerance and the latest measured value is compared to the goal.

In general, setup and hold values are acquired using the BISECTION method. Minimum pulse width is typically acquired using the PASSFAIL method. All methods require that the input stimulus be properly defined with both inputs and output identified. The optimization automatically alters the input stimulus relationship to achieve the desired goal.

To perform bisection optimization, the `.TRAN / TR`, `.PARAM / PARAMETER / PARAMETERS`, `.MEASURE / MEAS`, and `.MODEL` statements must be used. See the *PrimeSim™ Continuum Reference Manual: Commands and Control Options* for more information on these and other PrimeSim HSPICE-compatible commands.

Syntax

```
.TRAN [TranStep] [TranTime] SWEEP
+ OPTIMIZE=Optxxx
+ RESULTS=MeasName
+ MODEL=ModName
.PARAM ParName=Optxxx (Initial, Lower, Upper)
.MEASURE TRAN MeasName MeasureClause
+ [[GOAL=<|>|<=|>=val]|GOALMAX|GOALMIN]
.MODEL ModName OPT METHOD=BISECTION|PASSFAIL Relin=val Relout=val
+ Itropt=val
```

The `OPTIMIZE` keyword is followed by the Bisection optimization parameter.

The `Optxxx` optimization parameter reference name must agree with the `OPTxxx` name given in the `.TRAN` statement associated with the keyword `OPTIMIZE`.

Table 28 .PARAM Statement Keywords

Keyword	Meaning
ParName	Name of the bisection optimization parameter.
Initial	Initial value for bisection optimization parameter.
Lower	Left-bound for bisection optimization parameter.
Upper	Right-bound for bisection optimization parameter.

The measure results for *Lower* and *Upper* limits of *ParName* must be on opposite sides of the `GOAL` value in the `.MEASURE` statement. For the `PASSFAIL` method, the measure must pass for one limit and fail for the other limit. The process ignores the value of the *Initial* field.

Table 29 `.MEASURE TRAN` Statement Keywords

Keyword	Meaning
<code>RESULT</code>	This keyword is followed by the target for optimization.
<i>MeasName</i>	Name of the measure calculated by a <code>.MEASURE</code> statement.
<code>GOAL= < > <= >=val</code>	<code>GOAL</code> value compared to the associated goal measurement: <code>=, >=</code>: If the associated goal measurement equals or exceeds <i>val</i>, it is a "pass"; otherwise it is a "fail". <code>></code>: If the associated goal measurement exceeds <i>val</i>, it is a "pass"; otherwise it is a "fail". <code><</code>: If the associated goal measurement falls below <i>val</i>, it is a "pass"; otherwise it is a "fail". <code><=</code>: If the associated goal measurement falls below or equals <i>val</i>, it is a "pass"; otherwise it is a "fail".
<code>GOALMAX GOALMIN</code>	Use bisection method to get maximum/minimum measure value.
<code>MODEL</code>	This keyword is followed by the optimization model.
<i>ModName</i>	The model name, used by bisection optimization to reference a particular model.
<code>OPT</code>	This keyword indicates that optimization is to be performed.
<code>METHOD</code>	This keyword indicates which optimization method to use.
<i>Relin</i>	Error tolerance. This specifies the relative input parameter variation for convergence. The default value is $1e-3$, which means the last two input parameter variations should be less than $1e-3 * (\text{Lower}, \text{Upper})$ before the bisection succeeds and ends.
<i>Relout</i>	Error tolerance. This specifies the relative output measure variation for convergence. The default value is $1e-3$, which means the last two output measure variations should be less than $1e-3$. That is: $ (current_sweep_measurement_value - last_successful_sweep_measurement_value) / last_successful_sweep_measurement_value < Relout$
<i>Itropt</i>	Sets the maximum number of iterations. The default value is 20.

Bisection is further described in the following sections:

- [Example for Setup Time Analysis With Bisection](#)
- [Example for Minimal Pulse Width With Passfail](#)
- [Pushout Bisection](#)
- [Bisection Output Convention](#)
- [Bisection Analysis With Two Measurements](#)
- [Concurrent Bisection for Independent Circuit Blocks](#)

Example for Setup Time Analysis With Bisection

```
*include your model and netlist here
.inc "modelfile"
.inc " dff.sp"

* apply clock/data stimulus
vdata data gnd PWL
+ 0s 5v
+ 1n 5v
+ 2n 0v
+ 'Delay' 5v

vclock clock gnd PWL
+ 0s 0v
+ 3n 0v
+ 4n 5v

.TRAN 1n 10n Sweep Optimize = Opt1 Result = MaxVout Model = OptMod
.PARAM Delay= Opt1 ( 0n, 0n, 5.0n )
.MEASURE Tran MaxVout Max v(D_Output) Goal = 4.8
.MEASURE Tran SetupTime Trig v(Data) Val = 2.5 Rise = 1 Targ v(Clock)
+ Val = 2.5 Rise = 1
.MODEL OptMod Opt Method = Bisection Relin=0.1
.end
```

The PrimeSim tool reports the calculation result for each sweep:

```
*****
.....
SWEEP #5
bisec-opt iter = 4, delay = 1.56253218750000031045e-09, status: fail
.....
SWEEP #6
bisec-opt iter = 5, delay = 1.40633218750000031045e-09, status: success
*****
```

The measure file (mt0) also shows how the PrimeSim tool calculates optimum value of 'delay':

```
*****
$DATA1 SOURCE='PrimeSimPro' VERSION='2020.12-SP1'
.TITLE ''
delay      maxvout      setuptime
0.0000e+00  5.0006e+00    2.0000e-09
5.0000e-09  9.5481e-01    -3.0000e-09
2.5000e-09  1.1689e+00    -5.0000e-10
1.2500e-09  5.0005e+00    7.5000e-10
1.8750e-09  1.0573e+00    1.2500e-10
1.5625e-09  8.9082e-01    4.3700e-10
1.4063e-09  5.0017e+00    5.9400e-10
*****
```

For the last two iterations, the last one succeeds, and the input value variation is $|1.4063\text{n}-1.5625\text{n}| = 0.0562\text{n}$, less than $\text{relin} * (\text{upper}-\text{lower}) = 0.1 * 5\text{n} = 0.5\text{n}$. Then the bisection succeeds and ends, and the setup time calculated is 0.594ns .

Example for Minimal Pulse Width With Passfail

```
.MEASURE tran prop_time trig v(wrclk) val='0.5*hi' Td=22.5n rise=1
+ targ v(wgbt) val='0.5*hi' Td=22.5n fall=1

.MODEL optmod opt method=passfail itropt=60 relin=0.0005

.PARAM Tdelay=Opt1(25p, '-0.5*10n', '0.5*10n')
.TRAN 2p 't2' Sweep Optimize = Opt1
+      Model      = 'optmod'
+      Result      = 'prop_time'
```

The PrimeSim tool reports the calculation result for each sweep, the information for the last two iterations:

```
*****
.....
SWEEP #11
bisec-opt iter = 10, tdelay = -2.83203124999999976845e-10, status:
  success
.....
SWEEP #12
bisec-opt iter = 11, tdelay = -2.88085937499999981794e-10, status: fail
*****
```

For the last two iterations, one passes, and the other one fails. The input value variation is $|0.2880\text{n}-0.2832\text{n}| = 4.89\text{e-}12$, less than $\text{relin} * (\text{upper}-\text{lower}) = 5\text{e-}12$.

Pushout Bisection

The pushout analysis is very similar to passfail methodology, the difference is that a maximum allowed pushout time is added in the analysis. In pushout bisection analysis, the first successful bisection result is considered as a standard value. For the following analysis, the result must meet two criteria, then it can be looked as a success. Based on the bisection methodology, the standard value must be taken from the upper or lower boundary, because the PrimeSim tool first analyzes these two input values, and one must result in success:

- Meet the criteria of passfail
- Meet the criteria of $|\text{meas_result} - \text{standard}| < \text{pushout}$

Example

```
.MEASURE tran out_rise when v(wgbt)='hi/2' Td=22.5n fail=1 pushout=2e-12

.MODEL optmod opt method=passfail itropt=itrnum relin=0.0005

.PARAM Tdelay=Opt1(25p, '-0.5*10n', '0.5*10n')

.TRAN 2p 't2' Sweep Optimize = Opt1
+           Model      = 'optmod'
+           Result     = 'out_rise'
```

The PrimeSim tool reports the calculation results for each sweep:

```
*****
out_rise= 2.3074e-08
bisec-opt upper boundary, tdelay = 5.000000000010461e-09, status:
  success
.....
SWEEP #11
out_rise= 2.3078e-08
bisec-opt iter = 10, tdelay = -2.83203124999976845e-10, status: fail
.....
SWEEP #12
out_rise= 2.3074e-08
bisec-opt iter = 11, tdelay = -2.78320312500003595e-10, status: success
*****
```

For sweep #11, the measure for `out_rise` can get value, but the status is still `fail`; this is because it does not meet the criteria for `pushout`.

In this example, the standard is `2.3074e-08`, which is taken from the upper boundary. The `pushout` is defined as `2e-12`.

For sweep #11, $|2.3078e-12 - 2.3074e-12| > 2e-12$, so the status is `fail`.

For sweep 12, the `out_rise` can also get a value, and it meets the `pushout` criteria, so the status is `success`.

The PrimeSim tool supports the keywords, `POSITIVE` and `NEGATIVE`, when performing pushout bisection analysis.

- Specify `POSITIVE` if you want the pushout constraints only to take effect when the measured results are larger than the first iteration's standard value.
- Specify `NEGATIVE` if you want the pushout constraints only to take effect when the measured results are smaller than the first iteration's standard value.

Bisection Output Convention

The standard file output convention for bisection analysis is as follows:

```
prefix#alter_temp_sweep.extension
```

With the same procedure as the transient output, the first `temp/alter` does not have `_t0` or `#0` in the naming convention. For data sweeps, the `mt0s` are lumped into a single file, unless `primesim_measout` is set to 1, in which case the `mt0` for each data sweep is separated into an individual file with `_sX` appended in the prefix.

In addition, the following options control the bisection output:

- `primesim_bisection_output`
- `primesim_bisection_summary`

Bisection Analysis With Two Measurements

The PrimeSim tool can support evaluating bisection criteria using multiple measure results. To use this function, a keyword `"result1"` must be added on `.TRAN` analysis.

Syntax

```
.TRAN t1 tstop Sweep Optimize=opl Model=ModName  
+ Result=MeasName1  
+ Result1=MeasName2
```

Concurrent Bisection for Independent Circuit Blocks

The PrimeSim tool can support performing concurrent bisection analyses that sweep multiple bisection parameters and results at the same time. However, it is up to the user to provide circuits where each set of bisection parameters/results is independent of each other. The number of results and parameters specified should be the same as the number of optimized parameters.

Syntax

```
.TRAN t1 tstop Sweep Model=ModName  
+ Optimize =opt1 opt2 ....  
+ Results = res1 res2 .....
```

11

PrimeSim SPICE Commands and Control Options

This chapter is a reference for commonly used PrimeSim SPICE-compatible commands and control options.

Note:

For more information on PrimeSim HSPICE-compatible commands, see [PrimeSim HSPICE-Compatible Commands](#) in this user guide and the PrimeSim HSPICE Simulation Command Reference chapter in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

This chapter covers the following topics:

- [PrimeSim SPICE Commands](#)
- [PrimeSim SPICE Control Options](#)

PrimeSim SPICE Commands

The PrimeSim SPICE tool supports the following commands:

- [.AC](#)
- [.ACMATCH](#)
- [.DC](#)
- [.DCMATCH](#)
- [.ENV](#)
- [.HB](#)
- [.HBAC](#)
- [.HBLIN](#)
- [.HBLSP](#)

- [.HBNOISE](#)
- [.HBOSC](#)
- [.HBSTB](#)
- [.HBXF](#)
- [.LIN](#)
- [.LSTB](#)
- [.NOISE](#)
- [.PHASENOISE](#)
- [.SENS](#)
- [.SN](#)
- [.SNAC](#)
- [.SNLIN](#)
- [.SNNOISE](#)
- [.SNOSC](#)
- [.SNSTB](#)
- [.SNXF](#)
- [.STATEYE](#)
- [.TRAN](#)
- [.XF](#)

.AC

Description

The `.AC` command is used in AC analysis to sweep source values. An AC analysis calculates the behavior of the circuit when AC voltage or current is applied.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

Usage #1

```
.AC [swept_param[@dev]] type np [[START=]start] [[STOP=]stop]
+ [[STEP=]step] FREQ=freq
+ [WRITE="write_file"] [WRITE_FINAL="write_final_file"]
+ [SWEEP [swept_param[@dev]] type np [[START=]start] [[STOP=]stop]
+ [[STEP=]step] FREQ=freq
```

Usage #2

```
.AC [swept_param[@dev]] [POI np val1 val2 ... valn]
+ [WRITE="write_file"] [WRITE_FINAL="write_final_file"]
+ [SWEEP [swept_param[@dev]] type np [[START=]start] [[STOP=]stop]
+ [[STEP=]step] FREQ=freq
```

Arguments

Argument	Description
swept_param[@dev]	Specifies the parameter to sweep of the (optionally) specified device.
type	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
number	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
start, stop, step	Starting and final points of a sweep and the interval of sweeping.
freq	Specifies the AC frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
"write_file"	Specifies the file to write out the IC.
"write_final_file"	Specifies the file to write out the final sweep IC.
SWEEP	Specifies a parameter sweep.

Examples

```
.AC DEC 5 10 100e6
.AC LIN 10 1 100e6
.AC LOG 5 10 100e6
.AC POI 3 1k 5k 100k
.AC hertz DEC 200 1 1G
.AC dc@v1 DEC 10 1 2
```

```
.AC DEC 200 1 1e+09 write="s1.ic" write_final="s5.ic"
+ SWEEP vdd LIN 5 1.0 2.0
```

.ACMATCH

Description

Use this command to run ACMatch analysis which is used to observe the effects of variations of all devices on the frequency response of a circuit at a specified node voltage or branch current.

The following [Monte Carlo-Specific Variation Block Options](#) affect ACMatch analysis:

- Option Ignore_Global_Variation=Yes|No
- Option Ignore_Local_Variation=Yes|No

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.ACMATCH outvar [THRESHOLD=threshold]
+ [WHERE=ASCII|BINARY] [FILE=filename]
+ [NSIGMA=nsigma] [GROUPBY=PARAM|INST]
+ type number start stop
```

Arguments

Argument	Description
<i>outvar</i>	Specifies the element to be probed.
<i>threshold</i>	A value specifying the relative contribution threshold to filter the output reporting.
WHERE	Specifies the output file format: • ASCII: Default • BINARY: Not currently supported
<i>filename</i>	Output report file name. Default is <i>output-prefix.acm</i> .
<i>nsigma</i>	A value specifying the parameter variation multiplier. Default is 3.
GROUPBY	Specifies how to group contributions in the report: • PARAM: Group by parameters • INST: Group by subcircuit instances

Argument	Description
<i>type</i>	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending points of a sweep.

Examples

```
.ACMATCH v(net1) where=ascii file=acmatch.out threshold=0.01 nsigma=2
+ groupby=param dec 10 1e+06 1e+08
```

.DC

Description

The **.DC** statement is used in DC analysis to sweep source values. DC analysis calculates the behavior of the circuit when DC voltage or current is applied. In most cases, this simulation analysis is performed first. The result of this analysis is commonly referred to as the DC bias or operating point characteristic. Currently, the PrimeSim SPICE tool supports the sweeping of the independent voltage or current source.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.DC var1 START=start1 STOP=stop1 STEP=step1 [hysteresis=0|1]
or
.DC var1 start1 stop1 step1 [hysteresis=0|1] [var2 start2 stop2 step2
+ [hysteresis=0|1]]
or
.DC var1 start1 stop1 step1 [hysteresis=0|1] [SWEEP var2 type np fstart
+ fstop [SWEEPTYPE=PARAM|SOURCE] [hysteresis=0|1]]
or
.DC var1 type np fstart fstop [hysteresis=0|1] [SWEEP data=dataname]
or
.DC data=dataname
or
.DC var1 type np fstart fstop [SWEEP MONTE=MCcommand] [hysteresis=0|1]
```


Arguments

Argument	Description
<i>var1</i>	Name of an independent source (voltage, current).
<i>start1, stop1, step1</i>	Starting, final and increment value of <i>var1</i> , respectively.
<i>hysteresis</i>	When <i>hysteresis</i> =1 is set, the PrimeSim tool automatically performs a hysteresis sweep. The voltage is swept up and down in the same analysis. The default, if the option is not specified, is <i>hysteresis</i> =0. Note: Hysteresis is not supported for <i>POI</i> type sweeps.
<i>var2</i>	Name of an independent source (voltage, current).
<i>start2, stop2, step2</i>	Starting, final and increment value of <i>var2</i> , respectively.
<i>type</i>	Sweeping type. One of the following: • <i>DEC</i>: Decade variation starts at <i>fstart</i>, stops at <i>fstop</i>, with total number of points <i>np</i>. • <i>LIN</i>: Linear variation starts at <i>fstart</i>, stops at <i>fstop</i>, with total number of points <i>np</i>. • <i>OCT</i>: Octave variation starts at <i>fstart</i>, stops at <i>fstop</i>, with total number of points <i>np</i>. • <i>POI</i>: List of <i>np</i> points with value <i>val1, val2, ...valn</i>.
<i>np</i>	Number of points per decade, per octave, or just number of points depending on the preceding keyword.
<i>fstart, fstop</i>	Starting and final points of a decade (<i>DEC</i>), linear (<i>LIN</i>), or octave (<i>OCT</i>) sweep.
<i>SWEEPTYPE</i>	Specifies whether to sweep a variable or a voltage current source, as follows: • <i>PARAM</i>: Sweep a defined variable. • <i>SOURCE</i>: Sweep a voltage or current source.
<i>val1, val2, ...valn</i>	List of points for a point (<i>POI</i>) sweep.
<i>dataname</i>	Name for data driven name (see <i>.DATA</i> description).

Examples

```
.DC Vds 0 5 0.1 Vgs 1 5 1
.DC Vds 0 5 0.1 SWEEP Vgs POI 5 1 2 3 4 5
.DC Vds 0 5 0.1 SWEEP Vgs LIN 5 1 5
.DC Vds LIN 51 0 5 SWEEP Vgs POI 5 1 2 3 4 5
.DC data=Vds_point SWEEP Vgs 1 5 1
.DC data=bias_point
```

```
.DC temp -40 80 10
.DC Vds 0 5 0.1 sweep MONTE=100
```

The first six examples are the same command with different syntax. These commands are usually used for MOSFET I-V characterization, which sweep drain voltage (V_{ds}) from 0V to 5V with an increment of 0.1V using V_{gs} as parameters with values of 1, 2, 3, 4, and 5V.

The seventh example sweeps simulation temperature from -40°C to 80°C with increments of 10°C .

Circuit Examples

```
**** DC analysis for MOSFET I-V ****
.option post=2
.model mn nmos level=3
.param vd_val=5.0
.param vg_val=5.0
vd d 0 vd_val
vg g 0 vg_val
vb b 0 0
.param wn=0.5u ln=0.5u
m1 d g 0 b mn w=wn l=ln
.DC vg 0 5 0.01
.DC vd 0 5 0.1 vg 1 5 1
.DC vd POI 3 3 4 5 SWEEP data=size
.DC temp -25 75 25
.DC data=all_condition
.data size
+index wn ln
+ 1 10u 10u
+ 2 10u .5u
+ 3 10u .4u
+ 4 .5u .5u
.enddata
.data all_condition
+ wn ln temp vd_value vg_value
+ 10u 0.4u 25 0.05 3.3
+ 0.5u 0.5u -25 5.0 5.0
+ 20u 20u 125 5.0 5.0
.enddata
.end
```

This example is a good demonstration for MOSFET characterization. The graphic output for simulation results are `xxx.sw0`, `xxx.sw1`, ..., `xxx.sw4` for the five .DC analysis statements. You can probe the current, capacitance, or other information of the MOSFET.

.DCMATCH

Description

Use this command to run DCMatch analysis which is used to observe the effects of variations of all devices on a specified node voltage or branch current.

The following [Monte Carlo-Specific Variation Block Options](#) affect DCMatch analysis:

- Option Ignore_Global_Variation=Yes|No
- Option Ignore_Local_Variation=Yes|No

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.DCMATCH outvar [THRESHOLD=threshold] [WHERE=ASCII|BINARY]  
+ [FILE=filename] [NSIGMA=nsigma]
```

Arguments

Argument	Description
<i>outvar</i>	Specifies the element to be probed.
<i>threshold</i>	A value specifying the relative contribution threshold to filter the output reporting.
WHERE	Specifies the output file format: • ASCII: Default • BINARY: Not currently supported
<i>filename</i>	Output report file name. Default is <i>output-prefix.dcm</i> .
<i>nsigma</i>	A value specifying the parameter variation multiplier. Default is 3.

Examples

```
.DCMATCH i(V3) threshold=0.001 nsigma=1 file=dcmatch.txt
```

.ENV

Description

Use this command to perform standard envelope simulation.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.ENV METHOD=HB|SN [ENV_START=env_start] ENV_STOP=env_stop
+ [ENV_STEP=env_step] [ENV_NSTEP=env_nstep] [ENV_DELMAX=env_delmax]
+ [ENV_ITERLIMIT=env_iterlimit]
+ TONES=tones NHARMS=nharms PROBENODE=probenode
+ [TRINIT=trinit] [TRINIT_METHOD=trinit_method]
+ [OVERSAMPLING=oversampling] [KRYLOVTOI=krylovtol]
+ [READ_IC="read_ic"] [READ_NS="read_ns"]
+ [WRITE="write"] [UIC]
+ [FREQUENCY_DIVIDER=frequency_divider]
```

Arguments

Argument	Description
METHOD	Specifies which engine to use: • HB: Harmonic balance • SN: Shooting Newton
env_start, env_stop, env_step	Envelope start and stop times for the simulation, and the time interval between any two envelope steps.
env_nstep	Number of cycles to skip between any two envelope steps.
env_delmax	Limits maximum time envelope step size.
env_iterlimit	Maximum number of Newton iterations per envelope step.
tones	Fundamental frequency.
nharms	Number of harmonics.
probenode	One or more nodes used to probe for oscillation conditions.
trinit	Initialization time.
trinit_method	Integration method for initial transient.
oversampling	Oversampling ratio.
krylovtol	Relative tolerance for GMRES solver.
"read_ic"	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
"read_ns"	Initialize nodal voltage from the specified file.

Argument	Description
<code>"write"</code>	Write initial conditions (IC) to the specified file.
<code>UIC</code>	Use initial conditions (IC) to initialize nodes (do not run DCOP).
<code>frequency_divider</code>	Divide ratio on large signal tone.

Examples

```
.env method=sn env_stop=100u tones=1G nharms=3
+ env_step=101.7255n

.env method=sn env_stop=100u env_start=1u tones=1G nharms=3
+ env_step=101.7255n

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
.env method=sn env_stop=1e-4 tones=1G nharms=3 env_nstep=1000
.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
+ env_delmax=1u

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
+ env_iterlimit=100

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_nstep=1000
+ trinit=100n

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
+ trinit_method=trap

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
+ read_ic="env.ic"

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
+ read_ns="env.ic"

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n
+ write="env.ic"

.env method=sn env_stop=1e-4 tones=1G nharms=3 env_step=101.7255n uic
```

.HB

Description

Use this command to invoke the single or multi-tone harmonic balance (HB) algorithm for periodic steady state analysis for a driven circuit.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HB TONES=tones FUNDS=fundname NHARMS=nharms
+ [TRINIT=trinit] [TRINIT_METHOD=GEAR|EGEAR|TRAP|TRAPONLY|BE]
+ [SAVE_TRINIT=YES|NO] [AUTOTRINIT=YES|NO]
+ [NUMBEROUT=numperout]
+ [KRYLOVTOL=krylovtol] [KRYLOVITER=kryloviter]
+ [OVERSAMPLING=oversampling] [HBMODE=TSTAB|TONE]
+ [READ_IC="read_ic"] [READ_NS="read_ns"]
+ [PARAM=param] [PARAM_DATA=param_data]
+ [UIC] [INTMODMAX=intmodmax] [DETECT_STEADY=YES|NO]
+ [AUTOHARMS] [FREQUENCY_DIVIDER=frequency_divider] [MAXITER=maxiter]
+ [LOADSTATE="loadstate"|"loadstate_init"|"loadstate_1tone"]
+ [SAVESTATE="savestate"|"savestate_init"|"savestate_1tone"]
+ [SWEEP parameter_sweep] [SWEETYPE=PARAM|SOURCE]
+ [COMPRESS=compress] [COMPRESS_GAIN=POWER|VOLTAGE]
+ [COMPRESS_REF=LINEAR|MAX] [COMPRESS_SOURCE=compress_source]
+ [COMPRESS_LOAD=compress_load] [COMPRESS_REFNODE=compress_refnode]
+ [COMPRESS_PNODE=compress_pnode] [COMPRESS_NNODE=compress_nnode]
+ [COMPRESS_HARM=compress_harm] [COMPRESS_MAXSTEPS=compress_maxsteps]
+ [COMPRESS_STOP=compress_stop] [COMPRESS_START=compress_start]
+ [COMPRESS_TOL=compress_tol]
```

Arguments

Argument	Description
<i>tones</i>	One or more fundamental frequencies or frequency names.
<i>fundname</i>	One or more fundamental frequency names.
<i>nharms</i>	Number of harmonics to use for the specified tone. Note: If <code>AUTOHARMS=YES</code> and the determined optimal number of harmonics differs from <i>nharms</i> , the tool uses whichever value is larger because the user-defined argument sets the lower limit.
<i>trinit</i>	Defines the length of time to run the initial transient to initialize state variables for harmonic balance analysis. Default is 5*T0 where T0 is the period of the fundamental frequency specified by <i>tones</i> . For multi-tone, T0 is determined from the first tone. Note: If <code>AUTOTRINIT=YES</code> , this argument is ignored.

Argument	Description
TRINIT_METHOD	Integration method for initial transient: - GEAR: Second order of Gear - EGEAR: Enhanced second order of Gear - TRAP: Trapezoidal - TRAPONLY: Trapezoidal only - BE: Backwards Euler
SAVE_TRINIT	Specifies whether to save the initial transient waveforms.
AUTOTRINIT	Specifies whether to allow the harmonic balance engine to determine how long to run initial transient. Note: When this argument is set to YES, the <i>trinit</i> argument is ignored and DETECT_STEADY is set to YES automatically.
<i>numperout</i>	Specifies the number of output periods of the signal that are captured in the HB time domain solution *td.hb or *.hr0 file. Default is 1.
<i>krylovtol</i>	Relative tolerance for GMRES solver. Default is 0.1 for single-tone; 0.4 for multi-tone.
<i>kryloviter</i>	Maximum number of GMRES iterations. Default is 100.
<i>oversampling</i>	Oversampling ratio for each tone.
HBMODE	Selects harmonic balance (HB) homotopy methods: - TSTAB: Runs transient analysis to make an initial guess for HB analysis. - TONE: Multi-tone circuit only. Solves a single tone, then uses the single-tone solution as the initial guess for the multi-tone.
" <i>read_ic</i> "	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
" <i>read_ns</i> "	Initialize nodes from the specified file.
<i>param</i>	Parameter name or <i>tol</i> scale, or <i>del</i> max to be updated dynamically during initial transient.
<i>param_data</i>	The time-value pairs for parameter <i>param</i> .
UIC	Use initial conditions (IC) to initialize nodes (do not run DCOP).
<i>intmodmax</i>	Maximum number of intermodulation.

Argument	Description
DETECT_STEADY	Specifies whether to run initial transient until reaching steady state, which can have the effect of running for a different length of time than specified by <i>trinit</i>. Note: If AUTOTRINIT=YES, this argument is set to YES automatically.
AUTOHARMS	Allows the harmonic balance engine to detect the optimal number of harmonics for best performance and accuracy. Note: This argument is honored only when either <i>trinit</i>>0 or AUTOTRINIT=YES. If AUTOHARMS=YES and the determined optimal number of harmonics differs from <i>nharms</i>, the tool uses whichever value is larger because the user-defined argument sets the lower limit.
<i>frequency_divider</i>	Divide ratio on large signal tone.
<i>maxiter</i>	Maximum number of harmonic balance Newton iterations. Default is 100.
"loadstate" "loadstate_init" "loadstate_1tone"	• If the <i>loadstate</i> file name is normal, loads the harmonic balance solution from the specified file. • If the <i>loadstate</i> file name ends with <i>_init</i>, skips initial transient analysis and loads the initial transient solution from the specified file. • If multi-tone HB analysis and the <i>loadstate</i> file name ends with <i>_1tone</i>, skips 1tone simulation and loads the 1tone simulation results from the specified file.
"savestate" "savestate_init" "savestate_1tone"	• If the <i>savestate</i> file name is normal, saves the harmonic balance solution to the specified file. • If the <i>savestate</i> file name ends with <i>_init</i>, saves the initial transient solution to the specified file. • If multi-tone HB analysis and the <i>savestate</i> file name ends with <i>_1tone</i>, writes the 1tone simulation result to the specified file.

Argument	Description
<i>parameter_sweep</i>	Parameter sweep. You can sweep up to three variables, as follows: <pre>var start stop incr</pre> where <i>var</i> is a parameter variable and <i>start stop incr</i> are the starting, final, and increment value of <i>var</i>, respectively. You can specify the sweep type as either LIN, DEC, OCT, or POI. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: - LIN <i>nsteps start stop</i> - OCT <i>nsteps start stop</i> - DEC <i>nsteps start stop</i> - POI <i>npoints p1, p2, ...pn</i> Where: <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). <i>start</i> = starting value. <i>stop</i> = ending value. <i>p1, p2, ... pn</i> = point values.
SWEETYPE	Specifies whether to sweep a variable or a voltage current source, as follows: - PARAM: Sweep a defined variable. - SOURCE: Sweep a voltage or current source.
<i>compress</i>	Enables compression analysis and specifies the compression level. Default is 0 which disables compression analysis, while a positive real number defines the compression level.
COMPRESS_GAIN	Specifies how gain is defined, as follows: - POWER: (default) Power gain $G \text{ (dBm)} = P_{load} \text{ (dBm)} - P_{source} \text{ (dBm)}$ - VOLTAGE: Voltage gain $G \text{ (dB)} = dB20(V_{load} / V_{source})$
COMPRESS_REF	Specifies the reference for gain compression calculation, as follows: - LINEAR: (default) Small signal gain is used as the reference. - MAX: Maximum gain is used as the reference.
<i>compress_source</i>	Specifies the instance name of the excitation source, depending on the setting of COMPRESS_GAIN, as follows: - POWER: Instance name must be a Port. - VOLTAGE: Instance name must be a Port or Vsource. Default is null.
<i>compress_load</i>	Specifies the instance name of the load element (port, resistor, or current probe) to calculate the power of load. Default is null. Note: This parameter is valid only when COMPRESS_GAIN=POWER.

Argument	Description
<i>compress_refnode</i>	Specifies the reference circuit node when <i>compress_load</i> is a current probe. Default is ground.
<i>compress_pnode</i>	Specifies the positive node to measure voltage, Vload. Default is null.
<i>compress_nnode</i>	Specifies the negative node to measure voltage, Vload. Default is ground.
<i>compress_harm</i>	Specifies the harmonic index, at which output voltage or power is measured. Default is null.
<i>compress_maxsteps</i>	Specifies the maximum sweep steps in sweeping to search the compression point. Default is 100.
<i>compress_stop</i>	Specifies the upper limit of power or voltage in the sweep. Default is: • Power: 30 dBm • Voltage: 2 V
<i>compress_start</i>	Specifies the starting value of power or voltage in the sweep. Default is: • Power: <i>compress_stop</i> - 70 dBm • Voltage <i>compress_stop</i> / 20000 V
<i>compress_tol</i>	Specifies the tolerance for compression point calculation and compression curve fitting. Default is 0.01.

Examples

```
.hb tones=flo frf nharms=12 3
.hb tones=1G nharms=10 trinit=200n
.hb tones=1G nharms=10 trinit=200n krylovtol=1e-9
.hb tones=1G nharms=10 trinit=200n oversampling=5
.hb tones=1G nharms=10 hbmode=tone
.hb tones=1G nharms=10 trinit=200n read_ic="ff.ic"
.hb tones=1G nharms=10 trinit=200n read_ns="ff.ic"
.hb tones=1G nharms=10 trinit=200n uic
.hb tones=1G nharms=10 trinit=200n detect_steady=yes
.hb tones=1G nharms=10 trinit=200n trinit_method=trap
.hb tones=1G nharms=10 trinit=200n save_trinit=yes
.hb tones=1G nharms=10 autotrinit=yes
.hb tones=1G nharms=10 trinit=200n maxiter=20
.hb tones=1G nharms=10 trinit=200n kryloviter=50
.hb tones=1G nharms=10 trinit=200n loadstate="hb.sav"
.hb tones=1G nharms=10 trinit=200n savestate="hb.sav"
.hb tones=flo frf1 frf2 nharms=15 7 7 oversampling=4 1 1 save_trinit=yes
+ autoharms=yes autotrinit=yes runlvl=6 SWEEP dbvin '-60' '-40' 5
.hb tones=1G,1.06G,1.05G nharms=10,5,5 oversampling=1,1,1 name=hb
+ sweep v2 dec '5' '0.001' '0.1' sweeptype=param
.hb tones=3.5G nharms=12 trinit=100n oversampling=2 \
```

```
compress=1 compress_gain=power compress_ref=linear compress_start=-60 \  
compress_source=PORT0 compress_load=PORT1 compress_harm=1
```

.HBAC

Description

Use this command to perform harmonic-balance–based periodic AC analysis on circuits operating in a large-signal periodic steady state.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBAC type number [[START=]start] [[STOP=]stop] [POI np val1 val2 ... valn]  
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms] [SAMPLED=YES|NO]  
+ [MAXSAMPLES=maxsamples]  
+ [SAMPLE_DEF=measname] [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]  
+ [PERTURBATION=perturbation] [LINEAR_FOUT=linear_fout]  
+ [NONLINEAR_FOUT=nonlinear_fout] [OUTPUT1=output1] [OUTPUT2=output2]  
+ [RF_INPUT1=rf_input1] [RF_INPUT2=rf_input2]  
+ [RF_INPUT_MAG=rf_input_mag] [RF_INPUT_PWR=rf_input_pwr]  
+ [TOLERANCE=tolerance] [KRYOVITER=kryoviter] [NAME=name]  
+ [lowmem = 0|1]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=measname has the following syntax:

```
.MEASURE HBTRAN measname SAMPLE_TYPE=WHEN signal SAMPLES=samples VAL=val  
+ RISE|FALL|CROSS=rfc_value
```

Arguments

Argument	Description
<i>type</i>	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.

Argument	Description
<code>POI</code>	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
<code>SWEEP_TYPE</code>	Specifies sweep frequency type: • ABSOLUTE • RELATIVE
<code>rel_harms</code>	Reference harmonic index when <code>SWEEP_TYPE=RELATIVE</code> .
<code>SAMPLED</code>	Specifies whether the evaluation is sampled at particular point in time.
<code>maxsamples</code>	Specifies the maximum number of sampled events.
<code>measname</code>	Specifies how the sample is defined in the corresponding <code>.MEASURE</code> statement. See the arguments: <i>signal</i> , <code>SAMPLE_TYPE</code> , <i>samples</i> , <i>val</i> , and <i>rfc_value</i> .
<code>sidebands</code>	Specifies the sideband index.
<code>maxsideband</code>	Specifies the maximum sideband index.
<code>perturbation</code>	To perform third-order intermodulation, set this argument to <code>ip3</code> or <code>ip2</code> .
<code>linear_fout</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the frequency of the linear signal.
<code>nonlinear_fout</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the frequency of the intermodulated signal.
<code>output1</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the first signal to be output.
<code>output2</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the second signal to be output.
<code>rf_input1</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the list of RF1 sources.
<code>rf_input2</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the list of RF2 sources.
<code>rf_input_mag</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the magnitude of RF input.
<code>rf_input_pwr</code>	In a modulated signal when <i>perturbation</i> is set as <code>ip3</code> or <code>ip2</code> , specifies the power in dBm of RF input.
<code>tolerance</code>	Specifies the tolerance for the linear solver in HBAC analysis.

Argument	Description
<i>kryoviter</i>	Specifies the maximum number of iterations of the linear solver used in HBAC analysis.
<i>name</i>	Specifies the name of the HBAC analysis, which affects the output name of the waveforms.
<i>SAMPLE_TYPE</i>	Specifies the sample type in the <code>.MEASURE</code> statement: • <code>WHEN signal</code>: Specifies the sampled probe by <i>inst</i> or <code>v(node1,node2)</code> where <i>inst</i> is the name of a two-terminal instance and <i>node1,node2</i> are net names.
<i>samples</i>	Specifies the sampled event at specified time(s).
<i>val</i>	Specifies the sampled voltage.
<i>rfc_value</i>	Specifies the sampled event is evaluated at the rising (<code>RISE=</code>), falling (<code>FALL=</code>), or crossing (<code>CROSS=</code>) edge.
<i>lowmem</i>	Enables a low-memory mode for HB small signal analysis. This mode reduces the peak memory usage during the analysis. • 0: Disables the mode (by default). • 1: Enables a low-memory mode. Note: Enabling this mode may negatively impact the performance or convergence of the analysis. It is only recommended when the memory requirements of the HB small signal analysis are prohibitively high.

Examples

```
.hbac dec 10 1k 100M
.hbac lin 100 1k 1M
.hbac log 100 1k 1M
.hbac poi 3 100k 500k 1M

.hbac lin 51 10000 20000 perturbation=ip3 linear_fout=110000
+ nonlinear_fout=100000 output1=vout rf_input1=P12 rf_input2=P12
+ rf_input_pwr=-40

.hbac lin 51 10000 20000 perturbation=ip3 linear_fout=110000
+ nonlinear_fout=100000 output2=vout rf_input1=P12 rf_input2=P12
+ rf_input_mag=1e-10

.hbac lin 100 1k 1M sampled=yes sample_def=pm0
.measure hbtran pm0 sample_type=when v(vip2,0) samples=2e-10,2.5e-10
+ val=1.2 fall=0
```

.HBLIN

Description

Use this command to extract frequency translation S-parameters and noise figures.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBLIN type number [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [NOISECALC=0|1] [NAME=name]
+ [PORTHARMSARRAY=portharmsarray] [MAXSIDE BAND=maxsideband]
+ [PORTS=ports] [KRYOVITER=kryoviter]
+ [lowmem = 0|1]
```

Arguments

Argument	Description
type	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
number	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
start, stop	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
SWEEP_TYPE	Specifies sweep frequency type: • ABSOLUTE • RELATIVE
NOISECALC	Specifies whether to perform analysis with noise.
name	Output file name for the extracted S-parameters.
portharmsarray	Specifies the port harmonic array.

Argument	Description
<i>maxsideband</i>	Specifies the maximum sideband (absolute) for <code>.HBLIN</code> with noise calculation. If unspecified, the default is the <i>nharms</i> setting from the Harmonic Balance command (<code>.HB</code>).
<i>ports</i>	List of ports. Note: You must specify at least two ports if <code>NOISECALC=1</code> . The first port is considered the input source; the second port is the output source.
<i>kryoviter</i>	Specifies the maximum number of iterations of the linear solver used in HBLIN analysis.
<i>lowmem</i>	Enables a low-memory mode for HB small signal analysis. This mode reduces the peak memory usage during the analysis. 0: Disables the mode (by default). 1: Enables a low-memory mode. Note: Enabling this mode may negatively impact the performance or convergence of the analysis. It is only recommended when the memory requirements of the HB small signal analysis are prohibitively high.

Output

The `.HBLIN` waveform includes the following signals if `NOISECALC=1` is specified:

Signal	Description
gain	Gain of circuit From the input port to the output port, which is the circuit actual gain
in	Input-Referred Noise $out / gain$
RN	Equivalent noise resistance
NNR	Normalized equivalent noise resistance
Gopt	Optimum noise conductance
Bopt	Optimum noise susceptance
Fmin	Minimum Noise Factor
NFmin	Minimum Noise Figure $10\log_{10}(F_{min})$

Signal	Description
F	Single-sideband Noise Factor (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the input harmonic)
NF	Single-sideband Noise Figure $10\log_{10}(F)$
F_{dsb}	Double-sideband Noise Factor (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the input harmonic and its image harmonic) - The numerator is the same as the single-sideband noise factor (F) - The denominator takes the noise contribution from the input source from its reference sideband and image sideband
NF_{dsb}	Double-sideband Noise Figure $10\log_{10}(F_{dsb})$
F_{ieee}	IEEE definition of Noise Factor Same as the single-sideband noise factor (F)
NF_{ieee}	IEEE definition of Noise Figure Same as the single-sideband noise figure (NF)
$F_{ieee3rdparty}$	Compatible to the F_{ieee} for 3rd-party tool (Total noise power at the output with all sidebands - Noise power from load at the output harmonic - Noise power at output due to the harmonics except the input harmonic at the source) / (Noise power due to source noise from the input harmonic) - The denominator is the same as the single-sideband noise factor (F) - It only includes the contribution due to source noise from the input harmonic in the numerator of the single-sideband noise factor
$NF_{ieee3rdparty}$	Compatible to the NF_{ieee} for 3rd-party tool $10\log_{10}(F_{ieee3rdparty})$

Examples

```
.hblin poi 1 1e+08 sweep_type=relative noisecalc=1 name=psp1
portharmsarray=0,0
+ maxsideband=10 ports=Prf Ploadm
```


.HBLSP

Description

Use this command to invoke periodically-driven nonlinear circuit analyses for power-dependent S-parameters.

A harmonic balance large-signal S-parameter (HBLSP) analysis provides the following analyses for periodically-driven nonlinear circuits, such as those that employ power amplifiers and filters:

- Two-port power-dependant (large-signal) S-parameter extraction
- Two-port small-signal S-parameter extraction
- Two-port small-signal noise parameter calculation

Unlike small-signal S-parameters, which are based on linear analysis, power-dependent S-parameters are based on the harmonic balance simulation. Its solution accounts for nonlinear effects such as compression and variation in the power levels.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBLSP TONES=tones NHARMS=nharms [TRINIT=trinit] [OUT_HARMS=out_harms]  
+ [PORTS=ports] [FILENAME=filename]  
+ [DATAFORMAT=RI|MA|DB] [FORMAT=TOUCHSTONE|TOUCHSTONE2|CITI|SELEM]
```

Arguments

Argument	Description
<i>tones</i>	One or more fundamental frequencies or frequency names.
<i>nharms</i>	Number of harmonics to use for the specified tone.
<i>trinit</i>	Run transient analysis to initialize harmonic balance analysis.
<i>out_harms</i>	Harmonic for each tone to be output for the <code>.HBLSP</code> statement.
<i>ports</i>	List of ports to consider for the <code>.HBLSP</code> statement.
<i>filename</i>	Specifies the file where the <code>.HBLSP</code> result is written.

Argument	Description
DATAFORMAT	S-parameter output data type: • RI: Real and imaginary • MA: Magnitude and phase • DB: Magnitude and phase in dB
FORMAT	S-parameter output file data type: • TOUCHSTONE: Creates a .s2p output file in Touchstone v1.0 format. • TOUCHSTONE2: Creates a .s2p output file in Touchstone v2.0 format. • CITI: Creates a CITIfile format output file. • SELEM: Creates an S-element .sc# format output file which you can include in the netlist.

Examples

```
.hblsp tones=1e+09 nharms=5 trinit=5e-09 out_harms=1 ports=PORT1 PORT2
+ filename=newlsp.sp dataformat=ma format=touchstone
```

.HBNOISE

Description

Use this command to perform cyclo-stationary noise analysis on circuits operating in a large-signal periodic steady state.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBNOISE output [iprobe_section] type number
+ [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn] [NAME=name]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms]
+ [SAMPLED=YES|NO] [SAMPLE_DEF=measname] [SAMPLE_RATIO=sample_ratio]
+ [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]
+ [KRYLOVITER=kryloviter] [REFERENCE_SIDEBAND=reference_sideband]
+ [NOISE_OUTPUT=USB|LSB|PM|AM] [SEPARATE_NOISE=YES|NO]
+ [OUTPUT_XF=YES|NO]
+ [lowmem = 0|1]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=measname and output in syntax should be also defined in .MEASURE, not following .hbnoise has the following syntax, depending on the SAMPLE_TYPE:

```
.MEASURE HB measname
+ SAMPLE_TYPE=CROSS
```

```
+ TRIG trig_signal [td=td] RISE|FALL|CROSS=rfc_value
+ TARG targ_signal

.MEASURE HB measname
+ SAMPLE_TYPE=DELAY
+ TRIG trig_signal VAL=val [td=td] RISE|FALL|CROSS=rfc_value
+ TARG targ_signal VAL=val RISE|FALL|CROSS=rfc_value

.MEASURE HB measname
+ SAMPLE_TYPE=TIMESAMPLED sample_signal
+ NUMBER=number [SAMPLES=samples] [PHASE=phase]
```

Arguments

Argument	Description
<i>output</i>	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. - If specified as the voltage of a node or node pair, the PrimeSim SPICE tool sums up the output noise on this node or between the pair of nodes. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. - If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool sums up the output noise on its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. - If specified as an element name of other types, the PrimeSim SPICE tool sums up the output noise between the first two terminals of this element. Note: This argument is ignored if <code>SAMPLED=YES</code>.
<i>iprobe_section</i>	Specifies the noise source as a reference to be evaluated. Valid values are: <code>vsource</code> <code>isource</code> <code>port</code>
<i>type</i>	Sweeping type. One of the following: DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.

Argument	Description
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
<i>name</i>	Specifies the name of the HBNOISE analysis, which affects the output name of the waveforms.
SWEEP_TYPE	Specifies sweep frequency type: • ABSOLUTE • RELATIVE
<i>rel_harms</i>	Sideband to which relative frequency sweep should be referenced.
SAMPLED	Specifies whether the noise evaluation is sampled at particular point in time.
<i>sample_ratio</i>	The ratio of the sampling frequency over the fundamental frequency. The default is 1. Usually used along with sampled noise for signals in the frequency divider whose frequency is an integer multiple of the fundamental frequency.
<i>measname</i>	Specifies how the sampled noise is defined in the corresponding .MEASURE statement. See the arguments: <i>SAMPLE_TYPE</i> , <i>trig_signal</i> , <i>td</i> , <i>rfc_value</i> , <i>targ_signal</i> , <i>val</i> , <i>sample_signal</i> , <i>number</i> , <i>samples</i> , and <i>phase</i> .
<i>sidebands</i>	Specifies the relative, absolute and upper/lower sideband frequency.
<i>maxsideband</i>	Specifies the maximum sideband frequency.
<i>kryloviter</i>	Maximum number of GMRES iterations. Default is 100.
<i>reference_sideband</i>	Used to compute the input-referred noise or noise figure.
NOISE_OUTPUT	Specified the noise type to be evaluated: • USB • LSB • PM • AM
SEPARATE_NOISE	Specifies whether to separate output noise contribution from any noise source into input noise and noise gain on every sideband. Default is NO.
OUTPUT_XF	Specifies whether to output the harmonic balance transfer function from independent voltage/current sources to nodes specified by harmonic balance noise analysis. Default is NO.

Argument	Description
<code>SAMPLE_TYPE</code>	Specifies the sample type in the <code>.MEASURE</code> statement: <code>CROSS</code> <code>DELAY</code> <code>TIMESAMPLED</code>
<code>trig_signal</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=CROSS DELAY</code> . - When <code>SAMPLE_TYPE=CROSS</code>, it specifies the signal where the cross event happens. - When <code>SAMPLE_TYPE=DELAY</code>, it specifies the signal where the trigger event happens.
<code>td</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=CROSS DELAY</code> to specify the time delay of the <code>CROSS</code> or <code>DELAY</code> measurements.
<code>rfc_value</code>	Specifies when the noise event is evaluated at the rising (<code>RISE=</code>), falling (<code>FALL=</code>), or crossing (<code>CROSS=</code>) edge.
<code>targ_signal</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=CROSS DELAY</code> . - When <code>SAMPLE_TYPE=CROSS</code>, it specifies the signal where the output noise is probed. - When <code>SAMPLE_TYPE=DELAY</code>, it specifies the signal where the target event happens.
<code>val</code>	Specifies noise sampled at the specified voltage.
<code>sample_signal</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the signal where the output noise is probed.
<code>number</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the number of samples which equally divide the whole period.
<code>samples</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the points of the additional samples.
<code>phase</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the phases of the additional samples.
<code>lowmem</code>	Enables a low-memory mode for HB small signal analysis. This mode reduces the peak memory usage during the analysis. 0: Disables the mode (by default). 1: Enables a low-memory mode. Note: Enabling this mode may negatively impact the performance or convergence of the analysis. It is only recommended when the memory requirements of the HB small signal analysis are prohibitively high.

Output

The `.HBNOISE` waveform includes the following signals:

Signal	Description
out	Total output noise
out_dbc	SSB jitter phase noise

If an input equivalent noise source is specified, the following signals are also generated:

Signal	Description
gain	Gain of circuit
in	Input-Referred Noise out / gain
F	Single-sideband Noise Factor (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the reference sideband)
NF	Single-sideband Noise Figure $10\log_{10}(F)$
Fdsb	Double-sideband Noise Factor (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the reference sideband and its image harmonic) - The numerator is the same as the single-sideband noise factor (F) - The denominator takes the noise contribution from the input source from its reference sideband and image sideband
NFdsb	Double-sideband Noise Figure $10\log_{10}(F_{dsb})$
Fieee	IEEE definition of Noise Factor Same as the single-sideband noise factor (F)
NFieee	IEEE definition of Noise Figure Same as the single-sideband noise figure (NF)

Signal	Description
Fieee3rdparty	Compatible to the Fieee for 3rd-party tool • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic - Noise power at output due to the harmonics except the reference sideband at the source) / (Noise power due to source noise from the reference sideband) • The denominator is the same as the single-sideband noise factor (F) • It only includes the contribution due to source noise from the reference sideband (input harmonic) in the numerator of the single-sideband noise factor
NFieee3rdparty	Compatible to the NFieee for 3rd-party tool • $10\log_{10}(F_{\text{ieee3rdparty}})$

Examples

```
.hbnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ reference_sideband=0

.hbnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ sidebands=-1, 1, 2

.hbnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ maxsideband=10

.hbnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ noise_output=lsb,pm

.hbnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure hb pm0 sample_type=cross trig v(vin,0) val=0.8 rise=1
+ targ v(von,vop)

.hbnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure hb pm0 sample_type=cross trig v(vin,0) val=0.8 rise=1
+ targ v(von,vop)

.hbnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure hb pm0 sample_type=delay trig v(RFOUT,gnd) val=1.13 td=0
+ rise=1 targ v(RFOUT,gnd) val=1.13 fall=1

.hbnoise log '10+1' 1000 1e+07 name=hbnoise0 maxsideband=40
+ sampled=yes sample_def=pm0
.measure hb pm0 sample_type=timesampled v(RFOUT,0) phase=135
+ samples=2e-10,2.5e-10 number=1

//P6 is the name of one port, V4 is the name for one vsource
.hbnoise P6 lin '100+1' 3e+09 4e+09 name=hbnoise
.hbnoise V4 lin '100+1' 3e+09 4e+09 name=hbnoise1
```

.HBOSC

Description

Use this command to invoke the single or multi-tone harmonic balance (HB) algorithm for periodic steady state analysis for an autonomous circuit.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBOSC TONES=tones NHARMS=nharms PROBENODE=probenode
+ [TRINIT=trinit] [TRINIT_METHOD=GEAR|EGEAR|TRAP|TRAPONLY|BE]
+ [SAVE_TRINIT=YES|NO] [AUTOTRINIT=YES|NO]
+ [NUMBEROUT=numberout]
+ [KRYLOVTOL=krylovtol] [KRYLOVITER=kryloviter] [PINNODE=pinnode]
+ [OVERSAMPLING=oversampling] [HBMODE=TSTAB|TONE]
+ [READ_IC="read_ic"] [READ_NS="read_ns"]
+ [PARAM=param] [PARAM_DATA=param_data]
+ [UIC] [INTMODMAX=intmodmax] [DETECT_STEADY=YES|NO]
+ [AUTOHARMS] [FREQUENCY_DIVIDER=frequency_divider] [MAXITER=maxiter]
+ [LOADSTATE="loadstate"|"loadstate_init"|"loadstate_1tone"]
+ [SAVESTATE="savestate"|"savestate_init"|"savestate_1tone"]
+ [SWEEP parameter_sweep] [SWEETYPE=PARAM|SOURCE]
```

Arguments

Argument	Description
<i>tones</i>	One or more fundamental frequencies or frequency names.
<i>nharms</i>	Number of harmonics to use for the specified tone. Note: If AUTOHARMS=YES and the determined optimal number of harmonics differs from <i>nharms</i> , the tool uses whichever value is larger because the user-defined argument sets the lower limit.
<i>probenode</i>	One or more nodes used to probe for oscillation conditions.
<i>trinit</i>	Defines the length of time to run the initial transient to initialize state variables for harmonic balance analysis. Default is 50*T0 where T0 is the period of the fundamental frequency specified by <i>tones</i> . For multi-tone, T0 is determined from the first tone. Note: If AUTOTRINIT=YES, this argument is ignored.

Argument	Description
TRINIT_METHOD	Integration method for initial transient: - GEAR: Second order of Gear - EGEAR: Enhanced second order of Gear - TRAP: Trapezoidal - TRAPONLY: Trapezoidal only - BE: Backwards Euler
SAVE_TRINIT	Specifies whether to save the initial transient waveforms.
AUTOTRINIT	Specifies whether to allow the harmonic balance engine to determine how long to run initial transient. Note: When this argument is set to YES, the <i>trinit</i> argument is ignored and DETECT_STEADY is set to YES automatically.
<i>numberout</i>	Specifies the number of output periods of the signal that are captured in the HB time domain solution *td.hb or *.hr0 file. Default is 1.
<i>krylovtol</i>	Relative tolerance for GMRES solver. Default is 0.1 for single-tone; 0.4 for multi-tone.
<i>kryloviter</i>	Maximum number of GMRES iterations. Default is 100.
<i>pinnode</i>	Circuit node to detect oscillation.
<i>oversampling</i>	Oversampling ratio for each tone.
HBMODE	Selects harmonic balance (HB) homotopy methods: - TSTAB: Runs transient analysis to make an initial guess for HB analysis. - TONE: Multi-tone circuit only. Solves a single tone, then uses the single-tone solution as the initial guess for the multi-tone.
"read_ic"	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
"read_ns"	Initialize nodes from the specified file.
<i>param</i>	Parameter name or <i>tol scale</i> to be updated dynamically during initial transient.
<i>param_data</i>	The time-value pairs for parameter <i>param</i> .
UIC	Use initial conditions (IC) to initialize nodes (do not run DCOP).
<i>intmodmax</i>	Maximum number of intermodulation.

Argument	Description
DETECT_STEADY	Specifies whether to run initial transient until reaching steady state, which can have the effect of running for a different length of time than specified by <i>trinit</i>. Note: If AUTOTRINIT=YES, this argument is set to YES automatically.
AUTOHARMS	Allows harmonic balance engine to detect the optimal number of harmonics for best performance and accuracy. Note: This argument is honored only when either <i>trinit</i>>0 or AUTOTRINIT=YES. If AUTOHARMS=YES and the determined optimal number of harmonics differs from <i>nharms</i>, the tool uses whichever value is larger because the user-defined argument sets the lower limit.
<i>frequency_divider</i>	Divide ratio on large signal tone.
<i>maxiter</i>	Maximum number of harmonic balance Newton iterations. Default is 100.
"loadstate" "loadstate_init" "loadstate_1tone"	• If the <i>loadstate</i> file name is normal, loads the harmonic balance solution from the specified file. • If the <i>loadstate</i> file name ends with <i>_init</i>, skips initial transient analysis and loads the initial transient solution from the specified file. • If multi-tone HBOSC analysis and the <i>loadstate</i> file name ends with <i>_1tone</i>, skips 1tone simulation and loads the 1tone simulation results from the specified file.
"savestate" "savestate_init" "savestate_1tone"	• If the <i>savestate</i> file name is normal, saves the harmonic balance solution to the specified file. • If the <i>savestate</i> file name ends with <i>_init</i>, saves the initial transient solution to the specified file. • If multi-tone HBOSC analysis and the <i>savestate</i> file name ends with <i>_1tone</i>, writes the 1tone simulation result to the specified file.

Argument	Description
<i>parameter_sweep</i>	Parameter sweep. You can sweep up to three variables, as follows: <pre>var start stop incr</pre> where <i>var</i> is a parameter variable and <i>start stop incr</i> are the starting, final, and increment value of <i>var</i>, respectively. You can specify the sweep type as either LIN, DEC, OCT, or POI. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: - LIN <i>nsteps start stop</i> - OCT <i>nsteps start stop</i> - DEC <i>nsteps start stop</i> - POI <i>npoints p1, p2, ...pn</i> Where: <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). <i>start</i> = starting value. <i>stop</i> = ending value. <i>p1, p2, ... pn</i> = point values.
SWEETYPE	Specifies whether to sweep a variable or a voltage current source, as follows: - PARAM: Sweep a defined variable. - SOURCE: Sweep a voltage or current source.

Examples

```
.hbosc tones=flo frf nharms=12 3
.hbosc tones=1G nharms=10 trinit=200n
.hbosc tones=1G nharms=10 trinit=200n krylovtol=1e-9
.hbosc tones=1G nharms=10 trinit=200n oversampling=5
.hbosc tones=1G nharms=10 hbmode=tone
.hbosc tones=1G nharms=10 trinit=200n read_ic="ff.ic"
.hbosc tones=1G nharms=10 trinit=200n read_ns="ff.ic"
.hbosc tones=1G nharms=10 trinit=200n uic
.hbosc tones=1G nharms=10 trinit=200n detect_steady=yes
.hbosc tones=1G nharms=10 trinit=200n trinit_method=trap
.hbosc tones=1G nharms=10 trinit=200n save_trinit=yes
.hbosc tones=1G nharms=10 autotrinit=yes
.hbosc tones=1G nharms=10 trinit=200n maxiter=20
.hbosc tones=1G nharms=10 trinit=200n kryloviter=50
.hbosc tones=1G nharms=10 trinit=200n loadstate="hb.sav"
.hbosc tones=1G nharms=10 trinit=200n savestate="hb.sav"

.hbosc tones=1e+07 nharms=15 probenode=emitter,0,1.0 intmodmax=5
```

.HBSTB

Description

Use this command to invoke harmonic balance (HB) stability analysis to analyze small-signal loop gain for circuits with negative feedback loops, in the presence of a large signal, such as an amplifier.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBSTB type number [[START=]start] [[STOP=]stop]
+ [MODE=SINGLE|DIFF}COMMON] [VSOURCE=vsource]
+ [NAME=name]
+ [lowmem = 0|1]
```

Arguments

Argument	Description
type	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
number	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
start, stop	Starting and ending frequency.
MODE	Specifies the mode: • SINGLE: Single-ended test. Single mode analysis is used to deal with feedback circuits with only one single signal path. • DIFF: Differential mode test. • COMM: Common mode test.
vsource	Specifies the voltage source.
name	Specifies the name of the HBSTB analysis, which affects the output name of the waveforms.

Argument	Description
<code>lowmem</code>	Enables a low-memory mode for HB small signal analysis. This mode reduces the peak memory usage during the analysis. • 0: Disables the mode (by default). • 1: Enables a low-memory mode. Note: Enabling this mode may negatively impact the performance or convergence of the analysis. It is only recommended when the memory requirements of the HB small signal analysis are prohibitively high.

Examples

```
.hbstb lin 2 100 100e6 mode=single vsorce=v1 name=stb1
```

.HBXF

Description

Use this command to calculate the transfer function from a given source in the circuit to a designated output in a circuit under periodic steady-state conditions.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.HBXF output type number [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn] [NAME=name]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms] [SAMPLED=YES|NO]
+ [MAXSAMPLES=maxsamples]
+ [SAMPLE_DEF=measname] [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]
+ [TOLERANCE=tolerance] [KRYOVITER=kryoviter]
+ [STIMULI=SOURCES|NODES_AND_TERMINALS]
+ [modulation=single| first|second| no]
+ [out_modharm=1]
+ [in_modharms=0]
+ [in_moduppersideband=0]
+ [lowmem = 0|1]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=measname has the following syntax:

```
.MEASURE HBTRAN measname SAMPLE_TYPE=WHEN signal SAMPLES=samples VAL=val
+ RISE|FALL|CROSS=rfc_value
```

Arguments

Argument	Description
<i>output</i>	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. - If specified as the voltage of a node or node pair, the PrimeSim SPICE tool calculates the transfer function to the voltage of this node or node pair. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. - If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool calculates the transfer function to its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. - If specified as an element name of other types, the PrimeSim SPICE tool calculates the transfer function to the voltage between the first two terminals of this element.
<i>type</i>	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
<i>name</i>	Specifies the name of the HBXF analysis, which affects the output name of the waveforms.
SWEEP_TYPE	Specifies sweep frequency type: - ABSOLUTE - RELATIVE
<i>rel_harms</i>	Reference harmonic index when SWEEP_TYPE=RELATIVE.
SAMPLED	Specifies whether the evaluation is sampled at particular point in time.
<i>maxsamples</i>	Specifies the maximum number of sampled events.

Argument	Description
<i>measname</i>	Specifies how the sample is defined in the corresponding <code>.MEASURE</code> statement. See the arguments: <i>signal</i> , <code>SAMPLE_TYPE</code> , <i>samples</i> , <i>val</i> , and <i>rfc_value</i> .
<i>sidebands</i>	Specifies the sideband index.
<i>maxsideband</i>	Specifies the maximum sideband index.
<i>tolerance</i>	Specifies the tolerance for the linear solver in HBXF analysis.
<i>kryoviter</i>	Specifies the maximum number of iterations of the linear solver used in HBXF analysis.
STIMULI	Specifies which signals are output: <code>SOURCES:</code> (default) Specifies signals from independent voltage/current sources. <code>NODES_AND_TERMINALS:</code> Specifies nodes or terminals.
<code>SAMPLE_TYPE</code>	Specifies the sample type in the <code>.MEASURE</code> statement: <code>WHEN signal:</code> Specifies the sampled probe by <i>inst</i> or <code>v(node1,node2)</code> where <i>inst</i> is the name of a two-terminal instance and <i>node1,node2</i> are net names.
<i>samples</i>	Specifies the sampled event at specified time(s).
<i>val</i>	Specifies the sampled voltage.
<i>rfc_value</i>	Specifies the sampled event is evaluated at the rising (<code>RISE=</code>), falling (<code>FALL=</code>), or crossing (<code>CROSS=</code>) edge.
<i>modulation</i>	Specifies whether compute transfer functions between modulated sources and outputs. The following are the modulation types: <i>single</i> <i>first second</i> <i>no</i> Default is <i>no</i> . <i>first second</i> should be specified as a pair with <code>SWEEP_TYPE=RELATIVE</code> .
<i>out_modharm</i>	Specifies the harmonic number of the output modulation, when <code>modulation=first second</code> . Default is <i>1</i> .
<i>in_modharms</i>	Specifies a list of harmonic numbers of the source modulation. Default is <i>0</i> .
<i>in_moduppersideband</i>	Specifies the upper sideband of source modulation when <code>modulation=single</code> . Default is <i>0</i> .

Argument	Description
lowmem	Enables a low-memory mode for HB small signal analysis. This mode reduces the peak memory usage during the analysis. • 0: Disables the mode (by default). • 1: Enables a low-memory mode. Note: Enabling this mode may negatively impact the performance or convergence of the analysis. It is only recommended when the memory requirements of the HB small signal analysis are prohibitively high.

Examples

```
.hbxf dec 10 1k 100M
.hbxf lin 100 1k 1M
.hbxf log 100 1k 1M
.hbxf poi 3 100k 500k 1M

.hbxf lin 100 1k 1M sampled=yes sample_def=pm0
.measure hbtran pm0 sample_type=when v(vip2,0) samples=2e-10,2.5e-10
+ val=1.2 fall=0

//P6 is the name of one port, V4 is the name for one vsource
.hb tones=3.5e+09 nharms=36 trinit=1e-07
.hbxf P6 name=hbxf lin '100+1' 3e+09 4e+09 sidebands=-2
.hbxf V4 name=hbxf1 lin '100+1' 3e+09 4e+09 sidebands=-2
```

.LIN

Description

Use this command to extract noise and linear transfer parameters for a general multi-port network.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.LIN type number [[START=]start] [[STOP=]stop]
+ [PORTS=port1 port2 ... portN] [FILENAME=filename]
+ [FORMAT=TOUCHSTONE|TOUCHSTONE2|CITI|SELEM] [DATAFORMAT=RI|MA|DB]
+ [NOISECALC=0|1] [IPOINT=iport] [OPOINT=oport]
+ [MIXEDMODE2PORT=mixedmode2port]
```


Arguments

Argument	Description
<i>type</i>	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points per log, starting at <i>fstart</i>, stopping at <i>fstop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade, per log, or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
<i>port1 port2 ... portN</i>	List of ports to consider for the .HBLSP statement.
<i>filename</i>	Defines the output file name.
FORMAT	S-parameter output file data type: - TOUCHSTONE: Creates a .s2p output file in Touchstone v1.0 format. - TOUCHSTONE2: Creates a .s2p output file in Touchstone v2.0 format. - CITI: Creates a CITI file format output file. - SELEM: Creates an S-element .sc# format output file which you can include in the netlist.
DATAFORMAT	S-parameter output data type: - RI: Real and imaginary - MA: Magnitude and phase - DB: Magnitude and phase in dB
NOISECALC	Specifies whether to perform analysis with noise.
<i>iport</i>	Specifies the input port to be used. This argument is an integer that represents the placement order of the ports as defined in the PORTS= argument. For example, if PORTS=P1 P2 and IPORT=2, then P2 is specified as the input port.
<i>oport</i>	Specifies the output port to be used. This argument is an integer that represents the placement order of the ports as defined in the PORTS= argument. For example, if PORTS=P1 P2 and OPORT=1, then P1 is specified as the output port.
<i>mixedmode2port</i>	Describes the mixed-mode data map of the output mixed mode S-parameter matrix.

Output

The `.LIN` waveform includes the following signals if `NOISECALC=1` is specified:

Signal	Description
out	Total output noise
gain	Gain of circuit
in	Input-Referred Noise • $out / gain$
RN	Equivalent noise resistance
NNR	Normalized equivalent noise resistance
Gopt	Optimum noise conductance
Bopt	Optimum noise susceptance
F	Noise Factor
NF	Noise Figure • $10\log_{10}(F)$
Fmin	Minimum Noise Factor
NFmin	Minimum Noise Figure • $10\log_{10}(F_{min})$

Examples

```
.lin dec 10 1k 1M ports=P1 P2 filename="o.sp" format=touchstone
+ dataformat=ri

.lin dec 10 1k 1M ports=P1 P2 filename="o.sp" format=touchstone
+ dataformat=ri noisecalc=1 oport=1 iport=2

.lin filename=s3.s4p format=touchstone dataformat=ri
+ mixedmode2port=m13s2s4 lin '100+1' 0 300000
```

.LSTB

Description

Use this command to invoke linear loop stability analysis.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.LSTB type number [[START=]start] [[STOP=]stop] [MODE=SINGLE|DIFF|COMMON]  
+ [VSOURCE=vlstb|vlstbp,vlstbn]
```

Arguments

Argument	Description
type	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
number	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
start, stop	Starting and ending frequency.
MODE	Specifies the mode: - SINGLE: Single-ended test. Single mode analysis is used to deal with feedback circuits with only one single signal path. - DIFF: Differential mode test. - COMM: Common mode test.
vlstb vlstbp,vlstbn	The 0V voltage sources indicating the insertion point of the test circuit: - vlstb: The only voltage source for a single-ended mode test. - vlstbp,vlstbn: Two voltage sources for a differential or common mode test.

Examples

```
.lstb lin 2 100 100e6 mode=single vsource=v1
```

.NOISE

Description

Use this command to perform noise analysis.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

Usage #1

```
.NOISE output_section [iprobe_section] type number
+ [[START=]start] [[STOP=]stop] [NAME=name]
+ [WRITE="write_file"] [WRITE_FINAL="write_final_file"]
+ [READ_IC="read_ic"] [READ_NS="read_ns"] [SEPARATE_NOISE=YES|NO]
+ [SWEEP swept_param[@dev] type number start stop] |
+ [SWEEP swept_param[@dev] start stop step] |
+ [SWEEP POI np val1 val2 ... valn]
```

Usage #2

```
.NOISE output_section [iprobe_section] POI np val1 val2 ... valn
+ [NAME=name]
+ [WRITE="write_file"] [WRITE_FINAL="write_final_file"]
+ [READ_IC="read_ic"] [READ_NS="read_ns"] [SEPARATE_NOISE=YES|NO]
+ [SWEEP swept_param[@dev] type number start stop] |
+ [SWEEP swept_param[@dev] start stop step] |
+ [SWEEP POI np val1 val2 ... valn]
```

Usage #3

```
.NOISE output_section [iprobe_section] POI=1 val1
+ [NAME=name]
+ [WRITE="write_file"] [WRITE_FINAL="write_final_file"]
+ [READ_IC="read_ic"] [READ_NS="read_ns"] [SEPARATE_NOISE=YES|NO]
+ [SWEEP swept_param[@dev] type number start stop] |
+ [SWEEP swept_param[@dev] start stop step] |
+ [SWEEP POI np val1 val2 ... valn]
```

Arguments

Argument	Description
<i>output_section</i>	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. - If specified as the voltage of a node or node pair, the PrimeSim SPICE tool sums up the output noise on this node or between the pair of nodes. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. - If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool sums up the output noise on its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. - If specified as an element name of other types, the PrimeSim SPICE tool sums up the output noise between the first two terminals of this element.
<i>iprobe_section</i>	Specifies the noise source as a reference to be evaluated.
<i>type</i>	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
<i>name</i>	Specifies the name of the NOISE analysis, which affects the output name of the waveforms.
"write_file"	Specifies the file to write out the IC.
"write_final_file"	Specifies the file to write out the final sweep IC.
"read_ic"	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
"read_ns"	Initialize nodes from the specified file.
SEPARATE_NOISE	Specifies whether to separate output noise contribution from any noise source into input noise and noise gain. Default is NO.
SWEEP	Specifies a parameter sweep.
<i>swept_param[@dev]</i>	Specifies the parameter to sweep of the (optionally) specified device.

Argument	Description
POI	List of np points of interest with values $val1$, $val2$, ..., $valn$, respectively.

Examples

```
.noise v(n1, n2) dec 10 1K 1x
.noise v(n1, n2) vin dec 10 1K 1x
.noise v(n1,n2) portn log 15 25K 750K
.noise v(n1,n2) poi 4 1K 10K 1x 100x
.noise v(n1,n2) poi=1 1K SWEEP spVar lin 3 1 2
.noise v(n1,n2) poi=1 1K SWEEP spVar poi 4 1 10 12 13
.noise v(n1, n2) lin 50 10 100x SWEEP spVar 1 2 0.25
.noise v(n1,n2) poi 2 100 1x SWEEP spVar 1 2 0.25
```

For Usages #1 and #2, if you add the `SWEEP` parameter, the output file number is swept multiple times in each file and the noise value is based on different frequencies. For example:

```
.noise v(3,0) poi 2 100 1x SWEEP spVar poi 4 1 10 12 13
```

In the preceding example, the output files are `noise0_s0.noise`, `noise0_s1.noise`, `noise0_s2.noise`, and `noise0_s3.noise` for different `spVar` values, and each output file contains noise values at frequency 100 and 1M.

For Usage #3, `poi=1` is specified for parameter sweeping for one frequency point. For example:

```
.noise v(3,0) poi=1 1K SWEEP spVar poi 4 1 10 12 13
```

In the preceding example, there is only one output file, `noise0.noise`, and it contains all noise values of all `spVar` sweeps.

The following example shows how an output probe is performed:

```
//P6 is the name of one port, V4 is the name for one vsource
.noise V4 lin '100+1' 3e+09 4e+09 name=noise
.noise P6 lin '100+1' 3e+09 4e+09 name=noise1
```

.PHASENOISE

Description

Use this command to run phase noise analysis which requires first running either the harmonic balance (`.HBOSC`) or Shooting Newton (`.SNOSC`) analysis, and then this analysis.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.PHASENOISE output [iprobe_section] type number
+ [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn] [NAME=name]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms]
+ [SAMPLED=YES|NO] [SAMPLE_DEF=measname] [SAMPLE_RATIO=sample_ratio]
+ [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]
+ [FULLSPECTRUM=YES|NO] [REFERENCE_SIDEBAND=reference_sideband]
+ [NOISE_OUTPUT=USB|LSB|PM|AM] [SEPARATE_NOISE=YES|NO]
+ [LORENTZ=0|1]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=measname and output in syntax should be also defined in .MEASURE, not following .phasenoise has the following syntax, depending on the SAMPLE_TYPE:

```
.MEASURE SN measname
+ SAMPLE_TYPE=CROSS
+ TRIG trig_signal [td=td] RISE|FALL|CROSS=rfc_value
+ TARG targ_signal

.MEASURE SN measname
+ SAMPLE_TYPE=DELAY
+ TRIG trig_signal VAL=val [td=td] RISE|FALL|CROSS=rfc_value
+ TARG targ_signal VAL=val RISE|FALL|CROSS=rfc_value

.MEASURE SN measname
+ SAMPLE_TYPE=TIMESAMPLED sample_signal
+ NUMBER=number [SAMPLES=samples] [PHASE=phase]
```

Arguments

Argument	Description
<i>output</i>	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. - If specified as the voltage of a node or node pair, the PrimeSim SPICE tool sums up the output noise on this node or between the pair of nodes. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. - If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool sums up the output noise on its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. - If specified as an element name of other types, the PrimeSim SPICE tool sums up the output noise between the first two terminals of this element. Note: This argument is ignored if <code>SAMPLED=YES</code>.
<i>iprobe_section</i>	Specifies the noise source as a reference to be evaluated. Valid values are: <code>vsource</code> <code>isource</code> <code>port</code>
<i>type</i>	Sweeping type. One of the following: DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i>, <i>val2</i>, ..., <i>valn</i>, respectively.
<i>name</i>	Specifies the name of the PHASENOISE analysis, which affects the output name of the waveforms.
<code>SWEEP_TYPE</code>	Specifies sweep frequency type: <code>ABSOLUTE</code> <code>RELATIVE</code> (default)

Argument	Description
<i>rel_harms</i>	Number of harmonic references to <code>SWEEP_TYPE</code> . Default is 1. Note: This argument can be aliased to the PrimeSim HSPICE argument, <code>carrierindex</code> .
<code>SAMPLED</code>	Specifies whether the noise evaluation is sampled at particular point in time.
<i>sample_ratio</i>	The ratio of the sampling frequency over the fundamental frequency. The default is 1. Usually used along with sampled noise for signals in the frequency divider whose frequency is an integer multiple of the fundamental frequency.
<i>measname</i>	Specifies how the sample noise is defined in the corresponding <code>.MEASURE</code> statement. See the arguments: <code>SAMPLE_TYPE</code> , <i>trig_signal</i> , <i>td</i> , <i>rfc_value</i> , <i>targ_signal</i> , <i>val</i> , <i>sample_signal</i> , <i>number</i> , <i>samples</i> , and <i>phase</i> .
<i>sidebands</i>	Specifies the relative, absolute and upper/lower sideband frequency.
<i>maxsideband</i>	Specifies the maximum sideband frequency.
<i>iprobe_section</i>	Specifies the noise source as a reference to be evaluated.
<code>FULLSPECTRUM</code>	Specifies whether to use full noise spectrum analysis.
<i>reference_sideband</i>	Used to compute the input-referred noise or noise figure.
<code>NOISE_OUTPUT</code>	Specified the noise type to be evaluated: • USB • LSB • PM • AM
<code>SEPARATE_NOISE</code>	Specifies whether to separate output noise contribution from any noise source into input noise and noise gain on every sideband. Default is <code>NO</code> .
<code>LORENTZ</code>	Specifies whether to calculate and output the Lorentzian corner frequency and apply the Lorentzian model on phase noise at a low frequency offset, as follows: • 0: Neither the Lorentzian corner frequency nor the Lorentzian model are calculated/applied. • 1: The Lorentzian corner frequency is calculated and output, and the Lorentzian model is applied on the phase noise result at a low frequency offset. • Unspecified: The Lorentzian corner frequency is calculated and output, but the Lorentzian model is not applied on the phase noise result.

Argument	Description
<code>SAMPLE_TYPE</code>	Specifies the sample type in the <code>.MEASURE</code> statement: <code>CROSS</code> <code>DELAY</code> <code>TIMESAMPLED</code>
<code>trig_signal</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=CROSS DELAY</code> . - When <code>SAMPLE_TYPE=CROSS</code>, it specifies the signal where the cross event happens. - When <code>SAMPLE_TYPE=DELAY</code>, it specifies the signal where the trigger event happens.
<code>td</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=CROSS DELAY</code> to specify the time delay of the <code>CROSS</code> or <code>DELAY</code> measurements.
<code>rfc_value</code>	Specifies when the noise event is evaluated at the rising (<code>RISE=</code>), falling (<code>FALL=</code>), or crossing (<code>CROSS=</code>) edge.
<code>targ_signal</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=CROSS DELAY</code> . - When <code>SAMPLE_TYPE=CROSS</code>, it specifies the signal where the output noise is probed. - When <code>SAMPLE_TYPE=DELAY</code>, it specifies the signal where the target event happens.
<code>val</code>	Specifies noise sampled at the specified voltage.
<code>sample_signal</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the signal where the output noise is probed.
<code>number</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the number of samples which equally divide the whole period.
<code>samples</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the points of the additional samples.
<code>phase</code>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the phases of the additional samples.

Output

The `.PHASENOISE` waveform includes the following signals:

Signal	Description
<code>out</code>	Total output noise
<code>out_dbc</code>	SSB jitter phase noise or PM phase noise

If an input equivalent noise source is specified, the following signals are also generated:

Signal	Description
gain	Gain of circuit
in	Input-Referred Noise • $out / gain$
F	Single-sideband Noise Factor • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the reference sideband)
NF	Single-sideband Noise Figure • $10\log_{10}(F)$
Fdsb	Double-sideband Noise Factor • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the reference sideband) • The numerator is the same as the single-sideband noise factor (F) • The denominator takes the noise contribution from the input source from its reference sideband and image sideband
NFdsb	Double-sideband Noise Figure • $10\log_{10}(F_{dsb})$
Fieee	IEEE definition of Noise Factor • Same as the single-sideband noise factor (F)
NFieee	IEEE definition of Noise Figure • Same as the single-sideband noise figure (NF)
Fieee3rdparty	Compatible to the Fieee for 3rd-party tool • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic - Noise power at output due to the harmonics except the reference sideband at the source) / (Noise power due to source noise from the reference sideband) • The denominator is the same as the single-sideband noise factor (F) • It only includes the contribution due to source noise from the reference sideband (input harmonic) in the numerator of the single-sideband noise factor
NFieee3rdparty	Compatible to the NFieee for 3rd-party tool • $10\log_{10}(F_{ieee3rdparty})$

Examples

```
.phasenoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ fullspectrum=yes
```

```
.phasenoi se v(out,0) vin lin 100 1k 1M rel_harms=1 sweep_type=relative
+ reference_sideband=0

.phasenoi se v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ sidebands=-1, 1, 2

.phasenoi se v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ maxsideband=10

.phasenoi se v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ noise_output=lsb,pm

.phasenoi se lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=cross trig v(vin,0) val=0.8 rise=1
+ targ v(von,vop)

.phasenoi se lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=cross trig v(vin,0) val=0.8 rise=1
+ targ v(von,vop)

.phasenoi se lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=timesampled v(vin,0) samples=0.2p,0.5p,1.2p
number=5

.phasenoi se lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=timesampled v(vin,0) phase=30,50,160

.phasenoi se lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=delay trig v(vin,0) val=1.2 rise=1 targ
v(vip2,0) val=1.2 fall=0
```

.SENS

Description

Use this command to perform DC and AC sensitivity analysis as follows:

- If *design_parameters_list* is not specified, then the sensitivities of all instance parameters of all devices in the design are computed.
- If only instance names are listed in *design_parameters_list*, then the sensitivities of all instance parameters of listed devices are computed.
- If only model names are listed in *design_parameters_list*, then the sensitivities of all model parameters of listed models are computed.
- If voltage source is set as the response in `sens()`, then it means the sensitivity of the current through the voltage source.

For MOSFETs, this command supports only DC sensitivity analysis of BSIM4, BSIM-CMG, FDSOI, and PSP.

For the response, this command supports only node voltage or current through voltage source in the *output_variables_list*.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SENS [output_variables_list] [design_parameters_list]  
+ analist=analyses_list
```

Arguments

Argument	Description
<i>output_variables_list</i>	Specifies the outputs for which sensitivity is computed with respect to parameter variations in <i>design_parameters_list</i> .
<i>design_parameters_list</i>	Specifies the variable design parameters.
<i>analyses_list</i>	Analyses list specified in the analysis tag.

Examples

```
.sens V(2 Out) param= m1 m2 analist=analDC
```

For this command, the PrimeSim SPICE tool computes the DC sensitivities of nodes 2 and Out with respect to the m1 and m2 parameters for the DC analysis analDC. The results are stored in the raw directory with the name in the format *analysisname.sens.analysistype*, that is, analDC.sens.dc.

.SN

Description

Use this command to perform Shooting Newton (SN) analysis for a driven circuit.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SN PERIOD=period TONE=tone NHARMS=nharms
+ [TRINIT=trinit] [TRINIT_METHOD=GEAR|EGEAR|TRAP|TRAPONLY|BE]
+ [SAVE_TRINIT=YES|NO] [AUTOTRINIT=YES|NO] [KRYLOVITER=kryloviter]
+ [READ_IC="read_ic"] [READ_NS="read_ns"]
+ [WRITE="write"]
+ [PARAM=param] [PARAM_DATA=param_data]
+ [UIC] [DETECT_STEADY=YES|NO]
+ [STEADYRATIO=steadyratio] [MAXITER=maxiter]
+ [LOADSTATE="loadstate"|"loadstate_init"]
+ [SAVESTATE="savestate"|"savestate_init"]
+ [SWEEP parameter_sweep] [SWEEPTYPE=PARAM|SOURCE]
```

Arguments

Argument	Description
<i>period</i>	Expected period of the steady state waveforms. The period of the steady state waveform may be entered either as <i>period</i> or its reciprocal, <i>tone</i> .
<i>tone</i>	The fundamental frequency.
<i>nharms</i>	Number of harmonics to use for the specified tone.
<i>trinit</i>	Run transient analysis to initialize Shooting Newton analysis.
TRINIT_METHOD	Integration method for initial transient: - GEAR: Second order of Gear - EGEAR: Enhanced second order of Gear - TRAP: Trapezoidal - TRAPONLY: Trapezoidal only - BE: Backwards Euler
SAVE_TRINIT	Specifies whether to save the initial transient waveforms.
AUTOTRINIT	Specifies whether to allow the Shooting Newton engine to determine how long to run initial transient.
<i>kryloviter</i>	Maximum number of GMRES iterations. Default is 100.
" <i>read_ic</i> "	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
" <i>read_ns</i> "	Initialize nodes from the specified file.
" <i>write</i> "	Write operating point to the specified file.
<i>param</i>	Parameter name or <i>tolyscale</i> to be updated dynamically during initial transient

Argument	Description
<i>param_data</i>	The time-value pairs for parameter <i>param</i> .
UIC	Use initial conditions (IC) to initialize nodes (do not run DCOP).
DETECT_STEADY	Specifies whether the Initial transient can be terminated early after steady-state is detected.
<i>steadyratio</i>	Specifies the ratio of steady state tolerance.
<i>maxiter</i>	Maximum number of Shooting Newton iterations. Default is 100.
"loadstate"	- If the <i>loadstate</i> file name is normal, loads the Shooting Newton solution from the specified file. - If the <i>loadstate</i> file name ends with <i>_init</i>, skips initial transient analysis and loads the initial transient solution from the specified file.
"savestate"	- If the <i>savestate</i> file name is normal, saves the Shooting Newton solution to the specified file. - If the <i>savestate</i> file name ends with <i>_init</i>, saves the initial transient solution to the specified file.
<i>parameter_sweep</i>	Parameter sweep. You can sweep up to three variables, as follows: <pre>var start stop incr</pre> where <i>var</i> is a parameter variable and <i>start stop incr</i> are the starting, final, and increment value of <i>var</i>, respectively. You can specify the sweep type as either LIN, DEC, OCT, or POI. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: - LIN <i>nsteps start stop</i> - OCT <i>nsteps start stop</i> - DEC <i>nsteps start stop</i> - POI <i>npoints p1, p2, ...pn</i> Where: <i>nsteps</i> = number of steps per decade (DEC) or octave (OCT). <i>npoints</i> = number of points for linear (LIN) or points of interest (POI). <i>start</i> = starting value. <i>stop</i> = ending value. <i>p1, p2, ... pn</i> = point values.
SWEEPTYPE	Specifies whether to sweep a variable or a voltage current source, as follows: - PARAM: Sweep a defined variable. - SOURCE: Sweep a voltage or current source.

Examples

```
.sn period=1n nharms=10 trinit=200n
.sn tone=1G nharms=10 trinit=200n
.sn tone=1G nharms=10 trinit=200n read_ic="sn.ic"
```

```
.sn tone=1G nharms=10 trinit=200n read_ns="sn.ic"
.sn tone=1G nharms=10 trinit=200n write="sn.ic"
.sn tone=1G nharms=10 trinit=200n uic
.sn tone=1G nharms=10 trinit=200n save_trinit=yes
.sn tone=1G nharms=10 trinit=200n trinit_method=trap
.sn tone=1G nharms=10 autotrinit=yes
.sn tone=1G nharms=10 autotrinit=yes detect_steady=yes
.sn tone=1G nharms=10 trinit=200n savestate="sn.sav"
.sn tone=1G nharms=10 trinit=200n loadstate="sn.sav"
```

.SNAC

Description

Use this command to run a frequency sweep across a range for the input signal based on a Shooting Newton algorithm.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SNAC type number [[START=start] [[STOP=stop] [POI np val1 val2 ... valn]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms] [SAMPLED=YES|NO]
+ [MAXSAMPLES=maxsamples]
+ [SAMPLE_DEF=measname] [SIDEBANDS=sidebands]
+ [MAXSIDEBAND=maxsideband]
+ [TOLERANCE=tolerance] [KRYOVITER=kryoviter] [NAME=name]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=*measname* has the following syntax:

```
.MEASURE SN measname SAMPLE_TYPE=WHEN signal SAMPLES=samples VAL=val
+ RISE|FALL|CROSS=rfc_value
```

Arguments

Argument	Description
<i>type</i>	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.

Argument	Description
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1, val2, ..., valn</i> , respectively.
SWEEP_TYPE	Specifies sweep frequency type: • ABSOLUTE • RELATIVE
<i>rel_harms</i>	Reference harmonic index when SWEEP_TYPE=RELATIVE.
SAMPLED	Specifies whether the evaluation is sampled at particular point in time.
<i>maxsamples</i>	Specifies the maximum number of sampled events.
<i>measname</i>	Specifies how the sample is defined in the corresponding .MEASURE statement. See the arguments: <i>signal</i> , SAMPLE_TYPE, <i>samples</i> , <i>val</i> , and <i>rfc_value</i> .
<i>sidebands</i>	Specifies the sideband index.
<i>maxsideband</i>	Specifies the maximum sideband index.
<i>tolerance</i>	Specifies the tolerance for the linear solver in SNAC analysis.
<i>kryoviter</i>	Specifies the maximum number of iterations of the linear solver used in SNAC analysis.
<i>name</i>	Specifies the name of the SNAC analysis, which affects the output name of the waveforms.
SAMPLE_TYPE	Specifies the sample type in the .MEASURE statement: • WHEN <i>signal</i>: Specifies the sampled probe by <i>inst</i> or <i>v(node1,node2)</i> where <i>inst</i> is the name of a two-terminal instance and <i>node1,node2</i> are net names.
<i>samples</i>	Specifies the sampled event at specified time(s).
<i>val</i>	Specifies the sampled voltage.
<i>rfc_value</i>	Specifies the sampled event is evaluated at the rising (RISE=), falling (FALL=), or crossing (CROSS=) edge.

Examples

```
.snac dec 10 1k 100M
.snac lin 100 1k 1M
.snac log 100 1k 1M
.snac poi 3 100k 500k 1M
```

```
.snac lin 100 1k 1M rel_harms=1 sweep_type=relative
.snac lin 100 1k 1M sidebands=1,2,-2
.snac lin 100 1k 1M maxsideband=10

.snac lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=when v(vip2,0) samples=2e-10,2.5e-10
+ val=1.2 fall=0
```

.SNLIN

Description

Use this command to extract frequency translation S-parameters and noise figures.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SNLIN type number [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [NOISECALC=0|1] [NAME=name]
+ [PORTHARMSARRAY=portharmsarray] [MAXSIDEBAND=maxsideband]
+ [PORTS=ports] [KRYOVITER=kryoviter]
```

Arguments

Argument	Description
type	Sweeping type. One of the following: - DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. - LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. - LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
number	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
start, stop	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
SWEEP_TYPE	Specifies sweep frequency type: - ABSOLUTE - RELATIVE
NOISECALC	Specifies whether to perform analysis with noise.

Argument	Description
<i>name</i>	Output file name for the extracted S-parameters.
<i>portharmsarray</i>	Specifies the port harmonic array.
<i>maxsideband</i>	Specifies the maximum sideband (absolute) for <code>.SNLIN</code> with noise calculation. If unspecified, the default is 7.
<i>ports</i>	List of ports. Note: You must specify at least two ports if <code>NOISECALC=1</code> . The first port is considered the input source; the second port is the output source.
<i>kryoviter</i>	Specifies the maximum number of iterations of the linear solver used in SNLIN analysis.

Output

The `.SNLIN` waveform includes the following signals if `NOISECALC=1` is specified:

Signal	Description
gain	Gain of circuit • From the input port to the output port, which is the circuit actual gain
in	Input-Referred Noise • $out / gain$
RN	Equivalent noise resistance
NNR	Normalized equivalent noise resistance
Gopt	Optimum noise conductance
Bopt	Optimum noise susceptance
Fmin	Minimum Noise Factor
NFmin	Minimum Noise Figure • $10\log_{10}(F_{min})$
F	Single-sideband Noise Factor • $(\text{Total noise power at the output with all sidebands} - \text{Noise power from load at the output harmonic}) / (\text{Noise power due to input source noise from the input harmonic})$
NF	Single-sideband Noise Figure • $10\log_{10}(F)$

Signal	Description
Fdsb	Double-sideband Noise Factor • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the input harmonic and its image harmonic) • The numerator is the same as the single-sideband noise factor (F) • The denominator takes the noise contribution from the input source from its reference sideband and image sideband
NFdsb	Double-sideband Noise Figure • $10\log_{10}(F_{dsb})$
Fieee	IEEE definition of Noise Factor • Same as the single-sideband noise factor (F)
NFieee	IEEE definition of Noise Figure • Same as the single-sideband noise figure (NF)
Fieee3rdparty	Compatible to the Fieee for 3rd-party tool • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic - Noise power at output due to the harmonics except the input harmonic at the source) / (Noise power due to source noise from the input harmonic) • The denominator is the same as the single-sideband noise factor (F) • It only includes the contribution due to source noise from the input harmonic in the numerator of the single-sideband noise factor
NFieee3rdparty	Compatible to the NFieee for 3rd-party tool • $10\log_{10}(F_{ieee3rdparty})$

Examples

```
.snlin poi 1 1e+08 sweep_type=relative noisecalc=1 name=psp1
portharmsarray=0,0
+ ports=Prf Ploadm
```

.SNNOISE

Description

Use this command to run a periodic, time-varying AC noise analysis based on a Shooting Newton algorithm.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SNNOISE output [iprobe_section] type number
+ [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn] [NAME=name]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms]
+ [SAMPLED=YES|NO] [SAMPLE_DEF=measname] [SAMPLE_RATIO=sample_ratio]
+ [SIDEBANDS=sidebands] [MAXSIDEBAND=maxsideband]
+ [FULLSPECTRUM=YES|NO] [REFERENCE_SIDEBAND=reference_sideband]
+ [NOISE_OUTPUT=USB|LSB|PM|AM] [SEPARATE_NOISE=YES|NO]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=measname and output in syntax should be also defined in .MEASURE, not following .snnoise has the following syntax, depending on the SAMPLE_TYPE:

```
.MEASURE SN measname
+ SAMPLE_TYPE=CROSS
+ TRIG trig_signal [td=td] RISE|FALL|CROSS=rfc_value
+ TARG targ_signal

.MEASURE SN measname
+ SAMPLE_TYPE=DELAY
+ TRIG trig_signal VAL=val [td=td] RISE|FALL|CROSS=rfc_value
+ TARG targ_signal VAL=val RISE|FALL|CROSS=rfc_value

.MEASURE SN measname
+ SAMPLE_TYPE=TIMESAMPLED sample_signal
+ NUMBER=number [SAMPLES=samples] [PHASE=phase]
```

Arguments

Argument	Description
output	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. • If specified as the voltage of a node or node pair, the PrimeSim SPICE tool sums up the output noise on this node or between the pair of nodes. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. • If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool sums up the output noise on its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. • If specified as an element name of other types, the PrimeSim SPICE tool sums up the output noise between the first two terminals of this element.

Note:
This argument is ignored if SAMPLED=YES.

Argument	Description
<i>iprobe_section</i>	Specifies the noise source as a reference to be evaluated. Valid values are: • <i>vsource</i> • <i>isource</i> • <i>port</i>
<i>type</i>	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1</i> , <i>val2</i> , ..., <i>valn</i> , respectively.
<i>name</i>	Specifies the name of the SNNOISE analysis, which affects the output name of the waveforms.
SWEEP_TYPE	Specifies sweep frequency type: • ABSOLUTE (default) • RELATIVE
<i>rel_harms</i>	Number of harmonic references to SWEEP_TYPE.
SAMPLED	Specifies whether the noise evaluation is sampled at particular point in time.
<i>sample_ratio</i>	The ratio of the sampling frequency over the fundamental frequency. The default is 1. Usually used along with sampled noise for signals in the frequency divider whose frequency is an integer multiple of the fundamental frequency.
<i>measname</i>	Specifies how the sample noise is defined in the corresponding .MEASURE statement. See the arguments: <i>SAMPLE_TYPE</i> , <i>trig_signal</i> , <i>td</i> , <i>rft_value</i> , <i>targ_signal</i> , <i>val</i> , <i>sample_signal</i> , <i>number</i> , <i>samples</i> , and <i>phase</i> .
<i>sidebands</i>	A vector which specifies all the sidebands (relative to <i>rel_harms</i>) selected for Shooting Newton noise calculation. If specified, <i>maxsideband</i> is ignored. Note that sideband 0 is always included in noise calculation so it need not be specified in the vector.

Argument	Description
<i>maxsideband</i>	Specifies the maximum sideband (relative to <i>rel_harms</i>) for Shooting Newton noise calculation. If unspecified, the default is 7.
FULLSPECTRUM	Specifies whether to include all sidebands for Shooting Newton noise calculation. If specified, either <i>maxsideband</i> or <i>sidebands</i> is ignored.
<i>reference_sideband</i>	Used to compute the input-referred noise or noise figure.
NOISE_OUTPUT	Specified the noise type to be evaluated: • USB • LSB • PM • AM
SEPARATE_NOISE	Specifies whether to separate output noise contribution from any noise source into input noise and noise gain on every sideband. Default is NO.
SAMPLE_TYPE	Specifies the sample type in the .MEASURE statement: • CROSS • DELAY • TIMESAMPLED
<i>trig_signal</i>	Used in the .MEASURE statement with SAMPLE_TYPE=CROSS DELAY. • When SAMPLE_TYPE=CROSS, it specifies the signal where the cross event happens. • When SAMPLE_TYPE=DELAY, it specifies the signal where the trigger event happens.
<i>td</i>	Used in the .MEASURE statement with SAMPLE_TYPE=CROSS DELAY to specify the time delay of the CROSS or DELAY measurements.
<i>rfc_value</i>	Specifies when the noise event is evaluated at the rising (RISE=), falling (FALL=), or crossing (CROSS=) edge.
<i>targ_signal</i>	Used in the .MEASURE statement with SAMPLE_TYPE=CROSS DELAY. • When SAMPLE_TYPE=CROSS, it specifies the signal where the output noise is probed. • When SAMPLE_TYPE=DELAY, it specifies the signal where the target event happens.
<i>val</i>	Specifies noise sampled at the specified voltage.
<i>sample_signal</i>	Used in the .MEASURE statement with SAMPLE_TYPE=TIMESAMPLED to specify the signal where the output noise is probed.
<i>number</i>	Used in the .MEASURE statement with SAMPLE_TYPE=TIMESAMPLED to specify the number of samples which equally divide the whole period.
<i>samples</i>	Used in the .MEASURE statement with SAMPLE_TYPE=TIMESAMPLED to specify the points of the additional samples.

Argument	Description
<i>phase</i>	Used in the <code>.MEASURE</code> statement with <code>SAMPLE_TYPE=TIMESAMPLED</code> to specify the phases of the additional samples.

Output

The `.SNNOISE` waveform includes the following signals:

Signal	Description
<code>out</code>	Total output noise
<code>out_dbc</code>	SSB jitter phase noise

If an input equivalent noise source is specified, the following signals are also generated:

Signal	Description
<code>gain</code>	Gain of circuit
<code>in</code>	Input-Referred Noise $out / gain$
<code>F</code>	Single-sideband Noise Factor (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the reference sideband)
<code>NF</code>	Single-sideband Noise Figure $10\log_{10}(F)$
<code>Fdsb</code>	Double-sideband Noise Factor (Total noise power at the output with all sidebands - Noise power from load at the output harmonic) / (Noise power due to input source noise from the reference sideband and its image harmonic) - The numerator is the same as the single-sideband noise factor (F) - The denominator takes the noise contribution from the input source from its reference sideband and image sideband
<code>NFdsb</code>	Double-sideband Noise Figure $10\log_{10}(F_{dsb})$
<code>Fieee</code>	IEEE definition of Noise Factor Same as the single-sideband noise factor (F)
<code>NFieee</code>	IEEE definition of Noise Figure Same as the single-sideband noise figure (NF)

Signal	Description
Fieee3rdparty	Compatible to the Fieee for 3rd-party tool • (Total noise power at the output with all sidebands - Noise power from load at the output harmonic - Noise power at output due to the harmonics except the reference sideband at the source) / (Noise power due to source noise from the reference sideband) • The denominator is the same as the single-sideband noise factor (F) • It only includes the contribution due to source noise from the reference sideband (input harmonic) in the numerator of the single-sideband noise factor
NFieee3rdparty	Compatible to the NFieee for 3rd-party tool • $10\log_{10}(F_{\text{ieee3rdparty}})$

Examples

```
.snnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ fullspectrum=yes

.snnoise v(out,0) vin lin 100 1k 1M rel_harms=1 sweep_type=relative
+ reference_sideband=0

.snnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ sidebands=-1, 1, 2

.snnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ maxsideband=10

.snnoise v(out,0) lin 100 1k 1M rel_harms=1 sweep_type=relative
+ noise_output=lsb,pm

.snnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=cross trig v(vin,0) val=0.8 rise=1
+ targ v(von,vop)

.snnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=cross trig v(vin,0) val=0.8 rise=1
+ targ v(von,vop)

.snnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=timesampled v(vin,0) samples=0.2p,0.5p,1.2p
number=5

.snnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=timesampled v(vin,0) phase=30,50,160

.snnoise lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=delay trig v(vin,0) val=1.2 rise=1 targ
v(vip2,0) val=1.2 fall=0
```

```
//P6 is the name of one port, V4 is the name for one vsource
.snnoise P6 lin '100+1' 3e+09 4e+09 name=snnoise
.snnoise V4 lin '100+1' 3e+09 4e+09 name=snnoise1
```

.SNOSC

Description

Use this command to perform Shooting Newton (SN) analysis for an autonomous circuit..

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SNOSC PERIOD=period TONE=tone NHARMS=nharms OSCNODE=oscnode
+ [TRINIT=trinit] [TRINIT_METHOD=GEAR|EGEAR|TRAP|TRAPONLY|BE]
+ [SAVE_TRINIT=YES|NO] [AUTOTRINIT=YES|NO] [KRYLOVITER=kryloviter]
+ [READ_IC="read_ic"] [READ_NS="read_ns"]
+ [WRITE="write"]
+ [PARAM=param] [PARAM_DATA=param_data]
+ [UIC] [DETECT_STEADY=YES|NO]
+ [STEADYRATIO=steadyratio] [MAXITER=maxiter]
+ [LOADSTATE="loadstate"|"loadstate_init"]
+ [SAVESTATE="savestate"|"savestate_init"]
+ [SWEEP parameter_sweep] [SWEETYPE=PARAM|SOURCE]
```

Arguments

Argument	Description
<i>period</i>	Expected period of the steady state waveforms. The period of the steady state waveform may be entered either as <i>period</i> or its reciprocal, <i>tone</i> .
<i>tone</i>	The fundamental frequency.
<i>nharms</i>	Number of harmonics to use for the specified tone.
<i>oscnode</i>	One or more nodes used to probe for oscillation conditions.
<i>trinit</i>	Run transient analysis to initialize Shooting Newton analysis.
TRINIT_METHOD	Integration method for initial transient: - GEAR: Second order of Gear - EGEAR: Enhanced second order of Gear - TRAP: Trapezoidal - TRAPONLY: Trapezoidal only - BE: Backwards Euler

Argument	Description
SAVE_TRINIT	Specifies whether to save the initial transient waveforms.
AUTOTRINIT	Specifies whether to allow the Shooting Newton engine to determine how long to run initial transient.
<i>kryloviter</i>	Maximum number of GMRES iterations. Default is 100.
" <i>read_ic</i> "	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
" <i>read_ns</i> "	Initialize nodes from the specified file.
" <i>write</i> "	Write operating point to the specified file.
<i>param</i>	Parameter name or <i>tolscale</i> to be updated dynamically during initial transient.
<i>param_data</i>	The time-value pairs for parameter <i>param</i> .
UIC	Use initial conditions (IC) to initialize nodes (do not run DCOP).
DETECT_STEADY	Specifies whether the Initial transient can be terminated early after steady-state is detected.
<i>steadyratio</i>	Specifies the ratio of steady state tolerance.
<i>maxiter</i>	Maximum number of Shooting Newton iterations. Default is 100.
" <i>loadstate</i> "	- If the <i>loadstate</i> file name is normal, loads the Shooting Newton solution from the specified file. - If the <i>loadstate</i> file name ends with <i>_init</i>, skips initial transient analysis and loads the initial transient solution from the specified file.
" <i>savestate</i> "	- If the <i>savestate</i> file name is normal, saves the Shooting Newton solution to the specified file. - If the <i>savestate</i> file name ends with <i>_init</i>, saves the initial transient solution to the specified file.

Argument	Description
<code>parameter_sweep</code>	Parameter sweep. You can sweep up to three variables, as follows: <pre>var start stop incr</pre> where <i>var</i> is a parameter variable and <i>start stop incr</i> are the starting, final, and increment value of <i>var</i>, respectively. You can specify the sweep type as either <code>LIN</code>, <code>DEC</code>, <code>OCT</code>, or <code>POI</code>. Specify the <i>nsteps</i> or <i>npoints</i>, <i>start</i> and <i>stop</i> values using the following syntax for each sweep type: • <code>LIN nsteps start stop</code> • <code>OCT nsteps start stop</code> • <code>DEC nsteps start stop</code> • <code>POI npoints p1, p2, ...pn</code> Where: • <i>nsteps</i> = number of steps per decade (<code>DEC</code>) or octave (<code>OCT</code>). • <i>npoints</i> = number of points for linear (<code>LIN</code>) or points of interest (<code>POI</code>). • <i>start</i> = starting value. • <i>stop</i> = ending value. • <i>p1, p2, ... pn</i> = point values.
<code>SWEETYPE</code>	Specifies whether to sweep a variable or a voltage current source, as follows: • <code>PARAM</code>: Sweep a defined variable. • <code>SOURCE</code>: Sweep a voltage or current source.

Examples

```
.snosc period=1n nharms=10 trinit=200n
.snosc tone=1G nharms=10 trinit=200n
.snosc tone=1G nharms=10 trinit=200n read_ic="sn.ic"
.snosc tone=1G nharms=10 trinit=200n read_ns="sn.ic"
.snosc tone=1G nharms=10 trinit=200n write="sn.ic"
.snosc tone=1G nharms=10 trinit=200n uic
.snosc tone=1G nharms=10 trinit=200n save_trinit=yes
.snosc tone=1G nharms=10 trinit=200n trinit_method=trap
.snosc tone=1G nharms=10 autotrinit=yes
.snosc tone=1G nharms=10 autotrinit=yes detect_steady=yes
.snosc tone=1G nharms=10 trinit=200n savestate="sn.sav"
.snosc tone=1G nharms=10 trinit=200n loadstate="sn.sav"
```

.SNSTB

Description

Use this command to run a periodic, time-varying stability analysis to analyze small-signal loop gain for circuits with negative feedback loops.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SNSTB type number [[START=]start] [[STOP=]stop]  
+ [MODE=SINGLE|DIFF|COMMON] [VSOURCE=vsource]  
+ [NAME=name]
```

Arguments

Argument	Description
<i>type</i>	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
MODE	Specifies the mode: • SINGLE: Single-ended test. Single mode analysis is used to deal with feedback circuits with only one single signal path. • DIFF: Differential mode test. • COMM: Common mode test.
<i>vsource</i>	Specifies the voltage source.
<i>name</i>	Specifies the name of the SNSTB analysis, which affects the output name of the waveforms.

Examples

```
.snstb dec 10 1 100e6 mode=single vsource=v1 name=stb1
```

.SNXF

Description

Use this command to calculate transfer functions from an arbitrary number of small signal sources to a designated output in a circuit under periodic steady-state conditions.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.SNXF output type number [[START=]start] [[STOP=]stop]
+ [POI np val1 val2 ... valn] [NAME=name]
+ [SWEEP_TYPE=ABSOLUTE|RELATIVE] [REL_HARMS=rel_harms] [SAMPLED=YES|NO]
+ [MAXSAMPLES=maxsamples]
+ [SAMPLE_DEF=measname] [SIDE BANDS=sidebands]
+ [MAXSIDE BAND=maxsideband]
+ [TOLERANCE=tolerance] [KRYOVITER=kryoviter]
+ [STIMULI=SOURCES|NODES_AND_TERMINALS]
+ [modulation=single| first|second| no]
+ [out_modharm=1]
+ [in_modharms=0]
+ [in_moduppersideband=0]
```

If SAMPLED=YES, the .MEASURE statement defined by SAMPLE_DEF=measname has the following syntax:

```
.MEASURE SN measname SAMPLE_TYPE=WHEN signal SAMPLES=samples VAL=val
+ RISE|FALL|CROSS=rfc_value
```

Arguments

Argument	Description
output	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. • If specified as the voltage of a node or node pair, the PrimeSim SPICE tool calculates the transfer function to the voltage of this node or node pair. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. • If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool calculates the transfer function to its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. • If specified as an element name of other types, the PrimeSim SPICE tool calculates the transfer function to the voltage between the first two terminals of this element.

Argument	Description
<i>type</i>	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1, val2, ..., valn</i> , respectively.
<i>name</i>	Specifies the name of the SNXF analysis, which affects the output name of the waveforms.
SWEEP_TYPE	Specifies sweep frequency type: • ABSOLUTE • RELATIVE
<i>rel_harms</i>	Reference harmonic index when SWEEP_TYPE=RELATIVE.
SAMPLED	Specifies whether the evaluation is sampled at particular point in time.
<i>maxsamples</i>	Specifies the maximum number of sampled events.
<i>measname</i>	Specifies how the sample is defined in the corresponding .MEASURE statement. See the arguments: <i>signal</i> , SAMPLE_TYPE, <i>samples</i> , <i>val</i> , and <i>rfc_value</i> .
<i>sidebands</i>	Specifies the sideband index.
<i>maxsideband</i>	Specifies the maximum sideband index.
<i>tolerance</i>	Specifies the tolerance for the linear solver in SNXF analysis.
<i>kryoviter</i>	Specifies the maximum number of iterations of the linear solver used in SNXF analysis.
STIMULI	Specifies which signals are output: • SOURCES: (default) Specifies signals from independent voltage/current sources. • NODES_AND_TERMINALS: Specifies nodes or terminals.

Argument	Description
<code>SAMPLE_TYPE</code>	Specifies the sample type in the <code>.MEASURE</code> statement: <code>WHEN signal</code>: Specifies the sampled probe by <code>inst</code> or <code>v(node1,node2)</code> where <code>inst</code> is the name of a two-terminal instance and <code>node1,node2</code> are net names.
<code>samples</code>	Specifies the sampled event at specified time(s).
<code>val</code>	Specifies the sampled voltage.
<code>rfc_value</code>	Specifies the sampled event is evaluated at the rising (<code>RISE=</code>), falling (<code>FALL=</code>), or crossing (<code>CROSS=</code>) edge.
<code>modulation</code>	Specifies whether compute transfer functions between modulated sources and outputs. The following are the modulation types: <code>single</code> <code>first second</code> <code>no</code> Default is <code>no</code> . <code>first second</code> should be specified as a pair with <code>SWEEP_TYPE=RELATIVE</code> .
<code>out_modharm</code>	Specifies the harmonic number of the output modulation, when <code>modulation=first second</code> . Default is <code>1</code> .
<code>in_modharms</code>	Specifies a list of harmonic numbers of the source modulation. Default is <code>0</code> .
<code>in_moduppersideband</code>	Specifies the upper sideband of source modulation when <code>modulation=single</code> . Default is <code>0</code> .

Examples

```
.snxf dec 10 1k 100M
.snxf lin 100 1k 1M
.snxf log 100 1k 1M
.snxf poi 3 100k 500k 1M
.snxf lin 100 1k 1M rel_harms=1 sweep_type=relative
.snxf lin 100 1k 1M sidebands=1,2,-2
.snxf lin 100 1k 1M maxsideband=10

.snxf v(vip2,0) lin 100 1k 1M sampled=yes sample_def=pm0
.measure sn pm0 sample_type=when v(vip2,0) samples=2e-10,2.5e-10
+ val=1.2 fall=0

//P6 is the name of one port, V4 is the name for one vsorce
.sn tone=3.5e+09 nharms=25 trinit=1e-07
.snxf P6 name=snxf lin '100+1' 3e+09 4e+09 sidebands=-2
.snxf V4 name=snxf1 lin '100+1' 3e+09 4e+09 sidebands=-2
```


.STATEYE

Description

Use this command to perform statistical eye analysis to evaluate high-speed serial and memory interfaces. The statistical eye diagram provides fundamental performance metrics such as bit error rate (BER), eye opening, bathtub curves, and others.

When setting up a Statistical Eye Analysis, the Port element is used to designate the incident (input) and probe (output) ports for the system to be analyzed. Ports can be specified as single-ended or mixed mode. Random jitter can be applied to each incident and probe point in the system. Each incident port acts as random bit pattern source with specified voltage magnitude. If an incident port element does not have a time domain voltage magnitude specification, the defaults, $V_{high}=1.0$, $V_{low}=-1.0$ are used. Probe ports are used as observation points where `.PRINT`, `.PROBE`, and `.MEASURE` commands can be defined.

Note:

This analysis is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

For examples of use, see the `/SI` subdirectory under the `/tutorials` directory in your PrimeSim SPICE installation.

For more information, see Statistical Eye Analysis in the *PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis*.

Syntax

```
.STATEYE
```

Required keywords

+ T = <i>time_interval</i>	\$ Unit Interval
+ Trf = <i>rise_fall_time</i>	\$ rise/fall time
+ Incident_port = <i>idx1[,idx2,...,idxN]</i>	\$ input source
+ Probe_port = <i>idx1[,idx2,...,idxN]</i>	\$ observation points

Optional keywords for edge response (initial transient) control and test

+ [Tran_Init = <i>n_periods</i>]	\$ initial tran length
+ [Tran_Init_DELMAX = <i>val</i>]	\$ delmax for initial tran
+ [Save_tr = <i>ascii</i>] [Load_tr = <i>ascii</i>]	\$ save/reuse channel response
+ [Save_Dir = <i>string</i>] [Load_Dir = <i>string</i>]	\$ save/reuse directory name
+ [ISI_depth = <i>n</i>]	\$ length of ISI effect
+ [TD_In = <i>val</i>]	\$ initial waiting time
+ [TD_Probe = <i>val</i>] [TD_Probe_AMI = <i>val</i>]	\$ discard initial part
+ [V_low = <i>val</i>] [V_high = <i>val</i>]	\$ input high/low level
+ [Tran_Init_Windowing <i>tn_beg tn_end tf_beg tf_end</i>]	

Optional keywords for time domain waveform processing

```
+ [Mode = EDGE|CONV|TRAN]           $ analysis mode
+ [MAX_Pattern = n] [Pattern_Repeat = n] $ pattern length control
+ [Unfold_Length = n]                 $ length of unfold output
+ [Tran_Bit_Seg = val]                 $ block size of waveform process
+ [Stream_File = 'wdf_file_name']      $ reuse existing tr0 waveform
+ [Stream_Signal = 'signal_name1'[, signal_name2,..., signal_nameN]]
```

Optional keywords for eye diagram display setting

```
+ [T_resolution = n] [V_resolution = n] $ output V/T resolutions
+ [VD_range = val | val1 val2 ... valN] $ output V range
+ [TD_NUI = n]                           $ number of output UI
+ [Align_Eye = CENT [percentage] | SYNC] $ eye centering/alignment
```

Optional keywords mainly for SerDes designs

```
+ [AMI_T_resolution = n]           $ sample per bit for AMI flow
+ [Ignore_Bits = n]                 $ discard initial n bits
+ [Redriver = (probe_port_index,incident_port_index)]
+ [Retimer = (probe_port_index,incident_port_index)]
+ [TxJitter_Mode = 1|2]             $ Tx jitter injection method
+ [Impulse_Norm = val]              $ integration of impulse res
```

Optional keywords mainly for DDR designs

```
+ [Xtalk_Type = SYNC|ASYN|DDP|NO|ONLY] $ crosstalk type
+ [Edge= 1|2|4|8]                     $ # of patterns for edge mode
+ [Edge_Apps_Type = 1|2]               $ set 2 if extremely nonlinear
+ [Initial_Low = val] [Initial_Zero = val] $ for buffer initialization
+ [Clock_Port = (port_index, vth)]     $ clock output port index
+ [Clock_Mode = 1|2]                   $ clock jitter injection method
+ [Tr = rise_time] [Tf = fall_time]    $ use when input rise != fall
+ [T_Factor = n1 [, n2, ... nN]]       $ to mix SDR and DDR
+ [VBOX_Ref_Type = 1|2|3]               $ reference point for BOX meas
+ [PTB_Data = string]                  $ (VDD) perturbation analysis
```

Arguments

Argument	Description
T	Time (in seconds) of single bit width of the incident signal, normally referred as Unit Interval (UI).
Trf	Single value (in seconds) to set both the rise and fall times of the incident pulse.
Tr	Rise time (in seconds) of the incident port.
Tf	Fall time (in seconds) of the incident port.
Incident_port	An array of the index numbers of the incident port elements.
Probe_port	An array of the index numbers of the probing port elements.

Argument	Description
V_low	Low voltage level of the incident pulse. The value is used when the voltage level is not specified in the incident ports. Default: -1.0.
V_high	High voltage level of the incident pulse. The value is used when the voltage level is not specified in the incident ports. Default: 1.0.
Tran_init	An integer number that specifies the numbers of unit intervals (T) that is used by the initial transient analysis to determine the response of the system. Default: 60.
T_resolution	An integer number used to specify the probability density function (PDF) image resolution of the time axis. Default: 200.
AMI_T_resolution	An integer number used to specify the time point density (per unit interval) in IBIS-AMI filtering. By default, the T_resolution value is used.
V_resolution	An integer number used to specify the probability density function (PDF) image resolution of the voltage axis. Default: 200.
TD_In	Applies the specified time delay to the incident pulse/step in the initial transient analysis. Default: 0 (no delay).
TD_Probe	When a positive time value is specified, .STATEYE only uses initial transient analysis waveforms after the specified time for the eye diagram generation. Default: 0.
TD_Probe_AMI	Similar to the TD_Probe keyword. .STATEYE only uses AMI filtered waveform after the specified time for the eye diagram generation. Default 0.
TW_RELTOL	Relative tolerance used to determine the output waveform transition time window. .STATEYE considers the output settled when the output voltage reaches the value of the maximum output swing multiplied by TW_RELTOL. Default: 0.01.
TW_ABSTOL	Absolute tolerance used to determine the output waveform transition time window. This value overrides TW_RELTOL based tolerance. By default, TW_RELTOL is used to determine the time window.
ISI_depth	An integer value to force time window as <code>time window=n×Unit Interval</code> . This value overrides TW_RELTOL and TW_ABSTOL. By default, .STATEYE automatically determines the time window. It is recommended not to specify this keyword unless you have a special need.

Argument	Description
<code>Initial_Low</code>	Forces all the edge input to begin at a low state for specified time. For edges which begin at a high state, the input ramps up to a high state at the specified time and then proceeds to the given edge shape. When this keyword is specified, <code>TD_In</code> defaults to <code>2*Initial_Low</code> , so the edge transition begins after the specified <code>Initial_Low</code> period. <code>TD_Probe</code> and <code>TD_Probe_AMI</code> also default to <code>TD_In</code> , so the edge response during the <code>Initial_Low</code> period is ignored.
<code>Initial_Zero</code>	Similar to <code>Initial_Low</code> keyword, <code>Initial_Zero=val</code> forces all the edge inputs to begin at 0V for a specified time. The input ramps up or down to the initial logic state at the specified time and then proceeds to the given edge shape. When this keyword is specified, <code>TD_In</code> defaults to <code>2*Initial_Zero</code> , so the edge transition begins after the specified <code>Initial_Zero</code> period. <code>TD_Probe</code> and <code>TD_Probe_AMI</code> also default to <code>TD_In</code> , so the edge response during the <code>Initial_Zero</code> period is ignored.
<code>VD_range</code>	Specifies the voltage display (output data) range. By default, the <code>.STATEYE</code> analysis engine automatically determines the optimum voltage display range. When a single value is specified, <code>.STATEYE</code> applies the display range to all the probe ports defined in the <code>Probe_port</code> keyword. When an array of values is specified, each entry corresponds to the display range of each probe point and the number of values must correspond to the number of ports defined in the <code>Probe_port</code> keyword. Note: StatEye automatically estimates the suitable voltage range by superposition of initial edge response waveform(s) with continuous ones and zeros. Normally this gives good StatEye display range. However, when IBIS-AMI model is in the channel and has a filter which may drastically change the output swing voltage, the initial estimation may not give suitable range. For such case, use relatively conservative <code>VD_range</code> at first with relatively short pattern to check if the <code>VD_range</code> covers the output. Also, if the AMI comes with a document which shows example output, using that information may provide guidance on the value of <code>VD_range</code>.
<code>TD_NUI</code>	An integer number to specify time display range relative to the unit interval. Default value is 2 to display a single eye. <code>TD_NUI</code> value may be from 1 to 5. The higher value requires a larger data size.

Argument	Description
EDGE	Number of edges to be used. 1: Conventional statistical eye generation using the single pulse response (default). 2: Double-edge mode. The rising and falling edges are evaluated separately. 4: Four edge patterns are modeled. Aids in increasing accuracy for transmission line systems' linear memory effect. 8: Eight edge patterns are modeled for greater accuracy in nonlinearity. As the number of edge patterns increases, higher nonlinearity can be modeled accurately.
MAX_PATTERN= <i>n</i>	Limits the number of bits to be examined for a custom bit pattern specified with LFSR/PAT in incident Port-elements. For example, if MAX_PATTERN=100 is specified, .STATEYE examines only the first 100 bits in the LFSR/PAT sources. This keyword is especially effective when LFSR is used with very high (over 20-bit) feedback taps since it generates an extremely long bit stream. The default value of MAX_PATTERN is 1000 when mode=tran is used and 2^{n-1} when the default mode=edge and edge=<n> are used where n=1 (default), 2, 4, or 8. When you specify MAX_PATTERN=0, .STATEYE examines all the given patterns.
PATTERN_REPEAT	When a positive number, <i>n</i>, is specified, .STATEYE repeats pattern examination <i>n</i> times until the number of bits hits the MAX_PATTERN value. Default: 0 (no repeats).
SAVE_TR=ascii	Saves initial transient data in text files.
LOAD_TR=ascii	Loads initial transient data from text files when available.
SAVE_DIR= <i>string</i>	Specifies a target directory other than the one specified by the -o command option for the SAVE_TR and LOAD_TR transient files. In saving initial transient result, the PrimeSim SPICE tool creates netlist.save0/ directory under the specified SAVE_DIR directory. For example, SAVE_DIR = my_dir is specified in my_netlist.sp, "my_dir/my_netlist.save0/" directory is the target of SAVE_TR and LOAD_TR operations.
LOAD_DIR= <i>string</i>	LOAD_DIR works the same as SAVE_DIR but LOAD_DIR can be used to specify different directory target than SAVE_DIR for initial transient data read in. LOAD_DIR defaults to SAVE_DIR.

Argument	Description
<code>PTB_DATA=string</code>	Specifies the name of directory that contains initial edge response data with arbitrary perturbation. <code>.STATEYE</code> takes differences between normal edge responses and perturbed edge responses. Then <code>.STATEYE</code> applies voltage-wise fluctuation at each voltage level of the resulting eye diagram. For more information on perturbation analysis, see the <i>PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis</i> .
<code>Ignore_Bits=n</code>	When a positive number is specified, the <code>.STATEYE</code> pattern-specific eye diagram generation process ignores the first <i>n</i> bits. The value is overridden by one specified in the AMI parameter file when one exists for each probe port.
<code>AMIInit_Use_Impulse=0 1</code>	Specifies the waveform input for the <code>AMI_Init</code> function: 0: Edge (for example, pulse and step) responses are used for the <code>AMI_Init</code> function call. 1: (default) <code>.STATEYE</code> extracts system impulse response from edge responses then uses it for the <code>AMI_Init</code> function call.
<code>Impulse_Norm</code>	Specifies normalization factor in channel impulse response generation. Area integration of the impulse response function is normalized to this value. By default, the target channel's low frequency gain (V_{out}/V_{in}) is used.
<code>ALIGN_EYE</code>	Specifies horizontal eye diagram display alignment options. <code>.STATEYE</code> uses <code>ALIGN_EYE=CENT</code> by default. <code>CENT [percentage]</code>: Eye diagrams are placed in the center of the display. You can add the percentage value as an optional argument. When the percentage value is specified, <code>.STATEYE</code> retrieves the time point when the pulse response reaches the given percentage of the full swing. <code>.STATEYE</code>, then, uses the time point as the reference for the eye centering. If the percentage value is not given, the reference point is the beginning of the pulse response. <code>SYNC</code>: Takes timing offset into account based on the common ideal clock ticks. Note: When the receiver-side AMI model is attached and the CDR is activated, this option is ignored and the resulting eye position is based on the CDR clock.

Argument	Description
<code>Xtalk_Type</code>	Specifies type of crosstalk. <code>SYNC</code>: (Default) Crosstalk aggressors and victims are in sync under a single system clock. <code>ASYN</code>: Crosstalk noises are considered asynchronous. <code>DDP</code>: Input data dependent crosstalk. This mode requires input bit patterns for both aggressor and victim lines. <code>NO</code>: No crosstalk. <code>ONLY</code>: Suppress signal propagation and only show the crosstalk coupling effect.
<code>MODE=EDGE CONV TRAN</code>	Specifies the <code>.STATEYE</code> simulation mode. By default, <code>MODE=EDGE</code> is used. <code>EDGE</code>: <code>.STATEYE</code> performs edge superposition to generate eye diagrams. The number of edges to be used is specified by the <code>EDGE</code> keyword. <code>CONV</code>: Based on the initial edge response, <code>.STATEYE</code> encapsulates the whole system into single linear transfer function (that is, impulse response) and performs convolution integral with the input bit stream to generate eye diagrams. The <code>EDGE</code> keyword is ignored when this convolution mode is specified. <code>TRAN</code>: <code>.STATEYE</code> uses full non-linear transient analysis to generate eye diagrams. The <code>EDGE</code> keyword is ignored when this full transient analysis mode is specified.
<code>Tran_Bit_Seg=n</code>	Specifies the number of bits per each eye diagram generation process. When AMI objects are used, <code>AMI_GetWave</code> is called at each <code>Tran_Bit_Seg</code> bits. For example, when a given pattern has 10,000 bits and <code>Tran_Bit_Seg=1000</code>, <code>.STATEYE</code> sequentially calls <code>AMI_GetWave</code> 10 times with stream waveform of 1000 bits. Also in <code>MODE=TRAN</code> mode, <code>.STATEYE</code> processes transient waveforms at each <code>Tran_Bit_Seg</code> bits. The default values are: 200 for <code>MODE=TRAN</code> mode and 1000 for <code>MODE=EDGE</code> mode. For more details about <code>AMI_GetWave</code>, see chapter 10 of the IBIS version 5.0 specification.
<code>Unfold_Length=n</code>	Specifies the bit pattern length of the unfold eye probe/print. Default value is 100.
<code>redriver</code>	Specifies a probe port index and incident port index pair. A mid-channel repeater used as a re-driver is placed between the defined probe port and incident port pair. The output of the upstream channel as the waveform at the probe port is fed into the incident port as an input to the downstream channel. This analysis requires that the convolution mode (<code>mode=conv</code>) is set.

Argument	Description
<code>retimer</code>	Specifies a probe port index and incident port index pair. A mid-channel repeater used as a re-timer is placed between the defined probe port and incident port pair. The output of the upstream channel as the waveform at the probe port is used to generate a digital stream where the CDR clock information is used when an AMI model is attached to the probe port. The generated digital stream is fed into the incident port as an input to the downstream channel. This analysis requires that the convolution mode (<code>mode=conv</code>) is set.
<code>TXJITTER_MODE=1 2</code>	• Specifies how the transmitter-side jitter is added. • 1 (default): The jitter distribution is convoluted with the probability density of the eye diagram. • 2: Use this mode when the receiver-side equalizer model exists or when the convolution mode (<code>mode=conv</code>) is set. In this mode, the transmitter-side jitter is added as a numerical perturbation and the resulting bit-stream waveform is processed on the receiver side.
<code>TRAN_INIT_DELMAX=val</code>	Applies the <code>TRAN_INIT_DELMAX</code> value only to the <code>.STATEYE</code> initial transient when <code>MODE=EDGE</code> is used. <code>TRAN_INIT_DELMAX</code> does not have any effect when <code>MODE=TRAN</code> or <code>MODE=CONV</code> is specified.
<code>CLOCK_PORT=(port_index, vth)</code>	Specifies the clock port. <code>.STATEYE</code> takes the timing jitter at the specified voltage and the specified port and applies it to the other signal ports. The clock port must be one of the <code>Probe_port</code> arguments. By default, <code>.STATEYE</code> uses ideal clock trigger.
<code>CLOCK_MODE=1 2</code>	Specifies the method of non-ideal clock trigger treatment, when <code>CLOCK_PORT</code> is specified. • If <code>CLOCK_MODE=</code> • 1 (default): Extracts clock jitter distribution at the given logical threshold and then convolutes it with the data signal eye diagram. • 2: Perturbs the clock signal in the time domain by injecting a random number sequence, extracts trigger timings from the resulting clock waveform, and then uses these in the eye diagram generation.

Argument	Description
VBOX_Ref_Type	Specifies how to retrieve the reference time for VBOX and WorstBits measurements (see .MEASURE Syntax for .STATEYE analysis results, in the <i>PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis</i>). By default, VBOX_Ref_Type = 1 is used. 1: Sets the reference time as the maximum vertical opening point. 2: Sets the reference time as the center of the maximum horizontal eye opening point. 3: Sets the reference time as the cross point of the clock port. You must specify the CLOCK_PORT keyword with the appropriate arguments, for this option.
<pre>STREAM_FILE ='wdf_file_name' STREAM_SIGNAL = 'signal_name1'[, signal_name2, ..., signal_nameN]</pre>	When MODE=TRAN is specified, these two keywords allow the PrimeSim SPICE tool to load presimulated transient waveforms instead of rerunning the transient analysis. Consider for example, STREAM_FILE='my_wdf.tr0' STREAM_SIGNAL = 'v(port1)' This command allows .STATEYE to use the v(port1) waveform from the my_wdf.tr0 file for the first Probe_port.
T_Factor	Specifies integer multipliers to the unit interval of each incident port. The number of the arguments must be either one or the number of incident ports. By default, .STATEYE uses T_Factor = 1. See also Multiple Data Rate Eye Diagram Analysis with StatEye in the <i>PrimeSim™ HSPICE® User Guide: Signal Integrity Modeling and Analysis</i>.
<pre>TRAN_INIT_WINDOWING tn_beg tn_end tf_beg tf_end</pre>	To eliminate numerical/measurement noise, you can use this keyword TRAN_INIT_WINDOWING to .STATEYE, to apply windowing to the initial edge responses. The arguments tn_beg, tn_end, tf_beg, and tf_end can all be specified in seconds or unit interval, but the two units cannot be mixed. For the near end, .STATEYE applies voltage settlement from tn_beg to tn_end. For the far end, .STATEYE applies voltage settlement from tf_beg to tf_end. This windowing improves the stability of the near t=0 waveform and the settlement of the tail of the waveform.

Examples

```
.STATEYE T=400p Trf=20p
+ incident_port= 1, 2
+ probe_port= 3, 4
```

.TRAN

Transient analysis, probably the most important analysis type, simulates the circuit as a function of time over a specified voltage range with the specified initial conditions.

Syntax

```
.TRAN tstep tstop [start=Tstart] [UIC]
or
.TRAN tstep1 tstop1 [tstep2 tstop2 ... tstepN tstopN]
+ [start=Tstart] [UIC]
+ [PARAM=param] [PARAM_DATA=param_data]
or
.TRAN tstep tstop [start=Tstart] [UIC]
+ [SWEEP type] [SWEEPTYPE=PARAM|SOURCE]
+ [PARAM=param] [PARAM_DATA=param_data]
or
.TRAN tstep tstop [start=Tstart] [UIC]
+ [SWEEP data="dataname"]
+ [PARAM=param] [PARAM_DATA=param_data]
or
.TRAN tstep tstop [start=Tstart] [UIC]
+ [SWEEP MONTE=MCcommand data="dataname"]
+ [PARAM=param] [PARAM_DATA=param_data]
or
.TRAN tstep tstop [start=Tstart]
+ [tempvec=(t1 Temp1 t2 Temp2 t3 Temp3 ...) [tempstep=tempstep_val]]
+ [PARAM=param] [PARAM_DATA=param_data]
```

Arguments

Argument	Description
<i>tstep tstop</i>	User-defined time step.
<i>Tstart</i>	Starting time for the graphic output file. The default is 0 seconds. <i>Tstart</i> can save memory in the graphic output file because it stores only the data of time >= start.
<i>UIC</i>	The usage of <i>UIC</i> can be found in the <i>.IC</i> statement description.
<i>param</i>	Parameter names or <i>tolscale</i> to be updated dynamically during transient.
<i>param_data</i>	The time, single or multiple values pairs for parameter <i>param</i> . .tran 1n 1u param="rin,vin" param_data="1n,50,2,10n,100,2.5" Above example parametrizes <i>rin</i> and <i>vin</i> over transient time at 1ns and 10ns.

Argument	Description
<i>type</i>	Sweep and the types of sweep can be found in the <code>.DC</code> statement description. The sweep variable can be any one of the voltage or current sources (<code>Vxxx</code> or <code>Ixxx</code>), <code>TEMP</code> keyword, a variety of parameters, or data names referred to in <code>.DATA</code> statements. The sweep types are <code>DEC</code>, <code>LIN</code>, <code>OCT</code>, or <code>POI</code> (see the <code>.DC</code> description). For each sweep condition, the simulation results are stored in the files <code>xxx_s0.fsdb</code>, <code>xxx_s1.fsdb</code>, and so on if the option <code>primesim_output=fsdb</code> was set.
<code>SWEEPTYPE</code>	Specifies whether to sweep a variable or a voltage current source, as follows: • <code>PARAM</code>: Sweep a defined variable. • <code>SOURCE</code>: Sweep a voltage or current source.
<i>dataname</i>	Name for data driven name (see <code>.DATA</code> description).
<i>t1 Temp1 t2 Temp2</i> <i>t3 Temp3 ...</i>	Sets different temperature change values (<i>Temp1</i>, <i>Temp2</i>, ...) at the user-defined time points (<i>t1</i>, <i>t2</i>, ...).
<i>tempstep_val</i>	Defines the time interval of the temperature update. For example, if <code>tempstep=1n</code>, the temperature is updated at following time points: <i>t1</i>, <i>t1+1n</i>, ..., <i>t1+1n*N</i>, <i>t2</i>, <i>t2+1n</i>, ..., <i>t2+1n*M</i>, <i>t3</i>, ..., and the corresponding temperature values are given by linear interpolation. To probe temperature changes in transient simulation, add the following lines: <pre>.param ckt_temp=temper .probe tran par(ckt_temp)</pre>

Description

Because a transient analysis first performs a DC operating point analysis (unless the `UIC` option is specified in the `.TRAN` statement), most of the DC analysis algorithms, control options, and initialization and convergence issues apply to transient analysis.

Some circuits, such as oscillators or circuits with feedback, do not have stable operating point solutions. For these circuits, either the feedback loop must be broken so that a DC operating point can be calculated, or the initial conditions must be provided in the simulation input. The DC operating point analysis is bypassed if the `UIC` parameter is included in the `.TRAN` statement. If `UIC` is included in the `.TRAN` statement, a transient analysis is started using node voltages specified in an `.IC` statement. If a node is set to 5V in the `.IC` statement, the value at that node for the first time point (time 0) is 5V.

Periodic Output (strobeperiod/strobedelay)

The PrimeSim tool supports using `strobeperiod` to periodically output the time point into waveform database. This can be useful when the user wants to perform FFT on a simulation result without any interpolation. However, using this option can increase the

number of time points and slow down simulation. By default, `strobeperiod` starts from time 0, and can be changed with `strobedelay`. For example:

```
.tran tstep tstop strobeperiod=tperiod strobedelay=tdelay
```

Starting Analysis From Different Times (simstart)

The PrimeSim tool supports starting the simulation from a time not equal to 0. When using this syntax, it is similar to a restore simulation. The PrimeSim tool starts the simulation from the specified time without the user modifying any stimulus. For example:

```
.tran tstep tstop simstart=x
```

Transient Noise Analysis Support

Transient noise analysis is large signal noise analysis in the time domain. This analysis is very useful for all kinds of noise-sensitive circuits to determine the performance deterioration due to device noise. Both white and flicker noise can be supported.

Note:

Transient noise analysis is not supported on multiple partitions.

Transient noise analysis is supported only in the PrimeSim SPICE tool; it is not supported in the PrimeSim Pro tool.

```
.TRAN tstep tstop fmax=val1 [fmin=val2] [scale=val3]  
+ [seed=val4] [nstart=val5]
```

where:

- `fmax` – Parameter to invoke the transient noise analysis. Default is 0. A nonzero value turns on noise sources during transient analysis.
- `fmin` – If specified, the flicker noise is included in transient analysis. The default is `fmax`. The noise power density below `fmin` is constant.
- `scale` – Noise scale for all noise sources. Default is 1.
- `seed` – Seed for random number generator. Default is -1 (0 is noiseless).
- `nstart` – Start time during transient analysis when noise sources are activated. Default is 0.

Examples

```
.tran 0.5n 1m fmax=1e6 scale=1e2  
.tran 1ps 0.1u fmax=5e6 fmin=1e3
```

In the first example, transient noise analysis takes only white noise sources into account, and noise sources are scaled by 1e2 times.

In the second example, both white and flicker noise sources are taken into account.

Transient Noise Analysis Using PrimeSim HSPICE Syntax

PrimeSim HSPICE syntax for transient noise analysis is supported by the PrimeSim tool. In addition to the transient analysis statement (`.TRAN`), the transient noise analysis is performed by adding the following:

```
.TRANNOISE output [SEED=val] [START=val] [FMIN=val]  
+ [FMAX=val] [SCALE=val]
```

For more information, see the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

Transient Noise Analysis With Monte Carlo Samples

Running transient noise analysis with Monte Carlo samples is performed by sweeping the seed parameter.

```
.param param  
.TRAN tstep tstop fmax=val1 [fmin=val2] [scale=val3]  
+ [seed=val4] [nstart=val5] [sweep parameter_sweep]
```

Transient Noise Analysis Block-Level Noise Control

Transient noise analysis simulation often requires a higher accuracy simulation mode and therefore a longer simulation time. With block-level noise control, you can choose to disable noise in some blocks or subcircuits in order to speed up the simulation.

Specify `NOISE=0` to disable noise generation for the specified block or subcircuit and reduce transient noise analysis simulation time. `NOISE=1` is the default.

In the first example, `NOISE=0` is added to a `.SUBCKT` statement to disable noise in a specified subcircuit:

```
.SUBCKT subname n1 n2 n3 ... [param=val] [NOISE=0]  
.ENDS
```

The second example disables noise in subcircuit instances beginning with "X":

```
Xamplifier n1 n2 n3 Xamp NOISE=0
```

Capacitance Tables

Capacitance tables can be generated at any time during transient analysis. For example:

```
.save type=[captab|captab1|captab2] time=time_value file=file_name
```

where `type=captab` is equivalent to `type=captab1`. Here is another example:

```
.save type=captab time=100n file=captab_file
```

In the previous example, the capacitance table *captab_file* is generated at time 100ns. Note that `.option captab` is equivalent to `.save type=captab` and that `.option captab=2` is equivalent to `.save type=captab2`.

Examples

```
.TRAN 0.1n 100n
.TRAN 1n 20n UIC
.TRAN 0.1n 20m start=10m
.TRAN 0.1n 100n sweep temp -40 80 20
.TRAN 0.1n 100n sweep vcc LIN 3 3 5
.TRAN 0.1n 100n sweep vcc POI 3 3 4 5
.TRAN 0.1n 100n sweep vcc 3 5 1
.TRAN 0.1n 100n sweep data=v_data
.TRAN 0.1n 100n sweep monte=100
.TRAN 0.1n 100n sweep monte=900 data=dataname
```

In the first example, the transient analysis starts with a 0.1ns time step for 100ns.

In the second example, the transient analysis starts with a 1ns time step for 20ns with UIC.

In the third example, the transient analysis starts with a time step of 0.1ns for 20ms. However, only the simulation results at `time>10ms` are stored in the graphic output file.

In the fourth example, temperature is swept from -40°C to 80°C with a 20°C increment.

In the fifth through seventh examples, the transient analysis uses the parameter `vcc=3, 4, and 5`.

The eighth example is a data-driven transient analysis with a `.DATA` statement named `v_data`.

In the ninth example, the PrimeSim Pro tool performs a Monte Carlo simulation. The transient analysis is run 100 times randomly varying the statistical parameters.

In the final example, the PrimeSim Pro tool performs a Monte Carlo simulation on the results of a data-driven transient analysis with a `.DATA` statement named `dataname`. The Monte Carlo transient analysis is run 900 times randomly varying the statistical parameters. Note that the outer loop is a Monte Carlo simulation and the inside loop is a data sweep transient simulation, and so the PrimeSim tool is not calculating the measurement result's mean/sigma. In addition, only the FSDB output format is supported in this example.

Single Time Step Transient Analysis

```
**** Inverter Chain ****
.option post
.global vdd
**** MOSFET model
.model nch nmos level=49
.model pch pmos level=49
```

```
**** subckt description
.subckt inv out inp vdd
mp1 out inp vdd vdd pch w=20u l=0.5u
mn1 out inp 0 0 nch w=10u l=0.4u
.ends inv

.subckt inv5 out inp vdd
x1 a1 inp vdd inv
x2 a2 a1 vdd inv
x3 a3 a2 vdd inv
x4 a4 a3 vdd inv
x5 out a4 vdd inv
.ends inv5

**** Main circuit
v dd vdd 0 3.3
v in o1 0 pulse (0 3.3 2n 0.1n 5n 30n 2000n)
x 1 o6 o5 vdd inv5
x 2 o2 o1 vdd inv5
x 3 o3 o2 vdd inv5
x4 o4 o3 vdd inv5
x5 o5 o4 vdd inv
.save

**** single time step
.tran 0.01n 2000n
.end
```

In this example, the power supply of the inverter chain changes abruptly in the first 30ns of simulation time and then is latent until the end of the transient analysis time.

Examples

Data Sweep

This example runs 200ns transient with temperature variation. Temperature begins at 25C then changes to 50C at 50ns, 100C at 100ns; temperature is incremented at every 10ns. The additional data sweep allows multiple transient runs to be performed per data sweep parameter settings.

```
.tran 1n 200n tempvec=(0n 25 50n 50 100n 100) tempstep=10n sweep
data=temptest
.data temptest
+ rnom_v rtemp_v
+ 1k 1k
+ 2k 2k
+ 3k 3k
.enddata
```

Examples

Monte Carlo

This example runs 200ns transient with temperature variation. Temperature begins at 25C then changes to 50C at 50ns, 100C at 100ns; temperature is incremented at every 10ns. The additional monte carlo sweep allows multiple transient runs to be performed.

```
.tran 1n 200n tempvec=(0n 25 50n 50 100n 100) tempstep=10n sweep monte=5
```

Limitations: alter sweep is not supported.

.XF

Description

Use this command for AC small signal transfer functions.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by the PrimeSim Pro tool.

Syntax

```
.XF output type number [[START=]start] [[STOP=]stop]  
+ [POI np val1 val2 ... valn] [NAME=name]  
+ [STIMULI=SOURCES|NODES_AND_TERMINALS]  
+ [WRITE="write_file"] [WRITE_FINAL="write_final_file"]  
+ [READ_IC="read_ic"] [READ_NS="read_ns"]
```

Arguments

Argument	Description
<i>output</i>	Specifies the voltage of a node or node pair, or the name of an element where the PrimeSim SPICE tool sums up the output noise. - If specified as the voltage of a node or node pair, the PrimeSim SPICE tool calculates the transfer function to the voltage of this node or node pair. Specify a pair of nodes as $V(n+, n-)$. If you specify only one node, $V(n+)$, then the PrimeSim SPICE tool assumes that the second node is ground. - If specified as the name of a voltage source or an inductor, the PrimeSim SPICE tool calculates the transfer function to its branch current. Specify the name of a two-terminal device as an output probe. When a voltage source or inductor is used as an output probe, the output type is current. For other devices, the output type is voltage. - If specified as an element name of other types, the PrimeSim SPICE tool calculates the transfer function to the voltage between the first two terminals of this element.

Argument	Description
<i>type</i>	Sweeping type. One of the following: • DEC: Decade variation with <i>number</i> points per decade, starting at <i>start</i>, stopping at <i>stop</i>. • LOG: Logarithmic variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>. • LIN: Linear variation with <i>number</i> points in the entire range, starting at <i>start</i>, stopping at <i>stop</i>.
<i>number</i>	Number of points per decade or just number of points depending on the preceding <i>type</i> keyword.
<i>start, stop</i>	Starting and ending frequency.
POI	List of <i>np</i> points of interest with values <i>val1, val2, ..., valn</i> , respectively.
<i>name</i>	Specifies the name of the XF analysis, which affects the output name of the waveforms.
STIMULI	Specifies which signals are output: • SOURCES: (default) Specifies signals from independent voltage/current sources. • NODES_AND_TERMINALS: Specifies nodes or terminals.
" <i>write_file</i> "	Specifies the file to write out the IC.
" <i>write_final_file</i> "	Specifies the file to write out the final sweep IC.
" <i>read_ic</i> "	Use initial conditions (IC) from the specified file as an estimate of the operating point and then evaluate the operating point.
" <i>read_ns</i> "	Initialize nodes from the specified file.

Examples

```
.xf v(OUT,gnd) dec 5 100 1g name=xf
.xf P6 lin '100+1' 3e+09 4e+09 stimuli=nodes_and_terminals name=xf1
.xf V4 lin '100+1' 3e+09 4e+09 name=xf2
```

PrimeSim SPICE Control Options

The PrimeSim SPICE tool supports the following global control options:

- [S_IGNORE_PARAM](#)

S_IGNORE_PARAM

Description

Specifies whether to ignore additional [S-Element](#) parameters in a PrimeSim SPICE or 3rd-party netlist.

Note:

This command is supported only by the PrimeSim SPICE tool; it is not supported by other PrimeSim Continuum simulators.

Syntax

```
.option S_IGNORE_PARAM=[0|1|2]
```

Arguments

Argument	Description
0	(default) Does not ignore any PrimeSim SPICE and 3rd-party options.
1	Ignores additional options such as: FMAX, FDELTA, FBASE, ENFORCE_PASSIVE, and so on.
2	Ignores all additional options, INTERPOLATION, and RATIONAL_FUNC=0, and defaults to self-resonant frequency (SRF).

Part 5: Multi-CPU and GPU Support

Describes the PrimeSim tool's multi-CPU and GPU support.

This Part presents the following chapters:

- [Chapter 12, PrimeSim Multi-CPU](#)
- [Chapter 13, PrimeSim GPU](#)

12

PrimeSim Multi-CPU

This chapter details the multi-CPU features of the PrimeSim Pro and PrimeSim SPICE tools.

This chapter covers the following topics:

- [Introduction to Multi-CPU](#)
- [Why Multi-CPU SPICE?](#)
- [Multi-CPU Computers vs. Clusters of Computers](#)
- [System Requirements](#)
- [Command-Line Options for Running Multi-CPU Simulations](#)
- [Running PrimeSim Multi-CPU on a Single Machine](#)
- [Running PrimeSim Multi-CPU on Multiple Machines](#)
- [PrimeSim Client-Server Mode](#)
- [Distributed Processing, CDPL, and Dynamic Scaling](#)
- [Using .primesim_parallel.ini to Simplify Multi-CPU Calls](#)
- [Automatic Determination of Optimal Number of Parallel Processes](#)

Introduction to Multi-CPU

Both the PrimeSim Pro and PrimeSim SPICE tools offer multi-CPU technology to run simulations on multiple CPU and machines. This multi-CPU feature gives you near-linear scalability in simulation performance and capacity with respect to the number of CPUs used versus a serial simulation with the same or very close accuracy.

Note:

It is recommended that you use the same CPU type/frequency and OS between your multi-CPU simulations that span across multiple machines. You can get unexpected results or slowdown due to different machine configurations.

Why Multi-CPU SPICE?

For many large digital, analog and memory circuits, the PrimeSim Pro tool's Pro modes can not only simulate circuits at different levels of accuracy and speed to meet different design requirements, but they also can handle a circuit capacity that traditional, serial SPICE simulators cannot. However, for some analog circuits of any size, such as ADC, PLL, and analog power rail analysis, the accuracy in PrimeSim Pro Pro modes is insufficient. Those circuits require SPICE mode accuracy to get the expected circuit behavior. The only way to increase SPICE simulation performance dramatically is through parallel execution distributed among multiple CPUs.

PrimeSim multi-CPU uses a proprietary technique to distribute the circuit over multiple CPUs and run the jobs on those CPUs synchronously to maintain the full accuracy of the SPICE algorithm. The PrimeSim SPICE algorithm minimizes communication overhead so that the performance gain from parallel execution is maximized.

PrimeSim multi-CPU can speed up the simulation of circuits with as few as several hundred transistors and nodes by more than 3 times on one single 4-way Symmetric Multi-Processing (SMP) machine. With circuits of hundreds of thousands of transistors and nodes, the improvement is more than 20 times on a cluster of 20 to 30 CPUs. Besides offering increased speed, multi-CPU SPICE execution can handle circuits too large to simulate serially. This increased speed and capacity enables you to further verify the accuracy of the PrimeSim Pro or PrimeSim SPICE tools on a much larger scale than a traditional SPICE simulator can because of prohibitively long runtimes and/or excessive memory usage.

Multi-CPU Computers vs. Clusters of Computers

As CPU speed has leveled off, the computer industry has been putting multiple CPUs together to achieve higher performance through parallel execution. These CPUs can be on a single machine or on multiple machines connected by a high-speed network.

Computers with multiple CPUs in the Symmetric Multiprocessing (SMP) architecture are the most common multi-CPU computers. An SMP computer may have two or more identical processors connected to a shared main memory. The major benefit of SMP for multi-CPU is that the inter-process communication is achieved through the shared main memory instead of the network, which can lead to communication overhead. For circuits of fewer than 1000 nodes, you should run multi-CPU on an SMP machine because the communication overhead may be a significant portion of the total runtime.

For large circuits, one SMP machine can still be too slow or too small in capacity to make multi-CPU a practical solution. In these cases, you can apply multi-CPU to a cluster of single-CPU computers or SMPs. Because the communication overhead increases as the number of machines increases, the simulation performance gain of multi-CPU eventually saturates. The saturation number of CPUs depends on the size of the circuit and the

network. PrimeSim multi-CPU automatically calculates and provides the optimal number of CPUs based on the circuit size in the log file.

System Requirements

The following are the minimum system requirements to run PrimeSim multi-CPU.

Hardware

- An SMP machine, or:
- A cluster of single-CPU and/or SMP machines with identical CPUs that are connected through Gigabit Ethernet

Software

- SMP capable operating system
- Python 3.6 or later
- LSF, SGE, Secure Shell (SSH), or RSH

SMP servers and workstations with 16, 20, or 32 CPUs are becoming more and more common in IT infrastructures. Multi-CPU is distributed in such a way that each CPU process gets the same amount of computing resources. If a process were to run slower than others due to either a slower machine or competing jobs on the machine, all the other processes would have to wait for its completion and the overall performance would be reduced. So, before you start multi-CPU simulation, consult your IT team to arrange for dedicated machines for you.

Command-Line Options for Running Multi-CPU Simulations

All options for serial (single CPU) simulations are still valid, and there are no additional options added for this feature. This means that you can use your input setup from a serial simulation to run a multi-CPU simulation directly without changing either your stimulus or netlist. It creates all of the same output files a serial SPICE run creates.

Running multi-CPU simulation is easy. The shell script manages all of the execution details. To run it on a single SMP machine, you do not have to make any set-up changes. To run it on a cluster of computers, use LSF or SGE to distribute jobs onto your LSF/SGE queue, or set up a password-less SSH or RSH login once for all the computers and set up a machine file that contains the machine names and number of the dedicated computers for use in your multi-CPU simulation.

For multi-CPU, the PrimeSim command line supports the following options (this list can also be obtained by typing `primesim -p` in the command line; for additional command-line options, see [Table 2](#) in [Running the PrimeSim Tool](#)).

Table 30 Multi-CPU Command-Line Options

Command	Description
<code>-h</code>	Prints a message that lists the command-line options (see Table 2).
<code>-p</code>	Prints usage information for parallel simulation.
<code>-np N lsf sgc</code>	<code>N</code>: Specifies the number of processes (or the maximum number of processes when the <code>-auto</code> command-line option is specified) to be used. Note: This number should not be greater than the total number of CPUs on a single multi-CPU machine or specified in the <i>machinefile</i>. <code>lsf</code>: Used with <code>bsub -n [N]</code> for LSF farm simulations. When submitting to LSF, <code>-np lsf</code> uses the process number defined in <code>bsub -n [N]</code>. For example, <pre>%> bsub -q bnormal -n 4 primesim in.sp -np lsf -o results/</pre> Equivalent to: <pre>%> bsub -q bnormal -n 4 primesim in.sp -np 4 -o results/</pre> <code>sgc</code>: Used with <code>qsub -pe mt [N]</code> for SGE farm simulations. When submitting to SGE, <code>-np sgc</code> uses the process number defined in <code>qsub -pe mt [N]</code>. For example, <pre>%> qsub -P normal -pe mt 4 -shell n primesim -np sgc -o results/ in.sp</pre> Equivalent to: <pre>%> qsub -P normal -pe mt 4 -shell n primesim -np 4 -o results/ in.sp</pre>
<code>-ngpu N</code>	Enables GPU simulations combined with CPU, where <i>N</i> specifies the number of GPU cards. For more information, see PrimeSim GPU. Note: This option must be used with the <code>-np</code> option and the number of CPUs/processes specified must be greater than or equal to the number of GPU cards specified.

Table 30 Multi-CPU Command-Line Options (Continued)

Command	Description
<code>-prelock_lic_gpu</code>	Checks out GPU licenses according to the number of user-specified <code>-ngpu</code> command-line option before start parsing. This helps reduce waiting time when GPU licenses are insufficient for the simulator to handle large input designs with long parsing times. It applies to standalone licenses only Note: Checked-out GPU licenses are released immediately if not required for simulating the input design after parsing.
<code>-dp N</code>	Specifies the number of workers of distributed processing (DP).
<code>-mt N lsf sgs</code>	<code>N</code>: Specifies the number of threads/CPU's per process in parallel simulation. It can be used with <code>-dp</code> or by itself. <code>lsf</code>: Used with <code>bsub -n [N]</code> for LSF farm simulations. When submitting to LSF, <code>-mt lsf</code> uses the process number defined in <code>bsub -n [N]</code>. For example, <pre>%> bsub -q bnormal -n 4 primesim in.sp -mt lsf -o results/</pre> Equivalent to: <pre>%> bsub -q bnormal -n 4 primesim in.sp -mt 4 -o results/</pre> <code>sgs</code>: Used with <code>qsub -pe mt [N]</code> for SGE farm simulations. When submitting to SGE, <code>-mt sgs</code> uses the process number defined in <code>qsub -pe mt [N]</code>. For example, <pre>%> qsub -P normal -pe mt 4 -shell n primesim -mt sgs -o results/ in.sp</pre> Equivalent to: <pre>%> qsub -P normal -pe mt 4 -shell n primesim -mt 4 -o results/ in.sp</pre>
<code>-dpconfig dp_config_file</code>	Specifies the configuration file for distributed processing (DP).
<code>-dplocation TMP NFS</code>	Specifies the location for all the results files generated by the DP simulation: <code>TMP</code>: The results go to the <code>/tmp</code> folder, and are then moved to the <code>-o</code> PrimeSim Pro and SPICE output working directory after the simulation task from the worker is completed. <code>NFS</code> (default): The results files go directly to the <code>-o</code> PrimeSim Pro and SPICE output working directory after they are generated.

Table 30 Multi-CPU Command-Line Options (Continued)

Command	Description
<code>-dpworkerchk</code> <code>none mem load disk all</code>	Specifies the DP worker monitor level. Default is <code>all</code>. This option applies to <code>-dp -dpconfig</code> flow. <code>none</code>: No feature is monitored. <code>mem</code>: Memory monitor. Master drops the worker if the worker host's memory consumption is too high. The lower threshold of available physical memory is controlled by <code>.OPTION DP_MEMFREE</code>. <code>load</code>: CPU load monitor. Master drops the worker if the worker host's CPU load is high. The upper threshold of the load average of one worker CPU core is controlled by <code>.OPTION DP_LOADAVG</code>. <code>disk</code>: Disk monitor. Master drops the worker if the worker's CDPL scratch disk usage is too high. Master stops the whole DP simulation if the DP master output disk usage is too high. The lower threshold of the available working (scratch) disk space is controlled by <code>.OPTION DP_DISKFREE</code>. <code>all</code>: (default) Memory, load, and disk usage are all monitored. Note: Multiple monitoring types for memory, load, and disk options are supported by combining two options with "+" syntax. For example: <code>-dpworkerchk mem+load</code>
<code>-dp_preprocess [0 1]</code>	Specifies whether the master and workers perform preprocessing at the same time. <code>0</code>: (default) The master performs its preprocessing first, then submits the workers to perform preprocessing afterwards. <code>1</code>: The master submits the workers immediately to perform preprocessing (that is, read all input files) while waiting for a DP task from the master. Note: This option applies to the <code>-dp -mt -dpconfig</code> flow.
<code>-mf machinefile</code>	Specifies the machine file for using multiple machines.
<code>-np_per_m N</code>	<i>N</i> is the maximum number of processes for each machine or CPU for an LSF (<code>-bsub</code>), SGE (<code>-qsub</code>), or user-specified (<code>-usub</code>) job. The default value of <i>N</i> is 1.

Table 30 Multi-CPU Command-Line Options (Continued)

Command	Description
<code>-auto [local exit]</code>	Automatically determines the proper number of processors to use based on the number of devices and nodes from the input netlist. Note that this number is not necessarily the best number of processors for the best performance. N specified in <code>-np</code> becomes the maximum value. Note: <code>-auto</code> and <code>-dp</code> cannot be used together. If the <code>local</code> option is specified, the initial run for finding out the optimal number of processes is run on the local machine where this job is submitted with LSF or SGE. By default, the initial run is a job submitted to an LSF or SGE farm. If the <code>exit</code> option is specified, the simulation exits after finding out the optimal number of processes. It does <i>not</i> restart the simulation with the optimal number of processes.
<code>-rsh</code>	If specified, the PrimeSim tool uses rsh instead of the default ssh.
<code>-no_netcheck</code>	Turns off network speed checking which is on by default. This can improve runtimes for short simulations, but is not recommended for long simulations.
<code>-no_retry</code>	Stops the PrimeSim tool from reducing the number of CPUs used if the number requested is not available.
<code>-no_lsfllog</code>	When you specify this option for a parallel submission, the PrimeSim parallel code no longer outputs the LSF output into <code>prefix.lsf_log</code> .
<code>-lsf_hpc</code>	Use LSF HPC instead of LSF.
<code>-bsub "lsf_options"</code>	Using LSF for multiple machines.
	Note: For <code>-bsub</code> submissions, you must enable an RSH connection between LSF farm machines.
<code>-qsub "sge_options"</code>	Using Sungrid for multiple machines.
	Note: For <code>-qsub</code> submissions, you must enable an RSH connection between SGE farm machines.
<code>-usub "primesim_manager_header options"</code>	User-specified grid command for <code>primesim_manager</code> .
<code>-timeout seconds</code>	Maximum wait time to obtain LSF/SGE resources. Default is 600 (10 minutes).

Table 30 Multi-CPU Command-Line Options (Continued)

Command	Description
<code>-wait</code>	Process does not exit until the multi-CPU job is finished. The return code for parallel simulation is always 0. Note: This option is only used for network parallel simulation, including <code>-bsub</code> or <code>-qsub</code>. It has no effect on a single-machine run.
<code>-idle</code>	Determines the number of idle CPUs at startup time and only uses that many CPUs at startup and does not adjust it if something else is started on the machine later.
<code>-merge</code>	Specifies whether to merge the *.tr files generated by the <code>-dp</code> run into a single *.tr0 file. If this argument is not specified, which is the default, the *.tr files generated by the <code>-dp</code> run are not merged and they are grouped in a <code>wv *.tr.grp</code> file. However, when this argument is specified, the *.tr files generated by the <code>-dp</code> run are merged into a single *.tr0 file, and there is no *.tr.grp file created.
<code>-dp_master_waittime value</code>	Defines the maximum time interval in seconds that the DP master waits for the first DP worker to become active. If no worker becomes active during this interval, DP simulation terminates. The unit is time (seconds) and the default is 1800. Note: This option has higher priority than <code>.OPTION DP_MASTER_WAITTIME</code>.
<code>-dpincremental</code>	Reruns a Monte Carlo simulation for unfinished or new samples. After rerun, it automatically combines the measure results from the original runs. Example: Original run: <pre>primesim -dp 4 -o run1/out lib.spi</pre> Re-run: <pre>primesim -dpincremental run1/out -dp 4 -o run2/out lib.spi</pre> After the rerun, the merged measure file is <code>run2/out.mt0</code>. The measure file for the rerun's actual run sweeps is <code>run2/out_incr.mt0</code>. For more information about how to use the incremental distributed processing (DP) feature to rerun Monte Carlo simulation, see the Automatic Incremental DP Monte Carlo topic in <i>PrimeSim HSPICE User Guide: Basic Simulation and Analysis</i>.

Running PrimeSim Multi-CPU on a Single Machine

Running the PrimeSim Pro or PrimeSim SPICE tools in multi-CPU on a single machine is as simple as specifying the number of CPU required. In the command line, add `-np x` to specify the number of CPUs you want to use.

For PrimeSim Pro FastSPICE modes, due to circuit partitioning, the multi-CPU performances are not linearly scalable and the performance gain saturates around 16 CPUs. Although there is no limit on the number of CPUs you can run, it is recommended to keep it at 16 CPUs maximum for FastSPICE modes.

Examples

```
%> primesim -spice -np 8 input.sp
%> primesim -np 4 input.sp
```

The first example runs a 8-CPU PrimeSim SPICE simulation.

The second example runs a 4-CPU PrimeSim Pro simulation.

Running PrimeSim Multi-CPU on Multiple Machines

The PrimeSim tool supports three methods to run multi-CPU simulations across multiple machines: using password-less SSH to tunnel the connection, LSF, and Sun Grid (SGE). This section explains the setup requirements and commands for each of the methods.

- [How to Set Up Password-less SSH Login](#)
- [Machine File](#)
- [LSF](#)
- [SGE](#)
- [License Suspend/Resume Feature](#)
- [Time-Out Messages](#)

How to Set Up Password-less SSH Login

PrimeSim multi-CPU can be started on any single machine, but it requires SSH to propagate processes on to other machines. By default, SSH to a network machine requires you to type a password. To avoid having to type in a password for each machine, use the following instructions to set up a *password-less* SSH login.

Run `ssh-keygen` on your machine, and press Enter when asked for a password.

Example

```
% ssh-keygen -t(type) rsa
```

where `t(type)` is `rsa1` for protocol version 1 or `rsa` for protocol version 2.

This generates both a private and a public key. In older SSH versions, they are stored in `~/.ssh/identity` and `~/.ssh/identity.pub`. In newer versions, they are stored in `~/.ssh/id_rsa` and `~/.ssh/id_rsa.pub`.

Next, add the contents of the public key file into `~/.ssh/authorized_keys` on the remote site (The file should be mode 600.)

You should then be able to use SSH to log in to any computer on the network without being asked for a password. SSH to all the computers first as a test before you send any PrimeSim multi-CPU job over the network.

Machine File

To run multi-CPU on a cluster of computers using password-less SSH, you must know the name of the computers and the number of CPUs each computer has. A simple text file, called a machine file, is required for the task. The format for a machine file is intuitive. Each line should contain the name of a machine along with the number of CPUs on that machine.

Example

```
orange:4  
peach:2
```

The machine `orange` has four CPUs, while `peach` has two CPUs for the PrimeSim multi-CPU run. The first line must contain the machine that is submitting the job.

For the above example, you can start six CPU jobs using:

```
orange> primesim -np 6 -mf machine_file input.sp
```

LSF

If you use LSF, you do not need a machine file because LSF uses whatever machine is available on the queue at the moment.

For running multi-CPU on single machine, you do not have to change the way you submit to LSF, other than adding `-n X` in the `bsub` command to specify how many CPUs to request for resources. The same command you use to explicitly select machines still applies.

For example, you can select four CPUs with model X5355 that runs on RedHat 5.6 with the following command:

```
bsub -n 4 -q queue_name -R "select[(type==RHEL5)&&(model==X5355)]"  
primesim -np 4 input.sp
```

In order to run multi-CPU across multiple machines, you must use PrimeSim commands to submit the LSF jobs. The number of CPU requests `-n x` is now the number of CPUs to request per machine (LSF job), and not the total number of CPUs. Also, you must add `span[hosts=1]` in the resource requirement (`-R`).

Using the previous example, you can modify it to use two CPUs from two machines using the following:

```
primesim -np 4 -bsub '-n 2 -q queue_name -R  
"select[(type==RHEL5)&&(model==X5355)] span[hosts=1]"' input.sp
```

LSF HPC

When using LSF HPC, regardless whether you are running single host or multi-host, for LSF HPC, you must use the `bsub` command to submit a simulation and specify `-lsf_hpc` in the PrimeSim command line. All the system allocations are performed by the LSF HPC. For example:

```
bsub -n 4 -q queue_name -R "select[(type==RHEL5)&&(model==X5355)] "  
primesim -lsf_hpc input.sp
```

This command asks LSF HPC to allocate 4 process for the PrimeSim job.

Using *.bkill to Terminate LSF Processes

After you submit your multi-CPU through regular LSF, a `.bkill` file is generated. By executing this file, you can terminate all jobs automatically. It is not necessary to obtain the LSF process ID and manually terminate them all by using `bkill` one by one.

For example, the following command terminates all LSF processes for the multi-CPU run with the prefix `3X4CPU_parallel_run`.

```
% 3X4CPU_parallel_run.bkill
```

SGE

The PrimeSim tool supports Sun Grid Environment (SGE) Version N1GE 6.0u9. You must have administrative privileges to configure SGE. Complete the following steps to set up SGE on your computer:

Note:

Complete the initial SGE configuration. This must be done by an SGE manager or administrator.

1. Add a multi-CPU environment (PE) such as `primesim`. Enter `qconf -ap primesim`, and edit the file with the following options:

```
pe_name          primesim
slots            999
user_lists       NONE
xuser_lists      NONE
start_proc_args  /bin/true
stop_proc_args   /bin/true
allocation_rule  $pe_slots
control_slaves   FALSE
job_is_first_task TRUE
urgency_slots    min
```

2. Add the PE to the queue. The default queue is `all.q`. Enter `qconf -mq all.q`, and add “`primesim`” to the “`pe_list`.”

The following code exemplifies a PrimeSim command example:

```
primesim -np 4 -qsub '-q queue_name -pe primesim 4' -o test_sge_4 in.sp
```

Using *.qdel to Terminate SGE Processes

After you submit your multi-CPU through regular SGE, a `.qdel` file is generated. By executing this file, you can terminate all jobs automatically. It is not necessary to obtain the SGE process ID and manually terminate them all by using `qdel` one by one.

For example, the following command terminates all SGE processes for the multi-CPU run with the prefix `3X4CPU_parallel_run`.

```
% 3X4CPU_parallel_run.qdel
```

License Suspend/Resume Feature

You can temporarily suspend a simulation using `kill -10 $PID`, where `$PID` is the process ID of the PrimeSim job. After suspending with `kill -10`, the PrimeSim tool releases the license until you resume the simulation using `fg` or `kill -18 $PID`. This feature can be useful when your license pools are fully utilized and you want to free up some licenses for another critical simulation.

Note:

This feature is not recommended for multi-CPU simulations. You may free up the licenses, but the system resources may not be freed up, nor can the simulation be resumed.

Suspend Job

```
%> kill -10 $PID
```

Resume Job

```
%> kill -18 $PID
```

Time-Out Messages

Parallel SPICE issues an informative message to standard output when a time-out occurs. For example, the reason for a time-out might be reported as follows:

```
Queued 40 primesim jobs via LSF.  
Network speed check is logged to 'x40.nc_log'.  
LSF output is logged to 'x40.lsf_log'.  
PrimeSim output is logged to 'x40.log'.  
Trying to get 10 machines (40 CPU's)...  
ERROR: could not get any machine after 600 seconds.  
Please check the queue and your command.
```

In this instance, you might move to a larger LSF queue, or reduce the number of processes, or have a longer time-out wait time such as `-timeout 3600`.

PrimeSim Client-Server Mode

The PrimeSim tool supports Client-Server mode. In this mode, the Server can hold licenses for the Clients (that is, the simulation jobs) to save license check-in/check-out time for better performance. This mode benefits cell characterization type jobs the most because if simulation jobs are small, the license check-in/check-out time may contribute a larger portion of total run time.

Client-Server mode is described in the following sections:

- [Start the Server](#)
- [Start the Clients](#)
- [Stop the Server](#)

Start the Server

To get into Client-Server mode, start the Server to check out and hold the licenses you require for the future simulation jobs (that is, the Clients). When the Server starts, it returns a port number on the screen (stdout); you may need this number to [Start the Clients](#) or [Stop the Server](#), if it is not the default (32001).

The PrimeSim tool exits if the required licenses are not available. For example, with the following environment variable setting, the PrimeSim tool waits up to 2 minutes for the required licenses to be available:

```
setenv PRIMESIM_WAIT_LICENSE_TIMEOUT 2
```

By default, the Server starts in the foreground, attempts to check out the required licenses, and waits for Client simulation job requests with no immediate return code. However, if you use the `-sim_background` command-line option, the PrimeSim tool first attempts to check out the required licenses, then:

- If the required licenses cannot be obtained, the PrimeSim tool exits with a non-zero return code.
- If the required licenses are successfully obtained, the PrimeSim front-command initiates the Server in the background, then the front-command itself exits with a zero return code.

Syntax

The command to start the Server is as follows:

```
%> primesim -sim_server [-sim_port=####] -sim_lic LICENSE_TOKEN_NAME=num  
[-sim_lic LICENSE_TOKEN_NAME=num] [-sim_background]
```

The following are defaults for this command:

- `-sim_port`: 32001

Examples

The following are examples of how to start the Server:

- Check out 1 CKTSIMMC, 2 CKTSIMPROFS, and 1 PRIMESIM_LIC licenses at port 6080:

```
%> primesim -sim_server -sim_port=6080 -sim_lic CKTSIMMC=1 -sim_lic  
CKTSIMPROFS=2 -sim_lic PRIMESIM_LIC=1
```

- Check out 1 CKTSIMMC, 4 CKTSIMSPICE, and 2 PRIMESIMSPICE_LIC licenses at default port 32001:

```
%> primesim -sim_server -sim_lic CKTSIMMC=1 -sim_lic CKTSIMSPICE=4 -sim_lic  
PRIMESIMSPICE_LIC=2
```

Start the Clients

In Client-Server mode, the Clients are those simulation jobs that do not check out licenses by themselves from the license server; instead, they use the licenses checked out by the Server. Other than the licensing, the Clients are same as the original runs. The Clients

should be submitted sequentially, because they can only use the Server's licenses one after another, not at the same time.

Syntax

The command to start a Client is as follows:

```
%> primesim -sim_client[=server_ip_address] "simulation  
command" [-sim_port=#####] [-sim_ipv6]
```

The following are defaults for this command:

- `-sim_client`: The Server's IP address; this can be waived if the Client is on the same machine as the Server.
- `-sim_port`: 32001
- Protocol: IPv4. If the `server_ip_address` is in IPv6, specify `-sim_ipv6`

Examples

The following are examples of how to start the Client:

- Start the Client to run `input.sp` on the same machine as the Server, and with default port 32001:

```
%> primesim -sim_client -o output/ input.sp
```

- Start the Client to run `input.sp`, where the Server is running on a machine with IP address 10.232.159.4, and with the port 32005:

```
%> primesim -sim_client=10.232.159.4 -sim_port=32005 -o output/ input.sp
```

Stop the Server

When all of the Clients have finished their simulation runs, stop the Server to release all of the licenses that you checked out.

Syntax

The command to stop the Server is as follows:

```
%> primesim -sim_client[=server_ip_address] [-sim_port=#####] [-sim_ipv6]  
-sim_exit_cs
```

The following are defaults for this command:

- `-sim_client`: The Server's IP address; this can be waived if the Client is on the same machine as the Server.
- `-sim_port`: 32001
- Protocol: IPv4. If the `server_ip_address` is in IPv6, specify `-sim_ipv6`

Examples

The following are examples of how to stop the Server:

- Stop the Server on the same machine from which the Server is started, and with the port 32003:

```
%> primesim -sim_client -sim_port=32003 -sim_exit_cs
```

- Stop the Server on the machine with IP address 10.232.159.4, and with the default port 32001:

```
%> primesim -sim_client=10.232.159.4 -sim_exit_cs
```

Distributed Processing, CDPL, and Dynamic Scaling

Besides native multi-CPU technology, the PrimeSim tool also supports distributed processing (DP) simulation. Distributed processing can be used in the simulation that contains alter, Monte Carlo, temp sweep, and/or data sweep. Use the `-dp` command-line option to invoke this feature (see [Table 30](#)).

When any of the above operations exist in the deck, and the `-dp x` command-line options are specified, the PrimeSim tool distributes all of the simulations across x different workers.

If there is a mixture of analyses (for example, alter + data sweep), then the PrimeSim tool first distributes the analysis with the largest number of iterations across x different workers. Those simulations run at the same time. The smaller set of analyses iterations, or any of the analyses greater than x , are queued to run when one of the x workers frees up. The PrimeSim tool iterates through all analyses until all simulations are complete.

The PrimeSim tool also supports multi-threaded parallel simulation within different workers of distributed processing simulation. If you specify the number of CPUs with the `-mt y` command-line option, then each worker is run in parallel with y CPUs. The maximum value for y is 16.

Use the following syntax to start PrimeSim with distributed processing mode:

```
primesim -dp number  
+ [-mt thread_number]  
+ [-dpconfig dp_config_file]  
+ [-dscale]  
+ Other_Options
```

For more information, see the following sections:

- [Running Distributed Processing on the Current Machine](#)
- [Running Distributed Processing on a Network Grid](#)

- [Dynamic Scaling](#)
- [Terminating Distributed Processing Simulation in an LSF Farm Environment](#)

Running Distributed Processing on the Current Machine

When running on one multicore machine, the PrimeSim tool starts a master process to manage/dispatch jobs and collect the results from worker processes, and the worker processes are distributed on this same machine. Then the configuration option (`-dpconfig`) is not needed for distributed processing on a multicore machine.

Example 1

The following example distributes a job into four processes on a local machine, where each process runs as a single thread (`-mt 1` is the default setting and can be ignored in the command line).

```
primesim -dp 4 -o output input.sp
```

Example 2

The following example distributes a job into ten processes on a local machine, where each process runs as a single thread.

```
primesim -dp 10 -mt 1 -o output input.sp
```

Example 3

The following example distributes a job into five processes on a local machine, where each process runs as four threads.

```
primesim -dp 5 -mt 4 -o output input.sp
```

Output Files

`output.log_p1`, `output.log_p2`, `output.log_p3` ... `output.log_p(n)` — Log file for each worker process, where n is the number of worker processes.

`output.progress` — Log file of master process, information of dispatch jobs to worker processes.

`output_dp.progress` — Log file to track the progress of each distributed task in table format. The wall time, cpu time, worker ID, hostname, status, and progress are reported for each task.

Running Distributed Processing on a Network Grid

You can submit a PrimeSim simulation job to a network of machines for parallel processing on the following network grids: SGE, LSF, RTDA, RSH, and SSH. These machine cluster/ compute farm manager modes are controlled by the *dp_config_file*.

Distributed processing uses the Synopsys Common Distributed Processing Library (CDPL).

Typical configuration files have the following forms, depending on the type of network grid:

Table 31 *Distributed Processing Configuration Files*

	#flag	hostname	slots	tmpDir	protocol	command
SGE	1		-1	/tmp	SGE	qsub -p bnormal
LSF	1		-1	/tmp	LSF	bsub -q bnormal
RTDA	1		-1	/tmp	RTDA	nc run -e "SNAPSHOT"
NetBatch	1		-1	/tmp	NB	nbjob run
RSH	1	host1	2	/tmp	RSH	rsh
	1	host2	4		RSH	rsh
SSH	1	host1	3	/tmp	SSH	ssh
	1	host2	3		SSH	ssh
CUSTOM	1		4	/tmp	CUSTOM	/path/myfarm_submit_comm and

Note:

For “CUSTOM”, provide the full pathname of the submit command.

[Table 32](#) lists the columns found in a CDPL configuration file. Columns are divided by "|".

Table 32 *CDPL Configuration File Columns*

Column	Description
Number of Flag:	1 denotes the host can be a worker machine, 0 denotes not.
Hostname:	Blank for computer farms, hostnames for RSH/SSH.
Slot:	-1 denotes unlimited for computer farms, or numbers for slots in each worker.
Temp. Directory:	Specifies where the temporary files go; /tmp is recommended.
Protocol:	Specifies which platform to use: LSF, SGE, RTDA, RSH, or SSH.
Command:	The command for farm submission.

Note:

Only a single thread per process is enabled with custom/unknown job schedulers.

For more information on the CDPL configuration file, see the *Common DP Library User Guide*.

Example 1

The following example distributes a job into eight processes on an SGE farm, where each process runs as a single thread.

```
primesim -dp 8 -mt 1 -dpconfig sge.cfg -o output input.sp
```

sge.cfg file example:

```
l||-l|/tmp|SGE|qsub -P bnormal -shell n -l os_version=CS6.0 -e qerr -o  
qout
```

Example 2

The following example distributes a job into ten processes on an LSF farm, where each process runs as two threads.

```
primesim -dp 10 -mt 2 -dpconfig lsf.cfg -o output input.sp
```

lsf.cfg file example:

```
l||-l|/tmp|LSF|bsub -q bnormal -n 2 -R "arch==glinux &&  
os_version==WS6_0"
```

Output Files

`output.log_p1`, `output.log_p2`, `output.log_p3` ... `output.log_p(n)` — Log file for each worker process, where n is the number of worker processes.

`output.progress` — Log file of master process, information of dispatch jobs to worker processes.

`output.dynamic` — File used for dynamic scaling, automatically deleted when the simulation finishes.

`output_dp.progress` — Log file to track the progress of each distributed task in table format. The wall time, cpu time, worker ID, hostname, status, and progress are reported for each task.

Dynamic Scaling

Dynamic scaling lets you change the number of distributed workers during the period of a simulation. The PrimeSim tool runs simulation with distributed processing CDPL mode, and when simulation starts, the tool creates a file, `output.dynamic`, where `output` is the output prefix. You can then change the number of cores/CPU's using a PrimeSim command (`output.dynamic` cannot be changed manually).

```
primesim -dscale output.dynamic -dp number
```

The `-dscale` option enables the dynamic scaling feature, and `-dp number` sets the distributed processing number that you would like to change. Successful and unsuccessful changes of this action are logged into the `output.progress` file.

Example 1

The following example distributes a job into 100 processes on an SGE farm, where each process runs as a single thread.

```
primesim -dp 100 -mt 1 -dpconfig sge.cfg -o output input.sp
```

You can use the following command to dynamically scale down the job into 20 processes.

```
primesim -dscale output.dynamic -dp 20
```

After down scaling, the simulation runs as though you had first invoked it as follows:

```
primesim -dp 20 -mt 1 -dpconfig sge.cfg -o output input.sp
```

Example 2

The following example distributes a job into ten processes on an SGE farm, where each process runs as four threads.

```
primesim -o output -dp 10 -mt 4 -dpconfig sge.cfg input.sp
```

You can use the following command to dynamically scale up the job into 50 processes.

```
primesim -dscale output.dynamic -dp 50
```

After up scaling, the simulation runs as though you had first invoked it as follows:

```
primesim -dp 50 -mt 4 -dpconfig sge.cfg -o output input.sp
```

Note:

Backward compatible for -dscale syntax:

```
primesim -dscale -dp number output.dynamic
```

Terminating Distributed Processing Simulation in an LSF Farm Environment

Use the `bkill` command to terminate distributed processing simulation in an LSF farm environment as follows:

```
bkill -s INT job_id
```

This terminates the primary job which then terminates all of the distributed processes.

Using .primesim_parallel.ini to Simplify Multi-CPU Calls

For multi-CPU runs, many common options can be stored in a `.primesim_parallel.ini` file so that you do not have to specify them from the command line every time. Options specified in a `.primesim_parallel.ini` file also take precedence over those set in the submitted netlist.

When running a multi-CPU simulation, the PrimeSim tool searches for a `.primesim_parallel.ini` file. The directory names and their priorities in searching are as follows:

1. `$PRIMESIM_HOME` – Installation directory
2. `$PRIMESIM_PROJ_CFG` – Project directory
3. `$HOME` – Home directory
4. Current working directory

The search is performed in the order listed, with the current working directory having the highest priority. The highest priority `.primesim_parallel.ini` file can incrementally add the options not yet set and also override the options set in the lower priority `.primesim_parallel.ini` files.

For example, if no `$PRIMESIM_PROJ_CFG` variable is defined, the search for `primesim_parallel.ini` is initiated in the order: `$PRIMESIM_HOME`, then `$HOME`, and then the current working directory. If a `.primesim_parallel.ini` file exists in the current working directory and contains options already specified in a lower-level `.primesim_parallel.ini` file, the options in the current working directory's `.primesim_parallel.ini` file take precedence over any `.primesim_parallel.ini` instances in `$PRIMESIM_HOME` or `$HOME`. Any unique options defined in `$PRIMESIM_HOME` or `$HOME` and not defined in the `.primesim_parallel.ini` file in the current working directory are retained.

A sample `.primesim_parallel.ini` file is provided in the `$PRIMESIM_HOME` installation directory where all possible options are listed but commented out for your reference.

Automatic Determination of Optimal Number of Parallel Processes

You can use the `-auto` option when running a multi-CPU job. When enabled, it tells the PrimeSim tool to calculate the ideal number of processes to use for a run. When `-auto` is specified, the `X` in option `-np X` then becomes the upper bound on the number of processes that can be used. When `-auto exit` is specified, it prints to output buffer and recommended number of GPU as "Recommended #GPU". This information is also available in the `.elog` file as "Recommended #GPUs{*}:"

An optimal number of processes is calculated by the PrimeSim tool. When the specified number of processes differs from the optimal number, the PrimeSim tool reports a warning. You can use this functionality with the following modes:

- Dedicated machines (single or multiple)
- Regular LSF
- SGE

This functionality is not available for `LSF_HPC`.

13

PrimeSim GPU

This chapter explains how to run the PrimeSim tool on a GPU machine. The PrimeSim tool's powerful GPU solver and efficient process management flow provides significant performance improvement on large, post-layout simulations.

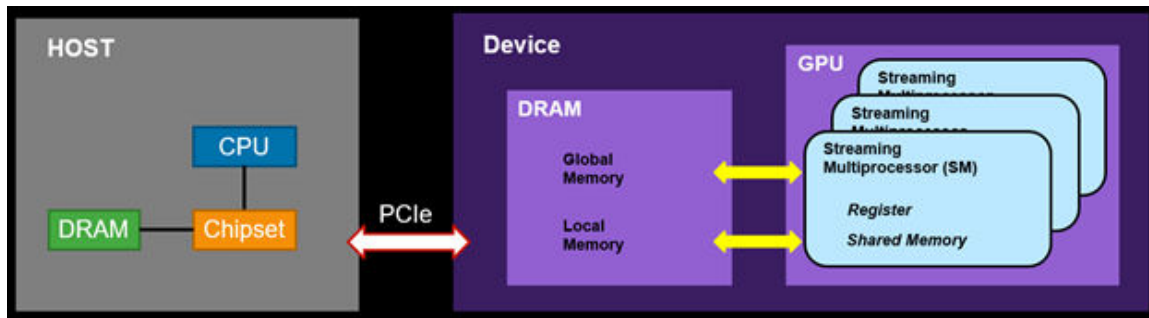
This chapter covers the following topics:

- [Introduction](#)
- [GPU Hardware and Software Requirements](#)
- [Running GPU Simulation](#)
- [GPU Card Selection](#)
- [GPU Extended Log File \(.glog\)](#)
- [CUDA Error Codes](#)
- [NVIDIA System Management Interface](#)

Introduction

The PrimeSim tool has a powerful GPU solver and efficient process management flow, which provides a significant performance improvement on large, post-layout simulations by maximizing the parallel computing capability of GPU hardware. Compared to CPUs, GPUs have massive light cores/processors, higher memory throughput, and higher memory latency.

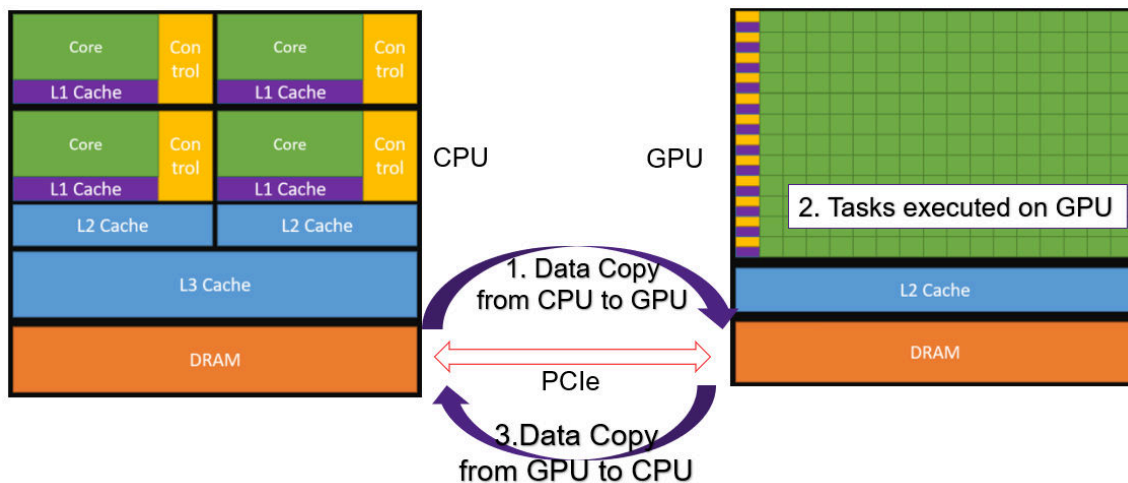
Figure 12 PrimeSim GPU Simulation Concept



The PrimeSim Pro and PrimeSim SPICE tools support GPU transient simulation and provide a simple use model. The PrimeSim tool manages the GPU flow as follows:

1. Data is copied from the CPU (Host) to the GPU (Device).
2. Tasks are executed on the GPU.
3. Results are sent from the GPU back to the CPU.

Figure 13 PrimeSim GPU Simulation Flow



GPU Hardware and Software Requirements

GPU simulation using the PrimeSim tool has the following hardware requirements:

- NVIDIA® Tesla® H100, A100, V100, or newer models with NVLink® to interconnect multi-GPU cards
- PCIe to connect GPU cards to a CPU server host
- Intel® / AMD servers

Hardware Recommended Details

- H100
 - CPU: Intel(R) Xeon(R) Gold 6448Y, @2.1 GHz (2 socket, 32 cores)
 - Memory: 1TB or more
 - GPU: NVIDIA H100 PCIeor
- A100
 - CPU: 32 cores x 3500 MHz, AMD EPYC 73F3 CPU (2 sockets, 16 cores)
 - Memory: 1TB or more
 - GPU: 4 x A100-SXM4 80GBor
- V100
 - CPU: 40 cores x 2400 MHz, Intel Xeon Gold 6148 CPU @ 2.40 GHz (2 sockets, 20 cores)
 - Memory: 1TB or more
 - GPU: 4 x Tesla V100-SXM2 32GB

Software Requirement Details

Software	Recommended Version
GPU Driver	535.183.01 and newer versions
CUDA	12.2 and newer versions
OS	AlmaLinux 8.6 and newer versions

Running GPU Simulation

As noted in [Command-Line Options for Running Multi-CPU Simulations](#), the `-ngpu` command-line option for running GPU simulation must be used with the `-np` option using the following syntax:

```
primesim -np X -ngpu Y -o output netlist.sp ...
```

where:

- X is the number of CPUs
- Y is the number of GPU cards
- Y must be $\leq X$

Note:

Use the `primesim_gpu_mode` option to set the GPU mode of operation.

Examples

In the following example, the PrimeSim SPICE tool runs on four CPUs plus one GPU card:

```
primesim -spice -np 4 -ngpu 1 -o output netlist.sp
```

In the following example, the PrimeSim SPICE tool runs on four CPUs plus four GPU cards:

```
primesim -spice -runlvl 5 -np 4 -ngpu 4 -o output netlist.sp
```

In the following example, the PrimeSim Pro tool runs on four CPUs plus four GPU cards because while eight GPU cards are specified, the number of GPU cards must be $\leq$ the numbers of CPUs so it is automatically reduced to match the specified number of CPUs:

```
primesim -mode promd -np 4 -ngpu 8 -o output netlist.sp
```

In the following example, the PrimeSim Pro tool runs on four CPUs because no GPU cards are specified and the `-ngpu` is required for GPU simulation:

```
primesim -mode spicemd -np 4 -o output netlist.sp
```

In the following example, the PrimeSim Pro tool runs on four CPUs plus two GPU, and the GPU license checkouts at beginning of the simulation:

```
primesim -np 4 -ngpu 2 -prelock_lic_gpu -o output netlist.sp
```

GPU Card Selection

The PrimeSim tool supports the following methods for GPU card selection:

- **Auto:** (default) The PrimeSim tool automatically selects the GPU card with the most available memory when the simulation begins.
- **Specified:** Use the `primesim_gpuid` option to specify the GPU card for the current process. Card IDs can be determined using the output from the [NVIDIA System Management Interface](#).

Note:

The minimum available GPU card memory requirement is 5120MB.

Example 1

In this example using the Auto method, the PrimeSim Pro tool runs on four CPUs plus the GPU card with the most available memory:

```
primesim -np 4 -ngpu 1 -o output netlist.sp
```

Log file information:

```
INFO: GPU card 0: memory total 32510 MB, free memory 27659 MB
INFO: GPU card 1: memory total 32510 MB, free memory 17750 MB
INFO: GPU card 2: memory total 32510 MB, free memory 17896 MB
INFO: GPU card 3: memory total 32510 MB, free memory 19056 MB
Used 1 GPU cards [0 - 0] (auto)
```

Because it had the most available memory, the job runs on card #0.

Example 2

In this example using the Auto method, the PrimeSim Pro tool runs on four CPUs plus two GPU cards, the one with the most available memory and the next card in order:

```
primesim -np 4 -ngpu 2 -o output netlist.sp
```

Log file information:

```
INFO: GPU card 0: memory total 32510 MB, free memory 27659 MB
INFO: GPU card 1: memory total 32510 MB, free memory 17750 MB
INFO: GPU card 2: memory total 32510 MB, free memory 17896 MB
INFO: GPU card 3: memory total 32510 MB, free memory 19056 MB
Used 2 GPU cards [0 - 1] (auto)
```

Because it had the most available memory, the job runs on card #0 and the next card in order, card #1.

Example 3

In this example using the Specified method, the PrimeSim Pro tool runs on four CPUs plus two GPU cards, the card specified by `.option primesim_gpuid=1` and the next card in order:

```
INFO: GPU card 0: memory total 40536 MB, free memory 26492 MB, ...
INFO: GPU card 1: memory total 40536 MB, free memory 28388 MB, ...
INFO: GPU card 2: memory total 40536 MB, free memory 28452 MB, ...
INFO: GPU card 3: memory total 40536 MB, free memory 28506 MB, ...
Used 2 GPU cards [1 - 2] (set)
```

Because it was specified, the job runs on card #1 and the next card in order, card #2.

Note:

- If there are fewer GPU cards available for running a job than specified by the `-ngpu` command-line option, the PrimeSim tool automatically reduces the GPU card requirement and prints a warning message in the log file.

For example, if `-ngpu 4` is specified and there are only two GPU cards available, the PrimeSim tool reduces the GPU card requirement to `-ngpu 2` and notes this in the log file with a warning message.

- If multiple GPU cards are specified, but no continuous GPU cards are available, the PrimeSim tool automatically uses the distributed GPU cards.

For example, if `-ngpu 2` is specified, but only GPU cards #1 and #3 are available, the PrimeSim tool runs the simulation on cards #1 and #3.

GPU Extended Log File (.glog)

When using GPU cards for simulation, the PrimeSim tool generates a `.glog` file that provides extended information about the current GPU process, including:

- The recommended number of GPU cards for the job
- A list of jobs also running on the same GPU card in the current process

In the following example, in addition to the current process, another job (PID 123860) is also running on GPU card #3 and is consuming more than half of the GPU card's processing capacity.

```
<<<<<< PrimeSim GPU Extended Log File >>>>>>
Recommended #GPUs: 1
20210515 15:10:28: GPU card 3 is being shared with other process
(Command:      primesim; PID:    123860; GPU Usage:   61%)
20210515 15:29:34: GPU card 3 is being shared with other process
(Command:      primesim; PID:    123860; GPU Usage:   65%)
```

CUDA Error Codes

Table 33 lists common CUDA error codes and recommended action.

Table 33 CUDA Error Codes

Error Code	Description	Suggestions for Action
=2 <code>cudaErrorMemoryAllocation</code>	Unable to allocate enough memory to perform the requested operation.	1.Check the free memory of the GPU card using <code>nvidia-smi</code> to see if there is enough free memory for the job. (Minimum required: >5120MB) 2.If there is not enough free memory, wait until memory is released or end other jobs and rerun.
=46 <code>cudaErrorDevicesUnavaible</code>	All CUDA devices are busy or unavailable at the current time.	1.Check the GPU compute mode with <code>nvidia-smi</code> to see if it is in "default" mode. 2.If it is not in "default" mode, wait until the other job completes or change the GPU compute mode to "default" with the command: <code>nvidia-smi -c 0</code>

NVIDIA System Management Interface

You can use the `nvidia-smi` command to monitor and manage (requires root admin privileges) NVIDIA GPU devices.

Use the `nvidia-smi` command to obtain the following information:

- Driver version and CUDA version
- Status of the GPU cards, such as temperature, power, memory usage, GPU utilization, and GPU compute mode
- Status of the running processes

Figure 14 shows example output from the `nvidia-smi` command:

Figure 14 Example `nvidia-smi` Output

NVIDIA-SMI 515.48.07 Driver Version: 515.48.07 CUDA Version: 11.7									
GPU	Name	Persistence-M	Bus-Id	Disp.A	Volatile	Uncorr. ECC			
Fan	Temp	Perf	Pwr:Usage/Cap	Memory-Usage	GPU-Util	Compute M.	MIG	M.	
0	NVIDIA A100-SXM...	On	00000000:01:00.0	Off	0%	0			
N/A	28C	P0	61W / 500W	3MiB / 81920MiB		Default			Disabled
1	NVIDIA A100-SXM...	On	00000000:41:00.0	Off	72%	0			
N/A	35C	P0	122W / 500W	4888MiB / 81920MiB		Default			Disabled
2	NVIDIA A100-SXM...	On	00000000:81:00.0	Off	0%	0			
N/A	27C	P0	74W / 500W	3654MiB / 81920MiB		Default			Disabled
3	NVIDIA A100-SXM...	On	00000000:C1:00.0	Off	0%	0			
N/A	27C	P0	72W / 500W	3654MiB / 81920MiB		Default			Disabled
Processes:									
GPU	GI	CI	PID	Type	Process name	GPU Memory			
ID	ID	ID				Usage			
1	N/A	N/A	36581	C	primesim_spice	4755MiB			
2	N/A	N/A	22191	C	primesim_spice	3651MiB			
3	N/A	N/A	22191	C	primesim_spice	3651MiB			

This example output provides the following information:

- The machine has four GPU cards and the compute mode (Compute M.) is in Default mode.
- Three cards are being used by the PrimeSim SPICE tool, and one card is free.

- Two jobs are running:
 - Job 36581 is running on card #1.
 - Job 22191 is running on cards #2 and #3.
- The `Volatile GPU-Util` reports the GPU tasks and resource consumption in real time.

For more command details, use `nvidia-smi -h` or visit the NVIDIA website.

Part 6: PrimeSim Features

Describes PrimeSim features.

This Part presents the following chapters:

- [Chapter 14, Back-Annotation](#)
- [Chapter 15, Circuit Checks](#)
- [Chapter 16, Tcl Interactive Mode](#)
- [Chapter 17, Monte Carlo Analysis](#)
- [Chapter 18, Multi-Technology Simulation of 3-D Integrated Circuit \(3DIC\)](#)

14

Back-Annotation

This chapter describes how to use post-layout simulation with back-annotation in the PrimeSim tool.

A successful simulation for a pre-layout netlist does not necessarily mean you get a successful simulation after layout and RC parasitics are taken into account. Full-chip transistor-level simulation considering the post-layout parasitics has therefore become an essential step for a successful design.

The PrimeSim Pro tool supports post-layout simulation by allowing pre-layout netlists to be back-annotated with the extracted interconnect RC parasitics as well as extracted device parameters. the PrimeSim Pro tool can read interconnect parasitics from Detailed Standard Parasitic Format (DSPF) files and extracted device parameters from Device Parameter Format (DPF) files or DSPF files.

No matter what back-annotation method you use in the PrimeSim Pro tool, you can achieve the most accurate results for post-layout simulation, including dynamic voltage drop, and EM violations, to complete the design verification successfully at the transistor level.

Back-annotation is further described in the following sections:

- [Setting Defaults for DSPF and DPF Options](#)
- [DSPF Annotation](#)
- [SPEF Annotation](#)
- [DPF Annotation](#)
- [SPF RC Reduction](#)
- [Active Net Flow](#)

Setting Defaults for DSPF and DPF Options

The PrimeSim Pro tool supports setting default values for DSPF and DPF-related options. DSPF and DPF options apply to the DSPF or DPF file they follow. When multiple DSPF

or DPF files are back-annotated, the options must be repeated for each file. Consider the following example:

```
.option primesim_spf="file1.spf"
.option primesim_spfrmin=0

.option primesim_spf="file2.spf"
.option primesim_spfrmin=0

.option primesim_spf="file3.spf"
.option primesim_spfrmin=.1
```

Because it is often the case that the same options should be applied to all DSPF or DPF files, the DSPF or DPF options that come before the first file change the default value for all of the files. This allows one to achieve the same effect as in the previous example with the following code excerpt:

```
* Set the defaults for DSPF
.option primesim_spfrmin=0

* DSPF files
.option primesim_spf="file1.spf"
.option primesim_spf="file2.spf"
.option primesim_spf="file3.spf"
.option primesim_spfrmin=.1 $ Override rmin for file3.spf
```

The PrimeSim Pro tool supports temperature-dependent DSPF resistors with the following options:

- `primesim_spfrtc1` – You can use this option to set the `tc1` parameter for DSPF resistors. The default value is 0.
- `primesim_spfrtc2` – You can use this option to set the `tc2` parameter for SPF resistors. The default value is 0.
- `primesim_spftref` – You can use this option to set the reference temperature for DSPF resistors. The default value is 25.

When these options are specified, the resistances in the DSPF file are scaled according to the following formula:

$$(1.0 + (\text{temp} - \text{tref}) * \text{tc1})$$

Consider the following example:

```
.option primesim_spf="file1.spf"
.option primesim_spftref="30"
.option primesim_spfrtc1=.0035
```

or:

```
.option primesim_spfrtc2=5.34e-7
```

DSPF Annotation

To support post-layout simulation, the PrimeSim Pro tool supports back-annotating interconnect parasitics contained in a DSPF file onto a pre-layout netlist.

A DSPF file contains networks of RC parasitics that correspond to nodes in the ideal netlist. Because annotating the full RC network for all of the nets in a DSPF file can significantly slow down a simulation, the PrimeSim Pro tool is flexible on how the DSPF is read, annotated and reduced. Depending on the accuracy requirements, on a net-by-net basis, the PrimeSim Pro tool can annotate a nodal network as full RC trees, a single lumped capacitance or with no back-annotation at all.

During DSPF reading, the following characters are not allowed in net names in `*|NET` (net definition): `( ) * + / \ @ .`

For RC tree parasitics, the PrimeSim Pro tool can further reduce the RC tree with different levels of constraints, such as model order reduction. Some floating (cross-coupling) capacitors and resistors that are either too small or too large can be eliminated from the back-annotation process to improve performance without compromising accuracy much. For power rail back-annotation, the challenge is the vast RC networks and their impact on voltage drops along the rail. The PrimeSim Pro tool has a special separate-partitioning-synchronizing-event-control algorithm to analyze this non-ideal power scenario to achieve the most accurate results in the industry.

Parasitic extraction tools can generate a flat or hierarchical DSPF file. A flat DSPF is the result of a full extraction, and contains parasitic networks at the lowest level of the design connected directly to basic devices such as transistors. In contrast, a hierarchical DSPF comes from extracting only the higher-level, for instance, gate level, interconnect parasitics. This hierarchical approach is useful for simulation of a standard-cell based design where a cell library with pre-extracted parasitics at the transistor level for each cell is already available.

When the PrimeSim Pro tool does back-annotation, either in a flat or hierarchical DSPF, onto a pre-layout netlist, it automatically and intelligently flattens the design as necessary to add the parasitics at the appropriate level. In the case of a flat DSPF, back-annotation may end up completely flattening the design to transistors and therefore disable the PrimeSim Pro tool ability to take advantage of the adaptive hierarchy database (`primesim_hiersim` option) because no subcircuit modules are identical anymore.

The PrimeSim Pro tool supports the use of different MOS models between pre-layout netlists and SPF. The SPF model is used to perform simulation.

The PrimeSim Pro tool provides the following options you can use for DSPF annotation:

Table 34 DSPF Annotation Commands

Command	Description
<code>primesim_enprefix</code>	Specifies the prefix characters in a pre-layout netlist that should be removed to match with the DSPF instance name during back-annotation.
<code>primesim_eqr2spres, primesim_spresout</code>	Enables and specifies the output format for the SPRES solver. The SPRES solver is a faster means of calculating equivalent resistance (EQR) of resistor networks from the SPF file and is used in conjunction with: <code>primesim_spfeqr</code> , <code>primesim_spf2eqr</code> , <code>primesim_spfeqrfile</code> , <code>primesim_spfeqr_only</code> , <code>primesim_spf2eqr_select</code> .
<code>primesim_gpd</code>	Back-annotates a corner used in the Galaxy Parasitic Database (GPD).
<code>primesim_repdot</code>	Replaces a period (.) with another character.
<code>primesim_simple_em_naming</code>	Uses the period (.) as a single hierarchical delimiter when periods (.) and slash (/) characters are mixed in signal names. This is used in EM analysis.
<code>primesim_spf</code>	Specifies the DSPF file that contains SPF data extracted from the layout interconnects.
<code>primesim_spf_acc_irem</code>	Performs dynamic EM simulation of a huge back-annotated circuit with better performance than standalone IR/EM.
<code>primesim_spf_add_netpin</code>	Allows you to add a small number of netpins in a DSPF file.
<code>primesim_spf_add_netpin_file</code>	Allows you to add a large number of netpins in a DSPF file.
<code>primesim_spf_add_irem_window</code>	Specifies analysis time range for the IR/EM .em file.
<code>primesim_spf_allow_rc_parasitic_instance</code>	Allows you to program the RC net of a Standard Parasitic Format (SPF) file with a custom subckt module in order to estimate the RC effect of that SPF NET.
<code>primesim_spf_bus_char</code>	Used when extracted SPEF file bus notation is different from schematic netlist bus notation.
<code>primesim_spf_cc_parse</code>	Specifies whether to parse all coupling capacitor NETs even if one of the NETs is not found in the schematic netlist.
<code>primesim_spf_delete_netpin</code>	Allows you to delete a small number of netpins in a DSPF file.
<code>primesim_spf_delete_netpin_file</code>	Allows you to delete a large number of netpins in a DSPF file.
<code>primesim_spf_exclude</code>	Excludes the specified instances from SPF back-annotation.
<code>primesim_spf_fingerchars</code>	Translates the special SPF finger delimiter from the specified string to the at-sign (@).

Table 34 *DSPF Annotation Commands (Continued)*

Command	Description
<code>primesim_spf_finger_res_policy</code>	Specifies whether finger resistors must be in parallel to the base resistor or are allowed to be a series or a mix of parallel and sequential order.
<code>primesim_spf_histogram</code>	Specifies whether to print an RC Histogram for each DSPF file during back-annotation.
<code>primesim_spf_layout_only_device_models</code>	Specifies the device model names in the DSPF file to be added to the back-annotation simulation.
<code>primesim_spf_layout_only_net</code>	Use this option to specify one or more NETs which exist only in layout so that you can apply RC annotation options to them.
<code>primesim_spf_map_subckt_port</code>	Sets mapping between the port names of the schematic file and the pin names of the back-annotation file.
<code>primesim_spf_matcheffort</code>	Performs more rigorous naming matching for SPF net and devices.
<code>primesim_spf_port_name_check</code>	Specifies whether to check for multiple instances of port names connecting to another NET.
<code>primesim_spf_removetoprc</code>	Removes the top-level parasitics in the <code>primesim_spfinst=2</code> back-annotation flow.
<code>primesim_spf_sch_check_mode</code>	Turn this option on in back-annotation simulations when the schematic netlist changes faster than layout and DSPF files can be generated.
<code>primesim_spf_sensitive</code>	Supports case sensitivity for netlist back-annotation flow.
<code>primesim_spf_skip_res_for_ivsrc</code>	Tests to see whether any SPF nets for RC annotation are connected to any current sources or voltage sources. If so, all of the resistors for those nets are shorted automatically.
<code>primesim_spf_spice_names</code>	Specifies whether the instance names in the DSPF file include SPICE type prefix.
<code>primesim_spf_user_defined_gnd_list</code>	Specifies the node list that should be treated as ideal ground in SPF back-annotation.
<code>primesim_spfallowerror</code>	Continues simulation even when there is a parsing error of SPF.
<code>primesim_spfcnet</code>	Specifies the net names to be annotated with lumped capacitance only.
<code>primesim_spfeqr, primesim_spf2eqr, primesim_spfeqrfile, primesim_spfeqr_only, primesim_spf2eqr_select</code>	Controls equivalent resistance (EQR) analysis of a DSPF file. To use the SPRES solver, see <code>primesim_eqr2spres</code> , <code>primesim_spresout</code> .
<code>primesim_spffcmin</code>	Sets the minimum floating capacitor value allowed for back-annotation.

Table 34 *DSPF Annotation Commands (Continued)*

Command	Description
<code>primesim_spffcnct</code>	Keeps the floating capacitance for C-only back-annotation.
<code>primesim_spfirst</code>	Controls whether the instance section of DSPF file is used for back-annotation.
<code>primesim_spfkeepnodecap</code>	Controls whether to keep or remove the pre-layout ground capacitance of back-annotated nets.
<code>primesim_spfmergeport</code>	Merges ports in a DSPF net.
<code>primesim_spfmodule</code>	Specifies the module for SPF back-annotation.
<code>primesim_spfnonet</code>	Specifies the nets that are not going to be back-annotated by DSPF data.
<code>primesim_spfpost</code>	Sets the node names for post analysis of DSPF back-annotated simulation.
<code>primesim_spfpost_out_only</code>	Use this option to probe the internal nodes of power/ground during EM simulation.
<code>primesim_spfprb</code>	Sets the probe mode for DSPF nodes.
<code>primesim_spfprb_finger</code>	Specifies whether to probe fingered device pins based on the <code>primesim_spfprb</code> option setting.
<code>primesim_spfprb_mode</code>	Specifies whether to flatten an SPF-annotated node name.
<code>primesim_spfprb_sync</code>	Specifies whether the probed NET's device's pin nodes or internal subnodes are probed.
<code>primesim_spfprefix</code>	Lists the prefixes that should be removed from device names.
<code>primesim_spfrcnct</code>	Specifies the net names that are annotated with RC trees.
<code>primesim_spfregate</code>	Picks the first GATE pin defined at <code>(*) </code> / <code>(*) NET</code> from the DSPF file, only if the GATE pin exists under <code>(*) NET</code> from the DSPF file.
<code>primesim_spfreplast</code>	Controls how the representative node is chosen when annotating DSPF.
<code>primesim_spfrgcnet</code>	Specifies the net names that are annotated with parasitic resistors and ground capacitors.
<code>primesim_spfrmax</code>	Sets the maximum resistor value allowed for back-annotation.
<code>primesim_spfrmin</code>	Sets the minimum resistor value allowed for back-annotation.
<code>primesim_spfrnet</code>	Specifies the net names to be annotated with parasitic resistor only.
<code>primesim_spfrptrmax</code>	Sets the warning threshold number for SPF resistors.

Table 34 *DSPF Annotation Commands (Continued)*

Command	Description
<code>primesim_spfscale</code>	Sets the scale for devices in the DSPF file.
<code>primesim_spfspef</code>	Sets <code>primesim_spf</code> input to be DSPF or SPEF format.
<code>primesim_spfsplitnet</code>	Splits nets into multiple nets after layout is complete.
<code>primesim_spfsuffix</code>	Defines the suffix in an SPF file.
<code>primesim_spfwarn</code>	Controls warning message output when performing and debugging back-annotation.

Option Precedence

The PrimeSim Pro tool adheres to the following rules in regard to the precedence of DSPF annotation-related options:

1. The precedence is defined as follows with `primesim_spfnonet` being at the highest level:
 - a. `primesim_spfnonet`
 - b. `primesim_spfrcnet`
 - c. `primesim_spffcnet`
 - d. `primesim_spfcnet`
2. The exact net name match without wildcards takes precedence over the match with wildcards. For example, with the following commands, net `A` is annotated with lumped capacitance only while all the other nets are annotated with RC trees.

```
.option primesim_spfrcnet='*'
.option primesim_spfcnet= 'A'
```

Probing Internal Nodes

When DSPF is back-annotated onto a design, additional nodes are created. What was one node in the original design becomes an RC network with additional nodes. The PrimeSim Pro tool supports `.measure`, `.probe`, and `.print` statements involving nodes from the DSPF file. For example, consider the following example where DSPF nets are annotated:

```
*|NET B 0.00269325PF
*|I (XND5/MXM2:GATE XND5/MXM2 GATE I 3e-15 0 0)
*|P (B B 0 0.565 9.685)
*|I (XND5/MXM4:GATE XND5/MXM4 GATE I 2e-16 0 0)
Cg1 XND5/MXM2:GATE GND 2e-16
```

```
R1 XND5/MXM2:GATE B 100
R2 XND5/MXM2:GATE XND5/MXM4:GATE 1000
R3 B XND5/MXM4:GATE 200

*|NET XND5/NET2 0PF
*I (XND5/MXM2:DRN XND5/MXM2 DRN B 0 0 0)
*I (XND5/MXM0:SRC XND5/MXM0 SRC B 0 0 0)
R4 XND5/MXM2:DRN XND5/MXM0:SRC 0.001
```

The PrimeSim Pro tool supports statements such as the following examples:

```
.measure a avg v(xnd5.mxm2:gate) FROM=10ns TO=20ns
.probe v(xnd5.mxm2:gate)
```

Notice that for hierarchical node names, the hierarchy separator must be changed to a period (.)

Back-Annotation Support Using Expressions

The PrimeSim tool supports back-annotation of resistor/capacitor values using expressions that can include:

- Controlled voltage probes. For example:

```
R3 B XND5/MXM4:GATE r='8.33333e-06*v(B,XND5/MXM4:GATE)'
```

The two nodes of the voltage probe should be either separated by a comma "," or hyphen "-".

- Built-in functions. For example:

```
C3 C XND5/MXM2:GATE c='6.88359e-16*tanh(2)'
```

SPEF Annotation

The PrimeSim Pro tool supports back-annotation for the SPEF file through the use of the `primesim_spfspef` option. The syntax and options are otherwise the same as [DSPF Annotation](#).

DPF Annotation

In addition to interconnect parasitics, extraction tools can also extract the device parameters from layout and save these in a Device Parameter Format (DPF) file. The PrimeSim Pro tool can annotate these device parameters onto the pre-layout netlist. If there are device parameters already in the pre-layout netlist, those in DPF file override pre-layout values for a correctly annotated simulation.

The PrimeSim Pro tool provides the following options you can use for DPF annotation:

Table 35 *DPF Annotation Commands*

Command	Description
<code>primesim_dpf</code>	Specifies the DPF file that contains the extracted device parameter data.
<code>primesim_dpf_mos_only</code>	Specifies whether to back-annotate all devices or only MOSFET devices.
<code>primesim_dpfsadddev</code>	Controls whether DPF devices that are not in the netlist are added.
<code>primesim_dpfdhdiv</code>	Sets the divider character for hierarchical names in DPF files.
<code>primesim_dpfsmodule</code>	Specifies the module for DPF back-annotation.
<code>primesim_dpfsprefix</code>	Lists the prefixes that should be removed from DPF device names.
<code>primesim_dpfscale</code>	Sets the scale for devices in the DPF file.
<code>primesim_dpfsuffix</code>	Defines the suffix in a DPF file.

SPF RC Reduction

During SPF back-annotation, you can select different thresholds and settings to trim the RC elements of each individual SPF file. The following are list of equivalent options for the SPF file:

- `primesim_spfmin` – Shorting of small resistors
- `primesim_spffmin` – Splitting of small coupling capacitors

The RC reduction for SPF is done during the back-annotation, and the fully annotated netlist then goes through RC reduction set by `primesim_spred`.

Active Net Flow

The active net flow runs a pre-layout simulation and generates the active net information to use in the back-annotation simulation.

.spf_active_ba

Description

Enables the active net back-annotation flow based on the activity of pre-layout design.

Syntax

```
.spf_active_ba mode="auto|read|write|force" dv=value start=tstart_time  
stop=tstop_time file=file_name exclude="net_name1 net_name2..."
```

Arguments

Argument	Description
mode="auto read write force"	Specifies one of the following modes: • auto: Specifies to do pre-layout simulation, create the activity file, and run simulation with those results. If the activity file is not present, it is created. • read: Specifies to read the activity file provided by <code>file=</code> or use the default file name and do the active net simulation directly. • write: Specifies to only run pre-layout simulation and writes out an activity file provided by <code>file=</code> or use the default file name. • force: Specifies to do a pre-characterization run and create the activity file irrespective of the file already available and runs active net simulation.
dv=value	Specifies the voltage tolerance. A node is considered active if the node value varies more than the specified value. The default is 100mV.
start=tstart_time	Specifies the time to start checking activity. The default is simulation start time.
stop=tstop_time	Specifies the time to stop checking activity. The default is simulation end time.
file=file_name	Specifies the user-defined file name for the activity file. The default is the simulation output prefix name in the following format: <pre>output_name.spf_inactive</pre> Note: If the path to the activity file includes commas, you must include quotation marks around the entire string.
exclude="net_name1 net_name2..."	Exclude the specified nets from the active net check.

Examples

```
.spf_active_ba mode=auto dv=200m start=10n stop=20n
```

Runs active net back-annotation automatically by first creating the activity file. If this file is not already present, run SPF back-annotation simulation of only the active nets.

Nodes whose values change by more than 200mV in the time interval of 10n to 20n are considered active nets.

```
.spf_active_ba mode=write file=active_file
```

Runs pre-layout simulation to create only the `active_file` activity file. It does not perform active net annotation automatically.

```
.spf_active_ba mode=read file=active_file
```

Runs active net simulation using the specified `active_file` activity file.

```
.spf_active_ba mode=auto
```

Same as the first example, but it considers `dv=100m` for the entire simulation to create the active file.

```
.spf_active_ba mode=auto exclude="vdd vss"
```

Same as previous example, but excludes the `vdd` and `vss` nets from the active net check.

```
.spf_active_ba file=active_file mode=force
```

Forces active net file generation, even the `active_file` file is already present.

15

Circuit Checks

This chapter contains information on various circuit check commands.

This chapter covers the following topics:

- [Circuit Check Commands](#)
- [Circuit Check Post-Processing Options](#)

Circuit Check Commands

The circuit check commands are:

- [Check Active/Inactive Nodes \(.CHKANODE\)](#)
- [Check Block Power \(.CHKBLKPWR\)](#)
- [Check DC Path \(.CHKDCPATH\)](#)
- [Check Leakage Current Path \(.CHKDCPATH, zgate=on\)](#)
- [Check Device Current \(.CHKDEVCUR\)](#)
- [Check Device Operation Point \(.CHKDEVOP\)](#)
- [Check Expression \(.CHKEXPR\)](#)
- [Check OFF Device Leakage Current \(.CHKOFFDEVLEAK\)](#)
- [Check Rise/Fall Transition Time \(.CHKRFTIME\)](#)
- [Check Signal Voltage Difference \(.CHKSIGDIFF\)](#)
- [Static Circuit Checks \(.CHKSTATICERC\)](#)
- [Check Timing Setup/Hold/Delay/Width \(.CHKTIMING\)](#)
- [Check and Report Toggle Count \(.CHKTOGGLE\)](#)
- [Check High Impedance State Node \(.CHKZNODE\)](#)

Check Active/Inactive Nodes (.CHKANODE)

Description

This command is used to check both the inactive and active nodes of the circuit.

Syntax

```
.chkanode [file=filename] [subckt=subckt_name]  
+ [name=nodename] [level=hierarchy_level_to_match]  
+ [start=start_time] [stop=stop_time]  
+ [dv=threshold_voltage] [type=all|gate]  
+ [report=none|inactive|active|both] [format=rcxt]
```

Arguments

Argument	Description
dv	The threshold of voltage change. The default value is 0.1v.
file	Specifies the output file name.
format	Specifies that the output is formatted for StarRC selective net extraction.
level	Specifies the hierarchy level. If level=-1, the PrimeSim Pro tool checks all hierarchy depth.
name	Check a specific node if it is active or inactive.
report	- inactive: (default) All nodes which are changed less than dv are reported. - active: If a node is changing more than dv for the time-range, then the node would be reported as active. - both: Both inactive nodes and active nodes are reported.
start	Specifies the start-time of the window.
stop	Specifies the stop-time of the window.
subckt	Specifies the name of the subcircuit.
type	- gate: (default) Check only nodes that is connected to transistor's gate. - all: Check all nodes.

Examples

```
.chkanode type=all dv=0.3
```

If the maximum voltage change of a node is lesser than 0.3v, the PrimeSim tool reports this node as inactive in the report and all nodes are checked.


```
.chkanode type=all dv=0.3 name=x156 report=both
```

The above command checks whether node x156 is active or inactive.

```
.chkanode dv=0.1 type=all report=active format=rcxt
```

The above command formats its output for StarRC selective net extraction.

Check Block Power (.CHKBLKPWR)

Description

This command is used for checking block power. Multiple blocks can be specified with one command by using a wildcard with the block parameter, rather than using multiple `.chkblkpwr` commands.

The command also reports the maximum (max) and minimum (min) current, in addition to the average (avg) and root mean square (rms) current, in the block report section.

The `.chkblkpwr` circuit check also honors the `primesim_ignore_chkfunc_error` option to ignore invalid circuit check commands.

Note:

Performance can slow down because of affected partitioning/filtered devices. You should selectively keep the number of checked blocks and their depth of hierarchy to the minimum necessary by using the command arguments `at_depth`, `depth`, and `block` in order to lessen the performance impact.

For more information, see the [SolvNetPlus article, Simulation Performance Impact of .CHKBLKPWR Command in FineSim Pro Mode](#).

You must be logged in to SolvNetPlus for the preceding link to connect directly to the article. If you are prompted to log in to SolvNetPlus upon clicking the link to the article, log in, then click the link again to reach the article.

Note:

This command is not supported when the `primesim_output=none` option is used.

Syntax

```
.chkblkpwr [pth=real] [block=instance] [depth=value]  
+ [start=time] [stop=time] [type=hier|block|both]  
+ [file=filename] [at_depth=n] [tag=tagname] [iunit=m|u|p|f]  
+ [multi_supply=0|1]
```

Arguments

Argument	Description
<code>at_depth</code>	Reports only level <i>n</i> instances. For example, <code>at_depth=3</code> reports all <code>*.*.*</code> instances not including <code>*</code> and <code>*.*</code> .
<code>block</code>	Reports one or more top-level blocks.
<code>depth</code>	Depth into hierarchy. Only one occurrence of this command is permitted.
<code>file</code>	Specifies the output file name.
<code>iunit</code>	Specifies the current value unit: • <code>m</code>: 1e-3 • <code>u</code>: 1e-6 • <code>p</code>: 1e-9 • <code>f</code>: 1e-12
<code>multi_supply</code>	Reports multiple supply powers independently in a hierarchy report when <code>multi_supply=1</code> . Default is 0.
<code>pth</code>	Specifies the power or current threshold.
<code>start</code>	Specifies the start time of the window.
<code>stop</code>	Specifies the stop time of the window.
<code>tag</code>	Defines the tag name of the result. If <code>tag</code> is not specified, the PrimeSim tool uses <code>[blockname]_[depth]_[type]</code> as the default tag name.
<code>type</code>	Report type: <code>hierarchy</code> , <code>block</code> , or <code>both</code> . Default is <code>both</code> .

Check DC Path (.CHKDCPATH)

Description

This command is used to check the DC current path (IPath) among voltage sources in the circuit. The generated output is `prefix.filename.chkdcpath`.

Note:

Checking the DC path through a MOSFET's Source/Drain to Bulk forward bias condition results in inordinate runtimes, so the PrimeSim tool skips this search.

Syntax

```
.chkdcpath [name="sig_name1 sig_name2 sig_name3 ..."]
+ [ith=threshold_current] [ithsub=threshold_sub_current]
+ [tag=tag_name] [file=filename]
+ [repeat_report=[0|1]] [subckt=subckt_name[:0|1]]
```

```
+ [instance=instance_name[:0|1]] [pwr_off=0]
+ [zgate=on] [exclude_znode="node1 node2 node3 ..."]
+ [conductive=on] [xgate=on]
+ [gnd_vth=xx] [start=time_val] [stop=time_val]
+ [include|exclude=name] [waivefile=file_name]
+ [period=time | interval=time | delay=time |
+ at=time|(time1,time2,time3...)]
+ [zperiod=time] [gperiod=time] [xperiod=time]
```

Arguments

Argument	Description
at	When this parameter is used, the DC path check is performed at the specified times.
conductive	Reports the pure conductive path (GPath).
delay	When this parameter is specified, the DC path is checked at each time defined as <i>t</i> +specified <i>delay</i> . <i>t</i> is the time at which an input voltage source changes to a new voltage level.
exclude_znode	See Check Leakage Current Path (.CHKDCPATH, zgate=on) .
file	Specifies the output file name.
gnd_vth	The threshold used in <i>pwr_off</i> to determine power down state. Default is 0.
gperiod	This option is used with <i>conductive=on</i> . When you specify this option, the pure conductive path (GPath) is reported only if the conductive state of device exceeds <i>gperiod</i> threshold. The default is 1n.
include exclude	- include=<i>name</i>: Checks only the include device in the circuit check. - exclude=<i>name</i>: Excludes the device in the circuit check.
instance	Specifies the instance name to perform DC path analysis. You can append [0 1] to indicate if it is an include or exclude. Wildcards are supported.
interval	When this parameter is specified, the DC path is checked for every specified interval.
ith	Defines threshold current. The default value is 50uA.
ithsub	When defined and greater than 0, then the substrate current DC path checker is executed.
name	Defines node names. This option needs two or more node names.

Argument	Description
<code>period</code>	Specifies that the DC path reports any current path violation that exceeds period threshold. The default is <code>1n</code> if the <code>interval</code>, <code>at</code>, or <code>delay</code> options are not specified (<code>period</code>, <code>interval</code>, <code>at</code>, and <code>delay</code> cannot be used together). Note: When using <code>period</code>, the same DC path is reported only for the first occurrence (reducing the same DC path in multiple time points).
<code>pwr_off</code>	When set to <code>1</code>, the <code>.CHKDCPATH</code> command checks if the nodes specified under <code>name</code> have paths to voltage sources. If not, these nodes are skipped and a warning is given. For all other nodes, if the voltage is less than the <code>gnd_vth</code> parameter, it is treated as a ground and ignores all DC path to ground (a warning is also given). Default is <code>0</code>.
<code>repeat_report</code>	By default (<code>0</code>), the <code>.CHKDCPATH</code> command reports only the first occurrence of a violation during transient analysis. When set to <code>1</code>, the <code>.CHKDCPATH</code> command reports all violation occurrences during transient analysis.
<code>start</code>	Specifies the start-time of the window. Default is <code>0</code>.
<code>stop</code>	Specifies the stop-time of the window. Default is the minimum of <code>tstop</code> of <code>1s</code>.
<code>subckt</code>	Specifies the subcircuit to perform DC path analysis. You can append <code>[0 1]</code> to indicate if it is an include or exclude. Wildcards are supported.
<code>tag</code>	Defines the tag name of the result.
<code>waivefile</code>	Specifies the file containing the device list of waived results.
<code>xgate</code>	Reports the DC path (XPath) when the MOSFET Gate node's state is unknown and over <code>xperiod</code>.
<code>xperiod</code>	This option is used with <code>xgate=on</code>. When you specify this option, the DC path (XPath) is reported only if the MOSFET Gate node's state is unknown and exceeds <code>xperiod</code> threshold. The default is <code>1n</code>.
<code>zgate</code>	See Check Leakage Current Path (.CHKDCPATH, zgate=on).
<code>zperiod</code>	This option is used with <code>zgate=on</code>. When you specify this option, the Hi-Z path is reported only if the Hi-Z mode exceeds <code>zperiod</code> threshold. The default is <code>1n</code>.

Examples

```
.chkdcpth name="BATT G" ith=100n delay=0
```

The above command conducts DC path check from node `BATT` to `G`. If the current value of a path is larger than 100nA, the PrimeSim Pro tool regards this path as error and put it in the output file.

Check Leakage Current Path (.CHKDCPATH, zgate=on)

Description

The PrimeSim Pro tool includes a floating gate check for leakage paths. Use option `zgate=on` with `.chkdcpath` to support `cckAnalogPDown` in the legacy HSPICE tool.

In addition to `exclude_znode`, all other `.chkdcpath` options can be used.

`zgate=on` with `.chkdcpath` uses the same MOSFET on/off status rules as `.chkznode`.

Syntax

```
.chkdcpath zgate=on exclude_znode="node1 node2 ..." ...
```

Arguments

Argument	Description
<code>exclude_znode</code>	Excludes the Hi-Z DC Path if the specified Hi-Z nodes are connected to the gate which is a part of Hi-Z DC Path. Wild cards (* or ?) are supported for Hi-Z node names.

Examples

```
cckAnalogPDown tag=chk_2 time=5n time=7n vsrc=vdd vsrc=gnd  
cckAnalogPDownIth 10u
```

The legacy HSPICE commands in the example above can be equivalently expressed in the PrimeSim Pro tool as follows:

```
.chkdcpath zgate=on tag=chk_2 at=(5n,7n) name="vdd gnd" ith=10u
```

This command conducts a power-down leakage-path check at both 5.0ns and 7.0ns. It then reports DC paths with leakage currents larger than the default 10uA current flowing from vdd to gnd, or the conducting paths with devices driven by a Hi-Z node.

Check Device Current (.CHKDEVCUR)

Description

This command checks whether the current of a device exceeds a given threshold and reports all violations in two log files: the main file that reports violations on every time step, and the summary file that reports the average (by default) or RMS (root-mean-square) of

the window-based violations per device. The main file can get very large, so you can set `primesim_chkdevcur_report=0` to generate only the summary file.

Syntax

```
.chkdevcur start=start_time stop=stop_time tag=tagname
+ ith=threshold_current [avg=1] [rms=1]
+ [model=modelpattern] [subckt=subckt_name] [name=device_name]
```

Arguments

Argument	Description
avg	Reports the average current of the device if there are any violations. This is the default option.
ith	Threshold current, device current (<code>rms</code> and/or <code>avg</code>) exceeding this number is reported.
model	Specifies the model pattern filter. Only models matching the given pattern are reported. Supports wildcard matching.
name	Specifies the device instance name. Supports wildcard matching.
rms	Reports the RMS (root-mean-square) current of the device if there are any violations.
start	Specifies the start time of the check window.
stop	Specifies the stop time of the check window.
subckt	Specifies the subcircuit name. If specified, the command checks the device current under that subcircuit. Does <i>not</i> support wildcard matching.
tag	Each violation has a tag name when reported. Violations sharing the same tag are summarized in a separate log file with that tag. If the user does not supply a tag, the tag name is <code>default</code> and a warning is issued.

Examples

```
.chkdevcur start=0 stop=400n tag=avg_tag ith=200n avg=1
```

Note:

The center log file's name is `*.chkdevcur`. Log files for separate tags are named `*.tagname.chkdevcur`.

Check Device Operation Point (.CHKDEVOP)

This command checks the operating point of a device such as MOSFET, diode, and bipolar transistors and issues warnings if the operating point falls outside the user-specified safe operation range or meets a user-specified violation condition.

Syntax

```
.chkdevop [type=m|q|j|d]c|r]
+ [model=modelname [modelname2...]]
+ [subckt=subckt_name] [name=instance_name]
+ [level=none|warn|error]
+ [lmin=min_length] [lmax=max_length]
+ [wmin=min_width] [wmax=max_width]
+ [period=period_time] [tag=tag_name]
+ [start=start_time] [stop=stop_time] [file=filename]
+ [report=[0|1|2]]
+ Vab=(min_value,max_value)... [when Vab=(min_value,max_value)...]
+ [message="user_message"]
OR
.chkdevop [type=m|q|j|d]c|r]
+ [model=modelname [modelname2...]]
+ [subckt=subckt_name] [name=instance_name]
+ [level=none|warn|error]
+ [lmin=min_length] [lmax=max_length]
+ [wmin=min_width] [wmax=max_width]
+ [period=period_time] [tag=tag_name]
+ [start=start_time] [stop=stop_time] [file=filename]
+ [report=[0|1|2]]
+ cond="expression"
+ [message="user_message"]
```

Arguments

Argument	Description
cond	Specifies a violation condition in <i>expression</i> .
file	Specifies the output file name.
level	Reports warnings/errors when a .CHKDEVOP violation occurs: • none: Default. • warn: Generate warning. • error: Error out the PrimeSim tool.
lmax	Defines the maximum MOSFET length. Default is infinite.
lmin	Defines the minimum MOSFET length. Default is 0.

Argument	Description
message	Specifies a custom message. Note: To get a user message in the log file, set <code>level=warn</code> . For example: <code>.chkdevop vgs=(-0.1,0.8) period=1n message="Vgs exceeds 0.8V" level=warn</code>
model	Sets the model name of the devices.
name	Sets the instance name of the devices.
period	If any device is staying in the violation state longer than period, it is reported in the output file.
report	0: (default) Specifies the original format 1: Specifies detail table format with violation 2: Specifies detail format with violation in one line
start	Specifies the start-time of the window.
stop	Specifies the stop-time of the window.
subckt	Specifies the subcircuit name to be checked.
tag	Defines the tag name of the result.
type	- m: MOSFET - q: Bipolar transistor - d: Diode - j: Junction transistor - c: Capacitor - r: Resistor
Vab	Can be any of the following: - Vd, Vg, Vs, Vb, Vds, Vgs ... for MOSFET - Ve, Vb, Vc, Vs, Vec, Vbc ... for BJT - Vp, Vn, Vpn, Vnp for Capacitor, Resistor, and Diode - Vp, Vn, Vc, Vpn, Vnp, Vpc, Vnc, Vcp, Vcn for R3 CMC Resistor Model
when	Specifies an additional condition for checking the device operation range.
wmax	Defines the maximum MOSFET width. Default is infinite.
wmin	Defines the minimum MOSFET width. Default is 0.

Description

For this command conditional expressions can have operators such as `>`, `>=`, `<=`, `!`, `&&`, and `||`. The output file has a `.chkdevop` extension.

Examples

```
.chkdevop model="nch* pch*" vds=(-2,2) vgs=(-2,2) vbs=(-2,2)
.chkdevop type=q vcb=(-1,1) vbe=(-1,1)
.chkdevop model='d*' type=d vpn=(-0.7, 0) vnp=(-0.1,0)
.chkdevop type=j vdb=(-0.1,0) vgd=(0,1) vbg=(0,0.5) file='jfet'
```

For the preceding examples, if the operating point is out of the given range, the PrimeSim Pro tool reports it as violation.

Here are the supported terminals for .chkdevop.

Devices	Description	Terminals
Capacitor	c	P: positive node N: negative node
Diode	d	vpn: forward bias voltage threshold vnp: reverse bias voltage threshold
Junction Transistor	j	G: gate D: drain S: source B: bulk
MOSFET	m	G: gate D: drain S: source B: bulk
Resistor	r	P: positive node N: negative node
Bipolar Transistor	q	B: base C: collector E: emitter S: substrate

Check Expression (.CHKEXPR)

This command checks a custom script during the simulation.

Syntax

```
.chkexpr expr="[expression_to_perform_checking |
+ "A=expr1; B=expr2; C=expr3; condition_expr"]"
+ [file=filename] [tag=tagname] [boolean=0|1]
+ [level=none|warn|error] [subckt=subckt_name]
```

```
+ [instance=instance_name] [period=period_time]  
+ [start=start_time] [stop=stop_time]  
+ [window="start1 stop1 start2 stop2..."]  
+ [type=m|q|d|j|c|x] [model=model_name]  
+ [message="user_message"] [csv=0|1]
```

Arguments

Argument	Description
boolean=0 1	Set to report a violation when the expression is true or false. The default is true.
csv=0 1	If set to 1, the result is output in CSV format. The default is 0.
expr	Specifies the expression of the probing syntax. Algebraic functions and conditional statements are supported.
file	Specifies the output file name.
instance	Specifies the instance name to check.
level	Reports warnings/errors when a .chkexpr violation occurs: • none: Default. • warn: Generate warning. • error: Error out the PrimeSim tool.
message	Specifies a custom message. Note: To get a user message in the log file, set level=warn. For example: .chkexpr expr="(id(*)>1u) (id(*)<-1u)" type=m period=1n message="Drain current is over 1u for 1nsec" level=warn
model	Sets the model name of the devices.
period	If the violation period exceeds this specified period, it is reported. The default is 0.
start	Specifies the start time for the violation check.
stop	Specifies the stop time for the violation check.
subckt	Specifies the subcircuit to check.
tag	Defines the tag name of the result.
type	• m: MOSFET • q: Bipolar transistor • d: Diode • j: Junction transistor • c: Capacitor • x: Subcircuit

Argument	Description
<code>window="start1 stop1 start2 stop2..."</code>	Specifies multiple start/stop windows.

Description

Application of the `.chkexpr` command is similar to `.chkdevop` and the expression syntax is similar to `.MEASURE` / `.PROBE`. You can use `.chkexpr` to perform stress check on a subcircuit/instances. Note that `.chkexpr` can dramatically slow down simulation if the check is performed on a large scale. The output file has a `.chkexpr` extension.

Multiple Sub-expression

The PrimeSim tool allows sub-expressions to be defined in the `.chkexpr` command, evaluates each of the sub-expressions, and prints out their values at the beginning and end time of each violation period.

For example, the following `.chkexpr` command:

```
.chkexpr type=m model=nmos expr="a='sqrt(id(*))*0.7'; b='ig(*)*0.3'; c='(a+b)/1e-3';  
c<1e-5"
```

Results in the following output:

```
* expr      : a='sqrt(id(*))*0.7'; b='ig(*)*0.3'; c='(a+b)/1e-3'; c<1e-5  
*  
[tag] [begin_time] [end_time] [period] [expr] [inst/dev]  
      2.013n      2.039n      0.026n      a='sqrt(id(*))*0.7'; b='ig(*)*0.3';  
c='(a+b)/1e-3'; c<1e-5 xinv.mn  
a      -0.002929      0.01154  
b      1.455e-05      2.607e-05  
c      -2.914      11.56  
      2.571n      2.571n      0n      a='sqrt(id(*))*0.7'; b='ig(*)*0.3';  
c='(a+b)/1e-3'; c<1e-5 xinv.mn  
a      -0.0005596      0.0003547  
b      -1.45e-08      5.779e-09  
c      -0.5596      0.3547
```

Conditional Statements

The usage of `IF/ELSE` keywords is not supported in `.chkexpr`. However, you can use the simplified form (`?/:`) for specifying if/else:

```
If (cond) then (expr1) else (expr2) endif
```

The preceding example can be translated into `cond?expr1:expr2`.

Additional Functions

The `.chkexpr` command supports additional functions that do not exist for regular SPICE expressions.

The following functions are supported for `.chkexpr`:

- `IN(expression, lower_bound, upper_bound)` – Returns `true` if the expression falls between lower and upper bound.
- `OUT(expression, lower_bound, upper_bound)` – Returns `true` if the expression falls outside the lower and upper bound.
- `TRACK(expression)` in the `.chkexpr` report specifies to report the min/max value of the expression during the violation period.

You can also specify multiple track functions in the expression. For example:

```
.chkexpr expr='track(ib(*))>1u' type=m
```

In the previous example, the PrimeSim tool reports the time window where bulk current of any MOS exceeds 1u. The report also shows the min/max value of the bulk current during the period in which the violation occurred.

Here is an example report:

```
[tag] [begin_time] [end_time] [period] [expr] [inst/dev]
tag 2.014n 2.098n 0.084n track(ib(*))>1u mp , track(ib(*)) =
(5.598e-05, 1.251e-04)
```

- The `LP` (local parameter) and `RMS` (root-mean-square) function. For example:

```
.chkexpr subckt=nmos_mac expr='OUT((RMS(x(d))/LP(w)), -1m, 1m)
```

- `dp(device_name, device_parameter)` – Allows you to use a device parameter as part of the `.chkexpr` expression. You can use an asterisk (*) wildcard to match the device name, but only MOSFET is supported (`type=m`).

Only length (`l`) and width (`w`) are supported for MOSFET parameters. For example:

```
.chkexpr expr="id(*) > 0.1*dp(*,l)" type=m
```

In this example, the `.chkexpr` command reports any drain current greater than 10% of the MOSFET length.

Checking Device Currents

You can use the `.chkexpr` command to perform stress checks on device current. The usage model is similar to `.chkdevop`, with following parameters supported:

- `ID, IS, IB, IG` – Current through specific MOSFET terminals
- `IB, IC, IE` – Current through specific BJT terminals

For example:

```
.chkexpr expr="id(*) > 1n" type=m model=nch
```

The previous example reports any `nch` MOSFET device's drain current exceeding `1n`.

Checking Subcircuit (Macro) Model Operating Point

Use the `.chkexpr` command to check device operating points for subcircuit models. Unlike the `.chkdevop` command, the `.chkexpr` command checks on the boundary of the subcircuit, and treats the subcircuit model as a single device. Add `type=x` for subcircuit model checking:

- When using `type=x`, you can check the operating point of the subcircuit model by using `Vxy` syntax, where `x` and `y` are the terminal of the equivalent device.
- BJT is not supported for `type=x`.
- Using the `Vxy` syntax automatically converts to the corresponding port based on the port order of the equivalent device. For instance, `Vgs` for MOSFET are the second and third port of the subcircuit.

Supported MOSFET Parameters

MOSFET subcircuit model: `Vd`, `Vg`, `Vs`, `Vb`, `Vds`, `Vdg`, `Vdb`, `Vgs`, `Vgd`, `Vgb`, `Vsb` (and all reverse combinations)

Diode/Resistor/Capacitor subcircuit model: `Vp`, `Vn`, `Vpn`, `Vnp`

For example, use the following to check whether `Vgs` of a subcircuit MOSFET `nch_mac` is outside of an operating range of `-1~1V`:

```
.chkexpr type=x subckt=nch_mac expr="OUT( vgs(*), -1, 1)"
```

It is recommended that you explicitly specify the subcircuit model name instead of using a wildcard.

Note:

The PrimeSim tool supports MOSFET templates `LX0-LX7`, `LV9`, and `LV10` in the `.chkexpr` command.

Examples

```
.chkexpr expr="abs( V(A) - V(Z) ) > 1" subckt=sub1
```

The previous example reports any time where the absolute value of `V(A) - V(Z)` is greater than 1 for the `sub1` subcircuit.

Check OFF Device Leakage Current (.CHKOFFDEVLEAK)

This command detects the OFF transistors, decides whether it has a leakage current at the standby time, and then computes the leakage current based on a given leakage

current table only for leaking transistors. This method is useful when the transistor model does not fit well in a sub-threshold region.

Syntax

```
.chkoffdevleak src=vsrc_node_name time=time1[,time2,...]  
+ [vth_leak=threshold] [ioff=techfile]  
+ [hzstate=hystate_limit] [detail=0|1|2]  
+ [subckt=subckt_name] [instance=instname]  
+ [include=include_pattern] [exclude=exclude_pattern]  
+ [tag=tagname] [file=output_file_name] [csv=0|1]
```

Arguments

Argument	Description
csv	If set to 1, the OFF Devices list is output in CSV format. The default is 0.
detail	If set to 1, all of the OFF transistors and the Hi-Z stage (OffStage) for each instance are printed. If set to 2, only leak devices are printed; noleak devices are not printed. The default is 0.
exclude	Excludes the specified pattern-matched devices to check with this command.
file	Specifies the output file name: output_prefix.output_file_name.chkoffdevleak.
hzstage	A series of OFF devices causes some nodes to be in a high impedance state. In this case, each OFF device has a Hi-Z stage (OffStage) number that indicates the distance from the voltage source to the non-Hi-Z nodes. The default is 2.
include	Includes the specified pattern-matched devices to check with this command.
instance	Specifies the instance name to check with this command.
ioff	Specifies the file that contains the current ratios per unit length/width for each model.
src	Specifies the voltage source node name from which OFF devices are traced.
subckt	Specifies the subcircuit to check with this command.
tag	Defines the tag name used in the result file.
time	Specifies the standby time when the check is performed.

Argument	Description
vth_leak	Specifies the voltage threshold used to decide whether a device has a leakage current. The default is 0.1V. Note: vth_leak is not the value that is used for the ON/OFF state decision.

Description

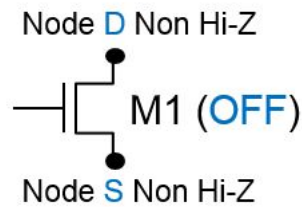
This command is described in more detail in the following sections:

- [Leakage Device Decision in OFF Transistors](#)
- [Leakage Current Calculation](#)
- [Check Example and Result File](#)

Leakage Device Decision in OFF Transistors

As shown in [Figure 15](#), the leakage current of an OFF device depends on the voltage of its drain (D) and source (S) node.

Figure 15 OFF Device Leakage Decision Rule 1

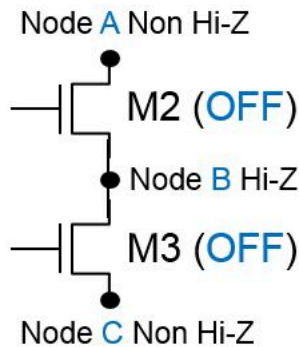


M1 has a leakage current
if $V(D) - V(S) > vth_leak$

If a trace is run from a given voltage source node to an OFF device and reaches the S node first, then if $V(D) > V(S) + vth_leak$ (given, default is 0.1V), then this OFF device has a leakage current.

As shown in [Figure 16](#), if the source or drain node is in a high impedance (Hi-Z) state, that node is not driven by power/ground.

Figure 16 OFF Device Leakage Decision Rule 2



M2 has a leakage current with OffStage=2
if $V(A) - V(C) > vth_leak$
M3 is not counted as a leakage device

In this case, the command keeps tracing to find a non-Hi-Z node and considers how many stacked (OffStage) before the non-Hi-Z node. Generally, if stacked, the leakage current is smaller and you can define a more precise value for that condition leakage current.

Leakage Current Calculation

The leakage current is calculated from the given technology file (`ioff=techfile`) when the device is detected as a leaking MOSFET. The command also considers the MOSFET's stacked number via Hi-Z node (OffStage).

The ratio in the current `loff` technology file is the leakage current per unit width for each model and length.

Syntax:

```
Model_name L_value [to L_value2] [W_value [to W_value2]] ioff1
[z2 ioff2] [z3 ioff3]
```

For example, in the following command:

```
nch 0.18u 2e-4 z2 1.5e-4 z3 1e-4
```

- `nch` is the model name.
- `0.18u` is the length of the transistor.
- The leakage current per unit width is `2e-4` (Amp/meter) for an OFF device with OffStage (Hi-Z stage) 0 or 1.

- If an OFF device is at OffStage 2, the unit width leakage current is $1.5e-4$ (Amp/meter).
- If an OFF device is at OffStage 3, then its leakage current is $1e-4$ (Amp/meter).

The higher the Hi-Z stage, the less its leakage current.

Notes:

- Leakage Current Unit = Ampere / Meter of Width
- Ioff current is calculated by:
Leak_OFF_Device_Width * Leakage_Current_ratio_in_tech_file
- If you define one L or W value, then only that size is calculated.
- If multiple ranges are overlapped, then this command stops with an error message.
- If some ranges' boundaries are the same, then use the boundary in the lower range.
For example:

```
nmos 0.1u to 0.2u 0.1u to 5u 5.0e-4 z2 2.0e-5 z3 1.0e-6 → 0.1u<=L<=0.2u /
0.1u<=W<=5u
nmos 0.1u to 0.2u 5u to 20u 5.5e-4 z2 2.1e-5 z3 1.1e-6 @ 0.1u<=L<=0.2u / 5u <
W<=20u
nmos 0.2u to 0.5u 0.1u to 5u 4.5e-4 z2 1.5e-5 z3 0.5e-6 → 0.2u < L<=0.5u /
0.1u<=W<=5u
nmos 0.2u to 0.5u 5u to 20u 4.7e-4 z2 1.7e-5 z3 0.7e-6 @ 0.2u < L<=0.5u / 5u
< W<=20u
```

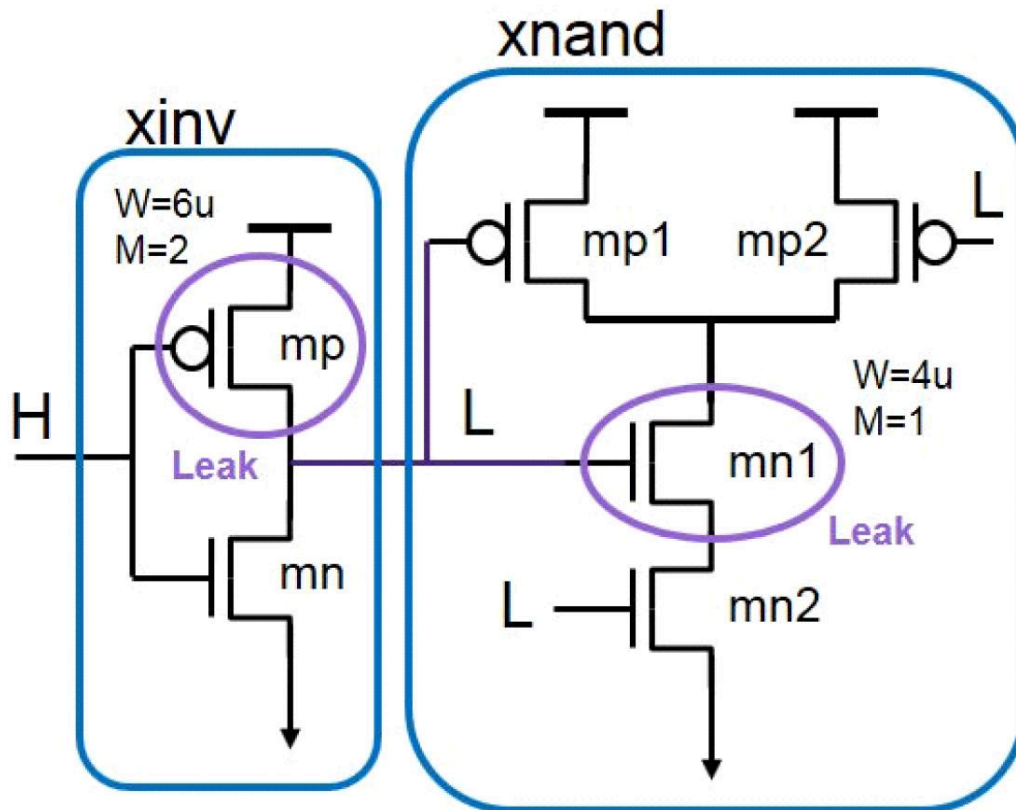
- If z2, z3, or zn are not defined, then the first *ioff1* current ratio is applied.
- If z3 is not defined and OffStage 3 devices are detected, then z2's *ioff2* is applied.
- Use "//" for commenting out a line and "/* ... */" for commenting out a block or multiple lines.
- If a Leak OFF device is detected and Size is not defined in the technology file, the command still reports leak devices without calculating the Ioff current.

Check Example and Result File

For the example shown in [Figure 17](#), invoke the command as follows:

```
.chkoffdevleak time=38n src=vss detail=1 ioff=ioff.tech
```

Figure 17 OFF Device Leakage Example



The command uses the following technology file, `ioff.tech`:

```
my_nmos 0.1u to 10u 5e-4 z2 2e-05 z3 1e-07
my_pmos 0.1u to 10u 5e-4 z2 2e-05 z3 1e-07
```

The result file, `output.chkoffdevleak`, contains the following:

```
*-----
* At 38.0000n SRC=vss
*-----
* Summary for Specs specified by users
*-----
my_nmos L=0.1 to 10u
  W_no_leak=4u (#device=1)
  W_leak=0u Ioff=0 (i_ratio=0.0005) (#device=0)
  W_leak=4u Ioff=8e-11 (i_ratio=2e-05) OffStage=2 (#device=1)
my_pmos L=0.1 to 10u
  W_no_leak=0u (#device=0)
  W_leak=12u Ioff=6e-09 (i_ratio=0.0005) (#device=2)
  W_leak=0u Ioff=0 (i_ratio=2e-05) OffStage=2 (#device=0)
*-----
#Total Off Transistor with leak: 3
- OffStage=1 : 2
- OffStage=2 : 1
```

```
* Detailed Off Devices (default is noleak)
*-----
xnand.mn2          : my_nmos L=0.3u W=4u
xinv.mp            : my_pmos L=0.3u W=6u m=2 leak
xnand.mn1          : my_nmos L=0.3u W=4u leak OffStage=2
*-----
```

In this example result file, you can see `xinv.mp` and `xnand.mn1` are detected as a leaking MOSFETs in the last section, Detailed Off Devices.

The `my_nmos` model's total leaking MOSFET width is 4um and the `i_ratio` (from the `ioff` tech file's `z2` value) is $2e-5$ (Amp/Meter), so `Ioff` (leakage current) is $8e-11$ ($=4e-06*2e-05$).

The `my_pmos` model's total leaking MOSFET width is 12um (6um*2) and the `i_ratio` is 0.0005 (Amp/Meter), so `Ioff` is $6e-09$ ($=12e-6*5e-04$).

Check Rise/Fall Transition Time (.CHKRFTIME)

Description

This command checks if the time of rising or falling transitions on a single node is bigger than a predefined threshold, then reports the violated results into an output file. The output file has a `.chkrftime` extension.

Syntax

```
.chkrftime vl=low_threshold_voltage
+ vh=high_threshold_voltage [subckt=subckt_name]
+ [name=node_name] [level=hierarchy_level_to_match]
+ [type=gate|all] [tr=rise_time_limit] [tf=fall_time_limit]
+ [tx=unknown_time_limit] [start=start_time]
+ [stop=stop_time] [file=filename]
+ [report=all|max] [report_current=0|1]
+ [exclude_node=node_name] [tag=tagname] [verbose=0|1]
```

Arguments

Argument	Description
exclude_node	Specifies the nodes <i>not</i> to check for rising and falling transitions. You can specify to exclude nodes under multiple instances, for example: exclude_node="xiv2.* xiv3.*"
file	Specifies the output file name.
level	Specifies the hierarchy level. If level=-1, the PrimeSim Pro tool checks all hierarchy depth.

Argument	Description
<code>name</code>	Check a specific node. If <code>name</code> is not given, all nodes which are matching <code>type</code> are checked with the warning message.
<code>report</code>	<code>all</code>: (default) Report all violations. <code>max</code>: Report only the maximum (worst) violation for each signal during the entire simulation period. The maximum (worst) violation has the longest time length for (<code>violation_period-time_limit</code>).
<code>report_current</code>	Reports the current of MOSFETs connected by the gate to the node in violation.
<code>start</code>	Specifies the start-time of the window.
<code>stop</code>	Specifies the stop-time of the window.
<code>subckt</code>	Specifies the name of the subcircuit.
<code>tag</code>	Specifies the tag name of the result.
<code>tf</code>	Specifies the fall time limit. If any node's fall time is larger than <code>tf</code> , it is reported in the output file. Default value is <code>5ns</code> .
<code>tr</code>	Specifies the rise time limit. If any node's rise time is larger than <code>tr</code> , it is reported in the output file. Default value is <code>5ns</code> .
<code>tx</code>	Specifies the unknown time limit. An unknown state occurs when any node's voltage cross the same <code>v1/vh</code> threshold twice without reaching the other threshold or when a node's voltage never crosses a <code>v1/vh</code> threshold. If any node's unknown time is larger than <code>tx</code> , it is reported in the output file. Default value is <code>5ns</code> .
<code>type</code>	<code>gate</code>: (default) Check only nodes connected to transistor's gate. <code>all</code>: Check all nodes.
<code>verbose</code>	<code>0</code>: (default) No additional information is provided in the <code>*.chkrftime</code> report. <code>1</code>: Additional information is provided in the <code>*.chkrftime</code> report for Tx violations: <code>State=v1/vh/vlx/vhx</code> indicates whether the Tx violation is based on <code>v1/vh</code> threshold violation. - When the <code>v1/vh</code> threshold is passed twice, the maximum/minimum of the voltage is reported with <code>mag=value</code>.
<code>vh</code>	High voltage threshold to start fall time, or stop rise time measurement.
<code>v1</code>	Low voltage threshold to start rise time, or stop fall time measurement.

Examples

```
.chkrftime v1=0.5 vh=2.0 tr=1p tf=1p file=test.chkrftime type=gate
```

This example checks rising transitions and falling transitions of all the nodes that are connected to the transistor's gate. If a node's rise time is larger than 1p or its fall time is larger than 1p, it is reported. The rise time of a node is the time range that a node rises across vl to vh. The fall time of a node is the time range that a node falls across vh to vl.

The result is output in the file `test.chkkrftime`.

Check Signal Voltage Difference (.CHKSIGDIFF)

Description

This command checks the voltage difference of two signals and issues a warning if the difference is maintained for longer than the specified period.

Syntax

```
.chksigdiff [name='signal_name1 signal_name2']  
+ [dv=threshold_voltage] [report=match|diff]  
+ [period=period_time] [start=start_time] [stop=stop_time]  
+ [tag=tag_name] [file=file_name]  
+ [subckt=subckt_name] [instance=instance_name]
```

Arguments

Argument	Description
<code>dv</code>	Defines the threshold of voltage difference. The default is 0.01v.
<code>file</code>	Specifies the output file name.
<code>instance</code>	Specifies the instance name to check with this command. if specified, the PrimeSim tool finds signals only under that instance. - If <code>tag</code> is specified, it is changed to: <code>tag_name:instance_name:v(signal_name1, signal_name2)</code>. - If <code>tag</code> is not specified, <code>instance_name:v(signal_name1, signal_name2)</code> is created.
<code>name</code>	Defines the two signal names.
<code>period</code>	Reports time-range if the voltage difference stays in the violation state longer than the specified period. The default is 0.
<code>report</code>	Reports time-range: <code>match</code>: (default) Voltage difference is less than <code>dv</code>. <code>diff</code>: Voltage difference is more than <code>dv</code>.

Argument	Description
subckt	Specifies the name of the subcircuit. if specified, the PrimeSim tool finds instance names and signals under those instances. • If <code>tag</code> is specified, it is changed to: <code>tag_name:instance_name:v(signal_name1, signal_name2).</code> • If <code>tag</code> is not specified, <code>instance_name:v(signal_name1, signal_name2)</code> is created.
start	Specifies the start-time of the window.
stop	Specifies the stop-time of the window.
tag	Defines the tag name of the result.

Examples

```
.chksigdiff name="a b" dv=0.2 period=10p
```

Static Circuit Checks (.CHKSTATICERC)

Description

The PrimeSim Pro tool supports the static electrical checking (ERC) command. This command is used to check a variety of electrical rules. The following are currently supported:

- Voltage level checking: Gate, drain, and source of MOS devices
- Floating bulk node checking: Bulk voltage checking less than drain/source for pmos or greater than drain/source for nmos
- Gate node checking: Connected to power or ground
- Path checking: Connected to voltage source from the given nodes
- Antenna diode checking: At top-level circuit or at given nodes

Syntax

```
.chkstaticerc type=[levelchk|bulkchk|gatechk|pathchk|adiochk]  
+ [mode={0|1|2|3|4|5|9|L|H|F|P|N|Z|G|V|A}]  
+ [file=filetag] [tag=name]  
+ [subckt=subname] [stop=1|0]  
+ [vt=vth] [vsrnode="node1(min,max) node2(min,max) ..."]  
+ [node="node1,node2,..."] [dest_node="node5,node6,..."]  
+ [digital_mode=1|0] [floating_gate=0|1]  
+ [grounded_gate=0|1]  
+ [model='[global_model_name1 global_model_name2 ...]  
[subckt_name1:local_model_name1 subckt_name2:local_model_name2 ...]...']
```

Arguments

Argument	Description
<code>dest_node</code>	Specifies destination node names for <code>pathchk</code> type. Several node names can be given by using double quotation marks (""). Wild cards (* or ?) can be used in the part of <code>dest_node</code> names.
<code>digital_mode</code>	As a <code>levelchk</code> option, specifies digital mode. Default value is 1. If <code>digital_mode=1</code>, the <code>levelchk</code> is applied only for CMOS digital devices and reports nodes for required level shifters. If <code>digital_mode=2</code>, the same as <code>digital_mode=1</code> except the algorithm used and output format. If <code>digital_mode=0</code>, vdd propagation algorithm is used and it is applied for all PMOS and NMOS devices. In case this algorithm is applied, diffamp's used as level shifter are detected and excluded from output. The result of <code>digital_mode=1</code> or 2 might be different from that of <code>digital_mode=0</code> because the digital device detection is based on the power block detection.
<code>file</code>	Specifies output file tag name. If <code>file="tag"</code> and output file is "outfile", the real output name is <code>outfile.tag.levelchk</code> for <code>levelchk</code> type.
<code>floating_gate</code>	As a <code>levelchk</code> option, specifies whether to include floating gate checking or not. If <code>floating_gate=1</code> , the level checking for floating gate is included, otherwise, it is excluded. Default value is 0.
<code>grounded_gate</code>	As a <code>levelchk</code> option, specifies whether to include grounded gate checking or not. If <code>grounded_gate=1</code> , the level checking for grounded gate is included, otherwise, it is excluded. Default value is 0.

Argument	Description
mode	Specifies the checking mode for <code>levelchk</code>, <code>bulkchk</code>, <code>gatechk</code>, <code>pathchk</code>, and <code>adiochk</code>. <code>levelchk</code>: • <code>mode=1 L</code> : LoV driving HiV (level shifter) (default). • <code>mode=2 H</code> : HiV driving LoV (overdrive). • <code>mode=3 9 A</code> : All modes. <code>bulkchk</code>: • <code>mode=0 F</code> : Floating bulk checking (default). • <code>mode=1 P</code> : $\text{Max}(V_b) < \text{max}(V_d/s) - V_t$ for pmos • <code>mode=2 N</code> : $\text{Min}(V_b) > \text{min}(V_d/s) + V_t$ for nmos • <code>mode=3 9 A</code> : All modes <code>gatechk</code>: • <code>mode=0 F</code> : Floating gate checking (default). • <code>mode=1 P</code> : $\text{Max}(V_g) < \text{max}(V_d/s) - V_t$ for pmos. • <code>mode=2 N</code> : $\text{Min}(V_g) > \text{min}(V_d/s) + V_t$ for nmos. • <code>mode=3 Z</code> : High impedance gate. • <code>mode=4 G</code> : Gate connected to gnd for pmos. • <code>mode=5 V</code> : Gate connected to vdd for nmos. • <code>mode=9 A</code> : All modes. <code>pathchk</code>: • <code>mode=1 V</code> : Path to vsrc from nodes given (default when dest node is not specified). • <code>mode=2 N</code> : Path to destination nodes from given nodes (default when dest node is specified). • <code>mode=3 9 A</code> : All modes. <code>adiochk</code>: • <code>mode=1 V</code> : Diode checking at the vsrc in top-level (default when node is not specified). • <code>mode=2 N</code> : Diode checking at the given nodes (default when node is specified). • <code>mode=3 9 A</code> : All modes. When “dest_node” is given in <code>pathchk</code>, the default mode is 2, otherwise, the default is 1. When “node” is given in <code>adiochk</code>, the default mode is 2, otherwise, the default is 1.
model	Model name for violation checking (available <code>type=gatechk bulkchk levelchk</code>).
node	Specifies starting node names for <code>pathchk</code> type and the node names considered as antenna node and checks if each node is protected by diode for <code>adiochk</code> type. Several node names can be given by using double quotation marks ("). Wildcards (* or ?) can be used in the part of node names. If <code>type=adiochk</code> and the “node” parameter is not given, detects an antenna node to be protected by diode. It is defined as an input at top level, driven by ideal voltage sources, and directly connected or through resistor/inductor to transistor's gate.

Argument	Description
stop	Specify the stop mode. If <code>stop=1</code> , the simulation is discontinued after checking static ERC (<code>stop=0</code> is the default).
subckt	Specifies a subcircuit for static ERC checking. Note: If this subcircuit is flattened during simulation, then this argument is not supported.
tag	Specifies tag name. If <code>tag="label"</code> is specified in a command, " <code>[tag=label]</code> " is appeared in each line of output file. This feature is usually used to classify the result output to the same file for the same kind commands.
type	Specifies the ERC command. Currently, level check, bulk check, gate check, path check, antenna diode check commands are supported: <code>levelchk</code>: Checks if level shifter is required on gate node of MOS devices. (Refer to <code>digital_mode</code>, <code>floating_gate</code>, <code>grounded_gate</code> parameters) <code>bulkchk</code>: Checks if bulk node is floating, or its voltage level is less than or greater than drain/source voltage. (Refer to <code>mode</code> setting) <code>gatechk</code>: Checks if gate node is floating, or PMOS gate is connected to <code>gnd</code>, or NMOS gate is connected to <code>vdd</code>. (Refer to <code>mode</code> setting) <code>pathchk</code>: Checks if there is a path to voltage source or destination node from the given nodes. <code>adiodchk</code>: Checks if an antenna node is protected by diode at top-level circuit or a given subcircuit.

Examples

```
*level check test
.global vdd vcc vss
v1 vdd 0 dc=2v
v2 vcc 0 dc=3v
v3 vss 0 dc=0
.subckt inv in out vdd vss
.param w=1u
mp out in vdd vdd pmos w='w*0.1' l=0.3u
mn out in vss vss nmos w=3u l=0.3u
.ends inv
.model pmos pmos level=53
.model nmos nmos level=53
x1 in n1 vdd vss inv
x2 n1 n2 vdd vss inv
x3 n1 n5 vcc vss inv
x4 n5 out2 vcc vss inv
x5 n4 out1 vcc vss inv
mp1 n3 n4 vcc vcc pmos
mn1 n3 n2 vss vss nmos
mp2 n4 n3 vcc vcc pmos
mn2 n4 n1 vss vss nmos
.tran 0.1n 40n
```

```
.probe v(*)
(1) .chkstaticerc type=levelchk file=aaa
(2) .chkstaticerc type=levelchk file=aaa_stop stop=1
(3) .chkstaticerc type=levelchk file=digital1 stop=1 digital_mode=1
(4) .chkstaticerc type=levelchk file=digital2 stop=1 digital_mode=2
(5) .chkstaticerc type=levelchk file=digital0 stop=1 digital_mode=0
(6) .chkstaticerc type=levelchk file=gndgate stop=1 digital_mode=0
+ floating_gate=1 grounded_gate=1
(7) .chkstaticerc type=bulkchk mode=0 file=floating stop=1
(8) .chkstaticerc type=bulkchk mode=2 stop=1
(9) .chkstaticerc type=gatechk mode=0 file=floating stop=1
(10) .chkstaticerc type=gatechk mode=2 file=nmosvdd stop=1
(11) .chkstaticerc type=pathchk mode=1 file=vsrc node="n1,n2" stop=1
(12) .chkstaticerc type=pathchk mode=2 file=node node="n1,n2"
dest_node="n3" stop=1
(13) .chkstaticerc type=pathchk file=node node="*" dest_node="n3" stop=1
(14) .chkstaticerc type=adiochk mode=1 file=antenna stop=1
(15) .chkstaticerc type=adiochk mode=2 file=antnode node="n1,n2,n3"
stop=1
.end
```

When you run `primesim test.sp`, you can get the following results for each command used.

- (1) generates `test.aaa.levelchk` file for level check.
- (2) is the same as (1), but terminates the simulation right after the `levelchk` command.
- (3),(4),(5) generate `test.digital1.levelchk`, `test.digital2.levelchk`, and `test.digital0.levelchk` by using different algorithms, respectively, and stop.
- (6) generates `test.gndgate.levelchk`. This does level checking for floating and grounded gates.
- (7) generates `test.floating.bulkchk`. This does check floating bulk node.
- (8) generates `test.bulkchk`. This checks if V_b is greater than V_d/V_s for nmos.
- (9) generates `test.floating.gatechk`. This does check floating gate node.
- (10) generates `test.nmosvdd.gatechk`. This does check gate nodes connected to vdd.
- (11) generates `test.vsrc.pathchk`. This checks if there is a path to voltage sources from the nodes `n1` or `n2`.
- (12) generates `test.node.pathchk`. This checks if there is a path to destination node `n3` from the nodes `n1` or `n2`.
- (13) generates `test.allnode.pathchk`. This checks if there is a path to destination node `n3` from at least one node of all the nodes.

- (14) generates `test.antenna.adiochk`. This checks if an antenna node is protected by the diode, that is, the diode is connected backward to the antenna node.
- (15) generates `test.antnode.adiochk`. This checks if each of the given nodes (`n1`, `n2`, `n3`) is protected by the diode.

Check Timing Setup/Hold/Delay/Width (.CHKTIMING)

This command performs checks for setup ([Setup Time Check](#)), hold ([Hold Time Check](#)), delay ([Delay Check](#)), and pulse_width ([Pulse Width Check](#)). The PrimeSim Pro tool supports `.chktiming` wildcards for tag names. You can also use this command to define a time range used by all other `.chktiming` statements (see [Check Timing Window](#)).

Syntax

```
.chktiming type=[setup|hold|delay|width|gwidth]
+ name=sig_name ref=ref_name
+ time=val [tag=tag_name]
+ vlow=low_min_voltage vhigh=high_max_voltage
+ vlth=low_threshold_voltage vhth=high_threshold_voltage
+ refvlth=ref_low_threshold_voltage
+ refvhth=ref_high_threshold_voltage
+ [edge=[r|f|x]] [refedge=[r|f|x]] [window=window_limit]
+ [subckt=ckt_name] [file=filename]
+ [trigger=[0|1|2]] * only for delay type
+ lowtime=(val1,val2) * only for width/gwidth type
+ hightime=(val1,val2) * only for width/gwidth type
```

Arguments

Argument	Description
edge	Defines the state transition type of the node: • <code>r</code>: Check is performed with transition from 0 to 1. • <code>f</code>: Check is performed with transition from 1 to 0. • <code>x</code>: Check is performed with any transition.
file	Specifies the output file name.
hightime	Defines minimum and maximum width values of the high state pulse.
lowtime	Defines minimum and maximum width values of the low state pulse.
name	Defines the node name. When using a <code>subckt</code> parameter, <code>sig_name</code> is the node name in the subcircuit. Note: You can use a wildcard (*) in the name pattern, but cannot use it in the reference node name.
ref	Defines the reference node name.

Argument	Description
refedge	Defines state transition type of the reference node.
refvhth	Represents logic state 1 threshold voltage of the reference node.
refvlth	Represents logic state 0 threshold voltage of the reference node.
subckt	When defined, indicates that checking timing is performed on all instances of the subcircuit.
tag	Defines the tag name of the result.
time	Defines time value to check. Delay type checking must have a time pair represented by minimum and maximum times.
trigger	0: (default) Any permissible state transition at either signal node or reference node triggers the timing edge check. 1: Only the permissible state transition at the signal node triggers the check. 2: Only the permissible state transition at the reference node triggers the check.
type	- setup: Setup Time Check - hold: Hold Time Check - delay: Delay Check - width gwidth: Pulse Width Check
vhigh	Defines the high threshold voltage, above which the signal must travel during a transition in order to register as a pulse.
vhth	Represents logic state 1 threshold voltage of the node.
vlow	Defines the low threshold voltage, below which the signal must travel during a transition in order to register as a pulse.
vlth	Represents logic state 0 threshold voltage of the node.
window	Specifies the window limit.

Description

Use the `.CHKTIMING` statement to check the following:

- [Setup Time Check](#)
- [Hold Time Check](#)
- [Delay Check](#)

- [Pulse Width Check](#)
- [Check Timing Window](#)

Setup Time Check

This command checks whether the time from the reference node transition to the signal node transition is less than the setup time.

Syntax

```
.chktiming type=setup name=sig_name ref=ref_name  
+ time=val [tag=tag_name]  
+ vlth=low_threshold_voltage vhth=high_threshold_voltage  
+ refvlth=ref_low_threshold_voltage  
+ refvhth=ref_high_threshold_voltage  
+ [edge={r|f|x}] [refedge={r|f|x}] [window=window_limit]  
+ [subckt=ckt_name] [file=filename]
```

Examples

```
.chktiming type=setup name=n1 ref=clk time=1n vlth=0.2 vhth=2.0  
+ refvlth=0.2 refvhth=2.0
```

This example checks whether the time from the reference node `clk` transitions to the signal node `n1` transition is less than the setup time `1ns`. If the time is less than `1ns`, the PrimeSim Pro tool reports it as a violation. For both `clk` and `n1`, the low threshold voltage is `0.2v` and high voltage threshold is `2.0v`.

Hold Time Check

This command checks whether the time from the signal node transition to the reference node transition is less than the hold time.

Syntax

```
.chktiming type=hold name=sig_name ref=ref_name  
+ time=val tag=tag_name  
+ vlth=low_threshold_voltage vhth=high_threshold_voltage  
+ refvlth=ref_low_threshold_voltage  
+ refvhth=ref_high_threshold_voltage  
+ [edge={r|f|x}] [refedge={r|f|x}] [window=window_limit]  
+ [subckt=ckt_name] [file=filename]
```

Examples

```
.chktiming type=hold name=n1 ref=clk time=1n vlth=0.2 vhth=2.0  
+ refvlth=0.2 refvhth=2.0
```

This example checks whether the time from the reference node `clk` transitions to the signal node `n1` transition is less than the hold time `1ns`. If the time is less than `1ns`, the

PrimeSim Pro tool reports it as a violation. For both `clk` and `n1`, the low threshold voltage is 0.2v and high voltage threshold is 2.0v.

Delay Check

This command checks whether the time delay between the signal node and the reference falls outside the specified time window.

Syntax

```
.chktiming type=delay name=sig_name ref=ref_name  
+ time=(val1,val2) tag=tag_name  
+ vlth=low_threshold_voltage vhth=high_threshold_voltage  
+ refvlth=ref_low_threshold_voltage  
+ refvhth=ref_high_threshold_voltage  
+ [edge=[r|f|x] [refedge=[r|f|x] [window=window_limit]  
+ trigger=[0|1|2] [subckt=ckt_name] [file=filename]
```

Examples

```
.chktiming type=delay name=n1 ref=clk time=(1n,2n) vlth=0.2  
+ vhth=2.0 refvlth=0.2 refvhth=2.0
```

Pulse Width Check

This command checks the time between the rise and fall transitions of the signal node.

Note:

Specify `type=gwidth` to check the pulse width only for nodes connected to the MOSFET Gate.

Syntax

```
.chktiming type=[width|gwidth] name=sig_name tag=tag_name  
+ lowtime=(val1,val2) hightime=(val1,val2)  
+ vlth=low_threshold_voltage vhth=high_threshold_voltage  
+ vlow=low_min_voltage vhigh=high_max_voltage  
+ [edge=[r|f|x] [subckt=ckt_name] [file=filename]
```

Examples

```
.chktiming type=width name=n1 lowtime=(1n,2n) hightime=(1n,2n)  
+ vlth=0.2 vhth=2.0
```

In the first example, if the high or low pulse width are less than 1ns or greater than 2ns, a pulse width violation is reported. Node `n1` has a logic 0 state if its node voltage is lower than 0.2v and has a logic 1 state if the node voltage is higher than 2v.

```
.chktiming type=width name="a*"
```

In the second example, `a*` can be a bus of `a[15:0]`.

Check Timing Window

The PrimeSim Pro tool can specify windows for `.chktiming`. All timing check commands use the same window sets that are specified by this command. When a window is not specified, the default window is the whole simulation time.

Syntax

```
.chktiming window start_time1 stop_time1 start_time2 stop_time2 ...
```

Examples

```
.chktiming window 0n 10n 40n 60n 80n
```

In this example, the command specifies three sets of windows. The first window is from 0ns to 10ns, the second window is from 40ns to 60ns, and the last window is from 80ns to the end of simulation.

Check and Report Toggle Count (.CHKTOGGLE)

Description

Toggle count is reported for every node at the end of the simulation. If any node rises across `vh` or declines across `vl`, the toggle count at this node is increased by 1. At the end of the simulation, the toggle count is reported for every node. The output file has a `.chktoggle` extension.

Syntax

```
.chktoggle vl=low_threshold_voltage  
+ vh=high_threshold_voltage  
+ [subckt=subckt_name] [name=nodename]  
+ [level=hierarchy_level_to_match]  
+ [type=gate|all] [report=(min,max)] [start=start_time]  
+ [stop=stop_time] [file=filename]
```

Arguments

Argument	Description
file	Specifies the output file name.
level	Specify the hierarchy level. If <code>level=-1</code> , the PrimeSim Pro tool checks all hierarchy depth.
name	Sets the specific node name for toggle behavior check. If <code>name</code> is not given, all nodes matching <code>type</code> are checked with the warning message.
report	Sets the minimum number and the maximum number of nodes reported in the output file.

Argument	Description
start	Specifies the start-time of the window.
stop	Specifies the stop-time of the window.
subckt	Specify the name of subcircuit.
type	- gate: (default) Check only nodes connected to transistor's gate. - all: Check all nodes.
vh	high_threshold_voltage. If any node rises across vh, the toggle count at this node is increased by 1.
vl	low_threshold_voltage. If any node declines across vl, the toggle count at this node is increased by 1.

Examples

```
.chktoggle name="2" vl=0.5 vh=2.0 start=0 stop=2n file=test.chktoggle
+type=all report=(0,10000)
```

This command counts the toggle number of all nodes in the circuit between 0ns and 2ns. The low threshold voltage is 0.5v and the high threshold voltage is 2v. If the number of nodes that meet the condition is between 0 and 10000, the PrimeSim Pro tool reports all of them.

The result is put in the file `test.chktoggle`.

Check High Impedance State Node (.CHKZNODE)

This command checks for the high impedance state in the circuit.

Syntax

```
.chkznode [file=filename] [type=gate|bulk|all]
+ [start=time_val] [stop=time_val]
+ [subckt=subckt_name[:0|1]]
+ [instance=instance_name[:0|1]]
+ [period=time_val] [include|exclude=name] [waivefile=file_name]
```

Arguments

Argument	Description
file	Specifies the output file name.
include exclude	- include=name: Only checks the include node in the circuit check. - exclude=name: Excludes the node in the circuit check.

Argument	Description
<code>instance</code>	Specifies the instance name to check. You can append [0 1] to indicate if it is an include or exclude. Wildcards are supported.
<code>period</code>	If any node is staying in the high impedance state longer than period, it is reported in the output file. The default is 1n.
<code>start</code>	Specifies the start-time of the window.
<code>stop</code>	Specifies the stop-time of the window.
<code>subckt</code>	Specifies the subcircuit to check. You can append [0 1] to indicate if it is an include or exclude. Wildcards are supported.
<code>type</code>	• <code>gate</code>: (default) Check only nodes connected to transistor's gate. • <code>all</code>: Check all nodes. • <code>bulk</code>: Check bulk nodes.
<code>waivefile</code>	Specifies the file containing the node list of waived results.

Description

This command reports a Hi-Z node when no voltage source is encountered while traversing the DC path from that node. While tracing the DC path, if a device's state in the path is OFF, the tool stops tracing and traverse in the other direction. The ON/OFF state judgment rule is optional.

The default rule (`primesim_check_vth=1 + primesim_chkznnode_rule=2`) basically determines the device's ON/OFF state based on its conductivity, which is judged by its terminal voltage level and Vth.

The `primesim_goff` option is used by Gds to determine the state of a MOSFET instead of `primesim_check_vth`.

The PrimeSim tool supports multiple `.chkznnode` commands in the same simulation.

Note:

The ON/OFF state described here applies specifically to CHKZNODE, not the general device ON/OFF state.

Examples

```
.chkznnode period=1n type=gate start=160u stop=240u
```

This command checks the high impedance state of nodes that are connected to transistors' gates between 160us and 240us and which stay in the high impedance state for more than 1ns. The generated output is `prefix.filename.chkznnode`.

Table 36 .CHKZNODE Related Options

PrimeSim Option	Description
<code>primesim_check_vth</code>	Specifies whether to use <code>primesim_goff</code> (0), <code>vth</code> (1) or operating region (2) to determine the ON/OFF state. The default is 1.
<code>primesim_chkznode_rule</code>	Specifies a global option in checking conductivity of MOSFET. The default is 2.
<code>primesim_chkznode_vth</code>	Specifies the Vth value for checking ON/OFF state.
<code>primesim_goff</code>	Determines the ON/OFF state of a MOSFET.

Note:

These options also apply to `CHKDCPATH zgate=on`.

Circuit Check Post-Processing Options

The PrimeSim tool supports post-processing for some circuit check commands. [Table 37](#) lists the post-processing options.

Table 37 *Circuit Check Post-Processing Options*

PrimeSim Option	Description
<code>primesim_ck_post</code>	Sets all three post-processing options: • <code>primesim_chkdevop_post</code> • <code>primesim_chkrftime_post</code> • <code>primesim_chktiming_post</code>
<code>primesim_chkdevop_post</code>	Allows you to post-process <code>.chkdevop</code> (see Check Device Operation Point (.CHKDEVOP)).
<code>primesim_chkrftime_post</code>	Allows you to post-process <code>.chkrftime</code> (see Check Rise/Fall Transition Time (.CHKRFTIME)).
<code>primesim_chktiming_post</code>	Allows you to post-process <code>.chktiming</code> (see Check Timing Setup/Hold/Delay/Width (.CHKTIMING)).
<code>primesim_print_rchk_resource</code>	Reports the detailed time and memory usage for the <code>-rchk</code> command-line option when rechecking the database (see Command-Line Options).

16

Tcl Interactive Mode

This chapter describes the PrimeSim tool support for Tcl Interactive mode.

This chapter covers the following topics:

- [Interactive Mode](#)
- [Interactive Commands](#)

Interactive Mode

You can start the PrimeSim tool in interactive mode from the command line using the `-istop [time]` option. The *time* argument is optional. If used, the PrimeSim Pro tool enters interactive mode at the time you specify.

In addition, you can use the `-istop_file file_name` command-line option to specify a batch file containing interactive commands.

Note:

If `-istop_file` is used without `-istop`, the tool exits with an error.

If you do not specify a time, the PrimeSim tool enters interactive mode when you type **Ctrl+C**. If you type **Ctrl+C** during a simulation without using the `-istop` command, the simulation enters interactive Tcl mode, but only with limited functionality related to voltage probing.

For more information on the commands supported in interactive mode, refer to [Interactive Commands](#).

Interactive Commands

With the need of complex and intensive simulations, there is an increasing need to get the most information out of each run. It extends beyond simple probing of signals and parameters; the status of certain devices or subcircuits in a time period of the run can potentially reveal valuable methods for verification of the design. Such techniques are also useful in investigating problem areas in the circuit. The PrimeSim tool supports a Tcl

interface to provide access to real-time simulation data in transient simulations. To access this data, use the command-line argument, `-istop`.

With the `primesim -istop` option, the PrimeSim Pro tool keeps all nodal information so that the interactive mode can access them. At the `PrimeSim>` interactive prompt, the commands for querying the simulation data can be entered.

User-defined scripts in the Tcl syntax can be passed into the PrimeSim tool for running in batch mode using the `primesim_tcl_init_file` option. It is possible to create control structures and mathematical expressions using the Tcl language to evaluate information as desired. The `pause` command provides the capability to switch to Interactive Mode if necessary.

Note:

The following interactive commands can be used with Tcl commands and pass their data to a Tcl variable: `ni`, `pd`, `pdp`, `pi`, `pn`, and `tn`.

For example, if you run the PrimeSim tool in interactive mode, you can store the `pn a*` command in a Tcl variable:

```
PrimeSim> set avar "pn a*"
```

Then you can reuse the variable to run the same command again later on:

```
PrimeSim> eval $avar
```

[Table 38](#) lists the interactive commands. Click a command name to jump to a more detailed description with examples.

Table 38 Interactive Commands

Command	Description
circheck	Tcl circuit check.
clearlog	Clears the interactive command log.
cont	Continues the simulation.
dataflush	Flushes output logs and FSDB files.
dn	Outputs user-specified nodes.
exi	Reports devices with excessive current.
exit	Terminates the simulation.
fn	Forces node, forces voltage to nodes.
fset	Configures Tcl interactive mode parameters.

Table 38 *Interactive Commands (Continued)*

Command	Description
<code>help</code>	Lists command usage.
<code>ni</code>	Prints the current of nodes that match the given set of node patterns.
<code>now</code>	Reports simulation progress.
<code>op</code>	Saves the <code>.op</code> into an output file.
<code>openwv</code>	Invokes the PrimeWave Design Environment WaveView tool and opens the specified waveform files.
<code>pd</code>	Prints device information.
<code>pdp</code>	Prints DC path information.
<code>pi</code>	Prints instance information.
<code>pn</code>	Prints node information.
<code>pvar</code>	Prints a Verilog-A variable.
<code>quit</code>	Quits the current sweep.
<code>rn</code>	Releases node, releases constant voltage set by <code>fn</code> .
<code>snapshot save</code>	Takes a snapshot.
<code>stop</code>	Sets a stop point for the simulator.
<code>svar</code>	Sets a Verilog-A variable.
<code>tn</code>	Traces node, shows devices that are connected to the node.
<code>tnet</code>	Iterates over the nodes in the same net with the given node, and prints the information of the devices connected, except resistors and capacitors.

circheck

This command adds, lists, and removes customized functions for checking nodes and devices.

Syntax

```
circheck [node|device] add nodenamePattern checkFunc  
circheck [node|device] ls  
circheck [node|device] del -index index [index]
```

Description

where:

- `[node|device]` – Specifies either node level or device level.
- `add nodenamePattern checkFunc` – Attaches the user-defined check function *checkFunc* to nodes matched by *nodenamePattern*. *checkFunc* must take the node handle as the only argument. The return value of the function is neglected.
- `ls` – Prints the current set of nodes and the check function name.
- `del -index index [index]` – Removes set of nodes with checking functions by specifying the index. Use `circheck node ls` to get the indexes.

clearlog

Description

This command clears and truncates the interactive command log (`*.inter.log`).

Syntax

```
clearlog
```

cont

Description

This command continues the interrupted simulation for time seconds or until a previously set stop point. The time sets the running time of the simulation until it stops again.

Syntax

```
cont [time]
```

dataflush

Description

This command flushes all of the PrimeSim tool output logs and `.fsdb` files.

Note:

If your simulation is long and you want to check the output at the start (<10%), then the `dataflush` command can be used. Generally, the `.fsdb` and `.log` files are not updated until at least 10% of the analysis has been performed, but if you want to check before 10%, once in interactive mode, you can execute

`dataflush` on the interactive prompt and all of the `.fsdb` waveforms and `.log` files up to that time are updated.

Syntax

```
dataflush
```

dn

This command outputs user-specified nodes.

Syntax

```
dn [-dv [voltage_change_value]] | [-dt [time_step_value]] | [-v  
[voltage_value]]
```

Description

where:

- `-dv voltage_change_value` – Outputs the node with maximum absolute voltage change when `voltage_change_value` is not specified; when `voltage_change_value` is specified (in volts), it outputs the nodes with the absolute voltage change greater than `voltage_change_value`.
- `-dt time_step_value` – Outputs the node with minimum time step when `time_step_value` is not specified; when `time_step_value` is specified, it outputs the nodes with the time step smaller than `time_step_value`.

Note:

The unit of `time_step_value` can be specified as: s (default), ms, us, ns, or ps.

- `-v voltage_value` – Outputs the node with maximum absolute voltage when `voltage_value` is not specified; when `voltage_value` is specified (in volts), it outputs the nodes with the absolute voltage greater than `voltage_value`.
-

exi

This command identifies and reports the devices matching a given set of device patterns. The patterns support wildcard matching.

Syntax

```
exi [options] device_pattern [device_pattern]
```


Description

where possible options are:

- `-ith threshold_current` – Sets the threshold current; all devices with current exceeding `threshold_current` are reported. The default value is `5e-5`.
- `-type device_type` – Sets the type of device; only devices matching `device_type` are reported.

Supported device type are: Res, VarRes, RDiff, Cap, VarCap, ChgCap, Ind, Reluc, VarInd, MutInd, MutReluc, NMOS, PMOS, VSrc, ISrc, VCVS, VCVSD, VCCS, VCCSD, CCVS, CCVSD, CCCS, CCCSD, VCRes, VCCap, BVS, BCS, Diode, BJT, JFET, WElem, SElem, BElem, Norton, EMM, User, CModel.

- `-sort` – If set, sort the devices by current, with larger coming first.

Note:

Any of the above options can be used before device name patterns.

exit

Description

This command stops the entire simulation process and exits the PrimeSim tool.

Syntax

```
exit
```

fn

This command forces the specified data bus or the nodes matching a set of node patterns to keep a constant voltage.

Syntax

```
fn [-s slope] [-bus bus_node[i:j](bus_node<i:j>) -vh high_voltage  
-vl low_voltage signal_mask] [node_pattern [node_pattern] voltage]
```

Description

To avoid abrupt changes of voltage, the voltage of the forced nodes change from their current voltage to the specified voltage at the given slope. Slope sets the slope of the voltage change.

When specifying a range of lines in a data bus, the order of the indexes (*i:j*) can be low-to-high or high-to-low, but the forcing of constant voltage according to *signal_mask* always begins with the lower index value.

Units are ps/V, and the default slope is 10 ps/V.

Note:

Voltage source nodes cannot be forced.

Examples

In the first example, the constant voltage for the node `out` is set to 1.2.

```
fn out 1.2
```

In the second example, the constant voltage for data bus lines `DATA[5]`, `DATA[7]`, and `DATA[9]` is set to 1.8, for `DATA[6]` and `DATA[10]` it is set to 0, and for `DATA[8]` it is left unchanged.

```
fn -bus DATA[5:10] 101x10 -vh 1.8 -vl 0
```

The examples for the [rn](#) command show how to release the forced constant voltage from the preceding examples.

fset

Description

This command configures some of the Tcl interface parameters where the output precision is set to `[X]`.

Syntax

```
fset precision [X]
```

help

Description

This command lists available commands and their relative usage models.

Syntax

```
help [command]
```

ni

Description

This command prints the current of nodes matching the given set of node patterns. The pattern supports wildcard matching. The current of the node is defined as the sum of branch current flows into the node.

Syntax

```
ni node_pattern [node_pattern]
```

now

Description

This command reports current simulation time and progress percentage of the whole simulation.

Syntax

```
now
```

op

Description

This command saves the `.op` into an output file, `prefix.filename.op`.

Syntax

```
op save filename
```

openwv

Description

This command invokes the PrimeWave Design Environment WaveView tool and opens the specified waveform files. The WaveView module must be loaded to the terminal before simulation starts. After this command is executed, the voltage waveform of the node can be plotted by the command `tn -wv`.

Syntax

```
openwv waveform_filename [waveform_filename ...]
```

pd

Description

This command prints the device name and related information of devices matching the given set of device patterns. The pattern supports wildcard matching. Related information includes the type of the device, basic parameters, and all of the device terminal name with voltages and currents. There is extra information available for MOSFETs.

Syntax

```
pd device_pattern [device_pattern]
```

pdp

This command prints DC path information.

Syntax

```
pdp [-ith threshold_current] [node_pattern] [node_pattern]
```

Description

where:

- `-ith threshold_current` – Sets the threshold current; all DC paths with current exceeding *threshold_current* are reported. The default *threshold_current* value is 0.
-

pi

Description

This command prints instance information. The pattern supports wildcard matching.

Syntax

```
pi instance_pattern [instance_pattern]
```

pn

This command prints the name and voltage (and optionally, capacitance) or Z-state of nodes matching the given set of node patterns. The pattern supports wildcard matching.

Syntax

```
pn [-cap] | [-zstate] node_pattern [node_pattern]
```

Description

where:

- `-cap` – Prints capacitance information of the nodes.
- `-zstate` – Prints only the nodes in Z-state.

pvar

Description

This command prints a Verilog-A variable. The pattern supports wildcard matching.

Syntax

```
pvar instance_name_pattern:variable_name_pattern
```

quit

Description

This command quits the current sweep and continues with the rest of the simulation.

Syntax

```
quit
```

rn

Description

This command releases the constant voltage set by the [fn](#) command for the specified data bus or the nodes matching the given set of node patterns. The data bus or node voltages are therefore determined by normal simulation result.

Syntax

```
rn [-bus bus_node[i:j] (bus_node<i:j>) [node_pattern [node_pattern]]
```

Examples

The following examples show how to release the constant voltage forced by the [fn](#) command examples.

The first example releases the constant voltage for the node, `out`.

```
rn out
```

The second example releases the constant voltage for the data bus, `DATA[5:10]`.

```
rn -bus DATA[5:10]
```

snapshot save

Description

This command controls taking a snapshot in interactive mode and saving it to *filename*.

Syntax

```
snapshot save [filename]
```

stop

Description

This command sets a time point at which the simulator schedules to stop at *time*, in unit of seconds.

Syntax

```
stop time
```

svar

Description

This command sets a Verilog-A variable. The pattern supports wildcard matching.

Syntax

```
svar instance_name_pattern:variable_name variable_value
```

tn

This command prints information of devices connected to certain nodes matching a given set of node patterns. The pattern supports wildcard matching. Each device is printed with its terminal nodes. The one which directly connects to the given node is marked with an asterisk (*).

Syntax

```
tn [options] node_pattern [node_pattern...]
```

Description

where possible *options* are:

- `-ith threshold_current` – Sets the threshold current; only devices with connected port currents that exceed *threshold_current* are reported.
- `-inst inst_name` – Specifies instance names on which to report; only devices in the specified instance are reported.
- `-etype device_type [device_type...]` – Excludes the specified device types, for example, ResPMOS. Devices with *device_type* are not reported. *device_type* is case-insensitive.

Note:

Do not use this option with the `-itype` option.

- `-itype device_type [device_type...]` – Includes the specified device types, for example, ResPMOS. Devices with `device_type` are reported. `device_type` is case-insensitive.

Note:

Do not use this option with the `-etype` option.

- `-eterm device_terminal [device_terminal...]` – Excludes the specified device terminals, for example, SDGB0123. Devices which connect the node through `device_terminal` are not reported. `device_terminal` is case-insensitive.

Note:

Do not use this option with the `-iterm` option.

- `-iterm device_terminal [device_terminal...]` – Includes the specified device terminals, for example, SDGB0123. Devices which connect the node through `device_terminal` are reported. `device_terminal` is case-insensitive.

Note:

Do not use this option with the `-eterm` option.

- `-sort sort_option` – Sorts the printed devices by its type (`sort_option = device_type`) or by the current (`sort_option = current`) flowing through the terminal connected to the node.
- `-n maximum_number` – Specifies the maximum number of devices on which to report.
- `-m` – Outputs merged parallel devices with the head device.
- `-c` – Specifies that output from this command is in concise mode where it reports only the total current flowing into the node.
- `-wv nodename [nodename ...]` – Plots the voltage waveform of the specified hierarchical `nodename` using the PrimeWave Design Environment WaveView tool which is invoked by the [openwv](#) command.

tnet

This command iterates over the nodes in the same net with the given node, and prints the information of the devices connected, except resistors and capacitors. The pattern supports wildcard matching. Each device is printed with its terminal nodes. The one which directly connects to the given node is marked with an asterisk (*).

Syntax

```
tnet options node_pattern [node_pattern...]
```

Description

where possible *options* are:

- `-ith threshold_current` – Sets the threshold current. Only devices with connected port current that exceeds *threshold_current* are reported.
- `-inst instance_name` – Prints only the devices in the specified instance.
- `-etype device_type [device_type...]` – Sets the device types to be excluded, for example, ResPMOS. The device types are case-insensitive. Devices with *device_type* are not reported.

Note:

Do not use this option with `-itype`.

- `-itype device_type [device_type...]` – Sets the device types to be included, for example, ResPMOS. The device types are case-insensitive. Only the devices with *device_type* are reported.

Note:

Do not use this option with `-eterm`.

- `-eterm device_terminal [device_terminal...]` – Sets the device terminals to be excluded, for example, SDGB0123. The device terminals are case-insensitive. Devices which connect to the node through *device_terminal* are not reported.

Note:

Do not use this option with `-iterm`.

- `-iterm device_terminal [device_terminal...]` – Sets the device terminals to be included, for example, SDGB0123. The device terminals are case-insensitive. Only devices which connect to the node through *device_terminal* are reported.

Note:

Do not use this option with `-eterm`.

- `-sort sort_option` – Sorts the printed devices by its type (*sort_option* = *device_type*) or by the current (*sort_option* = *current*) flowing through the terminal connected to the node.
- `-n maximum_number` – Specifies the maximum number of devices on which to report.

- `-a` – An advanced option which outputs the number of RC and does not output the device-zero node.
- `-m` – Outputs merged parallel devices with the head device.

17

Monte Carlo Analysis

This chapter describes the PrimeSim tool support for Monte Carlo Analysis.

The Monte Carlo (MC) method is a well known technique for modeling manufacturing variations in a statistical manner. The simulations involve randomizing some process parameters and observing the impact on the desired outputs. Typically, it requires a very large number of simulations and measurements to ensure confidence in boundaries of variations. Speed-up can be further improved by the fact that the PrimeSim tool offers scalability at runtime by executing on multiple processors and by the inherent speed-up of PrimeSim simulation. This can be achieved by using `-dp -mt X` with the command.

Random variations can be defined by using functions such as `unif`, `aunif`, `gauss`, and `agauss`, or in the Variation Block format.

You should use the following SPICE analysis statements to set MC analysis:

```
.dc ... sweep monte=NMC [firstrun=N]
.tran ... sweep monte=NMC [firstrun=N]
.ac ... sweep monte=NMC [firstrun=N]
```

where N_{MC} is the maximum number of Monte Carlo runs and `firstrun` defines the index from which to start using the random number sequence.

To set the seed for random generation to be the same across multiple runs, set `primesim_mcseed` to a positive integer value; see [primesim_mcseed](#) for details.

There must be at least one `.measure` statement in the input deck to output scalar values from each Monte Carlo run. The PrimeSim tool generates a `.mt0/.ma0/.md0` file for the run; the file includes the list of evaluated values in each Monte Carlo run, followed by a summary of the distribution.

It is highly recommended that you set `primesim_output=none` to avoid generating several FSDB files that can cause a disk space issue. For debugging purposes, set `primesim_output=tr0` to save the family of plots; set `primesim_output=fsdb/tr0` when a single run is rerun (`monte=1 firstrun=Nrerun`).

Monte Carlo analysis is further described in the following sections:

- [Monte Carlo Analysis in the PrimeSim Tool](#)
- [Bi-Section Runs](#)
- [Incremental Runs](#)
- [Random Number Generation Sequence](#)
- [Overview of the Variation Block](#)
- [Variation Block Structure](#)
- [External Sampling](#)
- [Data Mining Using PrimeSim With PrimeSim AVA](#)

Monte Carlo Analysis in the PrimeSim Tool

Monte Carlo analysis has long been available in the PrimeSim tool and it uses three approaches:

- Define distributions on global parameters (using `AGAUSS`, `GAUSS`, `UNIF`, and `AUNIF`) in a netlist. For example:

```
.param var=AGAUSS(20,1.2,3)
```

- Define distributions on model parameters with `DEV` and `LOT` constructs in a model file. For example:

```
vth0=0.6 lot/0.1 dev/0.02
```

- Define distributions on model parameters with Variation Block syntax. For example:

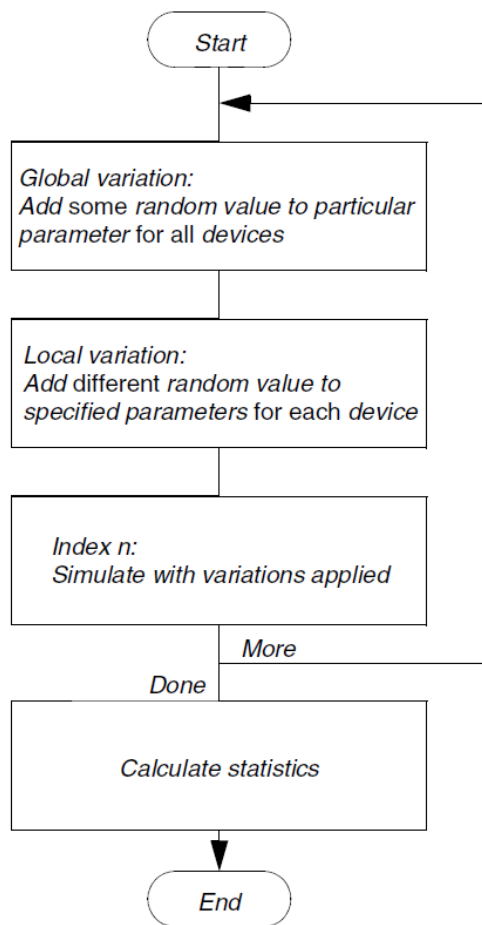
```
.Variation  
  Definitions for Global and Local Variations  
.End_Variation
```

Figure 18 shows the Monte Carlo simulation flow when you specify global and local variations.

- Sample number 1 of a Monte Carlo analysis always executes with nominal values and no variation.
- For subsequent samples, the PrimeSim tool updates the parameters specified for variation in the Variation Block with random values.
- For global variations, the same random value for all elements that share a common model changes a specified parameter.

- For local variation, a different random value for each element changes the specified parameter.
- The changes due to global and local variations are additive and saved in a file for post-processing.
- After an update of the elements, the PrimeSim tool executes the simulation and saves the measurement results. When the PrimeSim tool simulates all the requested samples, it calculates the statistics of the measurement results and includes them in the run listing.

Figure 18 Monte Carlo Analysis Flow in the PrimeSim Tool



See the following sections for these topics:

- [Input Syntax](#)
- [Monte Carlo-Specific Variation Block Options](#)
- [Output for Variation Block Monte Carlo](#)

Input Syntax

You always execute Monte Carlo analysis in conjunction with another analysis:

```
.DC sweepVar start stop step [SWEEP MONTE=MCcommand]  
.AC type step start stop [SWEEP MONTE=MCcommand]  
.TRAN step start stop [SWEEP MONTE=MCcommand]
```

Syntax for *MCcommand*:

```
MONTE=val | list (num) | val Firstrun=num |  
+ list (num1:num2 [num3] [num4:num5])
```

Parameter	Description
<i>val</i>	Specifies the number of random samples to produce.
<i>Firstrun=num</i>	Specifies the sample number on which the simulation starts.
<i>list (num)</i>	Specifies the sample number to execute.
<i>list (num1:num2 num3 num4:num5)</i>	Executes samples from <i>num1</i> to <i>num2</i> , sample <i>num3</i> , and samples from <i>num4</i> to <i>num5</i> .

The parameter values and results are always the same for a particular sample, whether generated in one pass or using *Firstrun* or the *list* syntax. Therefore, you can split Monte Carlo analyses or distribute them across multiple machines.

DC Sweep Examples

These examples apply a DC sweep to a parameter *k*. The first case produces 10 samples. The second case produces 5 samples, starting with sample number 6. The last example produces sample numbers 5, 6, 7, and 10.

```
.dc k start=2 stop=4 step=0.5 monte=10  
.dc k start=2 stop=4 step=0.5 monte=5 firstrun=6  
.dc k start=2 stop=4 step=0.5 monte=list (5:7 10)
```

Monte Carlo-Specific Variation Block Options

When you execute simulations by using the Variation Block, the PrimeSim tool ignores control options for the traditional Monte Carlo style. However, you can specify the following Monte Carlo-specific options in the first section of the Variation Block:

- `Option External_File=filename`

Enables read-in of an external block line-by-line-during the simulation stage. This command distributes memory consumption and avoids overtaxing the front end with a data block containing large samples. This option is also available for DP + DC Monte Carlo.

- `Option Normal_Limit=Value`

Limits the range for the numbers generated by the random number generator for standard normal distributions. The default value is 20 for sampling methods SRS, LHS, SOBOL, and NIED. The default value is 4 for the OFAT and FACTORIAL sampling methods. For example, numbers in the range $-/+4$ are created. The allowed range for the option is 0.1 to 20. Negative values are automatically reset to the default.

- `Option Ignore_Local_Variation=Yes`

Excludes effects of local variations in simulation. Default is `No`.

- `Option Ignore_Global_Variation=Yes`

Excludes effects of global variations in simulation. Default is `No`.

In the following example, global variations are not simulated.

```
.Variation
  Option Ignore_Global_Variation=Yes
  .Global_Variation
    Definitions for global variations
  .End_Global_Variation
  .Local_Variation
    Definitions for local variations
  .End_Local_Variation
.End_Variation
```

- `Option Vary_Only Subckts=SubcktList`

Limits variation on the specified subcircuits. Use this option to limit variation to the specified subcircuits, or use the `Do_Not_Vary` option, *but not both*. Actual subcircuit names are specified here (hierarchical names are also supported).

- `Option Do_Not_Vary Subckts=SubcktList`

Excludes variation on the specified subcircuits. Use this option to limit variation to the specified subcircuits, or use the `Vary_Only` option, *but not both*. Actual subcircuit names are specified here (hierarchical names are also supported).

- Option `Vary_Only instances=instance1[, instance2...]`

Limits the variations on the specified instances only.

- Option `Do_Not_Vary instances=instance1[, instance2...]`

Excludes the specified elements varying during the Monte Carlo simulation.

- Option `MC_File_Only=yes|no`

Generates a random number sample file (*.mc0) without invoking any analysis (applicable to AGAUSS style also). The feature is useful for an external block sampling simulation when you want to modify the samples before running the Monte Carlo simulation. If the netlist has a Monte Carlo command, then the MC command provides the number of samples; if the netlist has no MC command, then the number of samples is zero.

- Option `Modified_Distribution = Scale(val)`

Modifies/transforms the built-in distribution for Monte Carlo analysis.

Note:

This option affects only the scaling of Normal/Gaussian random parameters (while not affecting LIMIT/UNIFORM distributions). When `val > 0`, the sample points are multiplied by `val`.

Output for Variation Block Monte Carlo

The following sections cover these topics:

- [Simulation Listing](#)
- [Measurement Output File](#)

Simulation Listing

The output listing file contains a summary of the names of all input parameters that are subject to global or local variations. The measured results print for each sample. Then the PrimeSim tool reports the statistics for the measured data.

Partial printout of an output listing:

```
MONTE CARLO DEFINITIONS
Random number generator is default, and stream = 1
Global variations:      model                parameter
```

```

                                snps20n          vth0
                                snps20n          u0

Local variations:      model      parameter
                        snps20n    vth0
                        snps20n    u0

Element variations:    element    parameter
                        r1         r

*** monte carlo  index =      1 ***
    systoffset1=  1.3997E-03
*** monte carlo  index =      2 ***
    systoffset1= -9.2694E-04

MONTE CARLO STATISTICS
  meas_variable = systoffset
  mean  =    1.4398m      varian =    1.2391u
  sigma =    1.1132m      avgdev = 893.3815u
  max   =    5.3035m      min    =   -1.4532m
  1-sigma =  1.1132m      median =    1.4184m

```

More detailed statistics are given in the *.ava.report file.

Measurement Output File

Measure commands save simulation results depend on the analysis type, the name of the result file has an extension of .ms0, .ma0, or .mt0.

Bi-Section Runs

The PrimeSim tool also supports Monte Carlo analysis performed during bisection simulations.

Syntax

```
.tran 10p 'tmax' sweep Optimize = Opt1 Result = GoalValue Model = optmod
monte=1000
```

Incremental Runs

Use the -incremental *directory/prefix* command-line option to add more samples to a completed Monte Carlo analysis run and merge the results.

Example

For this example, run the initial Monte Carlo analysis with the following netlist and run commands:

- Netlist command:

```
.tran lp 30n sweep monte=10
```

- Run command:

```
primesim in.sp -o ./Outdir/test
```

This generates a measure file, `Outdir/test.mt0` with results for Monte Carlo indexes 1-10.

Next, to run additional samples:

1. Modify the original netlist command by either:

- Editing it directly in the netlist:

```
.tran lp 30 n sweep monte=list(11:20)
```

- Using the [primesim_monte_cmd_overwrite](#) option:

```
.option primesim_monte_cmd_overwrite=list(11:20)
```

2. Run the simulator using the `-incremental directory/output_prefix` command-line option:

```
primesim in.sp -o ./Outdir/test2 -incremental Outdir/test
```

This generates a measure file, `./Outdir/test2.mt0` which has the merged measure results for Monte Carlo indexes 1-20. The run also generates an `Outdir/test2_incr.mt0` file which has the incremental measure results from Monte Carlo results 11-20.

Random Number Generation Sequence

The PrimeSim tool can generate the same random number sequence for a different subcircuit; see [primesim_identical_mc_instance_file](#) for more information.

Overview of the Variation Block

The characteristics of circuits produced in semiconductor processing are subject to variability, as is the case for any manufactured product. For a given target technology, the nominal device characteristics are described with a set of parameters, which applies to a certain device model (for example, BSIM4). In the PrimeSim tool, the variability of the model parameters is described through a *Variation Block*. A Variation Block is a container

for specifying variations introduced by the effects in manufacturing on geometry and model parameters.

Variations in the PrimeSim tool are modeled as Global and Local variations.

- *Global Variations* are variations in device characteristics from lot to lot, wafer to wafer, and chip to chip; they are caused by variations in starting material and differences between equipment and manufacturing procedures. Global Variations affect all devices with the same model name in the same way.
- *Local Variations* are defined as variations between devices in proximity, or with common centroid layout on the same chip; they are caused by microscopic variations in materials and geometry, and affect different devices differently.

Both classes can be described in the Variation Block in a flexible way by user-defined expressions. Because there are currently no industry-wide standards for specifying process variability, this feature allows each company to implement their own proprietary model for variability. The Variation Block is generally provided by a modeling group, very similar to device models (for example, BSIM) because it must be created specifically for each technology from test circuits.

Like a model, the Variation Block can be part of a library which is encrypted; therefore, the content is not accessible to the designers. They can introduce additional Variation Blocks in their netlist to define options and variations on generic elements.

The structure of the Variation Block allows for building expressions to model interdependence and hierarchy of the variations. For example, one random variable can control the variation in oxide thickness of both PMOS and NMOS devices, as it is generally the same for both types of devices.

Note that the earlier methods for specifying variation are not compatible with the Variation Block.

Variation Block Structure

A Variation Block is divided into three sections:

- General section
- Subblock for Global Variations
- Subblock for Local Variations

This section presents the syntax of a Variation Block, followed by a discussion of the contents of the three sections.

```
.Variation
  Define options
  Define common parameters that apply to all subblocks
```

```
.Global_Variation
    Define the univariate independent random variables
    Define additional random variables through transformations
    Define variations of model parameters
.Element_Variation
    Define variations of element parameters
.End_Element_Variation
.End_Global_Variation
.Local_Variation
    Define the univariate independent random variables
    Define additional random variables through transformations
    Define variations of model parameters
.Element_Variation
    Define variations of element parameters
.End_Element_Variation
.End_Local_Variation
.End_Variation
```

General Section

In the general section, options can be defined that control the variability analyses that use the content of the Variation Block. Options can be specified, one per logical record.

Note:

`.OPTION` (with a leading period) does not work for options specified in the Variation Block.

The correct Variation Block syntax is:

```
Option OptionName = value
```

Parameters, also, can be defined that apply to all subblocks in which variations are specified; however, these cannot contain any distribution-related functions. Parameters defined within a Variation Block have local name scope and are completely independent of parameters defined outside it.

For example: `parameter PI=3.1416`

Subblocks for Global and Local Variations

Within the variation subblocks, univariate independent random variables can be defined. These are random variables with specific distributions over a certain sample space. Additional random variables can be generated through transformations. These random variables form the basis for correlations and complicated distributions.

A basic rule of the Variation Block approach is to place the model definition on the top level, instead of inside a subcircuit, as necessary in the old approach. In both subblocks,

variations on model parameters can be defined. This is where Global or Local Variations on the parameters of semiconductor devices are specified.

Note:

The `.MALIAS` command is supported for diode, BJT, JFET, and MOSFET models in `.Global_Variation` and `.Local_Variation` blocks.

A special section within the subblock for Local Variations allows for defining Local Variations on elements. Use this section either for specifying local temperature variations or variations on generic elements that do not have a model, as used early in the pre-layout design phase, or for off-chip components; for example, resistors and capacitors. Local and Global variation support block operator brackets.

The following sections discuss these topics:

- [Independent Random Variables](#)
- [Dependent Random Variables](#)
- [Absolute Versus Relative Variation](#)
- [Variations on Model Parameters](#)
- [Variations on Subcircuit Parameters](#)
- [Variations on Top-Level Parameters](#)
- [Variations on Temperature](#)
- [Access Functions, `Get\_E\(\)`, `Get\_P\(\)`, `Get\_M\(\)`, `Get\_O\(\)`](#)

Independent Random Variables

When describing variations, a standard normal (Gaussian) distribution is assumed, unless otherwise specified explicitly. This default behavior is explained in later sections. Other types of distributions or correlations are modeled by applying transformations to the independent random variables. These independent random variables are derived from three basic distributions:

- Uniform distribution, `U()`: Defined over the range from -0.5 to 0.5
- Normal distribution, `N()`: With mean=0 and variance=1, default range +/-4
- Use the `LIMIT` distribution to create discrete “corners” for a variable. Specify it as:

```
Parameter xx=LIMIT(nominal_val, abs_variation)
```

These distributions cannot be defined within expressions; variables must be assigned and then the variables can be used within expressions.

The `LIMIT` keyword provides a random-limit distribution function by using absolute variation. The syntax adds `+/- abs_variation` to `nominal_val` based on whether the random outcome of an underlying uniform distribution is over the range of -1 to 2.

In the following Variation Block example, `aa=Limit(-2,2)` means the `nominal_value` is -2, and the `abs_variation` is 2.

```
.Variation
.Local_Variation
  Parameter aa = Limit(-2,2)
  R Rmodel res= Perturb('aa')
.End_Local_Variation
.End_Variation
```

- User-defined cumulative distribution function, `CDF(xyPairs)`:

If $f(x)$ is the probability density of a random variable x , then the cumulative distribution function is the integral of $f(x)$. A cumulative distribution function can be approximated by a piecewise linear function, which can be described as a sequence of pairs of points $[x_i, y_i]$. The following rules apply:

- At least two pairs are required
- White space or a comma is required between each number
- The CDF starts at zero: $y_1=0$
- The CDF ends at one: $y_n=1$
- x_i values must be monotonically increasing $x_{i+1} > x_i$
- y_i values must be monotonically non-decreasing $y_{i+1} \geq y_i$
- $\int_{-\infty}^{\infty} x \cdot f(x) \cdot dx = 0$

where, the probability density function, $f(x)$, is the derivative of the cumulative density.

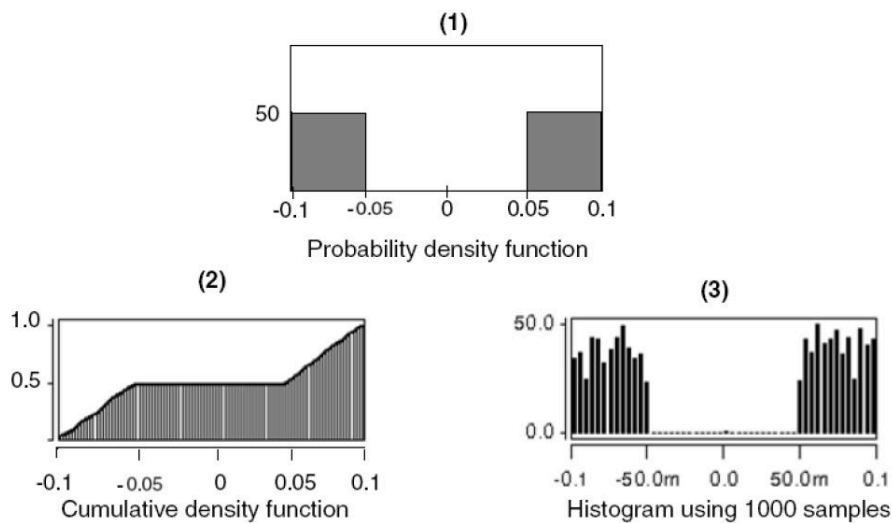
Example

The probability density function is shown in [Figure 19 \(1\)](#). [Figure 19 \(2\)](#) gives the corresponding cumulative distribution function. This is coded in the Variation Block as:

```
Parameter var=CDF(-0.1 0 -0.05 0.5 0.05 0.5 0.1 1.0)
```

The histogram generated by taking 1000 samples is shown in [Figure 19 \(3\)](#).

Figure 19 Probability, Cumulative Density, and 1000-sample Histogram



The distributions $N()$ and $U()$ do not accept any arguments.

The syntax for defining independent random variables is:

Parameter $a=U()$ $b=N()$ $c=CDF(x_1, y_1, \dots, x_n, y_n)$

These distributions cannot be defined within expressions; variables must be assigned and then the variables can be used within expressions.

The **LIMIT** keyword provides a random-limit distribution function by using absolute variation. The syntax adds $\pm \text{abs_variation}$ to nominal_val based on whether the random outcome of a -1 to 1 distribution is greater than or less than 0.

Dependent Random Variables

To model distributions which are more complicated than the ones which are available through the predefined independent random variables, transformations can be applied by using expressions on independent random variables. A dependent variable can also be created as a function of more than one independent random variable to express correlation.

Example 1

This example creates a random variable with normal distribution, with mean A and standard deviation B:

Parameter $\text{var}=N()$ $Y='A + B * \text{var}'$

Example 2

This example creates a random variable with a uniform distribution from D to E, where D and E are arbitrary constants:

```
Parameter var=U() Y='0.5*(D+E) + (E-D) * var'
```

Example 3

A variable x has a log-normal distribution if $\ln(x)$ is normally distributed. The probability density function for the log-normal distribution is: $f(x) = \frac{1}{\sigma x \sqrt{2\pi}} \cdot \exp\left(-\frac{(\ln(x)-\mu)^2}{2\sigma^2}\right)$

where μ is the mean and σ the standard deviation of associated normal distribution. Samples from such a distribution can be generated as:

```
Parameter      var=N()
Parameter      nor='mu+sigma*var'
Parameter      lognor='exp(nor)'
```

Absolute Versus Relative Variation

By default, the specified variation is absolute, which means additive to the original model or element parameter; however, sometimes it is more appropriate to specify relative variations that are defined by appending a space and a percent sign (%) to the expression. The simulator divides the result of the expression by 100, and multiplies the result by the original parameter value and the random number from the appropriate generator to calculate the change.

Example

In the following example, the variation on the threshold parameter `vth0` is specified as normal with absolute sigma of 80 or 70 mV, and the variation on the mobility `u0` is specified as relative 15 or 13 percent.

```
.Global_Variation
  Nmos snps20N vth0=0.08 u0=15 %
  Pmos snps20P vth0=0.07 u0=13 %
.End_Global_Variation
```

Variations on Model Parameters

Variations on model parameters can be defined in subblocks for Global and Local Variation. In the course of the simulation, these variations are then applied to the specified device model parameters. Model parameter variations are described with the following syntax:

```
Model_Type Model_Name Model_Parameter=Expression for Sigma
```

The syntax `Model_Parameter=Expression` for Sigma is a shorthand notation for `Variation_in_Model_Parameter='Expression for Sigma'`.

If the expression references only constants and parameters that evaluate to constants, then a Gaussian variation with zero mean and a sigma equal to the expression is automatically implied. To describe variation as a function of previously defined independent random variables, use the construct `Perturb()`, with the following syntax:

```
Model_Type Model_Name Model_Parameter=Perturb('Expression')
```

The expression for sigma should be enclosed in quotation marks. The following lines define a global Variation, with implicit normal distribution, with zero mean and sigma of 10, on the parameter `rsh` of resistors with model `Rpoly`.

```
.Global_Variation
  R Rpoly rsh=10
.End_Global_Variation
```

In the next example, the independent variable `Toxvar` is used to model global Variations on oxide thickness. `Toxvar` is an independent random variable with a normal distribution, with mean=0 and sigma=1. In the device models `nch` and `pch`, `Toxvar` is applied to the parameters `tox` with a different multiplier. The oxide thicknesses in the two models vary in parallel; they are correlated.

```
.Global_Variation
  Parameter Toxvar=N()
  Nmos nch tox=Perturb('7e-10*Toxvar')
  Pmos pch tox=Perturb('8e-10*Toxvar')
.End_Global_Variation
```

The PrimeSim tool supports the following model types: NMOS, PMOS, R, Q, D, and C.

Variations can only be defined on parameters that are explicitly specified in the associated device model.

For binned models, variations can be defined separately by specifying the model name with the bin extension; for example, devices from bins 1 and 2 receive different variation on the parameter `lint`, which models length variation:

```
Nmos snps20N.1 lint=10n
Nmos snps20N.2 lint=12n
```

Variations on Subcircuit Parameters

The Variation Block allows for defining variation on parameters, which are specified in the subcircuit definition record with a default value. Default values can be overwritten by specifying them at subcircuit instantiation.

The syntax is:

```
Subckt SubcktName Parameter='expression for sigma'
```


The following rules apply for these types of definitions:

- Only parameters that are defined as formal numeric arguments on the subcircuit definition record can be subject to variation. (This is the line which starts with `.SUBCKT` and possibly has continuation lines.)
- The subcircuit must not be defined within another subcircuit.
- If the subcircuit contains a model, then variations on the model parameters (as described in [Variations on Model Parameters](#)) are not supported. Instead, variations need to be defined on a subcircuit parameter and the parameter used inside the model.

The subcircuit parameters variation feature addresses the following three needs:

- A component is defined with an expression, not available in a model:

```
r1 1 0 'Rsh*1/w*(1+b1*(tanh(b2*abs(v(1,0)/1))))'
```

This expression models a voltage-dependent resistor, with non-linear dependence not available in a traditional model. If this resistor is called within a subcircuit, and parameters are specified on the subcircuit definition record, then variation can be modeled, for example, on `Rsh`, `l`, and `w`.

```
i1 0 1 1m
x1 1 0 rtanh
.subckt rtanh a b rsh=1k w=1u l=1u
.param b1 = -0.4 b2 = 8u
r1 a b 'Rsh*1/w*(1+b1*(tanh(b2*abs(v(a,b)/1))))'
.ends
.Variation
.Global_Variation
    subckt rtanh rsh=10 %
.End_Global_Variation
.Local_Variation
    subckt rtanh rsh=3 %
.End_Local_Variation
.End_Variation
```

- A component is represented by a network and the subcomponents must have the same value when local variations are specified:

```
r1 end1 middle rmodel w='w+dw' l='l/2+d1'
r2 end2 middle rmodel w='w+dw' l='l/2+d1'
c1 middle sub cmodel w=w l=l
```

In this example, the resistor has a center tap, where a capacitor is connected. If these three components are defined within a subcircuit and parameters `dw`, `dl`, and `rsh` are defined on the subcircuit definition record, then the two resistors of the same network always have the same value; local variations only cause different instantiations of the subcircuit to have different equivalent resistance between the network terminals.

- You must calculate the value of a device model parameter through an equation because the built-in equations are not adequate. For example:

```
.subckt nch n1 n2 n3 n4 dvth0_glob=0 dvth0_loc=0 du0_glob=0
+ du0_loc=0
+ dtox=0 dlnt=0 dwint=0 l=60n w=120n as='w*90n' ad='w*90n' ...
.param vth0_base=0.345 u0_base=0.015
.param vth0_geo=function1(w,l,temper,vth0_base)
.param u0_geo=function2(w,l,temper,u0_base)
.param dvth0_geo=function3(w,l,dvth0_loc)
.param du0_geo=function4(w,l,temper,du0_loc)
M1 n1 n2 n3 n4 nch25 w=w l=l as=as ad=ad ...
.MODEL nch25 NMOS LEVEL = 54
+ vth0='vth0_geo+dvth0_glob+dvth0_geo'
+ u0='u0_geo*(1+du0_glob)*(1+du0_geo)'
+ tox='2.6n+dtox' lint='2.1n+dlnt' wint='5.3n+dwint' ...
.ends
X1 d1 g1 s1 b1 nch l=60n w=150n
X2 d2 g2 s2 b2 nch l=80n w=120n
.Variation
  .Global_Variation
    subckt nch dvth0_glob=0.03 dtox=0.12n du0_glob=0.2 dlnt=2n
    + dwint=3n
  .End_Global_Variation
  .Local_Variation
    subckt nch dvth0_loc=2.0m dtox=0.03n du0_loc=0.03 dlnt=21p
    + dwint=47p
  .End_Local_Variation
.End_Variation
```

The values of model parameters `vth0` and `u0` are defined through user-defined equations (`function1` and `function2`), with dependency on device size and temperature. This necessitates a local model (`nch25`). The parameters with variations are declared on the subcircuit definition line.

In this example, global and local variations are processed differently through the subcircuit, therefore the respective variations have to be specified on separate parameters.

- Global variations (`dvth0_glob` and `du0_glob`) are applied to the model parameters `vth0` and `u0` directly.
- Local variation definitions (`dvth0_loc` and `du0_loc`) are adjusted for device size using `function3` and `function4`, and result then in the variations `dvth0_geo` and `du0_geo` applied to the model parameters `vth0` and `u0`.

While this use model is supported, it is not desirable because it leads to one model per device, which is inefficient in terms of memory and performance.

Variations on Top-Level Parameters

Variations on top-level parameters can be defined for Global Variation. The keyword `Top` is available to specify top-level parameter variation.

```
Top top_level_parameter='expression'  
Top top_level_parameter=Perturb('expression')
```

The following example defines the +/-10% global variation on parameter VDD with uniform distribution.

```
.param vdd=2.5  
.Variation  
    .Global_Variation  
        Parameter uniV=U()  
        Top vdd=perturb('20*uniV') %  
    .End_Global_Variation  
.End_Variation
```

Note:

The top-level parameter variation can only be specified in Global variation.

Variations on Temperature

You can define Variation on temperature for global variations to support the whole space of Process-Voltage-Temperature. The temperature variations affect all devices in the netlist.

The keyword `Temp` (or `Temper`) specifies temperature variation.

```
Temp temp='expression'  
Temp temp=Perturb('expression')
```

The following example defines the global variation on temperature.

```
.Variation  
    .Global_Variation  
        Temp temp='10'  
    .End_Global_Variation  
.End_Variation
```

Note:

You can only specify temperature variation in global variation.

Access Functions, `Get_E()`, `Get_P()`, `Get_M()`, `Get_O()`

Caution:

An error results if any of the arguments to these functions is invalid or undefined. This strong action is due to the need for the technology library to be bug free.

Get_E()

Certain variations depend on element geometry, as defined with parameters at instantiation. The `Get_E()` access function (only supported by the Variation Block) allows accessing these parameters in expressions by using the following syntax:

```
Get_E(Element_Parameter)
```

where *Element_Parameter* is the name of an element parameter, which you must define on the instantiation line (except for the `DTEMP` parameter and the multiplier `M` which have implicit values). Use this access function for specifying variations as a function of device geometry. The `Get_E()` access function reports the effective device geometry, after resolving parameters, scales and adjustments by process parameters, such as, `xw`, `xl`, `wint`, `lint`.

For example, you often specify that the local variation on the threshold is as inversely proportional to the square root of the total area of the device, as calculated from the product of the element parameters `W`, `L`, and `M`.

```
Nmos nch vth0='1.234e-9/sqrt(Get_E(W)*Get_E(L)*Get_E(M))'
```

In addition, Variation Block can handle cases of calling `NF` and `M` for elemental variation with the `Get_E()` function. For example:

```
.Variation
  .Global_Variation
    parameter var = N()
  .Element_Variation
    M(model_name ~= 'NCH') W = Perturb('var*100n*Get_E(NF)')
  .End_Element_Variation
  .End_Global_Variation
  .End_Variation
```

Get_E() Function for Obtaining Channel L/W for MOSFETs

You can also use the `get_e()` function to obtain channel length and width with MOSFET models as follows:

- `get_e(LDRAWN) | get_e(WDRAWN)` – Returns user-defined L/W for MOSFET models.

Get_P()

Another function allows for accessing the values of global parameters by using the following syntax:

```
Get_P(Global_Parameter)
```

The circuit context provides the parameter value, for example, from the subcircuit, if defined inside, otherwise from the top level. In the following example, sweep parameter `tol` determines the resistor variation:

```
.param tol=1
ral 1 0 1k
il 0 1 1m
.Variation
    .Local_Variation
        .Element_Variation
            R R='Get_P(tol)' %
        .End_Element_Variation
    .End_Local_Variation
.End_Variation
.dc tol 1 5 1 monte=100
```

Get_M()

The Variation Block format also allows the access of model parameters. The PrimeSim tool provides the function `Get_M()` for this purpose.

```
Get_M(Model_Parameter_Name)
```

The `Get_M()` function normally has a single argument.

```
.Variation
    .Global_Variation
        nmos snps20N u0 ='2.345e-6*GET_M(u0)'
    .End_Global_Variation
.End_Variation
```

Get_O() to Return the Value of PrimeSim Options

If a variation must be expressed as a function of a simulator option (specified as `.option=optionval` outside the Variation Block), the access function `Get_O()` is available, using the construct `Get_O(option_name)`.

Variations of Element Parameters

Variations affect devices not only in the underlying model parameters, but also through variations of properties specified at instantiation of an element, or variations on implied properties, such as local temperature. Also, early in the design phase, passive devices sometimes have only a nominal value, but no model as yet because designers have made no decision on a specific implementation. For these elements, you can specify variations on the implicit value parameter; for example: `R1 1 0 1k`.

You define variations on element parameters for Local Variations in a section within the Local Variation subblock.

You describe element parameter variations with the following syntax:

```
Element_Type Element_Parameter = 'Expression for Sigma'
```

The syntax `Element_Parameter = 'Expression for Sigma'` is a shorthand notation for:

```
Variation_in_Element_Parameter = Expression for Sigma
```

If the expression references only constants and parameters that evaluate to constants, then it automatically implies that a Gaussian variation with zero mean and a sigma is equal to the expression. To describe variation as a function of previously defined independent random variables, use the construct `Perturb()`, with the following syntax:

```
Element_Type Element_Parameter = Perturb('Expression')
```

Enclose the expressions in quotation marks.

The following lines define a normal distribution with a sigma of 10 on the resistors without a model:

```
.Element_Variation
  R R=10
.End_Element_Variation
```

In the following example, the specified temperature variation affects only resistor `ra2` with a uniform distribution from 0 to 10 degrees (the resistor resides next to a power device).

```
ra1 1 0 1k
rb1 2 0 1k
ra2 3 0 rpoly l=10u w=1u
rb2 4 0 rpoly l=10u w=1u
.model rpoly r rsh=100 tc1=0.01
.Variation
  .Local_Variation
    .Element_Variation
      Parameter tempvar=U()
      R(Element_Name~='ra*' && Model_Name~='rpoly')
      + dtemp=Perturb('10*tempvar+5')
    .End_Element_Variation
  .End_Local_Variation
.End_Variation
```

Because different classes of devices might be affected differently, use of a condition clause provides a selection mechanism based on element name and model name:

```
Element_Type(condition_clause) Element_Parameter= 'Expression for Sigma'
```

The condition clause allows for specifying variations on selected elements, according to their name or associated model. You can indicate wildcard substitutions as a question mark (?) for a single character and an asterisk (*) for multiple characters.

Examples for condition clause syntax are:

```
Element_Type(model_name~='modelNameA')
Element_Type(element_name~='elNameB')
Element_Type(model_name~='modelNameC' OPERATOR
    element_name~='elNameD') par='expression'
Element_Type(model_name~='modelNameE' OPERATOR
    model_name~='modelNameF') par='expression'
Element_Type(element_name~='elNameG' OPERATOR    element_name~='elNameH')
    par='expression'
```

where OPERATOR can be && (AND), || (OR). The operator “~=” stands for “matches”.

All pattern matching operations are case-insensitive. The PrimeSim tool ignores a leading subcircuit prefix when matching the element name.

Example

In this example, only resistor `ra1` varies.

```
ra1 1 0 1k
rb1 2 0 1k
.Variation
    .Local_Variation
        .Element_Variation
            R(element_name~='ra*') R=20
        .End_Element_Variation
    .End_Local_Variation
.End_Variation
```

Supported element types and their parameters are:

Table 39 Supported Elements and Parameters

Element	Parameters
M	DTEMP, L, W, AD, AS, PD, PS, NRD, NRS, RDC, RSC, VDS, VGS, VBS, DELVTO
R	Rval* DTEMP, L, W, TC1, TC2, C, AC, SCALE
C	Cval* DTEMP, L, W
Q	AREA DTEMP, AREA, AREAB, AREAC, TEMP
J	L, W, DTEMP
D	DTEMP, L, W
L	Lval* DTEMP, TC1, TC2
I	DCval* mag phase
V	DCval* mag phase

Table 39 Supported Elements and Parameters (Continued)

Element	Parameters
Note: The asterisk “*” denotes implicit value parameter. The DTEMP parameter is implicit; it needs specification on the element instantiation line.	

Example for Voltage Source

Netlist element:	V1 1 0 0.1 AC
Variation definition:	• V DC=5% (5% of 0.1) • V MAG=5% (5% of 1) • V PHASE=5 (5 degrees)

You define variations on element parameters for Global Variations in a section within the Global Variation subblock with almost the same syntax as within the Local Variation. But there is a limitation to when you can use a condition clause: If there is a condition clause in the variation definition line of a Global Element Variation, then use the Variation Block keyword `Parameter` to designate at least one independent random variable. The `Perturb` function uses an independent random variable element parameter perturbation. Other forms of perturbation are illegal. For example:

```
Parameter var1=N() var2=U()  
Element_Type (condition_clause) Element_Parameter = Perturb('Expression')
```

Example

In this example, `ra1` and `rb1` varies with global variation. So in each Monte Carlo trial, `ra1` and `rb1` are of the same resistor value.

```
ra1 1 0 1k  
rb1 2 0 1k  
.Variation  
  .Global_Variation  
    .Element_Variation  
      Parameter a=N()  
      R(element_name~='r*') R=Perturb('20*a')  
    .End_Element_Variation  
  .End_Global_Variation  
.End_Variation
```

Note:

If there are only two resistors of `ra1` and `rb1` in the netlist, then an equivalent definition of Global Element Variation is:

```
.Variation  
  .Global_Variation
```



```
.Element_Variation
  R R=20
.End_Element_Variation
.End_Global_Variation
.End_Variation
```

But the next definition is *invalid* because the condition clause matches all resistors in the netlist:

```
.Variation
.Global_Variation
.Element_Variation
  R(element_name~='r*') R=20
.End_Element_Variation
.End_Global_Variation
.End_Variation
```

Special Rules Regarding Variation Block Usage

Generally, a foundry creates the contents of the Variation Block. To safeguard against unintentional overwriting of these variation definitions:

- The name-space of the Variation Block is separate from the netlist contents.
- Once you specify a variation on a parameter, you cannot redefine it later, even in `.ALTER` statements. For example, if you want to change the corners defined in a model library file with a `.ALTER` statement, then you must specify the Variation Block in a separate `*.lib` section.

External Sampling

You can also execute a data set of externally created perturbations instead of relying on one of the built-in sampling methods. External sampling allows design and process exploration tools to run statistical experiments with the variables for each sample under their full control. In this way, for example, you can exclude certain transistors from the analysis (that is, have the parameters remain fixed instead of being randomly varied).

Note:

External sampling does not support encrypted or protected model libraries.

The following sections provide these details:

- [Syntax](#)
- [Controlling the Read-in of an External File](#)

Syntax

The external sampling feature is defined in two parts in the Variation Block: a data block and an option.

The data block syntax is the same as for the regular PrimeSim data block from `.Data` to `.EndData`. The first variable is always the index. All identifiers for the variables start with `I` because this is the only variable type which can be set externally. You invoke the feature itself by specifying the external sampling method, with the appropriate block name. For example:

```
.Variation
.Data test1
Index p1:@:IGN      snps20p:@:vth0:@:ILN  x1.mn1:@:q1:@:ILN  x2.mn1:@:vth0:@:ILN
1      .0            .0            .0            .0
2      .0            0.1086        .0            .0
3      .3            .0            .0            .0
4      .0            .0            1.0            .0
5      .0            .0            .0            1.0
6      .0            .0            1.0            -1.0
.EndData

Option Sampling_Method=External Block_Name=test1
.End_Variation
```

To run a particular sample from the data block, use the following:

```
monte = list(<num>) or
```

```
monte = list(<num1:num2> <num3> <num4:num5>)
```

```
monte = list(n1:n2) can exceed a limit in the data block.
```

If the netlist or the model, or both, are encrypted, the hash codes printed in the parameter file are recognized by the PrimeSim tool when reading in the external sampling data block.

Additional rules:

1. The PrimeSim tool does not check the range of values in the supplied data block against option value `Normal_Limit`.
2. Independent random variables which are not specified in the data block are assigned new random values generated by a new seed.

Controlling the Read-in of an External File

Use `Option External_File=filename` to enable read in of an external data block line-by-line during the simulation stage. This command distributes memory consumption and

avoids overtaxing the front end with a data block containing large samples. This option is also available for DP+ DC Monte Carlo. For example:

```
Option Sampling_Method=External Block_Name=extern_data
+ External_File=extern.mc0
.Data extern_data
...
.Enddata
```

Reusing a Modified .mc0 File

The PrimeSim tool generates an .mc0 file which you can modify and reuse for a new Monte Carlo analysis with the following procedure.

1. Add the following option to the initial Monte Carlo analysis:

```
.option primesim_gen_mc0=1
```

2. Run the Monte Carlo simulation, then modify the generated .mc0 file as follows:

- a. Remove the \$ comment character from the following lines:

```
$.Option Sampling_Method = External Block_Name = extern_data
$.data extern_data
```

- b. Modify any or all of the parameter values in columns 2 through N on the rows of data following the index line.

Note:

The first column is the index and must remain as integers 1 through N .

3. Add a .INCLUDE statement to the input deck that points to the modified .mc0 file:

```
.inc path_to_file/modified_mc0_filename
```

4. Rerun the Monte Carlo simulation with the modified variations.

Data Mining Using PrimeSim With PrimeSim AVA

Running Monte Carlo simulations is expensive and sometimes the raw output does not provide sufficient insight into the circuit behavior or guidance on how the circuit could be improved. The PrimeSim tool is compatible with the PrimeSim AVA tool's data mining flow, outputting *.mc#, *.ava.report, *.ava.annotate, *.ava.qqt0.csv, *.ava.qqt0.wv_plt, and *.ava.qqt0.pdf files, which can help you to analyze all the Monte Carlo results more efficiently.

The following flows show how the PrimeSim tool can make use of the PrimeSim HSPICE tool's data mining feature:

- [One-Step Flow](#)
- [Two-Step Flow](#)

One-Step Flow

For this one-step flow, simply invoke the PrimeSim tool with the `-datamining` command-line option. The PrimeSim tool runs the Monte Carlo simulation first, and then automatically invokes the PrimeSim AVA tool to execute the data mining post-processing procedure.

Syntax

```
primesim -datamining [-o outname] input.sp
```

In this flow, the PrimeSim tool automatically adds option `primesim_gen_mc0=1` and generates the following data mining files:

- `outname_datamining.json` – Setting options for the PrimeSim AVA simulation.
- `outname_datamining.ava.log` – Log file of data mining post-processing.

Notes:

- With `primesim_gen_mc0=1`, the `*.mt#` (or `*.ma#`, `*.md#`) files also differ slightly compared to the default setting, in that the statistic results of average (`avg`), standard (`std`), minimum (`min`), and maximum (`max`) do not show up in the bottom of the measurement file.

Two-Step Flow

The PrimeSim tool also provides a standalone data mining flow which is compatible with the PrimeSim AVA tool.

Syntax

For this two-step flow, you must add `primesim_gen_mc0=1` to the netlist and prepare file `primesim_datamining.json` manually. Then run your Monte Carlo simulation using the PrimeSim tool.

Next, enter the following command to execute the PrimeSim AVA standalone data mining flow based on the Monte Carlo results from the PrimeSim tool:

```
primesim [-spice] -adv primesim_datamining.json [-o outname]
```

Note:

`primesim_datamining.json` is the same as `outname_datamining.json` mentioned previously in [One-Step Flow](#).

For more detailed information on Monte Carlo data mining support, see Standalone Data Mining in PrimeSim HSPICE in the *PrimeSim™ HSPICE® User Guide: Basic Simulation and Analysis*.

18

Multi-Technology Simulation of 3-D Integrated Circuit (3DIC)

This chapter describes the PrimeSim solution to modularize IC chips inside a 3-D integrated circuit (3DIC) to be simulated as a single circuit.

A three-dimensional integrated circuit (3DIC) is a single chip that integrates two or more layers of active electronic components into a single circuit. All components on the layers communicate use on-chip signaling, whether vertically or horizontally. The layers in a 3DIC are IC designs that might be fabricated using different technology processes and operate at different sets of parameters, including temperature and geometry scaling.

The PrimeSim tool modularizes these IC designs and enables the simulation of a 3DIC as a single circuit. The PrimeSim tool supports PrimeSim Continuum syntax for 3DIC netlists as described in Multi-Technology Simulation of 3D Integrated Circuit in the *PrimeSim™ Continuum Overview*, with the exception of the `.ivth` feature.

This appendix covers the following:

- [3DIC Netlist Construct and Usage](#)
- [Module-Based PrimeSim Options](#)
- [Module-Based Back-Annotation](#)
- [Applying a 3DIC Module for a Regular Netlist](#)

3DIC Netlist Construct and Usage

The 3DIC netlist construct and usage in the PrimeSim tool follow the same rules as described in 3DIC Netlist Construct and Usage in the *PrimeSim™ Continuum Overview*.

The current 3DIC solution applies to transient analysis only. It supports sweeps for temperature and parameters, as well as `.ALTER` features as described in Transient Analysis and Alters Simulation Features in the *PrimeSim™ Continuum Overview*.

Full Circuit Example

A full circuit example of a 3DIC is described in Full Circuit Example in the *PrimeSim™ Continuum Overview*.

Module-Based PrimeSim Options

In a regular netlist, the PrimeSim tool supports many local options at the subcircuit level. With the introduction of modules in a 3DIC netlist, the PrimeSim tool extends the support of selected local options to the module level.

Table 40 lists the PrimeSim options with 3DIC module support.

Table 40 PrimeSim Options With 3DIC Module Support

Option	Description
<code>primesim_loadmodel</code>	Sets the capacitive load model that a partition output drives.
<code>primesim_mode</code>	Sets the simulation mode.
<code>primesim_model</code>	Sets the modeling method of the MOSFETs.
<code>primesim_partition</code>	Specifies how partitioning is done (FastSPICE only).
<code>primesim_speed</code>	Sets the speed level of the simulation.

These options can be set for a subcircuit within a module using the module label as follows:

```
.option primesim_option="module_name::subckt_name:option_value"
```

For example:

top_3d.sp file:

```
*** Top level netlist with inverter chains inside two 3DIC modules
.option primesim_model="mod1::chain:3"
.option primesim_model="mod2::chain:5"
.global vdd gnd
.module mod1
.inc './defcir.sp'
.endmodule
.module mod2
.inc './defcir.sp'
.endmodule
v1 vdd gnd 1.8
vg n0 gnd pulse 0 1.8 5n 1n 1n 20n 40n
xc1 n0 n1 mod1::chain
```

```
xc2 n0 n2 mod2::chain
.tran 1n 100n
.print v(*)
.end

defcir.sp file:

*** Circuit definition of inverter chain
.subckt inv in out
mp out in vdd vdd pch l=1u w=3u
mn out in gnd gnd nch l=1u w=2u
.ends
.subckt chain in out
x1 in a inv
x2 a b inv
x3 b out inv
.ends
```

In this example, the top-level netlist has two inverter chains inside two 3DIC modules. The commands in the configuration file set the model level to 3 for the first module, and to 5 for the second module, so the inverter chain in the second module is simulated with better accuracy.

Module-Based Back-Annotation

Table 41 lists the PrimeSim DPF and SPF back-annotation options with 3DIC module support.

Table 41 DPF and SPF Back-Annotation Options With 3DIC Module Support

Option	Description
primesim_dpftime	Specifies the module for DPF back-annotation.
primesim_spftime	Specifies the module for SPF back-annotation.

SPF Back-Annotation Example

Assumed Netlist Configuration:

```
*** 3DIC netlist ***
...
.module mod1
.subckt sub1 {ports_list}
...
.ends sub1
.subckt sub2 {ports_list}
...
.ends sub2
```



```
.endmodule

.subckt top {ports_list}
  X2 {ports_list} mod1::sub1
.ends top

X1 {ports_list} mod1::sub1
X2 {ports_list} top
**<simulation conditions>
.end

** DSPF (filename : case1.spf)
.SUBCKT SUB1
<Instance Section>
<RC extracted net Section>
<Device Section>
.ENDS
```

As shown previously, `.SUBCKT` is defined in the SPF file.

```
.option primesim_spf = case1.spf
.option primesim_spfmodule = mod1
```

In this example, the PrimeSim tool annotates `case1.spf` onto subcircuit `sub1` inside module `mod1` by searching `.SUBCKT SUB1` from the `case1.spf` file because the subcircuit name defined inside module is `SUB1`.

The log file reports: `case1.spf` back-annotated onto `subckt sub1` inside module `mod1`.

DPF Back-Annotation

For DPF back-annotation, the PrimeSim tool's behavior is the same as in the SPF back-annotation case described previously, but the `primesim_spf` and `primesim_spfmodule` commands are replaced with the `primesim_dpf` and `primesim_dpfmodule` commands, respectively.

Applying a 3DIC Module for a Regular Netlist

The PrimeSim tool supports using the `primesim_module_instance` option to apply a 3DIC module for a regular netlist without modifying the input netlist. Using this option, you can specify the instance name to apply to a 3DIC module and subcircuit without any netlist modification.

Note:

This option is recommended only for simple 3DIC application structures when running the PrimeSim tool standalone. Applying a donut style structure is not supported.

Part 7: Output

Describes PrimeSim tool output and utilities.

This Part presents the following chapters:

- [Chapter 19, Output Details](#)
- [Chapter 20, Probing and Measuring](#)
- [Chapter 21, PrimeSim Utilities](#)

19

Output Details

This chapter describes the output files and exit codes defined in the PrimeSim tool.

This chapter covers the following topics:

- [Output Naming Conventions](#)
- [Output Files](#)
- [Log Files](#)
- [Exit Codes](#)

Output Naming Conventions

The PrimeSim Pro and PrimeSim SPICE tools support the following output naming conventions.

Output Naming Convention 1

`output_format/ana_name_aalter_id_ttemp_id_mmc_id_ssw_id_bsbs_id.ana_keyword@[ra|t]=time`

where:

- The files are output to the `output_format` directory, `output` is the value set with the `-o` command-line argument, and `format` is the output format, such as `wdf` for WDF format. Currently it is used for PSF and WDF waveform outputs.
- If the analysis statement includes the `name` argument, the `ana_name` is the value of `name`. Otherwise, it begins with the analysis keyword, plus an index, starting with 0, such as `tran0` where `tran` is the `ana_keyword`.
- If there is an `.ALTER` statement, add `aalter_id`, where the first index is 0.
- If there is a `.TEMP` statement, add `ttemp_id`, where the first index is 0.
- If there is Monte Carlo, add `mmc_id`, where the start index is the first run, such as `m6`.
- If there is sweep, add `ssw_id`, where the start index is 0, such as `s0`.

- For multi-loop sweep, such as two loop sweeps, the convention is `ssw_id1_ssw_id2`, such as `s0_s0`, where the first is the outer loop, and the second is inner loop.
- If there is bisection optimization, add `bsbs_id`, where the first index is 0.
- If there is a degradation file, add `@ra=time` at the end.
- If there is analysis based on transient analysis, such as `.OP` time, add `@t=time` at the end.

Example

If the output is PrimeSim, the format is WDF, and the netlist includes Monte Carlo and `.ALTER` statements, then an example of a name for a transient waveform file is `prime_wdf/tran0_al_m6.tran`.

Output Naming Convention 2

`output_ana_name_aalter_id_ttemp_id_mmc_id_ssw_id_bsbs_id.postfix`

where:

- The files are output to the current working directory, and where `output` is the value set with the `-o` command-line argument.
- If the analysis statement includes the `name` argument, the `ana_name` is the value of `name`. Otherwise, it begins with the analysis keyword, plus an index, starting with 0, such as `tran0` where `tran` is the `ana_keyword`.
- If there is an `.ALTER` statement, add `aalter_id`, where the first index is 0.
- If there is a `.TEMP` statement, add `ttemp_id`, where the first index is 0.
- If there is Monte Carlo, add `mmc_id`, where the start index is the first run, such as `m6`.
- If there is sweep, add `ssw_id`, where the start index is 0, such as `s0`.
- For multi-loop sweep, such as two loop sweeps, the convention is `ssw_id1_ssw_id2`, such as `s0_s0`, where the first is the outer loop, and the second is inner loop.
- If there is bisection optimization, add `bsbs_id`, where the first index is 0.
- For `postfix`, for a text report, it can be `mt0`, `pt0`, and so on; while for a waveform, it can be `tr0`, `sw0`, and so on.

Example

If the output is PrimeSim and the netlist includes Monte Carlo and `.ALTER` statements, then an example of a name of an `.mc0` file for transient analysis is `prime_tran0_al.mc0`.

PrimeSim SPICE Default Output Format

If you do not define the output format, the output format is determined by the analysis being performed. [Table 42](#) lists the default output format for each analysis.

Table 42 *Default Output Formats*

Analysis Type	Output Format
TRAN	FSDB
DC/OP	WDF
Small Signal Analysis (AC/NOISE/LSTB and so on)	WDF
RF Analysis	WDF

Output Files

The PrimeSim tool supports the following output formats:

Table 43 *PrimeSim Output Formats*

File Extension	Description
.ac0	AC analysis output file.
.fast	Transient analysis output file (Veritool™ format).
.fsdb	Transient analysis output file (Synopsys format).
.ic	Initial node voltage file.
.ins	Instance table file.
.log	Simulation log file.
.ma0	AC analysis measurement.
.md0	DC analysis measurement.
.mt0	Transient analysis measurement output file.
.op0	Node/device operating point file.
.pa0	AC analysis print output file.
.pd0	DC analysis print output file.

Table 43 PrimeSim Output Formats (Continued)

File Extension	Description
.pt0	Transient analysis print output file.
.sw0	DC analysis output file.
.tr0	Transient analysis output file (PrimeSim HSPICE format). The PrimeSim tool supports the SPICE 9601, 2001, and 2013 output formats during the generation of a .tr0 output file. For more details, see .OPTION POST_VERSION.

Transient Analysis

Results are written to `fsdb`, `tr0`, `fast`, or `psf` format depending on the setting in the `primesim_output` option. `fsdb` is the default, but you can specify `primesim_output` with other formats such as `primesim_output=tr0`. These files contain a list of transient analysis numerical results and are the result of an input file `.TRAN` statement together with an `.OPTION POST[=1 or 2]` statement that creates a post-analysis file, where `POST=1` specifies a binary format (default) and `POST=2` creates a text format when format is `tr0`.

For data driven transient sweep analysis, output files are written with the following terminology (output file name followed by sweep number):

- `output_name_s#.fsdb` for FSDB output.
- `output_name.mt0` for measure output files.

`.PRINT` statements in the input file saves the results of transient analysis to `output_name.pt0`, which is a tabular text format.

Note:

Transient analysis measurement results are written to `xxx.mt0`. This output file is the result of an input file `.MEASURE TRAN` statement.

DC Analysis

Results appear in the file `output_name.sw0`, which is produced as a result of a `.DC` statement. This file contains the results of the applied stepped or swept DC parameters defined in that statement. If `.PRINT` statements are used in the input file, the DC analysis results are written to the file `output_name.pd0`.

DC analysis measurement results are stored in the file `output_name.md0` when a `.MEASURE DC` statement is used in the input file.

A `output_name.fsdb` output file is a binary format supported by the WaveView tool. The FSDB is a more compact format than text output formats. The `output_name.tr0` and `output_name.sw0` format is PrimeSim HSPICE compatible text or binary output format.

Output Control Statements

To generate an output file in the PrimeSim Pro tool, include the following control statements in the input files:

- `.PRINT` prints numeric analysis results in the output listing file in a tabular text format.
- `.PROBE` prints the output of the specified variables to post-processor output files.
- `.option post` outputs all nodal voltage waveforms, and all current waveforms of independent voltage sources.
- `.MEASURE` prints the results of user-specified analysis to the output file.

Log Files

By default, the PrimeSim tool automatically outputs to a `filename.log` log file. For example, if you want to perform [Transient Analysis](#), the PrimeSim Pro tool outputs to the default log.

Use the `primesim_skipwarn` option when you want to limit warnings reported to the log file.

Exit Codes

The following is a list of exit codes issued by the PrimeSim tool.

Table 44 *PrimeSim Exit Codes*

Code	Description
0	Simulation succeeded.
1	Memory related error.
2	File related error.
3	Not supported yet.
4	Input argument error.
5	General parsing error.

Table 44 *PrimeSim Exit Codes (Continued)*

Code	Description
6	I/O file parsing error.
7	SPF file parsing error.
8	General elaboration error.
9	Unable to resolve expression.
10	Option related error.
11	Netlist connection error.
12	Matrix related error.
13	Unable to converge.
14	Output related error.
15	License related error.
16	Unable to reinvoke by <code>execvp()</code> .
17	Parallel matrix error.
18	ADDMI related error.
19	S-element module error.
20	B-element module error.
21	Monte Carlo module error.
22	Verilog-A error.
23	TMI2 error.
24	Z-Transform module error.
25	ETMI SOA error.
28	PrimeSim API multiple analysis statements error.
29	PDMI related error.
30	Voltage Loop error.
31	Obsolete Option error for the PrimeSim API.
32	Bisection error.

Table 44 PrimeSim Exit Codes (Continued)

Code	Description
33	TNA error.
34	DC not converged - only when <code>primesim_exit_dc_fail</code> is set.

20

Probing and Measuring

This chapter describes the many advance probing and measurement features supported by the PrimeSim tool to generate the output that the user desires.

This chapter covers the following topics:

- [Algebraic Expressions](#)
- [External Table Parameter File Interface](#)
- [String Compare Function \(strcmp\)](#)
- [.PROBE](#)
- [.MEASURE](#)
- [.LPROBE](#)

Algebraic Expressions

This section describes the algebraic expressions used in the SPICE netlist. Any parameter defined in the netlist may be replaced by an algebraic expression. The expressions can be in a .PARAM / PARAMETER / PARAMETERS statement, .MEASURE / MEAS statement, .PRINT / .PROBE statements, and device parameters of element statements. See the *PrimeSim™ Continuum Reference Manual: Commands and Control Options* for more information on these and other PrimeSim HSPICE-compatible commands.

An algebraic expression is enclosed within quotation marks, and consists of simple arithmetic operations (+, -, *, /), conditional operators (==, !=, <, <=, >, >=), logic operators (&&, ||), bitwise operators (&, |), ternary operators (a?b:c) and built-in functions. You can change the precedence of calculation by using a pair of parentheses as in normal expressions. In .PROBE (.PRINT) statements, the expressions must be defined inside a PAR () statement.

Examples

```
.PARAM x='y+3'
.PARAM y='3*(b<1.0?1.0:2.0)'
R1 1 0 r='ABS(v(1)/i(m1))+10'
```

```
Rtest n1 n2 R='2.345*1.4' scale=1e6
Cval 13 4 C='22p+11*4e-12'
Lcouple 2 3 L='10uH*3' M=3
K12 K1 K2 K=0.9

.MEASURE vmax MAX V(1)
.MEASURE imax MAX I(q2)
.MEASURE ivmax param='vmax*imax'

.PROBE x=PAR('i(m1)/v(22)')
```

The PrimeSim tool also supports function parameter definition. For example:

```
.PARAM myfunc(a,b)='3*a + b'
.PARAM x=... y=...
.PARAM k='2*myfunc(x,y)'
R10 2 0 r='1+myfunc(10,100)'
```

Built-In Functions

Table 45 Built-in Function Descriptions

Function Name (args)	Description (Return value)
abs(x)	Absolute value of x: $ x $
acos(x)	Inverse cosine of x (in radians)
asin(x)	Inverse sine of x (in radians)
atan(x)	Inverse tangent of x (in radians)
avg(x)	Average value of x sequence
ceil(x)	Smallest integer $\geq x$
cos(x)	Cosine of x (in radians)
cosh(x)	Hyperbolic cosine of x (in radians)
db(x)	Base 10 logarithm of the absolute value of x, multiplied by 20, with the sign of x: $(\text{sign of } x)20\log_{10}(x)$
exp(x)	e raised to the power x: e^x
floor(x)	Largest integer of $\leq x$
fmod(x,y)	Floating-point remainder of x/y
int(x)	Largest integer less than or equal to x

Table 45 *Built-in Function Descriptions (Continued)*

Function Name (args)	Description (Return value)
<code>log(x)</code>	Natural logarithm of the absolute value of x, with the sign of x: $(\text{sign of } x) \log(x)$
<code>log10(x)</code>	Base 10 logarithm of the absolute value of x, with the sign of x: $(\text{sign of } x) \log_{10}(x)$
<code>max(x, y, ... n)</code> <code>mmax(x, y, ... n)</code>	Numeric maximum of x, y, ... n
<code>min(x, y, ... n)</code> <code>mmin(x, y, ... n)</code>	Numeric minimum of x, y, ... n
<code>nint(x)</code>	Round to nearest integer of x
<code>pow(x, y)</code>	Value of x raised to power of the integer part of y: $x(\text{integer part of } y)$
<code>pwr(x, y)</code>	Absolute value of x raised to the y power, with the sign of x: $(\text{sign of } x) x ^y$
<code>round(x)</code>	Round to nearest integer of x
<code>sgn(x)</code>	-1 if x is less than 0, 0 if x is equal to 0, and 1 if x is greater than 0
<code>sign(x, y)</code>	Absolute value of x, with the sign of y: $(\text{sign of } y) x $
<code>sin(x)</code>	Sine of x (in radians)
<code>sinh(x)</code>	Hyperbolic sine of x (in radians)
<code>sqrt(x)</code>	Square root of the absolute value of x: $\text{sqrt}(-x) = -\text{sqrt}(x)$
<code>table_param(str('table_file_name'), #disc, nd1, nd2, ..., #cont, nc1, nc2, ..., value_col)</code>	Linear interpolated value from table parameter file; see External Table Parameter File Interface for more information.
<code>tan(x)</code>	Tangent of x (in radians)
<code>tanh(x)</code>	Hyperbolic tangent of x (in radians)
<code>trunc(x)</code>	Integer part of x

External Table Parameter File Interface

Description

You can use the `table_param()` built-in function to specify a file that contains a parameter values table and the PrimeSim tool returns the linear interpolated value from that table.

Syntax

```
table_param(str('table_file_name'), #disc, nd1, nd2, ..., #cont, nc1, nc2, ..., value_col)
```

Arguments

Argument	Description
<code>str('table_file_name')</code>	Name of file containing a parameter values table. Note: Specify the file name using the <code>str()</code> function.
<code>#disc</code>	Total number of discrete index columns.
<code>nd1, nd2, ...</code>	Discrete index column numbers.
<code>#cont</code>	Total number of continuous value columns.
<code>nc1, nc2, ...</code>	Continuous values.
<code>value_col</code>	Continuous column number used in linear interpolation.

Examples

The following examples use a one-dimensional table data file, `exam1.table`, defined as follows:

```
# type      width      res      cap
1          10         0.5      12e-15
1          20         0.9      20e-15

.param my_res_val=table_param(str('exam1.table'), 1, 1, 1, 15, 1)
```

In this example, the `table_param()` function returns 0.7 and assigns it to `my_res_val`.

```
.param my_cap_val=table_param(str('exam1.table'), 1, 1, 1, 15, 2)
```

In this example, the `table_param()` function returns 16e-15 and assigns it to `my_cap_val`.

The following examples use a two-dimensional table data file, `exam2.table`, defined as follows:

```
#
# type1 type2  a   b   c   d
#
#   1     1   10  10  0.5  50
#   1     1   10  20  0.9  90
#   1     1   20  10  0.4  40
#   1     1   20  20  0.8  80
#
# type1 type2  a   b   c   d
#
#   1     2   10  10  0.7  40
#   1     2   10  20  1.2  90
#   1     2   20  10  0.4  30
#   1     2   20  20  1.0  70

.param vddval1=table_param(str("exam2.table"), 2, 1, 1, 2, 15, 15, 1)
.param vddval2=table_param(str("exam2.table"), 2, 1, 1, 2, 15, 15, 2)
.param mytbl1=table_param(str("exam2.table"), 2, 1, 2, 2, 15, 15, 1)
.param mytbl2=table_param(str("exam2.table"), 2, 1, 2, 2, 15, 15, 2)
```

In these examples, the resolved variables are `vddval1=0.65`, `vddval2=65`, `mytbl1=0.825`, and `mytbl2=57.5`, respectively.

String Compare Function (strcmp)

The PrimeSim tool supports the `strcmp` function, which behaves like the C `strcmp()` function, so it returns 0 if the compared strings are equal. This can be used either with strings passed directly to the function or with string parameters. The arguments to `strcmp` must be strings and cannot be swept parameters.

For example:

```
.param mode=str('@model')
.param snps=str('snps')
.if (strcmp(mode, snps) == 0)
...
.else
...
.endif

or just

.if (strcmp(str('@model'), str('snps')) == 0)
...
.else
...
.endif
```

The `strcmp` function does not have to be used in `.if` statements.

For example:

```
.param p='strcmp(model, snps) ? 1 : 0'
```

.PROBE

The `.PROBE` statement saves waveforms of output variables into data files. If you use an `.OPTION POST` statement, the PrimeSim tool saves all of the voltages and supply currents without the `.PROBE` statement. If you are only interested in the output data file, use the `.OPTION POST PROBE` statement in conjunction with the `.PROBE` statement. Even when you only use a `.PROBE` statement, the PrimeSim tool saves output variables only. See [Output Files](#) for detailed information about output file name conventions.

When you use this statement, the PrimeSim tool can create additional nodes when DSPF is back-annotated onto a design. For more details, see [DSPF Annotation](#).

Syntax

```
.PROBE antype [name1=]V(NodeName) [name2=]I(DevName)
+ [vprbtol=value] [iprbtol=value] ... [subckt=name] [adonly=0|1]
+ [level=value] [filter=pattern]
.PROBE antype [name3=]Cap(DevName) ... [subckt=name] [level=value]
.PROBE antype [name4=]V(Node1,Node2) [vprbtol=value]
+ [iprbtol=value]... [subckt=name] [level=value]
.PROBE antype [name5=]PAR('algebraic expression') [vprbtol=value]
+ [iprbtol=value] ... [subckt=name] [level=value]
.PROBE antype [name6=]IALL(DevName) [name7=]VALL(DevName) [level=value]
```

Table 46 .PROBE Parameters

Parameters	Description
<i>antype</i>	Type of analysis (DC, TRAN, AC, OP) for the specified plots.
<i>DevName</i>	Device (element) name, which can contain wildcards * and/or ?. <code>I(DevName)</code> means the branch current through <i>DevName</i> .

Table 46 .PROBE Parameters (Continued)

Parameters	Description
<code>filter=pattern</code>	When printing node voltage(s) and/or element current(s) that are specified by wildcard patterns such as: <code>.probe v(x1.x2.*)</code>, nodes/elements that match the pattern specified in the filter clause are not probed. Note: Each filter applies to all wildcard voltages/currents being probed on the <code>.probe</code> statement. Consider for example: <code>.probe tran v(a) v(c) v(xsub5.*) v(xsub9.*) filter='*2*'</code> In the preceding example, the filter is applied to both the wildcard portions of the <code>.PROBE</code>.
<code>name1, name2, ...</code>	Optional user-defined output name.
<code>Node1, Node2</code>	Node names that don't allow wildcard characters. <code>V(Node1, Node2)</code> means the difference of <code>V(Node1) - V(Node2)</code>.
<code>NodeName</code>	Node name, which can contain wildcard characters <code>*</code> and/or <code>?</code>. <code>V(NodeName)</code> means the node voltage of <code>NodeName</code>. <code>Cap(NodeName)</code> means the node capacitance of <code>NodeName</code>. Note: <code>.PROBE Cap(NodeName)</code> is not supported in AC analysis.
<code>vprbtol=value</code>	Voltage tolerance for the specified node (see primesim_vprbtol to set voltage tolerance globally).
<code>iprbtol=value</code>	Current tolerance for the specified node (see primesim_iprbtol to set current tolerance globally).
<code>subckt=name</code>	Name of the subcircuit.
<code>adonly=0 1</code>	0: (default) Specifies that nodes connected with active or passive devices are probed. 1: Specifies that only those nodes connected with active devices are probed. Note: This argument is applied only when you define nodes using wildcards. If you define a node explicitly, this argument is ignored.

Table 46 .PROBE Parameters (Continued)

Parameters	Description
level=value	The .PROBE statement handles level identification for the use of wildcards across hierarchy. The level option is applied when there is wildcard character in an output variable. The following values are supported: • -1: (default) Find matched variables at all levels of hierarchy. • 0: Ignore wildcards in .ic or .probe statements. • n: Find matched variables at the current level of hierarchy and n-1 levels below. For example, a value of 1 would find matched variables only at the current level of hierarchy, a value of 2 would find matched variables at the current level of hierarchy and one level below, a value of 3 would find matched variables at the current level of hierarchy and two levels below, and so on.

The PrimeSim tool can also probe nodes inside a subckt without having .PROBE inside the .subckt block.

Example

```
{simulation deck}
:
.probe v(*) subckt=block1
:
.subckt a b c d e block1
:
.ends
:
```

The previous example is equivalent to the following deck:

```
{simulation deck}
:
.subckt a b c d e block1
.probe v(*)
:
.ends
:
```

When adonly=1 is added, only the nodes that are connected to at least one active device are probed, as in the following example:

```
.probe v(*) subckt=block1 adonly=1
```

Examples

```
.PROBE DC V(1) V(2) V(3) beta=PAR('I1(Q1)/I2(Q1)')
```

In this example, the PrimeSim tool probes the DC voltages at nodes 1, 2, and 3 as well as currents through device Q1. The variable beta is calculated by I1(Q1)/I2(Q1).

```
.PROBE TRAN V(3,1) V(4) V(7) minus_ten=V(0,10)
```

In this example, the PrimeSim tool probes voltage differences between nodes 3 and 1, and between nodes 0 and 10, and calls the latter as variable `minus_ten`. It also probes voltages at nodes 4 and 7.

```
.PROBE V(*)
```

In this example, the PrimeSim tool probes voltages at all nodes in the entire design.

```
.PROBE V(*.*)
```

In this example, the PrimeSim tool probes voltages at nodes from the second level of hierarchy and below.

```
.PROBE V(*.*.*)
```

In this example, the PrimeSim tool probes voltages at nodes from the third level of hierarchy and below.

```
.PROBE V(*) LEVEL=1
```

In this example, the PrimeSim tool probes voltages at the top level nodes only, which is also where wildcard character first met.

```
.PROBE V(*) LEVEL=2
```

In this example, the PrimeSim tool probes voltages at nodes for the top level where is wildcard character first met and one level below.

```
.PROBE V(*) LEVEL=3
```

In this example, the PrimeSim tool probes voltages at nodes for the top level, where is wildcard character first met, and two levels down to make three levels of hierarchy.

```
.PROBE V(X101.*) LEVEL=3
```

In this example, the PrimeSim tool probes from instance `X101.*` as the first level and two levels of hierarchy down for a total of three levels of hierarchy.

```
.PROBE V(X*.*) LEVEL=3
```

In this example, the PrimeSim tool probes from instance `X*` as the first level and two levels of hierarchy down for a total of three levels of hierarchy.

```
.PROBE V(X*.*) LEVEL=1
```

In this example, the PrimeSim tool does not probe anything.

```
.PROBE DC V(*) I(*.*)
```

In this example, the PrimeSim tool probes DC voltages for all nodes in the entire design and also currents on all devices from the second level of hierarchy down.

```
.PROBE DC V(A*) I(x1.*)
```

In this example, the PrimeSim tool probes voltage on nodes started with letter **A** and currents in instance **x1** for all the hierarchy levels in this instance.

```
.PROBE TRAN V(?????)
```

In this example, the PrimeSim tool probes all the voltage on nodes whose name is five characters long and in the top level.

```
.PROBE V(*EN*)
```

In this example, the PrimeSim tool probes all nodes in the entire design whose name includes the string, **EN**.

```
.PROBE TRAN V(*b?)
```

In this example, the PrimeSim tool probes all nodes in the entire design whose name ends with letter **b** followed by one more character, for example, **b0**, **b1**, and **enba**.

```
.PROBE IALL(M1)
```

In this example, the PrimeSim tool probes all port current of device **M1** and is the same as

```
.PROBE I1(M1) I2(M1) I3(M1) I4(M1).
```

```
.PROBE VALL(M1)
```

In this example, the PrimeSim tool probes all port voltage of device **M1** and is the same as **.PROBE V1(M1) V2(M1) V3(M1) V4(M1)**, which is representing drain/gate/source/bulk node voltage.

```
.PROBE OP v(1) v(2)
```

In this example, the PrimeSim tool probes only the operating point information of nodes **1** and **2**.

```
.PROBE OP I(VIN) I(X1.VSRC)
```

In this example, the PrimeSim tool probes only the operating point information of devices **VIN** and **X1.VSRC**.

Probing is described further in the following sections:

- [Probing Block Current](#)
- [Probing and Exceptions to Probing](#)
- [Logic Probes \(Digital Waveforms\)](#)
- [Probing With Regular Expressions](#)
- [Exceptions to Probing](#)

- [Support for Power Reporting](#)
- [Support for Measuring Power Dissipation: pd\(\)](#)
- [Probing Element Parameters](#)

Probing Block Current

You can measure the current into a block through a port or global node by using `x()` with this statement, as shown in the following example:

```
x(instance.port) or x(instance.global)
```

where:

- *instance* is the name of an instance in the design, and can include wildcards,
- *port* is the name of a port in the instance, and can include wildcards, and
- *global* is the name of a global node used by the instance, and can include wildcards.

Example

```
.probe tran x(x1.x2.a)  
.probe tran x(x1.vdd*)
```

In the first example, the PrimeSim tool probes the current into port `x1.x2.a`; in the second example, the PrimeSim tool probes the current into `x1` through any global nodes matching `vdd*`.

The PrimeSim tool supports syntax of `ISUB(X****,****)` to support current probing in spice netlist format. Internally, it is mapped into x-probes.

Note:

Current is considered positive when it is flowing into a block and negative when it is flowing out, so vdd current is usually negative while vss current is usually positive.

When you want to define the source or sink of a current explicitly, you can use the `primesim_chk_devport` option. For more details, see [PrimeSim Options](#).

Probing and Exceptions to Probing

The PrimeSim tool supports regular expressions for probes and lprobes as well as exceptions to probes and lprobes. Exceptions can be given to any probe statement. Independent of whether you use glob-patterns or regular expressions for the node name, the exception is always a glob-pattern. You can further refine your probe statement with more constraints such as the `level=` statement.

The PrimeSim tool implements the Posix ERE (extended regular expressions).

Logic Probes (Digital Waveforms)

In contrast to the normal analog waveform probing in which PrimeSim records every change in the waveform that exceeds `primesim_vprbtol`, digital waveforms or logic probes (`.LPROBE`) can be either logically zero (0), logically one (1) or undefined (x). Using logic probes can reduce your FSDB file size significantly.

For logic probes, you must define the threshold levels for logic-low and logic-high detection for the signals you probe, as in the following example:

```
.lprobe level=8 v(*) high=0.7 low=0.4 subckt=pmma02ndx20c
```

The preceding example probes all signals in the top eight levels of hierarchy of subcircuit `pmma02ndx20c`. All voltage levels up to 0.4V are displayed as logically zero, all values above 0.7V are displayed as logically one, and the values in between are undefined.

Probing With Regular Expressions

The PrimeSim tool supports regular expressions for probes and lprobes, as well as the Posix ERE (extended regular expressions). `regex=option` can be used to match nodes you want to probe.

The following table lists and describes the special characters that the PrimeSim tool supports.

Table 47 *Supported Characters for Regular Expressions*

Character	Description
^	Indicates the beginning of the string.
\$	Indicates the end of the string.
.	Matches anything, including spaces.
*	Matches the prepended item zero times or more often. For example, <code>x\d*\$</code> matches <code>xxx</code> as well as <code>x122321</code> and <code>x3</code> .
+	Matches the prepended item once or more often. For example, <code>x\d+\$</code> does not match <code>xxx</code> , but it matches <code>x122321</code> and <code>x3</code> .
?	Matches the prepended item zero times or only once. For example, <code>x\d?\$</code> matches <code>xxx</code> and <code>x3</code> , but does not match <code>x122321</code> .
()	Used to group certain characters. For example, <code>(abac)+</code> matches <code>abac</code> or <code>abacabac</code> , but not <code>abab</code> .

Table 47 Supported Characters for Regular Expressions (Continued)

Character	Description
[]	Used to define a set of characters, out of which either one can match. For example, <code>h[aeo]llo</code> matches <code>hallo</code> or <code>hello</code> or <code>hollo</code> , but not <code>hillo</code> .
\	Escape any of the above special characters.

Examples

```
.probe regexp=v(^[a-z]+$)
```

The previous example probes all nodes that consist of characters only, no numbers or any special characters.

```
.probe regexp=v(^[a-z0-9]+$)
```

The previous example probes all nodes that consist of characters and digits only, no special characters such as underscore.

```
.probe regexp=v(net[0-4][0-9]*)
```

The previous example probes all nets that begin with either 0, 1, 2, 3, or 4. For example: `net1`, `net49`, or `net33423`, but not `net7` or `net99`.

```
.probe regexp=isub(^[dgsbcepn]$)
```

The previous example probes subcircuit port currents having single character names, at all levels of hierarchy.

```
.probe regexp=isub(^([dgsbcepn]|bulk|well|sub)$)
```

The previous example probes subcircuit port currents having single character names, as well as `bulk`, `well`, and `sub`, at all levels of hierarchy.

Exceptions to Probing

When using patterns to match multiple signals such as an asterisk (*) or regular expressions, it often occurs that this pattern matches thousands of signals. Quite often you do not intend to probe them all, just for a few of them. That is especially true in post-layout simulations, where each node has several (sometimes as many as hundreds) derivatives.

To exclude certain nets from your probe statement, you can use the `except=` option. These exceptions are always interpreted as glob-patterns, not as regular expressions, even though your actual probe statement might be.

Exceptions can be given to any probe statement. Independent of whether you use glob-patterns or regular expressions for the node name, the exception is always a glob-pattern.

You can further refine your probe statement with more constraints such as the `level=` statement.

Examples

```
.probe v(*) except="v*" level=2  
.probe v(*) except="*$*" level=1  
.probe regexp=v([^\o][a-z0-5]+$) except="s15" level=1
```

The first example probes all nets on the first two levels of hierarchy, except those that start with the letter `v`.

The second example probes all nets in all hierarchies except those that contain the dollar sign (`$`).

The third example probes all nets that do not begin with `o` and after that only contain `0`, `1`, `2`, `3`, `4`, and `5`, with the exception of node `s15`.

Support for Power Reporting

Power reporting for each device, instance, and port is supported.

Syntax

```
p(device_full_name)  
pinst(instance_name)  
pport(port_name)
```

Example

```
probe p(x1.x2.m1) p(vvdd)
```

where `p(device_full_name)` is the sum of `terminal_voltage*terminal_current` for all terminals of the device where `terminal_current` is the incoming direction.

Note that:

- `pd()` reports only dissipated power, but `p()` basically reports the sum of dissipated power and stored power.
- PrimeSim HSPICE has the same syntax but its definition is different for the devices having both resistances and capacitances inside like MOSFET. In that case, PrimeSim's `pd()` corresponds to it.

Support for Measuring Power Dissipation: `pd()`

The PrimeSim tool supports `pd()` for measuring the power dissipation of an individual device or total circuit. For capacitors/inductors/voltage sources/current sources, there is no power dissipation and the `pd()` value is `0`.

Examples

```
.probe pd(x1.m1) pd(x1.r1) pd(*)  
.probe pd(%top%)  
.meas total_power INTEG pd(%top%)
```

In the first example, it probes the power dissipation for an individual device. In the second example, it probes the total power dissipation. In the third example, the total power dissipation can be used with `.measure`. For better compatibility with the PrimeSim HSPICE tool, the keyword `POWER` has been aliased with `pd(%top%)`.

Probing Element Parameters

The PrimeSim tool supports probing of the following element parameters, including resistor ([Table 48](#)), diode ([Table 49](#)), BJT ([Table 50](#)), MOSFET ([Table 51](#)), and BSIM-BULK (formerly BSIM6) output templates ([Table 52](#)).

Table 48 Supported Resistor Parameters for Probing

Parameters	Description
LV1	Conductance at analysis temperature.
LV2	Resistance at analysis temperature.

Table 49 Supported Diode Parameters for Probing

Parameters	Description
LV1	Diode area factor.
LX0	Voltage across diode (VD), excluding RS (series resistance).
LX1	DC current through diode (ID), excluding RS. Total diode current is the sum of IDC and ICAP.
LX2	Equivalent conductance (GD).
LX4	Current through the diode capacitor. Total diode current is the sum of IDC and ICAP.
LX5	Total diode capacitance.
LV23	Area after scaling

Table 50 *Supported BJT Parameters for Probing*

Parameters	Description
LX19	cbe capacitance (C_{π}).
LX20	cbc internal base-collector capacitance (C_{μ}).
LX21	csc substrate-collector capacitance for vertical transistors. csb substrate-base capacitance for lateral transistors.

Table 51 *Supported MOS Parameters for Probing*

Parameter	Description
LV1	Channel length (L)
LV2	Channel width (W)
LV3	Area of the drain diode (AD)
LV4	Area of the source diode (AS)
LV9	Threshold voltage
LV10	Saturation voltage (VDSAT)
LV11	Drain diode periphery (PD)
LV12	Source diode periphery (PS)
LV13	Drain resistance (squares) (RDS)
LV14	Source resistance (squares) RSS
LV15	Charge sharing coefficient
LV16	Effective drain conductance ($1/R_{Deff}$)
LV17	Effective source conductance ($1/R_{Seff}$)
LV36	Gate-source overlap capacitance
LV37	Gate-drain overlap capacitance
LV38	Gate-bulk overlap capacitance
LX1	Bulk-source voltage (VBS)
LX2	Gate-source voltage (VGS)

Table 51 *Supported MOS Parameters for Probing (Continued)*

Parameter	Description
LX3	Drain-source voltage (VDS)
LX4	DC-drain current (CDO)
LX5	DC source-bulk diode current (IBS)
LX6	DC drain-bulk diode current (IBD)
LX7	DC-gate transconductance (GMO)
LX8	DC drain-source conductance (GDSO)
LX9	DC-substrate transconductance (GMBSO)
LX18	$CGGBO=dQG/dVGB=CGS+CGD+CGB$
LX19	$CGDBO=dQG/dVDB$
LX20	$CGSBO=dQG/dVSB$
LX21	$CBGBO=dQB/dVGB$
LX22	$CBDBO=dQB/dVDB$
LX23	$CBSBO=dQB/dVSB$
LX28	Bulk-source capacitance
LX29	Bulk-drain capacitance
LX32	$CDGBO=dQD/dVGB$
LX33	$CDDBO=dQD/dVDB$
LX34	$CDSBO=dQD/dVSB$
LX62	Effective channel width (Weff)
LX63	Effective channel length (Leff)
LX70	Gate induced drain leakage current
LX82	Total gate capacitance (including intrinsic), and all overlap and fringing components
LX83	Total gate-to-drain capacitance (including intrinsic), and overlap and fringing components

Table 51 *Supported MOS Parameters for Probing (Continued)*

Parameter	Description
LX84	Total gate-to-source capacitance (including intrinsic), and overlap and fringing components
LX85	Total drain capacitance (including intrinsic), overlap and fringing components, and junction capacitance
LX86	Total drain-to-source capacitance
LX87	Total drain-to-gate capacitance (including intrinsic), and overlap and fringing components
LX88	Total bulk-to-gate (floating body-to-gate) capacitance, including intrinsic and overlap components
LX89	Total bulk-to-drain (floating body-to-drain) capacitance, including intrinsic and junction capacitance
LX90	Total bulk-to-source (floating body-to-source) capacitance, including intrinsic and junction capacitance
LX110	Gate-induced source leakage current

Table 52 *Supported Output Templates for BSIM-BULK (formerly BSIM6) (Level 77)*

Parameter	Description
LV5	Initial condition for the drain-source voltage (VDS)
LV6	Initial condition for the gate-source voltage (VGS)
LV7	Initial condition for the bulk-source voltage (VBS)
LV9	Threshold voltage (bias dependent)
LV21	BETA effective
LX0	Bulk-drain voltage
LX1	Bulk-source voltage (VBS)
LX2	Gate-source voltage (VGS)
LX3	Drain-source voltage (VDS)
LX4	Channel current (IDS)

Table 52 *Supported Output Templates for BSIM-BULK (formerly BSIM6) (Level 77)*
(Continued)

Parameter	Description
LX5	DC source-bulk diode current (CBSO)
LX6	DC drain-bulk diode current (CBDO)
LX7	DC MOSFET gate transconductance (GMO)
LX8	DC drain-source conductance (GDSO)
LX9	DC substrate transconductance (GMBSO)
LX133	Gate to bulk voltage
LX134	Drain to gate voltage
LX142	Constant current based VTH
LX286	Output based on <code>.ivdmargin</code> or <code>.OPTION ivdmargin</code> simulation
LX501	Effective channel length
LX502	Effective channel width
LX503	Effective channel length for CV
LX504	Effective channel width for CV
LX505	Source area
LX506	Drain area
LX507	Source perimeter
LX508	Drain perimeter
LX520	Poly Gate doping, effective
LX521	Low field mobility, effective
LX523	Saturation velocity, effective
LX524	Phonon / surface roughness scattering, effective
LX525	Columbic scattering, effective
LX526	Coulombic scattering parameter, effective
LX529	Correction factor for velocity saturation, effective

Table 52 *Supported Output Templates for BSIM-BULK (formerly BSIM6) (Level 77)*
(Continued)

Parameter	Description
LX531	S/D doping concentration, effective
LX534	Thermal conductance
LX535	Thermal capacitance
LV10 / LX536	Saturation voltage
LX537	Effective mobility
LX538	Total source resistance
LX539	Total drain resistance
LX540	Gate electrode resistance
LX541	Impact ionization current
LX542	Gate to body current
LX544	Gate-induced drain leakage current
LX545	Gate-induced source leakage current
LX546	Total gate-to-source current ($I_{gs}+I_{gcs}$)
LX547	Total gate-to-drain current ($I_{gd}+I_{gcd}$)
LX548	Intrinsic drain charge
LX549	Intrinsic gate charge
LX550	Channel current
LX551	Total Gate charge (intrinsic+extrinsic)
LX552	Total Drain charge (intrinsic+extrinsic)
LX553	Total Source charge (intrinsic+extrinsic)
LX554	Total Bulk charge (intrinsic+extrinsic)
LX555	Tunneling current due to gate-to-source overlap (I_{gs}).
LX556	Tunneling current due to gate-to-drain overlap (I_{gd}).
LX559	Source Partition of IGC

Table 52 *Supported Output Templates for BSIM-BULK (formerly BSIM6) (Level 77)*
(Continued)

Parameter	Description
LX560	Drain Partition of IGC
LX564	Gate transconductance (dI_{ds}/dV_g)
LX565	Source conductance (dI_{ds}/dV_s)
LX566	Drain conductance (dI_{ds}/dV_d)
LX567	Temperature dependence of channel current (dI_{ds}/dT)
LX568	Total gate capacitance ($CGG = dQ_g/dV_g$)
LX569	Total gate-to-source capacitance ($CGS = -dQ_g/dV_s$)
LX570	Total gate-to-drain capacitance ($CGD = -dQ_g/dV_d$)
LX571	Total source-to-gate capacitance ($CSG = -dQ_s/dV_g$)
LX572	Total source-to-drain capacitance ($CSD = -dQ_s/dV_d$)
LX573	Total Source capacitance ($CSS = dQ_s/dV_s$)
LX574	Total drain-to-gate capacitance ($CDG = -dQ_d/dV_g$)
LX575	Total drain capacitance ($CDD = dQ_d/dV_d$)
LX576	Total drain-to-source capacitance ($CDS = -dQ_d/dV_s$)
LX577	Temperature dependence of gate charge (dQ_g/dT)
LX578	Temperature dependence of source charge (dQ_s/dT)
LX579	Temperature dependence of drain charge (dQ_d/dT)
LX580	Channel power ($I_{ds} \cdot V_{ds}$)
LX581	Bias dependent bulk-drain capacitance
LX582	Bias dependent bulk-source capacitance
LX583	Gate-source overlap and fringing capacitances
LX584	Gate-drain overlap and fringing capacitances
LX585	Gate-bulk overlap capacitances
LX586	Intrinsic gate capacitance

Table 52 *Supported Output Templates for BSIM-BULK (formerly BSIM6) (Level 77)*
(Continued)

Parameter	Description
LX587	Intrinsic gate-to-drain capacitance
LX588	Intrinsic gate-to-source capacitance
LX589	Bias dependent bulk-source capacitance (alias to CJSB)
LX590	Bias dependent bulk-drain capacitance (alias to CJDB)
LX591	Total gate capacitance (alias to CGG)
LX593	Bulk transconductance (dI_{ds}/dV_b)
LX594	Self-heating temperature
LX595	Total gate-to-bulk capacitance ($CGB = -dQ_g/dV_b$)
LX596	Total drain-to-bulk capacitance ($CDB = -dQ_d/dV_b$)
LX597	Total source-to-bulk capacitance ($CSB = -dQ_s/dV_b$)
LX598	Total bulk-to-gate capacitance ($CBG = -dQ_b/dV_g$)
LX599	Total bulk-top-drain capacitance ($CBD = -dQ_b/dV_d$)
LX600	Total bulk-to-source capacitance ($CBS = -dQ_b/dV_s$)
LX601	Total bulk capacitance ($CBB = dQ_b/dV_b$)
LX602	Intrinsic gate-to-bulk capacitance
LX603	Intrinsic source-to-gate capacitance
LX604	Intrinsic source-to-drain capacitance
LX605	Intrinsic source capacitance
LX606	Intrinsic source-to-bulk capacitance
LX607	Intrinsic drain-to-gate capacitance
LX608	Intrinsic drain capacitance
LX609	Intrinsic drain-to-source capacitance
LX610	Intrinsic drain-to-bulk capacitance
LX611	Intrinsic bulk-to-gate capacitance

Table 52 Supported Output Templates for BSIM-BULK (formerly BSIM6) (Level 77)
(Continued)

Parameter	Description
Lx612	Intrinsic bulk-to-drain capacitance
Lx613	Intrinsic bulk-to-source capacitance
Lx614	Intrinsic bulk capacitance
Lx615	Total gate-to-source capacitance (alias to CGS)
Lx616	Total gate-to-drain capacitance (alias to CGD)
Lx617	Zero voltage bias bulk-source capacitance
Lx618	Zero voltage bias bulk-drain capacitance
Lx619	Fringing capacitance

Examples

```
.probe LV9 (Xab.Xcd.M1)
.meas name1 avg lv9(Xab.Xcd.M1) FROM=10ns TO=20ns
```

.MEASURE

The `.MEASURE` statement is useful in a wide range of applications. You can analyze simulation results by adding `.MEASURE` statements in input files. The PrimeSim tool has additional data-measurement capability whose syntax is compatible with popular SPICE conventions.

- [Support for EM in .MEASURE](#)
- [Hierarchical Parameter Measurements](#)
- [Measurement Analysis \(.MEASURE\) and its Modes](#)
- [Rise, Fall, and Delay](#)
- [FIND and WHEN Functions](#)
- [Equation Evaluation](#)
- [AVG, RMS, MIN, MAX, and Peak-to-Peak \(PP\)](#)
- [INTEGRAL Function](#)
- [DERIVATIVE Function](#)

- [Peak-to-Peak Period Jitter \(PPJITTER\)](#)
- [Continuous Measure](#)
- [Pulse Width Measurements](#)
- [.STROBEJITTER](#)

Support for EM in .MEASURE

The PrimeSim tool supports the EM function in measure statements. The related option is `primesim_em_factor` and the default value of this option is 0.5.

Hierarchical Parameter Measurements

The PrimeSim tool supports `.meas` hierarchical parameter measurements when the statement is in the top level of the hierarchy.

Measurement Analysis (.MEASURE) and its Modes

The `.MEASURE` statement prints user-defined electrical specifications of a circuit and is used extensively in optimization. The specifications include propagation, delay, rise time, fall time, peak-to-peak voltage, minimum and maximum voltage over the specified period, and a number of other user-defined variables.

The `.MEASURE` statement has several different formats, depending on the application. You can use it for either DC sweep or transient analysis. Fundamental measurement modes are rise-fall-delay, find-when, equation evaluation, functions (average, RMS, min, max, and peak-to-peak), integral evaluation, and derivative evaluation.

General expressions can be given in the `when` condition of a `.MEASURE` statement, as shown in the following examples:

```
.measure tran tm1 when 'v(1)+v(2)!='v(3)+1' rise=1
.measure tran tm2 when 'v(4)*2'=2 fall=3
```

Note:

- Wildcards are supported only in [AVG](#), [RMS](#), [MIN](#), [MAX](#), and [Peak-to-Peak \(PP\)](#) measurement.
- When using formulas in a `.MEAS` command, use the `PAR()` keyword to designate the formula.

Rise, Fall, and Delay

This format is used to measure independent-variable (time, or any parameter or temperature) differential measurements such as rise time, fall time, slew rate, and any measurement that requires the determination of independent variable values. The format has two sub-statements `TRIG` and `TARG`, which specify the beginning and ending of a voltage or current amplitude measurement.

The rise, fall, and delay measurement mode computes the time between a trigger value and a target value.

Syntax

```
.MEASURE [DC|AC|TRAN] Result
+ TARG targ_var VAL=targ_val [TD=time_delay]
+ [CROSS=c|LAST] [RISE=r|LAST] [FALL=f|LAST] [REVERSE]
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] Result
+ TRIG AT=val
+ TARG targ_var VAL=targ_val [TD=time_delay]
+ [CROSS=c|LAST] [RISE=r|LAST] [FALL=f|LAST] [REVERSE]
+ [PRINT=0|1]
```

Table 53 Rise, Fall and Delay Parameters

Parameters	Description
[DC AC TRAN]	Analysis type. If omitted, the last analysis mode used is assumed.
AT=val	The val is the time for the TRAN analysis or the parameter for the DC analysis, at which the measurement is to start.
CROSS=c	The measurement is performed when the designated signal has reached c crossing times, as a result of rising or falling.
FALL=f	The measurement is performed when the designated signal has fallen f times.
LAST	The measurement is performed when the last CROSS, FALL, or RISE event occurs. For CROSS=LAST, measurement is performed the last time the WHEN condition is true for a rising or falling signal. For FALL=LAST or RISE=LAST, the measurement is performed the last time the WHEN condition is true for a falling or rising signal, respectively.
MEASURE	Specifies measurements. Can be abbreviated as MEAS.
[PRINT=0 1]	0: Prevents the printing a .MEASURE result into the output file. 1: (default) Prints the .MEASURE result into the output file.

Table 53 Rise, Fall and Delay Parameters (Continued)

Parameters	Description
<i>Result</i>	The name given the measured value in the PrimeSim output. The item measured is the independent variable beginning at the trigger and ending at the target. For transient analysis it is time. For DC analysis it is the DC sweep variable. If the target is reached before the trigger is activated, the result is negative.
REVERSE	The REVERSE keyword is used to reverse the direction of the measure. For any measure where RISE, FALL, or CROSS is used, the REVERSE keyword allows the measure to start at the end of the simulation time and end at <code>time=0</code> or the delay time defined by TD.
RISE= <i>r</i>	The measurement is performed when the designated signal has risen <i>r</i> times.
TARG	The beginning of the target signal specification.
<i>targ_var</i>	The name of the output variable whose propagation delay is determined with respect to the <i>trig_var</i> .
TD= <i>time_delay</i>	The amount of simulation time that must elapse before the measurement is enabled. The number of crossings, rises, or falls is counted only after the <i>time_delay</i> value. The default value is 0.
TRIG	The beginning of the trigger signal specification.
<i>trig_var</i>	The name of the output variable that determines the logical beginning of measurement.
VAL= <i>targ_val</i>	The value of the <i>targ_var</i> at which the counter for crossing, rises, or falls is incremented by one.
VAL= <i>trig_val</i>	The value of <i>trig_var</i> at which the counter for crossings, rises, or falls is incremented by one.

Examples

```
.MEASURE TRAN tdelay TRIG V(1) VAL=2.5 TD=10n RISE=2
+ TARG V(2) VAL=2.5 FALL=2
.MEASURE TRAN riset TRIG I(Q1) VAL=0.5m RISE=3
+ TARG I(Q1) VAL=4.5m RISE=3
.MEASURE pwidth TRIG AT=10n TARG V(in) VAL=2.5 CROSS=3
```

In the first example, a propagation delay measurement is taken between nodes 1 and 2 for a transient analysis. The delay is measured from the second rising edge of the voltage at node 1 to the second falling edge of node 2. The measurement begins when the second rising voltage at node 1 is 2.5V and ends when the second falling voltage at node 2 reaches 2.5V. The TD=10ns parameter prevents the crossings from being counted until 10ns has elapsed. The results are printed as `tdelay=value`.

In the second example, the time measurement begins when the current through Q1 at the third rising edge is 0.5mA and ends when the current through Q1 at the third rising edge is 4.5mA. The variable `riset` is the printed output variable.

In the third example, the short form `TRIG AT=10n` specifies that the time measurement is to begin at time `t=10ns` in the transient analysis. The `TARG` parameters specify that the time measurement is to end when `V(in)=2.5V` on the third crossing. The variable `pwidth` is the printed output variable.

FIND and WHEN Functions

The `FIND` and `WHEN` functions allow you to measure any independent variable (time, parameter), any dependent variable, such as voltage or current, or the derivative of any dependent variables when some event occurs. These functions are useful for measuring the time or any parameter value when two signals cross each other, or when a signal crosses a constant value. The measurement starts after a specified time delay, `TD`. You can find a specific event by setting `RISE`, `FALL`, or `CROSS` to a value (or parameter) or `LAST` for the last event.

Syntax

```
.MEASURE [DC|AC|TRAN] result WHEN outvar=val [TD=val]
+ [RISE=r|LAST] [FALL=f|LAST] [CROSS=c|LAST] [REVERSE]
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] result WHEN outvar1=outvar2 [TD=val]
+ [RISE=r|LAST] [FALL=f|LAST] [CROSS=c|LAST] [REVERSE]
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] result FIND outvar1 WHEN outvar2=val
+ [TD=val] [RISE=r|LAST] [FALL=f|LAST] [CROSS=c|LAST] [REVERSE]
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] result FIND outvar1 WHEN outvar2=outvar3
+ [TD=val] [RISE=r|LAST] [FALL=f|LAST] [CROSS=c|LAST] [REVERSE]
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] result FIND outvar1 AT=val
+ [PRINT=0|1]
```

Table 54 FIND and WHEN Parameters

Parameters	Description
[DC AC TRAN]	Analysis type. If omitted, the last analysis used is assumed.
CROSS= <i>c</i>	The measurement occurs when the designated signal has <i>c</i> crossing times, as a result of either rising or falling.
FALL= <i>f</i>	The measurement is performed when the designated signal has fallen <i>f</i> times.
FIND	Selects the FIND function.
LAST	Measurement is performed when the last CROSS, FALL, or RISE event occurs. For CROSS=LAST, the measurement is performed the last time the WHEN condition is true for a rising or falling signal. For FALL=LAST or RISE=LAST, the measurement is performed the last time the WHEN condition is true for a falling or rising signal, respectively.
[PRINT=0 1]	0: Prevents the printing a .MEASURE result into the output file. 1: (default) Prints the .MEASURE result into the output file.
outvar(1,2,3)	The variables used to establish conditions at which measurement is to take place.
result	The name associated with the measured value in the PrimeSim output.
REVERSE	The REVERSE keyword is used to reverse the direction of the measure. For any measure where RISE, FALL, or CROSS is used, the REVERSE keyword allows the measure to start at the end of the simulation time and end at time=0 or the delay time defined by TD.
RISE= <i>r</i>	The number which occurrence of a RISE event causes a measurement to be performed. The measurement is performed when the designated signal has risen <i>r</i> rise times.
TD= <i>val</i>	The time at which measurement is to start.
WHEN	Selects the WHEN function.

Examples

```
.MEASURE TRAN result WHEN V(1)=3.3
.MEASURE TRAN result WHEN V(1)=V(2) TD=10ns
.MEASURE TRAN result FIND V(3) WHEN V(1)=2.3 RISE=3
.MEASURE TRAN result FIND V(3) WHEN V(1)=V(2) CROSS=LAST
```

In the first example the `result` is the time taken when the voltage at the node 1 is 3.3V.

In the second example, `result` is the time taken at 10ns after both node 1 and 2 have the same voltage value.

The third example finds the voltage at the node 3 when `V(1)=2.3V` on the third rising.

The fourth example finds the voltage at the node 3 when $V(1)=V(2)$ on the last crossing.

Equation Evaluation

This statement is used to evaluate an equation that is a function of the results of previous `.MEASURE` statements. The equation must not be a function of node voltages or branch currents.

Syntax

```
.MEASURE [DC|AC|TRAN] result PARAM='equation'  
+ [PRINT=0|1]
```

Table 55 Equation Evaluation Parameters

Parameters	Description
[DC AC TRAN]	Analysis type. If omitted, the last analysis type uses is assumed.
PARAM='equation'	Evaluations of any algebraic expression which may contain other measure outputs.
[PRINT=0 1]	0: Prevents the printing a <code>.MEASURE</code> result into the output file. 1: (default) Prints the <code>.MEASURE</code> result into the output file.

Examples

```
.MEASURE TRAN result1 TRIG V(1) VAL=3.3 FALL=2  
+TARG V(2) VAL=2.1 TD=40ns  
.MEASURE TRAN result2 TRIG AT=5ns  
+TARG V(4) VAL=3.0 RISE=LAST  
.MEASURE TRAN sum param='result1+0.5*result2'
```

The first example finds the time measurement from the time $V(1)=3.3V$ on the second fall to the time `40ns` after $V(2)=2.1V$. The result is stored in `result1` and output.

The second example finds the time measurement from `5ns` to the time $V(4)=3V$ on the last rise. The result is stored in `result2` and output.

The third example calculates the expression `result1+0.5*result2` and stores the result in `sum` and outputs `sum` as `sum=value`.

AVG, RMS, MIN, MAX, and Peak-to-Peak (PP)

The average (AVG), RMS, MIN, MAX, and peak-to-peak (PP) measurement modes report statistical functions of the output variable rather than the analysis value. `AVG` calculates the area under the output variable divided by the periods of interest. `RMS` takes the square root of the area under the output variable square divided by the period of interest. `MIN`

reports the minimum value of the output function over the specified interval. `MAX` reports the maximum value of the output function over the specified interval. `PP` (peak-to-peak) reports the maximum value minus the minimum value over the specified interval.

Syntax

```
.MEASURE [DC|AC|TRAN] result func outvar [FROM=val] [TO=val]
+ [PRINT=0|1]
```

Table 56 *AVG,RMS, MIN, MAX, INTEG, PP Parameters*

Parameters	Description
[DC AC TRAN]	Analysis type. If omitted, the last analysis mode used is assumed.
FROM=val	The beginning of the <code>func</code> calculation. This parameter can be expressed algebraically using the keyword <code>tend</code> which represents the end time of the simulation as determined by <code>.AUTOSETTLE</code> .
func	The type of measure statement; one of the following: • <code>AVG</code> (average): Calculates the area under <code>outvar</code> divided by the periods of interest. • <code>MAX</code> (maximum): Reports the maximum value of <code>outvar</code> over the specified interval. • <code>MIN</code> (minimum): Reports the minimum value of <code>outvar</code> over the specified interval. • <code>PP</code> (peak-to-peak): Reports the maximum value minus the minimum value of <code>outvar</code> over the specified interval. • <code>RMS</code> (root mean squared): Calculates the square root of the area under <code>outvar^2</code> divided by the period of interest.
[PRINT=0 1]	• 0: Prevents the printing a <code>.MEASURE</code> result into the output file. • 1: (default) Prints the <code>.MEASURE</code> result into the output file.
outvar	The name of any output variable whose function <code>func</code> is to be measured in the simulation.
result	The name associated with the measured value in the PrimeSim output. The value is a function of the variable (<code>outvar</code>) and <code>func</code> .
TO=val	The end of the <code>func</code> calculation. This parameter can be expressed algebraically using the keyword <code>tend</code> which represents the end time of the simulation as determined by <code>.AUTOSETTLE</code> .

Examples

```
.MEASURE TRAN avgval AVG v(10) FROM=10ns TO=55ns
.MEASURE TRAN maxval MAX v(1,2) FROM=15ns TO=100ns
.MEASURE TRAN minval MIN v(1,2) FROM=15ns TO=100ns
.MEASURE TRAN p2val PP I(M1) FROM=10ns TO=100ns
```


The first example calculates the average nodal voltage value for node 10 during the transient sweep from 10ns to 55ns and prints the result as `avgval`.

The second example finds the maximum voltage difference between nodes 1 and 2 from 15ns to 100ns and prints the result as `maxval`.

The third example finds the minimum voltage difference between nodes 1 and 2 from 15ns to 100ns and prints the result as `minval`.

The fourth example finds the peak-to-peak current through the element M1 from 10ns to 100ns and prints the result as `p2val`.

The following example finds the average current through the element vdd for the last 10ns before the end of the simulation and prints the result as `iavg`.

```
.AUTOSETTLE SIGNAL=vdd TYPE=avg TOLERANCE=10us START=400ps
.MEASURE TRAN iavg AVG I(vdd) FROM='tend-10ns' TO='tend'
```

By adding `par()` to the DC sweep variable it can be used in a `.MEASURE` command.

```
*sweep measure
v0 1 0 3
r0 1 0 x
.dc x 1 5 1
.meas res find par(x) when i(r0)=2
.end
```

INTEGRAL Function

The `INTEGRAL` function calculates the integral of an output variable over a specified period. It uses the same syntax used for the average (`AVG`), RMS, MIN, MAX, and peak-to-peak (`PP`) measurement modes with `func` defined as `INTEGRAL`.

Syntax

```
.MEASURE [DC|AC|TRAN] result INTEGRAL outvar [FROM=val] [TO=val]
```

Examples

```
.MEASURE TRAN charge INTEGRAL I(cload) FROM=10ns TO=100ns
```

This example calculates the integral of `I(cload)` from 10ns to 100ns.

The `INTEGRAL` function also illustrates the usage of the `par()` keyword to measure the integral of a formula.

```
.meas il integ par('v(a)+v(b)')
```

DERIVATIVE Function

The `DERIVATIVE` function calculates the derivative of an output variable at a given time or frequency or for any sweep variable, depending on the type of analysis. It also calculates the derivative of a specified output variable when some specific event occurs.

Syntax

```
.MEASURE [DC|AC|TRAN] result DERIVATIVE outvar AT=val  
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] result DERIVATIVE outvar WHEN var2=val  
+ [RISE=r|LAST] [FALL=f|LAST] [CROSS=c|LAST] [REVERSE] [TD=tdval]  
+ [PRINT=0|1]
```

or

```
.MEASURE [DC|AC|TRAN] result DERIVATIVE outvar WHEN var2=var3  
+ [RISE=r|LAST] [FALL=f|LAST] [CROSS=c|LAST] [REVERSE] [TD=tdval]  
+ [PRINT=0|1]
```

Table 57 DERIVATIVE Function Parameters

Parameters	Description
[DC AC TRAN]	Analysis type. If omitted, the last analysis mode used is assumed.
AT=val	Value of <i>outvar</i> at which the derivative is to calculated.
CROSS=c	The measurement is performed when the designated signal has reached <i>c</i> crossing times, as a result of rising or falling.
DERIVATIVE	Selects the derivative function. May be abbreviated to <code>DERIV</code> .
FALL=f	The measurement is performed when the designated signal has fallen <i>f</i> times.
LAST	The measurement is performed when the last <code>CROSS</code> , <code>FALL</code> , or <code>RISE</code> event occurs. For <code>CROSS=LAST</code> , measurement is performed the last time the <code>WHEN</code> condition is <code>true</code> for a rising or falling signal. For <code>FALL=LAST</code> or <code>RISE=LAST</code> , the measurement is performed the last time the <code>WHEN</code> condition is <code>true</code> for a falling or rising signal, respectively.
outvar	The variable for which the derivative is to be found.
[PRINT=0 1]	0: Prevents the printing a <code>.MEASURE</code> result into the output file. 1: (default) Prints the <code>.MEASURE</code> result into the output file.
result	The name given the measured value in the PrimeSim output.

Table 57 DERIVATIVE Function Parameters (Continued)

Parameters	Description
REVERSE	The REVERSE keyword is used to reverse the direction of the measure. For any measure where RISE , FALL , or CROSS is used, the REVERSE keyword allows the measure to start at the end of the simulation time and end at <code>time=0</code> or the delay time defined by TD .
RISE= <i>r</i>	The measurement is performed when the designated signal has risen <i>r</i> times.
TD= <i>tdval</i>	The time at which the measurement is to start.
<i>var</i> (<i>2,3</i>)	The variables used to establish conditions at which the measurement is to take place.
WHEN	Selects the WHEN function.

Examples

```
.MEASURE TRAN slew_rate DERIV v(out) AT=25ns  
.MEASURE TRAN slew DERIV v(1) WHEN v(1)='0.90*vdd'
```

The first example calculates the derivative of `v(out)` at 25ns.

The second example calculates the derivative of `v(1)` when `v(1)` is equal to `0.9*vdd`.

Peak-to-Peak Period Jitter (PPJITTER)

The **PPJITTER** function allows measuring the peak-to-peak jitter of a node directly from the PrimeSim simulator. This measurement is performed on the `.TRAN` output waveform. It measures the jitter of the signal as a function of the waveform frequency. The PrimeSim simulator then writes the result of this measurement to the `*.mt#` file.

Syntax

```
.MEASURE TRAN result_name PPJITTER out_var VAL=val  
+ EDGE=rise|fall|cross [REF_FREQ=val] [FROM=val] [TO=val]
```

Table 58 PPJITTER Parameters

Parameters	Description
TRAN	Specifies the transient analysis type.
<i>result_name</i>	The name of the measure peak-to-peak period jitter result.
PPJITTER	Specifies the measure function type, peak-to-peak period jitter.

Table 58 PPJITTER Parameters (Continued)

Parameters	Description
<code>out_var</code>	The target signal name for jitter versus time, continuous measurement.
<code>VAL</code>	The target signal threshold value at the rising and falling edge.
<code>EDGE</code>	The target and the reference signal edge type. The value can be <code>rise</code> , or <code>fall</code> , or <code>cross</code> . The default type is <code>rise</code> .
<code>REF_FREQ</code>	(Optional) Specifies the frequency of an ideal reference clock.
<code>FROM ... TO</code>	(Optional) Specifies the range of time to measure.

Examples

```
.measure tran jvstfreq1 PPJITTER v(out) VAL=3 edge=rise REF_FREQ=2.5E8
```

Continuous Measure

To use the continuous measurement feature you specify the `tran_cont` option in the `.measure` statement. This type of measure performs the specified measurement continuously until the end of simulation, or whenever the measurement condition is fulfilled. The feature works in post-measure, so if you use `.option autostop`, the continuous measure is disabled, in which case a `tran_cont` measure becomes a regular `tran` measure. The syntax is:

```
.measure tran_cont measure_statement [jitter=num_bin]
```

The output file is `output_prefix.measure_name.mt#`. If you use the `jitter` option, a histogram is stored in the file `output_prefix.measure_name.mt#.hist`.

For more information and examples of use, see `.MEASURE (Continuous Results)` in the *PrimeSim™ Continuum Reference Manual: Commands and Control Options*.

Pulse Width Measurements

The PrimeSim tool `.meas` statements have been enhanced to make correct measurements in the presence of glitches in the waveforms. The new parameters VHG (voltage high glitch) and VLG (voltage low glitch) are now valid in a `.meas` statement. For example, in:

```
.meas TRAN PulseWidthHigh
+ TRIG vtest VAL='0.9*vdd' VLG='0.5*vdd' RISE=1
+ TARG vtest VAL='0.9*vdd' VLG='0.5*vdd' FALL=1
```

The trigger (or first measurement point) is on the rising edge of signal `vtest`. Signal `vtest` must first cross `VLG (0.5*vdd)` and then cross `VAL (0.9*vdd)` for it to be a valid first rising edge (`RISE=1`). The trigger time point is the time when `vtest` crosses `VAL`. The target (or second measurement point) is on the falling edge of signal `vtest`. Signal `vtest` must first cross `VAL (0.9*vdd)` and then cross `VLG (0.5*vdd)` for it to be a valid first falling edge (`FALL=1`). The target time point is the time when `vtest` crossed `VAL`. The measurement reported is the target time point minus the trigger time point.

```
.meas TRAN PulseWidthLow
+ TRIG vtest VAL='0.1*vdd' VHG='0.5*vdd' FALL=1
+ TARG vtest VAL='0.1*vdd' VHG='0.5*vdd' RISE=1
```

The trigger (or first measurement point) is on the falling edge of signal `vtest`. Signal `vtest` must first cross `VHG (0.5*vdd)` and then cross `VAL (0.1*vdd)` for it to be a valid first falling edge (`FALL=1`). The trigger time point is the time when `vtest` crosses `VAL`. The target (or second measurement point) is on the rising edge of signal `vtest`. Signal `vtest` must first cross `VAL (0.1*vdd)` and then cross `VHG (0.5*vdd)` for it to be a valid first rising edge (`RISE=1`). The target time point is the time when `vtest` crossed `VAL`. The measurement reported is the target time point minus the trigger time point.

.STROBEJITTER

Description

Measures the edge-to-edge jitter based on sampled `snnoise|hbnoise|Phasenoise` analyses result. The PrimeSim Spice simulator then writes the result of this measurement to the `.msnoi#` or `.mp#` file.

Syntax

```
.MEASURE SNNOISE|HBNOISE|PHASENOISE result STROBEJITTER ONOISE [FROM=val]
[TO=val] SAMPLE_DEF=sample_name [PRINT=0|1]
```

Arguments

Parameters	Description
<i>SNNOISE HBNOISE PHASENOISE</i>	Specify analysis type, which could be <code>snnoise</code> , <code>hbnoise</code> or <code>phasenoise</code>
<i>Result</i>	The name associated with the measured value in the PrimeSim output
<i>STROBEJITTER</i>	Selects the STROBEJITTER function
<i>ONOISE</i>	Specify to calculate the strobejitter for <code>onoise</code>

Parameters	Description
<i>FROM TO</i>	Integrated frequency range of noise. If not specified, the corresponding snnoise hbnoise Phasenoise analysis's starting and stopping offset frequency will be used
<i>SAMPLE_DEF</i>	The names of measurements which defines the sample at which snnoise hbnoise phasenoise is evaluated
<i>PRINT</i>	0: prevents printing a measure result into the measure output file 1 (Default): prints the measure result into the output file.

Examples

```
snnoise log '10+1' 1000 1e+07 sampled=yes sample_def=pm0,pm1
.measure sn pm0 sample_type=cross trig v(RFOUT,0) val=1.13 fall=1 targ
v(RFOUT,0) val=0
.measure sn pm1 sample_type=cross trig v(RFOUT,0) val=1.14 rise=1 targ
v(RFOUT,0) val=0
.measure snnoise strobejit1 STROBEJITTER onoise FROM=1k TO=10meg
sample_def=pm0
.measure snnoise strobejit2 STROBEJITTER onoise FROM=1k TO=10meg
sample_def=pm1
```

Output format in file:

timepnt	integptdnoise	slewrates	strobejit1
timepnt	integptdnoise	slewrates	strobejit2
temper	alter#		
4.3230e-11	7.6185e-05	-1.0179e+09	7.4845e-14
1.8855e-10	7.5886e-05	1.0104e+09	7.5105e-14
2.5000e+01	1.0000e+00		

Note:

timepnt: Time value associated with the strobe jitter output. Unit: seconds.

integptdnoise: The onoise noise voltage integrated over the specified frequency sweep. Unit: V.

slewrates: The slew-rate of the steady-state output waveform at the time value associated with the strobe jitter output. Unit: V/seconds.

strobejit: integptdnoise/ slewrates

temper: temprature

.LPROBE

Description

You can use the `.LPROBE` statement to reduce your fsdb file size significantly. By default, the PrimeSim tool only adjusts the logic value of an `.LPROBE` statement when it writes out an analog simulation data point. Logic probes can be: logically zero (0), logically one (1), or undefined (U).

Note:

The `.LPROBE` statement is not PrimeSim HSPICE-compatible.

Syntax

```
.LPROBE v(*) [high=high_val] [low=low_val] [level=level_val]  
[check_znode=0|1] [subckt=subckt_name]
```

Arguments

Argument	Description
<code>high</code>	(Optional) The high threshold value of the signal, overriding the global value set by <code>primesim_lprobe_vh</code> , if any. Any value that is greater than this threshold is probed as logically one (1).
<code>low</code>	(Optional) The low threshold value of the signal, overriding the global value set by <code>primesim_lprobe_vl</code> , if any. Any value that is less than this threshold is probed as logically zero (0).
<code>level</code>	(Optional) The number of levels of hierarchy of the signal to logic probe.
<code>check_znode</code>	(Optional) Specifies whether to check the Hi-Z node of the signal and display the "Z" state in the waveform, overriding the global value set by <code>primesim_lprobe_hiz</code> , if any. Default is 0.
<code>subckt</code>	(Optional) Specifies the subcircuit to logic probe.

Examples

The following example probes all signals in the top eight levels of hierarchy of subcircuit `pmma02ndx20c`. All voltage levels up to 0.4V are displayed as logically zero, all values above 0.7V are displayed as logically one, and the values in between are undefined.

```
.lprobe v(*) high=0.7 low=0.4 level=8 subckt=pmma02ndx20c
```

21

PrimeSim Utilities

This chapter describes the PrimeSim utilities, fscript and IM.

This chapter covers the following topics:

- [Using fscript](#)
- [fscript Commands](#)
- [Wildcard Support for Signal Names](#)
- [fscript Examples](#)
- [Using IM](#)

Using fscript

This section describes the `fscript` utility, which supports the conversion or export of signals from a data file.

You can use the `fscript` command as shown in the following example:

```
fscript
```

When you give a command list file as an argument of `fscript`, the utility runs with commands in batch mode.

```
fscript command_list_file
```

fscript Timeout Feature

By default, the `fscript` utility does not time out, which means that it keeps holding a license until it exits. However, you can set a "timeout" using any of the following methods:

- `FSCRIPT_IDLE_TIME` environment variable

```
setenv FSCRIPT_IDLE_TIME value_in_minutes
```
- Global configuration file (`fscript.cfg`)


```
timeout value_in_minutes
```

- [fset](#) command

```
fset timeout value_in_minutes
```

Note:

If you set a timeout value, the minimum value is 15 (that is, 15 minutes). Any value specified that is less than 15 is automatically adjusted to 15.

fscript Commands

This section lists and describes the commands you can use with this utility.

- [calc](#)
- [close](#)
- [cmdlist](#)
- [dinit](#)
- [dir](#)
- [dlist](#)
- [dsignal](#)
- [dump](#)
- [exit/quit](#)
- [fsdb2vcd](#)
- [fsdbinfo](#)
- [fset](#)
- [list](#)
- [load](#)
- [meas/measure](#)
- [open](#)
- [phnmeas](#)
- [pn2tbl](#)
- [pwl](#)

- [run](#)
- [signal](#)
- [sp2fsdb](#)
- [valias](#)
- [vbus](#)
- [vinit](#)
- [vlist](#)
- [vrun](#)
- [vsignal](#)

calc

Calculates numerical math for the signal.

Syntax

```
calc type signal_name
```

Description

The following table lists supported *type* arguments.

Table 59 *calc Command: type Argument*

Parameter	Description
AVG	Average of the signal
YMIN	Min value of the signal
YMAX	Max value of the signal
YMINMAX	Min/max value of the signal

Examples

```
fscript> calc avg v(clk)

**** average from 0.0 to 800n
v(clk) = 2.5647
****
```

close

Description

Closes the opened output file. It can be closed automatically when the application is terminated. So this command can be used in order to open another output file.

Syntax

```
close
```

cmdlist

Description

Shows and/or saves the command history. When a file name is specified, this command saves all histories to the specified file.

Syntax

```
cmdlist [cmd_filename]
```

dinit

Description

Initializes the dump signal table.

Syntax

```
dinit
```

dir

Description

Reports a list of the directory contents.

Syntax

```
dir [args]
```

dlist

Description

Shows all registered signals in the dump signal table.

Syntax

```
dlist
```

dsignal

Description

Adds a signal to the dump signal table. To dump multiple signals, use separate command lines. The *signal_name* format is what is seen in PrimeWave Design Environment WaveView.

This command supports wildcard descriptions in signal names. For more details, see [Wildcard Support for Signal Names](#).

Note:

By default, the output is sorted. Use `fset sort_signal 0` to disable sorting.

Syntax

```
dsignal signal_name
```

dump

Description

Dumps all signals in the given format. By default, the fscript utility dumps the signal in `table` format. Before you run this command, the output file must be opened by using the `open` command.

You can also use this command to dump the desired signals into a new FSDB file using the `dump fsdb` command.

If you want to dump in IC format, you can add `ic` as a following argument and you can optionally define multiple times so that the fscript utility dumps an IC file for each time.

Syntax

```
dump [table|fsdb][ic [time1 time2 ...]]
```

Examples

```
fset start 5n
fset end 10n
fset step 0.5n
dinit
dsignal v(clk)
open output.dump
dump
close
```

TIME	v (clk)
5.000n	0.000
5.500n	2.500
6.000n	2.500
6.500n	416.667m
7.000n	0.000
7.500n	0.000
8.000n	2.500
8.500n	2.500
9.000n	416.667m
9.500n	0.000
10.000n	0.000

In the following example, the fscript utility dumps and generates two files in IC format: `output.ic` for 10ns and `output.ic@5e-08` for 50ns.

```
dump ic 10n 50n
```

exit/quit

Description

Terminates the application.

Syntax

```
exit  
quit
```

fsdb2vcd

Converts an FSDB file to a VCD file. Signals in the FSDB file are converted in digital with the `VLTH` / `VHTH` / `START` / `END` / `LEVEL` variables.

Syntax

```
fsdb2vcd input_fsdb_file output_vcd_file
```

Description

Support for Signal File

The `fsdb2vcd` function supports specifying a signal file to limit the number of signals to convert from FSDB to VCD.

Syntax

```
fsdb2vcd infile outfile [signal_file]
```

Signal File Format (One Signal per Line)

`v(x1.x2.node)`

fsdbinfo

Description

Reports the version of an FSDB format waveform file, as well as the number of total signals and unique signals within the file.

Syntax

`fsdbinfo input_fsdb_file`

Examples

```
fscript> fsdbinfo sample.fsdb
-----
FSDB Version      : 5.8
Simulator         : PrimeSimPro R-2020.12-SP1
Simulation Date   : Mon Apr  2 09:31:39 2020
Simulation Time   : 0.0  -> 2.00n
Signal Number     : 3
Unique Number     : 3
-----
```

fset

Sets the value of internal variables. These internal variables affect all other functions. See [fscript Examples](#) for more detailed examples of this command.

Syntax

`fset variable_name value`

Description

The following table lists supported *variable_name* arguments.

Table 60 *fset Command: variable_name Argument*

Name	Description	Default Value
TUNIT	Time unit	1n
SLOPE	Slope time	1.0
RISE	Rise time	0.5
FALL	Fall time	0.5
VOL	Output low voltage	0.5

Table 60 *fset Command: variable_name Argument (Continued)*

Name	Description	Default Value
VOH	Output high voltage	2.0
VIL	Input low voltage	0.5
VIH	Input high voltage	2.0
VLTH	Low threshold voltage	0.5
VHTH	High threshold voltage	2.0
VDD	Global supplied voltage	2.5
START	Start time	0.0
END	End time	0.0
STEP	Time step	0.0
NDIGIT	Significant number	6
SCALE	Time scale factor	1.0
TOLERANCE	Tolerance for output	10u
SCOPE	Current hierarchy scope	""
LEVEL	Limit hierarchy depth for wildcard "*" expansion	0
SORT_SIGNAL	Sort dump signals (see dsignal)	1
NOTATION	Value notation (suffix/scientific/ exponent)	Suffix
VCD2VEC	Settings specific to VCD2VEC.	ascope bus_separator check_window
TIMEOUT	Time in minutes (minimum 15) before fscript releases its license.	No timeout; fscript releases its license when it exits.
MASK	Sets a mask function for signals defined in the vector file. See the waveform parameter setting, Mask for details on syntax and use.	n/a

fset VCD2VEC

The following options are specific to converting VCD to vector files.

ascope

This option changes the output hierarchy of the vector file. It can be combined with `fset scope` (digital VCD hierarchy) to perform the mapping of hierarchy from digital to analog. If the converted vector file is at the top level of the spice netlist, you can leave the scope name as blank.

Syntax

```
fset vcd2vec ascope analog_scope_name
```

Examples

```
fset vcd2vec ascope ATOP
```

All the vector file output is `ATOP.signal_name`.

```
fset vcd2vec ascope
```

All the vector file output is `signal_name`.

bus_separator

In the VCD file, `[]` is standard bus notation. Use this option to change bus notation for the output vector file.

Syntax

```
fset vcd2vec bus_separator prefix [postfix]
```

Example

```
fset vcd2vec bus_separator _
```

This maps `a[0]` to `a_0`.

```
fset vcd2vec bus_separator \< \>
```

This maps `a[0]` to `a\<0\>`.

check_window

Using this command allows you to insert the `check_window` command into the vector file. Note that it is your responsibility to make sure the `check_window` command is valid for vector file. The fscript utility does not perform a syntax check for `check_window`.

Syntax

```
fset vcd2vec check_window ....
```

Example

```
fset vcd2vec check_window 1e-12 10e-12 1
```


This puts the line "check_window 1e-12 10e-12 1" into the vector file.

list

Description

Prints out internal data of the recently-opened database. The default value is to show all hierarchy names only.

Syntax

```
list [signal|scope|all]
```

Examples

```
list signal
=====+++++=====
SIGNALS
=====
v(clk)
v(clkb)
v(cntout<0>)
v(cntout<1>)
v(cntout<2>)
v(cntout<3>)
v(cntout<4>)
v(dll_fail)
v(down)
v(lock)
v(reset)
v(up)
v(vdd)
v(vss)
v(x3<0>.x2<0>.x1<0>.cntout<0>)
v(x3<0>.x2<0>.x1<0>.cntout<1>)
v(x3<0>.x2<0>.x1<0>.cntout<2>)
v(x3<0>.x2<0>.x1<0>.cntout<3>)
v(x3<0>.x2<0>.x1<0>.cntout<4>)
=====
TOTAL SIGNAL : 19
```

load

Description

Loads the simulation output file. It supports tr0/fsdb/vcd/extend vcd format. For VCD format, you can give a signal_info_file to generate the iovector file automatically.

Syntax

```
load [spice|fsdb|vcd] filename [signal_info_file]
```

meas/measure

Description

Runs the post simulation measurements. It can be run as standalone syntax or in conjunction with the `load` command. The fscript utility `meas` command also supports running `.measure` across multiple FSDB files that are split during PrimeSim simulation. You only need to perform `.measure` on the first FSDB file and fscript automatically looks for additional split files.

```
meas[sure] measfile [-i dbfile] [-o outfile] [-l logfile] [-s [0|1]]
```

The fscript utility supports case sensitivity for the `meas` post-measure command. The command-line option, `-s [0|1]`, enables/disables case-sensitivity detection in the netlist. By default, without specifying `-s`, the option is set to 0, so case sensitivity detection is turned off (spice mode). When you set this option to 1, case sensitivity detection is turned on. The sensitivity rules are determined based on the file extension (`.scs`) or simulator `lang` settings.

For example:

```
fscript> meas msr_cmd.scs -s 1
```

The previous example turns on case sensitivity for the `msr_cmd.scs` file.

Currently the following simulation options are supported:

- `.PARAM / PARAMETER / PARAMETERS`
- `primesim_measout`
- `.OPTION MEASDGT`
- Algebraic functions

Syntax

```
meas[ure] measfile [-i dbfile] [-o outfile] [-l logfile]
```

open

Description

Specifies the output file name. Usually, it is used in `dump/vec/pwl` commands and so on. If there is already an opened file, the PrimeSim tool closes the previous file automatically.

Syntax

```
open filename
```

phnmeas

Description

This command extracts phase noise from the resulting waveforms of Transient Noise Analysis. The underlying algorithm uses the Delay Line Method (DLM). An output file must be open before `phnmeas` can be run, and you should name the extension as `.pn0` so it can be read into the PrimeWave Design Environment WaveView tool automatically.

Syntax

```
phnmeas SIGNAME FUND_FREQ TSTART TSTOP Fsample FminRatio NumClockCycle
```

Arguments

Argument	Description
<i>SIGNAME</i>	Signal name in the waveform file that is being analyzed.
<i>FUND_FREQ</i>	Fundamental oscillation frequency reference.
<i>TSTART, TSTOP</i>	Stable oscillation period used.
<i>Fsample</i>	Sampling frequency for data in fscript. Default is
<i>FminRatio</i>	To determine minimum frequency to be considered for DLM Default is 10.
<i>NumClockCycle</i>	Number of clock cycles for DLM. Default is 10.

pn2tbl

Description

This command converts the `.pn0` file into `.tbl` format to load into PrimeWave Design Environment WaveView.

Syntax

```
pn2tbl input_filename output_filename
```

pwl

Description

Converts a signal to pwl SPICE command syntax, for use in another simulation. The result is written to the output file, so the output file must be opened using the `open` command. To specify multiple signals, use separate command lines.

This command supports wildcard descriptions in signal names. For more details see [Wildcard Support for Signal Names](#).

Syntax

`pwl signal_name [nodename type]`

Arguments

Argument	Description
<i>signal_name</i>	The name of the signal.
<i>nodename</i>	The name of the node.
<i>type</i>	Describes signal type <code>[v(voltage) i(current)]</code>

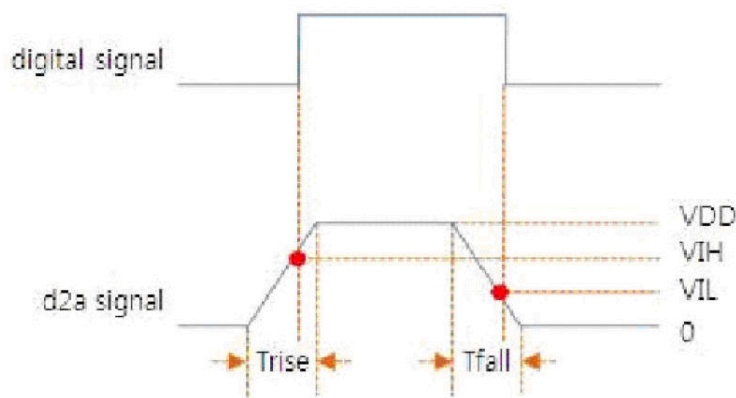
Examples

```
fset start 5n
fset end 10n
fset step 0.1n
open output.pwl
pwl v(clk)
close

vclk clk 0 pwl(
+ 5.0000e-09 0.0000e+00 5.1000e-09 8.3333e-01 5.3000e-09 2.5000e+00
+ 6.2000e-09 2.5000e+00 6.3000e-09 2.0833e+00 6.4000e-09 1.2500e+00
+ 6.5000e-09 4.1667e-01 6.6000e-09 0.0000e+00 7.5000e-09 0.0000e+00
+ 7.6000e-09 8.3333e-01 7.8000e-09 2.5000e+00 8.7000e-09 2.5000e+00
+ 8.8000e-09 2.0833e+00 8.9000e-09 1.2500e+00 9.0000e-09 4.1667e-01
+ 9.1000e-09 0.0000e+00 1.0000e-08 0.0000e+00 )
```

If a pwl signal is a digital signal, fscript automatically converts it to an analog signal with some global variables such as VDD/TRISE/TFALL/VIL/VIH, as shown in the following illustration.

Figure 20 Analog to Digital Conversion



run

Description

Runs commands in batch mode.

Syntax

```
run filename
```

Examples

```
fscript> run fscript.run
```

In this example, the command runs the following file, `fscript.run`, in batch mode:

```
load fsdb test/fs.fsdb
fset start 5n
fset end 100n
fset step 0.1n
vsignal v(*) in bin
open output.vec
vrun
close
```

signal

Description

Provides a convenient method for obtaining a signal list from an output file.

Syntax

```
signal name_pattern [depth(default=0)]
```

sp2fsdb

Description

Converts a PrimeSim HSPICE `.tr0` file to an FSDB file. If the `.tr0` file is swept, then the created FSDB file is named by `given_prefix#sweepnum.fsdb` automatically.

Syntax

```
sp2fsdb input_spice_file output_fsdb_file
```

valias

Description

Maps the node name to a different name to match the design for the vector signal.

Note:

This option is order dependent. If you alias a signal from A to B, you can perform another alias change from B to C.

Syntax

```
valias original_name aliased_name
```

Examples

Because `bus_separator` is a globally applied bus notation mapping, you can also use `valias` to change the vector bus notation for individual signals. Any `valias` used for bus mapping is performed at the very end.

```
fscript> fset vcd2vec bus_separator < >  
fscript> valias a<*> a_*
```

In the above example, all the bus signals use `<*>` as bus notation, except for `a`, which uses `_*` as bus notation.

vbuss

Description

Creates a bus from signals in a VCD file.

Note:

The signal name list must be specified using the `vsignal` command.

Syntax

```
vbuss bus_name signal1 signal2 [...]
```

Examples

```
vsignal v(a*) in hex
vbus a[3:0] v(a_3) v(a_2) v(a_1) v(a_0)
```

vinit

Description

Initializes the vector signal table.

Syntax

```
vinit
```

vlist

Description

Shows all registered signals in the vector signal table.

Syntax

```
vlist
```

vruntime

Description

Dumps all signals in a vector format file. The output file must be opened by using the [open](#) command.

Syntax

```
vruntime
```

Examples

```
fset start 5n
fset end 10n
fset step 0.1n
vsignal v(clk) in bin
open output.vec
vruntime
close

; vector pattern definitions
radix
+ 1
```

```
io
+ i

vname
+ v(clk)

; waveform parameter setting
tunit 1.0n
slope 1.0
vol 500.0m
voh 2.0
vil 500.0m
vih 2.0

; tabular data
0 0
1 1
2 0
3 1
4 0
```

vsignal

Description

Adds a signal to the vector signal table.

This command supports wildcard descriptions in signal names. For more details see [Wildcard Support for Signal Names](#).

Syntax

```
vsignal signal_name type radix
```

Arguments

Argument	Description
<i>signal_name</i>	The name of the signal.
<i>type</i>	Describes signal type. (in out both).
<i>radix</i>	Describes signal radix (bin oct hex).

Wildcard Support for Signal Names

The fscript utility supports wildcard descriptions in signal names for the `pwl`, `dsignal`, and `vsignal` commands. For example, using the following statements, you can convert all signals with names beginning with `n` to `pwl`:

```
fscript> pwl v(n1)
fscript> pwl v(n2)
fscript> pwl v(n3)
```

Or you can issue the following statement instead, using a wildcard:

```
fscript> pwl v(n*)
```

You can also define the hierarchy level which is used in name pattern expansion by using the `LEVEL` variable with the `fset` command:

```
fscript> fset level 2
fscript> vsignal v(*) in bin
fscript> dsignal v(*)
```

fscript Examples

This section includes examples to demonstrate how you can use the fscript utility.

- [Example 1: Convert FSDB to PWL](#)
- [Example 2: Convert VCD to Vector](#)
- [Example 3: Dump Signal](#)
- [Example 4: Convert tr0 to FSDB](#)
- [Example 5: Convert FSDB to VCD](#)
- [Example 6: Create IC File From Simulation Output Files](#)
- [Example 7: Post-measure With Output Data File](#)
- [Example 8: Phase Noise Analysis](#)

Example 1: Convert FSDB to PWL

Consider the following example when you want to get a PWL signal `clk`:

```
fscript> load fsdb sample.fsdb
fscript> fset start 5n
fscript> fset end 10n
fscript> fset step 0.5n
```

```
fscript> open output.pwl
fscript> pwl v(clk)
fscript> close
```

If you do not specify start/end/step variables, all time step points are output. In this case, the output file `output.pwl` is as follows:

```
vclk clk 0 pwl (
+ 5.0000e-09 0.0000e+00 5.1000e-09 8.3333e-01 5.3000e-09 2.5000e+00
+ 6.2000e-09 2.5000e+00 6.3000e-09 2.0833e+00 6.4000e-09 1.2500e+00
+ 6.5000e-09 4.1667e-01 6.6000e-09 0.0000e+00 7.5000e-09 0.0000e+00
+ 7.6000e-09 8.3333e-01 7.8000e-09 2.5000e+00 8.7000e-09 2.5000e+00
+ 8.8000e-09 2.0833e+00 8.9000e-09 1.2500e+00 9.0000e-09 4.1667e-01
+ 9.1000e-09 0.0000e+00 1.0000e-08 0.0000e+00 )
```

Example 2: Convert VCD to Vector

Consider the following example when you want vector out:

```
fscript> load vcd sample.vcd
fscript> fset start 0
fscript> fset end 100n
fscript> fset step 0
fscript> open output.vec
fscript> vinit
fscript> vsignal cont in bin
fscript> vrun
fscript> close
```

If you do not specify start/end/step variables, all time step points are output. In this case, the output file `output.vec` is as follows:

```
; vector pattern definitions
radix
+ 1
nodename
+ cont

Io
+ i
; waveform parameter setting
tunit 1.0n
slope 1.0
vol 500.0m
voh 2.0
vil 500.0m
vih 2.0
; tabular data
0.000 1
10.000 0
20.000 1
30.000 0
```

```
40.000 1
50.000 0
60.000 1
70.000 0
80.000 1
90.000 0
100.000 1
```

Example 3: Dump Signal

Consider the following example when you want signals `clk`, `ready`, and `outa`:

```
fscript> load fsdb sample.fsdb
fscript> fset start 5n
fscript> fset end 10n
fscript> fset step 0.5n
fscript> open output.dump
fscript> dinit
fscript> dsignal v(clk)
fscript> dsignal v(ready)
fscript> dsignal v(outa)
fscript> dump
fscript> close
```

If you do not specify start/end/step variables, all time step points are output. In this case, the output file `output.dump` for `clk` is as follows:

```
TIME v(clk)
5.000n 0.000
5.500n 2.500
6.000n 2.500
6.500n 416.667m
7.000n 0.000
7.500n 0.000
8.000n 2.500
8.500n 2.500
9.000n 416.667m
9.500n 0.000
10.000n 0.000
```

Example 4: Convert tr0 to FSDB

Consider the following example when you want to convert a `.tr0` file to FSDB format:

```
fscript> load spice sample.tr0
fscript> sp2fsdb sample.tr0 sample.fsdb
```

Example 5: Convert FSDB to VCD

Consider the following example when you want to convert an FSDB file to VCD format (all analog signals in the FSDB file are affected by the v_{lth} and v_{hth} variable):

```
fscript> load fsdb sample.fsdb
fscript> fset vlth 1.25
fscript> fset vhth 1.25
fscript> fsdb2vcd sample.fsdb sample.vcd
```

Example 6: Create IC File From Simulation Output Files

Consider the following example to create IC files from simulation output files, namely .fsdb:

```
fscript> load fsdb reference.fsdb
fscript> open output.ic
fscript> dsignal *
fscript> dump ic
fscript> exit
```

The `output.ic` file gets created uses the `.ic` statement. It can be modified to `.nodeset` if necessary.

Example 7: Post-measure With Output Data File

Consider the following example to load an output file `output.fsdb` using the original input deck `input.sp` and perform post-measure to generate `output.pmt`:

```
fscript> meas input.sp -i output.fsdb -o output.pmt -l fscript.log
```

or:

```
fscript> load fsdb output.fsdb
fscript> meas input.sp -o output.pmt -l fscript.log
```

Example 8: Phase Noise Analysis

Consider the following example to perform phase noise analysis on the FSDB output:

```
fscript> load fsdb input.fsdb
fscript> open output.pn0
fscript> phnmeas pll_out 1e9 50e-9 80e-9
fscript> close output.pn0
fscript> exit
```

The command loads the `input.fsdb` into `fscript` and performs phase noise analysis. The result is stored into `output.pn0`.

Using IM

IM is a GUI-based post-processing tool that you can use to inspect measurements. Invoke the IM tool using the `-im` command-line option (see [Table 2](#)).

Note:

After the GUI window opens, click **Help** for an additional description of the tool.

Syntax

```
primesim -im [-opt] file [file2 ...]
```

Values for `-opt` are described under [Filter Options](#) and [Batch Mode Options](#).

Values for `file` are described under [Input Files](#).

Filter Options

```
-ps[=abserr], -uA[=abserr], -V[=abserr]
```

Display only timing, current, or voltage measurements, respectively, optionally with minimum absolute error.

```
-leaf
```

Do not display compound measurements.

```
-match=pattern, -wmatch=pattern, -rmatch=pattern, -casesen
```

Display using pattern matching (strict, wildcard, or regex, respectively) and optional case-sensitivity.

Batch Mode Options

Note:

Batch mode options are used without the GUI.

```
-csv
```

Output processed measurements in `.csv` format.

```
-worst
```

Output worst processed measurements and elapsed runtime in `.csv` format.

```
-merge
```

Merge sets of worst processed measurements into a summary `.csv` table.

Input Files

Input files are ignored unless they have known extensions. The following types of files and extensions are supported:

- Measurement files: `.meas`, `.meas0`, `.mt`, `.mt0`
- IM files (optional): `.im`

Only the first `.im` file found is used. You can generate an `.im` file during a normal PrimeSim run by:

- Specifying `.opt PRIMESIM_IM=1`
- Putting `setenv PRIMESIM_IM 1` in your `.csh`
- Waveform files (optional): `.fsdb`, `.tr0`, `.out` (used with double-clicking)

If more than one input file is specified, the measurements are compared. The first measurement file found is treated as a reference result, while any relative and absolute errors are displayed for the second (target) file.

Examples

This example shell substitution mechanism generates a list of arguments which are processed according to the above rules.

```
alias im primesim -im
im refdir/* cmpdir/* someotherdir/*
im dir/ref* dir/cmp* dir/*
```

This example is a handy shortcut if you do not need to specify reference and compare files.

```
im outdir/*
```

In this example, only non-compound current measurements with absolute error exceeding 5nA matching the pattern are processed.

```
im -uA=0.005 -match=leak -leaf outdir/*
```

This batch mode example generates a summary table for the `worst.csv` files located in directories `case1` through `case24`.

```
im -merge case*/worst.csv
```